Object Oriented Programming with Java

by

Debasis Samanta

Associate Professor ,Department of Computer Science and Engineering, Indian Institute of Technology Kharagpur, India

C O N T E N T S

Overview

CHAPTER 1: *Fundamentals of Java*

- Introduction
- Java Programming Paradigms
  - Object-Oriented Programming
  - Platform Independent Programming Environment
  - Multithreading
  - Security in Java
- Advantages of Java
  - Easy Development of Large Applications
  - Simple, Portable, and Robust Programming
  - High Performance of Java
  - Internet Access Capability
  - Distributed Computing with Java
- Tools Available for Java Programming
  - Java Developer&#65533;s Kit (JDK)
  - Java Packages
  - Java -Enabled Web Browser
  - Other Third-party Tools
- Assignment

- Q&A

CHAPTER 2:‘‘ Programming in Java

- Introduction
- Building Java Applications
- Building Java applets
  - Basic Methods in Applet
- Differences between applet and Application

- Practice Questions
- Assignment
- Q&A

CHAPTER 3:‘‘ *Object Oriented Mechanism in Java*

- Introduction
- Class Definition in Java
- Constructors
- Inheritance
- Polymorphism in Java
  - Dynamic Method Dispatch
- Access Specification in Java
- Practice Questions
- Assignment
- Q&A

CHAPTER 4:‘‘ *Interfaces and Packages in Java*

- Introduction
- Interfaces in Java
  - Interface methods
  - Inheritance in Interfaces
- Packages in Java
  - API : The built-in Java Packages
  - Organization of user packages
  - Managing user packages
- Practice Questions
- Assignment
- Q&A

CHAPTER 5:‘‘ *Exception Handling in Java*

- Introduction
- Built-in classes for exceptions handling in Java
- Mechanism of Exceptions Handling in Java
- Error Handling Exception Classes
- Practice Questions
- Assignment

- Q&A

CHAPTER 6:‘‘ *Threads and Multithreading in Java*

- Introduction
- Basics of a thread
  - Creating and Running a Thread
  - Life cycle of threads
  - Status of a Thread
- Synchronization and Inter-Thread Communication
  - Synchronization
  - Inter-thread Communication
- Thread Groups and Daemon
- Practice Questions

- Assignment

- Q&A

CHAPTER 7:‘‘ *Application Development in Java*

- Introduction
- Designing GUI with Components and Layout Managers
  - Using Components for Java GUIs
  - Using Layout Managers for Java GUI
  - GridBagLayoutManager
- Event Handling
- Drawing methods for Graphics objects
- Practice Questions

- Assignment

- Q&A

CHAPTER 8:‘‘ *Java Input/Output and Networking*

- Introduction
- Java File I/O
  - Java Files
  - Input Stream
  - Output Stream
- Networking in Java
  - Internet addressing method in Java
  - Communication using UDP
  - Communication Using TCP
- Practice Questions

- Assignment

- Q&A

CHAPTER 9:‘‘ *Java Multimedia*

- Introduction
- Multimedia basics in Java
  - Audio basics
  - Image basics
- Web page in Java
  - World Wide Web basics
  - Preparing Web Documents in Java
  - Applet hosted HTML Document
  - HTML Snapshots
- Practice Questions
- Assignment
- Q&A

CHAPTER 10:‘‘ *Java Swing*

- Introduction
- AWT versus Swing
- Swing in depth
- Java Swing examples
- Practice Questions
- Assignment
- Q&A

CHAPTER 11:‘‘ *JDBC*

- Introduction
- MySQL Basics and setup installation
- JDBC: Java Database Connectivity
- JDBC: Characteristics
- Relational database
- Types of JDBC Drivers
- JDBC Architectures
- Inside JDBC
- Connecting to Database
- Examples

CHAPTER 12:‘‘ *The Applet Package*

- Introduction
- Class Applet
- Interface AppletContext
- Interface AppletStub
- Interface AudioClip

CHAPTER 13:‘‘ *The Java Language Package*

- Introduction
- Class Boolean
- Class Character
- Class Class
- Class ClassLoader
- Class Compiler
- Class Double
- Class Float
- Class Integer
- Class Long
- Class Math
- Class Number
- Class Object
- Class Process
- Class Runtime
- Class SecurityManager
- Class String
- Class StringBuffer
- Class System
- Class Thread
- Class ThreadGroup
- Class Throwable
- Interface Cloneable
- Interface Runnable
- Class ArithmeticException
- Class ArrayIndexOutOfBoundsException
- Class ArrayStoreException
- Class ClassCastException
- Class ClassNotFoundException
- Class CloneNotSupportedException
- Class Exception
- Class IllegalAccessException
- Class IllegalArgumentException
- Class IllegalMonitorStateException
- Class IllegalThreadStateException
- Class IndexOutOfBoundsException
- Class InstantiationException
- Class InterruptedException
- Class NegativeArraySizeException
- Class NoSuchMethodException
- Class NullPointerException
- Class NumberFormatException
- Class RuntimeException
- Class SecurityException
- Class StringIndexOutOfBoundsException
- Class AbstractMethodError
- Class ClassCircularityError
- Class ClassFormatError
- Class Error

- Class IllegalAccessError
- Class IncompatibleClassChangeError
- Class InstantiationError
- Class InternalError
- Class LinkageError
- Class NoClassDefFoundError
- Class NoSuchFieldError
- Class NoSuchMethodError
- Class OutOfMemoryError
- Class StackOverflowError
- Class ThreadDeath
- Class UnknownError
- Class UnsatisfiedLinkError
- Class VerifyError
- Class VirtualMachineError

CHAPTER 14:'' *The Java Utility Package*

- Introduction
- Class BitSet
- Class Date
- Class Dictionary
- Class Hashtable
- Class Observable
- Class Properties
- Class Random
- Class Stack
- Class StringTokenizer
- Class Vector
- Interface Enumeration
- Interface Observer
- The awt Event Handler
- Class EmptyStackException
- Class NoSuchElementException

CHAPTER 15:'' *The Abstract Window Toolkit (AWT) Package*

- Introduction
- Class BorderLayout
- Class Button
- Class Canvas
- Class CardLayout
- Class Checkbox
- Class CheckboxGroup
- Class CheckboxMenuItem
- Class Choice
- Class Color
- Class Component
- Class Container
- Class Dialog
- Class Dimension
- Class Event
- Class FileDialog

- Class FlowLayout
- Class Font
- Class FontMetrics
- Class Frame
- Class Graphics
- Class GridBagConstraints
- Class GridBagLayout
- Class GridLayout
- Class Image
- Class Insets
- Class Label
- Class List
- Class MediaTracker
- Class Menu
- Class MenuBar
- Class MenuComponent
- Class MenuItem
- Class Panel
- Class Point
- Class Polygon
- Class Rectangle
- Class Scrollbar
- Class TextArea
- Class TextComponent
- Class TextField
- Class Toolkit
- Class Window
- Interface LayoutManager
- Interface MenuContainer
- Class AWTException
- Class AWTError

CHAPTER 16:" *The AWT Image Package*

- Introduction
- Class ColorModel
- Class CropImageFilter
- Class DirectColorModel
- Class FilteredImageSource
- Class ImageFilter
- Class IndexColorModel
- Class MemoryImageSource
- Class PixelGrabber
- Class RGBImageFilter
- Interface ImageConsumer
- Interface ImageObserver
- Interface ImageProducer

CHAPTER 17:" *The AWT Peer Package*

- Introduction
- Interface ButtonPeer
- Interface CanvasPeer
- Interface CheckboxMenuItemPeer
- Interface CheckboxPeer

- Interface ChoicePeer
- Interface ComponentPeer
- Interface ContainerPeer
- Interface DialogPeer
- Interface FileDialogPeer
- Interface FramePeer
- Interface LabelPeer
- Interface ListPeer
- Interface MenuBarPeer
- Interface MenuComponentPeer
- Interface MenuItemPeer
- Interface MenuPeer
- Interface PanelPeer
- Interface ScrollbarPeer
- Interface TextAreaPeer
- Interface TextComponentPeer
- Interface TextFieldPeer
- Interface WindowPeer

CHAPTER 18:'' *The Java I/O Package*

- Introduction
- Class BufferedInputStream
- Class BufferedOutputStream
- Class ByteArrayInputStream
- Class ByteArrayOutputStream
- Class DataInputStream
- Class DataOutputStream
- Class File
- Class FileDescriptor
- Class FileInputStream
- Class FileOutputStream
- Class FilterInputStream
- Class FilterOutputStream
- Class InputStream
- Class LineNumberInputStream
- Class OutputStream
- Class PipedInputStream
- Class PipedOutputStream
- Class PrintStream
- Class PushbackInputStream
- Class RandomAccessFile
- Class SequenceInputStream
- Class StreamTokenizer
- Class StringBufferInputStream
- Interface DataInput
- Interface DataOutput
- Interface FilenameFilter
- Class EOFException
- Class FileNotFoundException
- Class IOException
- Class InterruptedIOException
- Class UTFDataFormatException

CHAPTER 19:'' *The Java Networking Package*

- Introduction
- Class ContentHandler
- Class DatagramPacket
- Class DatagramSocket
- Class InetAddress
- Class ServerSocket
- Class Socket
- Class SocketImpl
- Class URL
- Class URLConnection
- Class URLEncoder
- Class URLStreamHandler
- Throwing Exceptions
- Interface ContentHandlerFactory
- Interface SocketImplFactory
- Interface URLStreamHandlerFactory
- Class MalformedURLException
- Class ProtocolException
- Class SocketException
- Class UnknownHostException
- Class UnknownServiceException

Overview

Java, the next generation of C++ is a better solution for object oriented programming. It includes all the essential features of C++ and eliminates all the redundant and error-prone features in C++. Java is a unique language and it has all the features which makes it the language of the 21st century. Java is best suitable for distributed programming where a program is not necessarily to be confined within a single PC rather it spreads from single user environment to main frame, to network, and to network of networks. Java, thus makes platform independent programming in heterogeneous environment. This is why, Internet people prefer to solve their problems with this totally new language.

Learning Java, no doubt is an added advantage to the career of any IT professional, but it is tough indeed. Following few challenges are mentioned which a novice in this Java environment may face :

- ''It is based on object oriented programming concept.
- ''Java programmer must have knowledge of C++ programming.
- ''If contains so many diverse features that it is not a simple task to grasp within a quickest possible time.
- ''Java is designed for Internet programming and hence demands the knowledge of Internet standard, protocol etc. as an essential prerequisite. *Objective* Objective is to cope up with all the above mentioned challenges.It is composed to cover the following topics with easy, relevant to the topics, and small but most effective illustrations :
- ''Concept of object oriented programming.
- Brief and to the point discussions on programming with C++.
- Complete discussions for programming with Java.
- Complete reference of Application programming interface (API) which is known to be very essential for quick reference, and for advanced Java Programmer. In the first week you'll learn about the Java language itself:

Object Oriented Programming with Java.

All the features of Java Programming are discussed in this part with ready to use programs. Here, the whole discussion is distributed over 9 chapters.

API documentation :This part consists of 8 chapters where 8 Java packages are discussed with their full structures. This part is meant for advanced Java Programmers.

CHAPTER 1

Fundamentals of Java

by Debasis Samnata

CONTENTS

- Introduction
- Java Programming Paradigms
  - Object-Oriented Programming
  - Platform Independent Programming Environment
  - Multithreading
  - Security in Java
- Advantages of Java
  - Easy Development of Large Applications
  - Simple, Portable, and Robust Programming
  - High Performance of Java
  - Internet Access Capability
  - Distributed Computing with Java
- Tools Avaialbale for Java Programming
  - Java Developer's Kit (JDK)
  - Java Packages
  - Java -Enabled Web Browser
  - Other Third-party Tools
- Assignment
- Q&A

Introduction

IT industries goes on such a diversification that recent programming style should not confide within a single PC rather it spreads from a single user environment to main frame, to network, to network of networks . Or in other words, a giant program can be developed using mini programs which were developed by other geographically distributed programmers and this is also just on line. Now- a- days, such a distributed programming language and environment is no more an intellectual gossip but a reality and it is Java the Internet programming language which makes the whole world accessible from any point on it..

Java, the web-savvy programming language is developed by Sun and the Internet with the intent to create a dynamic, object oriented programming language suitable for use of the same types of development tasks as C and C++, but without the difficulties and sources of bugs common to those languages. This is why Java has been touted as a better C++; it contains all the essential OOP features but with reduced complexities than in C++. Sun describes Java as a simple, object-oriented, distributed, interpreted, robust, secure, architecture neutral, portable, high-performance, multithreaded, and dynamic language.

In this Chapter, we will briefly highlight over various features of Java programming, superiority of Java over other programming languages, and then tools available for Java programming.

Java programming paradigms

Java programming concept possesses all the advanced programming features that the recent programming languages hold, in addition to these it holds something special which makes it the first of its kind. Following sections are to brief the major characteristics of Java programming.

Object Oriented Programming

Java is an object Oriented Programming (OOP) language. It incorporates almost every OOP features. Object creation through object template i.e. class, data abstraction and encapsulation, data and code sharing through inheritance, overloading concept through polymorphism, and data/process hiding etc. are some basic OOP features in Java, Java is a bit like C++ but more simpler than aristocrat C++. In fact, it is elegant language wherein the poorly understood, erratic, and redundant features of C++ have been eliminated. There are no business of pointers in Java - only lightly bounded heterogeneous collection of data/object. Absence of any kind of pointers means free from huge bugs because pointers manipulation is one of the richest source of bugs in almost all programs. Java is sophisticated enough to help programmers to express complex ideas . Here, global data and stand-alone function are not possible, again, unlike C/C++. The basic programming components in Java is objects which directly or indirectly decedent from the forefathers of classes. The class definition in Java permits both static and dynamic binding and therefore full reuse of code and data sharing. Java is truly Object Oriented Programming in the sense that many class definition can be inherited dynamically. However, Java does not support multiple inheritance whose semantic and usage has been quite controversial. Type casting or operator overloading is also not allowed in Java.

Platform Independent Programming Environment

The microprocessor that is inside every computer can understand and execute only the machine codes i.e. in terms of 0′s and 1′s. But using high level programming, programmers use source code (English like) and using a translator, code is converted into the machine code relevant to the microprocessor. As the machine codes varies from microprocessor to microprocessor so what we need is that for same source code but for different microprocessor type different translator. Hence, a program (source code) when converted into the machine code for an Intel chip will vary from the version for a Motorola chip, which again will be different from the version for Sun workstation. The same problem exists in case of computers having same microprocessors but different operating systems.

Java has overcome this problem of architecture and environment dependency. Java solves this problem using the concept what is called Java Virtual Machine (JVM). This JVM specifies wonderfully detailed with a dummy CPU and dummy instruction set that looks like standard assembly code. The Java compiler transforms the source programs into the instructions of JVM. The compiled Java code in known as byte code instruction (which are assembly code according to JVM). Next, Java interpreter is there which is specific to each environment (processor and OS) and converts the byte code instructions into the native processor instructions before executing it. Thus, converting a program source into an object code is compile half way and interpret half way. **Figure 1.1** illustrates this mechanism.

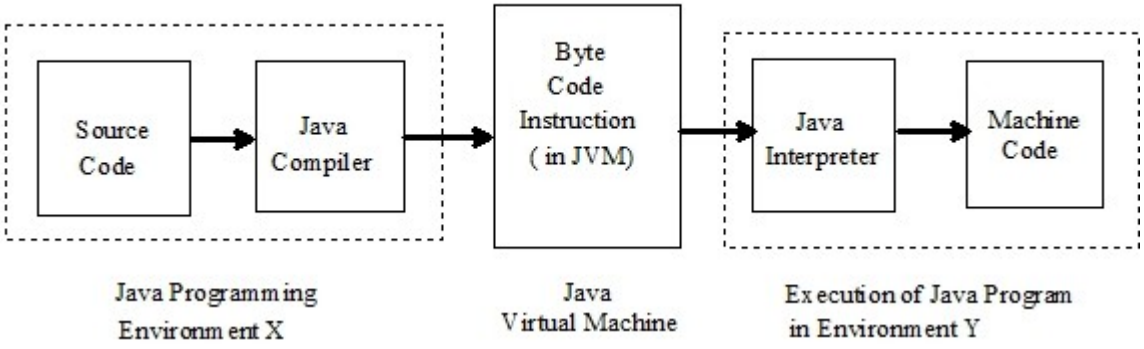


Figure 1.1 : Compilation of Java source code into object code

As the Java byte codes are architecture and operating system independent they are highly portable and can be executed on any system without change. This platform independence is illustrated in **Figure 1.2**

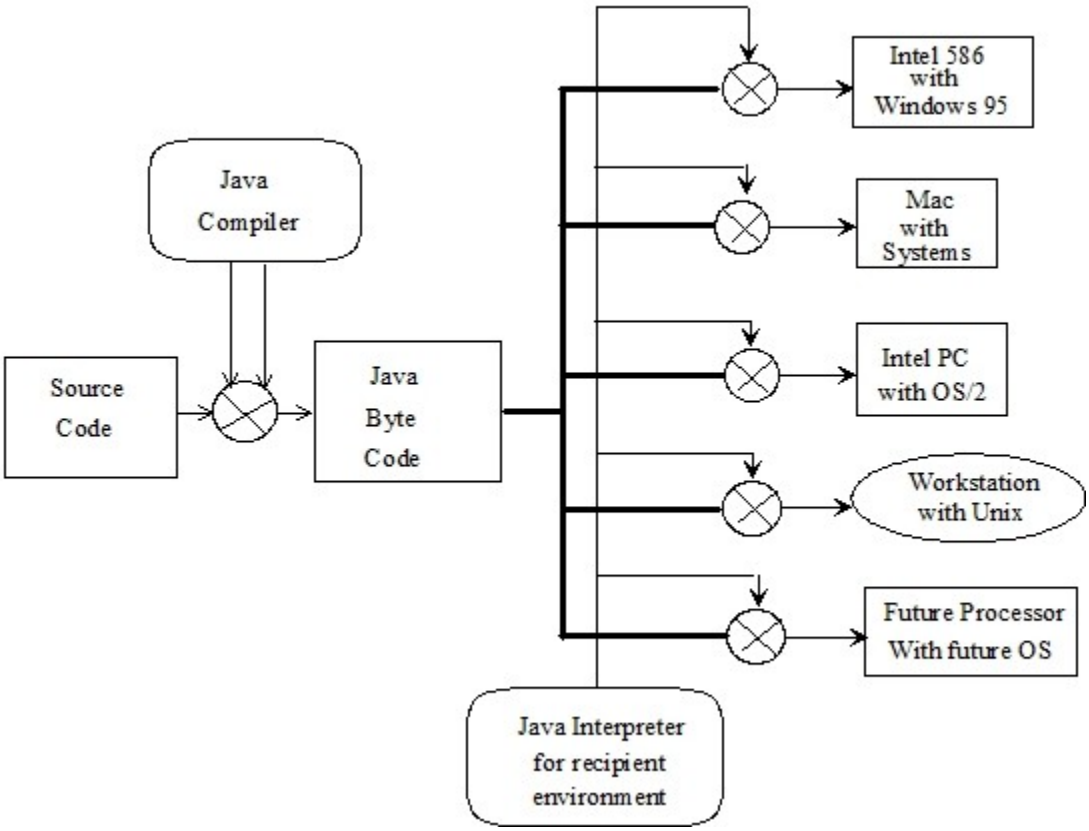


Figure 1.2 :Platform independent Java programming

Using the conventional programming languages like C or Pascal, if we need a slice of memory we have to allocate it and then to returns it to the system we are finished with it, Thus, here memory management is solely decided by the programmer. This overhead is eliminated in Java. Here, a program does not need to be malicious to suck up memory and not release it. Java controls all of the memory, Java′s run time system supports automatic garbage collectors based memory management system. Every once in a while, it will scan through all of the objects in a process to see if any of them are not used. The obsolete objects are reclaimed from the memory. As there no explicit memory manipulation is required by the programmer, so Java programs are highly robust

Multithreading

Another important contribution of Java is the support of multithreading. The multithreading is basically, the ability to run several tasks within the context of one large application. In more simpler version, a program is a list of instructions , and path through list of instructions is often called the thread of execution. In many cases, execution of a program through multiple threads makes more sense. For an example, suppose, a program needs to open a file, draw some thing on the screen, and print out something else. Each of these may require waiting for access to some resource. A multithreaded program can, in effect, start up three different threads of executions for these tasks. If the printing thread is waiting for more paper to be loaded in the printer, then the file thread and drawing thread can continue operating. In Java multithreading interpreter can keep track of all threads and switch among all of them, giving each a fair share of the processors attention. For most applications, multithreading improves interactive performance and real time system development. The multithreading encourages Java to interface and support several features of modern operating systems and network protocols.

Security in Java

Java is based on client /sever technology, After the accomplishment of source code into byte code translation, Java programs are in effect down loaded from host machines and then run on one′s client machine. Since Java programs are down loaded from all over the world, some precautions are needed to prevent malafide pieces of code called computer viruses embedded within the programs, causing problems to the recipient system and spreading it around. This therefore demands that distributed applications have to exhibit the highest level of security concerns. A byte

code verifier in the Java interpreter looks at the incoming byte codes and verifies whether the compiled code is strictly language compliant, in case it founds illegal code, the run time system rejects the code and refuses to run it; thus trapping all malafide codes. Once the strict verification is over, the application proceeds to run and by this Java impotents security holes if any.

Advantages of Java

This section is to give the answer of Why Java? . Following points are included as answer of this question.

Easy Development of Large Applications

Object oriented methodology in Java enables the software system developers to develop large complex applications with a little effort. Experienced software developers understands that software systems are dynamic in nature i.e. in continuous evolution. Keeping this in mind, the dynamic binding in Java can be utilized to furnish a surprising advantages of code reusability. Here, unlike C++, a change of a base class need not require to recompile of derived classes. Application can be embedded dynamically. Here, linkage manipulations are deferred until run time. This concept envisages easy to modification and ability to upgrade in the existing system

Simple, Portable, and Robust Programming

Writing Java code is almost English like and is similar to writing codes in other programming languages.

Java programs are portable because they can be executed in any environment without any substantial change. The Java bytecodes are independent of any underlying architecture, characters are uni-code based for internationally portable applications. If one use certain I/O classes, one can render data portable on Java code. For example, the DataInputStream and DataOutputStream, there are two classes in Java, utilize external data representation to normalize one′s data to a network format. In other words, data can be used by many different types of processors and machines. Currently, Java run time environments exists are SolariesTM 2.x, (SPARC and x86), Win32 (Windows 95 and Windows NT), OS/2, Macintosh Systems 7.5 for Power PCs, HP/Unix etc.

Java programs are robust because there is no need of explicit memory manipulation. Memory address cannot be deferred nor can pointer arithmetic be used to access object contents. Array bounds are checked so that array indices are never out-of-bounds.

High Performance of Java

Already mentioned that execution of programs in Java is a two steps process compile half way and interpret half way. But why an interpreter?

An interpreter reads every line, converts it into machine code of the system and then execute it . Every line of source code has to be converted to machine code every time it executes making the application painfully slow. On the other hand, compiler compiles the entire source code into machine code before starting the execution and speed up the overall process. The only draw backs with the compiler is that an executabel code can work only on one type of system often specific to an operating system. You can not compile code if you have no knowledge of the machine you are compiling it for.

But to match the same program into various environment. It was studied that the simplest solution is to look for an interpreter. Interpreter being as slow as they are, and compiled code being as fast as it is, the Java developer therefore decided to strike the middle ground in order to optimize the whole. In fact, most commands of one CPU machine code match with those of another, instruction to load and move data, to add and subtract numbers etc. form the basic building blocks of assembly programming and are often similar. Therefore, translating the source code into JVM code and then to that of, say, for Pentium, would not be as slow as in conventional interpreting. It is realized that by making a little compromise both ways, much has been achieved - the high performance computing environment.

Internet Access Capability

It is well known that marriage of computer and communication gave the birth to today′s information society. After the successful maturity of ARPANET (Advanced Research Projects Agency Network), now Internet is a rapidly evolving network of networks, which essentially consists of very powerful computing systems with a variety of capabilities through a high speed (even with the speed of light) and large band width communication channels. To be sure, Internet facilitating communication among the computers connected in it and data or information can be conveniently

transferred to and from any computer, anywhere in the world at the click of mouse. Through the Internet, a chunk of information can be simultaneously shared and read by people scattered around the globe. Information over the Internet is actually multimedia, which include text, graphics, images and sound. With Internet there is Web (short for World Wide Web, more commonly also written as WWW). It is a wide area hypermedia (also called hypertext) information retrieval system, introduced by a group of physicists at the CERN European Laboratory for Particle Physics in Geneva, Switzerland in 1991. Web is to facilitate online documentation to give universal access to a very large universe of documents. WWW on the Internet utilize hyperlinks to make immediate network connectivity to any site of the world.

It was a challenge to create a document that looked like a simple document to a user on a computer system, but were actually spread across multiple systems in different parts of the world. But with the WWW and the Internet boom, the whole world is now a simple page.

The Java programming concept is now poised to change the way Internet is currently being used. Currently hypertext documents comprise multimedia i.e. combination of text, data, audio, graphics, pictures - still or movie. Maintaining and retrieving such a multimedia base hypertext is a technological challenge to Internet people. For an example, if a web document with features that include non standard/ incompatible extension to the html are made, most Browser may ignore them because the underlying computer systems (clients) are incapable of supporting them. In addition, the transmission of multimedia based documents over the network involved a large channel band width - as a consequence, such documents may not be transmitted in their complete form on most client systems. Moreover, documents may run into trouble. This is because, the code marked in the text by the composer is specific to the local processor and operating system and hence may not be executed in the client systems.

Given these problems to the Internet people, here is an obvious need to devise a programming language which can yield a sophisticated, system independent hypertext so that user from different system compatibility can share information. Truly speaking, Java is to turn Internet and WWW into one giant computer so that Net and Web will be common storehouse for billion of data and application unit. One can down load the required application from the Net into one&#65533;s system, use it and it will be automatically deleted from the system after it use.

Distributed Computing with Java

Java is the first concrete example that moves to component and distributed computing. It envisages a new type of programming environment in which a program is truly task oriented and disposable. One can enjoy the benefits of the execution of ready made compiled code as single statement which can be dynamically down loaded from any where (may be too remote) in the network of heterogeneous systems and executed transparently. This is the dream most software developer have had for years.

Tools Available for Java Programming

The tools those are essential /available for Java programming environment are JDK(Java Developer&#65533;s Kit), Java Packages, Java Enabled web browsers and other Third party tools. Each of these are briefly outlined below :

Java Developer&#65533;s Kit (JDK)

The Java Developer&#65533;s Kit from Java Soft, a division of Sun Micro systems, contains the basic tools and libraries necessary for creating and executing Java programs. It consist of a library of standard classes and a collection of utilities for building, testing, and documenting Java programs. The core Java Application Programming Interface (API) is the aforementioned library of prefabricated classes. One need these classes to access the core functionality of the Java language. The core Java API includes some important language construct (including string data types and exceptions), as well as basic graphics, network, and file I/O. As for the JDK utilities, there are seven main programs in the kit :

```
1.javac:The Java compiler. This program compiles Java source codes into byte codes.

2.java:The Java interpreter. This program runs Java byte codes.

3.javadoc:Generates API documentation in HTML format from Java source code.

4.appletviewer:A Java interpreter that executes Java applet (a special kind of Java Program) classes.

5.jdb:The Java debugger.’‘Helps us find and fix bugs in Java programs.
```

6.**javap**:The Java disassembler. Displays the accessible functions and data in a compiled class file. It'‘also displays the meaning of the byte codes.

7.**javah**:Creates C header files that can be used to make C routines, that can call Java routines,or make C routines that can be called by Java programs.

JDK is a free software and can be down loaded from Java Softs web site at http://java .sun.com

Java Packages

The Java language provides suite of packages that include a windowing tool kit, utilities,, general I/O, tools, and networking. Following mentioned six packages are very popular:

1.**java.applet**: This package includes classes designed for use within an applet. There is one class, Applet and three interfaces AppletContex, AppletStub, and AudioClip.

2.**java.awt** :The Abstract Windowing Toolkit (AWT) package, awt, contains classes used to generate widgets and GUI'‘(Graphical User Interface) components. This package includes the classes: Button, CheckBox, Choice, Component, Graphics, Menu, Panel, TextArea, and TextField.

3.**java.io'‘**:The I/O Package include file input and output classes such as the classes FileInputStream and FileOutputStream. File'‘I/O in subject to Security Control in applets.

4.**java.lang**:This package includes the Java language classes, including Object, Thread, Exception, System, Integer, Float, Math, String, and so on.

5.**java.net** :This class supports the TCP/IP networking protocols and includes the Socket, SenverSocket, DatagramPacket, DategramSocket, URL, and URLConnection classes among others.

6.**java.util**:This class packages contains miscellaneons classes that are useful for a variety of programming chores. Those classes include Date, Dictionary, Random (for creating random number), Vector, and Stack (for implementing a last-in-first-out'‘(LIFO) stack).

Java -Enabled Web Browser

With Internet, Web Browser is a very important program. A Web Browser is nothing but a viewer program used by the client machine to display Web documents. A Jab-Enabled browser is a browser that supports Java applets. Currently many Java-capable Web Browser are known, out of which, most popular are HotJava, Netscape Navigator, and Microsoft Internet Explorer. All these Browser software are free releases and can be down loaded from the respective manufacturers web site as listed below:

1.HotiJova from Java softs Web site http://java.sun.com
2.Navigator from Netscapes home page http://home.netscape.com
3.Internet Explorer from Microsofts Web page http://www.microsoft.com

Other Third-party Tools

A lot of third party tools are currently available in the market in order to support the productivity of the software development using Java. Important of them are listed as below:

1.**Suns Java Workshop dev 5** from Suns Javasoft provids most robust development environment for Java. The recent addition with this is Visual Java.

2.**Ed for Windows** from Soft As It Get provides an Integrated Development Environment (IDE) for Java. It makes Java programming extremely easy and also provides one of the most powefull source code editor.

3.**Mojo** from Penumbra Software is one of the best visual environments for creating Java applets. Its primary goal is to allow rapid generation of Java applets without writing code.

4.**Jamba** from Aimtech and IBM is another graphical'‘applet builder and very powerful in Java environment.

```
5.Semantic Cafe from Semantic another very powerful and is fully integrated Development Environment (IDE).
This is aimed for professional application developers and programmers. Currently Cafe is the de facto
standard for Java development on Windows NT/95.
```

Assignment

- **Q: Explain JDK, JRE and JVM.**
- **Q: Why Java is platform independent?**
- **Q: Why java is not 100% Object-oriented?**
- **Q: List any five features of Java?**
- **Q: Which component is used to compile, debug and execute java program?**
- **Q: Which component is responsible to run java program?**
- **Q: What is use of interpreter?**

Q&A

Q: I know a lot about HTML, but not much about computer programming. Can I still write Java programs?

A: If you have no programming experience whatsoever, you most likely will find programming Java significantly more difficult than HTML. However, Java is an excellent language to learn programming with, and if you patiently work through the examples and the exercises in this book, you should be able to learn enough to get started with Java.

Q: What's the relationship between JavaScript and Java?

A: They have the same first four letters.
A common misconception in the Web world today is that Java and JavaScript have more in common than they actually do. Java is the general-purpose programming language that you'll learn about in this book; you use it to create applets. JavaScript is a Netscape-invented scripting language that looks sort of like Java; with it you can do various nifty things in Web pages. They are independent languages, used for different purposes. If you're interested in JavaScript programming, you'll want to pick up another book, such as Teach Yourself JavaScript in a Week or Laura Lemay's Web Workshop: JavaScript, both also available from Sams.net Publishing.

Q: What is Hot Java?

A: □'‘Hot Java was the first Web browser that could download and play (execute) Java applets.
□'‘Hot Java, created by Sun, is simply a browser, much like the Netscape Navigator or Microsoft's Internet Exploere.
□'‘Although Hot Java was the first browser to support Java applets, many browsers now support or will soon support applets.
□'‘Starting with Netscape Navigator 2.0 for example, you can play Java applets for many platforms (Windows 95, the Mac and so on.).
□'‘Another distinguishing feature of Hot Java is that unlike most browsers which

are written in C/C++.

□’‘Hot Java browser is written with the Java programming language.

Q: I followed all the directions you gave for creating a Java applet. I loaded it into HotJava, but Hello World didn't show up. What did I do wrong?

A: Don't use HotJava to view applets you've created in this book; get a more up-to-date browser such as Netscape or Internet Explorer. HotJava was an experimental browser and has not been updated since soon after its original release. The steps you take to define and write an applet have changed since then, and the applets you write now will not run on HotJava.

Q: What is static in Java?

A: Static means one per class, not one for each object no matter how many instance of a class might exist. This means that you can use them without creating an instance of a class.

Static methods are implicitly final, because overriding is done based on the type of the object and static methods are attached to a class, not an object.

A static method in a superclass can be shadowed by another static method in a sub class, as long as the original method was not declared final.

However, you can't override a static method with a non static method. In other words, you can't change a static method into an instance method of a subclass.

Q: Why java does not support pointers?

A: Because pointers are unsafe, Java uses reference types to hide pointer and programmers feel easier to deal with reference types without pointers.

Q: I'm using Notepad on Windows to edit my Java files. The program insists on adding a `.txt` extension to all my files, regardless of what I name them (so I always end up with files like `HelloWorld.java.txt`). Short of renaming them before I compile them, what else can I do to fix this?

Q: What is a platform?

A: A platform is the hardware or software environment in which a program runs. Most platforms can be described as a combination of the operating system and hardware, like Windows 2000/XP, Linux, Solaris and MacOs.

Q: What is a JVM consists of?

A: Each time of a Java Application is executed then an instance of JVM, responsible for its running, is created.

A JVM instance is described in terms of subsystems, memory areas, data types, and instructions.

Q: How Java enabled high performance?

A: Java uses Just In Time compiler to enable high performance.

Just-In-Time compiler is a program that runs Java bytecode, which is a program that contains instructions that must be interpreted into instructions that can be sent directly to the processor.

Q: What is JTI compiler?

A: Just In Time compiler: It is used to improve the performance.

JIT compiles parts of the byte code that have similar functionality at the same time, and hence reduces the amount of time needed for compilation.

Here the term compiler refers to translator from the instruction set of a Java virtual machine (JVM) to the instruction set of a specific CPU.

Q: What are the supported platforms by Java Programming Languages?
A: Java runs on a variety of platforms, such as

```
Windows

Mac OS and

The various versions of UNIX/Linux like

HP-Unix

Sun Solaris

Redhat Linux

Ubuntu

CentOS etc.
```

CHAPTER 2

Programming in Java

by Debasis Samnata

CONTENTS

- Introduction
- Building Java Applications
- Building Java applets
    - Basic Methods in Applet
- Differences between applet and Application
- Practice Questions
- Assignment
- Q&A

Introduction

Java provides us to write Java programs in two flavors : applets and Applications. These two kind of programs are differ by the way how a browser and Java interpreter will deal them. Simply speaking, a Java applet is a program that appears embedded in a web document, whereas, a Java Application (it is expressed with capital letters to differentiate from all programs as applications) is the term applied to all other kind of Java programs. There is however number of technical differences between applet and Application. Before gong to discuss about their differences, let us see how these two kind of programs can be developed.

Building Java Applications

Consider the following few codes as the first Java Application for you :

```
Illustration’‘2.1                          //’‘‘‘‘‘Hello Java Application’‘‘‘‘‘//


class helloworld{
          public static void main(String args[]){
                    System.out.println("Hello, World!");
                    System.out.println("Hi...."+" Debasis");
          }
}
```

These lines comprise the minimum components necessary to print **Hello Java !** onto the screen .

NOTE:

1. **How to edit this Program**: Java program Can be written with any text editor. In Windows or System 7.5 we can use Notepad or EDIT, in Unix vi or emacs can be used. Save this file having name HelloJavaAppln.Java. The name of the Java source file is not arbitrary; it must be the same as the name of the public class (here it is HelloJavaAppln) and extension must be java.
2. **How to compile this Program** : The Java compiler converts the Java programs into Java byte codes.

For Window 95 and NT users, you will need to open a DOS Shell window make sure that you are in the directory you are using to store your Java classes, then enter the following command :

```
javac’‘helloworld.java
```

After the successful compilation, Java byte codes will be produced which will be automatically stored in the same directory but with file name having **extension .class** (e.g. here the class file name will be HelloJavaAppln.class).

3.**How to run the Java Application**: To run the Application, you have to type the command java (from the commend prompt).

For example, our helloworld Application can be run as:

```
java’‘helloworld
```

With this the Java interpreter will be invoked and it executes the byte code stored in **helloworld.class** file. The output then will appear onto the screen.

4. **Basic structure of any Application** : An Application must contain at least one class which include the main function. It may contain more than one class also. In any case, the name of the Application (program file) will be according to the name of the class which contains **main (**) function.

Illustration 2.2

```
//‘‘ TestArray.java’‘‘‘‘‘‘‘‘‘‘‘‘
class TestArray{‘‘
‘‘ public static void main(String args[]){
‘‘‘‘‘‘ int b[]= {10, 20, 30, 40, 50};’‘‘‘‘‘//Initialization
‘‘‘‘‘‘//Traversing array’‘
‘‘‘‘‘‘for (int i=0; i < a.length; i++){‘‘‘‘//length is the property of array’‘
‘‘‘‘‘‘‘‘ System.out.print(a[i]+" ");’‘
‘‘‘‘‘‘}
          ‘‘System.out.println();
‘‘‘‘‘‘// Average calculation
‘‘‘‘‘‘float sum = 0, avg;
‘‘‘‘‘‘for(int i=0; i < a.length;i++)’‘‘‘//Calculating the sum of the numbers’‘
‘‘‘‘‘‘‘‘ sum += a[i];’‘
‘‘‘‘‘‘avg = sum/a.length;
‘‘‘‘‘‘System.out.println("Average = " + avg);’‘
‘‘ }
}’‘
```

Type following two commands :

```
javac TestArray.java          // To compile the source code
java TestArray                          // To run the Application
```

OUTPUT:
10 20 30 40 50

Average = 30.0

Illustration 2.3

```
//‘‘ a3DArray.java’‘‘‘‘‘‘‘‘‘‘‘‘‘
class a3DArray {
‘‘‘‘‘‘public static void main(String args[]) {

          int my3DArray [ ][ ][ ] = new int [3][4][5];

          int i, j, k;
          for(i=0; i<3; i++)
          ‘‘‘‘‘‘for(j=0; j<4; j++)
          ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘for(k=0; k<5; k++)
                    ‘‘‘‘‘‘‘‘‘‘‘‘‘‘ my3DArray[i][j][k] = i * j * k;
                    ‘‘
                    ‘‘
          for(i=0; i<3; i++) {
          ‘‘‘‘‘‘for(j=0; j<4; j++) {
          ‘‘‘‘‘‘‘‘‘‘‘‘ for(k=0; k<5; k++)
                    ‘‘‘‘‘‘System.out.print(my3DArray[i][j][k] + " ");
                    ‘‘‘‘‘‘System.out.println();
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ }
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘System.out.println();
‘‘‘‘‘‘‘‘‘‘}
‘‘‘‘ }
}
```

OUTPUT:

```
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

0 0 0 0 0
```

```
0 1 2 3 4
0 2 4 6 8
0 3 6 9 12

0 0 0 0 0
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24
```

Note:

The source code stored in a program file whose extension is java; the compiled version of the program file is stored in a file having same name as the source program file but having extension class. These class files are containing java byte code and can be interpreted by Java run time environment i.e. Java interpreter.

Illustration 2.4

```
// A Java program to demonstrate working of print() and println() together‘‘
// Edit Demonstration_41.java

class Demonstration_41
{
‘‘‘‘public static void main(String[] args) {
‘‘‘‘      int x=555;
‘‘‘‘      System.out.println("1. println ");
‘‘‘‘      System.out.println("2. println ");
‘‘‘‘
‘‘‘‘      System.out.print("1. print ");
                    System.out.print("\n");
‘‘‘‘      System.out.print("2. print "+ "3. Print "+ " " +x);
‘‘‘‘}
}
```

OUTPUT:

1. println
2. println
1. print
2. print 3. Print’‘555

Illustration 2.5

```
// A Java program to demonstrate working of printf() in Java
// Edit Demonstration_42.java


class Demonstration_42

{
‘‘public static void main(String args[])
‘‘{
‘‘‘‘int x = 100;
‘‘‘‘System.out.printf("Printing simple integer: x = %d\n", x);
‘‘
‘‘‘‘// this will print it upto 2 decimal places
‘‘‘‘System.out.printf("Formatted with precison: PI = %.2f\n", Math.PI);
‘‘
‘‘‘‘float n = 5.2f;
‘‘
‘‘‘‘// automatically appends zero to the rightmost part of decimal
‘‘‘‘System.out.printf("Formatted to specific width: n = %2.1f\n", n);
‘‘
‘‘‘‘n = 2324435.3f;
‘‘
‘‘‘‘// here number is formatted from right margin and occupies a
‘‘‘‘// width of 20 characters
‘‘‘‘System.out.printf("Formatted to right margin: n = %6.2f\n", n);
‘‘}
}
```

OUTPUT:

```
Printing simple integer: x = 100
Formatted with precison: PI = 3.14
Formatted to specific width: n = 5.2
Formatted to right margin: n = 2324435.25
```

Illustration 2.6

```
/*’‘Command line input in Java */
// Edit Demonstration_43.java

public class Demonstration_43{
‘‘‘‘‘‘ public static void main(String args[]){
                    ‘‘ int i=0;
                    ‘‘ i=Integer.parseInt(args[0]);
                    ‘‘ System.out.println("The value of i in Integer is "+ i);

          ‘‘‘‘
          //}
          //System.out.print(args[i]+" ");
          //System.exit(0);
‘‘‘‘‘‘ }
‘‘ }
```

Illustration 2.7

```
// Program to check for command line arguments
class Demonstration_44
 {
‘‘‘‘public static void main(String[] args)
```

```
‘‘‘‘{
‘‘‘‘‘‘‘‘// check if length of args array is
‘‘‘‘‘‘‘‘// greater than 0
‘‘‘‘‘‘‘‘if (args.length > 0)
‘‘‘‘‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println("The command line"+
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ " arguments are:");
‘‘
‘‘‘‘‘‘‘‘‘‘‘‘// iterating the args array and printing
‘‘‘‘‘‘‘‘‘‘‘‘// the command line arguments
‘‘‘‘‘‘‘‘‘‘‘‘for (String val:args)
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘System.out.println(val);
‘‘‘‘‘‘‘‘}
‘‘‘‘‘‘‘‘else
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println("No command line "+
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ "arguments found.");
‘‘‘‘}
}
```

OUTPUT:

```
No command line arguments found.
```

Illustration 2.8

```
/*The program demonstrates how to take Input using scanner class*/
//Edit Demonstration_45.java’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘import java.util.Scanner;

‘‘‘‘public class Demonstration_45 {
‘‘‘‘‘‘‘‘public static void main(String args[]) {
                                Scanner scnr = new Scanner(System.in);
                                // Calculating the maximum two numbers in Java
                                System.out.println("Please enter two numbers to find maximum of two");
                                int a = scnr.nextInt();
                                int b = scnr.nextInt();
                                if (a > b) {
```

```
                                        System.out.printf("Between %d and %d, maximum is %d \n", a, b,
a);
                                }
                                else {
                                        System.out.printf("Between %d and %d, maximum number is %d \n",
a, b, b);
                                }
                        }
‘‘‘‘}
```

Illustration 2.9

```
/*The following program snippet shows how to read and write to the console.*/
//Edit Demonstration_46.java’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

import java.util.*;

public class Demonstration_46
{
‘‘public static void main ( String args[] )
‘‘{
‘‘‘‘System.out.print ( "Enter the radius: " );

‘‘‘‘Scanner sc = new Scanner (System.in);

‘‘‘‘double radius = sc.nextDouble();

‘‘‘‘double area = 3.14159 * radius * radius;

‘‘‘‘System.out.println ( "Area is: " + area );
‘‘}
}
```

Illustration 2.10

```
/*Input with DataInputStream InterestCalculator Program */
//Edit Demonstration_47.java’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
```

```
import java.io.*;
class Demonstration_47{
‘‘ public static void main(String args[ ] ) throws Exception { // throws Exception
‘‘‘‘‘‘ Float principalAmount = new Float(0);
‘‘‘‘‘‘ Float rateOfInterest = new Float(0);
‘‘‘‘‘‘ int numberOfYears = 0;
          ‘‘ //try{
‘‘‘‘‘‘ DataInputStream in = new DataInputStream(System.in);
          ‘‘
‘‘‘‘‘‘ String tempString;
‘‘‘‘‘‘ System.out.print("Enter Principal Amount: ");
‘‘‘‘‘‘ System.out.flush();
‘‘‘‘‘‘ tempString = in.readLine();
          ‘‘
‘‘‘‘‘‘ principalAmount = Float.valueOf(tempString);
‘‘‘‘‘‘ System.out.print("Enter Rate of Interest: ");
‘‘‘‘‘‘ System.out.flush();
‘‘‘‘‘‘ tempString = in.readLine();
‘‘‘‘‘‘ rateOfInterest = Float.valueOf(tempString);
‘‘‘‘‘‘ System.out.print("Enter Number of Years: ");
‘‘‘‘‘‘ System.out.flush();
‘‘‘‘‘‘ tempString = in.readLine();
‘‘‘‘‘‘ numberOfYears = Integer.parseInt(tempString);
‘‘‘‘‘‘ // Input is over: calculate the interest
                    float interestTotal =‘‘‘‘
                    principalAmount*rateOfInterest*numberOfYears;
‘‘‘‘‘‘ System.out.println("Total Interest = " + interestTotal);
          //’‘ }
           //’‘catch (Exception ex)
           //’‘{}
‘‘‘‘}
}
```

Building Java applets

How to run an Applet?

There are two ways to run an applet
1. By html file.
2. By appletViewer tool (for testing purpose).

Applets are the most effective Java programs is shown in **Illustration 2.11** :

Illustration 2.11 : **// An applet version of HelloHavaAppln //**

```
/* create an applet and compile it. After that create an html file and place the applet code in html file.
Now click the html file. */


import’‘java.awt.Graphics;

import java.applet.Applet;

public class HelloJava extends Applet {

          public void paint (Graphics g ) {

‘‘‘‘‘‘‘‘‘‘‘g.drawString ( ﷺ Hello Java !ﷺ 50, 25 );

          }

}


// Html code


< html>‘‘

< body>‘‘

< applet code=" HelloJava.class" width="200" height="100">‘‘

< / applet>‘‘

< / body>‘‘

< / html>
```

These are the essential code to display the message **Hello Java !** onto the screen. Novice programmer may be puzzled with so many new lines, but this is just for an adventure, actual understanding will take place in due time. This applet will be edited in the same fashion as Application. The name of the applets will be same as the public class (here HelloJava.java). The program then can be compiled using **javac** in the same manner as an Application is compiled. The Java compiler produces a file named **HelloJava.class .**

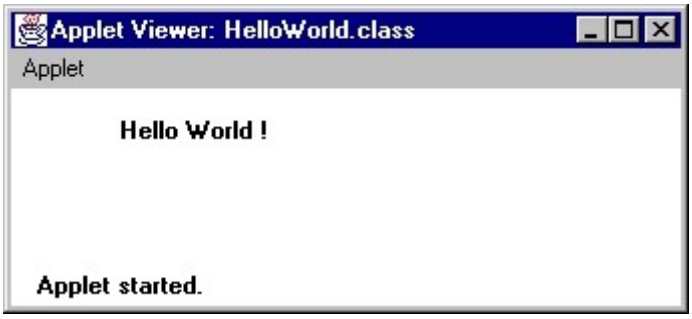
How to run the Java applet : Before going to run the Java applet, we have to create an HTML document to host it. In HTML, type this few line through any editor (donﷺt worry, everything will be discussed in detail).

```
< applet code = HelloJava.class width =200’‘height =100>‘‘< / applet >
```

This is the minimum HTML Page for running the Java Applet HelloJava. Save this file by giving a name **HelloJava.html** (the name of the file should be same as the name of the class and extension must be html).

Afte the HTML page is prepared, the applet is ready for execution.If you have any browser (like HotJava or Netscape), invoke this browser, open the HTML file and click then RUN button on it. Other way, the JDK includes a program **appletviewer** which also can be used to run the applet. Type the following command on command line:

appletviewer HelloJava.html.



NOTE:
Figure 2.1 represents the basic components in any Java applet : Let us explain the various parts in context of the applet HelloJava : **Part 1**: HelloJava imports two classes, namely : **java.awt.Graphics;** **java.applet.Applet;** The different classes that can be imported will be discussed in due time. **Part 2**:Name of the applet **HelloJava** is placed in place of **NewAppletName.** **Part 3** : There are no variable declared and used in this applet . This part is hence, optional. **Part 4**:Only one method vide **public void paint(Graphics g)** is declared in this applet which is defined with a function **drawString (��)** ; this function is defined in **Graphics.class.** Here, **Part 4** is the *main component in any applet*. This includes the various method(s) which actually defines the applet. These methods are responsible for interacting between the browser and applet. In fact, no user defined routines are required to define an applet. There are number of routines available in the class Applet. Programmer can use these or can override these function. Following **Section 2.2.1** gives an idea about these methods

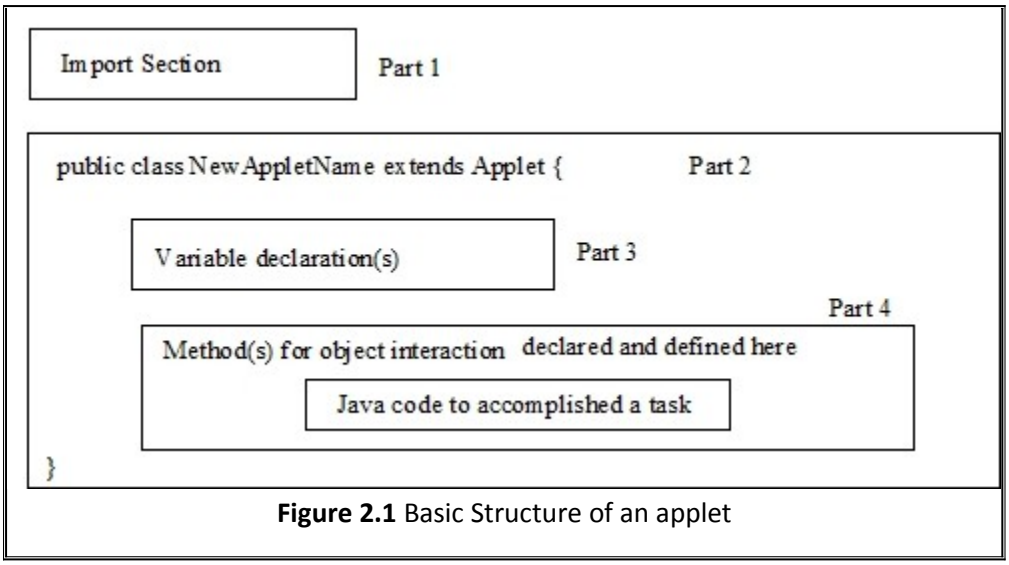


Figure 2.1 Basic Structure of an applet

Illustration 2.12 :

```
/* A "Hello, World" Applet*/
// Demonstration_22.java''

import java.applet.*;
import java.awt.*;

public class Demonstration_22 extends Applet {
'' public void paint (Graphics g) {
''''''g.drawString ("Hello World", 25, 50);
'' }
 }''
}''

// Html code
< html>
'' < title>The Hello, World Applet< / title>
'' < hr>
'' < applet code = "Demonstration_22.class" width = "320" height = "120">
''''''If your browser was Java-enabled, a "Hello, World"
''''''message would appear here.
'' < / applet>
'' < hr>
< / html>
```

Basic Methods in Applet

There is a number of applet member functions are define in java.applet.Applet class. These applet methods are very much useful to define any applet. During the building of an applet, a programmer can override them, of course, some methods can not be allowed for overriding. Another point is there, these method be called in an applet in a given order. These methods are briefly discussed in the order in which they should be placed in an applet.

Public void init ( )

The **init ( )** method gets called first. This member function is called only once and is called at the time the applet is created and first loaded into Java-enabled browser (such as the appletviewer).

Illustration 2.13

```
/* Resize Applet Window Example*/
// Demonstration_23.java''

import java.applet.Applet;
import java.awt.Graphics;

public class Demonstration_23 extends Applet{

          public void paint(Graphics g){
                    /*
                     * To resize an applet window use,
                     * void resize(int x, int y) method of an applet class.
                     */
                    resize(300,300);
                    g.drawString("Window has been resized to 300,300", 50, 50);
          }

}

// Html code
< html>
'' < title>The Hello, World Applet< / title>
'' < hr>
'' < applet code = "Demonstration_23.class" width = "200" height = "200">''''''
'' < / applet>
'' < hr>
< / html>
```

One can pass number of information to an applet through HTML document by init( ) method. Illustration 2.14 is such an attempt.

```
Illustration 2.14                                    // Use of'‘init( ) to pass value through HTML to
applet //


import java .awt . *;
import java.applet. * ;

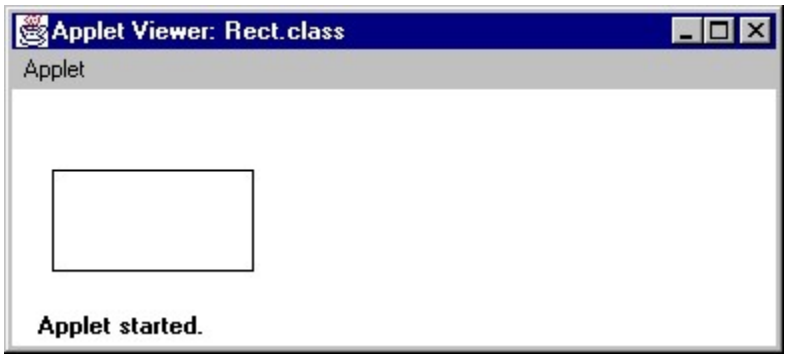

public class RectangleTest extends applet {
          int x, y, w, h;
          public void init (‘‘) {
                              x'‘= Integer.parseInt(get Parameter (□ xValue□ ));
                              y'‘= Integer.parseInt(get Parameter (□ yValue□ ));
                              w = Integer.parseInt(get Parameter (□ wValue□ ));
                              h'‘= Integer.parseInt(get Parameter (□ hValue□ ));
                    }

                    public void paint ( Graphics g ) {
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘g.drawRect (x, y, w, h );
‘‘‘‘‘‘‘}
‘‘‘‘}
```

Corresponding HTML document containing this applet and providing parameter values is mentioned as below :

< applet code = □ RectangleTest□ width = 150 height = 100 >
< param name = xValue value = 20 >
< param name = yValue value = 40 >
< param name = wValue value = 100>
< param name = hValue value = 50 >

Observe, how **< param □.>** tag and **getParameter (String S)** (defined in Applet Class) can be used. It is not necessary that ***getParameter(..) should be in the same order as parameter values passed in HTML; also if some parameter is not available, getParameter(..) will return a default value which is null.***



Public void start()
The start() method called after the init() method; it starts the applet. Suppose this method is overriden in an applet MusicApp. class :

```
‘‘‘‘public void start (‘‘) {
          ‘‘is Stopped = false ;
                    // Start the music //.
                    MusicClip.play (‘‘);’’’’’’’’’’’’’’’’ // This is’‘defined in some where.
          }
```

Suppose, this applet is loaded in an HTML file, and that HTML document is displayed on the screen; there is a link point on the document for the applet. If we click this link point (by mouse) then this applet will start its working. With the help of this function, then an applet can be called multiple times if the user keeps leaving and returning to the HTML document. It can be noted that, **init()** is called once - the first time an applet is loaded, start() is called each time the applet’s link is activated. Actual illustration of **start()** method will take place after discussing **Thread and Event.**

Public void stop()
The stop() method stops the applet running. This method is called when the browser wishes to stop executing the document or whenever the user leaves the applet. This method can be overriden in an applet, an example includes as below :

```
‘‘‘‘public void stop ( ) {
          ‘‘ is Stopped = true;
          ‘‘ if ( / * music is playing ? */) {
                              // Stop the music
                              ‘‘‘‘MusicClip.stop (‘‘)’’’’’’’’// This should be define in some where’‘          }
          }
```

Public void paint(Graphics g)
This method can be overriden in applet. Whenever browser needs to redraw the applet. A number of graphics methods are defined in java.awt.Graphics which can be included in this method to define it. What are the different graphics drawing method is available will be discussed during the discussion of ***Graphics in Java (Chapter 7, Part II and Chapter 4, Part III).***

Public void destroy( )

This method is called when the browser determines that the applets need to be remove completely from memory. The java.applet.Applet class does nothing in this member function, In the applet class (derived class), therefore, user should override this method to do final cleanup and freeing any resources holding the ceased applet. Detail use of this method will be illustrated during the discussion of **Multi-threading in Java (Chapter 6, Part II).**

Illustration 2.15

```
import java.applet.Applet;
import java.awt.Color;
import java.awt.Graphics;
```

```
public class Demonstration_24 extends Applet {
‘‘‘‘public void init() {
‘‘‘‘‘‘‘‘// Here we change the default grey color background of an
‘‘‘‘‘‘‘‘// applet to yellow background.
‘‘‘‘‘‘‘‘setBackground(Color.YELLOW);
‘‘‘‘}

‘‘‘‘public void paint(Graphics g) {
‘‘‘‘‘‘‘‘g.setColor(Color.BLACK);
‘‘‘‘‘‘‘‘g.drawString("Applet background example", 0, 50);
‘‘‘‘}
}

// Html code
< html>
‘‘ < title>The Hello, World Applet< / title>
‘‘ < hr>
‘‘ < applet code = "Demonstration_24.class" width = "200" height = "200">‘‘‘‘‘‘
‘‘ < / applet>
‘‘ < hr>
< / html>
```

Illustration 2.16

```
/* Specifying Applet Parameters*/



public void init () {
‘‘ String squareSizeParam = getParameter ("squareSize");
‘‘ parseSquareSize (squareSizeParam);’‘
‘‘ String colorParam = getParameter ("color");
‘‘ Color fg = parseColor (colorParam);’‘
‘‘ setBackground (Color.black);
‘‘ setForeground (fg);
}

private void Demonstration_25 (String param) {
```

```
‘‘ if (param == null) return;
‘‘ try {
‘‘‘‘‘‘squareSize = Integer.parseInt (param);
‘‘ } catch (Exception e) {
‘‘‘‘‘‘// Let default value remain
‘‘ }
}



// Html code
< html>
‘‘ < title> The Hello, World Applet < / title>
‘‘ < hr>
‘‘ < applet code = " Demonstration_25.class" width = "480" height = "320">
‘‘‘‘‘‘< param name = "color" value = "blue">
‘‘‘‘‘‘< param name = "squaresize" value = "30">
‘‘ < / applet>
‘‘ < hr>
< / html>
```

Illustration 2.17

```
/* Simple example of Applet by appletviewer tool:*/


import java.applet.Applet;’‘
import java.awt.Graphics;’‘
public class Demonstration_26 extends Applet{‘‘
‘‘
          public void paint(Graphics g){‘‘
                    g.drawString("welcome to applet",150,150);’‘
          }’‘
‘‘
}’‘


/*
< applet code="Demonstration_26.class" width="300" height="300">
< / applet>
```

```
*/''


To execute the applet by appletviewer tool, write in command prompt:

c:\>javac Demonstration_21.java

c:\>appletviewer Demonstration_21.java
```

Differences between applet and Application

In ths Section, we will find the technical differences between applet and Application. As we illustrated earlier, a Java interpreter is a class that the Java Application is a class that the Java interpreter knows is going to have a specific method in it called main( ), which it looks for when it tries to run the Application. On the other hand, a Java applet is a specific kind of application that is run from the browser (like HotJava, Netscape, appletviewer (known also as a mini browser)), rather than having a single main( ) method, a Java applet instead implements a set of methods that deal with of how to interact with the browser and applet.

Another technical difference between applet and Application stem from the context in which they run. A Java Application runs in the simplest possible environment - its only input from the outside world is a list of command line parameters. On the other hand, a Java applet needs a lot of information from environment; it needs to know when it is initialized, when and where to draw itself in the browser window, and when it is activated or deactivated. As a consequence of these two very different execution environments, applet and Application have different minimum requirement.

The major technical differences are summarized in **Table 2.1.**

Table 2.1

Technical Point	Java Application	Java applet
Method expected by the JVM	main ( ) - start up routine	No main ( ) but some methods
Environment Inputs	Only command line parameters	Parameters are embedded in the host HTML document
Distribution	Loaded from the file system or by a custom class loading process	Java application and transported via HTTP
Memory requirement	Minimal Java application Requirement	Java application plus browser memory requirement
User Graphics	It is optional	It is inherently graphical

Now, here the question that arises is to decide whether we write an applet or an Application to develop an application in Java. Java applets are preferred when graphical user interface is desirable, Java Applications are preferred over applets when graphical displays are undesirable. Java Applications are found in building applications for network servers, consumer electronic etc. (when no graphical displays is involved). On the other hand to prepare Web document, where huge graphical display may involved, and which are to be transported through the Internet, applets are the good solutions.

So for size and complexity are concerned, there is no limit to the size or complexity of the both kind of applications, however, with the Internet where communication speed is limited and down load times are long, most Java applets are small by necessity.

Practice questions

Practice 2.1

```
/*
One more simple Java Application. This application computes square root */
//Edit SquareRoot.java

          import java.lang.Math;
          class SquareRoot{
          public static void main (String args[ ]) {
          double x = 45; // Variable declaration and initialization
          double y;   // Declaration of another variable
          y = Math.sqrt(x);
          System.out.println("Square root of "+ x +"=" + y);
          }
          }
```

Is the compilation successful? What is the output?

Practice 2.2

```
/*
Example java program without any class
*/
// Edit HelloNoClass.java

          public static void main (String args[ ] ) {
                    System.out.println( "Hello Classless Java!]);
                    }


          Type following two commands to run the HelloNoClass.java Application:

                              javac HelloNoClass.java                  // To compile
                              java'' HelloNoClass             // To run the program
```

Is the compilation successful? What is about the execution?

Practice 2.3

```
/*
```

```
Application with more than one classes within same java file’‘
*/
//Edit PeopleApp.java’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
          class FirstClass {
          int idNo;
          idNo = 555;
                    public static void print(‘‘) {
                              System.out.println ( " First Class citizen"‘‘+ idNo );’‘
                    }
          }

          class SecondClass {
          int idNo;
                    idNo = 111;
                    public static void print(‘‘) {
                              System.out.println ( " Second Class citizen " + idNo) ;
                    }
          }
          public class PeopleApp {
          FirstClass female;
          SecondClass male;
                    public static void main( String args[ ] ) {
                              System.out.print("People from Java World");
                              female.print( );
                              male.print( );
                    }
          }
```

(This problem has a minor mistake. Identify the mistake and then write the correct code.)

Practice 2.4

```
/*
Application with more than one class in separate java file’‘
*/
//Edit AnotherFirstClass.java’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
          class AnotherFirstClass {
          static int idNo = 777;
                    public static void print(‘‘) {
                              System.out.println ( " First Class citizen"‘‘+ idNo );’‘
                    }
```

```
          }
          //Edit AnotherSecondClass.java
          class AnotherSecondClass {
          static int idNo = 888;
                    public static void print(‘‘) {
                              System.out.println ( " Second Class citizen " + idNo) ;
                    }
          }

          //Edit AnotherPeopleApp.java

          public class AnotherPeopleApp {
                    public static void main( String args[ ] ) {
                              AnotherFirstClass female = new AnotherFirstClass();
                              AnotherSecondClass male = new AnotherSecondClass();
                              System.out.print("People from Java World");
                              female.print( );
                              male.print( );
                    }
}
```

What is the output?

Practice 2.5

```
/*
This program passes inputs to the Application through command line arguments
*/
// CommnadLineInputTest.java

class CommnadLineInputTest{
public static void main(String args[ ] ) {
          int count;
          String aString;
          count = args.length;

          System.out.println( "Number of arguments ="+ count);

          for(int i = 0; i < count; i++) {
                    aString = args[0];
                    System.out.println( "args["+i+"]"+"="+ aString);
                    }
          ‘‘‘‘}
```

```
          }
Type following two commands to run the CommandLineInputTest.java Application :
          javac CommandLineInputTest.java
          java’‘CommandLineInputTest Kolkata Chennai Mumbai Delhi Goa
```

What is the output?

Practice 2.6

```
/*
This is a Java Application to read a number from the standard input * and display the number.
*/
// ReadNumber.java

          import java.io.*;

          class ReadNumber{
                    public static void main(String args[ ] ) {
                    Float number1 = new Float(0);
                    Float number2 = new Float(0);

          DataInputStream in = new DataInputStream(System.in);
                    String tempString;

                    System.out.print("Enter a number: ");
                    System.out.flush();
          tempString = in.readLine();
                    number1’‘= Float.valueOf(tempString);

                    System.out.print("Enter another number: ");
                    System.out.flush();
                    tempString = in.readLine();
                    number2 = Float.valueOf(tempString);

                    System.out.println("Number 1: "+number1);
          System.out.println("Number 2: "+number2);

           }
```

What is the output?

Practice 2.7

```
/* while loop example */
          class WhileDemo {
```

```
            public static void main(String[] args){
            int count = 1;
            while (count < 11) {
            System.out.println("Count is: " + count);
            count++;
}
}
}
```

What is the output?

Practice 2.8

```
/* DoWhile loop example */
            class DoWhileDemo {
            public static void main(String[] args){
            int count = 1;
            do {
            System.out.println("Count is: " + count);
            count++;
            } while (count <= 11);
            }
            }
```

What is the output?

Practice 2.9

```
/* while loop example */
            class WhileDemo {
            public static void main(String[] args){
            int count = 1;
            while (count < 11) {
            System.out.println("Count is: " + count);
            count++;
}
}
}
```

What is the output?

Practice 2.10

```
/* For loop example */
            class ForDemo {
            public static void main(String[] args){
```

```
           for(int i=1; i<11; i++){
           System.out.println("Count is: " + i);
           }
           }
           }
```

What is the output?

Practice 2.9

```
/* Loop’‘example with continue */
           class Continue {
           public static void main(String args[]) {
           for(int i=0; i<10; i++) {
           System.out.print(i + " ");
           if (i%2 == 0) continue;
           System.out.println("");
           }
           }
           }
```

What is the output?

Practice 2.11

```
/* Loop example with break*/
           class BreakLoop {
           public static void main(String args[]) {
           for(int i=0; i<100; i++) {
           if(i == 10) break; // terminate loop if i is 10
           System.out.println("i: " + i);
           }
           System.out.println("Loop complete.");
           }
           }
```

what is the output?

Practice 2.12

```
/* Test for primes */
           class FindPrime {
           public static void main(String args[]) {
           int num;
           boolean isPrime = true;
           num = 14;
```

```
            for(int i=2; i <= num/2; i++) {
            if((num % i) == 0) {
            isPrime = false;
            break;
            }
            }
            if(isPrime) System.out.println("Prime");
            else System.out.println("Not Prime");
            }
            }
```

what is the output?

Practice 2.13

```
/* Menu selection using do-while and switch-case */
            class Menu {
            public static void main(String args[]) throws java.io.IOException {
            char choice;
            do {
            System.out.println("Help on:");
            System.out.println(" 1. if");
            System.out.println(" 2. switch");
            System.out.println(" 3. while");
            System.out.println(" 4. do-while");
            System.out.println(" 5. for\n");
            System.out.println("Choose one:");
            choice = (char) System.in.read();
            } while( choice < '1' || choice > '5');

            System.out.println("\n");
            switch(choice) {
            case '1':
            System.out.println("The if:\n");
            System.out.println("if(condition) statement;");
            System.out.println("else statement;");
            break;
            case '2':
            System.out.println("The switch:\n");
            System.out.println("switch(expression) {");
            System.out.println(" case constant:");
            System.out.println(" statement sequence");
            System.out.println(" break;");
            System.out.println(" // ...");
```

```
            System.out.println("}");
            break;
            case '3':
            System.out.println("The while:\n");
            System.out.println("while(condition) statement;");
            break;
            case '4':
            System.out.println("The do-while:\n");
            System.out.println("do {");
            System.out.println(" statement;");
            System.out.println("} while (condition);");
            break;
            case '5':
            System.out.println("The for:\n");
            System.out.print("for(init; condition; iteration)");
            System.out.println(" statement;");
            break;
            }
            }
            }
```

what is the output?

Practice 2.14

```
/* Example of recursion */
            class Recursion {
            public static void main(String args[]) {
            myMethod(10); // pass positive integer
            }

            void myMethod( int counter)
            {
            if(counter == 0)
            return;
            else
            {
            System.out.println("hello" + counter);
            myMethod(--counter);
            System.out.println(""+counter);
            return;
            }
            }
```

```
        }


what is the output?
```

Assignment

Q: Write java program to calculate the Square of a number.

Q: Write java program to add two numbers, take input as command line argument.

Q: Write java program to multiply two numbers, numbers should be taken from standard input.

Q: Write Java program to read the three integers a, b and c from the keyboard and then print the largest value read.

Q: Write a Java Application to read the name of a student (studentName), roll Number (rollNo) and marks (totalMarks) obtained. rollNo may be an alphanumeric string. Display the data as read..

Q: Write Java program to calculate the perimeter and the area of the circle. The radius of the circle is taken as user input. Use to different functions to calculate the perimeter and area. Define the value of PI as class constant.

Q: Write a program to check the number is even or odd. Input is taken from keyboard.

Q: Write a program to find out the number of days in a month using switch-case. Month number and year is taken as input through keyboard.

Q: Write java program to calculate the Sum of the square of first 10 integers.

Q: Write java program to calculate the Factorial of a 10 (iteratively and recursively).

Q: Write a program to calculate the grade of a student. There are five subjects. Marks in each subject are entered from keyboard. Assign grade based on the following rule:

```
Total Marks >= 90    Grade: Ex
90 > Total Marks >= 80         Grade: A
80 > Total Marks >= 70         Grade: B
70 > Total Marks >= 60         Grade: C
60 > Total Marks'‘   Grade: F
```

Q&A

Q: Why are there no global variables in java?

A: Global variables are considered bad form for a variety of reasons: adding state variables breaks referential transparency, state variables lessen the cohesion of a program.

Q: What is Dynamic Method Dispatch?

A: Dynamic method dispatch is the mechanism by which a call to an overridden function is resolved at run time, rather than at compile time.

Q: What is the use of bin and lib in JDK?

A: Bin contains all tools such as javac, appletviewer, awt tool etc. whereas lib contains API and all packages.

Q: What are the different types of casting?

A: There are two types of casting

- ’‘Casting between primitive numeric types
- ’‘Casting between object references

Casting between numeric types is used to convert larger values into smaller values. For e.g. double values to byte values.
Casting between object references helps you refer to an object by a compatible class, interface, or array type reference.

Q: What is heap in Java?

A: Java is fully Object oriented language. It has two phases first one is Compilation phase and second one is interpretation phase.
The Compilation phase convert the java file to class file (byte code is only readable format of JVM) than interpretation phase interoperate the class file line by line and give the proper result.

Q: What type of parameter passing does Java support?

A: In Java the arguments are always passed by value.

Q: What is garbage collection?

A: If no reference to an object, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.This is known as garbage collection

Q: Describe the Garbage Collection process in Java?

A: The JVM spec mandates automatic garbage collection outside of the programmer’s control.
The System.gc() or Runtime.gc() is merely a suggestion to the JVM to run the GC process but is NOT guaranteed.

Q: How do you traverse through a collection using its Iterator?

A: To use an iterator to traverse through the contents of a collection, follow these steps:

□’‘Obtain an iterator to the start of the collection by calling the collection’s iterator() method.

□’‘Set up a loop that makes a call to hasNext(). Have the loop iterate as long as hasNext() returns true.

□’‘Within the loop, obtain each element by calling next().

Q: What are the main implementations of the List interface ?

A: The main implementations of the List interface are as follows :

- ArrayList : Resizable-array implementation of the List interface. The best all-around implementation of the List interface.
- Vector : Synchronized resizable-array implementation of the List interface with additional "legacy methods."
- LinkedList : Doubly-linked list implementation of the List interface. May provide better performance than the ArrayList implementation if elements are frequently inserted or deleted within the list. Useful for queues and double-ended queues (deques).

Q: What are the advantages of ArrayList over arrays ?

A: Some of the advantages ArrayList has over arrays are:

□’‘It can grow dynamically

□’‘It provides more powerful insertion and search mechanisms than arrays.

Q: Why insertion and deletion in ArrayList is slow compared to LinkedList ?

A: □’‘ArrayList internally uses and array to store the elements, when that array gets filled by inserting elements a new array of roughly 1.5 times the size of the original array is created and all the data of old array is copied to new array.

□’‘During deletion, all elements present in the array after the deleted elements have to be moved one step back to fill the space created by deletion. In linked list data is stored in nodes that have reference to the previous node and the next node so adding element is simple as creating the node an updating the next pointer on the last node and the previous pointer on the new node. Deletion in linked list is fast because it involves only updating the next pointer in the node before the deleted node and updating the previous pointer in the node after the deleted node.

Q: How do you decide when to use ArrayList and When to use LinkedList?

A: If you need to support random access, without inserting or removing elements from any

place other than the end, then ArrayList offers the optimal collection. If, however, you need to frequently add and remove elements from the middle of the list and only access the list elements sequentially, then LinkedList offers the better implementation.

Q: What are the main Implementations of the Set interface ?

A: The main implementations of the List interface are as follows:

```
HashSet
TreeSet
LinkedHashSet
EnumSet
```

Q: What Are the different Collection Views That Maps Provide?

A: Maps Provide Three Collection Views.

```
Key Set - allow a map's contents to be viewed as a set of keys.
Values Collection - allow a map's contents to be viewed as a set of values.
Entry Set - allow a map's contents to be viewed as a set of key-value mappings.
```

Q: What is String subsequence method?

A: Java 1.4 introduced Char Sequence interface and String implements this interface, this is the only reason for the implementation of sub Sequence method in String class. Internally it invokes the String substring method.

Q: What are different ways to create String Object?

A: We can create String Object using new operators like any normal java class or we can use double quotes to create a String object.
There are several constructors available in String class to get String from char array, byte array, StringBuffer and StringBuilder.

Q: Can we use String in switch case?

A: Java 7 extended the capability of switch case to use Strings also, earlier versions doesn't support this.

Q: How to convert String to byte array and vice versa?

A: We can use String getBytes() method to convert String to byte array and we can use String constructor new String(byte[] arr) to convert byte array to String.

Q: How do you check if two Strings are equal in Java?

A: There are two ways to check if two Strings are equal or not `&#xfffd;` using "==" operator or using equals method.
When we use "==" operator, it checks for value of Strings as well as reference but in our programming, most of the time we are checking equality of String for value only.

So we should use equal method to check if two Strings are equal or not.

Q: Difference between String, String Buffer and String Builder?

A: When the intern method is invoked, if the pool already contains a string equal to this string object as determined by the equal (Object) method, then the string from the pool is returned.
Otherwise, this String object is added to the pool and a reference to this String object is returned.
These method always return a String that has the same contents as this string, but is guaranteed to be from a pool of unique strings.

Chapter 3

Object Oriented Mechanism in Java

by Debasis Samanta

CONTENTS

- Introduction
- Class Definition in Java
- Declaring Variables
- Constructors
- Variable Types
- Polymorphism in Java
    - Dynamic Method Dispatch
- Access Specification in Java
- Practice Questions

- Assignment

- Q&A

Introduction

The basic element of object oriented Programming in Java is a class. Class is used to build an Application, to define an applet. A class defines the shape and behavior of an object. In Java, programmers are allowed to define their own classes or can borrow the class definition from outside (such as from built in classes, packages etc). In this Chapter, we will learn the theoretical notion of Object Oriented Programming : encapsulation, inheritance, and polymorphism. We will discuss how to create, extend, and instantiate our own classes. After the learning of this Chapter we will begin to utilize the real power of Java-oriented programming style.

Class Definition in Java

A Class is a template for an object. Every object has a class which defines the structure of an object (that means what are various component in it, termed as member elements) its functional inter feces (called methods) which decide what messages the object will respond to and how. The general form of class definition is shown below.

```
Class’‘Class Name    [extends SuperClassName ]
                     ‘‘‘‘‘‘‘‘[ implements Interface ] {
                                          [declaration of member elements ]
                                          [ declaration of methods ]
          }
```

Those are included with [...] are optional. Member elements will be declared with usual convention as in C++ with the following syntax :

type ElementName;

Methods are declared with the same convention of C ++ as :

```
[Modifier ] returnType’‘MethodName (parameter’‘list){
                              Statements (s) ;’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ // Code for definition of the method.
          ‘‘‘‘‘‘‘‘‘‘‘‘‘}
```

Putting the member elements and methods into together in the definition of a class called is called ***encapsulation.*** Following is a simple example to illustrate the defining a class.

Illustration 3.1 **// Encapsulation’‘//**

```
 // A program that uses the circle class
// Call this file Demonstration_31.java
class Circle {
          double x,y; // The coordinates of the center
          double r; // The radius
// Method that returns circumference
          double circumference(){
          return 2*3.14159*r;
          }
// Method that returns area
          double area(){
          return (22/7)*r*r;
          }
}

//The following class declares an object of type Circle
class Demonstration_31{
          public static void main(String args[]){
```

```
                    Circle c = new Circle();
                    c.x = 0.0;
                    c.y = 0.0;
                    c.r = 5.0;
                    System.out.println("Circumference" + c.circumference());
                    System.out.println("Area" + c.area());
          }
}
```

OUTPUT:
Circumference31.4159 Area75.0

In the above **Illustration 3.1**, we have defined two classes namely Circle and Demonstration_31. Circle contains two member elements namely x and y and a method namely **Circle( )**. The class Demonstration_31 act as the main class which contains main( ). Note that how an object (instance of a class) can be created with **new operator**. This **new operator** creates a single instance of a named class and returns a reference to that object.

For example, P1, P3 and P4 are three reference to three instances of object Point. These three objects are instantiated by the getPoint() method. It also should be noted the multiple references to the same object , e.g. P1 and P2 are the two reference to point the same object; this means that any change to the object, the same object to which P1is referencing. Another important point to note in this regard is that, memory is allocated only when they are referred; when control reaches to P1 = null one reference of the same object is ceased to exist and at P2 =null , the memory allocated to it automatically freed. This automatic memory allocation and deallocation is due to **automatic garbage collector in Java .**

Illustration 3.2 **// Encapsulation'‘//**

```
// A program that declares two objects of the Circle class
// Call this file CircleDemo2.java

class Circle {
          double x, y;
          double r;
          double circumference(){
          return 2*3.14159*r;
          }
          double area(){
          return (22/7)*r*r;
}
}
//The following class declares multiple objects of type Circle
```

```
class Demonstration_32
{
public static void main(String args[]){
          Circle c1 = new Circle();
          Circle c2 = new Circle();
          // Initialize the circles
          c1.x = 3.0;
          c1.y = 4.0;
          c1.r = 5.0;
          c2.x = -4.0;
          c2.y = -8.0;
          c2.r = 10.0;
          System.out.println("Circumference Circle 1" + c1.circumference());
          System.out.println("Area Circle 1" + c1.area());
          System.out.println("Circumference Circle 2" + c2.circumference());
          System.out.println("Area Circle 2" + c2.area());
}
}
```

OUTPUT:
Circumference Circle 131.4159 Area Circle 175.0 Circumference Circle 262.8318 Area Circle 2300.0

Constructors

In the last example, we have used the method viz **circle(..)** to initialize an object.

Illustration 3.3

```
class Circle {
          double x,y;
          double r;
          double circumference(){
          return 2*3.14159*r;
          }
```

```
          double area(){
          return (22/7)*r*r;
          }

}

class Box{
‘‘ double width;
‘‘ double height;
‘‘ double depth;
‘‘ double area(){
‘‘‘‘‘‘double a;
‘‘‘‘‘‘ a = (width*height + height*depth + width*depth) * 2;
‘‘‘‘‘‘ return a;
‘‘ }
‘‘ double volume(){
‘‘‘‘‘‘ double v;
‘‘‘‘‘‘ v = width*height*depth;
‘‘‘‘‘‘ return v;
‘‘‘‘}
}



// Declaring objects of type Circle and Box
class Demonstration_33 {
‘‘ public static void main(String args[]){
‘‘‘‘ Circle c = new Circle();
‘‘‘‘ Box b = new Box();
‘‘‘‘ // Initialize the circles
‘‘‘‘ c.x = 3.0; c.y = 4.0; c.r = 5.0;
‘‘‘‘ b.width = 3.0; b.height = 4.0; b.depth = 5.0;
‘‘‘‘ System.out.println("Circumference Circle" + c.circumference());
‘‘‘‘ System.out.println("Area Circle" + c.area());
‘‘‘‘ System.out.println("Area of Box" + b.area());
‘‘‘‘ System.out.println("Volume of Box" + b.volume());
‘‘ }
}
```

OUTPUT:
Circumference Circle31.4159 Area Circle75.0 Area of Box94.0 Volume of Box60.0

Illustration 3.4

```
// A program that uses simple Point class and na◆ve initialization of its data.

class Point {
          int x;
          int y;
          }

// This class declares an object of type Point.
class Demonstration_34{
          public static void main(String args[]) {
          Point mypoint = new Point();

          // assign values to mypoint's instance variables
          mypoint.x = 10;
          mypoint.y = 20;

          // access the values from mypoint◆s instance variable
          System.out.println("x " + mypoint.x);
          System.out.println("x " + mypoint.y);

          }
}
```

OUTPUT:
x 10 x 20

Illustration 3.5

```
/* Encapsulation: Defining a class with method */
class Point
{
          int x,y;
          void setPoint( ) {
          x = 10;
          y = 10;
          }
}

// definition of another class. This is a main class

class Demonstration_35
{
          float distance;
          public static void main (String args[ ]) {
          Point p = new Point( );
          p.setPoint();
          System.out.println ( " x = "+ p.x );
          System.out.println ( " y = "+ p.y );
          }
}
```

OUTPUT:
x = 10 y = 10

Illustration 3.6

```
/* Automatic''initialization in Java through the constructor as in C++*/
/* Encapsulation: Defining a class having method with parameter */
class Point
{
          int x,y;
          void setPoint( int a, int b     ) {
          x = a;
```

```
          y = b;
          }
}

// definition of another class. This is a main class

class Demonstration_36
{
          float distance;
          public static void main (String args[ ]) {
          Point p1 = new Point( );
          Point p2 = new Point( );
          p1.setPoint(15, 20);
          p2.setPoint(0, 0);
          System.out.println ( " x = "+ p1.x );
          System.out.println ( " y = "+ p1.y );
          System.out.println ( " x = "+ p2.x );
          System.out.println ( " y = "+ p2.y );
}
}
```

OUTPUT:
x = 15 y = 20 x = 0 y = 0

Illustration 3.7

```
/* Encapsulation: Utilization of objects in a program */
class Point
{
          int x,y;
          void getPoint(int a, int b) {
          x = a;
          y = b;
}
```

```
}

// definition of another class. This is a main class
class Demonstration_37
{
static double distance;
public static void main (String args[ ]) {
          Point p1 = new Point( );
          Point p2 = p1;
          Point p3 = new Point ( );
          Point p4 = new Point ( );
          p1.getPoint (5, 10 );
          p2.getPoint (15, 20);
          p3.getPoint (20, 30);
          p4.getPoint (30, 40);

          System.out.println (" X1 = " + p1.x +’‘"Y1’‘= " + p1.y );
          System.out.println ("X2=" + p2.x + "Y2 = " +p2.y );
          int dx = p3.x - p4. x;’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘// X2 - X1
          int dy = p3.y - p4. y;         ‘‘‘‘‘‘‘‘‘// y2 - y1
          distance = Math.sqrt (dx * dx + dy * dy );// (X2-X1)2 + (Y2-Y1)2
          System.out.println ( " Distance = "+ distance );
}
}
```

OUTPUT:
X1 = 15 Y1 = 20 X2=15 Y2 = 20 Distance = 14.142135623730951

Illustration 3.8 **//constructor//**

```
// A Java program to demonstrate working of constructor in Java

class Circle {
          double x,y;
          double r;
```

```
          double circumference(){
          return 2*3.14159*r;
}
double area(){
          return (22/7)*r*r;
}
Circle(double a, double b, double c){
          x = a; // Set center x-coordinate
          y = b; // Set center y-coordinate
          r = c; // Set radius
}
}
class Demonstration_38{
public static void main(String arge[]){
          Circle c1 = new Circle(3.0,4.0,5.0);
          Circle c2 = new Circle(-4.0,8.0,10.0);
          System.out.println("Circumference Circle 1" + c1.circumference());
          System.out.println("Area Circle 1" + c1.area());
          System.out.println("Circumference Circle 2" + c2.circumference());
          System.out.println("Area Circle 2" + c2.area());
}
}
```

OUTPUT:
Circumference Circle 131.4159 Area Circle 175.0 Circumference Circle 262.8318 Area Circle 2300.0

Illustration 3.9 **// Constructor //**

```
/* Demontration of constructor Overloading */
// Edit Demonstration_39.java

class Circle {
          double x,y;
          double r;
```

```
          Circle(double a, double b, double c){
          '' x = a; y = b; r = c;
}
Circle(double c){
          x = 0; y = 0; r = c;
}
Circle(Circle c){
          x = c.x; y = c.y; r = c.r;
}
Circle(){
          x = 0.0; y = 0.0; r = 1.0;
}
double circumference(){
          return 2*3.14159*r;
}
double area(){
          return (22/7)*r*r;
}
}

class Demonstration_39{
public static void main(String arge[]){
          Circle c1 = new Circle(3.0,4.0,5.0);
          Circle c2 = new Circle(5.0);
          Circle c3 = new Circle(c1);
          Circle c4 = new Circle();
          System.out.println("Circumference Circle 1" + c1.circumference());
          System.out.println("Area Circle 1" + c1.area());
          System.out.println("Circumference Circle 2" + c2.circumference());
          System.out.println("Area Circle 2" + c2.area());
          System.out.println("Circumference Circle 3" + c3.circumference());
          System.out.println("Area Circle 3" + c3.area());
          System.out.println("Circumference Circle 4" + c4.circumference());
          System.out.println("Area Circle 4" + c4.area());
}
}
```

OUTPUT:
Circumference Circle 131.4159

```
Area Circle 175.0
Circumference Circle 231.4159
Area Circle 275.0
Circumference Circle 331.4159
Area Circle 375.0
Circumference Circle 46.28318
Area Circle 43.0
```

Illustration 3.10

```
/*The following program shows the use of this() to avoid the name-space collision. */
//Edit Demonstration_310.java’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

class Student{‘‘
          int rollno;’‘
          String name, course;’‘
          float fee;’‘
          Student(int rollno, String name, String course){‘‘
          this.rollno = rollno;’‘
          this.name = name;’‘
          this.course = course;’‘
}’‘

Student(int rollno, String name, String course, float fee){‘‘
          this(rollno,name,course);//reusing constructor’‘
          this.fee=fee;’‘
}’‘
void display(){
          System.out.println(rollno+" "+name+" "+course+" "+fee);}’‘
}’‘
class Demonstration_310{‘‘
          public static void main(String args[]){‘‘
          Student s1=new Student(111,"ankit","java");’‘
          Student s2=new Student(112,"sumit","java",6000f);’‘
          s1.display();’‘
          s2.display();’‘
}
}’‘
```

OUTPUT:
111 ankit java 0.0 112 sumit java 6000.0

Illustration 3.11

```
/*Demontration of constructor Overloading */
// Edit Demonstration_311.java

class Circle {
          double x,y;
          double r;
          Circle (double x, double y, double r){
          this.x = x; this.y = y; this.r = r;
}
Circle (double r){
          x = 0; y=0; this.r = r;
}
Circle (Circle c){
          x = c.x; y = c.y; r = c.r;
}
Circle (){
          x = 0.0; y = 0.0; r = 1.0;
}
double circumference(){
          return 2*3.14159*r;
}
double area(){
          return (22/7)*r*r;
}
}

class Demonstration_311{
public static void main(String arge[]){
          Circle c1 = new Circle(3.0,4.0,5.0);
          Circle c2 = new Circle(5.0);
          Circle c3 = new Circle(c1);
          Circle c4 = new Circle();
```

```
          System.out.println("Circumference Circle 1" + c1.circumference());
          System.out.println("Area Circle 1" + c1.area());
          System.out.println("Circumference Circle 2" + c2.circumference());
          System.out.println("Area Circle 2" + c2.area());
          System.out.println("Circumference Circle 3" + c3.circumference());
          System.out.println("Area Circle 3" + c3.area());
          System.out.println("Circumference Circle 4" + c4.circumference());
          System.out.println("Area Circle 4" + c4.area());
}
}
```

OUTPUT:
Circumference Circle 131.4159 Area Circle 175.0 Circumference Circle 231.4159 Area Circle 275.0 Circumference Circle 331.4159 Area Circle 375.0 Circumference Circle 46.28318 Area Circle 43.0

Illustration 3.12

```
// Program to demonstrate use of this: to invoke current class method
// Edit Demonstration_312.java

class A{‘‘
          void m(){System.out.println("hello m");}’‘
          void n(){‘‘
          System.out.println("hello n");’‘
          this.m();’‘
}’‘
}’‘
class Demonstration_312{‘‘
public static void main(String args[]){‘‘
          A a=new A();’‘
          a.n();’‘
}
```

```
}
```

OUTPUT:
hello n hello m

Illustration 3.13

```
/* Special use of this keyword: No object creation to invoke method
*/
//Edit Demonstration_313.java’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
class A{‘‘
          A(){‘‘
          this(5);’‘
          System.out.println("hello a");’‘
}’‘
A(int x){‘‘
          System.out.println(x);’‘
}’‘
}’‘


class Demonstration_313{‘‘
public static void main(String args[]){‘‘
          A a=new A();’‘
}
}
```

OUTPUT:
5 hello a

Illustration 3.14

```
/* Demontration of constructor Overloading */
// Edit Demonstration_314.java

class Circle {
          double x,y;
          double r;

Circle (double x, double y, double r){
          this.x = x; this.y = y; this.r = r;
}

Circle (double r){
          this(0.0, 0.0, r);
}

Circle (Circle c){
          this(c.x, c.y, c.r);
}

Circle (){
          this(0.0, 0.0, 1.0);
}

double circumference(){
          return 2*3.14159*r;
}
double area(){
          return (22/7)*r*r;
}
}

class Demonstration_314{
public static void main(String arge[]){
          Circle c1 = new Circle(3.0,4.0,5.0);
          Circle c2 = new Circle(5.0);
          Circle c3 = new Circle(c1);
          Circle c4 = new Circle();
          System.out.println("Circumference Circle 1" + c1.circumference());
```

```
          System.out.println("Area Circle 1" + c1.area());
          System.out.println("Circumference Circle 2" + c2.circumference());
          System.out.println("Area Circle 2" + c2.area());
          System.out.println("Circumference Circle 3" + c3.circumference());
          System.out.println("Area Circle 3" + c3.area());
          System.out.println("Circumference Circle 4" + c4.circumference());
          System.out.println("Area Circle 4" + c4.area());
}
}
```

OUTPUT:
Circumference Circle 131.4159 Area Circle 175.0 Circumference Circle 231.4159 Area Circle 275.0 Circumference Circle 331.4159 Area Circle 375.0 Circumference Circle 46.28318 Area Circle 43.0

Inheritance

When encapsulation is the first mechanism in Object Oriented Programming, the second fundamental Object Oriented mechanism is inheritance. This allows descendants of a class to inherit all of its member elements and methods from its ancestors as well as creates its own. These descendants are known as subclasses. A class's immediate parent is called its super class. In Java, a special key word extends is used to implement this mechanism. Following is the example to illustrate this.

```
Illustration 3.15'''''''''''''''''''''          // Example of Inheritance in Java //

/* A simple example of inheritance. */

// Create a superclass.
class A {
          int i, j;

          void showij() {
                    System.out.println("i and j: " + i + " " + j);
          }
}
```

```
// Create a subclass by extending class A.
class B extends A {
          int k;

          void showk() {
                    System.out.println("k: " + k);
          }

          void sum() {
                    System.out.println("i+j+k: " + (i + j + k));
          }
}

class Demonstration_61 {
          public static void main(String args[]) {
                    A superOb = new A();
                    B subOb = new B();
                    // The superclass may be used by itself.
                    superOb.i = 10;
                    superOb.j = 20;
                    System.out.println("Contents of superOb: ");
                    superOb.showij();
                    System.out.println();

                    /* The subclass has access to all public members of its superclass. */
                    subOb.i = 7;
                    subOb.j = 8;
                    subOb.k = 9;
                    System.out.println("Contents of subOb: ");
                    subOb.showij();
                    subOb.showk();
                    System.out.println();
                    System.out.println("Sum of i, j and k in subOb: ");
                    subOb.sum();
          }
}
```

OUTPUT:
Contents of superOb: i and j: 10 20 Contents of subOb: i and j: 7 8 k: 9 Sum of i, j and k in subOb: i+j+k: 24

Illustration 3.16’‘‘‘‘‘‘‘‘‘‘‘‘‘

```
/* Inheritance example: initializing through constructor */
class Box {
           double width;
           double height;
           double depth;

           Box(){‘‘‘‘‘‘‘‘// Default setting by this constructor
                      width = 0.0;
                      height = 0.0;
                      depth = 0.0;
           }

           Box(double w, double h, double d) {
                      width = w;
                      height = h;
                      depth = d;
           }

           double volume() { // compute and return volume
                      return width * height * depth;
           }
}

// Here, Box is extended to include weight.
class BoxWeight extends Box {
           double weight; // weight of box

           // constructor for BoxWeight
           BoxWeight(double w, double h, double d, double m) {
```

```
                    width = w;
                    height = h;
                    depth = d;
                    weight = m;
          }
}
class Demonstration_62a {
          public static void main(String args[]) {
                    Box mybox1 = new Box();
                    BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
                    double vol;
                    vol = mybox1.volume();
                    System.out.println("Volume of mybox1 is " + vol);
                    System.out.println();

                    vol = mybox2.volume();
                    System.out.println("Volume of mybox2 is " + vol);
                    System.out.println("Weight of mybox2 is " + mybox2.weight);
          }
}
```

OUTPUT:
Volume of mybox1 is 0.0 Volume of mybox2 is 24.0 Weight of mybox2 is 0.076

Illustration 3.17’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

```
/* Inheritance example: initializing through constructor */
class Box {
          double width;
          double height;’‘
          double depth;

          Box(){‘‘‘‘‘‘‘‘// Default setting by this constructor
                    width = 0.0;
```

```
                    height = 0.0;
                    depth = 0.0;
          }

          Box(double w, double h, double d) {
                    width = w;
                    height = h;
                    depth = d;
          }

          double volume() { // compute and return volume
                    return width * height * depth;
          }
}

// Here, Box is extended to include weight.
class BoxWeight extends Box {
          double weight; // weight of box

          // Constructors for BoxWeight
          BoxWeight() {‘‘‘‘‘‘// Default constructor
                    super();’’’’’’’’’’ // Call the default constructor in the super class
                    weight = 0.0;
          }


          BoxWeight(double w, double h, double d, double m) {
                    super(w, h, d);’‘// Call the overloaded constructor in the super class
                    weight = m;
          }
}

class Demonstration_62b {
          public static void main(String args[]) {
                    Box mybox1 = new Box(10, 20, 15);
                    BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
                    double vol;
                    vol = mybox1.volume();
                    System.out.println("Volume of mybox1 is " + vol);
                    // System.out.println("Weight of mybox1 is " + mybox1.weight); ERROR!
                    System.out.println();
```

```
                    vol = mybox2.volume();
                    System.out.println("Volume of mybox2 is " + vol);
                    System.out.println("Weight of mybox2 is " + mybox2.weight);
          }
}
```

OUTPUT:
Volume of mybox1 is 3000.0 Volume of mybox2 is 24.0 Weight of mybox2 is 0.076

Illustration 3.18’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

```
/* Example of a superclass variable referring to a subclass Object*/

class Box {
          double width;
          double height;
          double depth;

          Box() {‘‘// default constructor
          }

          // constructor used when all dimensions specified
          Box(double w, double h, double d) {
                    width = w;
                    height = h;
                    depth = d;
          }

          double volume() { // compute and return volume
                    return width * height * depth;
          }
}
```

```
// Here, Box is extended to include weight.
class BoxWeight extends Box {
          double weight; // weight of box

          // constructor for BoxWeight
          BoxWeight(double w, double h, double d, double m) {
                    width = w;
                    height = h;
                    depth = d;
                    weight = m;
          }
}

class Demonstration_63 {
          public static void main(String args[]) {
                    BoxWeight weightbox = new BoxWeight(3, 5, 7, 8.37);
                    Box plainbox = new Box();
                    double vol;
                    vol = weightbox.volume();
                    System.out.println("Volume of weightbox is " + vol);
                    System.out.println("Weight of weightbox is " + weightbox.weight);
                    System.out.println();

                    // assign BoxWeight reference to Box reference
                    plainbox = weightbox;
                    vol = plainbox.volume(); // OK, volume() defined in Box
                    System.out.println("Volume of the box is " + vol);
                    /* The following statement is invalid because plainbox does not define a weight
member. */
                    // System.out.println("Weight of plainbox is " + plainbox.weight);
          }
}
```

OUTPUT:
Volume of weightbox is 105.0 Weight of weightbox is 8.37

Volume of the box is 105.0

Illustration 3.19’‘‘‘‘‘‘‘‘‘‘‘‘‘‘ **// Example of Inheritance in Java //**

```
/* Simple example of super concept */

// A complete implementation of BoxWeight.
class Box {
           double width;
           double height;
           double depth;

           // constructor used when all dimensions specified
           Box(double w, double h, double d) {
                      width = w;
                      height = h;
                      depth = d;
           }

           // compute and return volume
           double volume() {
                      return width * height * depth;
           }
}

// BoxWeight now fully implements all constructors.
class BoxWeight extends Box {
           double weight; // weight of box

           // constructor when all parameters are specified
           BoxWeight(double w, double h, double d, double m) {
                      super(w, h, d); // call superclass constructor
                      weight = m;
           }
}

class Demonstration_64 {
           public static void main(String args[]) {
```

```
                    BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
                    BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
                    double vol;
                    vol = mybox1.volume();
                    System.out.println("Volume of mybox1 is " + vol);
                    System.out.println("Weight of mybox1 is " + mybox1.weight);
                    System.out.println();
                    vol = mybox2.volume();
                    System.out.println("Volume of mybox2 is " + vol);
                    System.out.println("Weight of mybox2 is " + mybox2.weight);
                    System.out.println();
          }
}
```

OUTPUT:
Volume of mybox1 is 3000.0 Weight of mybox1 is 34.3 Volume of mybox2 is 24.0 Weight of mybox2 is 0.076

Illustration 3.20’‘‘‘‘‘‘‘‘‘‘‘‘‘‘

```
/* Example of super to overcome name hiding */

class A {
          int i;
}

// Create a subclass by extending class A.
class B extends A {
          int i; // this i hides the i in A

          B(int a, int b) {
                    super.i = a; // i in A
                    i = b; // i in B
          }
```

```
          void show() {
                    System.out.println("i in superclass: " + super.i);
                    System.out.println("i in subclass: " + i);
          }
}

class Demonstration_65 {
          public static void main(String args[]) {
                    B subOb = new B(1, 2);
                    subOb.show();
          }
}
```

OUTPUT:
i in superclass: 1 i in subclass: 2

Illustration 3.21’‘‘‘‘‘‘‘‘‘‘‘‘‘‘

```
/* Code sharing through super concept */

class Cat {
          void speak() {
                    System.out.println("Meaon ! ");
          }
}

class PetCat extends Cat {               // PetCat is one type of Cat
          void speak() {
                    System.out.println(" Meow ! ");
          }
}

class MagicCat extends Cat { // MagicCat is another kind of Cat
          static boolean noOne;
```

```
          void speak() {
                    if (noOne) {
                              super.speak(); // use the super class definition
                    } else {
                              System.out.println(" Hello World !");
                    }
          }
}

class Demonstration_66 {
          public static void main(String args[]) {
                    PetCat c1 = new PetCat();
                    MagicCat c2 = new MagicCat();
                    c2.noOne = true;
                    c2.speak();
                    c1.speak();
                    c2.noOne = false;
                    c2.speak();
          }
}
```

OUTPUT:
Meaon ! Meow ! Hello World !

Illustration 3.22’‘‘‘‘‘‘‘‘‘‘‘‘‘‘

```
/* Example of multilevel inheritance. */

// Start with Box.
class Box {
          private double width;
          private double height;
          private double depth;
```

```
	// constructor used when all dimensions specified
	Box(double w, double h, double d) {
		width = w;
		height = h;
		depth = d;
	}

	// compute and return volume
	double volume() {
		return width * height * depth;
	}
}

// Add weight.
class BoxWeight extends Box {
	double weight; // weight of box
	// constructor when all parameters are specified
	BoxWeight(double w, double h, double d, double m) {
		super(w, h, d); // call superclass constructor
		weight = m;
	}
}

// Add shipping costs
class Shipment extends BoxWeight {
	double cost;

	// constructor when all parameters are specified
	Shipment(double w, double h, double d, double m, double c) {
		super(w, h, d, m); // call superclass constructor
		cost = c;
	}
}


class Demonstration_67 {
	public static void main(String args[]) {
		Shipment shipment1 = new Shipment(10, 20, 15, 10, 3.41);
		Shipment shipment2 = new Shipment(2, 3, 4, 0.76, 1.28);
		double vol;
```

```
                    vol = shipment1.volume();
                    System.out.println("Volume of shipment1 is " + vol);
                    System.out.println("Weight of shipment1 is " + shipment1.weight);
                    System.out.println("Shipping cost: $" + shipment1.cost);
                    System.out.println();
                    vol = shipment2.volume();
                    System.out.println("Volume of shipment2 is " + vol);
                    System.out.println("Weight of shipment2 is " + shipment2.weight);
                    System.out.println("Shipping cost: $" + shipment2.cost);
          }
}
```

OUTPUT:
Volume of shipment1 is 3000.0 Weight of shipment1 is 10.0 Shipping cost: $3.41 Volume of shipment2 is 24.0 Weight of shipment2 is 0.76 Shipping cost: $1.28

Illustration 3.23’‘‘‘‘‘‘‘‘‘‘‘‘‘

```
/* A simple abstract class example */

abstract class Base {
‘‘‘‘abstract void fun();
}

class Derived extends Base {
‘‘‘‘void fun() { System.out.println("Derived fun() is called"); }
}

class Demonstration_68 {
‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘
‘‘‘‘‘‘‘‘// Uncommenting the following line will cause compiler error as the’‘
‘‘‘‘‘‘‘‘// line tries to create an instance of abstract class.
‘‘‘‘‘‘‘‘// Base b = new Base();
```

```
‘‘
‘‘‘‘‘‘‘‘// We can have references of Base type.
‘‘‘‘‘‘‘‘Base b = new Derived();
‘‘‘‘‘‘‘‘b.fun();’‘
‘‘‘‘}
}
```

OUTPUT:
Derived fun() is called

Illustration 3.24’‘‘‘‘‘‘‘‘‘‘‘‘‘‘

```
// An abstract class with constructor

abstract class Base {
‘‘‘‘Base() { System.out.println("Base constructor is called"); }
‘‘‘‘abstract void fun();
}

class Derived extends Base {
‘‘‘‘Derived() { System.out.println("Derived constructor is called"); }
‘‘‘‘void fun() { System.out.println("Derived fun() is called"); }
}



class Demonstration_69{
‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘ Derived d = new Derived();
‘‘‘‘‘‘ d.fun();
‘‘‘‘}
}
```

OUTPUT:
Base constructor is called

Derived constructor is called
Derived fun() is called

Illustration 3.25’‘‘‘‘‘‘‘‘‘‘‘‘‘

```
// An abstract class without any abstract method

abstract class Base {‘‘‘‘
‘‘‘‘void fun() { System.out.println("Base fun() is called"); }
}’‘

class Derived extends Base {
‘‘‘‘Derived() { System.out.println("Derived constructor is called"); }
‘‘‘‘void fun() { System.out.println("Derived fun() is called"); }
}
‘‘
class Demonstration_610 {
‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘Derived d = new Derived();
‘‘‘‘‘‘‘‘d.fun();
‘‘‘‘}
}
```

OUTPUT:

Derived constructor is called
Derived fun() is called

Illustration 3.26’‘‘‘‘‘‘‘‘‘‘‘‘‘

```
// Final Class Inheritance  An Example

final class Bike{}’‘
‘‘
class Honda1 extends Bike{‘‘
‘‘ void run(){
```

```
‘‘‘‘ System.out.println("Running safely with 100kmph");
‘‘ }’‘
}



final class Demonstration_611 {
‘‘ public static void main(String args[]){
                    Honda1 honda = new Honda1();
                    honda.run();
‘‘}
}
```

Illustration 3.27’‘‘‘‘‘‘‘‘‘‘‘‘‘

```
// An abstract class without any abstract method

abstract class Base {‘‘‘‘
‘‘‘‘void fun() { System.out.println("Base fun() is called"); }
}’‘

class Derived extends Base {
‘‘‘‘Derived() { System.out.println("Derived constructor is called"); }
‘‘‘‘void fun() { System.out.println("Derived fun() is called"); }
}
‘‘
class Demonstration_610 {
‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘Derived d = new Derived();
‘‘‘‘‘‘‘‘d.fun();
‘‘‘‘}
}
```

OUTPUT:
Final fun() is called

Illustration 3.28’‘‘‘‘‘‘‘‘‘‘‘‘‘

```
// An abstract class with a final method
abstract class Base {
‘‘‘‘final void fun() { System.out.println("Final fun() is called"); }
}
‘‘
class Derived extends Base {
‘‘‘‘Derived() { System.out.println("Derived constructor is called"); }
‘‘‘‘void fun() { System.out.println("Derived fun() is called"); }
}
‘‘
class Demonstration_612b {
‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘ Base b = new Derived();
‘‘‘‘‘‘ b.fun();
‘‘‘‘}
}
```

Polymorphism in Java

Another fundamental object oriented mechanism in Java is the polymorphism. Java allows polymorphism on methods. Polymorphism, meaning one object many shapes, is a simple concept that allows a method to have multiple implementations. This is also known as **method overloading**. Following is the **Illustration 3.29** to illustrate the idea of polymorphism .

Illustration 3.29 **// Polymorphism concept //**

```
class Point {
                    int x,y;
                    Point ( int x, int y ) {                 //’‘It is a constructor
                              this.x = x;
                              this.y = y;
                    }

 /*M1*/ float distance ( int x, int y)    {          // One definition of distance
                    int’‘dx = this.x -’‘x;
```

```
                    int’‘dy = this.y - y;
                    return (float) Math.sqrt ( dx* dx + dy * dy );
          ‘‘        }

 /*M2*/ float distance (Point p) {                  // Overloaded definition of distance .
                              return distance ( p.x,’‘p.y) ;’‘
                              }
‘‘‘‘}
          class Point3D extends Point {
                    int z ;
                    Point3D ( int x, int y, int z ) {                   // Constructor of Point3D
                              super ( x, y );
‘‘‘‘‘‘‘‘‘‘‘this.z = z;
                    }’‘
 /*M3*/ float distance (int x,int y, int z )’‘{     // Another definition of distance
                              int dx = this.x - x;
                              int dy = this.y - y;
                              int dz = this.z - z ;
                              return’‘(float) Math.sqrt ( dx * dx + dy*dy + dz*dz );
                    }

 /*M4*/ float distance (Point3D pt) {
                              return distance (pt.x, pt.y, pt.z );
                    }
          }

‘‘‘‘class PointDistance {
                    public static’‘void main ( String args [‘‘] ) {
                    Point p1 = new Point (10, 5) ;            // 2-D point
                    Point p3 = new Point3D (5, 10, 5);                  // 3-D point
                    Point p2 = new Point (4, 1) ;             // another 2-D point
                    Point p4 = new Point3D ( 2,3,4 );                   // another 3-D point
                    float d0 = p1.distance ( 0,0);             // M1 will be referred’‘
                    float’‘d1 = p1.distance ( p2);             // M2 will be referncd
                    System.out.println ( "Distance from P2 to Origin = " + d0);
                    System.out .println ( " Distance from P2 to P1 = " + d1) ;
                    d1 = p4.distance (p3);                              // M4 will be referred
                    System.out.println ( "Distance from P3 to P4= "+ d1);
                    }
‘‘‘‘}
```

OUTPUT:
Distance from P2 to Origin = 11.18034 Distance from P2 to P1 = 7.2111025 Distance from P3 to P4= 7.615773

In the above example, we have seen, how the same method can be implemented in different ways. The concept of this type of method overloading is same as in C++. However, C++ allows the operator overloading too, Java does not.

Dynamic Method Dispatch

Method can be called dynamically in Java. Whenever, a method is called on an object reference, the declared type of the object reference is checked at compile time to make sure that the method exists in the declared class. At run time, the object reference could be referring to an instance of some subclass of the declared reference type. This is illustrated in **Illustration 3.30.**

```
Illustration 3.30                          // Method resolution during execution //


class A {
          void callMe (‘‘) {
                    System.out. println ( "I am from A ") ;
                    }
          }
class B’‘extends A {
          void callMe (‘‘) {
          System.out.println ( "I am from B ");
          }
}’‘
public class Who {
          public static void main(String args [‘‘] ) {
                    A’‘a = new B (‘‘) ;
                    a.callMe (‘‘ );
          }
}
```

OUTPUT:
I am from B

In the above mentioned **Illustration 3.30**, we declared the variable to be of type A, then stored a reference to an object of class B in it. Now, when we call the method**callMe( )** on a, the Java run time notices that the reference is actually an instance of B, so it calls B s callMe( ) method instead of A s.

This form of dynamic run time resolution is one of the most powerful mechanisms that Object Oriented in Java brings to bear on code reuse and robustness. The ability for existing code libraries to call methods on instance of new classes without recompiling while maintaining a clean abstract interface is a profoundly powerful tool.

Access Specification in Java

The mechanism by which one can control the use of objects, its member elements and methods is known as the access specification. This is done in Java by using three access modifier key words public, private , and protected.

Public : Member elements and methods can be marked as public and then they can be accessed from any other method in Java Programs. To indicate a method or element is public, precede it with the public key word .
The public modifier can be applied to classes as well as methods and variables. It then allows to make a class accessible to other classes in other Packages.
The public access specification is automatic, in the sense that, if no access specifier is mentioned then by default it is having public accessibility.

Private : Member elements and methods marked private can be used only marked private can be used only from inside their class. A private element / method is not visible in any other class, including subclasses . Also, a subclass cannot override a non-private method and make the new method private.

Protected : Member elements and methods marked protected can be used only from inside their class or in subclasses of that class. A subclass can still override a protected method or variable.

Note : It can be noted that the private and protected modifier can not be applied to classes.

Final: Apart of these, there is another access modifier known in Java which is known as final. All methods and member elements may be overridden by default. In Java, one can declare that methods or member elements cannot be allowed to override by subclasses. For those method and member elements final key word can be specified. Like member elements and methods, classes also can be declared final and a final class cannot be sub classed.

Abstract : Just as one can specify that a method can never be sub classed by marking it as final, one can indicate that a method must always be sub classed by using the abstract keyword in the method definition. When applied to a class, the abstract modifier indicates that the class has not been fully implemented and that it should not be instantiated. If applied to a member function declaration, the abstract modifier means that the function will be implemented in a subclass. Since the function has no implementation, the class cannot be instantiated, and must be declared as abstract .

Illustration 3.31

```
/* Example-1 of default access modifier */

//Save the following classes in the same file named as Demonstration_71.java
‘‘
class A {‘‘
‘‘void msg(){System.out.println("Hi! I am in Class A");
 }’‘
}

class Demonstration_71 {‘‘
          public static void main(String args[]){‘‘
```

```
                    A obj = new A();’‘‘‘‘‘
                    obj.msg();’‘‘‘‘‘‘‘‘‘
‘‘ }’‘
}
```

OUTPUT:
Hi! I am in Class A

Illustration 3.32

```
/* Example-2 of default access modifier */

//Save the following class as A.java
‘‘
class A {‘‘
‘‘void msg(){System.out.println("Hi! I am in Class A");
 }’‘
}



//Save the following classes in the same file named as Demonstration_72.java

class Demonstration_72 {‘‘
          public static void main(String args[]){‘‘
                    A obj = new A();’‘‘‘‘‘
                    obj.msg();’‘‘‘‘‘‘‘‘‘
‘‘ }’‘
}
```

OUTPUT:
Hi! I am in Class A

Illustration 3.33

```
//Save this program as A.java in a sub-directory ﷺpack1ﷺ

/* Example-3 of default access modifier */

//Save the following class as A.java

class A {‘‘
‘‘‘‘‘‘void msg(){System.out.println("Hi! I am in Class A");
‘‘‘‘ }’‘
‘‘ }
‘‘

/* Save this program as Demonstration_73.java in another sub-directory ﷺpack2ﷺ’‘*/

//Save the following class as Demonstration_73.java

class Demonstration_73 {‘‘
          public static void main(String args[]){‘‘
                    A obj = new A();’‘‘‘‘‘
                    obj.msg();’‘‘‘‘‘‘‘‘‘
‘‘ }’‘
}’‘
```

OUTPUT:
Hi! I am in Class A

Illustration 3.34

```
/* Example-4 of default access modifier */

//Save this program as A.java in a sub-directory ﷺpack1ﷺ

package pack1;

public class A {‘‘
```

```
‘‘public void msg(){System.out.println("Hi! I am in Class A");
 }’‘
}



/* Save this program as Demonstration_74.java in another sub-directory ⍰pack2⍰’‘*/


package pack2;
import pack1.*;

class Demonstration_74 {‘‘
          public static void main(String args[]){‘‘
                     A obj = new A();’‘ //Compile Time Error’‘
                     obj.msg();’‘‘‘‘‘‘‘ //Compile Time Error’‘
‘‘ }’‘
}’‘

Note: Run the main class Demonstration_74 from pack2 ⍰
```

Illustration 3.35

```
/* Example-5 of public access modifier */

class A{
          public int data=40;
          public void msg(){
                    System.out.println("Class A: Hello Java!");
          }
}

class Demonstration_75 {
          public static void main(String args[]){
                    A obj = new A(); //OK : Class A is public
                    System.out.println(obj.data); //OK : data is public
                    obj.msg(); //OK: msg is public
          }
}
```

OUTPUT:
40 Class A: Hello Java!

Illustration 3.36

```
//Example-6 : Java program to illustrate’‘public modifier

package pack1;
public class A1 {
‘‘ int data = 100;
‘‘ public void display()
‘‘‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘System.out.println("NPTEL " + data);
‘‘‘‘‘‘}
}

package pack2;
import pack1.*;

class Demonstration_76’‘{
‘‘‘‘public static void main(String args[])
‘‘‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘A1 obj = new A1();
‘‘‘‘‘‘‘‘‘‘obj.display();
‘‘‘‘‘‘}
}
```

Illustration 3.37

```
/* Example-1 : Private access modifier */
```

```
public class A{
          private int data = 40;
          public void msg(){
          System.out.println("Class A: Hello Java!");
          }
}

public class Demonstration_77 {
          public static void main(String args[]){
          A obj = new A();’‘‘‘‘‘‘‘‘‘‘‘‘‘//OK : Class A is public
          System.out.println(obj.data); //Compile Time Error : data is private
          obj.msg();’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘//OK : msg is public
          }
}
```

Illustration 3.38

```
/* Example-2: Private access modifier */

class A{
          private int data = 40;
          public void msg(){
                    System.out.println("Class A: Hello Java!" + data);
          }
}

public class Demonstration_78 {
          public static void main(String args[]){
                    //private int data2 = 100;
                    //public int data2 = 100;
                    int data2 = 100;
                    A obj = new A(); //OK : Class A is public
                    System.out.println("Class B:"+ data2);’‘//OK: private data in the same class
                    obj.msg(); // Private data is accessable through public method
          }
}
```

OUTPUT:

Class B:100
Class A: Hello Java!40

Illustration 3.39

```
/* Example-3: Private access modifier */

class A{
          private A(){
          //private constructor
          }

          void msg(){
                    System.out.println("Class A: Hello Java!");
          }
}

public class Demonstration_79 {
          public static void main(String args[]){
                    A obj = new A(); //Compile Time Error
          }
}
```

Illustration 3.40

```
/* Example 1: protected modifier’‘*/

//Save by A.java in a sub-directory pack1

package pack1;

public class A {
‘‘‘‘‘‘protected int rollNo = 555;
```

```
‘‘‘‘‘‘protected void msg(){
‘‘‘‘‘‘System.out.println("Class A: Hello Java!" + rollNo);
‘‘‘‘}
}

//Save by B.java in another sub-directory pack2

package pack2;
import pack1.*;

class Demonstration_710’‘{
‘‘public static void main(String args[]){
          A obj = new A();
          obj.msg();
          }
}
```

OUTPUT:
Class A: Hello Java!555 Class A: Hello Java!555

Illustration 3.41

```
/* Example 2: protected access specifier*/

//Save by A.java in a subdirectory pack1

package pack1;

public class A{
 protected int rollNo = 555;
 protected void msg(){
‘‘‘‘ System.out.println("Class A: Hello Java!" +rollNo);
 }
}
```

```
//Save by B.java in another sub-directory pack2

package pack2;
import pack1.*;

class Demonstration_711 extends A{
public static void main(String args[]){
‘‘‘‘A’‘obj = new A();
‘‘‘‘obj.msg();
‘‘‘‘System.out.println("Class B: Hello Java!" +obj.rollNo);
}
}
```

OUTPUT:
Class A: Hello Java!555 Class B: Hello Java!555

```
Illustration 3.42

//Example-3: Java program to illustrate protected modifier

package p1;
‘‘
//Class A

public class A
{
‘‘ protected void display()’‘‘‘ {
‘‘‘‘‘‘‘‘System.out.println("NPTEL");
‘‘‘‘}
}
//Java program to illustrate protected modifier

package p2;
import p1.*; //importing all classes in package p1
‘‘
//Class B is sub-class of A
```

```
class Demonstration_712 extends A’‘{
‘‘ public static void main(String args[])
‘‘ {‘‘
‘‘‘‘‘‘ A’‘obj = new A ();’‘
‘‘‘‘‘‘ obj.display();’‘
‘‘ }’‘
}
```

OUTPUT:
NPTEL

Illustration 3.43

```
//Example-4: Java program to illustrate protected modifier


package p1;
‘‘
//Class A


protected class A’‘‘‘‘‘‘ // A class cannot be declared as protected.
{
‘‘ void display()’‘‘‘ {
‘‘‘‘‘‘‘‘System.out.println("NPTEL");
‘‘‘‘}
}
//Java program to illustrate protected modifier


package p2;
import p1.*; //importing all classes in package p1
‘‘
//Class B is sub-class of A
class Demonstration_712 extends A {
‘‘ public static void main(String args[])
‘‘ {‘‘
‘‘‘‘‘‘ A obj = new A();’‘‘‘
```

```
‘‘‘‘‘‘ obj.display();’’
‘‘ }’’
}
```

Illustration 3.44

```
/* Simple example of access modifier. In a class hierarchy, private members remain private to their class.
This program contains an error and will not compile. */


// Create a superclass.
class A {
          int i; // public by default
          private int j; // private to A

          void setij(int x, int y) {
                    i = x;
                    j = y;
          }
}

// A's j is not accessible here.
class B extends A {
          int total;

          void sum() {
                    total = i + j; // ERROR, j is not accessible here
          }
}


class Demonstration_714{
          public static void main(String args[]) {
                    B subOb = new B();
                    subOb.setij(10, 12);
                    subOb.sum();
                    System.out.println("Total is " + subOb.total);
          }
```

```
}
```

Illustration 3.45

```
/* Another example of access modifier with public, private and protected data */

class BaseClass {
           public int x = 10;
           private int y = 10;
           protected int z = 10;
           int a = 10; //Implicit Default Access Modifier

           public int getX() {
           return x;
}

public void setX(int x) {
           this.x = x;
}

private int getY() {
           return y;
}

private void setY(int y) {
           this.y = y;
}

protected int getZ() {
           return z;
}

protected void setZ(int z) {
           this.z = z;
}

int getA() {
           return a;
```

```
}

void setA(int a) {
          this.a = a;
}
}



public class Demonstration_715 extends BaseClass {
          public static void main(String args[]) {
                    BaseClass rr = new BaseClass();
                    rr.z = 0;
                    Demonstration_715 subClassObj = new Demonstration_715();
                    //Access Modifiers - Public
                    System.out.println("Value of x is : " + subClassObj.x);
                    subClassObj.setX(20);
                    System.out.println("Value of x is : " + subClassObj.x);
                    //Access Modifiers - Public
                    // If we remove the comments it would result in a compilaton
                    //’‘      error as the fields and methods being accessed are private

                    /* System.out.println("Value of y is : "+subClassObj.y);
                    subClassObj.setY(20);
                    System.out.println("Value of y is : "+subClassObj.y);*/

                    //Access Modifiers - Protected
                    System.out.println("Value of z is : " + subClassObj.z);
                    subClassObj.setZ(30);
                    System.out.println("Value of z is : " + subClassObj.z);
                    //Access Modifiers - Default
                    System.out.println("Value of x is : " + subClassObj.a);
                    subClassObj.setA(20);
                    System.out.println("Value of x is : " + subClassObj.a);
          }
}
```

OUTPUT:
Value of x is : 10

```
Value of x is : 20
Value of z is : 10
Value of z is : 30
Value of x is : 10
Value of x is : 20
```

Illustration 3.46

```
/* Example of method-overiding */


class A{
‘‘‘‘‘‘ public void msg1() {
‘‘‘‘‘‘‘‘‘‘ System.out.println("Class A: Public!");
‘‘‘‘‘‘ }
‘‘‘‘‘‘
‘‘‘‘‘‘ private void msg2() {
‘‘‘‘‘‘‘‘‘‘ System.out.println("Class A: Private!");
‘‘‘‘‘‘ }
‘‘‘‘‘‘ protected void msg3(){
‘‘‘‘‘‘‘‘‘‘System.out.println("Class A: Protected!");
‘‘‘‘‘‘ }’‘
‘‘‘‘}’‘
‘‘
public class Demonstration_716 extends A{‘‘
‘‘‘‘‘‘ void msg(){
‘‘‘‘‘‘‘‘‘‘ System.out.println("Class Main: Welcome!");
‘‘‘‘‘‘ }
‘‘‘‘‘‘ public void msg1() { // If modifire is set to default it cannot overide.
‘‘‘‘‘‘‘‘‘‘ System.out.println("Overriding public method!");
                    ‘‘
‘‘‘‘‘‘ }
‘‘‘‘‘‘
‘‘‘‘‘‘ void msg2() {
‘‘‘‘‘‘‘‘‘‘ System.out.println("Overriding private method!");
‘‘‘‘‘‘ }
          ‘‘ // If modifire is set to private it can be overidden.
‘‘‘‘‘‘ public void msg3(){
‘‘‘‘‘‘‘‘‘‘System.out.println("Overriding protected method!");
‘‘‘‘‘‘ }
```

```
‘‘‘‘‘‘ public static void main(String args[]){‘‘
‘‘‘‘‘‘‘‘‘ Demonstration_716 obj = new Demonstration_716();’‘
‘‘‘‘‘‘‘‘‘ obj.msg();
                    ‘‘ //obj.msg1();
                    ‘‘ //obj.msg2();
                    ‘‘ //obj.msg3();
‘‘‘‘‘‘ }’‘
‘‘‘‘}’‘
```

OUTPUT:
Class Main: Welcome!

Static method and variable

In general, each instance of a class has its own copy of any member variables. However, it is possible to designate a member variable as belonging to the class itself independent of any object of that class. Such member variables are called static members and are declared with the static key word. These are often work as global variable about the instances of a class. Likewise, methods can also be declared as static. Such static methods then can be used outside of the context of any instances.

In order to initialize static variable, one may declare a static block which gets executed exactly once, when the class is first loaded. The following is a complete listing in order to understand static modifier.

```
Illustration 3.47                          // Static extent of a variable or method //


public class Demo{
‘‘ public static void main(String args[]){
‘‘‘‘ int b=0;
‘‘‘‘ System.out.println("Value of b = "+b);
‘‘‘‘ Student s1 = new Student();
‘‘‘‘ s1.showData();
‘‘‘‘ Student s2 = new Student();
‘‘‘‘ s2.showData();’‘
‘‘}
}


class Student {
```

```
static int b;


Student(){
‘‘ //Constructor incrementing static variable b
‘‘ b++;
‘‘}


public static void showData(){
‘‘‘‘
‘‘‘‘System.out.println("Value of b = "+b);
‘‘ }
}
```

OUTPUT:
Value of b = 0 Value of b = 1 Value of b = 2

From the foregoing discussion, it is evident that static is useful for creating methods which may be called directly by referring to name of the class in which they are declared. This is how Java implements global functions and global variables. This is why our main() should always be declared as static so that the classes contain the scope of the names to avoid collisions.

Control, Scope and Recursion

Illustration 3.48

```
/* while loop example */
public class Demonstration_51{
          public static void main(String[] args){
                    int count = 1;
                    System.out.println("Printing first 10 odd numbers");
                    while (count < 11) {
                              System.out.print(" " +((2*count)-1));
                              count++;
                    }
          }
}
```

OUTPUT:
Printing first 10 odd numbers 1 3 5 7 9 11 13 15 17 19

Illustration 3.49

```
/* Do-While loop example */


public class Demonstration_52{
          public static void main(String[] args){
                    int count = 1;
                    System.out.println(" Printing first 10 even numbers");
                    do {
                              System.out.print(" " + 2*count);
                              count++;
                    } while (count < 11);
          }
}
```

OUTPUT:
Printing first 10 even numbers 2 4 6 8 10 12 14 16 18 20

Illustration 3.50

```
/* For loop example */
class Demonstration_53{
          public static void main(String[] args){
                    for(int i=1; i<11; i++){
                              System.out.println("Count is: " + i);
                    }
          }
}
```

OUTPUT:

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
Count is: 6
Count is: 7
Count is: 8
Count is: 9
Count is: 10
```

Illustration 3.51

```
/* Loop’‘example with continue */
public class Demonstration_54{
          public static void main(String args[]) {
                    for(int i=0;’‘i<10; i++) {
                              System.out.print(i + " ");
                              if (i%2 == 0) continue;
                              System.out.println(" ");
                    }
          }
}
```

OUTPUT:

```
0 1
2 3
4 5
6 7
8 9
```

Illustration 3.52

```
/* Loop example with break*/
class Demonstration_55{
          public static void main(String args[]) {
```

```
                    for(int i=1;’‘; i++) {
                              if(i%10 ==0 ) break; // terminate loop if i is 10
                              System.out.println("i: " + i);
                    }
                    System.out.println("Loop complete.");
          }
}
```

OUTPUT:
i: 1 i: 2 i: 3 i: 4 i: 5 i: 6 i: 7 i: 8 i: 9 Loop complete.

Illustration 3.53

```
/* Test for primes */
class Demonstration_56{
          public static void main(String args[]) {
                    int num;
                    boolean isPrime = true;
                    num = Integer.parseInt(args[0]);
                    for(int i=2; i <= num/2; i++) {
                              if((num % i) == 0) {
                                        isPrime = false;
                                        break;
                              }
                    }
                    if(isPrime) System.out.println("Prime");
                    else System.out.println("Not Prime");
          }
}
```

Illustration 3.54

```
/* Menu selection using do-while and switch-case */
class Demonstration_57{
          public static void main(String args[]) throws java.io.IOException {
                    char choice;
                    do {
                              System.out.println("Help on:");
                              System.out.println(" 1. if");
                              System.out.println(" 2. switch");
                              System.out.println(" 3. while");
                              System.out.println(" 4. do-while");
                              System.out.println(" 5. for\n");
                              System.out.println("Choose one:");
                              choice = (char) System.in.read();
                    } while( choice < '1' || choice > '5');

                    System.out.println("\n");
                    switch(choice) {
                    case '1':
                              System.out.println("The if:\n");
                              System.out.println("if(condition) statement;");
                              System.out.println("else statement;");
                              break;
                    case '2':
                              System.out.println("The switch:\n");
                              System.out.println("switch(expression) {");
                              System.out.println(" case constant:");
                              System.out.println(" statement sequence");
                              System.out.println(" break;");
                              System.out.println(" // ...");
                              System.out.println("}");
                              break;
                    case '3':
                              System.out.println("The while:\n");
                              System.out.println("while(condition) statement;");
                              break;
                    case '4':
                              System.out.println("The do-while:\n");
```

```
                                System.out.println("do {");
                                System.out.println(" statement;");
                                System.out.println("} while (condition);");
                                break;
                        case '5':
                                System.out.println("The for:\n");
                                System.out.print("for(init; condition; iteration)");
                                System.out.println(" statement;");
                                break;
                        }
                }
}
```

Illustration 3.55

```
/* A variable declared inside pair of brackets {  and  }  in a method has scope withing the brackets
only.*/
public class Demonstration_58
{
    public static void main(String args[])
    {
        {
            // The variable x has scope within
            // brackets
            int x = 10;
            System.out.println(x);
        }

        // Uncommenting below line would produce
        // error since variable x is out of scope.

        // System.out.println(x);''
    }
}
```

OUTPUT:
10

Illustration 3.56

```
class Demonstration_59
{
‘‘‘‘public static void main(String args[])
‘‘‘‘{
‘‘‘‘‘‘‘‘for (int x = 0; x < 4; x++)
‘‘‘‘‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println(x);
‘‘‘‘‘‘‘‘}
‘‘
‘‘‘‘‘‘‘‘// uncommenting Will produce error
‘‘‘‘‘‘‘‘//System.out.println(x);
‘‘‘‘}
}
```

OUTPUT:

```
0
1
2
3
```

Illustration 3.57

```
// Above program after correcting the error
class Demonstration_510
{
‘‘‘‘public static void main(String args[])
‘‘‘‘{
‘‘‘‘‘‘‘‘int x;
‘‘‘‘‘‘‘‘for (x = 0; x < 4; x++){
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println(x);
‘‘‘‘‘‘‘‘}
```

```
‘‘
‘‘‘‘‘‘ System.out.println(x);
‘‘‘‘}
}
```

OUTPUT:
0 1 2 3 4

Illustration 3.58

```
// Another example of scope of variable in a block

class Demonstration_511 {
          public static void main(String args[]) {
                    int x;
                    x = 10;
                    if(x == 10) {‘‘‘‘‘‘‘‘
                              int y = 20;’‘‘‘
                              System.out.println("x and y: " + x + " " + y);
                              x = y * 2;’‘‘‘‘‘
                    }
                    //y = 100; // Error: Out of scope’‘‘‘‘‘‘‘‘‘
                    System.out.println("x is " + x);
          }
}
```

OUTPUT:
x and y: 10 20 x is 40

Illustration 3.59

```
// Example of static‘‘variable’‘

public class Circle{
‘‘‘‘static int circlecount = 0; // class variable
‘‘‘‘public double x,y,r; // instance variables
‘‘‘‘public Circle(double x, double y, double r){
‘‘‘‘‘‘ this.x = x; this.y = y; this.r = r;
‘‘‘‘}
‘‘‘‘public Circle(double r){
‘‘‘‘‘‘ this(0.0,0.0,r);
‘‘‘‘‘‘ circlecount++;
‘‘‘‘}
‘‘‘‘public Circle(Circle c){
‘‘‘‘‘‘ this(c.x,c.y,c.r);
‘‘‘‘‘‘ circlecount++;
‘‘‘‘}
‘‘‘‘public Circle(){
‘‘‘‘‘‘ this(0.0,0.0,0.1);
‘‘‘‘‘‘‘‘circlecount++;
‘‘‘‘}

          public double circumference(){
‘‘‘‘‘‘ return (2*3.14159*r);
‘‘‘‘}
‘‘‘‘public double area(){
‘‘‘‘‘‘‘‘return(3.14159*r*r);
‘‘‘‘}
‘‘‘‘public static void main(String args[ ]){
‘‘‘‘‘‘ Circle c1 = new Circle();
          ‘‘ System.out.println("c1#"+ c1.circlecount);
          ‘‘
‘‘‘‘‘‘ Circle c2 = new Circle(5.0);
          ‘‘ System.out.println("c2#"+ c2.circlecount);
          ‘‘
‘‘‘‘‘‘ Circle c3 = new Circle(c1);
          ‘‘ System.out.println("c3#"+ c3.circlecount);
          ‘‘
‘‘‘‘‘‘ System.out.println("c1#"+ c1.circlecount + " c2# " + c2.circlecount + "     c3#"+ c3.circlecount);
‘‘‘‘}
```

```
}
```

OUTPUT:
c1#1 c2#2 c3#3 c1#3 c2# 3 c3#3

Illustration 3.60

```
// Example of static’‘method’‘
// A class method and instance method

public class Circle{
‘‘‘‘ public double x,y,r;
            static int circlecount=0;
‘‘‘‘ public Circle(double x, double y, double r){
‘‘‘‘‘‘ this.x = x; this.y = y; this.r = r;
‘‘‘‘‘‘ //circlecount++;
‘‘‘‘}
‘‘‘‘public Circle(double r){
‘‘‘‘‘‘ this(0.0,0.0,r);
‘‘‘‘‘‘ circlecount++;
‘‘‘‘}
‘‘‘‘public Circle(Circle c){
‘‘‘‘‘‘ this(c.x,c.y,c.r);
‘‘‘‘‘‘ circlecount++;
‘‘‘‘}
‘‘‘‘public Circle(){
‘‘‘‘‘‘ this(0.0,0.0,0.1);
‘‘‘‘‘‘‘‘circlecount++;
‘‘‘‘}
‘‘‘‘ // An instance method. Return the bigger of two circles.’‘‘‘‘‘‘‘‘
```

```
‘‘‘‘ }
‘‘‘‘ // A class method: Return the bigger of two classes.
‘‘‘‘ public static Circle bigger (Circle a, Circle b) {
‘‘‘‘‘‘‘‘ if (a.r > b.r) return a;
‘‘‘‘‘‘‘‘ else return b;
‘‘‘‘ }
          public static void main(String args[]){
‘‘‘‘‘‘‘‘ Circle a = new Circle (2.0);
‘‘‘‘‘‘‘‘ Circle b = new Circle (3.0);
‘‘‘‘‘‘‘‘ Circle c = a.bigger (b);’‘‘‘‘‘‘‘‘
‘‘‘‘‘‘‘‘ Circle d = Circle.bigger (a,b);
          }
}
```

Illustration 3.61

```
// Example of factorial calculation
‘‘
public class RecursiveFactorial{
‘‘int n;
‘‘int factorial(int n) {
‘‘‘‘‘‘if (n == 0)
‘‘‘‘‘‘‘‘‘‘return(1);
‘‘‘‘‘‘else
‘‘‘‘‘‘‘‘‘‘return(n*factorial(n-1));
‘‘ }

‘‘ public static void main(String[] args) {‘‘
‘‘‘‘‘‘ RecursiveFactorial x = new RecursiveFactorial();
‘‘‘‘‘‘ x.n = Integer.parseInt(args[0]);
‘‘‘‘‘‘ System.out.println("Factorial of "+ x.n + ": " + x.factorial(x.n));
‘‘ }’‘
}
```

Illustration 3.62

```
// Example of Fibonacci sequence

class RecursiveFibonacci {
‘‘‘‘int n;’’
‘‘‘‘int fibonacci(int n){
‘‘‘‘‘‘ if (n == 0)’‘‘‘
‘‘‘‘‘‘‘‘‘‘ return 0;
‘‘‘‘‘‘ else if (n == 1)
          ‘‘ return 1;
‘‘‘‘‘‘‘‘‘‘‘‘else’‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘ return(fibonacci(n-1) + fibonacci(n-2));’‘‘‘
‘‘‘‘ }’‘‘‘
‘‘‘‘ public static void main(String args[]){‘‘‘‘
‘‘‘‘‘‘‘‘ RecursiveFibonacci x = new RecursiveFibonacci();
‘‘‘‘‘‘‘‘ x.n = Integer.parseInt(args[0]);
‘‘‘‘‘‘‘‘ for(int i = 0; i <= x.n; i++){
          ‘‘ System.out.println(x.fibonacci(i));’‘‘‘‘‘
‘‘‘‘ }
‘‘ }’‘
}
```

Illustration 3.63

```
// Example of GCD Calculation
public class RecursiveGCD {
‘‘‘‘int m, n;
‘‘‘‘int gcd(int m, int n){
‘‘‘‘‘‘‘‘if(m>n) return gcd(n,m);
‘‘‘‘‘‘‘‘if(m==n)return m;
‘‘‘‘‘‘‘‘if(m==0)return n;
‘‘‘‘‘‘‘‘if(m==1)return 1;
‘‘‘‘‘‘‘‘return gcd(m,n%m);
‘‘‘‘}

‘‘‘‘public static void main(String[] args) {
‘‘‘‘‘‘‘‘RecursiveGCD g = new RecursiveGCD();
‘‘‘‘‘‘‘‘g.m = Integer.parseInt(args[0]);
‘‘‘‘‘‘‘‘g.n = Integer.parseInt(args[1]);
```

```
‘‘‘‘‘‘‘‘System.out.printf("GCD of %d and %d is %d.", g.m, g.n, g.gcd(g.m, g.n));
‘‘‘‘}
}
```

Illustration 3.64

```
/* Example of recursion : Practic 1 */
class Demonstration_517{

          static void myMethod( int counter){
                    if(counter == 0)
                              return;
                    else
                    {
                              System.out.println("Hello "+counter);
                              myMethod(--counter);
                              System.out.println(counter);
                              return;
                    }
          }

          public static void main(String args[]) {
                    myMethod(10); // pass positive integer
          }


}
```

OUTPUT:
Hello 10 Hello 9 Hello 8 Hello 7 Hello 6 Hello 5 Hello 4 Hello 3

Hello 2
Hello 1
0
1
2
3
4
5
6
7
8
9

Illustration 3.65

```
/* Example of recursion : Practic 2 */


public class Demonstration_518{‘‘
‘‘static int count = 0;’’
‘‘static void p(){‘‘
‘‘‘‘‘‘count++;’’
‘‘‘‘‘‘if(count <= 5){‘‘
‘‘‘‘‘‘‘‘‘System.out.println("Hello " + count);’’
‘‘‘‘‘‘‘‘‘p();’’
‘‘‘‘‘‘}’’
‘‘}
‘‘
‘‘ public static void main(String[] args) {‘‘
‘‘‘‘‘‘‘p();’’
‘‘ }’’
}
```

OUTPUT:

Hello 1
Hello 2
Hello 3
Hello 4
Hello 5

Practice questions

Practice 3.1

```
/* A program that uses simple Point class. Call this file PointDemo.java */
          class Point {
          int x;
          int y;
          }

          // This class declares an object of type Point.
          class PointDemo {
          public static void main(String args[]) {
          Point mypoint = new Point();

          // assign values to mypoint's instance variables
          mypoint.x = 10;
          mypoint.y = 20;

          // access the values from mypoint's instance variable
          System.out.println("x " + mypoint.x);
          System.out.println("x " + mypoint.y);

          }
          }
```

Is the compilation successful? What is the output?

Practice 3.2

```
          /* Encapsulation: Defining a class with method */
          class Point
          {
          int x,y;
          void setPoint ( ) {
          x = 10;
          y = 10;
          }
          }

          // definition of another class. This is a main class

          class PointDemo
          {
```

```
float distance;
public static void main (String args[ ] {
Point p = new Point( );
p.setPoint();
System.out.println ( " x = "+ p.x );
System.out.println ( " y = "+ p.y );
}
}
```

Is the compilation successful? What is about the execution?

Practice 3.3

```
/* Encapsulation: Defining a class with method with parameter */
class Point
{
int x,y;
void setPoint ( int a, int b, ) {
x = a;
y = b;
}
}

// definition of another class. This is a main class

class PointParameterDemo
{
float distance;
public static void main (String args[ ] {
Point p = new Point( );
p.setPoint(15, 20);
System.out.println ( " x = "+ p.x );
System.out.println ( " y = "+ p.y );
}
}
```

(This problem has a minor mistake. Identify the mistake and then write the correct code.)

Practice 3.4

```
/* Encapsulation: declare more than one object of a class */
```

```
class Point
{
int x,y;
void getPoint ( int a, int b ) {
x = a;
y = b;
}
}

// definition of another class. This is a main class
class PointsDemo
{
float distance;
public static void main (String args[ ] {
Point p1 = new Point( );
Point p2 = p1;
Point p3 = new Point ( );
Point p4 = new Point ( );
p1.getPoint (5, 10 );
p2.getPoint (15, 20);
p3.getPoint (20, 30);
p4.getPoint (30, 40);

System.out.println (" X1 = " + p1.x +’‘"Y1’‘= " + p1.y );
System.out.println ("X2=" + p2.x + "Y2 = " +p2.y );
int dx = p3.x - p4. x;’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘// X2 - X1
int dy = p3.y - p4. y;          ‘‘‘‘‘‘‘‘‘‘// y2 - y1
distance = Math.sqrt (dx * dx + dy * dy );// (X2-X1)2 + (Y2-Y1)2
System.out.println ( " Distance = "+ distance );
}
}
```

What is the output?

Practice 3.5

```
/* Example of constructors. (one default and one parameterized constructor) */
public class Cube {
int length;
int breadth;
int height;
public int getVolume() {
```

```
return (length * breadth * height);
}
Cube() {
length = 10;
breadth = 10;
height = 10;
}
}
Class DemoCube{
public static void main(String[] args) {
Cube cubeObj;
cubeObj = new Cube();
System.out.println("Volume of Cube is : " + cubeObj.getVolume());
}
}
```

What is the output?

Practice 3.6

```
/* Example of a cube class containing 2 constructors. (one default and one parameterized constructor) */
public class Cube1 {
int length;
int breadth;
int height;

public int getVolume() {
return (length * breadth * height);
}

Cube1() {
length = 10;
breadth = 10;
height = 10;
}

Cube1(int l, int b, int h) {
length = l;
breadth = b;
height = h;
}
}
```

```
class Cube1Demo{
public static void main(String[] args) {
Cube1 cubeObj1, cubeObj2;
cubeObj1 = new Cube1();
cubeObj2 = new Cube1(10, 20, 30);

System.out.println("Volume of Cube1 is : " + cubeObj1.getVolume());
System.out.println("Volume of Cube1 is : " + cubeObj2.getVolume());
}
}
```

What is the output?

Practice 3.7

```
/* Automatic initialization - concept of constructor */
class Point () {
int x, y;

Point () {
x = 10 ;
y = 5;
}

printPoint() {
System.out.println("X = "+ this.x + " Y= " + this.y);
}
}

class PointCreate1 {
public static void main ( String args [‘‘] ) {
Point p = new Point ();
p.printPoint();
}
}
```

What is the output?

Practice 3.8

```
/* Automatic initialization - concept of constructor */
class Point ()’‘{
int x, y;
```

```
          Point ( int x, int y ) {
          this.x = x ;
          this.y = y;
          }

          printPoint() {
          System.out.println("X = "+ this.x + " Y= " + this.y);
          }
          }

          class PointCreate2 {
          public static void main ( String args [‘‘] ) {
          Point p = new Point (10, 20 );
          p.printPoint();
          }
          }
```

What is the output?

Practice 3.9

```
/* Automatic initialization  more than one constructor */
          class Point ()’‘{
          int x, y;

          Point () {
          x = 30;
          y = 40;
          }

          Point ( int x, int y ) {
          this.x = x ;
          this.y = y;
          }

          printPoint() {
          System.out.println("X = "+ this.x + " Y= " + this.y);
          }
          }

          class PointCreate3 {
          public static void main ( String args [‘‘] ) {
          Point p1 = new Point ();
```

```
          Point p2 = new Point (10, 20 );
          p1.printPoint();
          p2.printPoint();
          }
          }
```

What is the output?

Practice 3.10

```
/* Passing objects as parameter */
          class Test {
          int a,b;
          Test(int i, int j) {
          a = i;
          b = j;
          }

          boolean equals(Test o) {
          if((o.a == a) && (o.b == b)) return true;
          else return false;
          }
          }

          class PassObjectParameter {
          public static void main(String[] args) {
          Test ob1 = new Test(100,22);
          Test ob2 = new Test(100,22);
          Test ob3 = new Test(-1,-1);

          System.out.println("ob1 == ob2: " + ob1.equals(ob2));
          System.out.println("ob1 <> ob3: " + ob1.equals(ob3));
          }
          }
```

What is the output?

Practice 3.11

```
/* Returning an object */
          class Test {
          int a;
```

```
          Test(int i) {
          a = i;
          }
          Test incrByTen() {
          Test temp = new Test(a+10);
          return temp;
          }
          }

          class RetOb {
          public static void main(String args[]) {
          Test ob1 = new Test(2);
          Test ob2;
          ob2 = ob1.incrByTen();
          System.out.println("ob1.a: " + ob1.a);
          System.out.println("ob2.a: " + ob2.a);
          ob2 = ob2.incrByTen();
          System.out.println("ob2.a after second increase: " + ob2.a);
          }
          }
```

What is the output?

Practice 3.12

```
/* Example of access modifier */
          class BaseClass {
          public int x = 10;
          private int y = 10;
          protected int z = 10;
          int a = 10; //Implicit Default Access Modifier

          public int getX() {
          return x;
          }

          public void setX(int x) {
          this.x = x;
          }

          private int getY() {
          return y;
```

```
}

private void setY(int y) {
this.y = y;
}

protected int getZ() {
return z;
}

protected void setZ(int z) {
this.z = z;
}

int getA() {
return a;
}

void setA(int a) {
this.a = a;
}
}

public class SubclassInSamePackage extends BaseClass {
public static void main(String args[]) {
BaseClass rr = new BaseClass();
rr.z = 0;
SubclassInSamePackage subClassObj = new SubclassInSamePackage();

//Access Modifiers - Public
System.out.println("Value of x is : " + subClassObj.x);
subClassObj.setX(20);
System.out.println("Value of x is : " + subClassObj.x);

//Access Modifiers - Public
// If we remove the comments it would result in a compilaton
// error as the fields and methods being accessed are private

/* System.out.println("Value of y is : "+subClassObj.y);
subClassObj.setY(20);
System.out.println("Value of y is : "+subClassObj.y);*/
```

```
          //Access Modifiers - Protected
          System.out.println("Value of z is : " + subClassObj.z);
          subClassObj.setZ(30);
          System.out.println("Value of z is : " + subClassObj.z);

          //Access Modifiers - Default
          System.out.println("Value of x is : " + subClassObj.a);
          subClassObj.setA(20);
          System.out.println("Value of x is : " + subClassObj.a);
          }
          }
```

what is the output?

Practice 3.13

```
/* The following program, ArrayDemo, creates an array of integers, puts some values in it, and prints each
value to standard output. */
          class ArrayDemo {
          public static void main(String[] args) {
          int[] anArray;’‘‘‘‘‘‘‘‘‘// declares an array of integers
          anArray = new int[10];’‘// allocates memory for 10 integers

          anArray[0] = 100; // initialize first element
          anArray[1] = 200; // initialize second element
          anArray[2] = 300; // etc.
          anArray[3] = 400;
          anArray[4] = 500;
          anArray[5] = 600;
          anArray[6] = 700;
          anArray[7] = 800;
          anArray[8] = 900;
          anArray[9] = 1000;

          System.out.println("Element at index 0: " + anArray[0]);
          System.out.println("Element at index 1: " + anArray[1]);
          System.out.println("Element at index 2: " + anArray[2]);
          System.out.println("Element at index 3: " + anArray[3]);
          System.out.println("Element at index 4: " + anArray[4]);
          System.out.println("Element at index 5: " + anArray[5]);
          System.out.println("Element at index 6: " + anArray[6]);
          System.out.println("Element at index 7: " + anArray[7]);
          System.out.println("Element at index 8: " + anArray[8]);
```

```
          System.out.println("Element at index 9: " + anArray[9]);
          }
          }
```

what is the output?

Practice 3.14

```
/* Another array example */
          class ArrayDemo {
          public static void main(String[] args) {
          int[] arrayOfInts = { 32, 87, 3, 589, 12, 1076, 2000, 8, 622, 127 };

          int i;
          for (i = 0; i < arrayOfInts.length; i++) {
          System.out.println("arrayOfInts[" + i + "] = " + arrayOfInts[i]);
          }
          }
          }
```

what is the output?

Practice 3.15

```
/* Average an array of values */
          class Average {
          public static void main(String args[]) {
          double nums[] = {10.1, 11.2, 12.3, 13.4, 14.5};
          double result = 0;
          int i;
          for(i=0; i<5; i++)
          result = result + nums[i];
          System.out.println("Average is " + result / 5);
          }
          }
```

what is the output?

Practice 3.16

```
/* Demonstrate a two-dimensional array */
          class TwoDArray {
          public static void main(String args[]) {
          int twoD[][]= new int[4][5];
          int i, j, k = 0;
          for(i=0; i<4; i++)
          for(j=0; j<5; j++) {
```

```
          twoD[i][j] = k;
          k++;
          }
          for(i=0; i<4; i++) {
          for(j=0; j<5; j++)
          System.out.print(twoD[i][j] + " ");
          System.out.println();
          }
          }
          }
```

what is the output?

Practice 3.17

```
/* Example of 2D Array:’‘Manually allocate differing size second dimensions. */
          class TestTwoDimArrays {
          static int [][] myArray = new int[3][]; // initialize # of rows

          public static void main(String[] args) {
          myArray[0] = new int[3];’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘// initialize # of cols
          myArray[1] = new int[4];’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘// in each row
          myArray[2] = new int[5];

          for(int i=0; i<3; i++)’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘// fill and print the array
          fillArray(i, i+3);
          System.out.println();
          }

          private static void fillArray(int row, int col) {
          for( int i=0; i
```

what is the output?

Practice 3.18

```
/* Example Multidimentional Array */
          class MultiDimArrayDemo {
          public static void main(String[] args) {
          String[][] names = {{"Mr. ", "Mrs. ", "Ms. "}, {"Smith", "Jones"}};
          System.out.println(names[0][0] + names[1][0]); //Mr. Smith
          System.out.println(names[0][2] + names[1][1]); //Ms. Jones
          }
          }
```

what is the output?

Practice 3.19

```
/* The following program, ArrayCopyDemo, declares an array of char elements, spelling the word
"decaffeinated". It uses arraycopy to copy a subsequence of array components into a second array */
          class ArrayCopyDemo {
          public static void main(String[] args) {
          char[] copyFrom = { 'd', 'e', 'c', 'a', 'f', 'f', 'e',
          'i', 'n', 'a', 't', 'e', 'd' };
          char[] copyTo = new char[7];
          System.arraycopy(copyFrom, 2, copyTo, 0, 7);
          System.out.println(new String(copyTo));
          }
          }
```

what is the output?

Practice 3.20

```
/* Example ArrayList */
          import java.util.*;

          class ArrayListDemo {
          public static void main(String args[]) {
          // create an array list
          ArrayList al = new ArrayList();
          System.out.println("Initial size of al: " + al.size());
          // add elements to the array list
          al.add("C");
          al.add("A");
          al.add("E");
          al.add("B");
          al.add("D");
          al.add("F");

          // display the array list
          System.out.println("Contents of al: " + al);
          al.add(1, "A2");
          System.out.println("Size of al after additions: " + al.size());

          // display the array list
          System.out.println("Contents of al: " + al);
          System.out.println("Index of D : " + al.indexOf("D"));
          System.out.println("Value at 2: " + al.size());
          System.out.println("The second elements is " + al.get(1));
```

```
            // Remove element from the array list
            al.remove("F");
            System.out.println("Size of al after deletions: " + al.size());

            // display the array list
            System.out.println("Contents of al: " + al);

            // replacing an element at a particular index
            al.set(2,"G");
            System.out.println("Size of al after deletions: " + al.size());

            // display the array list
            System.out.println("Contents of al: " + al);

            // Remove element from the array list
            al.remove(2);
            System.out.println("Size of al after deletions: " + al.size());
            System.out.println("Contents of al: " + al);

            // Remove all elements from the array list
            al.clear();
            System.out.println("Size of al after deletions: " + al.size());
            System.out.println("Contents of al: " + al);
            }
            }
```

what is the output?

Practice 3.21

```
            /* Example: convert an ArrayList into array */
            import java.util.*;

            class ArrayListToArray {
            public static void main(String args[]) {
            // Create an array list
            ArrayList al = new ArrayList();
            // Add elements to the array list
            al.add(new Integer(1));
            al.add(new Integer(2));
            al.add(new Integer(3));
            al.add(new Integer(4));
            System.out.println("Contents of al: " + al);
```

```
          // get array
          Object ia[] = al.toArray();
          int sum = 0;
          // sum the array
          for(int i=0; i
```
what is the output?

Practice 3.22

```
/* Example LinkList */
          import java.util.*;

          public class LinkedListDemo{
          public static void main(String[] args){
          LinkedList link=new LinkedList();
          link.add("a");
          link.add("b");
          link.add(new Integer(10));
          System.out.println("The contents of array is" + link);
          System.out.println("The size of an linkedlist is" + link.size());

          link.addFirst(new Integer(20));
          System.out.println("The contents of array is" + link);
          System.out.println("The size of an linkedlist is" + link.size());

          link.addLast("c");
          System.out.println("The contents of array is" + link);
          System.out.println("The size of an linkedlist is" + link.size());

          link.add(2,"j");
          System.out.println("The contents of array is" + link);
          System.out.println("The size of an linkedlist is" + link.size());

          link.add(1,"t");
          System.out.println("The contents of array is" + link);
          System.out.println("The size of an linkedlist is" + link.size());

          link.remove(3);
          System.out.println("The contents of array is" + link);
          System.out.println("The size of an linkedlist is" + link.size());
          //access element using iterator
          Iterator iterator;
          //Create a iterator
```

```
            iterator = link.iterator();
            while (iterator.hasNext()){
            System.out.print(iterator.next()+" ");
            }
            System.out.println();
            //Check list empty or not
            if (link.isEmpty()){
            System.out.println("Linked list is empty");
            }
            else{
            System.out.println( "Linked list size: " + link.size());
            }
            }
            }
```

what is the output?

Practice 3.23

```
/* Example Vector */
            //java.util.Vector and java.util.Enumeration;
            import java.util.*;

            public class VectorDemo{
            public static void main(String[] args){
            Vector< Object> vector = new Vector< Object>();
            int primitiveType = 10;
            Integer wrapperType = new Integer(20);
            String str = "tapan joshi";
            vector.add(primitiveType);
            vector.add(wrapperType);
            vector.add(str);
            vector.add(2, new Integer(30));
            System.out.println("the elements of vector: " + vector);
            System.out.println("The size of vector are: " + vector.size());
            System.out.println("The elements at position 2 is: " + vector.elementAt(2));
            System.out.println("The first element of vector is: " + vector.firstElement());
            System.out.println("The last element of vector is: " + vector.lastElement());
            vector.removeElementAt(2);
            Enumeration e=vector.elements();
            System.out.println("The elements of vector: " + vector);
            while(e.hasMoreElements()){
```

```
          System.out.println("The elements are: " + e.nextElement());
          }
          }
          }
```

what is the output?

Practice 3.24

```
/* Example Stack class */
          import java.util.*;

          public class StackDemo{
          public static void main(String[] args) {
          Stack stack=new Stack();
          stack.push(new Integer(10));
          stack.push("a");
          System.out.println("The contents of Stack is" + stack);
          System.out.println("The size of an Stack is" + stack.size());
          System.out.println("The number poped out is" + stack.pop());
          System.out.println("The number poped out is " + stack.pop());
          //System.out.println("The number poped out is" + stack.pop());
          System.out.println("The contents of stack is" + stack);
          System.out.println("The size of an stack is" + stack.size());
          }
          }
```

what is the output?

Practice 3.25

```
/* Implement Stack in Java */
          import java.io.*;
          import java.util.*;

          public class StackImplement{
          Stack stack;
          String str;
          int num, n;

          public static void main(String[] args){
          StackImplement q = new StackImplement();
          }
```

```
            public StackImplement(){
            try{
            stack = new Stack();
            InputStreamReader ir = new InputStreamReader(System.in);
            BufferedReader bf = new BufferedReader(ir);
            System.out.print("Enter number of elements : ");
            str = bf.readLine();
            num = Integer.parseInt(str);
            for(int i = 1; i <= num; i++){
            System.out.print("Enter elements : ");
            str = bf.readLine();
            n = Integer.parseInt(str);
            stack.push(n);
            }
            }
            catch(IOException e){}
            System.out.print("Retrieved elements from the stack : ");
            while (!stack.empty()){
            System.out.print(stack.pop() + "‘‘");
            }
            }
            }
```

what is the output?

Practice 3.26

```
/* Example queue */
            import java.util.Iterator;
            import java.util.LinkedList;
            import java.util.Queue;

            public class QueueExample {
            public static void main(String[] args) {
            Queue qe=new LinkedList();
            qe.add("b");
            qe.add("a");
            qe.add("c");
            qe.add("e");
            qe.add("d");
            // access queue using iterator
            Iterator it=qe.iterator();
```

```
            System.out.println("Initial Size of Queue :"+qe.size());
            while(it.hasNext()){
            String iteratorValue=(String)it.next();
            System.out.println("Queue Next Value :"+iteratorValue);
            }
            // get value and does not remove element from queue
            System.out.println("Queue peek :"+qe.peek());
            // get first value and remove that object from queue
            System.out.println("Queue poll :"+qe.poll());
            System.out.println("Final Size of Queue :"+qe.size());
            }
            }
```

what is the output?

Practice 3.27

```
/* Implement Queue in Java */
            import java.io.*;
            import java.util.*;

            public class QueueImplement{
            LinkedList list;
            String str;
            int num;

            public static void main(String[] args){
            QueueImplement q = new QueueImplement();
            }

            public QueueImplement(){
            try{
            list = new LinkedList();
            InputStreamReader ir = new InputStreamReader(System.in);
            BufferedReader bf = new BufferedReader(ir);
            System.out.println("Enter number of elements : ");
            str = bf.readLine();
            if((num = Integer.parseInt(str)) == 0){
            System.out.println("You have entered either zero/null.");
            System.exit(0);
            }
            else{
```

```
            System.out.println("Enter elements : ");
            for(int i = 0; i < num; i++){
            str = bf.readLine();
            int n = Integer.parseInt(str);
            list.add(n);
            }
            }
            System.out.println("First element :" + list.removeFirst());
            System.out.println("Last element :" + list.removeLast());
            System.out.println("Rest elements in the list :");
            while(!list.isEmpty()){
            System.out.print(list.remove() + "\t");
            }
            }
            catch(IOException e){
            System.out.println(e.getMessage() + " is not a legal entry.");
            System.exit(0);
            }
            }
            }
```

what is the output?

Practice 3.28

```
/* String operation example */
            public class stringmethod{
            public static void main(String[] args){

            // Construct one String from another
            char c[] = {'J', 'a', 'v', 'a'};
            String s1 = new String(c);
            String s2 = new String(s1);
            System.out.println(s1);
            System.out.println(s2);

            //calculate string length
            System.out.println(s2.length());

            //use the case functions:
            System.out.println("toUpperCase: " + s1.toUpperCase());
            System.out.println("toLowerCase: " + s1.toLowerCase());
```

```
            //check for a certain character using indexOf()
            System.out.println("indexOf('s'): " + s1.indexOf('s'));
            System.out.println("indexOf('v'): " + s1.indexOf('v'));
            System.out.println("indexOf('av'): " + s1.indexOf("av"));

            //print out the beginning character using charAt()
            System.out.println("first character: " + s1.charAt(0));

            // Construct string from subset of char array.
            byte ascii[] = {65, 66, 67, 68, 69, 70 };
            String s3 = new String(ascii);
            System.out.println(s3);
            String s4 = new String(ascii, 2, 3);
            System.out.println(s4);

            //string concatination
            String age = "9";
            String s = "He is " + age + " years old.";
            System.out.println(s);

            String string1 = "Hi";
            String string2 = new String("Hello");
            if (string1 == string2) {
            System.out.println("The strings are equal.");
            } else {
            System.out.println("The strings are unequal.");
            }
            String str = "Her name is Tamana and Tamana is a good girl.";
            String strreplace = "Sonia";
            String result = str.replaceFirst("Tamana", strreplace);
            System.out.println(result);
            }
            }
```

what is the output?

Practice 3.29

```
/* Simple version of bubble sort */

public class bubbleSort{
            public static void main(String a[]){
            int i;
```

```
           int array[] = {12,9,4,99,120,1,3,10};
           System.out.println("Values Before the sort:\n");
           for(i = 0; i < array.length; i++)
           System.out.print( array[i]+"‘‘");
           System.out.println();
           bubble_srt(array, array.length);
           System.out.print("Values after the sort:\n");
           for(i = 0; i <="" n;="" i++){="" for(j="1;" j="" (n-i);="" j++){="" if(a[j-1]="" style="box-
sizing: border-box;"> a[j]){
                       t = a[j-1];
                       a[j-1]=a[j];
                       a[j]=t;
                       }
                       }
‘‘‘‘       }
‘‘‘‘}
}
```

what is the output?

Practice 3.30

```
/* Simple version of bidirectional bubble sort */
‘‘‘‘public class BidirectionalBubbleSort{
           public static void main(String a[]){
           int i;
           int array[] = {12,9,4,99,120,1,3,10};
           System.out.println("Values Before the sort:\n");
           for(i = 0; i < array.length; i++)
           System.out.print( array[i]+"‘‘");
           System.out.println();
           bidirectionalBubble_srt(array, array.length);
           System.out.print("Values after the sort:\n");
           for(i = 0; i <="" n)="" {="" st++;="" n--;="" for="" (j="st;" j="" n;="" j++)="" if="
(array[j]=" style="box-sizing: border-box;"> array[j + 1]) {
           int T = array[j];
           array[j] = array[j + 1];
           array[j + 1] = T;
           }
           }
           for (j =‘‘n; --j >= st;) {
```

```
          if (array[j] > array[j + 1]) {
          int T = array[j];
          array[j] = array[j + 1];
          array[j + 1] = T;
          }
          }
          }
          }
          }
```

what is the output?

Practice 3.31

```
/* Simple version of bidirectional insertion sort */


public class InsertionSort{
‘‘‘‘public static void main(String a[]){
          int i;
          int array[] = {12,9,4,99,120,1,3,10};
          System.out.print("Values Before the sort:\n");
          for(i = 0; i < array.length; i++)
          System.out.print( array[i]+"‘‘");
          System.out.println();
          insertion_srt(array, array.length);
          System.out.print("Values after the sort:\n");
          for(i = 0; i <="" n;="" i++){="" j="i;" b="array[i];" while="" ((j="" style="box-sizing:
border-box;"> 0) && (array[j-1] > B)){
          array[j] = array[j-1];
          j--;                              }
          array[j] = B;
                    }
          }
          }
```

what is the output?

Practice 3.32

```
/* Simple version of bidirectional merge sort */


public class mergeSort{
‘‘‘‘public static void main(String a[]){
                    int i;
                    int array[] = {12,9,4,99,120,1,3,10};
                    System.out.print("Values Before the sort:\n");
```

```
                    for(i = 0; i < array.length; i++)
                    System.out.print( array[i]+"‘‘");
                    System.out.println();
                    mergeSort_srt(array,0, array.length-1);
                    System.out.print("Values after the sort:\n");
                    for(i = 0; i = high) {
                    return;              }
                    int middle = (low + high) / 2;
                    mergeSort_srt(array, low, middle);
                    mergeSort_srt(array, middle + 1, high);
                    int end_low = middle;
                    int start_high = middle + 1;
                    while ((lo <= end_low) && (start_high <= high)) {
                    if (array[low] < array[start_high]) {
                    low++;                          }
                    else {
                    int Temp = array[start_high];
                    for (int k = start_high- 1; k >= low; k--) {
                    array[k+1] = array[k];
                    }
                    array[low] = Temp;
                    low++;
                    end_low++;
                    start_high++;
                    }
                    }
          }’‘
          }
```

what is the output?

Practice 3.33

```
/* Selection sort as a method of array */

class ArraySel
          {
          ‘‘‘‘private double[] a;// ref to array a’‘
                    private int nElems; // number of data items
                    public ArraySel(int max) // constructor
                    {
```

```
                    a = new double[max]; // create the array
                    nElems = 0; // no items yet
                  }’‘
                  public void insert(double value)’‘// put element into array
                  {
                  a[nElems] = value; // insert it
                  nElems++; // increment size
                  }’‘
                  public void display() // displays array contents
                  {
                  for(int j=0;j< nElems; j++) // for each element
                  System.out.print(a[j] + " "); // display it
                  System.out.println("");
                  }’‘
                  public void selectionSort()
                  {
                  int out, in, min;
                  for(out=0; out< nElems-1; out++) // outer loop
                  {                              min = out; // minimum
                  for(in=out+1; in< nElems; in++) // inner loop
                  if(a[in] < a[min] ) // if min greater,
                  min = in; // we have a new min
                  swap(out, min); // swap them
                  } // end for(outer)
                  } // end selectionSort()
                  private void swap(int one, int two)
                  {
                  double temp = a[one];
                  a[one] = a[two];
                  a[two] = temp;
                  }
                  } // end class ArraySel
          ‘‘‘‘class SelectSortApp {
                  public static void main(String[] args)    {
                  int maxSize = 100; // array size
                  ArraySel arr; // reference to array
                  arr = new ArraySel(maxSize); // create the array
                  arr.insert(77); // insert 10 items
                  arr.insert(99);
                  arr.insert(44);
                  arr.insert(55);
```

```
                    arr.insert(22);
                    arr.insert(88);
                    arr.insert(11);
                    arr.insert(00);
                    arr.insert(66);
                    arr.insert(33);
                    arr.display(); // display items
                    arr.selectionSort(); // selection-sort them
                    arr.display(); // display them again
                    }
                    }
```

what is the output?

Practice 3.34

```
/* Quick sort as method of array */


class ArrayIns {
private double[] theArray; // ref to array theArray
private int nElems; // number of data items
public ArrayIns(int max) // constructor
{‘‘
‘‘‘‘theArray = new double[max]; // create the array
          nElems = 0; // no items yet
          }’‘
          public void insert(double value) // put element into array
          {
          theArray[nElems] = value; // insert it
          nElems++; // increment size
          }
          public void display() // displays array contents
          {
          System.out.print("A=");
          for(int j=0; j<="0)" if="" size="" return;="" already="" sorted="" else="" is="" 2="" or="" larger="" double="" pivot="theArray[right];" rightmost="" item="" partition="" range="" right,="" pivot);="" recquicksort(left,="" partition-1);="" sort="" left="" side="" recquicksort(partition+1,="" right);="" right="" end="" recquicksort()="" partitionit(int="" pivot)="" leftptr="left-1;" (after="" + +)="" rightptr="right;" right-1="" --)="" while(true)="" find="" bigger="" while(thearray[++leftptr]="" (nop)="" smaller="" while(rightptr="" style="box-sizing: border-box;"> 0 && theArray[--rightPtr] > pivot);
// (nop)
                    if(leftPtr >= rightPtr) // if pointers cross,
                    break; // partition done
                    else // not crossed, so
```

```
                    swap(leftPtr, rightPtr); // swap elements
                    } // end while(true)
                    swap(leftPtr, right); // restore pivot
                    return leftPtr; // return pivot location
                    } // end partitionIt()
          public void swap(int dex1, int dex2) // swap two elements
          {
          double temp = theArray[dex1]; // A into temp
          theArray[dex1] = theArray[dex2]; // B into A
          theArray[dex2] = temp; // temp into B
          } // end swap( } // end class ArrayIns
          class QuickSort1App {
          public static void main(String[] args)
‘‘
          {
                    int maxSize = 16; // array size
                    ArrayIns arr;
                    arr = new ArrayIns(maxSize); // create array
                    for(int j=0; j< maxSize; j++) // fill array with
                    {
                    // random numbers
                    double n = (int)(java.lang.Math.random()*99);
                    arr.insert(n);
                    }
                    arr.display(); // display items
                    arr.quickSort(); // quicksort them
          ‘‘ arr.display(); // display them again
          } // end main() } // end class QuickSort1App
```

what is the output?

Practice 3.35

```
/* Sorting array of objects using insertion sort */

class Person {
‘‘‘‘private String lastName;
          private String firstName;
          private int age;
          public Person(String last, String first, int a)
```

```
          {‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
          // constructor
          lastName = last;
          firstName = first;
          age = a;
          }
          public void displayPerson()    {
          System.out.print("‘‘Last name: " + lastName);
          System.out.print(", First name: " + firstName);
          System.out.println(", Age: " + age);      }
          public String getLast()’‘‘‘‘‘// get last name
          {
          return lastName;
          }
          } // end class Person
‘‘ class ArrayInOb {
‘‘‘‘private Person[] a;’‘‘‘‘‘‘‘// ref to array a
          private int nElems;’‘‘‘// number of data items
          public ArrayInOb(int max)’‘‘‘ // constructor
          {
          a = new Person[max];’‘‘‘‘‘‘‘// create the array
          nElems = 0;’‘‘‘‘‘‘‘‘‘‘‘// no items yet
          }’‘‘‘// put person into array
          public void insert(String last, String first, int age)
          {
          a[nElems] = new Person(last, first, age);
          nElems++;’‘‘‘‘‘‘‘‘‘‘‘ // increment size
          }’‘
          public void display()’‘‘‘‘‘ // displays array contents
          {
          for(int j=0; j0 && a[in-1].getLast().compareTo(temp.getLast())>0)         // until smaller one
found,
           {                              ‘‘
           a[in] = a[in-1];’‘// shift item to the right
          --in;’‘‘‘‘‘‘‘‘‘// go left one position
          }
          a[in] = temp;’‘‘‘‘‘‘‘// insert marked item
          } // end for         } // end insertionSort()
          } // end class ArrayInOb
////////////////////////////////////////////////////////////////
class ObjectSortApp {
```

```
          public static void main(String[] args)
          {         int maxSize = 100;’‘‘‘‘‘ // array size
                    ArrayInOb arr;’‘‘‘‘‘‘‘ // reference to array
                    arr = new ArrayInOb(maxSize); // create the array
                    arr.insert("Evans", "Patty", 24);
                    arr.insert("Smith", "Doc", 59);
                    arr.insert("Smith", "Lorraine", 37);
                    arr.insert("Smith", "Paul", 37);
                    arr.insert("Yee", "Tom", 43);
                    arr.insert("Hashimoto", "Sato", 21);
                    arr.insert("Stimson", "Henry", 29);
                    arr.insert("Velasquez", "Jose", 72);
                    arr.insert("Vang", "Minh", 22);
                    arr.insert("Creswell", "Lucinda", 18);
                    System.out.println("Before sorting:");
                    arr.display();’‘‘‘‘‘‘‘ // display items
                    arr.insertionSort();’‘‘‘‘‘// insertion-sort them
                    System.out.println("After sorting:");
                    arr.display();’‘‘‘‘‘‘‘ // display them again
                    } // end main()
} // end class ObjectSortApp
```

what is the output?

Practice 3.36

```
/* This example shows how to sort elements of Java ArrayList in descending order using comparator and
reverseOrder method of Collections class. */


import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
public class SortArrayListInDescendingOrderExample {
          public static void main(String[] args) {
                    //create an ArrayList object
                    ArrayList arrayList = new ArrayList();              //Add elements to Arraylist
                    arrayList.add("A");
                    arrayList.add("B");
                    arrayList.add("C");
                    arrayList.add("D");
                    arrayList.add("E");
```

```
                    /* To get comparator that imposes reverse order on a Collection use static Comparator
reverseOrder() method of Collections class */
                    Comparator comparator = Collections.reverseOrder();‘‘
                    System.out.println("Before sorting ArrayList in descending order : " + arrayList);
                    /* To sort an ArrayList using comparator use, static void sort(List list, Comparator
c) method of Collections class. */
                    Collections.sort(arrayList,comparator);
                    System.out.println("After sorting ArrayList in descending order : " + arrayList);
                    }
                    }
```

what is the output?

Practice 3.37

```
/* Example of binary search in sorted array */
public class BinarySearch {‘‘‘‘
           public static final int NOT_FOUND = -1;
                    public static int binarySearch( Comparable [ ] a, Comparable x )
                    {‘‘‘‘‘‘‘‘
                    int low = 0;’‘‘‘‘‘
                    int high = a.length - 1;’‘
                    int mid;’‘‘‘‘‘‘‘‘‘
                    while( low <= high )’‘‘‘‘‘
                    {‘‘‘‘‘‘‘‘‘‘‘‘
                    mid = ( low + high ) / 2;’‘‘‘
                    if( a[ mid ].compareTo( x ) < 0 )’‘‘‘
                    low = mid + 1;’‘‘‘‘‘‘‘‘‘‘‘
                    else if( a[ mid ].compareTo( x ) > 0 )
                              ‘‘high = mid - 1;’‘
                              ‘‘else’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                              ‘‘return mid;’‘‘‘‘‘
                              ‘‘}’‘‘‘‘‘‘‘
                              ‘‘return NOT_FOUND;’‘‘‘ // NOT_FOUND = -1’‘‘‘
                              ‘‘}

                    // Test program
                    public static void main( String [ ] args )’‘‘‘ {‘‘‘‘
                    int SIZE = 8;’‘‘‘‘‘‘‘
                    Comparable [ ] a = new Integer [ SIZE ];
                    for( int i = 0; i < SIZE; i++ )’‘‘‘‘‘‘‘‘‘
                              a[ i ] = new Integer( i * 2 );
                              for( int i = 0; i < SIZE * 2; i++ )’‘‘‘‘‘
```

```
                                System.out.println( "Found " + i + " at " +binarySearch( a, new
Integer( i ) ) );
                        }
                }
```

what is the output?

Practice 3.38

```
/* Binary search in ArrayList */


/*'' Perform Binary Search on Java ArrayList Example
''This java example shows how to search an element of Java ArrayList'' using binarySearch method of
Collections class. binarySearch method uses
''binary search algorithm to search an element.
*/
import java.util.ArrayList;
import java.util.Collections;
public class BinarySearchArrayListExample {
          ''public static void main(String[] args) {''
          ''//create an ArrayList object
                    ArrayList arrayList = new ArrayList();
                    //Add elements to Arraylist''''
                    arrayList.add("1");''''
                    arrayList.add("4");''''
                    arrayList.add("2");''
                    arrayList.add("5");''''
                    arrayList.add("3");
                    /*'''''' To Search an element of Java ArrayList using binary search algorithm
use,static int binarySearch(List list, Object element) method of Collections class.This method returns the
index of the value to be searched, if found in the'''''' ArrayList.'''''' Otherwise it returns (- (X) -
1)'''''' where X is the index where the the search value would be inserted.
                    ''i.e. index of first element that is grater than the search value'''''' or
ArrayList.size(), if all elements of an ArrayList are less than the search value.
          Please note that the ArrayList MUST BE SORTED before it can be searchedusing binarySearch
method.'''' */''''
                    //First sort an ArrayList using sort method of Collections class
                    Collections.sort(arrayList);''
                    System.out.println("Sorted ArrayList contains : " + arrayList);
                    //search an element using binarySearch method of Collections class''''
                    int index = Collections.binarySearch(arrayList,"4");
                    System.out.println("Element found at : " + index);''
                    }
                    }
```

what is the output?

Practice 3.39

```
/* Example of Binary Search Tree */


class Node {‘‘‘‘
                    private int key;’‘‘‘
                    private Node parent;
                    private Node leftChild;
                    private Node rightChild;
                    public Node(int key, Node leftChild, Node rightChild)
                    {
                              this.setKey(key);’‘‘‘
                              this.setLeftChild(leftChild);’‘‘‘
                              this.setRightChild(rightChild);’‘‘‘
                              }
                    public void setKey(int key) {
                    this.key = key;’‘
                    }
                    public int getKey() {‘‘
                    return key;’‘
                    }
                    public void setParent(Node parent) {
                    this.parent = parent;’‘
                    }
                    public Node getParent() {‘‘
                    return parent;’‘‘‘
                    }
                    public void setLeftChild(Node leftChild) {‘‘
                    this.leftChild = leftChild;’‘
                    }
                    public Node getLeftChild() {
                                        return leftChild;’‘‘‘ }
          ‘‘
                    public void setRightChild(Node rightChild)
                    {
                              this.rightChild = rightChild;’‘‘‘
                              }
                    public Node getRightChild() {‘‘
```

```
return rightChild;‘‘‘‘
}
}
class BinarySearchTree {‘‘‘‘
private Node root;
public void insert(int key) {
          insert(new Node(key, null, null));‘‘
          }
public void insert(Node z) {‘‘
Node y = null;‘‘‘‘‘‘‘
Node x = root;‘‘‘‘‘‘‘
while (x != null) {‘‘‘‘
y = x;‘‘‘‘‘‘‘‘‘‘‘
if (z.getKey() < x.getKey())
{‘‘
x = x.getLeftChild();‘‘‘‘‘‘‘
}
else {‘‘‘‘‘‘
x = x.getRightChild();‘‘‘‘‘‘
}‘‘‘‘‘‘
}‘‘‘‘‘‘‘
z.setParent(y);‘‘‘‘
if (y == null){
‘‘ root = z;‘‘‘‘‘‘
‘‘ }
‘‘ else if (z.getKey() < y.getKey()) {
                    y.setLeftChild(z);‘‘‘‘‘‘
                    }
                    else {‘‘
                    y.setRightChild(z);‘‘‘‘‘‘
                    }‘‘
                    }‘‘‘‘
                    public void preorderTraversal() {‘‘‘‘‘‘‘‘
                    preorderTraversal(root);‘‘‘‘ }
public void preorderTraversal(Node node) {
if (node != null) {‘‘‘‘‘‘‘‘‘‘
System.out.print(node.getKey() + " ");‘‘‘‘‘‘‘‘‘‘‘
preorderTraversal(node.getLeftChild());‘‘‘‘‘‘‘‘‘‘‘
preorderTraversal(node.getRightChild());‘‘‘‘‘‘‘
}
}
```

```
                    public void inorderTraversal() {‘‘‘‘‘‘
                    inorderTraversal(root);’‘‘‘ }
                    private void inorderTraversal(Node node) {‘‘
                    if (node != null) {‘‘‘‘‘‘‘‘‘‘‘‘
                    inorderTraversal(node.getLeftChild());’‘‘‘
                    System.out.print(node.getKey() + " ");’‘‘‘‘‘
                    inorderTraversal(node.getRightChild());’‘
                    }’‘‘‘
                    }’‘
                    public void postorderTraversal() {‘‘‘‘‘‘
                    postorderTraversal(root);’‘
                    }
                    private void postorderTraversal(Node node) {‘‘
                    if (node != null) {‘‘‘‘‘‘‘‘
                    postorderTraversal(node.getLeftChild());’‘
                    postorderTraversal(node.getRightChild());
                    System.out.print(node.getKey() + " ");’‘‘‘‘‘
                    }’‘‘‘
                    }
                    }
                    class BinarySearchTreeTest {‘‘‘‘
                    public static void main(String[] args) {‘‘
                    BinarySearchTree bst = new BinarySearchTree();
                    int[] input = new int[] { 8, 3, 10, 1, 6, 14, 4, 7, 13 };
                              for (int i : input) {‘‘‘‘‘‘‘‘
                              bst.insert(i);’‘‘‘
                              }
                              System.out.println("Preorder Traversal:");
                              bst.preorderTraversal();’‘‘‘‘‘
                              System.out.println( "\nInorder Traversal:");
                              bst.inorderTraversal();’‘‘‘‘‘‘‘‘‘
                              System.out.println("\nPostorder Traversal:");
                              bst.postorderTraversal();’‘‘‘
                              }
          }
```

what is the output?

Practice 3.40

```
/* Method overloading. */
```

```
class OverloadDemo {
void test() {
System.out.println("No parameters");
}

// Overload test for one integer parameter.
void test(int a) {
System.out.println("a: " + a);
}

// Overload test for two integer parameters.
void test(int a, int b) {
System.out.println("a and b: " + a + " " + b);
}

// overload test for a double parameter
double test(double a) {
System.out.println("double a: " + a);
return a * a;
}
}

class Overload {
public static void main(String args[]) {
OverloadDemo ob = new OverloadDemo();
double result;
// call all versions of test()
ob.test();
ob.test(10);
ob.test(10, 20);
result = ob.test(123.25);
System.out.println("Result of ob.test(123.25): " + result);
}
}
```

what is the output?

Practice 3.41

```
/* Constructor overloading */
```

```
public class Cube {
int length;
int breadth;
int height;

public int getVolume() {
return (length * breadth * height);
}

Cube() {
this(10, 10);
System.out.println("Initialized with Default Constructor");
}

Cube(int l, int b) {
this(l, b, 10);
System.out.println("Initialized with Constructor having 2 params");
}

Cube(int l, int b, int h) {
length = l;
breadth = b;
height = h;
System.out.println("Initialized with Constructor having 3 params");
}

Cube(Cube ob) { //pass object to constructor
length = ob.length;
breadth = ob.breadth;
height = ob.height;
System.out.println("Finished with Constructor having object params");
}

public static void main(String[] args) {
Cube cubeObj1, cubeObj2, cubeObj3;
cubeObj1 = new Cube();
cubeObj2 = new Cube(10, 20, 30);
cubeObj3 = new Cube(cubeObj2); // Creating clone object
System.out.println("Volume of Cube1 is : " + cubeObj1.getVolume());
System.out.println("Volume of Cube2 is : " + cubeObj2.getVolume());
System.out.println("Volume of Cube3 is : " + cubeObj3.getVolume());
```

```
          }
          }
```

what is the output?

Practice 3.42

```
/* A simple example of inheritance. */

          // Create a superclass.
          class A {
          int i, j;

          void showij() {
          System.out.println("i and j: " + i + " " + j);
          }
          }

          // Create a subclass by extending class A.
          class B extends A {
          int k;

          void showk() {
          System.out.println("k: " + k);
          }

          void sum() {
          System.out.println("i+j+k: " + (i + j + k));
          }
          }

          class SimpleInheritance {
          public static void main(String args[]) {
          A superOb = new A();
          B subOb = new B();
          // The superclass may be used by itself.
          superOb.i = 10;
          superOb.j = 20;
          System.out.println("Contents of superOb: ");
          superOb.showij();
          System.out.println();
```

```
            /* The subclass has access to all public members of its superclass. */
            subOb.i = 7;
            subOb.j = 8;
            subOb.k = 9;
            System.out.println("Contents of subOb: ");
            subOb.showij();
            subOb.showk();
            System.out.println();
            System.out.println("Sum of i, j and k in subOb:");
            subOb.sum();
            }
            }
```

what is the output?

Practice 3.43

```
/* Inheritance example: initializing through constructor */
            class Box {
            double width;
            double height;
            double depth;

            Box(){‘‘// constructor used when all dimensions specified
            }

            Box(double w, double h, double d) {
            width = w;
            height = h;
            depth = d;
            }

            double volume() { // compute and return volume
            return width * height * depth;
            }
            }

            // Here, Box is extended to include weight.
            class BoxWeight extends Box {
            double weight; // weight of box
```

```
            // constructor for BoxWeight
            BoxWeight(double w, double h, double d, double m) {
            width = w;
            height = h;
            depth = d;
            weight = m;
            }
            }

            class DemoBoxWeight {
            public static void main(String args[]) {
            BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
            BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
            double vol;
            vol = mybox1.volume();
            System.out.println("Volume of mybox1 is " + vol);
            System.out.println("Weight of mybox1 is " + mybox1.weight);
            System.out.println();
            vol = mybox2.volume();
            System.out.println("Volume of mybox2 is " + vol);
            System.out.println("Weight of mybox2 is " + mybox2.weight);
            }
            }
```

what is the output?

Practice 3.44

```
/* Inheritance example: initializing through constructor */
            class Box {
            double width;
            double height;
            double depth;

            Box(){‘‘// constructor used when all dimensions specified
            }

            Box(double w, double h, double d) {
            width = w;
            height = h;
            depth = d;
```

```
}

double volume() { // compute and return volume
return width * height * depth;
}
}

// Here, Box is extended to include weight.
class BoxWeight extends Box {
double weight; // weight of box

// constructor for BoxWeight
BoxWeight(double w, double h, double d, double m) {
width = w;
height = h;
depth = d;
weight = m;
}
}

class DemoBoxWeight {
public static void main(String args[]) {
BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
double vol;
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);
System.out.println("Weight of mybox1 is " + mybox1.weight);
System.out.println();
vol = mybox2.volume();
System.out.println("Volume of mybox2 is " + vol);
System.out.println("Weight of mybox2 is " + mybox2.weight);
}
}
```

what is the output?

Practice 3.45

```
/* Example of a superclass variable referring to a subclass Object*/
```

```
class Box {
double width;
double height;
double depth;

Box() {‘‘// default constructor
}

// constructor used when all dimensions specified
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}

double volume() { // compute and return volume
return width * height * depth;
}
}

// Here, Box is extended to include weight.
class BoxWeight extends Box {
double weight; // weight of box

// constructor for BoxWeight
BoxWeight(double w, double h, double d, double m) {
width = w;
height = h;
depth = d;
weight = m;
}
}

class RefDemo {
public static void main(String args[]) {
BoxWeight weightbox = new BoxWeight(3, 5, 7, 8.37);
Box plainbox = new Box();
double vol;
vol = weightbox.volume();
System.out.println("Volume of weightbox is " + vol);
System.out.println("Weight of weightbox is "
```

```
            + weightbox.weight);
            System.out.println();
            // assign BoxWeight reference to Box reference
            plainbox = weightbox;
            vol = plainbox.volume(); // OK, volume() defined in Box
            System.out.println("Volume of plainbox is " + vol);
            /* The following statement is invalid because plainbox does not define a weight member. */
            // System.out.println("Weight of plainbox is " + plainbox.weight);
            }
            }
```

what is the output?

Practice 3.46

```
/* Simple example of access modifier. In a class hierarchy, private members remain private to their class.
This program contains an error and will not compile. */

            // Create a superclass.
            class A {
            int i; // public by default
            private int j; // private to A

            void setij(int x, int y) {
            i = x;
            j = y;
            }
            }

            // A's j is not accessible here.
            class B extends A {
            int total;

            void sum() {
            total = i + j; // ERROR, j is not accessible here
            }
            }

            class Access {
            public static void main(String args[]) {
            B subOb = new B();
            subOb.setij(10, 12);
```

```
            subOb.sum();
            System.out.println("Total is " + subOb.total);
            }
}
```

what is the output?

Practice 3.47

```
/* Another example of access modifier with public, private and protected data */

            class BaseClass {
            public int x = 10;
            private int y = 10;
            protected int z = 10;
            int a = 10; //Implicit Default Access Modifier

            public int getX() {
            return x;
            }

            public void setX(int x) {
            this.x = x;
            }

            private int getY() {
            return y;
            }

            private void setY(int y) {
            this.y = y;
            }

            protected int getZ() {
            return z;
            }

            protected void setZ(int z) {
            this.z = z;
            }
```

```
            int getA() {
            return a;
            }

            void setA(int a) {
            this.a = a;
            }
            }

            public class SubclassInSamePackage extends BaseClass {
            public static void main(String args[]) {
            BaseClass rr = new BaseClass();
            rr.z = 0;
            SubclassInSamePackage subClassObj = new SubclassInSamePackage();
            //Access Modifiers - Public
            System.out.println("Value of x is : " + subClassObj.x);
            subClassObj.setX(20);
            System.out.println("Value of x is : " + subClassObj.x);
            //Access Modifiers - Public
            // If we remove the comments it would result in a compilaton
            //''      error as the fields and methods being accessed are private

            /* System.out.println("Value of y is : "+subClassObj.y);
            subClassObj.setY(20);
            System.out.println("Value of y is : "+subClassObj.y);*/

            //Access Modifiers - Protected
            System.out.println("Value of z is : " + subClassObj.z);
            subClassObj.setZ(30);
            System.out.println("Value of z is : " + subClassObj.z);
            //Access Modifiers - Default
            System.out.println("Value of x is : " + subClassObj.a);
            subClassObj.setA(20);
            System.out.println("Value of x is : " + subClassObj.a);
}
}
```

what is the output?

Practice 3.48

```
/* Sinple example of super concept */
```

```
// A complete implementation of BoxWeight.
class Box {
private double width;
private double height;
private double depth;

// constructor used when all dimensions specified
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}

// compute and return volume
double volume() {
return width * height * depth;
}
}

// BoxWeight now fully implements all constructors.
class BoxWeight extends Box {
double weight; // weight of box

// constructor when all parameters are specified
BoxWeight(double w, double h, double d, double m) {
super(w, h, d); // call superclass constructor
weight = m;
}
}

class DemoSuper {
public static void main(String args[]) {
BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
double vol;
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);
System.out.println("Weight of mybox1 is " + mybox1.weight);
System.out.println();
vol = mybox2.volume();
```

```
          System.out.println("Volume of mybox2 is " + vol);
          System.out.println("Weight of mybox2 is " + mybox2.weight);
          System.out.println();
          }
          }
```

what is the output?

Practice 3.49

```
/* Example of super to overcome name hiding */

          class A {
          int i;
          }

          // Create a subclass by extending class A.
          class B extends A {
          int i; // this i hides the i in A

          B(int a, int b) {
          super.i = a; // i in A
          i = b; // i in B
          }

          void show() {
          System.out.println("i in superclass: " + super.i);
          System.out.println("i in subclass: " + i);
          }
          }

          class UseSuper {
          public static void main(String args[]) {
          B subOb = new B(1, 2);
          subOb.show();
          }
          }
```

what is the output?

Practice 3.50

```
/* Code sharing through super concept */

          class Cat {
          void speak() {
          System.out.println("Meaon ! ");
          }
          }

          class PetCat extends Cat {               // PetCat is one type of Cat
          void speak() {
          System.out.println(" Meow ! ");
          }
          }

          class MagicCat extends Cat { // MagicCat is another kind of Cat
          static boolean noOne;

          void speak() {
          if (noOne) {
          super.speak(); // use the super class definition
          } else {
          System.out.println(" Hello World !");
          }
          }
          }

          class ManyCats {
          public static void main(String args[]) {
          PetCat c1 = new PetCat();
          MagicCat c2 = new MagicCat();
          c2.noOne = true;
          c2.speak();
          c1.speak();
          c2.noOne = false;
          c2.speak();
          }
          }
```

what is the output?

Practice 3.51

```
/* Example of multilevel inheritance. */

          // Start with Box.
          class Box {
          private double width;
          private double height;
          private double depth;

          // constructor used when all dimensions specified
          Box(double w, double h, double d) {
          width = w;
          height = h;
          depth = d;
          }

          // compute and return volume
          double volume() {
          return width * height * depth;
          }
          }

          // Add weight.
          class BoxWeight extends Box {
          double weight; // weight of box
          // constructor when all parameters are specified
          BoxWeight(double w, double h, double d, double m) {
          super(w, h, d); // call superclass constructor
          weight = m;
          }
          }

          // Add shipping costs
          class Shipment extends BoxWeight {
          double cost;

          // constructor when all parameters are specified
          Shipment(double w, double h, double d, double m, double c) {
          super(w, h, d, m); // call superclass constructor
          cost = c;
          }
          }
```

```
class DemoShipment {
public static void main(String args[]) {
Shipment shipment1 = new Shipment(10, 20, 15, 10, 3.41);
Shipment shipment2 = new Shipment(2, 3, 4, 0.76, 1.28);
double vol;
vol = shipment1.volume();
System.out.println("Volume of shipment1 is " + vol);
System.out.println("Weight of shipment1 is " + shipment1.weight);
System.out.println("Shipping cost: $" + shipment1.cost);
System.out.println();
vol = shipment2.volume();
System.out.println("Volume of shipment2 is " + vol);
System.out.println("Weight of shipment2 is " + shipment2.weight);
System.out.println("Shipping cost: $" + shipment2.cost);
}
}
```

what is the output?

Practice 3.52

```
/* Polymorphism and Overriding concept */

//base class
class Base
{
int i=1;
int j=2;
public Base(){
}
public void display(){
System.out.println("i="+i);
System.out.println("i="+j);
}
}

//derived class
class Derived extends Base{
int p=3;
int q=4;

public Derived(){
```

```
          }

          public void display(){
          System.out.println("p="+p);
          System.out.println("q="+q);
          }

          public void uniqueDisplay(){
          System.out.println(" This display is unique to Derived");
          }
          }

          public class AccessCheck{
          public static void main(String[] args){
          Base b=new Base();
          Derived d=new Derived();
          b.display();
          d.display();
          d.uniqueDisplay();
          // derived is assigned to base
          b=d;
          // Object is derived, reference is of base type
          b.display();
          //b.uniqueDisplay();'' // Uncomment this line, and check
          }
          }
```

what is the output?

Practice 3.53

```
/* Using run-time polymorphism. */

          class Figure {
          double dim1;
          double dim2;

          Figure(double a, double b) {
          dim1 = a;
          dim2 = b;
          }
```

```
double area() {
System.out.println("Area for Figure is undefined.");
return 0;
}
}

class Rectangle extends Figure {
Rectangle(double a, double b) {
super(a, b);
}

// override area for rectangle
double area() {
System.out.println("Inside Area for Rectangle.");
return dim1 * dim2;
}
}

class Triangle extends Figure {
Triangle(double a, double b) {
super(a, b);
}

// override area for right triangle
double area() {
System.out.println("Inside Area for Triangle.");
return dim1 * dim2 / 2;
}
}

class FindAreas {
public static void main(String args[]) {
Figure f = new Figure(10, 10);
Rectangle r = new Rectangle(9, 5);
Triangle t = new Triangle(10, 8);
Figure figref;
figref = r;
System.out.println("Area is " + figref.area());
figref = t;
System.out.println("Area is " + figref.area());
figref = f;
```

```
        System.out.println("Area is " + figref.area());
        }
        }


what is the output?
```

Assignment

Q: Read at most 10 names of students and store them into an array of String nameOfStudents[10]. Sort the names into the lexicographic order. Display the sorted list of names.

Q: Define a class Complex to represent an object for a complex number like Z = X + i.Y with the following methods:

```
Complex add(Complex z1, Complex z2) //To add two complex numbers
Complex sub(Complex z1, Complex z2) //To subtract two complex numbers
Complex mul(Complex z1, Complex z2) // To multiply two complex numbers
float magnitude(Complex z)     ‘‘‘‘‘‘ // To find the modulus
Complex conjugate(Complex z)’‘‘‘‘‘‘ // To find the complex conjugate
```

Write the main class and instantiate the objects of the above mentioned classes.

Q: Add the necessary methods in the class PointCreate3 (Practice 3.8) to calculate the area and perimeter of a rectangle given the two corner coordinates.

Q: Read at most 10 names of students and store them into an array of String nameOfStudents[10]. Sort the names into the lexicographic order. Display the sorted list of names.

Q: Define a class Employee with usual member for an employee like empCode(String), empName(String), dateOfBirth(Date), dateOfJoin(Date), designationCode(int), salary(float).Create a list to store data about 10 employees using Vector. Manipulate the list using the methods in class Vector.

Q: Define an one dimensional array "vivid" of type float. Read the values from the keyboard to load the array. Calcualte and then print the average of all the values in "vivid"..

Q: Define two 2D arrays of integers, namely A[3]4] and B[4][2]. Store the values into them. Store the result of matrix multiplication into an another 2D array, say C.

Q: Write a program to store a lists of name in a List. Reverse the order of the names in the list using Stack. You should use class ArrayList to store the names (String) and class Stack for reversing..

Q: Create a class ArrSort which contains one double array to store the data and one integer variable to store the number of elements.

Q: Write a program to read the content of a file and count the number of words

in the file.

Q: Write a program to copy contents from one file to another file.

Q: Write a program to merge two file contents into another file.

Q&A

Q: What are the principle concepts of OOPS?

A: There are four principle concepts upon which object oriented design and programming rest. They are:

- Abstraction
- Polymorphism
- Inheritance
- Encapsulation

(i.e. easily remembered as A-PIE).

Q: What is Abstraction?

A: Abstraction refers to the act of representing essential features without including the background details or explanations.

Q: What is Encapsulation?

A: Encapsulation is a technique used for hiding the properties and behaviors of an object and allowing outside access only as appropriate. It prevents other objects from directly altering or accessing the properties or methods of the encapsulated object.

Q: What is the difference between abstraction and encapsulation?

A: □'‘Abstraction focuses on the outside view of an object (i.e. the interface) Encapsulation (information hiding) prevents clients from seeing it's inside view, where the behavior of the abstraction is implemented.
□'‘Abstraction solves the problem in the design side while Encapsulation is the Implementation.
□'‘Encapsulation is the deliverables of Abstraction. Encapsulation barely talks about grouping up your abstraction to suit the developer needs.

Q: What is Inheritance?

A: Inheritance is the process by which objects of one class acquire the properties of objects of another class.
□'‘Inheritance is the process by which objects of one class acquire the properties of objects of another class.
□'‘A class that is inherited is called a superclass.
□'‘The class that does the inheriting is called a subclass.
□'‘Inheritance is done by using the keyword extends.
□'‘The two most common reasons to use inheritance are:

- To promote code reuse
- To use polymorphism

Q: What is Polymorphism?

A: Polymorphism is briefly described as "one interface, many implementations." Polymorphism is a characteristic of being able to assign a different meaning or usage to something in different contexts - specifically, to allow an entity such as a variable, a function, or an object to have more than one form.

Q: How does Java implement polymorphism?

A: (Inheritance, Overloading and Overriding are used to achieve Polymorphism in java). Polymorphism manifests itself in Java in the form of multiple methods having the same name.

- In some cases, multiple methods have the same name, but different formal argument lists (overloaded methods).
- In other cases, multiple methods have the same name, same return type, and same formal argument list (overridden methods).

Q: Explain the different forms of Polymorphism.

A: There are two types of polymorphism one is Compile time polymorphism and the other is run time polymorphism. Compile time polymorphism is method overloading. Runtime time polymorphism is done using inheritance and interface. **Note:** From a practical programming viewpoint, polymorphism manifests itself in three distinct forms in Java:

- Method overloading
- Method overriding through inheritance
- Method overriding through the Java interface

Q: What is runtime polymorphism or dynamic method dispatch?

A: In Java, runtime polymorphism or dynamic method dispatch is a process in which a call to an overridden method is resolved at runtime rather than at compile-time. In this process, an overridden method is called through the reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

Q: What is Dynamic Binding?

A: Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding (also known as late binding) means that the code associated with a given procedure call is not known until the time of the call at run-time. It is associated with polymorphism and inheritance.

Q: What is method overloading?

A: Method Overloading means to have two or more methods with same name in the same class with different arguments. The benefit of method overloading is that it allows you to implement methods that support the same semantic operation but differ by argument number or type.
Note:

- *Overloaded methods MUST change the argument list*
- *Overloaded methods CAN change the return type*
- *Overloaded methods CAN change the access modifier*

- *Overloaded methods CAN declare new or broader checked exceptions*
- *A method can be overloaded in the same class or in a subclass*

Q: What is method overriding?
A: Method overriding occurs when sub class declares a method that has the same type arguments as a method declared by one of its superclass. The key benefit of overriding is the ability to define behavior that's specific to a particular subclass type.
Note:

- *The overriding method cannot have a more restrictive access modifier than the method being overridden (Ex: You can't override a method marked public and make it protected).*
- *You cannot override a method marked final*
- *You cannot override a method marked static*

Q: Can overloaded methods be override too?
A: Yes, derived classes still can override the overloaded methods. Polymorphism can still happen. Compiler will not binding the method calls since it is overloaded, because it might be overridden now or in the future.
Q: Is it possible to override the main method?
A: NO, because main is a static method. A static method can't be overridden in Java.
Q: How to invoke a superclass version of an Overridden method?
A: To invoke a superclass method that has been overridden in a subclass, you must either call the method directly through a superclass instance, or use the super prefix in the subclass itself. From the point of the view of the subclass, the super prefix provides an explicit reference to the superclass' implementation of the method.

```
// From subclass
                    super.overriddenMethod();
```

Q: What is super?
A: super is a keyword which is used to access the method or member variables from the superclass. If a method hides one of the member variables in its superclass, the method can refer to the hidden variable through the use of the super keyword. In the same way, if a method overrides one of the methods in its superclass, the method can invoke the overridden method through the use of the super keyword.
Note:

- *You can only go back one level.*
- *In the constructor, if you use super(), it must be the very first code, and you cannot access any `this.xxx` variables or methods to compute its parameters.*

Q: How do you prevent a method from being overridden?
A: To prevent a specific method from being overridden in a subclass, use the final

modifier on the method declaration, which means "this is the final implementation of this method", the end of its inheritance hierarchy.

```
        ‘‘‘‘‘‘‘public final void exampleMethod() {
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘//’‘Method statements
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘}
```

Q: What is Constructor?

A:

- A constructor is a special method whose task is to initialize the object of its class.
- It is special because its name is the **same as the class name**.
- They do not have return types, not even **void** and therefore they cannot return values.
- They **cannot be inherited**, though a derived class can call the base class constructor.
- Constructor is invoked whenever an object of its associated class is created.

Q: How does the Java default constructor be provided?

A: If a class defined by the code does not have any constructor, compiler will automatically provide one no-parameter-constructor (default-constructor) for the class in the byte code. The access modifier (public/private/etc.) of the default constructor is the same as the class itself.

Q: Can constructor be inherited?

A: No, constructor cannot be inherited, though a derived class can call the base class constructor.

Q: How are this() and super() used with constructors?

A:

- Constructors use *this* to refer to another constructor in the same class with a different parameter list.
- Constructors use *super* to invoke the superclass's constructor. If a constructor uses *super*, it must use it in the first line; otherwise, the compiler will complain.

Q: What are Access Specifiers available in Java?

A:

- **Public**- *public* classes, methods, and fields can be accessed from everywhere.
- **Protected**- *protected* methods and fields can only be accessed within the same class to which the methods and fields belong, within its subclasses, and within classes of the same package.
- **Default(no specifier)-** If you do not set access to specific level, then such a class, method, or field will be accessible from inside the same package to which the class, method, or field belongs, but not from outside this package.
- **Private**- *private* methods and fields can only be accessed within the same class to which the methods and fields belong. *private* methods and fields

are not visible within subclasses and are not inherited by subclasses.

Q: What is final modifier?

A:
- ***final* Classes**- A final class cannot have subclasses.
- ***final* Variables**- A final variable cannot be changed once it is initialized.
- ***final* Methods**- A final method cannot be overridden by subclasses.

Q: What are the uses of final method?

A:
- Disallowing subclasses to change the meaning of the method.
- Increasing efficiency by allowing the compiler to turn calls to the method into inline Java code.

Q: What are static variables?

A: Variables that have only one copy per class are known as static variables. They are not attached to a particular instance of a class but rather belong to a class as a whole. They are declared by using the static keyword as a modifier.

```
static type’‘varIdentifier;
```

where, the name of the variable is varIdentifier and its data type is specified by type.

Note: Static variables that are not explicitly initialized in the code are automatically initialized with a default value. The default value depends on the data type of the variables.

Q: What is static block?

A: Static block which exactly executed exactly once when the class is first loaded into JVM. Before going to the main method the static block will execute.

Q: What is the difference between static and non-static variables?

A: A static variable is associated with the class as a whole rather than with specific instances of a class. Non-static variables take on unique values with each object instance.

Q: What are static methods?

A:
- *A static method can only call other static methods.*
- *A static method must only access static data.*
- *A static method* ***cannot*** *reference to the current object using keywords super or this.*

Chapter 4

Interfaces and Packages in Java

by Debasis Samanta

CONTENTS

- Introduction
- Interfaces in Java
  - Interface methods
  - Inheritance in Interfaces
- Packages in Java
  - API : The built-in Java Packages
  - Organization of user packages
  - Managing user packages
- Practice Questions
- Assignment
- Q&A

Introduction

There is no concept of multiple-inheritance in Java, but, Interfaces in Java are, for the most part, unique to the language, play a role similar to that of multiple-inheritance. Another unique feature in Java is Packages. Packages are containers for classes that are used to keep the class name space compartmentalized. By organizing the users classes into packages, their reusing to write other classes are easier. This Chapter is for getting a lesson of Interfaces and Packages.

Interfaces in Java

An interface in Java is essentially a special kind of class. Like classes, interfaces contain methods and variables; unlike classes, interfaces are always completely abstract. Let us first take a look at the general form of an interface definition.

```
‘‘interface’‘InterfaceName [ extends Name1, [ Name2, . . . .’‘]’‘{
                    [ Variable’‘declaration ; ]
                    [ Method declaration ; ]
          }
```

Thus, an interface is defined just like a class except the key word **interface** in place of class. Although member elements are present in an interface they are static and final; although methods are defined in an interface, the code to implement the method is not. While classes can inherit from only one super class but, interfaces are able to inherit from as many as interface one can provide.

After defining an interface, it is require to implement with classes which can be done with the following syntax :

```
‘‘class’‘ClassName [ extends SuperClassName ]
                    [ implements InterfaceName1 [, Interface Name2 ] )’‘{
                              class body
          }
```

Now, let us consider an example to illustrate the use of interface technique. See the **Figure 4.1** Here, **Teacher and Student** are two classes each of them having their own unique components. We want to define an interface, say, Resume which will include all the components both in Teacher and Student.

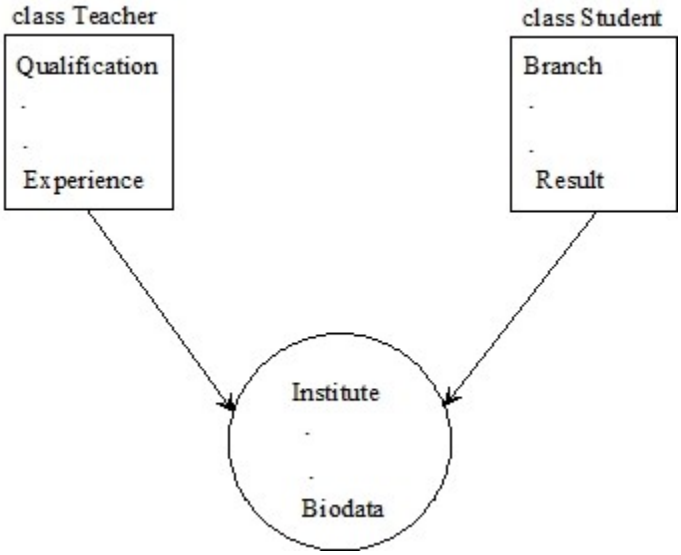


Figure 4.1 :Interface of classes

Following is the **Illustration 4.1** to snap shot the implementation of the above and its use.

Illustration 4.1

```
‘‘‘‘interface Resume {
          ‘‘‘‘��.
          ‘‘ void bioData (‘‘) ;                              // By default it is an abstract method.
                    ����                                ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘// when it is declared in an interface
          }


‘‘ class Teacher extends Employee implements Resume {
                    . . .’‘.                                  // Assume Employee is a class defined some
where
          ‘‘‘‘Qualification ;                        // A’‘member element
                    ��..
                    experience (‘‘)’‘{���}’‘     // A member function
                    ���.
                    public void bioData (‘‘) {
                              ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ // Code for this method for preparing the bio-data
of a teacher //
                    }
          }
```

```
‘‘‘‘class Student implements Resume {
          ‘‘‘‘��.
‘‘‘‘‘‘‘‘Discipline;                      // A member element.
‘‘‘‘‘‘‘‘���.
‘‘‘‘‘‘‘‘result (‘‘) { ��.}              // A member function
‘‘‘‘‘‘‘‘��..

‘‘‘‘public void bioData (‘‘) {
                                        // Code for’‘the method for preparing the bio-data of a
student //
                    }
          }
```

In this example, the interface Resume is declared which is implemented by two classes: Teacher and Students; Teacher is an inherited class from a class Employee (the definition of all classes are understood). With this definitions it should be noted that the method (s) which is / are in interface must be defined in their respective classes with which the interface is implemented. We can, therefore, run the following through a main class which extends the above defined interface, namely Resume.

```
Teacher x = new Teacher (‘‘) ;’‘          //’‘ X is an instance of Teacher
Student y = new Student (‘‘) ; //’‘Y is an instance of Student
Resume object ;                          //’‘ Object is reference to an instance of Resume.
object’‘= x;’‘                 ‘‘‘‘‘‘‘‘  //’‘ Object is now a teacher
object. bioData (‘‘);          ‘‘‘‘‘‘‘‘  //’‘ Obtain the bio-data of’‘a teacher X
object’‘= y ;                  ‘‘‘‘‘‘    //’‘ Object is now a student
object. bioData                ‘‘‘‘‘‘    //’‘ Obtain the bio-data of a student Y
```

Note that for **object.bioData( )**; *at line 5*, and **object.bioData ( )**; *at line 7 of the above piece of codes*, the respective codes for preparing bio-data of Teacher and Student will be in effect.

If the above mentioned example is a skeleton of how a method is sharable in two classes then following is the **Illustration 4.2** giving executable code how member elements are sharable.

Illustration 4.2

```
/*Creating an interface and using the same in a Java program*/

package myInterface;
```

```
‘‘
public interface anInterface {
‘‘‘‘
‘‘‘‘//final public static int a = 10; // public, static and final
‘‘‘‘int a = 10; // public, static and final
‘‘‘‘void display();’‘// public and abstract
}


import myInterface.*;
‘‘
// A class that implements interface.


class Demonstration_91 implements anInterface {
‘‘‘‘‘‘ public void display()’‘// Implementing the abstract method’‘‘‘
‘‘‘‘{
‘‘‘‘‘‘‘‘System.out.println("Fine!");
‘‘‘‘}
‘‘
‘‘‘‘// The main method()
‘‘‘‘public static void main (String[] args)’‘‘‘ {
‘‘‘‘‘‘‘‘Demonstration_91 t = new Demonstration_91();
‘‘‘‘‘‘‘‘t.display();
‘‘‘‘‘‘‘‘System.out.println("The final value a in myInterface "+ a);
‘‘‘‘}
}
```

Illustration 4.3

```
/* Interface, in many way, similar to a class; however, no object can be instantiated from an interface.
The following program is to illustrate this fact. */


interface C {
‘‘‘‘public static void int i = 555;
‘‘‘‘void printInterface();
}



//Attempting to create an object of interface
‘‘class Demonstration_92 {
```

```
‘‘‘‘‘‘‘public static void main (String[] args)’‘‘‘ {
‘‘‘‘‘‘‘‘‘‘‘‘‘C c = new C(); //ERROR: Object cannot be instantiated
‘‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘C c2; // OK: Declaration of object is possible

‘‘‘‘‘‘‘‘‘‘‘‘‘C c3[] = new C[]; // This is also OK: declaration of
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ // array of objects for an interface is permitted.
‘‘‘‘‘‘ }
‘‘}
```

Interface methods

Interface methods are those methods which are abstract methods appearing in the interface. Here, we will see, how they expect attention from the programmer. If a class implements an interface but does not implement one or more of interface's methods, the whole class becomes an abstract class and cannot be instantiated. Let us take a look at a brief example **(Illustration 4.4)** of this :

Illustration 4.4

```
interface I1 {
‘‘‘‘
‘‘‘‘public static final double PI = 3.1413434567;’‘ //OK
‘‘‘‘static final double lambda = 0.04; //OK: public is implicit

‘‘‘‘//int x;’‘//Declaration of any instance variable is NOT allowed.
          int x=100;// Initialization of variable is necessary.


          //private static final int p = 444;’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ //ERROR: private/ protected is NOT allowed.
          //abstract public static void methodI1(); // Static method is not allowed


          void methodI2(); //public abstract by default
}



class A1 implements I1 {
‘‘‘‘‘‘ public int a1 = 555;
‘‘‘‘‘‘ public void methodI1(){
‘‘‘‘‘‘‘‘‘ System.out.println("From I1 " + PI);
```

```
‘‘‘‘‘‘ }
‘‘‘‘‘‘ public void methodI2(){
‘‘‘‘‘‘‘‘‘‘System.out.println("Again from I1 " + lambda);
‘‘‘‘‘‘}
}

‘‘// Here, is the main class
public class Demonstration_93 {
          public static void main(String[] args) {
                    A1 a = new A1();
‘‘‘‘‘‘‘‘a.methodI1();
‘‘‘‘‘‘‘‘a.methodI2();
‘‘‘‘ }
}
```

Illustration 4.5

```
/* We have learnt that a class MUST implements an interface and then that
class will be used as a normal class. However, the reverse is not true.
That, is an interface cannot implement another interface */

interface I1 {
‘‘‘‘public static int i = 555;
‘‘‘‘void printInterface();
}

//The following code is invalid!

interface I2 implements I1 {
‘‘‘‘public static int j = 111;
‘‘‘‘public void printInterface(){
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.print("Geek!");
‘‘ }
}
```

Illustration 4.6

```
// We have learnt that a class MUST implements an interface and then that class will be used as a normal class. However, the reverse is not true. That is, an interface cannot implement another class (abstract or non-abstract */


/*OR
The following code that an interface implements a class is also invalid! */


//The following code is invalid!


abstract class’‘C {
‘‘‘‘public static int j = 111;
‘‘‘‘void print(){
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.print("Geek1!");
‘‘ }
}



interface I2 implements C {
‘‘‘‘public static int j = 222;
‘‘‘‘void print (){
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.print("Geek2!");
‘‘ }
}
```

Illustration 4.7

```
/* An interface method cannot be made final */


interface I1 {
‘‘‘‘public static int i = 555;
‘‘‘‘public final static void printInterface(); //ERROR
}


//The following code is invalid!


class C implements I1 {
‘‘‘‘public static int j = 111;
‘‘‘‘void printInterface(){
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.print("Geek!");
‘‘ }
```

```
}
```

Inheritance in Interfaces

Like classes, interfaces also have a chain of inheritance i.e. an interface can be derived from other interface. Consider the example, given below :

Illustration 4.8

```
/* An interface acts like a class.
That is, we can do everything as we do with class except the object instantiation. Here, is an example on the single inheritance with interface.
*/
interface I1 {
‘‘‘‘double x = 4.444;
          void methodI1(); //public static by default
}

interface I2 extends I1 {
‘‘‘‘double y = 5.555;
          void methodI2(); //public static by default
}


class A1 implements I2 {
‘‘‘‘‘‘ public int a1 = 555;
          ‘‘
‘‘‘‘‘‘ public void methodI1(){
‘‘‘‘‘‘‘‘‘‘ System.out.println("From I1"+x+y);
‘‘‘‘‘‘ }
‘‘‘‘‘‘ public void methodI2(){
‘‘‘‘‘‘‘‘‘‘System.out.println("From I2"+x+y);
‘‘‘‘‘‘}
}’‘


public class Demonstration_97 {
          public static void main(String[] args) {
          ‘‘A1 a = new A1();
          ‘‘a.methodI1();
```

```
          ‘‘a.methodI2();
‘‘‘‘ }
}
```

OUTPUT:
From I14.4445.555 From I24.4445.555

Illustration 4.9

```
/* An interface is a significant feature in Java in the sense that’‘it enables the multiple inheritance.
Here, is an example on the multiple inheritance with interface.
Case 1:’‘Example of class which "implements" two interfaces
*/

interface I1 {
‘‘‘‘int i = 123;
‘‘‘‘void printI1();
‘‘‘‘}



interface I2 {
‘‘‘‘public static int j = 555;
‘‘‘‘void printI2();
}

class A implements I1, I2{
‘‘‘‘ public int aValue = 999;
‘‘‘‘
‘‘‘‘ public void printI1() {
‘‘‘‘‘‘‘‘‘‘System.out.println("I am from I1 " + i);
‘‘‘‘ }
‘‘‘‘ public void printI2() {
‘‘‘‘‘‘‘‘ System.out.println("I am from I2 " + j);’‘‘‘
‘‘‘‘}
‘‘‘‘public void printA() {
```

```
‘‘‘‘‘‘‘‘ System.out.println("I am from A " + (aValue = i+j));’‘‘‘
‘‘‘‘}


}

class Demonstration_98 {
‘‘‘‘‘‘‘‘public static void main (String[] args)’‘‘‘ {
                    ‘‘A a = new A();
                    ‘‘a.printA();
                    ‘‘a.printI2();
                    ‘‘a.printI1();
‘‘‘‘}
}
```

OUTPUT:
I am from A 678 I am from I2 555 I am from I1 123

Illustration 4.10

```
/* Example on the multiple inheritance with interface.
Case 2:’‘Example of "extends" with "implements"
*/

class A {
‘‘‘‘protected int j = 1000;
‘‘‘‘void print() { System.out.println("I am from A "+j);
‘‘‘‘}
}

interface I {
‘‘‘‘public static int i = 555;
‘‘‘‘void printInterface();
}
```

```
class B extends A implements I{
‘‘‘‘ public int aValue = 999;
‘‘‘‘
‘‘‘‘ public void printInterface() {
‘‘‘‘‘‘‘‘‘System.out.println("I am from I " + i);
‘‘‘‘ }
‘‘‘‘ public void printB() {
‘‘‘‘‘‘‘‘ super.print();
‘‘‘‘‘‘‘‘ printInterface();
‘‘‘‘}
}


//Driver code in main class
public class Demonstration_99 {
‘‘‘‘‘‘‘‘public static void main (String[] args)’‘‘‘ {
                                ‘‘B b = new B();
                                ‘‘b.printB();
‘‘‘‘}
}
```

OUTPUT:
I am from A 1000 I am from I 555

Illustration 4.11

```
/* Example on the multiple inheritance with interface. */


/* Case 3:’‘Example of extends with implements two or more interfaces*/



class A {
‘‘‘‘protected int i = 1000;
‘‘‘‘public void print() { System.out.println("I am from A "+i);
‘‘‘‘}
}
```

```
interface C {
‘‘‘‘public static int j = 555;
‘‘‘‘void printInterfaceC();
}

interface D {
‘‘‘‘public static int k = 666;
‘‘‘‘void printInterfaceD();
}

class B extends A implements C, D{
‘‘‘‘ public int aValue = 999;
‘‘‘‘ public void printInterfaceC() {
‘‘‘‘‘‘‘‘‘‘System.out.println("I am from C " + j);
‘‘‘‘ }
‘‘‘‘ public void printInterfaceD() {
‘‘‘‘‘‘‘‘‘‘System.out.println("I am from D " + k);
‘‘‘‘ }
public void printB() {
‘‘‘‘‘‘‘‘ super.print();
‘‘‘‘‘‘‘‘ printInterfaceC();
‘‘‘‘‘‘‘‘ printInterfaceD();
‘‘‘‘}

}



class Demonstration_910 {
           public static void main (String[] args)’‘‘‘ {
                     ‘‘B b = new B();
                     ‘‘b.printB();
‘‘ }
}
```

OUTPUT:
I am from A 1000 I am from C 555

I am from D 666

Illustration 4.12

```
/* How an interface helps a programmer? A program demonstrating sharing variables like a library */

package myInterface;

public interface SharedConstants {
          ‘‘ int NO = 0;
          ‘‘ int YES = 1;
          ‘‘ int MAYBE = 2;
          ‘‘ int LATER = 3;
          ‘‘ int SOON = 4;
          ‘‘ int NEVER = 5;
}
import myInterface.*;

class Demonstration_911 implements SharedConstants {
‘‘‘‘static void answer(int result) {
‘‘‘‘‘‘‘‘switch(result) {
                    case NO:
          ‘‘‘‘                System.out.println("No");
          ‘‘‘‘                break;
‘‘‘‘‘‘‘‘‘‘case YES:
          ‘‘‘‘                System.out.println("Yes");
‘‘‘‘‘‘‘‘‘‘‘‘‘‘      break;
‘‘                  case MAYBE:
          ‘‘‘‘                System.out.println("Maybe");
          ‘‘‘‘                break;
                    case LATER:
          ‘‘‘‘                System.out.println("Later");
          ‘‘‘‘                break;
                    case SOON:
          ‘‘‘‘                System.out.println("Soon");
                              break;
                    case NEVER:
          ‘‘‘‘                System.out.println("Never");
          ‘‘‘‘                break;
```

```
                    }
‘‘‘‘‘‘ }
‘‘

public static void main(String args[]) {
          Question q = new Question();
          answer(q.ask());
          answer(q.ask());
          answer(q.ask());
          answer(q.ask());
          }
}
```

Illustration 4.13

```
/* Runtime polymorphism with interface objects
‘‘ GeoAnalyzer program implementation using interface */

package myInterface;

interface GeoAnalyzer {
          final static float pi = 3.142F;
          float area();
          float perimeter();
}

class Circle implements GeoAnalyzer {
          float radius;

          Circle(float r) {
                    radius = r;
          }

          public float area() {
                    return (pi * radius * radius);
          }

          public float perimeter() {
```

```
                    return (2 * pi * radius);
          }
}

class Ellipse implements GeoAnalyzer {
          float major;
          float minor;

          Ellipse(float m, float n) {
                    major = m;
                    minor = n;
          }

          public float area() {
                    return (pi * major * minor);
          }

          public float perimeter() {
                    return (pi * (major + minor));
          }
}
class Rectangle implements GeoAnalyzer {
          float length;
          float width;

          Rectangle(float l, float w) {
                    length = l;
‘‘‘‘‘‘‘‘width = w;
          }

          public float area() {
                    return (length * width);
          }

          public float perimeter() {
                    return (2 * (length + width));
          }
}

class Demonstration_92 {
          static void display(float x, float y) {
```

```
                    System.out.println("Area = " + x + "Perimeter = " + y);
          }

public static void main(String args[]) {
          Circle c = new Circle(5.2f);
          Ellipse e = new Ellipse(4.5f, 3.6f);
          Rectangle r = new Rectangle(6.5f, 4.3f);
          GeoAnalyzer geoItem;
          geoItem = c;
          display(geoItem.area(), geoItem.perimeter());
          geoItem = e;
          display(geoItem.area(), geoItem.perimeter());
          geoItem = r;
          display(geoItem.area(), geoItem.perimeter());
          }
}
```

OUTPUT:
Area = 84.95968Perimeter = 32.6768 Area = 50.9004Perimeter = 25.450201 Area = 27.95Perimeter = 21.6

Illustration 4.14

```
// Runtime polymorphism with abstract class
‘‘ GeoObjects program implementation using abstract class

// Abstract class versus Interface 

//An abstract class’‘and its sub class

package myInterface;

public abstract class Geometry {
‘‘public abstract double area ();
‘‘public abstract double circumference();
```

```
}
```

```
package myInterface;

import myInterface.*;


// Extending Geometry for Circle

class Circle extends Geometry {

‘‘‘‘public double r;

           protected static final double PI = 3.14159265358979323846;




           public Circle (){

           ‘‘‘‘‘‘‘‘ r=1.0

           }

           public Circle (double r){

                                this.r=r;

           }

           public double area() {

                                return PI*r*r;

           }

           public double circumference() {

                                return 2*PI*r;

           }

           public double getRadius() {

                                return r;

           }

}
```

```
class Rectangle extends Geometry {

‘‘‘‘protected double l,w;

‘‘‘‘public Rectangle() {

                    l = 0.0;

                    w = 0.0;

‘‘‘‘}



‘‘‘‘public Rectangle(double l, double w) {

                    this.l = l;

                    this.w = w;

‘‘‘‘}




          public double area() {

                    return l*w;

          }

          public double circumference() {

                    return 2*(l+w);

          }



          public double getwidth() {

                    return w;

          }



          public doub;le getlength() {

                    return l;

          }

}
```

```
package myInterface;

import myInterface.*;

class Ellipse extends Geometry {

‘‘‘‘protected double a,b;

‘‘‘‘public Ellipse() {

          a = 0.0;

          b = 0.0;

‘‘‘‘}



‘‘‘‘public Ellipse(double a,   double b) {

          this.a = a;

          this.b = b;

‘‘‘‘}




          public double area() {

                    return PI * a*b;

          }

          public double circumference() {

                    return PI*(a+b);

          }



          public double getMinorAxis() {

                    return a;

          }
```

```
            public double getMajorAxis() {
                        return b;
            }
}
```

```
package myInterface;
import myInterface.*;

public class GeoDemo {
            ‘‘ public static void main(String args[]) {

            // use the above class definition
            Geometry [] geoObjects = new Geometry[3]
            // create an array to hold Geometry objects
            geoObjects [0] = new Circle (2.0);
            geoObjects [1] = new Rectangle (1.0,3.0);
            geoObjects [2] = new Ellipse (4.0,2.0);

            double totalArea = 0;

            for (int i = 0; i < 3; i++) {
                        totalArea = totalArea + geoObjects[i].area();
                        }
                        Sytem.out.println(�Total area of the geo-objects is ► ► + totalArea);
}
```

Packages in Java

One of the main advantage of object oriented programming is its ability to reuse code by sub classing existing classes (that is by inheritance) where programmers have to define the difference between super class and sub class. Although this is an extremely powerful capability but its real benefit cannot be achieved unless one have the complete knowledge of library of classes. There are two problems occurs with library of classes : one is the problem in name collisions and the other is to control the visibility of classes, interfaces, methods, and the variables define within them. The solution to these problems is the use of systematic class libraries, or what Java refers to as packages.

Packages are collection of classes and interfaces, usually according to functionality, such as graphics, audio, text, or project. Here, we are to organize our classes and interface so that reusing these packages in other programs will be easier. Having a package organization, a programmer can refer this package by import statement as the first non-comment statement of a program. When a class is referred to in a program, the compiler will check all the imported packages to find out where the class is defined. If there is only one imported packages to find out where the class is defined, the definition of the class in that package will be used. If more than one imported package contains the definition of the named class, then there is ambiguity which can be resolved by specifying the exact reference (e.g. there is a class Abc located in both the package X and Y and we want to refer the Abc in Y, then its reference should be Y.Abc).

All classes and interfaces in a package are accessible to all other classes and interfaces in that package. All the member elements and method of a class can be accessed from any method of any class under the same package, except when the data and method are declared private. In the next few sections, we are to discuss about the handling and managing packages.

API : The built-in Java Packages

Most of the time, our primary dealings with packages provided with the Java release. The Java Development Kit contains over 15 packages that we can use to obtain classes for use in our programs. All these built-in packages are known as the core API (Application Programming Interface). This standard set of libraries are extremely useful for writing Java programs and evolved over several years of writing Java code to implement a variety of systems, ranging from consumer device networks to animated user interfaces to operating systems to compiler. Many people, both from inside and outside Sun, have been involved in the design of the API.

The API contains the following packages :

```
Core packages
          java.lang : The Java language Package
          java.io :'' The Java''I/O Package
          java.util'' : The Java Utility Package
          java.net'' : The Java Networking Package


Window Toolkit and applet
          java.awt   : Abstract Window Toolkit (AWT) package
          java.awt. image      : The AWT Image package
          java.awt. peer''''   : The AWT peer package
          java.applet''''''    : The Java Applet package
```

Let us take an overview of the API.

```
java. lang : The Java language package

''''The java.lang package provides the classes and interfaces that form the core of the Java language and
the Java Virtual Machine. For example, the classes Object, String, and Thread are used by almost every
program and are closely inter wined with the Java language definition. Other classes define the Exceptions
and Errors that the Java Virtual Machine can throw. Another set of java.lang classes provide wrappers for
primitive types. For example, the Integer class provides object to''contain int values.

''''Note :The java.lang package is imported automatically into''every Java program.

java. io : The Java I/O package
```

‘‘‘‘The java.io package provides a set of input and output (I/O) streams used to read and write data to files or other I/O sources. Java streams are byte oriented and the classes defined here can be chained to implement more’‘sophisticated stream functionality.’‘

java util : The Java’‘utility package

‘‘‘‘ The java.util package contains a collection of utility classes and related interfaces. It includes classes that provide generic data structures (Dictionary, Hashtable, Stack, Vectors), String’‘manipulation (String Tokekenizer), Calender’‘and date utilities (Date). The java.util package also contains the Observer interface and’‘Observable class, which allow objects to notify one another when they change.

java net : The Java Networking package

‘‘‘‘ The java.net’‘package contains networking classes and interfaces, including classes that represent a URL and URL’‘connection, classes that implement a socket connection, and a class that represent an Internet address.

java .awt : The Abstract Window Tool kit package

‘‘‘‘ The java.awt package provides the standard graphical user interface (GUI) elements’‘such as buttons, lists, menus, and text areas. It also includes containers (such as windows and menu bars) and higher-level components(such as dialogs for opening and saving files). The’‘AWT contains two more package java.awt.image and java.awt.peer.

java.awt.image : The AWT Image package

‘‘‘‘ The java.awt.image package contains classes and interfaces for performing sophisticated image processing. These classes and interfaces’‘can be used by applications that need to create or manipulate images and colors.

java.awt.peer : The AWT Peer package

‘‘‘‘ The java.awt.peer package contains interfaces used to connect AWT components to their window system-specific implementation’‘such as Motif widgets’‘). Unless you are creating a window system specific implementation’‘of the AWT, you should not need to use the interfaces in the java.awt.peer package.

java.applet : The Applet package

‘‘‘‘The java.applet package contains classes and interfaces for creating applets.

The Java API is generally consulted by the advance Java programmer, and this Part is to give the Java programming for beginners. The complete API documentation is presented in the Part III of this book.

Using API Packages :

After a stock of the API, we will see how they can be used in Java programs. Suppose, in the package java.util, there is a class called Date. If you want to use this class in order to instantiate an object from that you should use the fully qualified class name; a code fragment is given :

```
‘‘ java.util.Date toDay = new java.util.Date( );
          System.out.print (toDay);
```

One can pull a class or can pull all the classes from a package. Java provides the import keyword to allow one to import specific classes or entire packages. The import key word is used as follows :

```
import PackageName.ClassName ;‘‘‘‘// To get access of a’‘specific class ClassName
import PackageName.* ;                          // To get access the enter package PackageName
```

By importing this way, we no longer have to use its fully qualified ClassName to use it. An example is given below :

```
          ‘‘import java.util.Date;                              // or import java.util.*;
                    Date toDay = new Date( );
                    System.out.print (toDay);
```

Illustration 4.15

```
// Accessing a package from a Java built-in APIs

import java.lang.*;’‘//Math class is defined in this package

‘‘ class Calculator{
‘‘‘‘‘‘ double i;
‘‘‘‘‘‘ double x ;
          ‘‘ void p(){
                    ‘‘ x = Math.sqrt(i);
          ‘‘ }
          ‘‘
‘‘ }

‘‘ class Demonstration_81{
‘‘‘‘ public static void main(String args[]){
          ‘‘ Calculator a = new Calculator();
          ‘‘ a.i = 20.0;
          ‘‘ a.p();
          ‘‘ System.out.println("Square root of "+a.i+" is "+a.x);
‘‘‘‘ }
}
```

OUTPUT:
Square root of 20.0 is 4.47213595499958

Illustration 4.16

```
/* Java program to demonstrate accessing of members when it is imported */

import java.util.Vector;
‘‘
class Demonstration_82 {
‘‘ public Demonstration_82()’‘‘‘{
‘‘‘‘‘‘// java.util.Vector is imported, hence we are
‘‘‘‘‘‘// able to access directly in our code.
‘‘‘‘‘‘Vector newVector = new Vector();
‘‘‘‘‘‘// The code dealing with the object newVector..’‘
‘‘ }
‘‘
‘‘ public static void main(String arg[])
‘‘ {
‘‘‘‘‘‘//Here is the code in the main class
‘‘ }
}
```

Illustration 4.17

```
/* Java program to demonstrate accessing of members when it is not imported explicitly*/
‘‘
public class Demonstration_83 {
‘‘ public Demonstration_83()’‘‘‘{‘‘
‘‘‘‘‘‘java.util.ArrayList newList = new java.util.ArrayList();
‘‘‘‘‘‘
‘‘‘‘‘‘// java.util.ArrayList is not imported, hence we referring
‘‘‘‘‘‘// to it using the complete’‘package.

‘‘‘‘‘‘// The code dealing with the object newList..’‘

‘‘ }
‘‘
‘‘ public static void main(String arg[])’‘ {
‘‘‘‘‘‘//Here is the code in the main class
‘‘ }
}
```

Illustration 4.18

```
/* Importing an entire package, that is, all classes in a package */

import java.util.*;
‘‘
public class Demonstration_84 {
‘‘ public Demonstration_84()’‘‘‘{
‘‘‘‘‘‘Vector newVector = new Vector();
‘‘‘‘‘‘// java.util.Vector is imported, hence we are
‘‘‘‘‘‘// able to access directly in our code.

‘‘‘‘‘‘ArrayList newList = new ArrayList();
‘‘
‘‘‘‘‘‘// java.util.ArrayList is imported, hence’‘
‘‘‘‘‘‘// we are able to accees it directly in the program
‘‘‘‘‘‘// package.’‘
‘‘ }
‘‘
‘‘ public static void main(String arg[])
‘‘ {
‘‘‘‘‘‘// The code dealing with the object newVector and newList..’‘

‘‘ }
}
```

Organization of user packages

Till this point we have discussed about packages from the system packages. Now, a user can develop several classes and then wish to share with different programs. To do this, user has to construct his own package. Let us consider an example for this. Suppose, we want to create a simple package which will contain only one class, say MyClass. Let the name of the package be MyPackage. **Illustration 4.19** is for this purpose.

Illustration 4.19 **// User defined Package //**

```
/*Creating a user's own package */


/*Save the following code as myClass.java in a sub-directory, say myPackage. Compile the .java file and
save the .class file in the same sub-directory */
```

```
‘‘
package myPackage;


public class myClass {‘‘
‘‘public void msg(){System.out.println("Class: Hello! ");}’‘
}
```

This listing would be saved in a file called MyClass.java and would be located in directory named MyPackage. The class then should be compiled and compiled version (i.e MyClass.class) would be stored in the same directory as in source file (i.e. MyPackage). Now, let us finish the example by looking at a simple program that will import the class MyClass from the package MyPackage:

```
/*Save the following code as Demonstration_85.java in a sub-directory, say Demonstration_85 */


import myPackage.myClass;’‘
‘‘
class Demonstration_85{‘‘
          public static void main(String args[]){‘‘
                    myClass obj = new myClass();’‘
                    obj.msg();’‘
          }’‘
}
```

Illustration 4.20

```
/*Creating a user’s own package */


/*Save the following code as myClass1.java in a sub-directory, say myPackage. Compile the .java file and
save the .class file in the same sub-directory */
‘‘
package myPackage;


public class myClass1 {‘‘
‘‘public void msg(){
‘‘System.out.println("muClass1: Hello! ");
‘‘}’‘
}’‘
```

```
/*Save the following code as myClass2.java in the same sub-directory myPackage. Compile the .java file and save the .class file in the same sub-directory */
‘‘
package myPackage;


public class myClass2 {‘‘
‘‘public void msg(){
‘‘System.out.println("muClass2: Hello! ");
‘‘}’‘
}’‘

/*Save the following code as Demonstration_86.java in a sub-directory, say Demonstration-VIII */



import myPackage.*;’‘
‘‘
class Demonstration_86{‘‘
‘‘public static void main(String args[]){‘‘
‘‘ myClass1 obj1 = new myClass1();’‘
‘‘ obj1.msg();

‘‘ myClass2 obj2 = new myClass2();’‘
‘‘ obj2.msg();’‘


‘‘}’‘
}
```

Illustration 4.21

```
// Careful about default access specification. Check the following

package myPackage;’‘‘‘// Creating a package with class myClass1


class myClass1 {‘‘
‘‘public void msg(){
```

```
‘‘System.out.println("muClass1: Hello! ");
‘‘}’‘
}’‘

package myPackage;’‘ // Creating a package with class myClass2

class myClass2 {‘‘
‘‘public void msg(){
‘‘System.out.println("muClass2: Hello! ");
‘‘}’‘
}’‘

import myPackage.*;’‘
‘‘
class Demonstration_87{‘‘
‘‘public static void main(String args[]){‘‘
‘‘‘‘‘‘myClass1 obj1 = new myClass1();’‘
‘‘‘‘‘‘obj1.msg();
‘‘‘‘‘‘myClass2 obj2 = new myClass2();’‘
‘‘‘‘‘‘obj2.msg();’‘
‘‘}’‘
}
```

Illustration 4.22

```
//Example of package by import package.classname
//Save the class as A88.java’‘

package myPackage;

public class A88{‘‘
‘‘public void msg(){
‘‘System.out.println("Class A88: Hello");
‘‘}’‘
}’‘

//Save the following Java program as’‘Demonstration_88.java
```

```
import myPackage.A88;’‘
‘‘
class Demonstration_88{‘‘
‘‘public static void main(String args[]){‘‘
‘‘ A88 obj = new A88 ();’‘
‘‘ obj.msg();’‘
‘‘}’‘
}
```

Illustration 4.23

```
// Example of package by import package.classname
//Save the class as A88.java’‘

package myPackage;

public class A88{‘‘
‘‘public void msg(){
‘‘System.out.println("Class A88: Hello");
‘‘}’‘
}’‘

//Save the following Java program as’‘Demonstration_89.java

import myPackage.A88;’‘
‘‘
class Demonstration_89{‘‘
‘‘public static void main(String args[]){‘‘
‘‘ myPackage.A88 obj = new myPackage.A88 ();//using the fully qualified name’‘‘‘
```

Illustration 4.24

```
/* A simple package: Utilization of package in any java application */

package myPackage;’‘‘‘ // Adding another class into myPackage

public class Balance {
String name;
double bal;

Balance(String n, double b) {
          name = n;
          bal = b;
}

void show() {
          if (bal < 0) {
          System.out.print("Sorry! Negative balance ");
          }
          System.out.println(name + ": $" + bal);
          }
}


// Run the following java program from Demonstration-VIII directory

import myPackage.*;

class Demonstration_810{
public static void main(String args[]) {
          Balance current[] = new Balance[3];
          current[0] = new Balance("D. Samanta", 123.23);
          current[1] = new Balance("T. Ahmed", 157.02);
          current[2] = new Balance("N. Sinhababu", -12.33);

          for (int i = 0; i < 3; i++) {
          current[i].show();
‘‘‘‘ }
‘‘ }
}
```

Illustration 4.25

```
/* Can the same class name but in two different packages and the classes are then referred in a java program? */



package myPackage1;‘‘ // Creating a package myPackage1


 public class myClass {
‘‘‘‘public void msg1() {
‘‘‘‘System.out.println("Hello! It is myPackage1 Class!! ");
‘‘‘‘}
}


package myPackage2;‘‘ // Creating a package myPackage2


 public class myClass {
‘‘‘‘public void msg2() {
‘‘‘‘‘‘‘‘System.out.println("Hello! It is myPackage2 Class!! ");
‘‘‘‘}
}


import myPackage1.myClass;
import myPackage2.myClass;
‘‘
public class Demonstration_811{
‘‘‘‘public static void main(String[] args){
‘‘‘‘‘‘‘‘myPackage1.myClass a = new myPackage1.myClass();
‘‘‘‘‘‘‘‘myPackage2.myClass b = new myPackage2.myClass ();
‘‘‘‘‘‘‘‘a.msg1();
‘‘‘‘‘‘‘‘b.msg2();
‘‘‘‘}
}
```

Illustration 4.26

```
/* Can we use a class in a package to derive a sub-class? */

package myPackage;’‘‘‘ // A class in a myPackage

public class Balance {
          String name;
          double bal;

Balance(String n, double b) {
          name = n;
          bal = b;
}

void show() {
          if (bal < 0) {
          System.out.print("Sorry! Negative balance ");
          }
          System.out.println(name + ": $" + bal);
          }
}
/* Derive a sub-class in a program to be run from Demonstration-VIII directory */

import myPackage.Balance;

public class Savings extends Balance{
‘‘‘‘ String branch;
‘‘‘‘ Int customerID;
‘‘‘‘ Savings(String n, double bal, String br, int id) {
‘‘‘‘‘‘‘‘ Balance(n, bal);
‘‘‘‘‘‘‘‘ Branch = br;
‘‘‘‘‘‘‘‘ customerID = id;
‘‘‘‘ }

void show() {
          if (bal < 0) {
          System.out.print("Sorry! Negative balance ");
          }
          System.out.println(name + ": $" + bal + ☐ ☐+ branch + ☐ ☐+id);
}
```

```
class Demonstration_810{
          public static void main(String args[]) {
          Savings current[] = new Savings[3];
          current[0] = new Balance("D. Samanta", 123.23, ?Kolkata?, 555);
          current[1] = new Balance("T. Ahmed", 157.02, ?Chennai?, 420);
          current[2] = new Balance("N. Sinhababu", -12.33, ?Mumbai? + 999);

          for (int i = 0; i < 3; i++) {
          current[i].show();
          }
‘‘}
}
```

Illustration 4.27

```
/* This program shows all combinations of the access control modifiers. In this program, we define two
packages and five classes.

MyPackage1:
          class X
          class Y’‘extends X
          class A
MyPackage2:
          class Z extends X
          class B
*/
// Defining package MyPackage1

package MyPackage1;

public class X {
          int n = 1;
          private int p = 2;
          protected int q = 3;
          public int r = 4;
          // A constructor of the class protection
          public X() {
```

```
                    system.out.println("I am constructor from class X:");
                    system.out.println("n="+n);
                    system.out.println("p="+p);
                    system.out.println("q="+q);
                    system.out.println("r="+r);
          }
}
//Save this as X.java in Mypackage1 directory



package MyPackage1;


class Y extends X {
          Y() {
          system.out.println("I am constructor from class Y:");
          system.out.println("n="+n);
          system.out.println("p="+p); // Error p is a private
                                                ‘‘// member of X. Not accessible outside X.
          system.out.println("q="+q);    // Protected is accessible
          system.out.println("r="+r);    // public is accessible
          }
}
//Save this as Y.java in Mypackage1 directory




//Save this as A.java in Mypackage1 directory

class A {  ‘‘‘‘‘‘‘‘// class with default protection
          A() {‘‘   ‘‘‘‘‘‘‘‘// default constructor with default access
          X x = new X();’‘‘‘// create an object of class X
          System.out.println("Same package constructor ....");
          System.out.println("n from A"+x.n);’‘
          // Default variable is accessible in the same package
          System.out.println("p from A"+x.p);       // Error
          System.out.println("q from A"+x.q);       //Error
           System.out.println("r from A"+x.r); // OK: public
          }
}
//Save this as Z.java in Mypackage2 directory
```

```
‘‘ package MyPackage1;
‘‘ class Z extends MyPackage1.X {
‘‘‘‘‘‘Z() {
                    System.out.println("I am constructor from class Z:");
                    System.out.println("n from Z"+n); // Error:
                    // Default is not accessible outside its package.
                    System.out.println("p from Z"+p); // Error : private of X     ‘‘
                    System.out.println("q from Z"+q);
                    // Protected member is accessible by inheritance
                    System.out.println("r from Z"+r);
                    // public is accessible
          }
‘‘‘‘ }



//Save this as B.java in Mypackage2 directory

class B {  ‘‘‘‘// class with default protection
           B() {‘‘‘‘ // default constructor with default access
           MyPackage1.X x = new MyPackage1.X();’‘
           // create an object of class X
           System.out.println("I am constructor from class B of MyPackage2");
           System.out.println("n from B of myPackage2"+x.n);
           // default variable but is not accessible in this package
           System.out.println("p from B of myPackage2"+x.p);
           // Error
           System.out.println("q from B of myPackage2"+x.q);
           // Error protection
           System.out.println("r from A of myPackage2"+x.r);
          }
}


/* Finally, run the above0defined classes from any working directory*/

//This is the demo of MyPackage1

import MyPackage1.*;

public class Demo1{
```

```
          public static void main(String args[])
          ‘‘X x1 = new X();
          ‘‘Y y1 = new Y();
          ‘‘A a1 = new A();
}
//This is the demo of MyPackage2

import MyPackage2.*;
public class Demo2{
          public static void main(String args[])
          ‘‘Z z2 = new Z();
          ‘‘B b2 = new B();
}
```

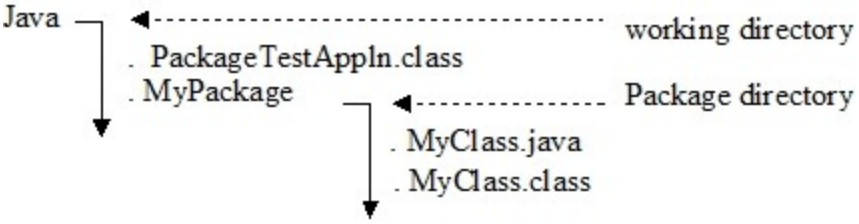
Managing user packages

Packages solves many problems from an access control and name space collision perspective. However, it does cause some curious difficulties when you are compiling and running your programs. Compiling your program to make use of your own packages can be slightly more difficult then using the standard packages. The Java compiler automatically knows where to look for the Java Packages. But for user packages, it is the responsibility of the user to specify it and hence a source of complexities. Three cares should be taken in order to avoid the complexities :

ð ’‘Exact location of the package should be mentioned in package statement. One can specify the directory hierarchy also, e.g.

```
package Project.Fractal.Geo;
```

‘‘This means that the class (es) is (/ are) stored in project / fractal / geo directory.
ð ’‘The directory in which package is stored should be under the directory where Java applications want to share the package classes. For example, for Illustration 4.7, directory hierarchy appear as :



ð ’‘By setting the system variable CLASSPATH. The specific location that the Java compiler will consider as the root of any package hierarchy can be controlled by CLASSPATH.

e.g. SET CLASSPATH = Java \ classes ; Java \ project \ packages;
‘‘with this then compiler will be enforced to resolve a package name for an import statement.

Practice Questions

Practice 4.1

```
/* A simple package */

          package MyPack;

          class Balance {
          String name;
          double bal;

          Balance(String n, double b) {
          name = n;
          bal = b;
          }

          void show() {
          if (bal < 0) {
          System.out.print("--> ");
          }
          System.out.println(name + ": $" + bal);
          }
          }

          class AccountBalance {
          public static void main(String args[]) {
          Balance current[] = new Balance[3];
          current[0] = new Balance("K. J. Fielding", 123.23);
          current[1] = new Balance("Will Tell", 157.02);
          current[2] = new Balance("Tom Jackson", -12.33);
          for (int i = 0; i < 3; i++) {
          current[i].show();
          }
          }
          }
```

what is the output?

Practice 4.2

```
/* An'‘example of interface in Java */

          interface GeoAnalyzer {
          final static float pi = 3.142F;
          float area();
          float perimeter();
```

```
}

class Circle implements GeoAnalyzer {
float radius;

Circle(float r) {
radius = r;
}

public float area() {
return (pi * radius * radius);
}

public float perimeter() {
return (2 * pi * radius);
}
}

class Ellipse implements GeoAnalyzer {
float major;
float minor;

Ellipse(float m, float n) {
major = m;
minor = n;
}

public float area() {
return (pi * major * minor);
}

public float perimeter() {
return (pi * (major + minor));
}
}

class Rectangle implements GeoAnalyzer {
float length;
float width;

Rectangle(float l, float w) {
```

```
            length = l;
            width = w;
            }

            public float area() {
            return (length * width);
            }

            public float perimeter() {
            return (2 * (length + width));
            }
            }

            class Geometry {
            static void display(float x, float y) {
            System.out.println("Area = " + x + "Perimeter = " + y);
            }

            public static void main(String args[]) {
            Circle c = new Circle(5.2f);
            Ellipse e = new Ellipse(4.5f, 3.6f);
            Rectangle r = new Rectangle(6.5f, 4.3f);
            GeoAnalyzer geoItem;
            geoItem = c;
            display(geoItem.area(), geoItem.perimeter());
            geoItem = e;
            display(geoItem.area(), geoItem.perimeter());
            geoItem = r;
            display(geoItem.area(), geoItem.perimeter());
            }
            }
```

what is the output?

Practice 4.3

```
            /* Interface with sharing member elements */

            import java.util.*;

            interface SharedConstants {
            int NO = 0;
            int YES = 1;
```

```
int MAYBE = 2;
int LATER = 3;
int SOON = 4;
int NEVER = 5;
}

class Question implements SharedConstants {
Random rand = new Random();

int ask() {
int prob = (int) (100 * rand.nextDouble());
if (prob < 30) {
return NO; // 30%
} else if (prob < 60) {
return YES; // 30%
} else if (prob < 75) {
return LATER; // 15%
} else if (prob < 98) {
return SOON; // 13%
} else {
return NEVER; // 2%
}
}
}

class AskMe implements SharedConstants {
static void answer(int result) {
switch (result) {
case NO:
System.out.println("No");
break;
case YES:
System.out.println("Yes");
break;
case MAYBE:
System.out.println("Maybe");
break;
case LATER:
System.out.println("Later");
break;
case SOON:
```

```
            System.out.println("Soon");
            break;
            case NEVER:
            System.out.println("Never");
            break;
            }
            }

            public static void main(String args[]) {
            Question q = new Question();
            answer(q.ask());
            answer(q.ask());
            answer(q.ask());
            answer(q.ask());
            }
            }
```

what is the output?

Practice 4.4

```
/* Inheritance in interface */

            interface I1 {
            void methodI1(); //public static by default
            }

            interface I2 extends I1 {
            void methodI2(); //public static by default
            }



            class A1 {
            public String methodA1() {
            String strA1 = "I am in methodC1 of class A1";
            return strA1;
            }

            public String toString() {
            return "toString() method of class A1";
            }
            }
```

```
class B1 extends A1 implements I2 {
public void methodI1() {
System.out.println("I am in methodI1 of class B1");
}
public void methodI2() {
System.out.println("I am in methodI2 of class B1");
}
}

class C1 implements I2 {
public void methodI1() {
System.out.println("I am in methodI1 of class C1");
}

public void methodI2() {
System.out.println("I am in methodI2 of class C1");
}
}

/* Note that the class is declared as abstract as it does not satisfy the interface contract */
abstract class D1 implements I2 {
public void methodI1() {
}
//This class does not implement methodI2() hence declared abstract.
}

public class InterFaceEx {
public static void main(String[] args) {
I1 i1 = new B1();
i1.methodI1();                              //OK as methodI1 is present in B1
// i1.methodI2(); Compilation error as methodI2 not present in I1
// Casting to convert the type of the reference from type I1 to type I2
((I2) i1).methodI2();
I2 i2 = new B1();
i2.methodI1(); //OK
i2.methodI2(); //OK
// Does not Compile as methodA1() not present in interface reference I1
// String var = i1.methodA1();
//Hence I1 requires a cast to invoke methodA1
String var2 = ((A1) i1).methodA1();
System.out.println("var2 : " + var2);
```

```
            String var3 = ((B1) i1).methodA1();
            System.out.println("var3 : " + var3);
            String var4 = i1.toString();
            System.out.println("var4 : " + var4);
            String var5 = i2.toString();
            System.out.println("var5 : " + var5);
            I1 i3 = new C1();
            String var6 = i3.toString();
            System.out.println("var6 : " + var6);     //It prints the Object
            // toString() method
            Object o1 = new B1();
            // o1.methodI1(); does not compile as Object class
            //does not define methodI1()
            //To solve the probelm we need to downcast o1 reference.
            // We can do it in the following 4 ways
            ((I1) o1).methodI1(); //1
            ((I2) o1).methodI1(); //2
            ((B1) o1).methodI1(); //3
            /* B1 does not have any relationship with C1 except they are "siblings".
            Well, you can't cast siblings into one another. */
            // ((C1)o1).methodI1(); Produces a ClassCastException
            }
            }
```

what is the output?

Assignment

Q: Define two different classes namely, Student and Employee. These classes are derived from a base class Person. Define other two classes Staff and Faculty. Staff and Faculty classes are derived from Employee class. The Person class has name and age data and display method to display the name and age of a person. The Student class has data like rollNo and branch and display method to display name, age, rollNo and branch of the student. Stuff has ecNo and doj(date of joining) data and display method to display name, age, ecNo, doj of the stuff. Faculty has designation data (Assistant Professor, Associate Professor and Professor) and display method to display the name, age, ecNo, doj and designation of the Faculty. Staff has designation data (Technical and Clerical) and display method to display the name, age, ecNo, doj and designation of the Staff. Each class have their own constructor to initialize the value of each data field. Finally create MainDemoClass and create an object of each class. Print the values of all objects in the MainDemoClass.

Q&A

Q: What is an Interface?

A: An interface is a description of a set of methods that conforming implementing classes must have.

Note:

- *You can't mark an interface as final.*
- *Interface variables must be static.*
- *An Interface cannot extend anything but another interfaces.*

Q: Can we instantiate an interface?

A: You can't instantiate an interface directly, but you can instantiate a class that implements an interface.

Q: Can we create an object for an interface?

A: Yes, it is always necessary to create an object implementation for an interface. Interfaces cannot be instantiated in their own right, so you must write a class that implements the interface and fulfill all the methods defined in it.

Q: Do interfaces have member variables?

A: Interfaces may have member variables, but these are implicitly public, static, and final- in other words, interfaces can declare only constants, not instance variables that are available to all implementations and may be used as key references for method arguments for example.

Q: What modifiers are allowed for methods in an Interface?

A: Only public and abstract modifiers are allowed for methods in interfaces.

Q: When should I use abstract classes and when should I use interfaces?

A: Use Interfaces when…

- You see that something in your design will change frequently.
- If various implementations only share method signatures then it is better to use Interfaces.
- you need some classes to use some methods which you don't want to be included in the class, then you go for the interface, which makes it easy to just implement and make use of the methods defined in the interface.

Use Abstract Class when…

- If various implementations are of the same kind and use common behavior or status then abstract class is better to use.
- When you want to provide a generalized form of abstraction and leave the implementation task with the inheriting subclass.
- Abstract classes are an excellent way to create planned inheritance hierarchies. They're also a good choice for nonleaf classes in class hierarchies.

Q: When you declare a method as abstract, can other nonabstract methods access it?

A: Yes, other nonabstract methods can access a method that you declare as abstract.

Q: Can there be an abstract class with no abstract methods in it?
A: Yes, there can be an abstract class without abstract methods.
Q: Can I import same package/class twice? Will the JVM load the package twice at runtime?
A: One can import the same package or same class multiple times. Neither compiler nor JVM complains wil complain about it. And the JVM will internally load the class only once no matter how many times you import the same class.
Q: Do we need to import java.lang.package ?
A: No, It is loaded by default by the JVM.
Q: Name few classes of package java.lang
A: [Throwable, Error, Object, Exception, IllegalArgumentException]
Q: What Restrictions Are Placed On The Location Of A Package Statement Within A Source Code File?
A: A package statement must appear as the first line in a source code file (excluding blank lines and comments).
Q: Explain the usage of Java packages.
A: A Java package is a naming context for classes and interfaces. A package is used to create a separate name space for groups of classes and interfaces. Packages are also used to organize related classes and interfaces into a single API unit and to control accessibility to these classes and interfaces.
Q: Why do we need to use package in Java?
A: Package provides encapsulation in Java program. Default access modifier for any variable or class is package-private i.e. they are only visible into package, on which they are declared. By using package you Encapsulate whole functionality which allows you to change the functionality, include new functionality or just change the implementation without breaking whole application. Though package is not the highest degree of Encapsulation in Java which is achieved using 'private' keyword, it is still the second best option and a must in order to encapsulate whole functionality rather than just a class.
Q: How to create a package in Java?
A: If you are using IDE like 'Eclipse' for developing your Java program then you don't need to do anything. Just click on new-->package and Eclipse will ask you name of the package, put name of the package and you are good to go. Now if you want to create Java class on that package, just select the package in package explorer and create new-->Java Class. If you are not using any IDE than you manually need to create directories corresponding to package in Java.

Chapter 5

Exception Handling in Java

by Debasis Samanta

CONTENTS

- Introduction
- Built-in classes for exceptions handling in Java
- Mechanism of Exceptions Handling in Java
- Error Handling Exception Classes
- Practice Questions
- Assignment
- Q&A

Introduction

This Chapter discusses exceptions handling mechanism in Java. An exception is an abnormal condition that can occur during the execution time of a program. If these exceptions are not prevented or at least handled properly, either the program will be aborted abnormally, or the incorrect result will be carried on. In traditional programming languages like C, Pascal etc. this exception handling is an overhead of the programmer to make the program robust by including lot of if statement and error-handler routine which make the programs more complicated. Java programmer are released from this overhead by the exception handling mechanism in Java.

Built-in classes for exceptions handling in Java

To handle the common possible exceptions ,Java defined a class hierarchy as shown below :

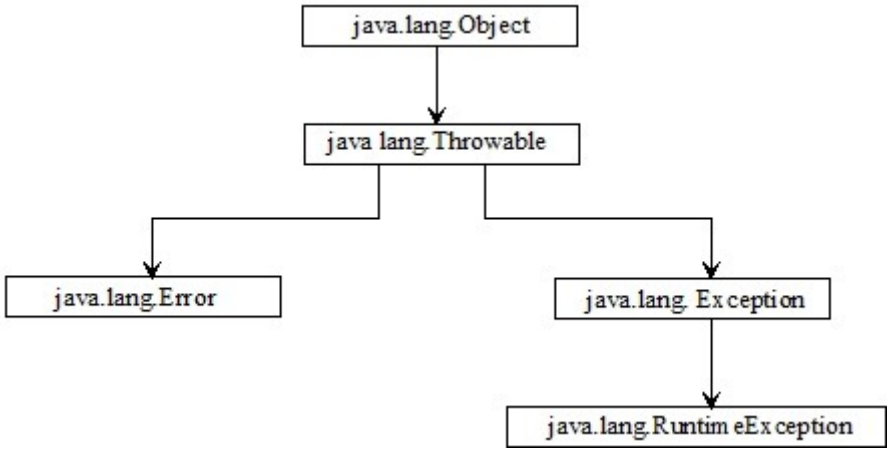


Figure 5.1 :Exception Class hierarchy in Java

Here, the class Throwable is used to represent all exceptional conditions. Two immediate subclasses of Throwable are Exception, and Error. The class Exception is used for exceptional conditions that user programs can catch. The other branch of the throwable tree is the class Error, which defines the conditions that should not be expected to be caught under normal circumstances. These class is responsible for giving errors in some catastrophic failures. A further refinement is there by a sub class of Exception, which is for exceptional condition that created by the run time called RuntimeException. These exceptions are typically created automatically during the run time in response to some execution error. A list of exceptions that a programmer can catch in the program is summarized below :

RuntimeException sub classes :	**Error sub classes :**
ArithmeticException	ClassCirculatoryError

```
ArrayIndexOutofBoundException                                   ClassFormatError
ArrayStoreException                                             Error
ClassCasteException                                             IllegalAccessError
IlegalArgumentException                                         IncompatibleClassChangeError
IndexOutofBoundException                                        InstantiationError
NegativeArraySizeException                                      LinkageError
NullPointerException                                                    NoCassDefFoundError
NumberFormatException                                                   NoSuchFieldError
SecurityException                                               NoSuchMethodError
StringIndexOutofBoundException                                  OutofMemoryError
‘‘‘
```

```
          }
          finally {
                    // Program code of exit (optional)
          }
```

This structure implements that, when you try to execute a block of code, and if an error occurs, you may catch based on what type of exception it is, or finally dealt with by a default handler.

It is better to illustrate the mechanism with few simple examples.

Illustration 5.1 **// simple example of Exception handling //**

```
          Consider the following code in Java :

          class DivideZero {
                    static int anyFunction ( int x, int y ) {
                              int a = x/y;
                              return (a);
                    }
                    public'‘static void main (String args [‘‘] ) {
                    int result = anyFunction (25, 0) ;                  // Exception occurs here as y = 0
                    System.out.println ( " Result : " + result );
‘‘‘‘‘‘‘‘}
‘‘‘‘}
```

This is a code where an exception will occur in this example (when the value of the second argument is passed as 0), we have not coded an exception handler; but Java provides a default run time handler. In this case, when the Java run time tries to execute the division, it notices that the denominator is zero and then instantiate an Exception object (namely ArithmeticException) to cause this code to stop and deal with this error condition. The default handler prints out the exception message. One can easily see it if this program is run. Following output is expected :

C:\> java DivideZero // To run the Application DivideZero

```
          One can notice the output then :

                    Java . lang . Arithmetic Exception : / by zero
                    at DivideZero.Any Function (DivideZero.Java : 3)
                    at DivideZero.main (DivideZero.Java : 7)
```

Note : Note that how default run time handler can print where is the source and what kind of exception it is.

Illustration 5.2

```
/* Showing compile time errors in a program. */

Class Error {
‘‘‘‘ Public static void main (string args [ ]) {
‘‘‘‘‘‘‘‘ system.out.print("Can you find errors in me?")
‘‘‘‘}
}

class AnotherError {
‘‘‘‘public void insert( ){
‘‘‘‘‘‘‘‘ System.out.print("To insert a text");
‘‘‘‘}

‘‘‘‘abstract void delete( ){
‘‘‘‘‘‘‘‘‘‘System.out.print("To delete a text");
‘‘‘‘ }

}
/*Note:
A class file can be compiled successfully, if it is syntactically correct, even there is no main class,
that is, with main method.
*/
```

It is observed that Java's default run time handler displays the detail of an exception and execution suspended as soon as an error encountered, but it is often more desirable to handle the exception yourself and continue running. The try key word can be used to specify a block of code that should be guarded against all exceptions. Immediately following a try block, you should include a catch clause which specifies the exception type that you wish to catch.

Illustration 5.3

```
/* Showing run-time errors in a program. */

class Demonstration_102 {
‘‘‘‘‘‘public static void main (String args [ ]) {
‘‘‘‘‘‘‘‘‘‘‘‘int a = Integer.parseInt(args[0]);
‘‘‘‘‘‘‘‘‘‘‘‘int b = Integer.parseInt(args[1]);
‘‘‘‘‘‘‘‘‘‘‘‘int c = a/b;
```

```
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println("Value of c =" + c);
‘‘‘‘‘‘ }
}
/*Note:
Run this program with the following input:
java Error 1 2
java Error 10 20 30
java Error 40
java Error 4.5 5
*/
```

Illustration 5.4

```
/* Run the following program without exception-handling mechanism for some input */

public class Demonstration_103 {
          static int anyFunction (int x, int y ){
          ‘‘‘‘      int a = x/y;
                              return a;
          }

          public static void main (String args[]) {
                    int a,b, result;
                    a=0;
                    b=0;
                    System.out.print("Enter any two integers : ");

                    a = Integer.parseInt(args[0]);
                    b = Integer.parseInt(args[1]);

                    result’‘= anyFunction (a, b);
                    System.out.println ( "Result : " + result);
          }
}
/*Note:
Run this program with the following input:
java Demonstration_103’‘200 10
java Demonstration_103 10 0
```

```
*/
```

Illustration 5.5

```
/* Run the following program with exception handling mechanism for some input */

// Case : Simple try-catch block��

public class Demonstration_104 {
          static int anyFunction (int x, int y ){
                    try {
                              int a = x/y;
                              return a;
                    }
                    catch (ArithmeticException e) {
                              System.out.println ( "Division by zero" );
                    }
                    return 0;
          }

          public static void main (String args[]) {
                    int a,b, result;
                    a=0;
                    b=0;
                    try{
                              a = Integer.parseInt(args[0]);
                              b = Integer.parseInt(args[1]);
                              System.out.print("Value of a and b =‘‘"+a+"‘‘"+b);
                    }catch(Exception e){}
                    result’‘= anyFunction (a, b);
                    System.out.println ( "\nResult : " + result);
          }
}
/*Note:
Run this program with the following input:
java Demonstration_104 200 10
java Demonstration_104 10 0
*/
```

Illustration 5.6

```
/* The following program with exception handling mechanism shows how robust it while it is in execution */

class Demonstration_105 {
          public static void main (String args[ ]){
                    int number, InvalidCount = 0, validCount = 0;
                    for (int i = 0; i < args.length; i++)‘‘
                    {
                              try {
                                        number = Integer.parseInt(args[i]);
                              } catch (NumberFormatException e){
                                        InvalidCount++;
                                        System.out.println ("Invalid number at " + i +"‘‘"+ args[i]);
                              }
                              validCount++;
                              System.out.println ("Valid number at " + i+"‘‘"+ args[i]);
                    }
                    System.out.println ("Invalid entries: " + InvalidCount);
                    System.out.println ("Valid entries: " + validCount);
          }
}
/*Note:
Run this program with the following input:
java Demonstration_105’‘1 2 3
java Demonstration_105’‘10 20 30 40
java Demonstration_105’‘1.0’‘2 3.0 40 java
java Demonstration_105’‘4.5 5
*/
```

Illustration 5.7

```
/* Run the following program without exception handling mechanism for some input */

// Case : try with multiple catch block��

public class Demonstration_106’‘{
```

```
          public static void main (String args[ ])
          ‘‘‘‘{


                              int i = args.length;           ‘‘// No of arguments in the command line
                              String myString[] = new String[i];
                              myString[0]=args[0];


                    if(myString[0].equals("Java"))
‘‘‘‘‘‘              ‘‘{
                                        ‘‘System.out.println("First word is Java !");
                    ‘‘‘‘‘‘}
                              System.out.println( " Number of arguments = " + i );
                    ‘‘‘‘int x = 12/ i;
                              int y[ ] = {555, 999};
                              y[ i ] = x;
          }
}
/*Note:
Run this program with the following input:
java Demonstration_106
java Demonstration_106 Java
java Demonstration_106 I love Java
java Demonstration_106 10 20 30 40
*/
```

Next let us see, the multiple use of catch clauses, let us take a look at the **Illustration 5.8.**

Illustration 5.8

```
/* Run the following program with exception handling mechanism for the same input */

// Case : try with multiple catch block��

public class Demonstration_107’‘{
          public static void main (String args[ ]) {
                    try {
                              int i = args.length;           ‘‘// No of arguments in the command line
                              String myString[] = new String[i];
                              // If i = 0 then’‘myString null pointer error
```

```
                              if(myString[0].equals("Java")){‘‘// #1 //
                                        System.out.println("First word is Java !");
                              }
                              System.out.println( " Number of arguments = " + i );
                              int x = 12/ i;       ‘‘‘‘// # 2 //
                              int y[ ] = {555, 999}; // y is an array of size 2 with index 0,1
                              y[ i ] = x;’‘ //#3// Index is out-of-range may occur if i > 1
                    }
                    catch (ArithmeticException’‘e ) {                   // To catch the error at #2
                              System.out.println ( " Div by 0 : "+ e );
                    }
                    catch (NullPointerException’‘e ) {                  // To catch the error at #1
                              System.out.println ( "A null pointer exception :" + e );
                    }
                    catch (ArrayIndexOutOfBoundsException e ) {
                              // To catch the error at’‘#3
                              System.out.println ("Array Index OOB : " + e);
                    }
          }
}
/*Note:
Run this program with the following input:
java Demonstration_106
java Demonstration_106 Java
java Demonstration_106 I love Java
java Demonstration_106 10 20 30 40
*/
```

For different kind of errors, corresponding catch clause will catch the respective exception.Instead of using multiple catches, only one catch clause can handle number of exceptions at a time. Following is an illustration for this :

Illustration 5.9

```
/* Multiple errors with single catch block□ */

class Demonstration_108’‘ {
‘‘‘‘public static int j;
‘‘‘‘public static void main (String args[ ] ) {
          for (int i = 0; i < 4; i++ ) {
‘‘‘‘‘‘‘‘ try {
```

```
                                switch (i) {
                                          case 0 :
                                                    int zero = 0;
                                                    j = 999/ zero; // Divide by zero’‘
                                                    break;
                                          case 1:
                                                    int b[ ] = null;
                                                    j = b[0] ; // Null pointer error’‘
                                                    break;
                                          case 2:
                                                    int c[] = new int [2] ;
                                                    j = c[10]; // Array index is out-of-bound
                                                    break;
                                          case 3:
                                                    char ch = "Java".charAt(9) ;// String index is out-of-
bound
                                                    break;
                                }
‘‘‘‘‘‘‘} catch (Exception e) {‘‘
‘‘‘‘‘‘‘‘‘‘‘ System.out.println("In Test case#"+i+ "\n");


‘‘‘‘‘‘‘‘‘‘‘ System.out.println (e.getMessage() );’‘‘‘‘‘‘‘‘‘
                    }
‘‘‘‘}
‘‘}
}’‘
```

OUTPUT:
In Test case#0 / by zero In Test case#1 null In Test case#2 10 In Test case#3 String index out of range: 9

Exception handling is in fact built with try-catch-finally construct, although the finally clause is purely optional. But in some cases, it is required to execute few codes regardless of whether an exception was caught or not. For example, before exiting a program, it may have to close some open files and freeing up any other resources that might have been allocated at the beginning of a method. The finally clause defines a block of code which will be executed always

irrespective of any exception occurs or not. Consider the use of the **Illustration 5.10** to give a demonstration of the use of finally :

Illustration 5.10

```
/* finally in try-catch block */


class Demonstration_109’‘‘‘{
          public static void main (String [ ] args ) {
          ‘‘ int i = 0;
          ‘‘ String greetings[] = {"Hello Twinkle !", "Hello Java !", "Hello World ! "};
          ‘‘ while ( i < 4) {
‘‘‘‘‘‘‘‘‘‘try {
          ‘‘‘‘‘‘ System.out.println (greetings [i] );
                    ‘‘ i++;
‘‘‘‘‘‘‘‘‘‘}catch (Exception e ) {
          ‘‘‘‘‘‘‘‘ System.out.println (e.toString() );
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ // Message of exception e in String format
          ‘‘‘‘‘‘}
          ‘‘‘‘‘‘finally {
          ‘‘‘‘‘‘‘‘ System.out.println (" Hi !");
          ‘‘‘‘‘‘‘‘ if (i < 3);’‘
‘‘‘‘‘‘‘‘‘‘‘‘ else’‘{System.out.println("You should quit and reset index value");break;}
                    ‘‘}
          ‘‘ } // while ( )
‘‘ }      // main ( )
} // class’‘‘‘‘‘‘‘‘‘‘‘‘‘
```

OUTPUT:
Hello Twinkle ! Hi ! Hello Java ! Hi ! Hello World ! Hi ! You should quit and reset index value

If you run this program, you will see that the code in finally block will be executed always the loop is iterated.

In Java, throw key word is known by which user can throw an exception of their own instead of automatic exception object generated by Java run time. To use this, we have to create an instance of Throwable object. Then the throw key word can be used to throw this exception. Following is the Illustration 5.11 to clear this idea :

Illustration 5.11

```
/* Use of throws clause in exception handling */


import java.lang.*;


public class Demonstration_1010 {
          public static void main(String Args[]) throws Exception{
                    int[] array = new int[3];
                    try{
                              for (int i=0;i<4;++i)
                              {
                                        array[i] = i;
                              }
                              System.out.println(array);
                    }
                    catch(ArrayIndexOutOfBoundsException e){
                              System.out.println("filIn: " + e.fillInStackTrace());
                              System.out.println("cause: " + e.getCause());
                              System.out.println("local: " + e.getLocalizedMessage());
                              System.out.println("messa: " + e.getMessage());
                              System.out.println("trace: " + e.getStackTrace());
                              System.out.println();
                              System.out.println();
                              System.out.print("trace: ");
                              e.printStackTrace();
                              System.out.println();
                              System.out.print("string: ");
                              e.toString();
                              System.out.println();
                              System.out.println();
                              //printed just to inform that we have entered the catch block
                              System.out.println("Oops, we went too far, better go back to 0!");
                              throw (Exception) new Exception().initCause(e);
                    }
                    finally{
```

```
                                System.out.println(array);
                                // method call to continue program
                        }
            }
}
```

Illustration 5.12a

```
/* Necessity of nested try-catch example */

class Demonstration_1011a‘‘{
‘‘‘‘public static void main(String args[]) {
‘‘‘‘‘‘‘‘‘‘int a = args.length;
‘‘‘‘‘‘‘‘‘‘int b = 42 / a;‘‘ // Divide-by-zero exception’‘‘‘
           ‘‘‘‘ System.out.println("a = " + a);
‘‘‘‘‘‘‘‘‘‘if(a==1)
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘a = a/(a-1);// // Another divide-by-zero exception
‘‘‘‘‘‘‘‘‘‘if(a==2) {
‘‘‘‘‘‘‘‘‘‘‘‘ int c[ ] = { 1 };
‘‘‘‘‘‘‘‘‘‘‘‘ c[2] = 99; // out-of-bound exception, if two argments
‘‘‘‘‘‘‘‘‘‘}
‘‘‘‘}
}
/*Note: Run the program with the following output
java Demonstration_1011a’‘1 2 3
java Demonstration_1011a’‘1 2
java Demonstration_1011a’‘1
java Demonstration_1011a’‘1
*/
```

Illustration 5.12b

```
/* Remedy with nested try-catch example */

class Demonstration_1011b {
           public static void main(String args[]) {
```

```
                    try {‘‘
                              // To catch divide-by-zero
                              int a = args.length;
                              int b = 42 / a;
                              // divide-by-zero exception
                              System.out.println("a = " + a);
                              if(a==1)
                                        a = a/(a-a);
                              // another divide-by-zero exception
                              try {
                                        // nested try block
                                        if(a==2) {
                                        // If two command-line args are used, then an       out-of-
bounds exception
                                        int c[ ] = { 1 };
                                        c[2] = 99;
                                        }

                              }catch(ArrayIndexOutOfBoundsException e) {
                                        System.out.println("Array      index out-of-bounds: " + e);
                                        }

                    }catch(ArithmeticException e) {
                              System.out.println("Divide by 0:"+e);
                              }
          }
}
/*Note: Run the program with the following output
java Demonstration_1011a’‘1 2 3
java Demonstration_1011a’‘1 2
java Demonstration_1011a’‘1
java Demonstration_1011a’‘1
*/
```

Error Handling Exception Classes

In previous Section of this Chapter, we have listed the different classes for handling exceptions in Java. In this Section, let us get a brief introduction about the main of them :

ArithmeticException: An ArithmeticException is thrown if one try to divide an integer by zero or take a modules by zero. For example, the following code causes an ArithmeticException to be thrown:

```
          int wrongMath ( ) {
                    int n = 100;
                    int result ;
                    for (int i = 9; i > -1; i- - )
                    result = n % i;                // modulo remainder.
                    return (result );
          }
```

ArrayIndexOutofBoundsException : In ArrayIndexOutofBoundsException is thrown when one try to access an array element that is out of bounds, meaning that one using an index of less than zero or greater than or equal to the size of the array. Here is a token example that would throw an ArrayIndexOutofBoundsException :

```
          void wrongArrayAccess (‘‘) {
                    int anArray = new int[10] ;    // An array of size having index 0,1,..,9
                    ��..
                    anArray[10] = 999 ;            // index out of range
          }
```

ArrayStoreException : This exception occurs when one try to store a value into an array of incompatible class or type. Following is an example where ArrayStoreException will be thrown.

```
          void’‘badArrayStore (‘‘) {
                    int’‘storeArray’‘= new int[15];                    // An array of integers
                    boolean boolArray’‘=new boolean[5];’‘‘‘‘‘‘‘‘‘// An array of booleans
                    System.arraycopy(storeArray, 2, boolArrary, 2, 4);
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ // Copy the element boolArray[3,4,5] into storeArray starting
at storeArray[2]
          }
```

ClassCastException : In Java, an instance of a class of one type can be possible to cast for another type. Here an instance of class can be casted to its super class but one can not cast an instance of class to its subclasses. If one attempt this cast, a ClassCasteException will occur. The following example, results a ClassCastException at run time :

```
          ‘‘class ClassA {                                  // a token of a simple class
                    ���
          ‘‘ }


‘‘‘‘‘‘‘‘class’‘ClassB extends ClassA{              // A sub class of ClassA
          ‘‘‘‘‘‘ ���.
```

```
‘‘‘‘‘‘void’‘bMethod ( )        {‘‘. . . . }
‘‘ }


‘‘ class Test {
            void wrongCast ( ) {
                      ClassA anInstanceA = new ClassA( );
                      ClassB anInstanceB = (Class B ) anInstanceA;        // Exception
                      anInstanceB.bMethod (‘‘);
                      }
           }
```

IllegalArgumentException : This IllegalArgumentException occurs when one attempt to pass a parameter that is not in valid range or value for the method. The following method throws an IllegalArgumentException if passed an illegal parameter value:

```
static void wrongArgumentPass (int agru ) {
           if (argu == 0)
                      throw new IllegalArgumentException ( "Argument cannot be 0 ");
                      int x = 555 / argu;
           }
```

Not that, in the above example, method wrongArgumentPass(int) throws an exception when caller passes unacceptable value.
IllegalThreadStateException : This exception is thrown by some methods in the system package classes when one try to illegally change the state of thread, for example, by trying to start a thread that is already running.

IndexOutofBoundsException : This exception can be thrown in a method when one passed an index value that is out side an acceptable range. Example is already visited in ealier discussions.

NegativeArraySizeException : This exception is thrown when an array with a negative size is attempted to be allocated. The following method results in a NegativeArraySizeException at run time :

```
Void negativeSizeArray (‘‘) {
           int theSize = -5;
           int’‘foolArray = new int[theSize];
           }
```

NullPointerException : This exception is thrown when one attempt to use a method or variable in a variable name that contains a null object reference. The following method results in a NullPointerException at run time:

```
 void’‘nullPointer (‘‘) {
          String myString = null;                  // myString is a null reference object
          if ( myString.equals (" Sahara" )) {
```

```
                    System.out.println (" Howz!"); }
                }
```

NumberFormatException : This exception is thrown by some methods in classes of System package when one try to convert an invalid string to a number or vice versa.

SecurityException : This exception is thrown by some methods in the System package when one attempt to call a method that will perform an action not allowed by the current security settings of the browser within which the applet code is running. It can also be thrown if the program denies permission when prompted whether to allow an action such as writing to a file.

StringIndexOutOfBoundsException : A StringIndexOutOfBoundsException is thrown when one try to access a character that is out of the bounds of a string, meaning that using an index of less than zero or greater than or equal to the length of the string. Following is an example that would throw a StringIndexOutOfBoundException :

```
     ‘‘void wrongStringIndex ( ) {
          String theString’‘= " N E R I S T",
          char theChar =‘‘theString.charat(20);    //’‘Index should be between 0 and 10
     ‘‘‘‘}
```

ClassNoFoundException : This exception is thrown by the class loader when a class file is not found when a class is attempted to be instantiated.

DataFormatException : This exception is thrown when data being read from a string appears to be in an invalid format.

IllegalAccessException : This exception is thrown by methods in java.lang class when instantiating a class by its name if the class is not public or there is no public constructor. One might encounter this exception if calling a method that, in turn, calls one of these methods.

InstantiationException : This exception is thrown when an attempt is made to instantiate an abstract class, primarily by methods in java.lang class when instantiating a class by its name.

InterruptedException : This exception is thrown within a thread when it is interrupted by some other thread. This exception will be illustrated during the discussion of Thread in Java.

NoSuchMethodException : This exception is thrown when a particular method in an object or class cannot be found.

The information about other less frequently used exceptions can be obtained in details from Chapter 2 of Part III in this book.

Practice Question

```
Practice 5.1
     public class DivideZero {
          static int anyFunction (int x, int y ){
               try {
                    int a = x/y;
                    return a;
               }
               catch (ArithmeticException e) {
```

```
                                        System.out.println ( "Division by zero" );
                                }
                                return 0;
                        }
                        public static void main (String args[]) {
                                int a,b, result;
                                a=0;
                                b=0;
                                System.out.print("Enter any two integers : ");
                                try{
                                a = System.in.read();
                                b = System.in.read();
                                }catch(Exception e){}
                                result’‘= anyFunction (a, b);
                                System.out.println ( "Result : " + result);
                        }
                }
```

Find out the types of exceptions.

Practice 5.2

```
                class CommandLineInput {
                public static void main (String args[ ] {
                            int number, InvalidCount = 0; validCount = 0;
                            for (int i = 0; i < args.length; i++)’‘
                {
                            try {
                                        number = Integer.parseInt(args[i]);
                                        } catch (NumberFormatException e)
                                {
                                ‘‘inavlidCount++;
                                                System.out.println ( ﷽Invalid number at ﷽ + i +
args.[i]);
                                        }
                                                ‘‘validCount++;
                                        ‘‘System.out.println ( ﷽Valid number at ﷽ + i + args.[i]);
                                 }
                                System.out.println ( ﷽Invalid entries: ﷽ + inValidCount);
                                System.out.println ( ﷽Valid entries: ﷽ + validCount);
                                }
                ‘‘ }
                }
```

Find out the types of exceptions.

Practice 5.3

```
          public class MultiCatch {
          public static void main (String args[ ]) {
                    try {
                              int i = args.length;           ‘‘// No of arguments in the command line
                              String myString[] = new String[i];
          // If i = 0 then’‘myString null pointer error
          // #1 //  if(myString[0].equals(�Java�));
                                        System.out.println("First word is Java !");
                                        System.out.println( " Number of arguments = " + i );
          // # 2 //’‘ int x = 18/ i;
                                        int y[ ] = {555, 999};
          // y is an array of size 2 and index are 0,1
          // #3 //                     y[ i ] = x;
          // Index is out-of-range may occur if i > 1
                              }
                              catch (ArithmeticException’‘e ) {                   // To catch the error
at #2
                                        System.out.println ( " Div by 0 : "+ e );
                              }
                              catch (NullPointerException’‘e ) {                  // To catch the error
at #1
                              System.out.println ( "A null pointer exception :" + e );
                              }
                              catch (ArrayIndexOutOfBoundsException e ) {
          // To catch the error at’‘#3
                                        System.out.println ("Array Index OoB : " + e);
                    }
          }
}
```

Find out the types of exceptions.

Practice 5.4

```
                    import java.lang.*;
                    public class exceptions{
                    public static void main(String Args[]) throws Exception{
                              int[] array = new int[3];
                              try{
                                        for (int i=0;i<4;++i)
```

```
                                        {
                                                array[i] = i;
                                        }
                                        System.out.println(array);
                                }
                                catch(ArrayIndexOutOfBoundsException e){
                                        System.out.println("filIn: " + e.fillInStackTrace());
                                        System.out.println("cause: " + e.getCause());
                                        System.out.println("local: " + e.getLocalizedMessage());
                                        System.out.println("messa: " + e.getMessage());
                                        System.out.println("trace: " + e.getStackTrace());
                                        System.out.println();
                                        System.out.println();
                                        System.out.print("trace: "); e.printStackTrace();
                                        System.out.println();
                                        System.out.print("string: ");e.toString();
                                        System.out.println();
                                        System.out.println();
                                        //printed just to inform that we have entered the catch block
                                        System.out.println("Oops, we went too far, better go back to
0!");
                                        throw (Exception) new Exception().initCause(e);
                                }
                                finally{
                                        System.out.println(array);
                                        //method call to continue program
                                }
                        }
                }
```

Find out the types of exceptions.

Practice 5.5

```
                        class ExceptionTest {
                        public static int j;
                        public static void main (String'' args[ ] ) {
                                for (int i = 0; i < 4; i++ ) {
                                        try {
                                                switch (i) {
                                                case 0 :
                                                        int zero = 0;
                                                        j = 999/ zero;                  // divide by
zero''
```

```
                                                  break;
                                        case 1:
                                                  int b[ ] = null;
                                                  j = b[ 0] ;          // Null pointer error
                                                  break;
                                        case 2 :
                                                  int c[] = new int [2] ;
                                                  j = c[10];                    // Array
index is out-of-bound
                                                  break;
                                        case 3 :
                                                  char ch = "Java".charAt(9) ;   // String
index is out-of-bound
                                                  break;
                                        }                   // switch
                              }                   // try
                              catch (Exception e) {                    // To catch an
exception
                                        System.out.println "In Test case # " + i + "\n" );
                                        System.out.println (e) ;
                              }         // catch
                    } // main
          }'' // class
     }
```

Find out the types of exceptions.

Practice 5.6

```
          // Use of finally in try-catch //
          class FinallyDemo {
          public static void main (String [ ] args ) {
          int i = 0;
          String greetings [ ] = {
                    "Hello Twinkle !",
                              "Hello Java !",
                    "Hello World ! " };


          while ( i < 4) {
           try {
                    System.out.println (greetings [i] );
                     }catch (Exception e ) {
                    System.out.println (e.toString());        // message of exception e in
String format
```

```
                              System.out.println("Resetting index value");


                    ‘‘ } finally {
                              System.out.println (" Hi !");
                              ‘‘ i ++;
                                                  }
                    }         // while ( )
                               }        // main ( )
          } // class
```

Find out the types of exceptions.

Practice 5.7

```
// File Name BankDemo.java
                    public class BankDemo
                    {
                    ‘‘ public static void main(String [] args)
                    ‘‘ {
                              ‘‘CheckingAccount c = new CheckingAccount(101);
                              ‘‘System.out.println("Depositing $500...");
                              ‘‘c.deposit(500.00);
                              ‘‘try
                              ‘‘{
                                        System.out.println("\nWithdrawing $100...");
                                        c.withdraw(100.00);
                                        System.out.println("\nWithdrawing $600...");
                                        c.withdraw(600.00);
                              ‘‘}catch(InsufficientFundsException e)
                              ‘‘{
                                        System.out.println("Sorry, but you are short $"+
e.getAmount());
                                        e.printStackTrace();
                              ‘‘}
                              }
                    }


                    // File Name CheckingAccount.java


                    //create a separate class file and name it as’‘CheckingAccount.java. Then paste the
following class //contents there.
```

```
                    public class CheckingAccount
                    {
                    ‘‘ private double balance;
                    ‘‘ private int number;
                    ‘‘ public CheckingAccount(int number)
                    ‘‘ {
                              ‘‘this.number = number;
                    ‘‘ }
                    ‘‘ public void deposit(double amount)
                    ‘‘ {
                              ‘‘balance += amount;
                    ‘‘ }
                    ‘‘ public void withdraw(double amount) throws InsufficientFundsException
                    ‘‘ {
                              ‘‘if(amount <= balance)
                              ‘‘{
                                        balance -= amount;
                              ‘‘}
                              ‘‘else
                              ‘‘{
                               double needs = amount - balance;
                               throw new InsufficientFundsException(needs);
                    ‘‘}
          ‘‘ }
          ‘‘ public double getBalance()
          ‘‘ {
                    ‘‘return balance;
          ‘‘ }
          ‘‘ public int getNumber()
          ‘‘ {
                    ‘‘return number;
          ‘‘ }
          }


          // File Name InsufficientFundsException.java
          //create a separate class file and name it as InsufficientFundsException.java. Then paste the
following class contents there.


          public class InsufficientFundsException extends Exception
          {
          ‘‘ private double amount;
```

```
‘‘ public InsufficientFundsException(double amount)
‘‘ {
        ‘‘this.amount = amount;
‘‘ }
‘‘ public double getAmount()
‘‘ {
        ‘‘return amount;
‘‘ }
}
```

Find out the types of exceptions.

Practice 5.8

```
import java.io.*;

public class exceptionHandle{
‘‘public static void main(String[] args) throws Exception{
‘‘try{
‘‘int a,b;
‘‘BufferedReader in =‘‘ new BufferedReader(new InputStreamReader(System.in));
‘‘a = Integer.parseInt(in.readLine());
‘‘b = Integer.parseInt(in.readLine());
‘‘}
‘‘catch(NumberFormatException ex){
‘‘System.out.println(ex.getMessage()’‘+ " is not a numeric value.");
‘‘System.exit(0);
‘‘}
‘‘}
}
```

Find out the types of exceptions.

Assignment

- **Q: Is there any way to "get around" the strict restrictions placed on methods by the throws clause?**
- **Q: Differences between exceptions, errors, and runtime exceptions.**

Q&A

Q: What is an exception?

A: An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

Q: What is error?

A: An Error indicates that a non-recoverable condition has occurred that should not be caught. Error, a subclass of Throwable, is intended for drastic problems, such as OutOfMemoryError, which would be reported by the JVM itself.

Q: Which is superclass of Exception?

A: "Throwable", the parent class of all exception related classes.

Q: What are the advantages of using exception handling?

A: Exception handling provides the following advantages over "traditional" error management techniques:

- Separating Error Handling Code from "Regular" Code.
- Propagating Errors Up the Call Stack.
- Grouping Error Types and Error Differentiation.

Q: Why Runtime Exceptions are Not Checked?

A: The runtime exception classes (RuntimeException and its subclasses) are exempted from compile-time checking because, in the judgment of the designers of the Java programming language, having to declare such exceptions would not aid significantly in establishing the correctness of programs. Many of the operations and constructs of the Java programming language can result in runtime exceptions. The information available to a compiler, and the level of analysis the compiler performs, are usually not sufficient to establish that such run-time exceptions cannot occur, even though this may be obvious to the programmer. Requiring such exception classes to be declared would simply be an irritation to programmers.

Q: What is the use of finally block?

A: The finally block encloses code that is always executed at some point after the try block, whether an exception was thrown or not. This is right place to close files, release your network sockets, connections, and perform any other cleanup your code requires.
Note: If the try block executes with no exceptions, the finally block is executed immediately after the try block completes. It there was an exception thrown, the finally block executes immediately after the proper catch block completes

Q: Can we have the try block without catch block?

A: Yes, we can have the try block without catch block, but finally block should follow the try block.
Note: It is not valid to use a try clause without either a catch clause or a finally clause.

Q: What is the difference throw and throws?

A: **throws:** Used in a method's signature if a method is capable of causing an exception that it does not handle, so that callers of the method can guard themselves against that exception. If a method is declared as throwing a particular class of exceptions, then any other method that calls it must either have a try-

catch clause to handle that exception or must be declared to throw that exception (or its superclass) itself.
A method that does not handle an exception it throws has to announce this:

```
‘‘public void myfunc(int arg) throws MyException {
‘‘‘‘‘‘‘‘
‘‘‘‘}
```

throw: Used to trigger an exception. The exception will be caught by the nearest try-catch clause that can catch that type of exception. The flow of execution stops immediately after the throw statement; any subsequent statements are not executed. To throw an user-defined exception within a block, we use the throw command:

```
‘‘throw new MyException("I always wanted to throw an exception!");
‘‘
```

Q: How to create custom exceptions?
A: By extending the Exception class or one of its subclasses.

```
Example:
class MyException extends Exception {
‘‘public MyException() { super(); }
‘‘public MyException(String s) { super(s); }
‘‘}
‘‘
```

Q: What are the different ways to handle exceptions?
A: There are two ways to handle exceptions:
□’‘Wrapping the desired code in a try block followed by a catch block to catch the exceptions.
□’‘List the desired exceptions in the throws clause of the method and let the caller of the method handle those exceptions.

Q: What are the types of Exceptions in Java?
A: There are two types of exceptions in Java, unchecked exceptions and checked exceptions.
□’‘**Checked exceptions:** A checked exception is some subclass of Exception (or Exception itself), excluding class RuntimeException and its subclasses. Each method must either handle all checked exceptions by supplying a catch clause or list each unhandled checked exception as a thrown exception.
□’‘**Unchecked exceptions:** All Exceptions that extend the RuntimeException class are unchecked exceptions. Class Error and its subclasses also are unchecked.

Q: Why Errors are Not Checked?
A: A unchecked exception classes which are the error classes (Error and its subclasses) are exempted from compile-time checking because they can occur at many points in the program and recovery from them is difficult or impossible. A program declaring such exceptions would be pointlessly.

Chapter 6

Threads and Multithreading in Java

by Debasis Samanta

CONTENTS

- Introduction
- Basics of a thread
  - Creating and Running a Thread
  - Life cycle of threads
  - Status of a Thread
- Synchronization and Inter-Thread Communication
  - Synchronization
  - Inter-thread Communication
- Thread Groups and Daemon
- Practice Questions
- Assignment

- Q&A

Introduction

Multi-threading means multiple flow of control. Multi-threading programming is a conceptual paradigm for programming where one can divide a program into two or more processes which can be run in parallel. There are two main advantages of multi-threading : Fist, program with multiple threads will, in general, result in better utilization of system resources, including the CPU, because another line of execution can grab the CPU when one line of execution is blocked. Second, there are several problems better solved by multiple threads. For example, we can easily write a multi-threaded program to show animation, play music, display documents, and down load files from the network at the same time.

Java is a multi-threaded language. Java allows to write a program where more than one processes can be executed concurrently within the single program. Java's threads are often referred to as light weight threads, which means that they run in the same memory space. Because Java threads run in the same memory space, they can easily communicate among themselves because an object in one thread can call a method in another thread without any overhead from the operating system. In this Tutorial we will learn how to do multi-threaded programming in Java.

Basics of a thread

As with the Java concepts, everything about thread are defined in a class Thread. The Thread class encapsulates all of the control one will need over threads. The Thread class is our only link to manage how threads behave. In this Section, we will learn about : how to create and run a thread, the life cycle of a thread, and the thread controlling methods.

Creating and Running a Thread

There are two ways to define a thread: one is sub classing Thread and other is using the Runnable interface.

Using the sub classing thread : With this method, we have to define a class as a sub class of the Thread class. This sub class should contain a body which will be defined by a method run(). This run() method contains the actual task that the thread should perform. An instance of this sub class is then to be created by a new statement, followed by a call to the thread's start() method to have the run() method executed. Let us consider the Illustration 6.1 which includes a program to create there individual threads that each print out their own " Hello World !" string.

Illustration 6.1 **// Creating and running'‘threads using sub classing Thread //**

```
/* Creating three threads using the class Thread and then running them concurrently. */
class ThreadA extends Thread{
‘‘‘‘ public void run( ) {
‘‘‘‘‘‘‘‘for(int i = 1; i <= 5; i++) {
‘‘‘‘‘‘‘‘‘‘ System.out.println("From Thread A with i = "+ -1*i);
‘‘‘‘‘‘‘‘}
‘‘‘‘‘‘‘‘System.out.println("Exiting from Thread A ...");
‘‘‘‘ }
}

class ThreadB extends Thread {
‘‘‘‘public void run( ) {
‘‘‘‘‘‘ for(int j = 1; j <= 5; j++) {
‘‘‘‘‘‘‘‘ System.out.println("From Thread B with j= "+2* j);
‘‘‘‘‘‘ }
‘‘‘‘‘‘ System.out.println("Exiting from Thread B ...");’‘
          }
}
class ThreadC extends Thread{
‘‘‘‘ public void run( ) {
‘‘‘‘‘‘‘‘ for(int k = 1; k <= 5; k++) {
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.println("From Thread C with k = "+ (2*k-1));
‘‘‘‘‘‘‘‘ }
‘‘‘‘‘‘‘‘ System.out.println("Exiting from Thread C ...");
‘‘‘‘ }
}

public class Demonstration_111 {
‘‘‘‘public static void main(String args[]) {
‘‘‘‘‘‘‘‘ ThreadA a = new ThreadA();
```

```
‘‘‘‘‘‘‘‘ ThreadB b = new ThreadB();
‘‘‘‘‘‘‘‘ ThreadC c = new ThreadC();
‘‘‘‘‘‘‘‘ a.start();
‘‘‘‘‘‘‘‘ b.start();
‘‘‘‘‘‘‘‘ c.start();
‘‘‘‘‘‘‘‘ System.out.println("... Multithreading is over ");
‘‘‘‘}
}
```

OUTPUT:

```
From Thread A with i = -1
From Thread A with i = -2
From Thread A with i = -3
From Thread B with j= 2
From Thread A with i = -4
From Thread A with i = -5
Exiting from Thread A ...
... Multithreading is over
From Thread C with k = 1
From Thread B with j= 4
From Thread B with j= 6
From Thread B with j= 8
From Thread B with j= 10
Exiting from Thread B ...
From Thread C with k = 3
From Thread C with k = 5
From Thread C with k = 7
From Thread C with k = 9
Exiting from Thread C ...
```

In the above simple example, three threads (all of them are of some type) will be executed concurrently. Note that a thread can be directed to start its body by start() method.

Using the Runnable interface : A second way to create threads is to make use of the Runnable interface. With this approach, first we have to implement the Runnable interface.[Runnable interface is already defined in the system package **java.lang** with a single method **run()** as below :

```
public interface Runnable {
                                public abstract void run( );
                    }’‘]
‘‘‘‘
```

When we will create a new thread, actually a new object will be instantiated from this Runnable interface as the target of our thread, meaning that the thread will look for the code for the run( ) method within our object's class instead of inside the Thread's class. This is illustrated with an example where two processes Brother and Sister will be executed simultaneously.

Illustration 6.2

```
/* Creating three threads using the Runnable interface and then running them concurrently. */
class ThreadX implements Runnable{
‘‘‘‘ public void run( ) {
‘‘‘‘‘‘‘‘for(int i = 1; i <= 5; i++) {
‘‘‘‘‘‘‘‘‘‘ System.out.println("Thread X with i = "+ -1*i);
‘‘‘‘‘‘‘‘}
‘‘‘‘‘‘ System.out.println("Exiting Thread X ...");
‘‘‘‘ }
}

class ThreadY implements Runnable {
‘‘‘‘ public void run( ) {
‘‘‘‘‘‘‘‘ for(int j = 1; j <= 5; j++) {
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.println("Thread Y with j = "+ 2*j);
‘‘‘‘‘‘‘‘ }
‘‘‘‘‘‘‘‘System.out.println("Exiting Thread Y ...");
‘‘‘‘ }
}
class ThreadZ implements Runnable{
‘‘‘‘ public void run( ) {
‘‘‘‘‘‘‘‘ for(int k = 1; k <= 5; k++) {
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.println("Thread Z with k = "+ (2*k-1));
‘‘‘‘‘‘‘‘ }
‘‘‘‘‘‘‘‘‘‘‘‘ System.out.println("Exiting Thread Z ...");
‘‘‘‘ }
}

public class Demonstration_112 {
‘‘‘‘public static void main(String args[]) {
                    ThreadX x = new ThreadX();
                    Thread t1 = new Thread(x);


                    ThreadY y = new ThreadY();
```

```
                    Thread t2 = new Thread(y);


                    //ThreadZ z = new ThreadZ();
                    //Thread t3 = new Thread(z);
                    Thread t3 = new Thread(new ThreadZ());


                    t1.start();
                    t2.start();
                    t3.start();


‘‘‘‘‘‘‘System.out.println("... Multithreading is over ");
‘‘‘‘}
}
```

OUTPUT:

```
Thread X with i = -1
Thread X with i = -2
Thread Z with k = 1
Thread Z with k = 3
Thread Z with k = 5
Thread Z with k = 7
Thread Z with k = 9
Exiting Thread Z ...
... Multithreading is over
Thread Y with j = 2
Thread Y with j = 4
Thread Y with j = 6
Thread Y with j = 8
Thread Y with j = 10
Exiting Thread Y ...
Thread X with i = -3
Thread X with i = -4
Thread X with i = -5
Exiting Thread X ...
```

Note : Note in the above example, how after implementing objects, their thread is created and their threads start execution. Also note that, a class instance with the run( ) method defined within must be passed in as an argument in creating the thread instance so that when the start() method of this Thread instance is called, Java run time knows

which run() method to execute. This alternative method of creating a thread is useful when the class defining run() method needs to be a sub class of other classes; the class can inherit all the data and methods of the super class.

Life cycle of threads

Each thread is always in one of five states, which is depicted in **Figure 6.1**

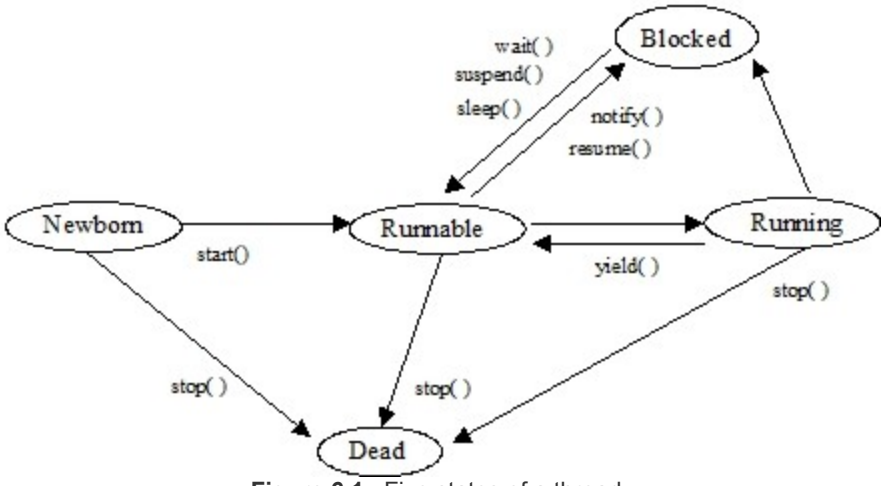


Figure 6.1 : Five states of a thread

Newborn : When a thread is created (by new statement) but not yet to run, it is called in Newborn state. In this state, the local data members are allocated and initialized.
Runnable : The Runnable state means that a thread is ready to run and is awaiting for the control of the processor, or in other words, threads are in this state in a queue and wait their turns to be executed.
Running : Running means that the thread has control of the processor, its code is currently being executed and thread will continue in this state until it get preempted by a higher priority thread, or until it relinquishes control.
Blocked : A thread is Blocked means that it is being prevented from the Runnable (or Running) state and is waiting for some event in order for it to reenter the scheduling queue.
Dead : A thread is Dead when it finishes its execution or is stopped (killed) by another thread.

Threads move from one state to another via a variety of means. The common methods for controlling a thread's state is shown in **Figure 6.1.** Below, we are to summarize these methods :
start ( ) : A newborn thread with this method enter into Runnable state and Java run time create a system thread context and starts it running. This method for a thread object can be called once only
stop( ) : This method causes a thread to stop immediately. This is often an abrupt way to end a thread.
suspend( ) : This method is different from stop( ) method. It takes the thread and causes it to stop running and later on can be restored by calling it again.
resume ( ) : This method is used to revive a suspended thread. There is no gurantee that the thread will start running right way, since there might be a higher priority thread running already, but, resume()causes the thread to become eligible for running.
sleep (int n) : This method causes the run time to put the current thread to sleep for n milliseconds. After n milliseconds have expired, this thread will become elligible to run again.
yield( ) : The yield() method causes the run time to switch the context from the current thread to the next available runnable thread. This is one way to ensure that the threads at lower priority do not get started.

Other methods like **wait(), notify(), join()** etc. will be discussed in subsequent discussion. Let us illustrate the use of these method in a simple Application.

Illustration 6.3

```
/* Demonstration of thread class methods : getID() */
/* Java code for thread creation by extending the Thread class */
```

```
class ThreadId extends Thread {
‘‘‘‘public void run()’‘{
‘‘‘‘‘‘‘‘try {
‘‘‘‘‘‘‘‘‘‘‘‘// Displaying the thread that is running
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println ("Thread " + Thread.currentThread().getId() + " is running");
‘‘‘‘‘‘‘‘}
‘‘‘‘‘‘‘‘catch (Exception e)’‘{
‘‘‘‘‘‘‘‘‘‘‘‘// Throwing an exception
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println ("Exception is caught");
‘‘‘‘‘‘‘‘}
‘‘‘‘}
}
‘‘
public class Demonstration_113{
‘‘‘‘public static void main(String[] args)
‘‘‘‘{
‘‘‘‘‘‘‘‘int n = 8; // Number of threads
‘‘‘‘‘‘‘‘for (int i=0; i<8; i++)
‘‘‘‘‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘‘‘ThreadId object = new ThreadId();
‘‘‘‘‘‘‘‘‘‘‘‘object.start();
‘‘‘‘‘‘‘‘}
‘‘‘‘}
}
```

OUTPUT:
Thread 21 is running Thread 22 is running Thread 23 is running Thread 25 is running Thread 26 is running Thread 27 is running Thread 24 is running Thread 28 is running

Illustration 6.4

```
/* Demonstration of thread class methods : getID() */
/* Java code for thread creation by implementing the Runnable Interface */

class ThreadId implements Runnable
{
‘‘‘‘public void run()’‘‘‘ {
‘‘‘‘‘‘‘‘try’‘{
‘‘‘‘‘‘‘‘‘‘‘‘// Displaying the thread that is running
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println ("Thread " + Thread.currentThread().getId() + " is running");
‘‘‘‘‘‘‘‘‘‘}
‘‘‘‘‘‘‘‘catch (Exception e)
‘‘‘‘‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘‘‘// Throwing an exception
‘‘‘‘‘‘‘‘‘‘‘‘System.out.println ("Exception is caught");
‘‘‘‘‘‘‘‘}
‘‘‘‘}
}
‘‘
// Main Class
class Demonstration_114’‘{
‘‘‘‘public static void main(String[] args)
‘‘‘‘{
‘‘‘‘‘‘‘‘int n = 8; // Number of threads
‘‘‘‘‘‘‘‘for (int i=0; i<8; i++){
                                Thread object = new Thread(new ThreadId());
                                object.start();
‘‘‘‘‘‘‘‘}
‘‘‘‘}
}
```

OUTPUT:
Thread 21 is running Thread 23 is running Thread 24 is running Thread 25 is running Thread 22 is running Thread 27 is running Thread 26 is running

Thread 28 is running

Illustration 6.5

```
/* Use of yield(), stop() and sleep() methods */

class ClassA extends Thread{
          public void run() {
                    System.out.println("Start Thread A ....");
                    for(int i = 1; i <= 5; i++) {
                              if (i==1) yield();
                              System.out.println("From Thread A: i = "+ i);
                    }
                    System.out.println("... Exit Thread A");
          }
}

class ClassB extends Thread{
          public void run() {
                    System.out.println("Start Thread B ....");
                    for(int j = 1; j <= 5; j++) {
                              System.out.println("From Thread B: j = "+ j);
                              if (j==2) stop();
                    }
                    System.out.println("... Exit Thread B");
          }
}

class ClassC extends Thread{
          public void run() {
                    System.out.println("Start Thread C ....");
                    for(int k = 1; k <= 5; k++) {
                              System.out.println("From Thread B: j = "+ k);
                              if (k==3){
                                        try{
                                                  sleep(1000);
                                        }catch(Exception e){}
                              }
```

```
                    }
                    System.out.println("... Exit Thread C");
          }
}


public class Demonstration_115{
          public static void main (String args[]) {
          ‘‘‘‘ClassA t1 = new ClassA();
                    ClassB t2 = new ClassB();
                    ClassC t3 = new ClassC();
                    t1.start(); t2.start(); t3.start();
                    System.out.println("... End of executuion ");
          }
}
```

OUTPUT:

```
Start Thread A ....
Start Thread C ....
Start Thread B ....
... End of executuion
From Thread A: i = 1
From Thread B: j = 1
From Thread B: j = 2
From Thread B: j = 1
From Thread A: i = 2
From Thread A: i = 3
From Thread A: i = 4
From Thread A: i = 5
... Exit Thread A
From Thread B: j = 2
From Thread B: j = 3
From Thread B: j = 4
From Thread B: j = 5
... Exit Thread C
```

Illustration 6.6

```
/* Use of suspend() and resume() methods */

class Thread1 extends Thread {
          public void run( ) {
                    try{
                              System.out.println (" First thread starts running" );
                              sleep(10000);
                              System.out.println (" First thread finishes running" );
                    }
                    catch(Exception e){  }
          }
}

class Thread2 extends Thread {
          public void run( ) {
                    try{
                              System.out.println ( "Second thread starts running");
                              System.out.println ( "Second thread is suspended itself ");
                              suspend( );
                              System.out.println (" Second''thread runs again" );
                    }
                    catch(Exception e){  }
          }
}

class Demonstration_116{
          public static void main (String args[ ] ){
                    try{
                              Thread1 first = new Thread1( );''// It is a newborn thread i.e. in Newborn
state
                              Thread2 second= new Thread2( );''// another new born thread

                              first.start( );''''// first is scheduled''for running
                              second.start( );'' // second is scheduled for running

                              System.out.println("Revive the second thread" );''// If it is suspended
                              second.resume( );
```

```
                                System.out.println ("Second thread went for 10 seconds sleep " );
                                second.sleep (10000);


                                System.out.println ("Wake up second thread and finishes running" );
                                System.out.println ( " Demonstration is finished ");
                        }
                        catch(Exception e){  }
                }
}
```

OUTPUT:
Revive the second thread First thread starts running Second thread starts running Second thread is suspended itself Second thread went for 10 seconds sleep

Status of a Thread

It is some time essential to know some information about threads. There are number of methods defined in Thread which can be called for getting information about threads. Some of the most commonly used methods for thread's status are listed here :

currentThread( ) : The CurrentThread() is a static method returns the Thread object which is the currently running thread.
setName(String s) : The SetName() method is to assign a name s for a thread prior its execution. This, therefore, identifies the thread with a string name. This is helpful for debugging multi-threaded programs.
getName( ) : This method returns the current string value of the thread's name as set by SetName().
setPriority (int p) : This method sets the thread's priotity to an integer value p passed in. There are several predefined priotiry constants defined in class Thread : **MIN-PRIORITY, NORM-PRIORTY and MAX-PRIORITY,** which are 1, 5, and 10 respectively.
getPriority ( ) : This method returns the thread's current priority, a value between 1 and 10.
isAlive ( ) : This method returns true if the thread is started but not dead yet.
isDaemon ( ) : This method returns true if the thread is a daemon thread.

Following is the **Illustration 6.7** to give an idea how the above mentioned method may be utilized.

```
Illustration 6.7                                        // Status information of threads //
/* Setting priority to threads */


class ClassA extends Thread{
          public void run() {
```

```
                    System.out.println("Start Thread A ....");
                    for(int i = 1; i <= 5; i++) {
                              System.out.println("From Thread A: i = "+ i);
                    }
                    System.out.println("... Exit Thread A");
          }
}

class ClassB extends Thread{
public void run() {
          System.out.println("Start Thread B ....");
          for(int j = 1; j <= 5; j++) {
                    System.out.println("From Thread B: j = "+ j);
          }
          System.out.println("... Exit Thread B");
          }
}

class ClassC extends Thread{
          public void run() {
          System.out.println("Start Thread C ....");
          for(int k = 1; k <= 5; k++) {
                    System.out.println("From Thread B: j = "+ k);
          }
          System.out.println("... Exit Thread C");
          }
}


class Demonstration_117{
          public static void main (String args[]) {
                    ThreadA t1 = new ThreadA();
                    ThreadB t2 = new ThreadB();
                    ThreadC t3 = new ThreadC();

                    t3.setPriority(Thread.MAX_PRIORITY);
                    t2.setPriority(t2.getPriority() + 1);
                    t1.setPriority(Thread.MIN_PRIORITY);

                    t1.start(); t2.start(); t3.start();
                    System.out.println("... End of executuion ");
```

```
          }
}
```

OUTPUT:

```
Start Thread A ....
From Thread A: i = 1
From Thread A: i = 2
From Thread A: i = 3
From Thread A: i = 4
From Thread A: i = 5
... Exit Thread A
... End of executuion
Start Thread B ....
Start Thread C ....
From Thread B: j = 1
From Thread B: j = 2
From Thread B: j = 3
From Thread B: j = 4
From Thread B: j = 5
... Exit Thread B
From Thread B: j = 1
From Thread B: j = 2
From Thread B: j = 3
From Thread B: j = 4
From Thread B: j = 5
... Exit Thread C
```

Illustration 6.8

```
/* Data race example. */

public class Demonstration_118 extends Thread {
‘‘‘‘public static int x;
‘‘‘‘public void run() {
‘‘‘‘‘‘‘‘for (int i = 0; i < 100; i++) {
‘‘‘‘‘‘‘‘‘‘‘‘x = x + 1;
```

```
‘‘‘‘‘‘‘‘‘‘‘‘x = x - 1;
‘‘‘‘‘‘‘‘}
‘‘‘‘}
‘‘‘‘public static void main(String[] args) {
‘‘‘‘‘‘‘‘ x = 0;
‘‘‘‘‘‘‘‘ for (int i = 0; i < 1000; i++){
‘‘‘‘‘‘‘‘‘‘‘‘new Demonstration_118().start();
                                System.out.println(x);          // x not always 0!
                        }
‘‘‘‘}
}
```

Synchronization and Inter-Thread Communication

It is already mentioned that threads in Java are running in the same memory space, and hence it is easy to communicate between two threads. Inter-thread communications allow threads to talk to or wait on each other. Again, because all the threads in a program share the same memory space, it is possible for two threads to access the same variables and methods in object. Problems may occur when two or more threads are accessing the same data concurrently, for example, one thread stores data into the shared object and the other threads reads data, there can be synchronization problem if the first thread has not finished storing the data before the second one goes to read it. So we need to take care to access the data by only one thread process at a time. Java provides unique language level support for such synchronization. in this Section we will learn how synchronization mechanism and inter-thread communications are possible in Java.

Synchronization

To solve the critical section problem, one usual concept is known what is called monitor. A monitor is an object which is used as a mutually exclusive lock (called mutex). Only one thread may own a monitor at a given time. When a thread acquires a lock it is said to have entered the monitor. All other threads attempting to enter the locked monitor will be suspended until the owner thread exits the monitor. But in Java, there is no such monitor. In fact, all Java object have their own implicit monitor associated with them. Here, the key word synchronized is used by which method (s) or block of statements can be made protected from the simultaneous access. With this, when a class is designed with threads in mind, the class designer decides which methods should not be allowed to execute concurrently. when a class with synchronized method is instantiated, the new object is given its own implicit monitor. The entire time that a thread is inside of a synchronized method, all other threads that try to call any other synchronized method on the same instance have to wait. In order to exit the monitor and relinquish control of the object to the next waiting thread the monitor owner simply needs to return from the method.

Let us illustrate this mechanism with a simple example.

Suppose, we want to maintain a bank account of customers. Several transactions, such as deposite some amount to an account and withdraw some amount from an account etc. are possible. Now, for a given account, if two or more transactions come simultaneously then only one transaction should be allowed at a time instead of simulataneous transaction processing so that data inconsistency will never occur. So, what we need is to synchronize the transaction. **Illustration 6.9** is to implement such a task.

```
Illustration 6.9
/* The following Java application shows how the transactions in a bank can be carried out concurrently. */
class Account {
```

```
	public int balance;
	public int accountNo;
	void displayBalance() {
		System.out.println("Account No:" + accountNo + "Balance: " + balance);
	}

	‘‘ synchronized void deposit(int amount){
			balance = balance + amount;
			System.out.println( amount + " is deposited");
			displayBalance();
	‘‘ }

	‘‘ synchronized void withdraw(int amount){
			‘‘balance = balance - amount;
			‘‘System.out.println( amount + " is withdrawn");
			‘‘displayBalance();
	‘‘ }
}

class TransactionDeposit implements Runnable{
	int amount;
	Account accountX;
	TransactionDeposit(Account x, int amount){
		accountX = x;
		this.amount = amount;
		new Thread(this).start();
	}

	public void run(){
		accountX.deposit(amount);
	}
}

class TransactionWithdraw implements Runnable{
	int amount;
	Account accountY;

	TransactionWithdraw(Account y, int amount) {
		accountY = y;
		this.amount = amount;
```

```
                    new Thread(this).start();
          }


          public void run(){
                    accountY.withdraw(amount);
          }
}


class Demonstration_119{
          public static void main(String args[]) {
                    Account ABC = new Account();
                    ABC.balance = 1000;
                    ABC.accountNo = 111;
                    TransactionDeposit t1;
                    TransactionWithdraw t2;
                    t1 = new TransactionDeposit(ABC, 500);
                    t2 = new TransactionWithdraw(ABC,900);
          }
}
```

OUTPUT:

```
500 is deposited
Account No:111Balance: 1500
900 is withdrawn
Account No:111Balance: 600
```

In the above example, the keyword synchronized is used for the methods **void deposite(..) and void withdraw(�)** so that these two methods will never run for the same object instance simultaneously.

Alternatively, if one wants to design a class that was not designed for multi-thread access and thus has non-synchronized methods, then it can be wrapped the call to the methods in a synchronized block. Here is the general form of the synchronized statement :

synchronized (Object) { block of statement(s) }

where Object is any object reference. For example, make all the methods in Account class as non-synchronized (remove the synchronized key word). Then modify the code for run( ) in class TransactionDeposite and class TransactionWithdraw are as under :

```
‘‘‘‘public void run( ) {                 // in TransactionDeposite
```

```
          ‘‘ synchronized (accountX ) {
                     accountX.deposite(amount );
                    }
          }

‘‘ public void run( )          {                    // in TransactionWithdraw
          ‘‘synchronized (accountY )’‘‘‘‘‘ {
                    ‘‘ accountY.withdraw(amount );
                    }
          }
```

You will get the same output.

Interested reader may try to run the program without using synchronization and observe the result.

Note : In a class, non-synchronized methods are concurrently executable.

Inter-thread Communication

There are three ways for threads to communicate with each other. The first is through commonly shared data. All the threads in the same program share the same memory space. If an object is accessible to various threads then these threads share access to that object's data member and thus communicate each other.

The second way for threads to communicate is by using thread control methods. There are such three methods by which threads communicate for each other :

suspend ( ): A thread can suspend itself and wait till other thread resume it.
resume ( ): A thread can wake up other waiting thread (which is waiting using suspend()) through its resume() method and then can run concurrently.
join ( ) :This method can be used for the caller thread to wait for the completion of called thread.

The third way for threads to communicate is the use of three methods; **wait(), notify(), and notifyAll();** these are defined in class Object of package **java.lang.** Actually these three methods provide an elegant inter-process communication mechanism to take care the deadlock situation in Java. As there is multi-threading in program, deadlock may occur when a thread holding the key to monitor is suspended or waiting for another thread's completion. If the other thread is waiting for needs to get into the same monitor, both threads will be waiting for ever. The uses of these three methods are briefed as below :

wait ( ):This method tells the calling thread to give up the monitor and make the calling thread wait until either a time out occurs or another thread call the same thread's notify() or notifyAll() method.
Notify ( ): This method wakes up the only one (first) waiting thread called wait() on the same object.
notifyAll( ): This method will wake up all the threads that have been called wait( ) on the same object.

Now, let us demonstrate the classical use of these methods. **Illustration 6.6** is for this purpose.

```
Illustration 6.10 :            // Inter thread communication : Producer & Consumer problem //

          class Q {                      // Q is a class containing two parallel processes
                    int n;
                    boolean flag = false;
‘‘‘‘//PRODUCER
                    synchronized void put( int n) {// Produce a value
```

```
                              if(flag) {                                                                    //
Entry      ‘‘‘‘‘‘     ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                              ‘‘ try’‘wait( );’‘catch(InterruptedException e);              // to the
           ‘‘‘‘‘‘
                              }                                                                             //
critical section     ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘


           ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

           ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘this.n = n;
           ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

                              ‘‘‘‘System.out.println( "Produce :" + n);                  // Critical
Section’‘‘‘‘‘‘‘‘‘


           ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

                              ‘‘‘‘flag = true;                                                          //
Exit from the        ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                              ‘‘‘‘notify( );                                                                //
critical section
                    ‘‘‘‘}
‘‘‘‘//CONSUMER
           ‘‘‘‘‘‘ synchronized int get( ) {                    // Consume a value
                              if(! flag) {                                                              //
Entry                ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                              ‘‘try’‘wait( );’‘catch(InterruptedException e);               // to the
           ‘‘‘‘‘‘
                              }                                                                             //
critical section     ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘


           ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘

                              System.out.println( "Consume :" + n);                     // Critical
Section’‘‘‘‘‘‘‘‘‘
           ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                              ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
```

```
                    }

          ‘‘ public void run( ) {                  // infinite running’‘thread for Producer
                    int i = 0;
                    while (true )
                              q.put ( i++ );
                    ‘‘ }
                    }

          class Consumer implement Runnable {       // Thread for consumer process
                    Q q;
          ‘‘ Consumer (Q q )   {          ‘‘‘‘‘‘‘‘‘‘// Constructor
          ‘‘ this.q’‘= q;
          ‘‘ new Thread (this).start ( );
                    }

           public void run( ) {                    // infinite running thread for Consumer
           while (true)
                    q.get ( );
                    }
          }

‘‘‘‘‘‘ class PandC’‘{
                    public static void main( String args[ ] ) {
                    Q q = new Q( );                // an instance of parallel processes’‘is created
                    new Producer(q) ;                         // Run the thread for producer
                    new Consumer (q);                         // Run consumer thread
                    ‘‘ }
                    }
```

To understand this critical section problem in operating system design, user may be referred to : *Operating system concept by Peterson and Sylberchotze , Addiction Wesley Inc.*

Note : All three methods i.e. wait(), notify(), and notifyAll() must only be called from the inside of synchronized methods.

Thread Groups and Daemon

There are two other variation in thread class known : ThreadGroup and Daemon. ThreadGroup, as its name implies, is a group of threads. A thread group can have both threads and other thread groups as its member elements. In Java, there is a default thread group called SystemThreadGroup, which is nothing but the Java run time itself. When a Java application is started, the Java run time creates the main thread group as a member of the system thread group. A main thread is created in the main thread group to run the main( ) method of the Application. In fact, every thread instance is member of exactly one thread group. By default, all new user created threads and thread groups)

will become the members of the main thread group. All the threads and thread in an application form a tree with the system thread group as the root.

Daemon threads are service threads. They exist to provide service threads. They exist to provide services to other threads. They normally run in an infinite loop and attending the client threads requesting services. When no other threads exist daemon thread is automatically ceased to exist.

A new thread group can be created by instantiating the thread group class. For example,

```
Threadgroup TG = new ThreadGroup ( ) ;
Thread T = new Thread ( TG) ;
```

These two statements creates a new thread group TG which contains a thread T as the only member.

To create a daemon thread, there is a method setDaemon() can be called just after the creation of a thread and before the execution is started. For example, following two statement is to make a thread as demon thread.

```
Thread T = new Thread ( ) ;
T setDaemon (true);
```

The constructor of the thread is a good candidate for making this method call, Also, by default, all the threads created by a daemon thread are also daemon thread.
Some commonly used methods for handling thread groups and daemon are listed below :

getName ( )’“ :Returns the name of the thread group .
setName ( )’“ :Sets the name of the thread group .
geParent ( )’“:Returns the parent thread group of the thread group .
getMaxPriority’“:Returns the current maximum priority of the thread group.
activeCount ( )’“:Returns the number of active threads in the thread group.
activeGroupCount’“:Returns the number of active threads groups in the thread group.
isDaemon ( )’“:Returns true if the thread is a daemon thread.
setDaemon ( )’“: Set the thread as a daemon thread prior its starting execution.

Practice Questions

Practice 6.1

```
          /* Practice of a multithreaded program using subclassing Thread */
          class ThreadA extends Thread{
                    public void run( ) {
                    for(int i = 1; i <= 5; i++) {
                    System.out.println("From Thread A with i = "+ -1*i);
                    }
                    System.out.println("Exiting from Thread A ...");
                    }
                    }


                    class ThreadB extends Thread{
                    public void run( ) {
                    for(int j = 1; j <= 5; j++) {
                    System.out.println("From Thread B with j = "+ 2*j);
                    }
                    System.out.println("Exiting from Thread B ...");
```

```
}
}

class ThreadC extends Thread{
public void run( ) {
for(int k = 1; k <= 5; k++) {
System.out.println("From Thread C with k = "+ (2*k-1));
}
System.out.println("Exiting from Thread C ...");
}
}

class MultiThreadClass{
public static void main(String args[]) {
ThreadA a = new ThreadA();
ThreadB b = new ThreadB();
ThreadC c = new ThreadC();

a.start();
b.start();
c.start();

System.out.println("... Multithreading is over ");
}
}
```

Practice 6.2

```
/* Practice of a multithreaded program using Runnable interface */
class ThreadX implements Runnable{
public void run( ) {
for(int i = 1; i <= 5; i++) {
System.out.println("Thread X with i = "+ i);
}
System.out.println("Exiting Thread X ...");
}
}

class ThreadY implements Runnable{
public void run( ) {
for(int j = 1; j <= 5; j++) {
System.out.println("Thread Y with j = "+ j);
```

```
                    }
                    System.out.println("Exiting Thread Y ...");
                    }
                    }


                    class ThreadZ implements Runnable{
                    public void run( ) {
                    for(int k = 1; k <= 5; k++) {
                    System.out.println("Thread Z with k = "+ k);
                    }
                    System.out.println("Exiting Thread Z ...");
                    }
                    }

                    class MultiThreadRunnable{
                    public static void main(String args[]) {
                    ThreadX x = new ThreadZ(); Thread t1 = new Thread(x);
                    ThreadY y = new ThreadY(); Thread t2 = new Thread(y);
                    ThreadZ z = new ThreadZ(); Thread t3 = new Thread(z);
                    t1.start();
                    t2.start();
                    t3.start();
                    System.out.println("... Multithreading is over ");
                    }
          }
```

Practice 6.3

```
          /* Use of yield(), stop() and sleep() methods */

          class ClassA extends Thread{

                    public void run() {

                              System.out.println("Start Thread A ....");

                              for(int i = 1; i <= 5; i++) {

                                        if (i==1) yield();
```

```
                                        System.out.println("From Thread A: i = "+ i);

                        }

                        System.out.println("... Exit Thread A");

            }

}

class ClassB extends Thread{

            public void run() {

                        System.out.println("Start Thread B ....");

                        for(int j = 1; j <= 5; j++) {

                                    System.out.println("From Thread B: j = "+ j);

                                    if (j==2) stop();

                        }

                        System.out.println("... Exit Thread B");

            }

}



class ClassC extends Thread{

            public void run() {

                        System.out.println("Start Thread C ....");

                        for(int k = 1; k <= 5; k++) {

                                    System.out.println("From Thread B: j = "+ k);
```

```
                                    if (k==3){
                                              try{
                                                        sleep(1000);
                                              }catch(Exception e){}
                                    }
                          }
                          System.out.println("... Exit Thread C");
                }
        }


        class ThreadControl{
                public static void main (String args[]) {
                          ThreadA t1 = new ThreadA();
                          ThreadB t2 = new ThreadB();
                          ThreadC t3 = new ThreadC();
                          t1.start(); t2.start(); t3.start();
                          System.out.println("... End of executuion ");
                }
        }
```

Practice 6.4

```
          /* Use of suspend() and resume() methods */
                    class Thread1 extends Thread {
                    public void run( ) {
                    System.out.println (" First thread starts running" );
                    sleep(10000);
                    System.out.println (" First thread finishes running" );
                    }
                    }

                    class Thread2 extends Thread {
                    public void run( ) {
                    System.out.println ( "Second thread starts running");
                    System.out.println ( "Second thread is suspended itself ");
                    suspend( );
                    System.out.println (" Second’‘thread runs again" ));
                    }
                    }

                    class AnotherThreadControl {
                    public static void main (String, args[ ] ) {
                    Thread1 fist = new Thread1( );’‘// It is a newborn thread i.e. in Newborn state
                    Thread2 second= new Thread2( );’‘// another new born thread
                    first.start( );’‘‘‘// first is scheduled’‘for running
                    second.start( );’‘ // second is scheduled for running

                    System.out.println("Revive the second thread" );’‘// If it is suspended
                    second.resume( );
                    System.out.println ("Second thread went for 10 seconds sleep " );
                    Second.sleep (10000);
                    System.out.println ("Wake up second thread and finishes running" );
                    System.out.println ( " Demonstration is finished ");
                    }
                    }
```

Practice 6.5

```
/* Setting priority to threads */
                    class ClassA extends Thread{
                    public void run() {
                    System.out.println("Start Thread A ....");
                    for(int i = 1; i <= 5; i++) {
```

```
                    System.out.println("From Thread A: i = "+ i);
                    }
                    System.out.println("... Exit Thread A");
                    }
                    }

                    class ClassB extends Thread{
                    public void run() {
                    System.out.println("Start Thread B ....");
                    for(int j = 1; j <= 5; j++) {
                    System.out.println("From Thread B: j = "+ j);
                    }
                    System.out.println("... Exit Thread B");
                    }
                    }

                    class ClassC extends Thread{
                    public void run() {
                    System.out.println("Start Thread C ....");
                    for(int k = 1; k <= 5; k++) {
                    System.out.println("From Thread B: j = "+ j);
                    }
                    System.out.println("... Exit Thread C");
                    }
                    }



                    class ThreadPriorityTest{
                    public static void main (String args[]) {
                    TheadA t1 = new ThreadA();
                    TheadB t2 = new ThreadB();
                    TheadC t3 = new Thread3();

                    t3.setPriority(Thread.MAX_PRIORITY);
                    t2.setPriority(Thread.getPriority() + 1);
                    t1.setPriority(Thread.MIN_PRIORITY);
                    t1.start(); t2.start(); t3.start();
                    System.out.println("... End of executuion ");
          }
‘‘ }
```

Practice 6.6

```
/* Following Java application create a list of numbers and then sort in ascending order as well as in
descending order simultaneously. */

‘‘‘‘import java.util.*;


          class Numbers {

          public int result[] = new int[10];

          void displayListOfNos()

          {

          System.out.println("Numbers stored in the array:");

          for( int idx=0; idx<10; ++idx) {

          System.out.println(result[idx]);

          }

          }


          void fillTheArray(int aUpperLimit, int aArraySize)

          {

          ‘‘if (aUpperLimit <=0) {

          throw new IllegalArgumentException("UpperLimit must be positive: " + aUpperLimit);

          }


          if (aArraySize <=0) {

          throw new IllegalArgumentException("Size of returned List must be greater than 0.");

          }


          Random generator = new Random();

          for( int idx=0; idx<="" result[j])="" int="" temp="result[i];" result[i]="result[j];"
result[j]="temp;" sortdescending()="" style="box-sizing: border-box;"> result[j]) {

          int temp = result[i];

          result[i] = result[j];

          result[j] = temp;

          }

          }

          }


          displayListOfNos();

          }

          }

          class ArrangementAscending implements Runnable {

          Numbers n1 ;
```

```
          ArrangementAscending(Numbers n) {
          n1 = n;
          new Thread(this).start();
          }
          public void run() {
          n1.sortAscending();
          }
          }

          class ArrangementDescending implements Runnable {
          Numbers n2;
          ArrangementDescending(Numbers n) {
          n2 = n;
          new Thread(this).start();
          }



          public void run() {
          n2.sortDescending();
          }
          }

          class ArrangingNos {

          public static void main(String args[]) {
          Numbers n = new Numbers();
          n.fillTheArray(20,10);
          ArrangementAscending’‘a1 = new ArrangementAscending(n);
          ArrangementDescending d1 = new ArrangementDescending(n);
          }
          }
```

Practice 6.7

```
/* The following Java application shows how the transactions in a bank can be carried out concurrently. */
          class Account {
          public int balance;
          public int accountNo;
          void displayBalance()
          {
          System.out.println("Account No:" + accountNo + "Balance: " + balance);
```

```
}


synchronized void deposit(int amount)
{
balance = balance + amount;
System.out.println( amount + " is deposited");
displayBalance();
}

synchronized void withdraw(int amount)
{
balance = balance - amount;
System.out.println( amount + " is withdrawn");
displayBalance();
}
}

class TransactionDeposit implements Runnable
{
int amount;
Account accountX;
TransactionDeposit(Account x, int amount)
{
accountX = x;
this.amount = amount;
new Thread(this).start();
}


public void run()
{
accountX.deposit(amount);
}
}

class TransactionWithdraw implements Runnable
{
int amount;
Account accountY;
TransactionWithdraw(Account y, int amount) {
```

```
accountY = y;
this.amount = amount;
new Thread(this).start();
}
public void run()
{
accountY.withdraw(amount);
}
}

class Transaction {
public static void main(String args[]) {
Account ABC = new Account();
ABC.balance = 1000;
ABC.accountNo = 111;
TransactionDeposit t1;
TransactionWithdraw t2;
t1 = new TransactionDeposit(ABC, 500);
t2 = new TransactionWithdraw(ABC,900);
}
}
```

Assignment

Q: Write a Java program which handles Push operation and Pop operation on stack concurrently.

Q: Write a Java program which first generates a set of random numbers and then determines negative, positive even, positive odd numbers concurrently.

Q&A

Q: What is the thread?

A: A thread is a lightweight subprocess. It is a separate path of execution because each thread runs in a different stack frame. A process may contain multiple threads. Threads share the process resources, but still, they execute independently..

Q: How to implement Threads in java?

A: Threads can be created in two ways i.e. by implementing **java.lang.Runnable interface or extending java.lang.Thread class and then extending run method.** Thread has its own variables and methods, it lives and dies on the heap. But a thread

of execution is an individual process that has its own call stack. Thread are lightweight process in java.

1)Thread creation by implementingjava.lang.Runnableinterface. We will create object of class which implements Runnable interface :

```
MyRunnable runnable=new MyRunnable();

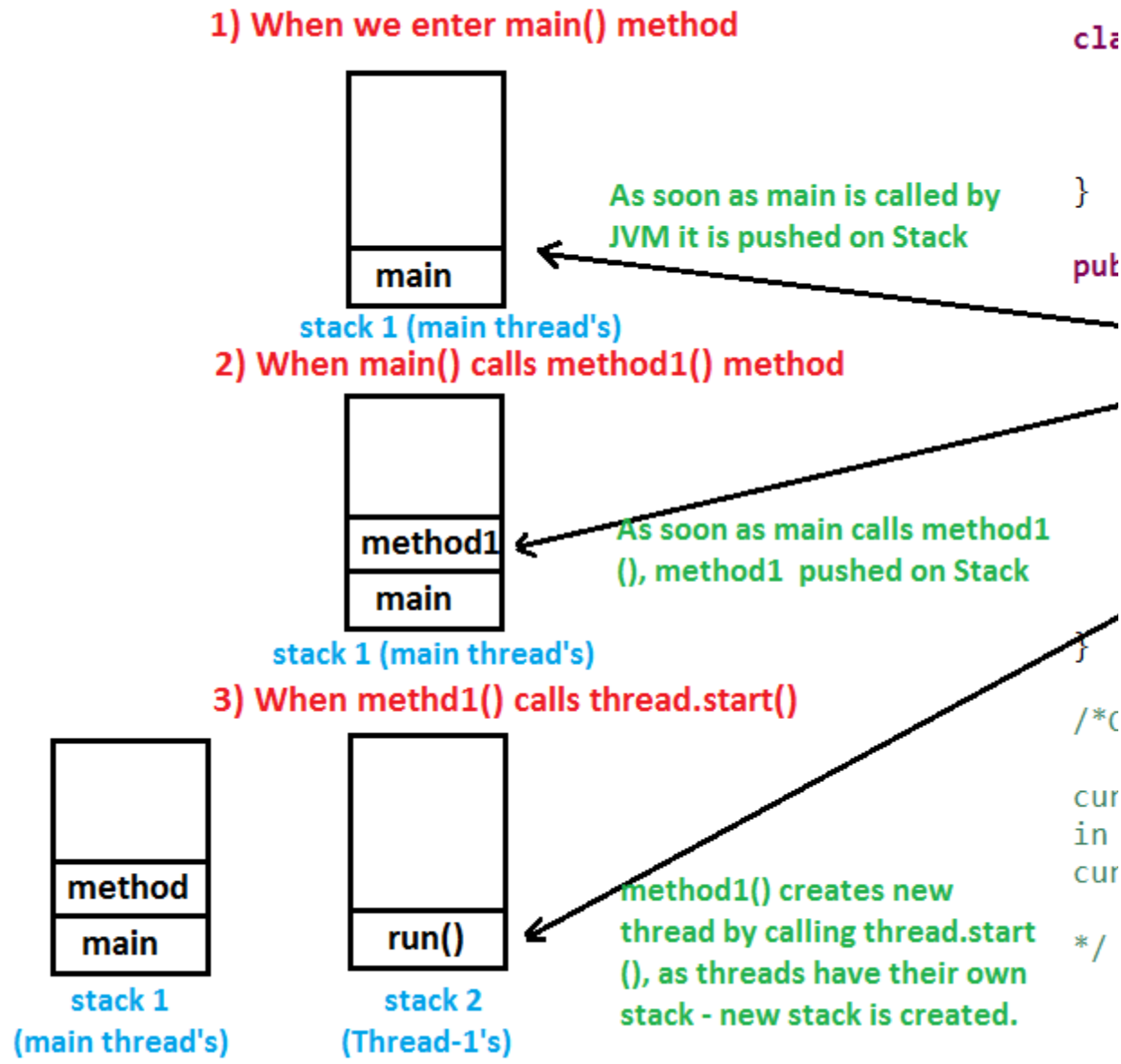

Thread thread=new Thread(runnable);
```

2) And then create Thread object by calling constructor and passing reference of Runnable interface i.e. runnable object :

```
Thread thread=new Thread(runnable);
```

Q : Does Thread implements their own Stack, if yes how?

A : Yes, Threads have their own stack. This is very interesting question, where interviewer tends to check your basic knowledge about how threads internally maintains their own stacks.Shown in figure below.



Q What is multithreading?
:

A Multithreading is a process of executing multiple threads simultaneously.
: Multithreading is used to obtain the multitasking. It consumes less memory and gives the fast and efficient performance. Its main advantages are:

- Threads share the same address space.
- The thread is lightweight.
- The cost of communication between the processes is low.

Q What do you understand by inter-thread communication?
:

A: ’‘The process of communication between synchronized threads is termed as inter-thread communication.

’‘Inter-thread communication is used to avoid thread polling in Java.

’‘The thread is paused running in its critical section, and another thread is allowed to enter (or lock) in the same critical section to be executed.

’‘It can be obtained by wait(), notify(), and notifyAll() methods.

Q: How can you say Thread behaviour is unpredictable?

A: Thread behaviour is unpredictable because execution of Threads depends on Thread scheduler, thread scheduler may have different implementation on different platforms like windows, unix etc. Same threading program may produce different output in subsequent executions even on same platform. To achieve we are going to create 2 threads on same Runnable Object, create for loop in run() method and start both threads. There is no surety that which threads will complete first, both threads will enter anonymously in for loop.

Q: How can you ensure all threads that started from main must end in order in which they started and also main should end in last?

A: We can use join() methodto ensure all threads that started from main must end in order in which they started and also main should end in last.In other words waits for this thread to die. Calling join() method internally calls join(0).

Q: What is difference between starting thread with run() and start() method?

A: When you call start() method, main thread internally calls run() method to start newly created Thread, so run() method is ultimately called by newly created thread.

When you call run() method main thread rather than starting run() method with newly thread it start run() method by itself.

Q: What is significance of using Volatile keyword?

A: Java allows threads to access shared variables. As a rule, to ensure that shared variables are consistently updated, a thread should ensure that it has exclusive use of such variables by obtaining a lock that enforces mutual exclusion for those shared variables.

If a field is declared volatile, in that case the Java memory model ensures that all threads see a consistent value for the variable.

Note: volatile is only a keyword, can be used only with variables.

Q: What is race condition in multithreading and how can we solve it?

A: When more than one thread try to access same resource without synchronization causes race condition.

So we can solve race condition by using either synchronized block or synchronized method. When no two threads can access same resource at a time phenomenon is also called as mutual exclusion.

Q: When should we interrupt a thread?

A: We should interrupt a thread when we want to break out the sleep or wait state of a thread. We can interrupt a thread by calling the interrupt() throwing the

InterruptedException.

Q: What is the purpose of the Synchronized block?

A: The Synchronized block can be used to perform synchronization on any specific resource of the method. Only one thread at a time can execute on a particular resource, and all other threads which attempt to enter the synchronized block are blocked.

- Synchronized block is used to lock an object for any shared resource.
- The scope of the synchronized block is limited to the block on which, it is applied. Its scope is smaller than a method.

Q: What is the difference between notify() and notifyAll()?

A: The notify() is used to unblock one waiting thread whereas notifyAll() method is used to unblock all the threads in waiting state.

Q: How to detect a deadlock condition? How can it be avoided?

A: We can detect the deadlock condition by running the code on cmd and collecting the Thread Dump, and if any deadlock is present in the code, then a message will appear on cmd. Ways to avoid the deadlock condition in Java:

- **Avoid Nested lock:** Nested lock is the common reason for deadlock as deadlock occurs when we provide locks to various threads so we should give one lock to only one thread at some particular time.
- **Avoid unnecessary locks:** we must avoid the locks which are not required.
- **Using thread join:** Thread join helps to wait for a thread until another thread doesn't finish its execution so we can avoid deadlock by maximum use of join method.

Q: Difference between wait() and sleep().

A: **called from synchronized block** :wait() method is always called from synchronized block i.e. wait() method needs to lock object monitor before object on which it is called. But sleep() method can be called from outside synchronized block i.e. sleep() method doesn't need any object monitor.
IllegalMonitorStateException : if wait() method is called without acquiring object lock than IllegalMonitorStateException is thrown at runtime, but sleep() methodnever throws such exception.
Belongs to which class: wait() method belongs to java.lang.Object class but sleep() method belongs to java.lang.Thread class.
Called on object or thread: wait() method is called on objects but sleep() method is called on Threads not objects.
Thread state: when wait() method is called on object, thread that holded object's monitor goes from running to waiting state and can return to runnable state only when notify() or notifyAll()method is called on that object. And later thread scheduler

schedules that thread to go from from runnable to running state.
when sleep() is called on thread it goes from running to waiting state and can return to runnable state when sleep time is up.
When called from synchronized block:when wait() method is called thread leaves the object lock. But sleep()method when called from synchronized block or method thread doesn't leaves object lock.

Q:Similarity between yield() and sleep().

A:

- yield() and sleep() method belongs to java.lang.Thread class.
- yield() and sleep() method can be called from outside synchronized block.
- yield() and sleep() method are called on Threads not objects.

Q:What are daemon threads?

A:Daemon threads are low priority threads which runs intermittently in background for doing garbage collection.
Few salient features of daemon() threads>

- Thread scheduler schedules these threads only when CPU is idle.
- Daemon threads are service oriented threads, they serves all other threads.
- These threads are created before user threads are created and die after all other user threads dies.
- Priority of daemon threads is always 1 (i.e. MIN_PRIORITY).
- User created threads are non daemon threads.
- JVM can exit when only daemon threads exist in system.
- we can use isDaemon() method to check whether thread is daemon thread or not.
- we can use setDaemon(boolean on) method to make any user method a daemon thread.
- If setDaemon(boolean on) is called on thread after calling start() method than IllegalThreadStateException is thrown.

Q:Can a constructor be synchronized?

A:No, constructor cannot be synchronized. Because constructor is used for instantiating object, when we are in constructor object is under creation. So, until object is not instantiated it does not need any synchronization.

''Enclosing constructor in synchronized block will generate compilation error.

''Using synchronized in constructor definition will also show compilation error.
COMPILATION ERROR = Illegal modifier for the constructor in type Constructor Synchronize Test; only public, protected & private are permitted
we can use synchronized block inside constructor.

Chapter 7

Application Development in Java

by Debasis Samanta

CONTENTS

- Introduction
- Designing GUI with Components and Layout Managers
    - Using Components for Java GUIs
    - Using Layout Managers for Java GUI
    - GridBagLayoutManager
- Event Handling
- Drawing methods for Graphics objects
- Practice Questions

- Assignment

- Q&A

Introduction

Java is a very powerful programming language to develop varieties of applications and makes the application development easy. Java provides a package of classes called the Abstract Windowing Toolkit (or also called the Advanced Window Toolkit), which we will simply refer to as the AWT. It is an Object Oriented Graphical User Interface (GUI) frame work that allows the application developer to design modern, mouse controlled, graphical application interfaces. A basic overview of this package is shown in the **Figure 7.1.** This package is so vast that it is not possible to discuss in this Chapter about each class and to illustrate their uses. However, we will pick up only few (those are marked in bold type) of them as a token and will be illustrated their application.

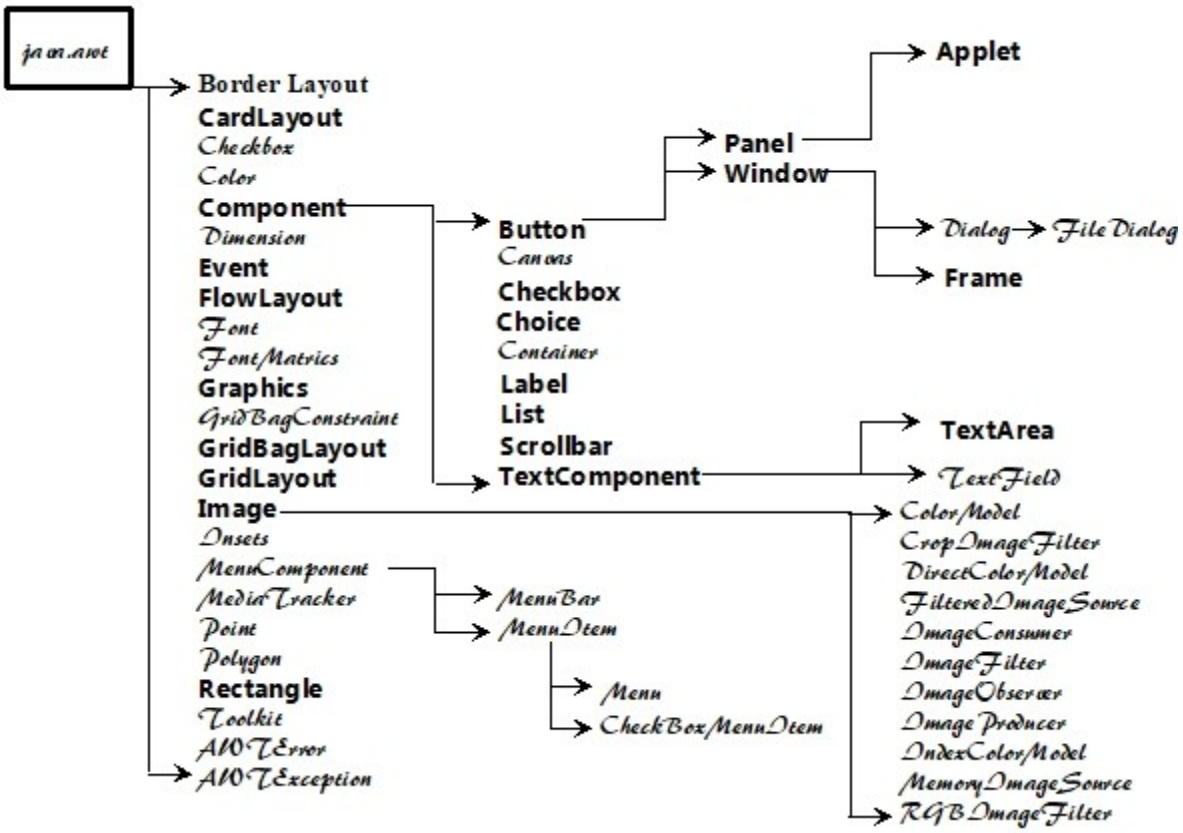


Figure 7.1 : Abstract Windowing Toolkit Package Hierarchy

Designing GUI with Components and Layout Managers

The Layout managers and Components are known to be very popular to Java GUI programming. The Components is a group of classes which belong to the class Component. These classes are very much powerful to deal with buttons, menus, sliders, text field and etc. On the other hand, Layout Managers is the group of classes which are important for managing layout within window in various ways. Following two subsections are to explore these two things.

Using Components for Java GUIs

We will restrict our discussion with the following basic components:

```
                        . Button          . Checkbox          . Choice
                        . Frame''         . Panel''''''''''. Scrollbar
                        . Label           . List              . TextField
                        . TextArea        ''''''''''''
''''
```

The main idea of using these classes is that to create the object of their type. Each class have their own constructors for initialization of the objects. When an object is created and initialized, it then can be placed on an applet by the add() method; this method is defined in their corresponding classes.

Following are 11 samples for illustrations of using these component. Readers are advised to test the illustrations.

```
Illustration 7.1 (A)                              //‘‘To create a button’‘//
          import java.applet.Applet;
          import java.awt.*;                                  // To import the Button class
          public class ButtonTest extends Applet {
                    public void init( ) {
                              Button’‘   b1,b2;                          // Create two Button objects
                              b1 = new Button ("Welcome");    // One button is initialized
                              add (b1);                       // Add it into the applet.
                              b2 = new Button (‘‘);                      // Second default button
                              add (b2);                       // Add second button to the applet.
                    ‘‘ }
          }
```

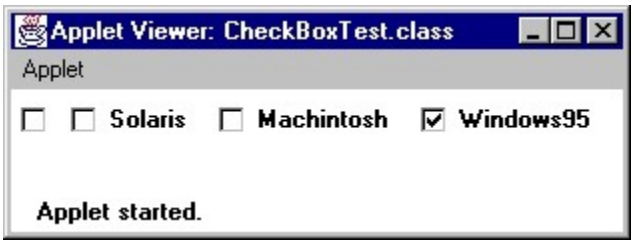


Note : Button class contains one default constructor where no argument has to be passed and a button will appear without any string label on it. One can change the label of a button at any time using the public void setLabel(String label) method defined in Button class. There is another function also defined as public String getLabel( ) to return the label of a button object.

```
Illustration’‘7.1 (B)                    // To create a check box’‘//

          import java.awt . *;           // To import the check bon class.
          import java.applet. *;
          public class CheckboxText extends’‘Applet           {
                    public void init ( ) {
                              Checkbox c1 = new Checkbox (‘‘);          ‘‘‘‘‘‘ // default constructor
without any label
                              Checkbox c2 = new Checkbox ( " Solaris 2.x");// Check box with default
layout
                              Checkbox c3 = new Checkbox ( "Macintosh", null, false);
                              Checkbox c4 = new Checkbox ( "Windows 95", null, true);
                              add (c1);
```

```
                              add (c2);
                              add (c3);
                              add (c4);
                    }
          }
```

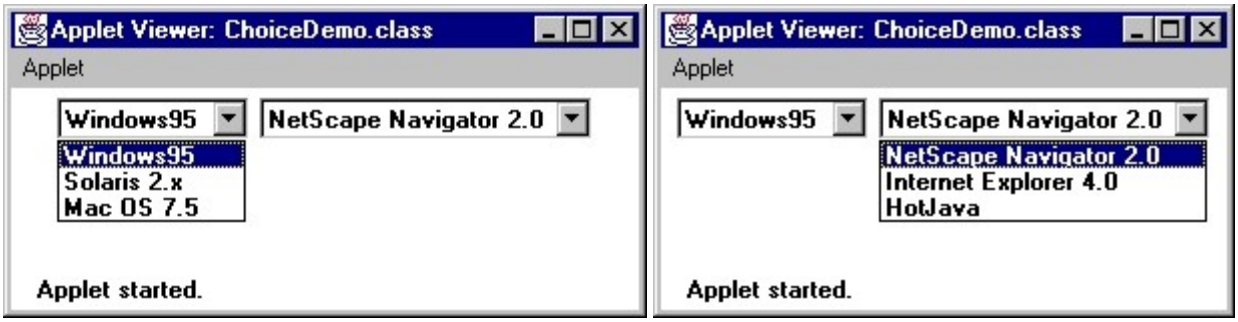


Note : This class has 3 constructors; one constructor with three arguments first for label, second for fencing type(default value is null) and third for initial state : false for deselected and true for selected.

```
Illustration 7.1 (C)                              //''To create Choice component'' //

          import java.awt.*;
          import java.applet.*;
          public class ChoiceDemo extends Applet {
                    public void init ( ) {
                              int width = Integer.parseInt (getParameter( " width" ));
                              int height = Integer.parseInt (getParameter(" height"));
                                                        // To get the width and height of the
applet.
                              Choice os = new Choice ( );    //''os is an instance of choice class.
                              Choice browser = new Choice ( );'' // browser is the another instance.
                              os.addItem ( " Windows 95" );
                              os.addItem ( " Solaris 2.x " );
                              os.addItem ( " Mac OS 7.5 ");
                              browser.addItem (" Netscape Navigator 2.0");
                              browser.addItem ( "Internet Explorer 4.0" );
                              browser.addItem ("Hot Java ") ;
                              add (os);
                              add (browser );
                              os.reshape ( 0, 0, width /2, height /2 );
                              browser.reshape (0, height /2, width, height);
                    }
```

```
        }
```



The Choice class is used for creating a pop-up selection menu. A Choice component is used to display menu item, each item in the menu is a String which appears as a left justified label in the order they are added to the Choice object. The Choice class uses addItem() to add new Strings to a Choice object. In the above example, two choice menus and their items are displayed. This time we have used the reshape() method to customarily place the choices on the applet view. It first two arguments are the left-top and remaining two argument are right-bottom coordinates.

```
Choice component can use other methods for selecting an item :


getSelectedItem (‘‘)                    Returns the string of the item itself
getSelectedIndex (‘‘)                   Return the index position of the item
select (int pos )               Selects an item by index position
select ( String name)                   Selects an item by name
```

In the Component class, there is Frame, a sub class of Window. This Frame class can be used to create a user defined frame. A frame is a window with a title and resize corners. The program given below creates a simple frame with a specific title, size and back ground color :

```
Illustration 7.1 (D)                    // To create a frame //
          import java.awt.*;
          public class MyFrame {
                    public static void main (String args[ ] ) {
                              Frame frame =‘‘new Frame ( " Frame in Java " );
                              frame.resize (500, 500);       ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ // component method to
resize the frame
                              frame.setBackground (Color.blue );’‘// Background of the frame will be blue
                              frame.show ( )                 ‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘// Method to show the frame
on the screen
                              }
                    }
```

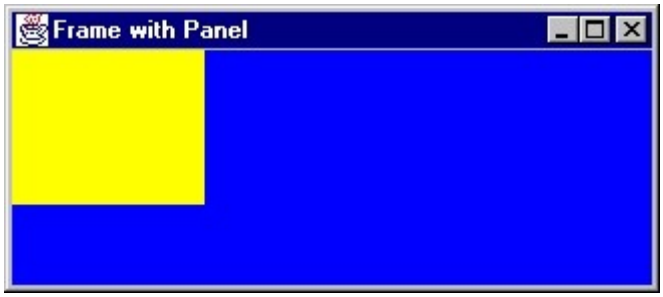
The program runs continuously. **To quit press CTRL+C.** It can be noted that, the show( ) and resize( ) methods must be called to make the frame visible.



Panels like frames, provide the space for you to attach any GUI component, including other panels. Panels are created using the constructor Panel(). Once a Panel object is created, it must be added to a Window or Frame object in order to be visible. This is done using the add() method of the container class. The following program creates a small yellow panel, and adds it to a Frame object :

```
Illustration'' 7.1 (E)                              // To create a panel //


          import java.awt.* ;
          public class MyPanel''{
                    public static void main ( String args [ ] )''{
                              Frame frame = new Frame( "Frame with panel");
                              Panel panel = new Panel('');
                              frame.resize(200, 200);
                              frame setBackground (Color.blue);
                              frame.setLayout (null);                    // override default layout
                              panel.resize (100, 100) ;
                              panel.setBackground (Color.yellow );
                              frame.add (panel);              //''add the panel into the frame
                              frame.show ( );                            //'' display the panel
                    }
          }
```

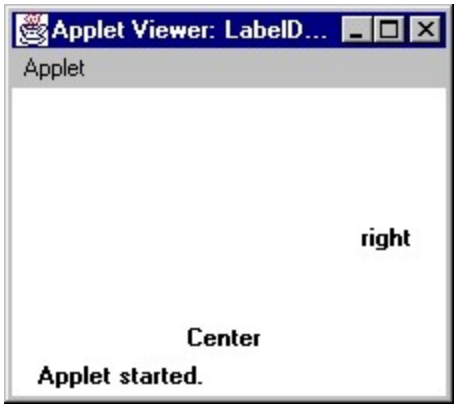


With the class **Label**, one can construct a label object with the string labeled that has to be displayed. An alternative constructor is provided to allow left, right, or center justification of the label text. Three constants LEFT, RIGHT, and CENTER are available for corresponding alignment. Here, is an example, that creates three labels, each with a different alignment.

```
Illustration 7.1 (F)                                //’‘‘‘To create label          //

          import.java.awt.*;
          import.java.applet.*;
          public class LabelDemo extends Applet’‘{
                    public void init ( ) {
                              setLayout (null );
                              int width = Integer.parseInt ( getParameter ("width" ));
                              int height = Integer.parseInt (get.Parameter (" height" ));
                              Label left = new Label ( );                         // default alighment is
left )
                              Label center = new label (‘‘" Center", Label.CENTER);’‘‘‘// For Centered
alignment
                              Label right = new Label("Right", Label.RIGHT); // for Right justified label

                              // Add the labels on the applet //
                              add(left);
                              add (right );
                              add (center );
                              // Place the label properly on the applet; otherwise deafault placing will
occur //
                              left.reshape ( 0, 0, width, height/3);
                              right.reshape(0, height/3, width, height/3);
                              center.reshape (0, 2 * hight /3 , width, height /3);
                    }
‘‘‘‘}
```



The **List** class provides a compact, multiple-choice, scrolling selection list. It can be noted that the Choice object shows only the single selected item in the menu, whereas, a List object can be constructed to show any number of

choices in the visible window. Following is an applet to display two selection lists, one multiple choice and other single choice.

Illustration 7.1 (G) **// To create a list //**

```
          import java.awt.*;
          import java.applet.*;
          public class ListDemo extends Applet’‘{
          ‘‘‘‘public void init ( ) {
                    setLayout (null );                         // It is a default Layout setting
                    int width = Integer.parseInt (getParameter (" width" ) );
                    int height = Integer.parseInt (getParameter (" height" ) );


                    List os’‘=‘‘new List (3, true);’‘                    // List object for list of Oss,
multiple selection
                    List browser’‘= new List (5, false);                 // List object for list of
Browsers, single choice

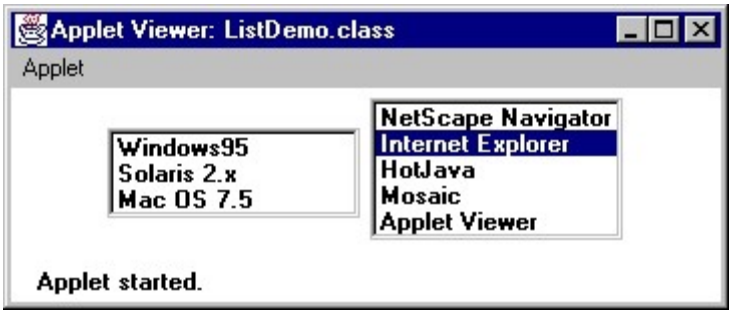

          // Add the Items into the lists
                    os.addItem ( " Windows 95" );
                    os.addItem ("Solaris’‘2.x " );
                    os.addItem (" Mac OS 7.5 " );
                    browser.addItem ("Netscape Navigator" );
                    browser.addItem ("Internet Explorer" );
                    browser.addItem ("Hot Java" );
                    browser.addItem ("Mosaic" );
                    browser.addItem ("Applet Viewer" );
                    browser.select(1) ;            // Second item in the Browser list is selected


          // Add and place the lists onto the applet
                    add(os);
                    add(browser);
                    os.reshape (0, 0, width,’‘ height /2 );
                    browser. reshape ( 0, height /2, width, height /2 );
                    }
          }
```

It can be noted that, the List class has two constructors namely :

```
public List(‘‘); and
public List ( int numberOfIemToDisplay, boolean multiSelectionChoice );
```

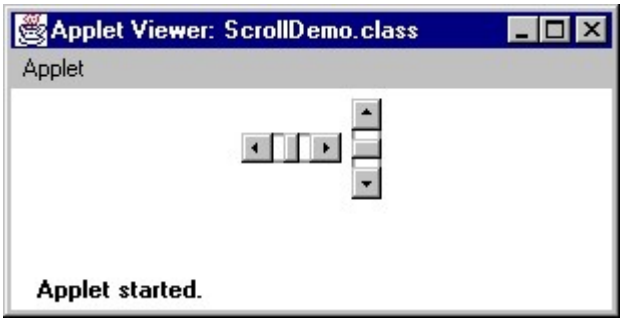
The first constructor is to create default list with no items and no selection, (when the list is created). Other constructor, comes in two varieties : one allows multiple selection with multiSelectionChoice as true; another with false behave exactly like a Choice such that it only accepts single (mutually exclusive) selection.



Scroll bars are used to select continuous values between a given minimum and maximum and one can slide over this range. The scroll bar is either in vertical or horizontal orientation. Following is an example that creates both a vertical and an horizontal scroll bar :

Illustration 7.1 (H) **// To create scroll bar’‘//**

```
          import java.awt.*;
          import java.applet.*;
          public class ScrollDemo extends Applet {
          ‘‘‘‘‘‘ public void init ( ) {
                              setLayout( new Borderlayout ( ) );                  // default border
layout setting
                              int width = Integer.parseInt (getParameter (" width" ));
                              int height = Integer.parseInt (getParameter (" height"));
                              Scrollbar hScroll = new Scrollbar ( Scrollbar. HORIZONTAL, 50, width /10,
0, 100 );
                              Scrollbar vScroll = new Scrollbar ( Scrollbar.VERTICAL, 50, hight /2, 0,
100);
                              add (hScroll );
                              add (vScroll );
                              }
          }
```



Note the numeric argument that have been passed to the Scrollbar constructor : First argument is to specify the kind of the scroll bar. Second argument dictate the starting position of the slider, third argument is to decide the proportion of the range that scroll bar will appear ie. 1/10th of the width of the applet for horizontal or ◆ of the height of the applet view for vertical scroll bar. The last two arguments specify the minimum and maximum range (for example, 1-12 for month , 1-31 for dates, 1 to 256 for colours etc.), we have taken 0 - 100 for something in both the scroll bar.

In the scroll bar class, therre are two methods vide :

setLineIncrement (int lineIncr); and setPageIncrement (int pageIncr);

with which one can specify the speed of the slider to be controlled to move one line or one page at a time on click on arrows at the ends in the scroll bar.
The **TextField** class can be used to implement a single line text input box. The constructors in this class allow you to create text fields of a specific character width and default values. The text is editable using keys and mouse selections. Also, this class can be used to accept text to echoing it or hide echoing just as in login and password entries. Following is an example towards the application of text field in an applet.

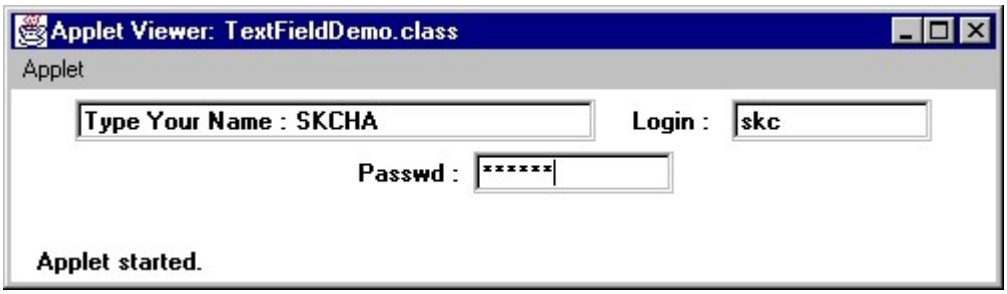
```
Illustration 7.1 (I)                                  // To create Text field'‘//

           import java.awt.*;
           import java.applet.* ;
           public class TextFieldDemo extends Applet          {
                    public void init ( ) {
                              add ( new TextField("Type your Name", 20);
                              // create a text field window of 20 character widths
                              Label login = new Label ( "Login : " , Label.LEFT );
                              Label pswd = new Label ( "Password : " , Label.CENTER );

                              TextField log'‘= new TextField (8) ;      // Text field for 8 characters
                              TextField'‘pas = new TextField (8);
                              pas.setEchoCharacter ( '*' );             // echo hide the type pass

                              add (login);'‘add (log);'‘                // add labels in the applet and
                              add (pswd );'‘add (pass );                //'‘add corresponding text fields
                    }
          }
```

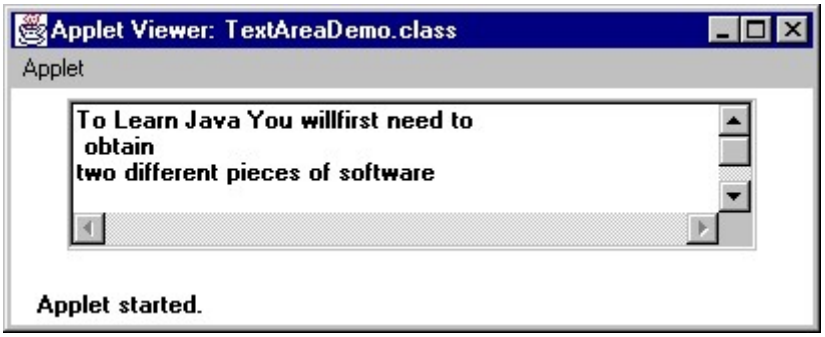
In the above example, note the use of setEchoCharacter() method which is used to mask the actual character on the screen.



The **TextArea** class, as its name suggests, is used when larger amount of text need to be input / displayed. Let us see the following illustration :

```
Illustration 7.1 (J)                                    // To create text area'‘//


          import java.awt.*;
          import java.applet.*;
          public class TextAreaDemo extends Applet {
                    TextArea text;
                    String multiLineText =
                    " To learn Java, you will first"
                              + " need to obtain \ n'‘ two different pieces of'‘"
                              + "softwares : \ n "
                              + " The first is the JDK (Java Development Kit"
                              + "and \n the second piece of software "
                              + "is a Java capable browser."
                              + " \n JDK software is needed for Writing, "
                              + " Compiling, and Testing Java applet \n and
                              + " Applications.\n \n" ;
                    public void init (‘‘) {
                               setLayout(null);
                               text = new TextArea (multiLineText, 20, 40);
                               add (text);
                    }
          }
```



Here, the constructor TextArea (String text, int rows, int columns) where rows and columns are to describe the rows and columns in the text area, to be displayed.

Using Layout Managers for Java GUI

So far we have discussed about various components and you can note that, where these components will be placed on the screen it has to specify explicitly. But this is very much tedious for laying large number of components. Thankfully, AWT includes the notion of Layout Managers : the layout of components in a container may be governed by a Layout Manager. A Container in Java is nothing but an applet which generally contains number of GUI attributes such as Frame, Panel, Label, Button etc. (which are individually known as Component). Each Container object has a Layout Manger which is an instance of any class that implements the Layout Manager interface. Thus, each component, such as a Panel, or a Frame has a default Layout Manager associated with it which may be changed by the Java developer when an instance of that container is created. Each Layout Manager keeps track of a list of Components. The Layout Manager is notified each time one adds a Component to a Panel, for example

```
Java has the following five predefined Layout Manager classes :

          . FlowLayout

          . BorderLayout

          . GridLayout

          . GridBagLayout

          . CardLayout.
```

Let us discuss about each of them
The most basic Layout Manager is the FlowLayout manager. The manager implements a simple layout style which is similar to how words flow in a text editor. Components are laid out from the upper-left corner, left to right and top to bottom. When no more components fit on a line, the next one appears on the next line. A small space is left between each component, above and below, as well as left and right; however, user can change this gap, alignment etc. The default gaps are five pixels and the default alignment is CENTER.
‘‘The applet example below adds several Labels to a Panel which is using FlowLayout with the justification set to RIGHT.

```
Illustration 7.2 (A)                                  //’‘‘‘Flow layout Manager’‘ //


          import java.awt.*;

          import java.applet.*;

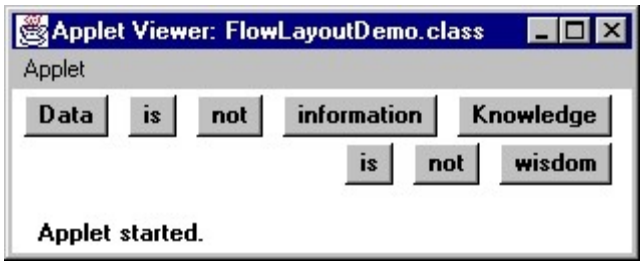
          import java.util.*;

          public class FlowLayoutDemo extends Applet {

                    public void init ( )           {

                              setLayout (new FlowLayout (FlowLayout.RIGHT, 10, 3));
```

```
                              String val =‘‘" Data is not Information" +
                                                  " is not knowledge is not wisdom";
                              StringTokenizer str = new StringTokenizer (val );’‘            // Read a
token from the String val
                              while (str. hasMoreTokens ( ) ) {
                                        add (new Button (str.nextToken( ) ) );’‘// For one token add a
label into the panel
                              }
                    }
          }
```



In the above example, the FlowLayout manager is invoked using the setLayout() method inherited from Component class. There are three forms of this method :

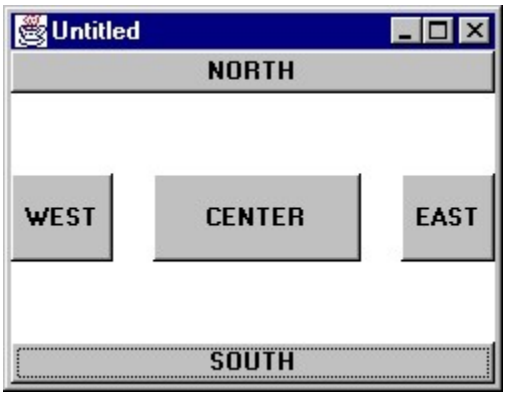
```
                    setLayout (new FlowLayout ( ) ;                              // with default value
                    setLayout (new FlowLayout (FlowLayout.alignment));  // with user defined alignment
                    setLayout (new FlowLayout (FlowLayout.alignment, int’‘hskip , int vskip ) );
                              ‘‘‘‘// with alignment, horizontal and vertical skips setting
```

alignment is LEFT, RIGHT or CENTER, hskip and vskip are the spaces between two components horizontally and vertically respectively.
The BorderLayout Manager implement a common layout style which has five zones - each of these zones is named with a string North, South, East, and West represent the four sides, and Center is the middle area.
Here is an example of BorderLayout with component in each layout area in a frame.

```
          import java.awt.*;
          import java.applet.*;
          import java.util.*;
          public class BorderLayoutDemo extends Applet {
                    public void intit ( ) {
                              Frame frame = new Frame ( );
                              frame.setLayout ( new BorderLayout (20,40));
                              Button’‘buttonN, buttonS, buttonE, buttonW,’‘buttonC;

                              buttonN = new Button ( " NORTH " );
```

```
                              buttonS'‘= new Button ( " SOUTH" );
                              buttonE = new Button ( " EAST" );
                              buttonW = new Button (" WEST" );
                              buttonC = new Button("CENTER");
                              frame.resize (250,250);        // resize the frame is height = 250 =pixels
, width 250 pixels
                              frame.add ( "North", buttonN );
                              frame.add (" South ", buttons );
                              frame.add ( "East", buttonE );
                              frame.add (" West" , buttonW );
                              frame.add("Center",buttonC);
                              frame.show ( );
          ‘‘‘‘}
‘‘‘‘}
```



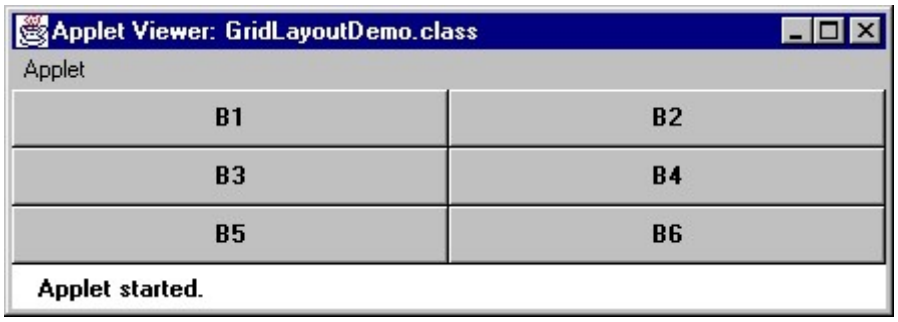
Note : The coding difference for add() method can be noted. Using FlowLayout, the method was add(Component) but with BorderLayout, it is add(String position, Component comp), an overloaded variant.
Again, Border layout relies on the position String being case-sensitive. Passing "NORTH" or "north" instead of "North" will not work.

The **GridLayout manager** provides flexibility for placing components. You create the manager with a number of rows and columns. Components then fill up the cells defined by the manager For example, the following **Illustration 7.2(C)** is to create an applet with three rows and three columns and containing 6 buttons :

```
Illustration 7.2 ( C )                              // Grid Llayout Manager //


          import java.awt.*;
          import java.applet.Applet;
          public class GridLayoutDemo extends Applet {
                    Button b1,b2,b3,b4,b5,b6 ;
```

```
                    public void init ( ) {
                              setLayout (new GridLayout (3,2) ) ;                // 3 rows, 2 columns
                              b1 = new Button ("B1");
                              b2 = new Button ("B2");
                              b3 = new Button ("B3" );
                              b4 = new Button ("B4" );
                              b5 = new Button ("B5" );
                              b6= new Button ("B6" );
                              add (b1); add (b2); add (b3);
                              add (b4); add (b5); add (b6);
                    }
          }
```



GridBagLayoutManager

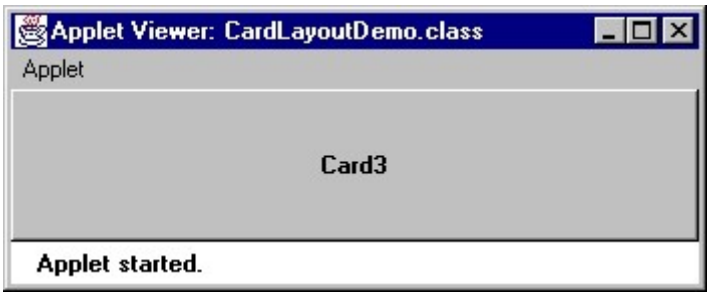
GridBagLayoutManager is the more flexible analog of GridLayoutManager. With this manager you can specify width, height etc. of individual grids in a container. It is not possible to illustrate this manager with a small example.

The **Card Layout Manager** is unique from the other layout managers in the sense that it represents several different layouts that can be thought of as living on separate index cards that can be shuffled so that any one card is on top at a given time. This can be useful for user interfaces that have optional components which can be dynamically enabled and disabled upon user input. The following program illustrates the use of CardLayout with five cards containing a single button each.

```
Illustration 7.2 (D)                                   // Card layout Manager'‘//


          import java.awt.*;
          public class Crads extends java.applet.aApplet {
                    CardLayout'‘ layout;
                    public void init ( ) {
                              layout = new CardLayout ( ) ;
                                        setLayout (layout );
```

```
                                        add ("1", new Button ("Card1" ));
                                        add ("2", new Button ("Card2" ));
                                        add ("3", new Button ("Card3" ));
                                        add ("4", new Button ("Card4" ));
                                        add ("5", new Button ("Card5" ));
                                ‘‘ }
                                public boolean key Down (Even e int key ){
                                        layout next (this);
                                        return (true );
                                        }
        ‘‘‘‘}
```



Note : The add() method used here to add cards to a CardLayout, and you have to use the labeled variety to pass a card name; in the above example 1,2 ��etc. are the name for five cards. Later, one can refer to cards by this name. Also note that a CardLayout variable (i.e. layout) was created. This is because, we actually need to refer to our CardLayout object after we have created it.
To demonstrate the ability to cycle through the cards, this applet was made to listen to the key board and select the next card whenever you press a key. This is a mechanism of event handling which is our next topic of discussion

Event Handling

All of the components we have created so far have looked right, but they have been essentially useless since we never talked about how to process the user input to these user interface controls. Suppose, there are a number of buttons in a container and each button is corresponding to a particular event. Thus, if a user selects a button then the system should respond accordingly and this type of GUI programming (this is why it is also known as Event Driven programming). In this very section we will explore, how Java provides the programmers event handling capabilities.

In the context of Java events, an event is an asynchronous data generated by the window server and results from some device activity. The devices are usually mouse and keyboard. Examples of event include:

Windows events - Destroying , exposing, iconifying, deiconifying and moving windows.
Mouse events - Moving the mouse, dragging the mouse, having the mouse enter or exit a component, pressing a mouse button, and releasing a mouse button.
Keyboard events - pressing a key, and releasing a key.
List events - selecting or deselecting a list item.
Scrolling events - scrolling a line up or down, a page up or down, or scrolling to an exact location.

For all the above type of events, Java provides event handling method which is defined in Event class under AWT. In fact, for each kind of Component object in AWT, there is a handleEvent() method. When a device activity occurs that generate an event, the event is first sent to the component in which call to the event handler of the Component class.

The event handler there after returns a boolean value either false or true depending on the event handling routine. If it returns false, the event is passed on to the component's parent (container). If the method returns true, it means that the component acted on the event and no more processing of the event takes place. Next we will see, what are the various event handling methods in Event class and their practical uses.

Event handler use number of information regarding the activity that took place. This information variables are defined in Event class. When an event occurs, the corresponding event object is instantiated and these variables are initialized. The variables includes :

```
Object target        :‘‘the component that originally received the event
long when’‘                    :‘‘the time stamp of when the event occurred
int id’‘             :‘‘the type of this event
int X’‘              ‘‘‘‘‘‘     :‘‘the X-coordinate of the event
int Y                ‘‘‘‘‘‘     :‘‘the Y-coordinate of the event
int key    ‘‘‘‘‘‘    :‘‘the key’‘that was pressed in a key board event
int modifier’‘‘‘‘‘‘   :‘‘the state of’‘the modifier key (Shift, Control, Alt ).
int clickCount       :‘‘the number of consecutive clicks
Object arg           ‘‘‘‘‘‘     :‘‘an event dependent argument, that helps to identify the event causing
GUI element (e.g.’‘the label of a’‘button etc).
```

The different event types possible are listed below :

```
Windows events :

WINDOW_ICONIFY, WINDOW_ DEICONIFY, WINDOW_MOVED, WINDOW_DESTROY, WINDOW_EXPOSE.These events occurs when
the corresponding activity is performed on a window object.


Mouse events :

‘‘‘‘MOUSE_ENTER, MOUSE_EXIT, MOUSE_MOVE, MOUSE_DRAG, MOUSE_UP, MOUSE_DOWN .These are generated by mouse
actions within the borders of’‘Canvas, Frame, and Panel components.


Focus events :

          GOT_FOCUS, LOST_FOCUS Generated as the mouse enters or exits the borders of a Canvas, Frame, or
Panel Component. They are also generated as you click a mouse in a Text Component.


Keyboard events :

          KEY_PRESS, KEY_ACTION, KEY_RELEASE, KEY_ACTION_RELEASE Generated by pressing or releasing keys
on the key board when you are in a Frame, Canvas, Panel or Text Component.


Action event :

‘‘‘‘ACTION_EVENT Generated when you click on a Button, Choice, or CheckBox Component, Press Enter in a
TextField, or double click on a List item.


Scroll bar events :

‘‘‘‘SCROLL_LINE_UP, SCROLL_LINE_DOWN, SCROLL_PAGE_UP, SCROLL_PAGE_DOWN, SCROLL_ABSOLUTE .Generated by
actions’‘involving the scroll bar in Scrollbar Component.
```

The id variable in the Event class will be initialized by the corresponding event type during the automatic instantiation of respective Event object.

Now, there are a number of event handling methods declared in Component class and Event class. The method handleEvent(Event evt) in Component class is the central event dispatcher for a component. This method for a given event type (evt.id) calls the corresponding method. These methods are defined in Component class and summarized in Section 8.4 of this Chapter:

However, all these method can be called exclusively in applets/Application with the overridden code.

evt . id	handleEvent calls
Event. MOUSE_ DOWN	mouseDown (Event e, int x, int y)
Event MOUSE_ UP	mouseUp (Event e, int x, int y)
Event MOUSE_DRAG	mouseDrag (Event e, int x int y)
Event MOUSE_ ENTER	mouseEnter (Event e, int x int y)
Event MOUSE - EXIT	mouseExit (Event e, int x, int y)
Event KEY_PRESS	keyDown (Event e, int key),
Event KEY_ACTION	key Down (Event e, int key)
Event KEY_RELEASE	keyUp (Event e, int key)
Event KEY_ACTION_RELEASE	key Up (Event e, int key)
Event LOST_FOCUS	lostFocus (Event e, Object o)
Event GOT_ FOCUS	getFocus (Event e, Object o)
Event ACTION_EVENT	action (Event e, Object o)

It can be noted that, there are few events which are generated by List and Scrolllbar components that do not have their helper functions. To handle these, it is required to override the handleEvent() method.

As an illustration let us consider the following example (Illustration 7.3) which creates 15-cells GridLayout and then the use of even handler namely action().

Illustration 7.3

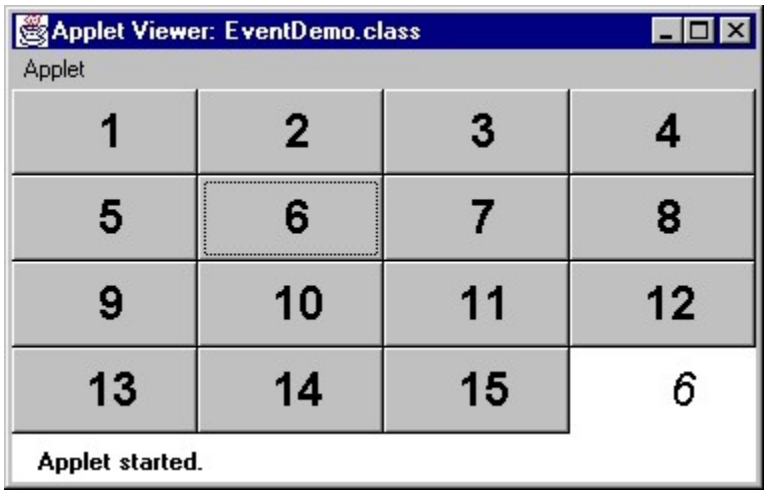
```
                                        // 15 puzzle - use of even handling methods'‘//

          import java.awt.*;
          import java.applet.*;
          public class EventDemo extends Applet {
                    static final int n = 4;
                    Label label;
                    public void init(‘‘) {                              // To initialize the applet for
16-cells grid
                              setLayout (new GridLayout (n,n);                  // To for 4x4 cells
grid
                              setFont (new Font ("Helvetica",’‘Font.BOLD, 24 ));’‘ // Font of Helvetica
of size 24
                              for (int i =0; i < n; i++ )    {
                                        for (int j = 0; j < n; j++ ) {
                                         int k = i * n + j;
                                                   ‘‘ if (k > 0)
                                         add (new Button ("‘‘" + k);’‘‘‘‘‘// Place a button at
(i,j)-th cell
                                        }
```

```
                                }
                                label = new Label (" * " , Label.CENTER );
                                label.setFont (new FONT (" Times Roman", Font.ITALIC, 24 ));
                                add (label );
                        }

                        public boolean action(Event e, Object obj) {        ‘‘‘‘‘‘ // Overridden event
handler
                                        if (e.target instanceof’‘Button ) ) {     ‘‘//’‘Wait for clicking
a button object
                                                label.setText ((String ) obj );’‘// Print the value of
the Clicked butoon
                                        }
                                        return false ;
                                }
          }
```

Whenever the applet EventDemo runs the event handler, namely, action(Event e, Object obj) will come into play till the user selects a button in the GridLayout, and it then returns false.



Note : Each event handling method returns boolean and they are in general public.

Drawing methods for Graphics objects

Java provides a good library for drawing various graphics object. In Java, Graphics objects (from the java.awt.Graphics class) have a method for displaying text, picture attributes and several other which are listed below :

```
public void drawLine (x1, y1, x2, y2 );
public void drawRect (x,’‘y,’‘width, height );
```

```
public void fillRect (x, y,’‘width, height );
public void clearRect (x,y, width,’‘height );
public void drawRoundRect (x, y, width, height’‘arcWidth, arcHeight );
public void fillRoundRect (x, y, width, height’‘arcWidth, arcHeight );
public void draw3DRect (x, y, width, height, boolean raised );
public void fill3DRect (x, y, width, height, boolean raised );
public void drawOval (x, y, width, height );
public void’‘fillOval (x, y, width, height );
public void drawArc (x, y, width, height’‘startAngle, arcangle );
public void fillArc (x, y,’‘width, height’‘startAngle, arcAngle );
public void drawPolygon (int [ ], xPoints, int [ ] yPoints );
public void drawString ( String s, x, y);
public void drawChars (char data [ ], offset, length, x, y );
public void drawBytes ( byte data [ ], offset, length, x, y);
public void drawBytes ( byte data [ ], offset, length, x,y );
```

Following is the illustration 7.4 to use few of these methods in an applet.

Illustration 7.4 **// Graphics Test’‘ //**

```
          import java.awt.*;
          import java.applet.*;
          public class GraphicsText extends Applet  {
                    public void init (‘‘) {
                              public void paint (Graphics g) {
                                        Dimension d = size ( ) ;                 // returns the width
and height’‘of the applet
                                        g.setColor (Color.black );               // set the drawing
color as black
                                        g.drawRect (0, 0, d.width-1, d.height -1);
                                        g.setColor (Color.blue);
                                        g drawRect (10, 10, d.width-10, d.height-10 );’‘ // draw another
inner blue Rectangle
                                        g.setColor (Color.red );
                                        g.drawOval (15,15, 50,50 );
                                        g.setColor( Color.yellow );
```

```
                                        g.fillOval ( 20, 20,40, 40 );
                                        g.setColor (Color.red );
                                        g.drawArc (230, 10, 50, 50, -90, 270 );
                                        g.fillArc (190, 10, 50, 50, -90, 270 );
                                        g.setColor (Color.pink );
                                        g.drawRoundRect (130, 70, 50, 50, 10, 10 );
                                        g.fillRoundRect (190, 70, 50, 50, 10, 10 );
                                        int [ ] xPoints'‘=   { 10,70,120,120,70,10,10 };
                                        int [ ] yPoints'‘=   { 130, 150, 130, 180, 170, 180, 130 };
                                        Color myColor = new Color (255,100,150);'‘‘‘// set a RGB color
                                                                        // A color can be choosed, R, G,
B'‘integers ranges between'‘0 to 255
                                        g.setColor (myColor );
                                        g.drawPolygon (xPoints , yPoints, 7);     // To draw a polygon
with 7 points
                                        g.setColor (Color.gray );
                                        g.draw3DRect (10, 190, 50, 50, true );
                                        g fill3DRect (70, 199, 50, 50, true);
                                        g.setColor (Color.green );
                                        g.draw3DRect (130, 190, 50, 50, false );
                                        g.fill3DRect (190, 190, 50, 50, false );
                    '‘‘‘‘‘‘}
          '‘‘‘ }
          }
```

Graphics objects are also capable of displaying images; there is a number of methods in Component and Applet class dealing with image. Two methods are mentioned here :

getImage (URL url, String s); Returns the image file img (say) having names and located at url. For stand alone machine, url is getDocumentBase( ).

drawImage (Image img; int x, int y, ImageObserver'‘observer);'‘‘‘This method requires an Image object and an ImageObserver observer.

Following is an example, which retrieves an image and displays it on an applet.

Illustration 7.5

```
                                        // Image display'‘//

          import java.awt.*;
          import java.applet.Applet;
          public class ImageDemo extends Applet'‘{
```

```
                    Image myImage;
                    public void init ( ) {
                                        myImage = getImage (getDocumentBase ( ), "graphics/title.gif
" );
                    }
                    public void paint (Graphics g) {
                              g.drawString (myImage, 25, 25, this);
                    }
          }
```



Here, we assume that an image file tille.gif is located in a directory graphics in the applet running machine. The getDocumentBase( ) method returns the URL url of the current machine.

The Graphics class is also very powerful handle string/font in several ways. But discussion of this is beyond the scope of this book.

Practice Questions

Practice 7.1

This example shows how an applet code retrieve the value of parameters from .html file.

```
          AppletParameter.java



          import java.applet.*;
          import java.awt.*;

          public class AppletParameter extends Applet {
          private String strDefault = "Hello! Java Applet.";
          public void paint(Graphics g) {
          String strParameter = this.getParameter("Message");
          if (strParameter == null) strParameter = strDefault; g.drawString(strParameter, 50, 25);
          }
          }
```

AppletParameter.htm

```
< HTML>
< HEAD>
< TITLE>Passing Parameter in Java Applet

< BODY>
This is the applet:< P>
< APPLET code="AppletParameter.class" width="800" height="100">
< PARAM name="message" value="Welcome in Passing parameter in java applet example.">
< / APPLET>
```

Practice 7.2

This example shows how to draw the different types of shapes like line, circle and rectangle.

Java code of the program :.

```
import java.applet.*;
import java.awt.*;

public class         CircleLine extends Applet{
int x=300,y=100,r=50;

public void paint(Graphics g){ g.drawLine(3,300,200,10); g.drawString("Line",100,100);
g.drawOval(x-r,y-r,100,100); g.drawString("Circle",275,100); g.drawRect(400,50,200,100);
g.drawString("Rectangel",450,100);
}
}
```

HTML code of the program:

```
< HTML>
< HEAD>

< BODY>
< div align="center">
< APPLET CODE="CircleLine.class" WIDTH="800" HEIGHT="500">
< / div>
< / BODY>
< / HTML>
```

Practice 7.3

This program illustrates how to draw the different types of colorful shapes like line, circle and the rectangle.

Java code of the program

```
import java.applet.*;
import java.awt.*;

public class         ShapColor extends Applet{
int x=300,y=100,r=50;

public void paint(Graphics g){
g.setColor(Color.red);         //Drawing line color is red g.drawLine(3,300,200,10);
g.setColor(Color.magenta);
g.drawString("Line",100,100);

g.drawOval(x-r,y-r,100,100);
g.setColor(Color.yellow);      //Fill the yellow color in circle
g.fillOval( x-r,y-r, 100, 100 );
g.setColor(Color.magenta);
g.drawString("Circle",275,100);

g.drawRect(400,50,200,100);
g.setColor(Color.yellow);      //Fill the yellow color in rectangel

g.fillRect( 400, 50, 200, 100 ); g.setColor(Color.magenta); g.drawString("Rectangel",450,100);
}
}
```

HTML code of the program:

```
< HTML>
< HEAD>
< / HEAD>
< BODY>
< div align="center">
< APPLET ALIGN = "CENTER"      CODE = "ShapColor.class" WIDTH = "800" HEIGHT =
"500" >
< / div>
< / BODY>
< / HTML>
```

Practice 7.4

This program illustrates how to create choice in applet

Java code of the program

```
import java.awt.*;
import java.applet.*;
public class ChoiceDemo extends Applet {
‘‘‘‘public void init ( ) {
          int width = Integer.parseInt (getParameter("width" ));
          int height = Integer.parseInt (getParameter("height"));

          Choice os = new Choice ( );
          Choice browser = new Choice ( );’‘‘‘
          os.addItem ( " Windows 95" );
          os.addItem ( " Solaris 2.x " );
          os.addItem ( " Mac OS 7.5 ");
          browser.addItem (" Netscape Navigator 2.0");
          browser.addItem ( "Internet Explorer 4.0" );
          browser.addItem ("Hot Java ") ;
          add (os);
          add (browser );
          os.reshape ( 0, 0, width/2, height/2 );
          browser.reshape (0, height/2, width, height);
‘‘‘‘}
}
```

HTML code of the program:

```
< HTML>
< HEAD>

< BODY>
< div align="center">
< APPLET ALIGN = "CENTER"      CODE = "ChoiceDemo.class" WIDTH = "800" HEIGHT = "500">
< PARAM name="width" value="400">
< PARAM name="height" value="500">
< / APPLET>
< / div>
< / BODY>
< / HTML>
```

Practice 7.5

This program illustrates how to create label

Java code of the program

```
import java.awt.*;
import java.applet.*;

public class LabelDemo extends Applet’‘{
‘‘‘‘public void init ( ) {
          setLayout (null );
          int width = Integer.parseInt (getParameter ("width" ));
          int height = Integer.parseInt (getParameter ("height"));
          Label left = new Label();                                        ‘‘
          Label center = new Label("Center", Label.CENTER);’‘‘‘
          Label right = new Label("Right", Label.RIGHT);                   ‘‘

          add(left);
‘‘        add(right );
          add(center );

          left.reshape ( 0, 0, width, height/3);
          right.reshape(0, height/3, width, height/3);
          center.reshape (0, 2 * height/3 , width, height/3);
‘‘‘‘}
}
```

HTML code of the program:

```
< HTML>
< HEAD>
< TITLE>Passing Parameter in Java Applet< / TITLE>
< / HEAD>
< BODY>
This is the applet:< P>
< APPLET code="LabelDemo.class" width="800" height="100">
< PARAM name="width" value="200">
```

```
< PARAM name="height" value="300">
< / APPLET>
< / BODY>
< / HTML>
```

Practice 7.6

This program illustrates how to create text field

Java code of the program

```
import java.awt.*;
import java.applet.* ;
public class TextFieldDemo extends Applet           {
‘‘ public void init ( ) {
                      add(new TextField("Type your Name", 20));

                Label login = new Label ( "Login : " , Label.LEFT );
                Label pswd = new Label ( "Password : " , Label.CENTER );

                TextField log’‘= new TextField (8) ;
                TextField’‘pas = new TextField (8);
                pas.setEchoCharacter ('*');    ‘‘‘‘‘‘‘‘‘‘‘‘‘ // echo hide the type pass

                add (login);’‘add (log);’‘
                add (pswd);’‘add (pas);                  ‘‘
‘‘ }
}
```

HTML code of the program:

```
< HTML>
< HEAD>
< TITLE>Passing Parameter in Java Applet< / TITLE>
< / HEAD>
< BODY>
This is the applet:< P>
< APPLET code="TextFieldDemo.class" width="800" height="100">
< / APPLET>
< / BODY>
```

```
          < / HTML>
```

Practice 7.7

This program illustrates Flow layout manager

Java code of the program

```
          import java.awt.*;
          import java.applet.*;
          import java.util.*;

          public class FlowLayoutDemo extends Applet {
                    public void init ( )           {
                    setLayout (new FlowLayout (FlowLayout.RIGHT, 10, 3));
                    String val =‘‘" Data is not Information" +
                                                  " is not knowledge is not wisdom";
                    StringTokenizer str = new StringTokenizer (val );’‘

                    while (str. hasMoreTokens ( ) ) {
                              add (new Button (str.nextToken( ) ) );’‘
                              }
                    ‘‘}
          }
```

HTML code of the program:

```
          < HTML>
          < HEAD>
          < TITLE>Passing Parameter in Java Applet< / TITLE>
          < / HEAD>
          < BODY>
          This is the applet:< P>
          < APPLET code="FlowLayoutDemo.class" width="800" height="100">
          < / APPLET>
          < / BODY>
          < / HTML>
```

Practice 7.8

This example shows an Applet program which performs Addition, Subtraction, Multiplication and Division operations. To do these operations, this applet implements ActionListener interface and use four buttons for four operations.

Java code of the program:

```
import java.applet.*;
import java.awt.event.*; import java.awt.*;

public class EventListeners extends Applet implements ActionListener{ TextArea txtArea;
String Add, Subtract,Multiply,Divide;
int i = 10, j = 20, sum =0,Sub=0,Mul = 0,Div = 0;

public void init(){
txtArea = new TextArea(10,20);
txtArea.setEditable(false);
add(txtArea,"center");
Button b = new Button("Add");
Button c = new Button("Subtract");
Button d = new Button("Multiply");
Button e = new Button("Divide");
b.addActionListener(this); c.addActionListener(this); d.addActionListener(this);
e.addActionListener(this);

add(b); add(c); add(d); add(e);
}



public void actionPerformed(ActionEvent e)
{
sum = i + j;
txtArea.setText("");
txtArea.append("i = "+ i + "\t" + "j = " + j + "\n"); Button source = (Button)e.getSource();
if(source.getLabel() == "Add"){
txtArea.append("Sum : " + sum + "\n");
}

if(i >j){
Sub = i - j;
}
else{
Sub = j - i;
}
if(source.getLabel() == "Subtract"){
```

```
txtArea.append("Sub : " + Sub + "\n");
}

Mul = i*j;



if(source.getLabel() == "Multiply"){
txtArea.append("Mul = " + Mul + "\n");
}

if(i > j){
Div = i / j;
}
else{
Div = j / i;
}

if(source.getLabel() == "Divide"){
txtArea.append("Divide = " + Div);
}
}
}
HTML code of the program:

< HTML>
< BODY>
< APPLET CODE ="EventListeners" WIDTH="800" HEIGHT="500">< / APPLET>
< / BODY>
< / HTML>
```

Practice 7.9

This example shows an interactive applet for summing two numbers.

Java code of the program:

```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;

public class InteractiveApplet extends Applet implements Runnable, ActionListener {
‘‘‘‘TextField inputA, inputB;
```

```
          String s;
          int x = 0; int y = 0; int z = 0;
‘‘‘‘Button b;

          public void init ( ) {
          inputA = new TextField(8);
          inputB = new TextField(8);
          b = new Button("Add");
          b.addActionListener(this);

          add(inputA); add(inputB); add(b);
          inputA.setText("0");’‘ inputB.setText("0");

          Thread t = new Thread(this);
‘‘‘‘t.start();
‘‘‘‘}

          public void actionPerformed(ActionEvent e)
‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘‘‘‘‘s = inputA.getText(); x = Integer.parseInt(s);
‘‘‘‘‘‘‘‘‘‘‘‘‘‘s = inputB.getText(); y = Integer.parseInt(s);
                              ‘‘z=x +y;
           }
          public void run () {
          ‘‘‘‘‘‘‘‘for(;;)
                              {
                              try{
                              Thread.sleep(100);
                              }
                              catch(Exception e){}
          ‘‘‘‘‘‘‘‘repaint();
                              }
‘‘‘‘‘‘ }

‘‘‘‘public void paint(Graphics g) {
‘‘‘‘‘‘‘‘
‘‘‘‘‘‘‘‘g.drawString("Enter two values", 10,50);
‘‘‘‘‘‘‘‘s = String.valueOf(z); g.drawString("The sum is :",10,75);
‘‘‘‘‘‘‘‘g.drawString(s, 100,75);
                    ‘‘
```

```
‘‘‘‘ }
}

HTML code of the program:

< HTML>
< HEAD>
< TITLE>Passing Parameter in Java Applet< / TITLE>
< / HEAD>
< BODY>
This is the applet:< P>
< APPLET code="InteractiveApplet.class" width="800" height="100">
< / APPLET>
< / BODY>
< / HTML>
```

Practice 7.10

This program illustrates displaying an image in an applet

Java code of the program:

```
import java.applet.*;
import java.awt.*;

public class AppletImage extends Applet{ Image img;
MediaTracker tr;
public void paint(Graphics g) {
tr = new MediaTracker(this);
img = getImage(getCodeBase(), "sample.JPG");
tr.addImage(img,0);
g.drawImage(img, 0, 0, this);
}
}

HTML code of the program:
< HTML>
< HEAD>
< / HEAD>
< BODY BGCOLOR="000000">
< CENTER>
< APPLET
```

```
code      = "AppletImage.class"
width = "500"
height    = "300">
< / APPLET>
< / CENTER>
< / BODY>
< / HTML>
```

Note: This example assumes that sample.JPG exists in the source folder

Practice 7.11

This program shows how to display the banner in applet.

Java code of the program:

```
import java.awt.*;
import java.applet.*;

public class SampleBanner extends Applet implements Runnable{ String str = "This is a simple
Banner ";
Thread t ;
boolean b;
public void init() {
setBackground(Color.gray);
setForeground(Color.yellow);
}

public void start() {
t = new Thread(this);
b = false;
t.start();
}

public void run () {
char ch;
for( ; ; ) {
try {
repaint();
Thread.sleep(250);
ch = str.charAt(0);
str = str.substring(1, str.length());
```

```
str = str + ch;
} catch(InterruptedException e) {}
}
}
public void paint(Graphics g) { g.drawRect(1,1,300,150); g.setColor(Color.yellow);
g.fillRect(1,1,300,150); g.setColor(Color.red); g.drawString(str, 1, 150);
}
}
```

HTML code of the program:

```
< HTML>
< BODY>
< APPLET CODE = "SampleBanner" WIDTH = "500" HEIGHT = "300">< / APPLET>
< / BODY>
< / HTML>
```

Practice 7.12

This example displays the time in an applet in the time format like: hours, minutes and then seconds (hh:mm:ss).

Java code of the program:

```
import java.applet.*; import java.awt.*; import java.util.*;

public class ClockApplet extends Applet implements Runnable{ Thread t,t1;
public void start(){
t = new Thread(this);
t.start();
}

public void run(){
t1 = Thread.currentThread();
while(t1 == t){ repaint(); try{
t1.sleep(1000);
}catch(InterruptedException e){}
}
}

public void paint(Graphics g){
Calendar cal = new GregorianCalendar();
```

```
            String hour = String.valueOf(cal.get(Calendar.HOUR));
            String minute = String.valueOf(cal.get(Calendar.MINUTE));
            String second = String.valueOf(cal.get(Calendar.SECOND));
            g.drawString(hour + ":" + minute + ":" + second, 20, 30);
            }
            }

            HTML code of the program:
            < HTML>
            < BODY>
            < div align = "center">
            < APPLET CODE = "ClockApplet" WIDTH = "500" HEIGHT = "400">< / APPLET>
            < / div>
            < / BODY>
            < / HTML>
```

Practice 7.13

This applet shows a rectangle that will change color when the mouse moves over it.

Java code of the program:

```
import java.awt.*;
import java.applet.*;
import java.awt.event.*;

public class MouseMotionExample extends Applet implements MouseMotionListener
{
 // The X-coordinate and Y-coordinate of the last Mouse Position.
‘‘‘‘ int x;
‘‘‘‘ int y;

‘‘‘‘ int rect1x,rect1y,rect1width,rect1height;
‘‘‘‘ boolean rect1Active;

‘‘‘‘ public void init()
‘‘‘‘ {
‘‘‘‘‘‘‘‘‘rect1x = 20;
```

```
‘‘‘‘‘‘‘‘‘‘rect1y = 20;
‘‘‘‘‘‘‘‘‘‘rect1width =100;
‘‘‘‘‘‘‘‘‘‘rect1height= 80;
‘‘‘‘‘‘‘‘‘‘addMouseMotionListener(this);
‘‘‘‘ }

‘‘‘‘ public void paint(Graphics g)
‘‘‘‘ {
‘‘‘‘‘‘‘‘‘‘if (rect1Active)g.setColor(Color.green);
‘‘‘‘‘‘‘‘‘‘else g.setColor(Color.red);
‘‘‘‘‘‘‘‘‘‘g.fillRect(rect1x,rect1y,rect1width,rect1height);
‘‘‘‘‘‘‘‘‘‘g.setColor(Color.blue);
‘‘‘‘‘‘‘‘‘‘g.drawString("("+x+","+y+")",x,y);
‘‘‘‘ }

‘‘ // This will be excuted whenever the mousemoves in the applet


‘‘‘‘public void mouseMoved(MouseEvent me)
‘‘‘‘ {
‘‘‘‘‘‘‘‘‘‘x = me.getX();
‘‘‘‘‘‘‘‘‘‘y = me.getY();
‘‘‘‘‘‘    ‘‘‘‘// Check if the mouse is in the rectangle
‘‘‘‘‘‘if (x > rect1x && x < rect1x+rect1width && y > rect1y && y < rect1y+rect1height)
‘‘‘‘‘‘‘‘‘‘‘‘‘‘ rect1Active = true;
‘‘‘‘‘‘‘‘‘‘else
‘‘‘‘‘‘‘‘‘‘‘‘‘‘rect1Active = false;
‘‘‘‘‘‘‘‘‘‘‘‘‘‘//show the results of the motion
‘‘‘‘‘‘‘‘‘‘repaint();
‘‘‘‘ }
‘‘‘‘ public void mouseDragged(MouseEvent me)
‘‘‘‘ {
‘‘‘‘ }
‘‘}
```

HTML code of the program:

```
< HTML>
< HEAD>
< / HEAD>
< BODY>
```

```
< div align="center">
< APPLET ALIGN = "CENTER"      CODE = "MouseMotionExample.class" WIDTH = "800" HEIGHT = "500">
< / APPLET>
< / div>
< / BODY>
< / HTML>
```

Practice 7.14

This applet draws an arc with mouse.

Java code of the program:

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;

public class MouseArc extends Applet
‘‘ implements MouseListener, MouseMotionListener {

‘‘ int width, height;
‘‘ Vector listOfPositions;

‘‘ public void init() {
‘‘‘‘‘‘width = getSize().width;
‘‘‘‘‘‘height = getSize().height;
‘‘‘‘‘‘setBackground( Color.black );
‘‘‘‘‘‘listOfPositions = new Vector();
‘‘‘‘‘‘addMouseListener( this );
‘‘‘‘‘‘addMouseMotionListener( this );
‘‘ }

‘‘ public void mouseEntered( MouseEvent e ) { }
‘‘ public void mouseExited( MouseEvent e ) { }
‘‘ public void mouseClicked( MouseEvent e ) { }
‘‘ public void mousePressed( MouseEvent e ) { }
‘‘ public void mouseReleased( MouseEvent e ) { }
‘‘ public void mouseMoved( MouseEvent e ) {

‘‘‘‘‘‘if ( listOfPositions.size() >= 50 ) {
```

```
‘‘‘‘‘‘‘‘ // delete the first element in the list
‘‘‘‘‘‘‘‘ listOfPositions.removeElementAt( 0 );
‘‘‘‘‘‘}
‘‘‘‘‘‘// add the new position to the end of the list
‘‘‘‘‘‘listOfPositions.addElement( new Point( e.getX(), e.getY() ) );
‘‘‘‘‘‘repaint();
‘‘‘‘‘‘e.consume();
‘‘ }
‘‘ public void mouseDragged( MouseEvent e ) { }

‘‘ public void paint( Graphics g ) {
‘‘‘‘‘‘g.setColor( Color.white );
‘‘‘‘‘‘for ( int j = 1; j < listOfPositions.size(); ++j ) {
‘‘‘‘‘‘‘‘ Point A = (Point)(listOfPositions.elementAt(j-1));
‘‘‘‘‘‘‘‘ Point B = (Point)(listOfPositions.elementAt(j));
‘‘‘‘‘‘‘‘ g.drawLine( A.x, A.y, B.x, B.y );
‘‘‘‘‘‘}
‘‘ }
‘‘ }
```

HTML code of the program:

```
< HTML>
< HEAD>
< / HEAD>
< BODY>
< div align="center">
< APPLET ALIGN = "CENTER" CODE = "MouseArc.class" WIDTH = "800" HEIGHT = "500">
< / APPLET>
< / div>
< / BODY>
< / HTML>
```

Practice 7.15

This applet generates color spectrum.

Java code of the program:

```
import java.applet.*;
import java.awt.*;

public class ColorSpectrum extends Applet {

‘‘ int width, height;
‘‘ int N = 25;’‘‘‘‘‘‘‘‘‘// the number of colors created
‘‘ Color[] spectrum;’‘‘‘
‘‘ Color[] spectrum2;’‘
‘‘ public void init() {
‘‘‘‘‘‘width = getSize().width;
‘‘‘‘‘‘height = getSize().height;
‘‘‘‘‘‘setBackground( Color.black );
‘‘‘‘‘‘spectrum = new Color[ N ];
‘‘‘‘‘‘spectrum2 = new Color[ N ];

‘‘‘‘‘‘// Generate the colors and store them in the arrays.
‘‘‘‘‘‘for ( int i = 1; i <= N; ++i ) {
‘‘‘‘‘‘‘‘ // The casting to (float) is done so that the divisions will be
‘‘‘‘‘‘‘‘ // done with floating point numbers, yielding fractional quotients.

‘‘‘‘‘‘‘‘ // As i goes from 1 to N, this color goes from almost black to white.
‘‘‘‘‘‘‘‘ spectrum[ i-1 ] = new Color( i/(float)N, i/(float)N, i/(float)N );

‘‘‘‘‘‘‘‘ // As i goes from 1 to N, this color goes from almost pure green to pure red.
‘‘‘‘‘‘‘‘ spectrum2[ i-1 ] = new Color( i/(float)N, (N-i)/(float)N, 0 );
‘‘‘‘‘‘}
‘‘ }

 public void paint( Graphics g ) {
‘‘‘‘‘‘int step = 90 / N;
‘‘‘‘‘‘for ( int i = 0; i < N; ++i ) {
‘‘‘‘‘‘‘‘ g.setColor( spectrum[ i ] );
‘‘‘‘‘‘‘‘ g.fillArc( 0, 0, 2*width, 2*height, 90+i*step, step+1 );
‘‘‘‘‘‘‘‘ g.setColor( spectrum2[ i ] );
‘‘‘‘‘‘‘‘ g.fillArc( width/3, height/3, 4*width/3, 4*height/3, 90+i*step, step+1 );
‘‘‘‘‘‘}
‘‘ }
}
```

HTML code of the program:

```
< HTML>
< HEAD>
< / HEAD>
< BODY>
< div align="center">
< APPLET ALIGN = "CENTER"      CODE = "ColorSpectrum.class" WIDTH = "800" HEIGHT = "500">
< / APPLET>
< / div>
< / BODY>
< / HTML>
```

Practice 7.16

This applet reduces flickering while painting with mouse dragging. Here, backbuffer is used to store results of drawing operations.

Java code of the program:

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;

public class BackBufferPaint extends Applet
‘‘ implements MouseMotionListener {

‘‘ int width, height;
‘‘ Image backbuffer;
‘‘ Graphics backg;

‘‘ public void init() {
‘‘‘‘‘‘width = getSize().width;
‘‘‘‘‘‘height = getSize().height;

‘‘‘‘‘‘backbuffer = createImage( width, height );
‘‘‘‘‘‘backg = backbuffer.getGraphics();
‘‘‘‘‘‘backg.setColor( Color.black );
‘‘‘‘‘‘backg.fillRect( 0, 0, width, height );
‘‘‘‘‘‘backg.setColor( Color.white );

‘‘‘‘‘‘addMouseMotionListener( this );
‘‘ }
‘‘ public void mouseMoved( MouseEvent e ) { }
‘‘ public void mouseDragged( MouseEvent e ) {
```

```
‘‘‘‘‘‘int x = e.getX();
‘‘‘‘‘‘int y = e.getY();
‘‘‘‘‘‘backg.fillOval(x-5,y-5,10,10);
‘‘‘‘‘‘repaint();
‘‘‘‘‘‘e.consume();
‘‘ }

‘‘ public void update( Graphics g ) {
‘‘‘‘‘‘g.drawImage( backbuffer, 0, 0, this );
          ‘‘}
‘‘ public void paint( Graphics g ) {
‘‘‘‘‘‘update( g );
‘‘ }
}
HTML code of the program:

< HTML>
< HEAD>
< / HEAD>
< BODY>
< div align="center">
< APPLET ALIGN = "CENTER"      CODE = "BackBufferPaint.class" WIDTH = "800" HEIGHT = "500">
< / APPLET>
< / div>
< / BODY>
< / HTML>
```

Assignment

- **Q: Write an applet which takes input from user and performs basic mathematical operations on the inputs and shows the result.**
- **Q: Write an applet which takes the input - a text and font color from the user and displays the text in the selected color.**
- **Q: Write an applet which prompts for dimension of circle/square/rectangle and displays the figure with proper dimension. After some time, figure will vanish from the screen and again ask to enter dimension of the figure.**

Q&A

- **Q: What are AWT peers?**
- **A:** A component is associated with a standard AWT button object, a peer object and an interfacing button object constructed per the native GUI.
- **Q: What are the different types of controls in AWT?**

A: The AWT supports the following types of controls:
Labels, Pushbuttons, Checkboxes, Choice lists, Lists, Scroll bars, Text components These controls are subclasses of component.

Q: What is difference between Swing and AWT in Java?

A: Swing is a considered as light weight and AWT is considered as heavy weight. Another difference between AWT and Swing is that, Swing offers uniform look and feel across platform while look and feel of AWT GUI application are platform dependent because AWT mostly use native components e.g. a AWT windows will look different in DOS and Windows operating system.

Q: What interface is extended by AWT event listener?

A: The java.util.EventListener interface is extended by all the AWT event listeners.

Q: What is the default layout for Applet?

A: The default layout manager for an Applet is FlowLayout, and the FlowLayout manager attempts to honor the preferred size of any components.

Q: Name Components subclasses that support painting?

A: The Canvas,Frame,Panel and Applet classes support painting.

Q: What is difference between BorderLayout and GridLayout ?

A: BorderLayout and GridLayout are two widely used LayoutManager from Swing API, former arranges components in predefined position e.g. NORTH, SOUTH, EAST and WEST while later arranges components one after other until there is space and wraps them afterwards.

Q: What is the lifecycle of an applet?

A:

```
init() method  can be called when an applet is first loaded.

start() method can be called each time an applet is started.

paint() method  can be called when the applets is minimized or maximized.

stop() method  can be used when the browser moves off the applet s page.

destroy() method  can be called when the browser is finished with the applet.
```

Q: Why is Model-View-Controller Architecture used in Swing?

A: Model-View-Controller Architecture is used to show the design of the components that are made up of three elements and these elements show the behavior of components as:
Model: It consists of the state of individual component. Different models are used for different components for example scrollbar component that consists of the information used to display the current position and the values that are associated with it. Menu, can consists of a simple list containing the menu items that user can select and take actions on. This information doesn't change from component to component and it remains the same in every case. The model remains independent of the visual representation of the component.
View: It defines the visual representation of the component. The view differs from one window to another window. The application can consist of different user interface platforms and specifications. It consists of all the components related to the visual appearance. It deals with different views of a particular component.
Controller: It provides the way to control the relationship between the model and the view and it allows the interaction to be done with the events. The events can have many forms like mouse click, having focus on some button or some kind of

triggers that are on some menu, etc. This is the controller that decides the component that is to be used to react to an event.

Q: What is the difference between exclusive and non exclusive?

A: Exclusive: Only one among a group of items can be selected at a time. If an item from the group is selected, the checkbox currently checked is deselected and the new selection is highlighted. The exclusive Checkboxes are also called as Radio buttons.
Non Exclusive: Checkboxes are not grouped together and each one can be selected of the other.

Q: What is the difference between choice and list?

A: A choice is displayed in a compact from that requires you to pull it down to see the list of available choices and only one item may be selected from a choice.
A list may be displayed in such a way that several list items are visible and it supports the selection of one or more list items.

Q: Why Swing is called light weight ?

A: Most of Swing component are inherited form JComponent and doesn't required a native peer and that's why they are referred as light weight component. light weight component implement look and feel in Java rather than using look and feel of native component and that's why Swing's look and feel remains same across platform.

Q: How will you communicate between two applets?

A: By creating URL objects, URL Connection object and getting Input Stream, Output Stream Using get Input Stream(), get Output Stream().

Q: Which method is called by applet class to load an image?

A: Get Image(URL object, filename) is used for this purpose.

Q: What is the relationship between clipping and repainting?

A: When a window is repainted by the AWT painting thread, it sets the clipping regions to the area of the window that requires repainting.

Chapter 9

Java Multimedia

by Debasis Samanta

CONTENTS

- Introduction
- Multimedia basics in Java
  - Audio basics
  - Image basics
- Web page in Java

    - World Wide Web basics
    - Preparing Web Documents in Java
    - Applet hosted HTML Document
    - HTML Snapshots
- Practice Questions
- Assignment
- Q&A

Introduction

From Chapter 1 to Chapter 8, we have learnt what we can do with Java. Truly speaking, we can do anything whatever we want. In the current Chapter, we are to explore how Java can do a lot even in the world of entertainment. Two main cooks for entertainment : Multimedia and Web page will be highlighted.

Multimedia basics in Java

In broad sense, multimedia is the processing of audio, video, image/graphics, and text in such a way that one can create them, linked them, interact with them and all these things in a real time fashion. Processing of all these, although requires extra hardware support, and presently, number of professional tools are readily available in the market, but Java is unique because of its incomparable feature of portability.

In this Section, we will elaborate the basic multimedia objects that Java supports.

Audio basics

To play an audio data, it requires a specific format that the working platform can support. Java can play back audio data over a wide range of platforms. According to Java convention, all audio data should come from a URL (Universal Resource Locator) in a .au audio file. In order to use existing sounds in formats such as .wav or .aiff, one need to convert these to .au using a sound program that supports audio conversion. If one try to use .wav then Java is unable to support it and throws an InvalidAudioFormatException, which will not play the audio. The .au file refers to an 8-bit, 8Khz encoded audio file. This format doesn't represent the state of the art in audio fidelity, but is adequate for most sound clips.

There are two ways to play a sound file from an applet. The first way to play a sound is through the play() method and the other through the AudioClip of the Applet class object.

Using play() method : This method comes in two versions, both of which take a URL for a sound file, load the file into the memory, and then play the sound for one time. The play() method is as follows :

```
          public void play (URL'‘url ) - plays an audio clip if the audio clip referred to by the URL is found.

          public void play (URL url, String name) - same as the previous method, but uses a base URL and file name to locate the audio file.
```

Following is the Illustration 9.1 to play a sound when a button is pressed.

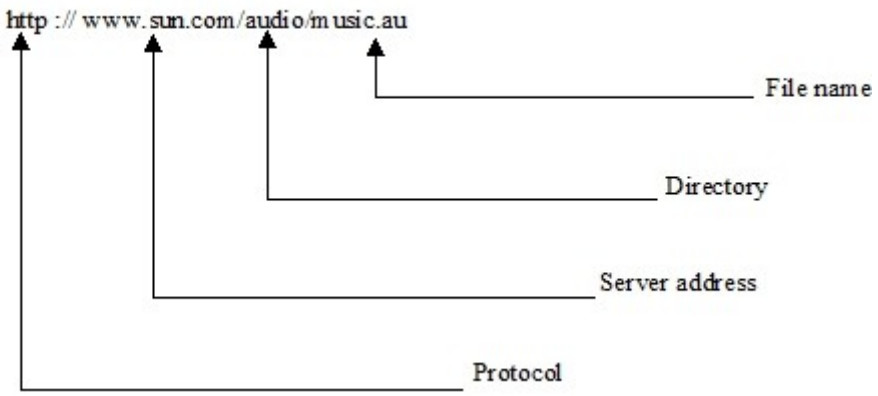
```
Illustration 9.1                                    //'‘Playing an audio clip'‘//
```

```
          import java.applet.*;
          import java.awt.*;
          public class AudioPlayTest extends Applet           {
                    public void init ( ) {                              // To Place a Play button in the
applet
                              setLayout (new FlowLayout (FlowLayout. CENTER ));
                              Button playButton = new Button ("Play" );
                              add (playButton );
                    }

                    public boolean action (Event e, object button) {
                              if ("Play".equals (button))’‘{
                                        play (getDocumentBase ( ), "audio\bell.au" );
                                                  ‘‘return true;
                              }
                    }
          }
```

This applet simply plays the audio file bell.au which is located in the audio sub directory of the directory of the HTML file that loaded the applet. The audio file is cached in memory before playing it.

Note : URL actually a source of an object in Internet site. A typical URL which is actually having various component may look like as below :



Here, we have specified an audio file "music.au" which is available in the sub directory "audio" at the server node "sun".

There are two methods defined in Applet class, getDocumentBase() and getCodeBase() which are very useful to load data from the directory that had the HTML file that started the applet(document base), and the directory that the class file was loaded from (code base). These two methods return a URL object.

Using Audio clip object : The disadvantage of using the play() method is that the first time you call it with a given audio file, it will have to down load the file if it hasn't been used before. This can happen responsiveness in cases like the previous example, where we want to play the sound in response to a user action, and the user ends up waiting for the sound to load in response to hitting a button. Also, the play() method is present only in the applet, which means

that to use it from a component, or from within a thread, we need to have a reference to the applet. Last, the play() method plays a sound only once and must be called again if one wants to play the sound next.

The AudioClip object solves many of these limitations. To obtain an AudioClip object, one has to call the Applet's getAudioClip() method, which is defined as follows :

```
          public AudioClip getAudioClip (URL url ) - returns an AudioClip object. This method will not
return until the AudioClip has been loaded from the specified URL, so one should consider placing it in a
separate thread’‘if the file is expected to take a while to down load.


          public AudioClip getAudioClip (URL’‘url, String name ) - same as the previous method, but finds
the audio file using the’‘base URL and file name.


The AudioClip object specifies three methods which are stated below :


          public abstract void loop(‘‘) - plays the clip in a continuous loop.

          public abstract void play(‘‘) - starts playing the clip from its beginning.

          public abstract void stop(‘‘) - stops the clip if it is currently playing.


Here, audio played by the AudioClip() is asynchronous play that can be possible with the above three
methods. Following is the Illustration 9.2 of an applet that plays a sound as long as the applet is on -
screen or until the 'Stop' button is clicked :
```

Illustration 9.2 `// Play audio through Audio clip //`

```
          import java.applet.*;
          import java.awt.*;
          public class AudioClipTest extends Applet {
                    AudioClip flute ;
                    AudioClip piano ;
                    public void init ( ) {          // applet initialize
                              setLayout (new FlowLayout (FlowLayout.CENTER ));
                              Button testButton = new Button ( "Start" );
                              add(testButton);
                              testButton = new Button( "Continue");
                              add(testButton);
                              testButton = new Button ( "Stop");
                              add(testButton);
                              flute = getAudioClip( getDocumentBase( ), "flute.au");
                              piano = getAudioClip( getDocumentBase( ), "piano.au");
                    }
                    public void start( ) {‘‘‘‘‘‘‘‘‘‘// Continue the play of Piano
                              piano.loop( );
```

```
                    }
                    public void stop( ) {‘‘‘‘‘‘‘‘ // Stop the Piano play
                              piano.stop( );
                    }
                    public boolean action( Event e, Object button) {
                              if( "Start".equals(button) )’‘
                                        flute.play( );
                              if( "Continue".quals(button) )
                                        piano.loop( );
                              if ( "Stop".equals(button) )
                                        piano.stop( );
                    return true;
                    }
          }
```

In the above illustration, applet loads two audio clips vide its init() method. First is "flute.au" and the second is "piano.au". The init() method will not finish until the sounds have been loaded. When the applet's start() is called, it starts the "piano.au". The thread of an event driven method action() is started, thus, if user hit "Start" button the "flute.au" audio clip will come into play. Note that, two audio clips will then come into play simultaneously. Thus, using AudioClip object multiple audio can be played, provided that the working platform supports multiple audio channel. The play will continue until the applet is closed.

Image basics

Java is popular because it has the promise of creating rich graphical experiences. Java has a number of classes designed to deal with images and image manipulation. Java follows both JPEG (Joint Photographic Expert Group) and GIF (Graphics Interchange Format) as the formats for image data. In general , JPEG images are better suited to natural color images such as photo graphs, and GIF images are best for graphical logos, rules, and button images.

```
          There are five types of objects that one will need to deal with or at least understand when
dealing with Images. These are :
          Image
          ImageObserver
          ImageProducer
          ImageConsumer
          ImageFilter
```

These are basics, all of them defined in java.awt package, and many more also related to image manipulation.
Image class: With this class one can load an image object that obtains its pixel data from a specific URL. The most common way to do this is through the use of the getImage() method of the java.applet.Applet class, which has the following two forms :

```
          public Image getImage (URL url ) - returns an Image object given a URL.
          public Image getImage (URL url, String name )- returns an Image object given a base URL and
file name.
```

Let us take a took at a very simple applet that loads and then draw an image (Illustration 9.3)

Illustration 9.3 // Draw an Image //

```
import java.applet.*;
import java.awt.*;
public class ImageTest extends Applet {
        Image mona;
        public void init (‘‘) {
                mona = getImage (getDocumentBase ( ),’‘"monalisa.gif ");
        }

        public void point (Graphics g ) {
                ‘‘ g.drawImage (mona, 0, 0, this);
‘‘‘‘}
}
```

In the above mentioned applet, first init() method loads the image object from the specified URL and file name. The paint() method uses drawImage() method (it is defined in Component class); it has the form as below :

public abstract boolean drawImage (Image imageName, int x, int y, ImageObserver imgObs);

imageName is the image reference that has to be drawn, the x, y coordinates to paint at, and imgObs is the ImageObserver that can monitors an image while it loads, we will learn more about it during the subsequent discussion. In the example Illustration 9.3, this represents the default ImageObserver. When this applet runs, it starts loading "monalisa.gif " in the init() method. On screen you can see the image as it loads from the Network (or local machine), because the default ImageObserver interface calls paint() method every time more image data arrives from the source.

ImageObserver class : ImageObserver is an abstract interface used to get notification as an image is being generated. As we have seen in the last example, the drawImage() method takes both an Image object and an ImageObserver object. In the case of an ImageObserver object, because ImageObserver is an interface, we need to pass a reference to an instance of a class that implements the ImageObserver interface. Classes that implements the ImageObserver interface are required to have an imageUpdate() method as follows :

public abstract boolean imageUpdate (Image img, int status , int x, int y, int width, int height);

Before going to know the use of this method in an applet, let us see about the various arguments in it :
Image img - the Image object references that has to be drawn on the applet.
int status - this gives us the status information of the image "img" being processed. The " status" integer is tested bit wise against the value of one or more flags. The available flags and the information they provide are listed in **Table 9.1**
int (x, y, width, height)- is a configuration for rectangle that reports different information about the image under loading.
The use of the **imageUpdate()** method can be summarized as stated below :
When an Image object is referred, the imageUpdate() of this object will call repaint() method which will cause the paint() method to eventually call again. The drawImage() is thus again called, and it again draws all the pixel data it has loaded so far, which at this point should be more than had been loaded the last time drawImage() was called. The partial image is again drawn, and this time, because more pixel data has been loaded, more of the image is drawn and appears on the screen. When more pixel data is loaded, imageUpdate() is again called, and the cycle repeats itself, until all the pixel data has been loaded and the complete image has been drawn.

Table 9.1

Bits	Indicator
WIDTH	The width of the base image is now available and can be taken from the `width` argument.
HEIGHT	The height of the base image is now available and can be taken from the `height` argument
PROPERTIES	The properties of the image are now available. One can access the properties with img.`properties`.
SOMEBITS	More pixels needed for drawing a scaled variation of the image are available. The bounding box of the new pixels can be taken from the `x`, `y`, `width`, and `height` argu ments.
FRAMEBITS	Another complete frame of a multiple frame image, which was previously drawn, is now available to be redrawn. The `x`, `y`, `height`, and `width` arguments should be ignored.
ALLBITS	The image being drawn is now complete and can be drawn again in its final form. The `x`, `y`, `width`, and `height` arguments are no longer meaningful.
ERROR	An image that was being tracked has encountered an error. No further image informa tion will be available and drawing the image will fail.
ABORT	An image that was being tracked was aborted before production was complete. No more image information will become available.

Now let us look at an example that uses ImageObserver to see how many scan lines of the image have been processed and then prints the progress to the console.

Illustration 9.4 **// Use of ImageObserver’‘//**

```
import java.applet.*;
import java.awt.*;
import java.awt. image.*;
public class ImageLoadTest extends Applet implement ImageObserver {
          image mona;
          Dimension d;
          int progress;
          boolean loaded;
          public void init (‘‘) {
                    mona = getImage ( getDocumentBase ( ), "monalisa.gif " );
                    loaded = false;
                    progress’‘= 0;
```

```
‘‘‘‘}
‘‘‘‘public void paint (Graphics’‘g )      {
                    d’‘= this.size;
                    loaded = g.drawImage (mona, 0, 0, this );
          }
‘‘‘‘public boolean imageUpdate (Image img, int info, int x,’‘ int y,’‘int width, int height ) {
‘‘‘‘‘‘‘‘if (( info & ALLBITS ) != 1) {
                              if (progress < d.height )
                                        progress = progress + height;
                              ‘‘‘‘System.out.println( progress + "/" + d.height);
                              ‘‘‘‘return true;
                    } else’‘{ return false; }
          }
          public void start( ) {
                    mover = new Thread (this);
                    mover.start( );’‘‘‘
‘‘‘‘‘‘‘‘}
          public void stop( ) {
          ‘‘‘‘mover.stop( );’‘‘‘
‘‘‘‘‘‘‘‘}
          public void run( ) {
                    mover.setPriority (Thread.MIN_PRIORITY);
                    while (!loaded ) {
                              repaint( );
                              try’‘mover.sleep( 200 );
                              catch (Exception e);
                    }
          }
}
```

This applet, loads an image just as in Illustration 9.3. The difference in this example is that we are implementing our own imageUpdate() method over the default one. The imageUpdate() method processes the status of the image load. A summary of the status is passed in through info, against which the static variable ALLBITS is tested. If we have not yet received all of the bits in our image, we add the height value to the total number of scan line processed which will be printed to the system console. The run() method repaints the applet five times a second (every 200 milliseconds) while the image "mona" is not loaded. How long the status monitor will run depends on how quickly the Network connection can deliver the image date.

ImageProducer class : ImageProducer is an abstract interface for objects that want to produce data for Images. An object that implements the ImageProducer interface will supply integer or byte arrays that represent image data. The createImage() method (will be discussed shortly) takes an ImageProducer object and returns an Image object.

We will examine a valuable class called MemoryImageSource that implements ImageProducer. We will create a new Image object from data generated by this producer.

MemoryImageSource : It is a class used to create a new Image from an array of pixels. Here is a constructor used to create a MemoryImageSource object :

public ImageProducer MemoryImageSource (int width, int height, int pixel[ ],int offset, int scan linewidth);

The MemoryImageSource object is constructed out of an array of integers pixel[ ], in the default RGB color model to produce data for an image object. The constructor for MemoryImageSource also takes the width and height of the result, the offset into the pixel array, and the width of a scan line, This constructor returns an ImageProducer object which is used with createImage() to result in a usable image. Following is an illustration of ImageProducer.

Illustration 9.5 **// Image Production //**

```
Import java.applet.*;
import java.awt.*;
import java.awt.image.*;
public class MemoryImageProduce extends Applet      {
          Image art;
          Dimension’‘d;
          public void init ( ) {
                    generateImage ( );
          }

          public void generateImage ( ) {
                    int pixels[ ] = new int [ d.width * d.height ];
                    int i = 0;
                    int r,’‘g,’‘b;
                    for (int’‘ y’‘= 0 ; y < d.height; y++ ) { // Create a set of pixels
                              for (int x = 0,’‘x < d.width; x++ )       {
                                        r = (x ^ y ) & 0xff;
                                        g = ( x * 2 ^ y * 2 ) & 0xff ;
                              b = ( x* 4’‘^ y * 4 ) 0xff ;
                              pixels [ i++ ] = (255< 24 ) | (r < 16 ) | (g < 8) | b;
                              }
                    }
                    art = createImage( new MemoryImageSource( d.width, pixels, 0, d.width ));
          }
          public void paint( Graphics g )
                    ‘‘g.drawImage (art, 0, 0, this ));
          }
}
```

Here, the data for the new MemoryImageSource is created in the generateImage() method. An array of integers is created to hold pixel values; Then integer array elements get shifted into a pixel in the pixel array. Finally, we call createImage() to create a new Image from the 'pixels' array with MemoryImageSource.

There are many more about image handling : OffScreen, MediaTracker, ImageFilter, ImageFilterSource, CropImageFilter, RGBImageFilter etc. which are not possible to discuss in this Tutorial.

Web page in Java

Web is a short form of very big concept World Wide Web, also nicknamed as WWW. Today, the most likely place you will find Java is on the web. The web act as a convenient transport mechanism for Java programs, and the web's ubiquity has popularized Java as an Internet development tool. In fact, Java dynamically extends the capability of web media. In this Section we are to explore how Java can support web.

World Wide Web basics

What is web? The web is a huge collection of interconnected hypertext document on the Internet. A hypertext document is a document which contains hot links (on line) to other documents. Hypertext links are usually visible as highlighted (colored/underlined) word(s) in the text, but they can also be graphics, and are frequently used for on line help systems. Links are activated by clicking on them with a mouse.

The web is based on two standards : the HTTP protocol and the HTML language : HTTP for Hyper Text Transmission Protocol, and it describes the way that hypertext documents are transferred over the Internet. HTML is the abbreviation for Hyper Text Markup Language, and it specifies the layout and linking commands present in the hypertext document themselves.

The HTTP protocol is implemented in software on the server and the users machine (client). The server software is called a web server or HTTP server, and the client software is called a web browser. To open an HTML document, the web browser sends an HTTP commands to the sever requesting the document by its location (called URL, Universal Resource Location). The web server responds by sending the HTML document to the client. The client then displays the document on the user's screen. The HTML documents, again, not only contain text, it also may contain multimedia objects like graphics, audio, animation etc. which are generally stored in files and in the server's sub directory where the HTML document itself is located. Location of the resources on the web are specified with a URL. A URL specifies the protocol used to fetch a document as well as its location. This is already discussed in Chapter, Part II of this book.

To achieve all these, web developers have to face number of hurdles. Important among various difficulties is the platform independency i.e. it should present information in a way that can be viewed on almost every type of machine and operating system. It does not matter whether the environment is Pentium, or Macintosh, Windows 95 or UNIX.

Java shorted out this platform independency problem easily. Java addresses this problem by using Java applets. An applet is a Java program which appears embedded in a web document. The Java applets runs when it is loaded by a Java enabled web browser like HotJava, Netscape Navigator or Internet Explorer. The running applet then draws itself in the user's browser window according to the programmer's instructions. The applet can create its own network connections and use whatever protocol is required to get information from the server.

The next section, is devoted how to prepare a web documents in Java.

Preparing Web Documents in Java

Java web pages are nothing but the HTML documents. Or in other words, an HTML document hosts number of applets to constitute a web page. In Section 2.2 of Chapter 2, Part II of this book, we have seen that an applet can not run independently unless it is embedded in a HTML document.

In this section, we are to discuss what are the basic elements to compose a HTML document and then how applet(s) can be embedded in it.

HTML files are text files with special character sequences that specify the document formatting characteristics. The special character sequences are called tags, and they consist of words placed between left and right angular brackets like word of interest here < /TAG>. Here a special tag TAG is used to achieve some attribute on the text "word of interest here" and /TAG is to reset the attribute. In general, every tag should have its corresponding resetting tag. All tags are with capital letters.

What are the various useful tags available for composing in HTML documents is discussed in Section 9.2.4 of the current Chapter.

For better understanding of full HTML, the reader may refer to a good HTML text entitled "The HTML Sourcs book" published by Ian Grahams, John Wiley Computer Publishing.

Following is an example, illustrating the basic formats of an HTML document

---

Illustration 9.6 (a) **// Basic format of an HTML document //**

```
‘‘‘‘ < HTML>
          ‘‘< HEAD>
                    < TITLE> A sample HTML demonstration < / TITLE>
‘‘‘‘‘‘< / HEAD>
          < BODY>
                    < H1> HTML Demo
                    Here is a simple normal text
                    < B> Here is some Bold text’‘
                    < I> Here is some text in Italic
          < / BODY>
          < / HTML>
```

This file should be saved with a suitable file name and having extension .html. In the **Illustration 9.6(a)** the tag HTML indicates that the file is an HTML document, the HEAD tag marks the start of an header section that is normally used for recording the title and author of the document; the phrase between < TITLE> and tags is the name of this document. Anything within the < TITLE> tags will not be displayed when the web page is run. The phrases between < BODY> and of the document are to compose the actual web page.

One can include an image (or audio etc.) file in an HTML file; an another HTML file can also be included. Following is an example taking these into consideration as given below :

Illustration 9.6 (b) **// Hyper text document //**

```
          < HTML>
          < HEAD>
          < TITLE> A hypertext Demo < / TITLE>


          < BODY>
                    < IMG SRC= "monalisa.gif" >
                    < B> Following is another image from NERIST server
                    < IMG SRC = " http: // www. nerist.in /etc/ emblem.gif" >
                     Get the information of NERIST < BR>
                    < A HREF = "http : // www. nerist in /etc/ info.html">
```

```
                    NERIST Information < / A>
          < / BODY>
          < / HTML>
```

In the Illustration 9.6 (b), an image file "monalisa.gif " is included in the HTML document with the IMG tag. This file will be down loaded by the browser from the directory in which these HTML file will be located (or any directory as the sub directory of the directory the HTML file is in. One can specify other location than the such default location, (in that case full URL has to be specified). As an example, next IMG tag is having a source "www. nerist. in /etc/emblem.gif" which means that the image file is emblem.gif is in the directory /etc of the server www.nerist.in. Also, one can connect an HTML page to another document via a hypertext link. To do this one must insert an anchor tag (< A>). A text between the anchor tag and the closing anchor tag will be highlighted, so the user knows that the highlighted text (or graphics) can be clicked on. In the current example a part of the page will appear as :

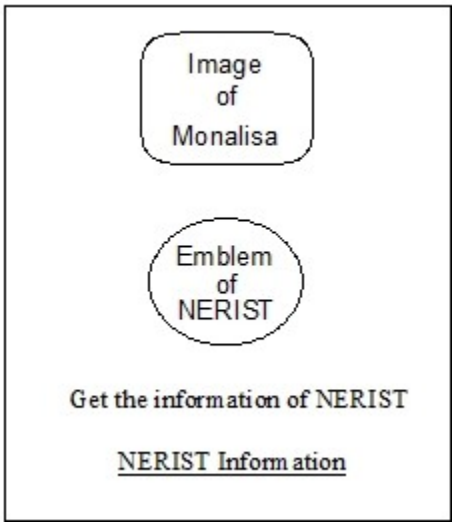


Figure 9.1 : A typical web page

This highlighted text may appear in blue color and underlined - so that if user click here, then jump to the document info.html which is in the NERIST's server. One can insert < BR> and < P> etc. to break for a new line and a new paragraph. This may require because, a web browser ignores excess spaces and new lines when displaying a document.

Applet hosted HTML Document

Next we will see how applet(s) can be embedded within an HTML document.

The < APPLET> tag is used to start an applet from both an HTML document and from the JDK appletviewer. The appletviewer will execute each applet tag that it finds in a separate window, while web browser like HotJava, Netscape Navigator, and Internet Explorer will allow many applets on a single page.

Following is the Illustration 9.7 which includes an applet to run HellowJavaApplet :

```
Illustrate 9.7                          // A web page with an applet’‘//

‘‘‘‘< HTML >
```

```
           < HEAD>
          ‘‘‘‘‘‘< TITLE > An HTML Containing HelloJavaApplet < / TITLE>
‘‘‘‘< / HEAD>


          < BODY>
                    < HI> Very simple web page Demo < / HI>
                              < APPLET CODE = " HelloJavaApplet.class">
                              WIDTH = 200’‘‘‘HEIGHT = 100 >< / APPLET>
          < / BODY>
          < / HTML>
```

In the last example, the applet is included with < APPLET> tag in a minimum way. There is much more, that one can specify with this tag. The detail syntax for the < APPLET> tag is given below :

```
Syntax for the < APPLET> Tag


< APPLET
          [CODE BASE = codeBaseURL]
          CODE = appletFileName
          [ALT = alternateText]
          [ NAME = appletInstanceName]
          WIDTH = pixels HEIGHT = pixels
          [ALLIGN = alignment]
          [VSPACE’‘= pixels] [HSPACE = pixels ]
          >
          [< PARAM NAME = attributeName1 VALUE = attributeValue>]
          [< PARAM NAME = attributeName2 VALUE = attributeValue>]


          [altermateHTML]
< / APPLET>
```

The various attributes indicate the options that can be used when placing an applet on a web page. Many of these options are optional which are marked within square brackets.

The meaning of attributes are summarized as below :

CODE BASE = codebaseURL: CODEBASE is an optional attribute that specifies the base URL of the applet code, which is the directory that will be searched for the applet's executable class file (specified with the CODE tag). The HTML document's URL directory is used as the CODEBASE if this attribute is not specified. The CODEBASE does not have to be on the host that the HTML document was read from.

CODE = appletFileName: CODE is a required attribute that gives the name of the class file that contains your applet's compiled code (of applet). This file is relative to the code base URL of the applet, which is the directory that the HTML file was in or the directory indicated any CODEBASE, if set.

ALT = alternateText: The ALT tag is an optional attribute which specifies a short text message that should be displayed if the browser understands the APPLET tag but can't currently run Java applets. This is distinct from the alternate HTML you provide for browsers that don't support applets.

NAME = appletInstanceName: NAME is an optional attribute used to specify a name for the applet instance, applet must be named for other applets or the same page to find them by name and communicate. For example, to get an Applet subclass MyApplet named Twinkle use :

MyApplet appletInstanceName = getAppletContext ( ).getApplet ("Twinkle");

Once you have the named instance, you can call methods on it as you would any object.
WIDTH= Pixels HEIGHT = Pixels: WIDTH and HEIGHT are required attributes that give the initial size (in pixels) of the applet display area.

ALIGN = alignment : ALIGN is an optional attribute that specifies the alignment of the applet. This many have the possible values : LEFT, RIGHT, TOP, TEXTOP, MIDDLE, ABSMIDDLE, BASELINE, BOTTOM, ABSBOTTOM. LEFT is the default alignment.

VSPACE = pixels HSPACE= pixels: These attributes optionally specify the space in pixels above and below the applet, VSPACE, and each side of the applet, HSPACE.

PARAM NAME = attributeName1 VALUE= attributeValue: The PARAM tag is how one can pass applet-specific arguments in from attributes with the getParameter() method, which is illustrated as in **Illustration 9.8.**

Illustration 9.8 **// Passing parameter from and to APPLET HTML'‘//**

```
< HTML>
.
.
< APPLET'‘CODE = ParamTest WIDTH = 40 HEIGHT = 40>
< PARAM'‘NAME = fontName VALUE = Universe>
< PARAM'‘NAME = fontSize VALUE =14>
< PARAM'‘NAME = leading VALUE=2>
.
.
```

```
Here is how one would extract'‘each parameter from the applet :
.
.
// In an applet

String font = getParameter (" fontName" );
int sizeFont = Integer.parseInt (getParameter ( " fontSize" ));
float leading = float.valueof (getParameter ( "leading" ));
```

```
.
.
```

If some parameter value is not specified in HTML's PARAMs but is referred in applet then the getParameter() will return a null.

HTML Snapshots

Following Table 9.2 summarizes the basic HTML tags which are frequently in use to develop a web document.

Table 9.2

PARAGRAPH FORMATTING	
<H1>..... </H1>	Heading Level 1 (similarly <H2> to"<H6>) <H4> is normal text.
<P ALIGN = = ﷺalignmentﷺ ..... </P>	Paragraph break, `alignment` can take one of the values- Left, Right or Center.
 	Break a line
<UL> ..... </UL>	Unordered or Bulleted List.
<OL> ..... </OL>	Ordered or Numbered List.
<L1>	An item in a list.
<CENTER> ...</CENTER>	Center align anything enclosed in tags.
<HR SIZE =x WIDTH=y>	Horizontal ruler line; x = number of pixels for thickness; y = number or percentage for length.
CHARACTER FORMATTING	
<B> .... </B>	Bold text.
<I> ..... </I>	Italicized text.
<U> ..... </U>	Underlined text
<BLINK> ...</BLINK>	Blinking text.
^{.....}	Superscript text
_{.....}	Subscript text.
<TT> ..... <TT>	Typewriter-like monospaced text
<FONT SIZE=x FACE = ﷺTypefaceﷺ COLOR =ﷺcolorﷺ>	Font enhancements: X can be a can start from 1-7 (1 being smallest) or relative specification like -1, -2, +1 etc; `typeface` is font name; `color` is either color name or color code in hex number.
<PRE> ....</PRE>	Pre-formatted text (retains alignment and displays text just as typed in.
<COMMENT> ..... </COMMENT>	This is a comment line which is ignored by the

	browser.
< !-COMMENT TEXT >	This is also a comment line which is ignored by the browser.
OTHER TAGS	
<BODY BGCOLOR=color BACKGROUND =�image�>	Body tag which controls general color and background settings; `color` is hex RGB or name as background color; `image` is location of background GIF/JPEG image file.
<A HREF = �URL�> ....</A>	Anchor tag to create a hypertext reference/ link. `URL` is the location of the external document/image etc.
<A NAME = �name�> .... <A/>	Anchor tag to create an internal jump to a particular line of the page; name is what the spot is called.
<A HREF = �#name� > ..... </A>	Create a link to jump internally to a line of the page; `name` is the spot which was created by the previous command.
<IMG SRC = �file� ALIGN = alignment ALT = �text� HEIGHT=xWIDTH=Y>	Add images in GIF/JPEG format; `file` is the name of the file to display; `alignment` = left, right or centre aligns image with text word wrap, or else `alignment` = top, middle or bottom aligns subsequent text to top, middle or bottom respectively; alt specifies alternative text if image is not displyed; `x` is height of image in pixels and `y` is width of image.

Special Characters : Special characters which are used in HTML document to control web pages are liste as in Table 9.3.

Table 9.3

Description	Numerical Code	Character Code	Entity
Non-breating space			
Copyright	&#169	&copy	
Registered trademark	&#174	&reg	
Quotation mark	&#34	&quot	"
Ampersand	&#38	&amp	&
Less-than sign	&#60	&lt	<
Greater than sign	&#62	&gt	>
Soft hyphen	&#173	&shy	-

```
Some’‘Color’‘Codes:

          In web document one can use color codes to manage various colors. These are listed as below :


                    Aqua      #00FFF              Black     #00000              Blue’‘    ‘‘#0000FF
          Fuchsia’‘‘‘#FF00FF
```

```
                    Brown’‘    #A52924    ‘‘‘‘Cyan    #00FFF              Darkblue’‘#000088    Orange’‘‘‘
#FFA500

                    Gray       #808080              Green’‘    #008000              Lime’‘     ‘‘#00FF00
          Maroon’‘‘‘ #80000

                    Navy       #000080    ‘‘‘‘Olive’‘          # 808000   Purple     ‘‘#800080
          Red’‘‘‘‘‘‘‘#FF0000

                    Silver     #C0C000    ‘‘‘‘Teal   #009080              White’‘    ‘‘#FFFFF   Yellow’‘‘‘
#FFFF00
```

URL: The URL for a web page that can be specified is as per the syntax given below :

```
‘‘                          Service: //hostname/directory(and/or filename)
                    Service can be:
                              HTTP - Hypertext transfer protocol for web pages
                              FTP - File Transfer Protocol
‘‘‘‘
```

For more information regarding the HTML document preparation, one may consult the following :

Abeginners's guide to HTML : WWW.ncsa.uiuc.edu/General/Internet/WWWW/HTMLPrimer.html This is last but not least. Java is a collection of huge concepts and day-by-day Java features are upgrading, even when you are going through this Tutorial, lot of improvements might have been achieved. To know the latest stock of Java , you are advised to set contact http:// java. sun.com

Practice Questions

Practice 9.1

Below, is a "complete" program. It draws 10 randomly placed lines of randomly selected colors on the image at a time; in a separate thread, it then displays the "updated" image. The drawing is done in the paint method, and the display is done in the run method. Other applications that used this architecture would move an object across the screen in the paint method one increment at a time, display one of a list of images for an animation, etc.

```
import java.applet.*;
import java.awt.*;
import java.util.*;

public class DoubleBuf extends Applet implements Runnable’‘{

‘‘boolean pause = false;
‘‘Thread runThread = null;
‘‘Image offScreen;
‘‘int rd, gr, bl;
‘‘int x1, y1, x2, y2;
```

```
‘‘Color c;
‘‘Color backGround = new Color (0xffffff);
‘‘int imageWidth, imageHeight;
‘‘Random random;

// ************* init

public void init ()’‘{
‘‘random = new Random ();
}’‘// end init

// ************* start

public void start ()’‘{
‘‘runThread = new Thread ( this );
‘‘runThread.start ();
}’‘// end start

// ************* stop

public void stop ()’‘{
‘‘if ( runThread != null ) runThread.stop ();
‘‘runThread = null;
}’‘// end stop

// ************* mouseDown

public boolean mouseDown ( Event event, int x, int y)’‘{
‘‘if ( runThread != null ) pause = true;
‘‘else’‘{
‘‘‘‘pause = false;
‘‘‘‘start ();
‘‘} // end else;
‘‘return true;
}’‘// end stop



// ************* paint

public void paint ( Graphics g, Dimension d )’‘{
‘‘for ( int i = 0; i < 10; i++ )’‘{
```

```
‘‘rd = (int)( 255 * random.nextFloat () );
‘‘gr = (int)( 255 * random.nextFloat () );
‘‘bl = (int)( 255 * random.nextFloat () );
‘‘x1 = (int)( d.width * random.nextFloat () );
‘‘y1 = (int)( d.height * random.nextFloat () );
‘‘x2 = (int)( d.width * random.nextFloat () );
‘‘y2 = (int)( d.height * random.nextFloat () );

‘‘g.setColor ( new Color ( rd, gr, bl ) );
‘‘g.drawLine ( x1, y1, x2, y2 );
‘‘}’‘// end for
}’‘// end paint

// ************* run thread

public void run ()’‘{
‘‘Dimension d;
‘‘Graphics g;

‘‘while ( ! pause )’‘{

‘‘‘‘d = this.size ();

‘‘‘‘if ( ( offScreen == null )’‘||
‘‘‘‘‘‘( imageWidth != d.width )’‘||
‘‘‘‘‘‘( imageHeight != d.height ) )’‘{
‘‘‘‘‘‘offScreen = this.createImage ( d.width, d.height );
‘‘‘‘‘‘imageWidth = d.width;
‘‘‘‘‘‘imageHeight = d.height;
‘‘‘‘}’‘// end if

‘‘‘‘g = offScreen.getGraphics ();
‘‘‘‘paint ( g, d );

‘‘‘‘g = this.getGraphics ();
‘‘‘‘g.drawImage ( offScreen, 0, 0, Color.white, this );

‘‘‘‘try {Thread.sleep ( 10 );} catch ( InterruptedException e) {;}

‘‘}’‘// end while
```

```
‘‘runThread = null;


}’‘// end run


}’‘// end DoubleBuf
```

Practice 9.2

```
//This will play an .au file:


import java.io.InputStream;
import java.net.*;
import java.applet.*;
import java.awt.Graphics;
import java.awt.Color;


public class Message extends java.applet.Applet {
‘‘private String file;
‘‘private AudioClip source;


‘‘public void init() {
‘‘‘‘setBackground(new Color(0));
‘‘‘‘file=getParameter("FILE");
‘‘‘‘source=getAudioClip(getDocumentBase(),file);
‘‘}
‘‘
‘‘public void start() {
‘‘‘‘if(source!=null) source.play();
‘‘}


‘‘public void stop() {
‘‘‘‘if(source!=null) source.stop();
‘‘}


}
```

Practice 9.3

The key objects and methods used are AudioClip, loop() and stop(). Another method not used here but perhaps of some value is the play() method which plays an audio clip one time.

```
import java.applet.Applet;
import java.applet.AudioClip;
import java.awt.*;
```

```
public class PlayAudio extends Applet
{

    AudioClip myAudio;
    Button b_play;
    Button b_stop;
    String textString;

    public void init()
    {
        resize(300, 100);

        b_play = new Button("Play audio");
        b_stop = new Button("Stop audio");

        myAudio = getAudioClip(getDocumentBase(), "computer.au");

        add(b_play);
        add(b_stop);
    }

    public void start()
    {
    }

    public void paint(Graphics g)
    {
        g.setColor(Color.blue);
        g.drawString(textString, 100,75);
    }

    public boolean action(Event e, Object arg)
    {
        if(e.target instanceof Button)
        {
            if("Play audio" == arg)
            {
                myAudio.loop();
                textString = "Playing audio...";
            }
            if("Stop audio" == arg)
```

```
‘‘‘‘‘‘‘‘‘‘‘‘{
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘myAudio.stop();
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘textString = "Not playing audio...";
‘‘‘‘‘‘‘‘‘‘‘‘}
‘‘‘‘‘‘‘‘ }
‘‘‘‘‘‘‘‘ repaint();
‘‘‘‘‘‘‘‘ return true;
‘‘‘‘ }
}
```

Practice 9.4

```
// load an Image into a standalone Java application panel (i.e. one that does not have a base URL
import java.awt.*;
import java.awt.image.*;

public class ImageWindow extends Frame
{
          Image im=null;

          public ImageWindow()
          {
          }
‘‘‘‘
          public void createImage()
          {
                    im = getToolkit().getImage("b01.gif");
          }

          public void paint(Graphics g)
          {
                    update(g);
          }

          public void update(Graphics g)
          {
                    g.drawImage(im, 0, 0, this);
          }

          public boolean handleEvent(Event evt)
          {
```

```
                    if (evt.id == Event.WINDOW_DESTROY)
                    {
                              System.exit(0);
                    }
                    return super.handleEvent(evt);
          }

          public static void main(String args[])
          {
                    ImageWindow window = new ImageWindow();
                    window.createImage();
                    window.setTitle("Java StandAlone Application");
                    window.pack();
                    window.resize(500,300);
                    window.show();
          }
}
```

Assignment

Design JPanel the plays media from a URL.Test simple media player .

Q&A

Q: What is multimedia?
A: Multimedia is a technique that incorporates

```
text,
graphics,
sound,
animations and
video elements.
```

Q: Mention some of the image formats used in multimedia?
A: Some of the image formats used in multimedia are

```
GIF files
JPG files
Animated GIF files
MPEG files
Shockwave files and
Nx View files.
```

Q: What Is Lossless Source Coding?
A: Lossless source coding is one of the data compression standards.

Images, audio and video files use lossless source compression standard. Lossless source encoding is suitable for compressing text files.

Chapter 10

Java Swing

by Debasis Samanta

CONTENTS

- Introduction
- AWT versus Swing
- Swing in depth
- Java Swing examples
- Practice Questions
- Assignment
- Q&A

Introduction

Java Swing is a part of Java Foundation Classes (JFC) that is used to create window-based applications. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in Java. Java Swing provides better lightweight components than AWT. The javax.swing package provides classes for Java Swing components such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

AWT versus Swing

Java AWT	Java Swing
AWT components are platform-dependent.	Java swing components are platform-independent.
AWT components are heavyweight.	Swing components are lightweight.
AWT doesn't support pluggable look and feel.	Swing supports pluggable look and feel.
AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane, etc.
AWT doesn't follows MVC(Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC.

Figure 10.1 :AWT versus Swing

NoteThe Java Foundation Classes (JFC) are a set of GUI components which simplify the development of desktop applications.

Swing in depth

Hierarchy of Java Swing classes:

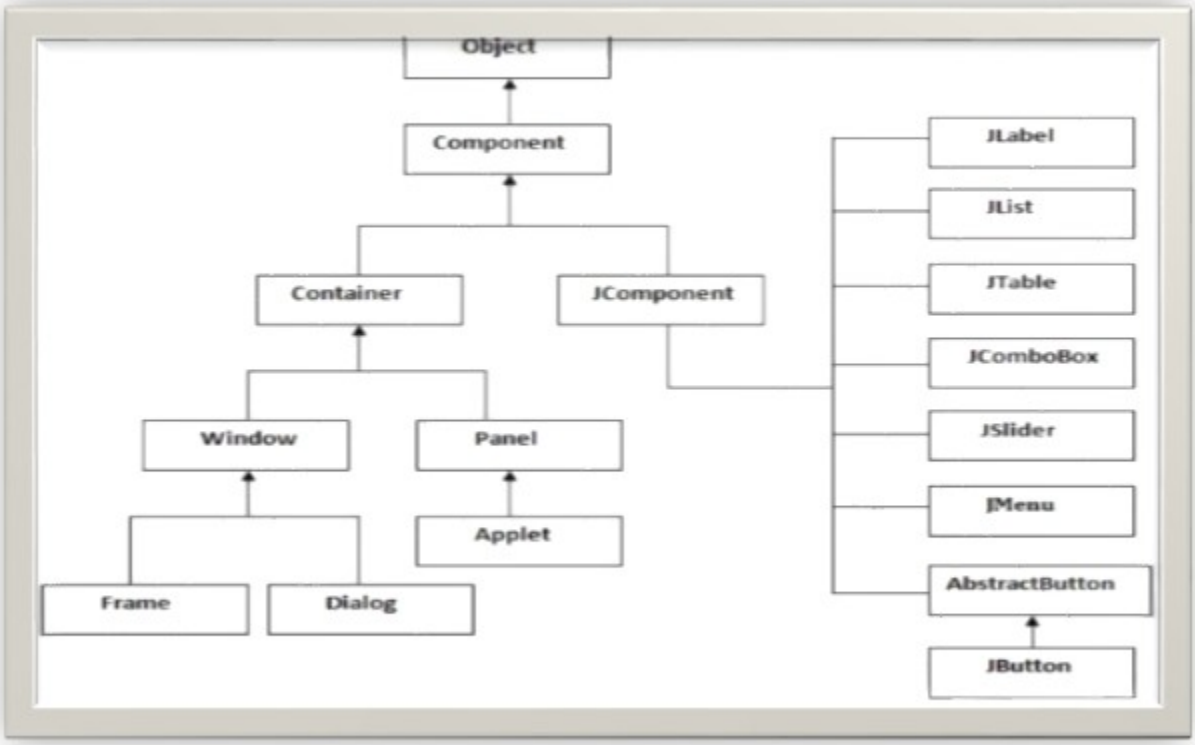


Figure 10.2 :Hierarchy of Java Swing classes

Commonly used methods in Component class:

Method	*Description*
`public void add(Component c)`	Add a component on another compon
`public void setSize(int width,int height)`	Sets size of the component.
`public void setLayout(LayoutManager m)`	Sets the layout manager for the comp
`public void setVisible(boolean b)`	Sets the visibility of the component.

Figure 10.3 :Commonly used methods in Component class:

Java Swing examples

There are two ways to create a frame:

- ''By creating the object of Frame class (Association)
- ''By extending Frame class (Inheritance)We can write the code of Swing inside the main(), constructor or any other method.

JFrame object :

Let's see a simple swing example where we are creating one button and adding it on the **JFrame** object inside the **main()** method.



Figure 10.4 :JFrame object

Illustration 10.1

```
// A program to add a label and button in a frame


import java.awt.FlowLayout;‘‘
import javax.swing.JButton;‘‘
import javax.swing.JFrame;‘‘
import javax.swing.JLabel;‘‘
import javax.swing.Jpanel;‘‘
public class JFrameExample {‘‘
‘‘‘‘public static void main(String s[]) {‘‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("JFrame Example");‘‘
‘‘‘‘‘‘‘‘JPanel panel = new JPanel();‘‘
‘‘‘‘‘‘‘‘panel.setLayout(new FlowLayout());‘‘
‘‘‘‘‘‘‘‘JLabel label = new JLabel("JFrame By Example");‘‘
‘‘‘‘‘‘‘‘JButton button = new JButton();‘‘
‘‘‘‘‘‘‘‘button.setText("Button");‘‘
‘‘‘‘‘‘‘‘panel.add(label);‘‘
‘‘‘‘‘‘‘‘panel.add(button);‘‘
‘‘‘‘‘‘‘‘frame.add(panel);‘‘
‘‘‘‘‘‘‘‘frame.setSize(200, 300);‘‘
‘‘‘‘‘‘‘‘frame.setLocationRelativeTo(null);‘‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘‘‘‘‘frame.setVisible(true);‘‘
```

```
‘‘‘‘}’‘
}’‘
```

Illustration 10.2

```
// A program to add panel to GUI

import java.awt.*;’‘
import javax.swing.*;’‘
public class PanelExample {‘‘
‘‘‘‘ PanelExample()’‘{‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame("Panel Example");’‘‘‘
‘‘‘‘‘‘‘‘JPanel panel=new JPanel();’‘
‘‘‘‘‘‘‘‘panel.setBounds(40,80,200,200);’‘‘‘
‘‘‘‘‘‘‘‘panel.setBackground(Color.gray);’‘
‘‘‘‘‘‘‘‘JButton b1=new JButton("Button 1");’‘‘‘
‘‘‘‘‘‘‘‘b1.setBounds(50,100,80,30);’‘‘‘
‘‘‘‘‘‘‘‘b1.setBackground(Color.yellow);’‘
‘‘‘‘‘‘‘‘JButton b2=new JButton("Button 2");’‘
‘‘‘‘‘‘‘‘b2.setBounds(100,100,80,30);’‘‘‘
‘‘‘‘‘‘‘‘b2.setBackground(Color.green);’‘
‘‘‘‘‘‘‘‘panel.add(b1);
                    panel.add(b2);’‘
‘‘‘‘‘‘‘‘f.add(panel);’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setSize(400,400);’‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setVisible(true);’‘‘‘
‘‘‘‘‘‘‘‘}’‘
‘‘‘‘‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘new PanelExample();’‘
‘‘‘‘‘‘‘‘}’‘
‘‘‘‘}’‘
```

We can also write all the codes of creating JFrame, JButton and method call inside the Java constructor.

Illustration 10.3

```
‘‘
//Create a Swing button
```

```
import javax.swing.*;‘‘
public class FirstSwingExample {‘‘
          public static void main(String[] args) {‘‘
                    //creating instance of JFrame
                    JFrame f=new JFrame();
‘‘‘‘‘‘‘‘‘          //creating instance of JButton’‘
                    JButton b=new JButton("click");
                    //x axis, y axis, width, height
                    b.setBounds(130,100,100, 40);
                    f.add(b);//adding button in JFrame
                    f.setSize(400,500);//400 width and 500 height’‘
                    f.setLayout(null);//using no layout managers’‘
                    f.setVisible(true);//making the frame visible’‘
          }’‘
}
```

Illustration 10.4

```
// Creating JFrame, JButton and method call inside the java constructor.

import javax.swing.*;‘‘
public class Simple {‘‘
          JFrame f;’‘
          Simple(){‘‘
          f=new JFrame();
          JButton b=new JButton("click");
          b.setBounds(130,100,100, 40);’‘
          f.add(b);
          f.setSize(400,500);
          f.setLayout(null);
          f.setVisible(true);
}
          public static void main(String[] args) {‘‘
                    new Simple();’‘
          }’‘
}
```

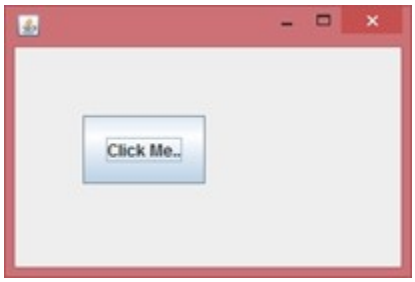
Illustration 10.5

```
// inherit the JFrame class, so there is no need to create the instance of JFrame class explicitly.


import javax.swing.*;
//inheriting JFrame’‘
public class Simple2 extends JFrame{
          JFrame f;’‘
          Simple2(){‘‘
                    JButton b=new JButton("click");
                    b.setBounds(130,100,100, 40);’‘
                    add(b);
                    setSize(400,500);’‘
          ‘‘‘‘‘‘ setLayout(null);’‘
                    setVisible(true);’‘
          }’‘
          public static void main(String[] args) {‘‘
                    new Simple2();’‘
          }
}
```

Java Swing JButton

The **JButton class** is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed. It inherits AbstractButton class. Below is the declaration for **javax.swing.JButton class.**
public class JButton extends AbstractButton implements Accessible



Class JButton : Constructors:

Constructor	Description
JButton()	It creates a button with no text and i
JButton(String s)	It creates a button with the specified
JButton(Icon i)	It creates a button with the specified

Class JButton : Methods:

Methods	Descrip
`void setText(String s)`	It is used to set specified text on but
`String getText()`	It is used to return the text of the bu
`void setEnabled(boolean b)`	It is used to enable or disable the bu
`void setIcon(Icon b)`	It is used to set the specified Icon or
`Icon getIcon()`	It is used to get the Icon of the butto
`void setMnemonic(int a)`	It is used to set the mnemonic on the
`void addActionListener(ActionListener a)`	It is used to add the action listener t

Creating a JButton : An example

Let's see a simple swing example where we are creating one button and adding it on the JFrame object inside the main() method. also illustarte Java JButton : An example with ActionListener

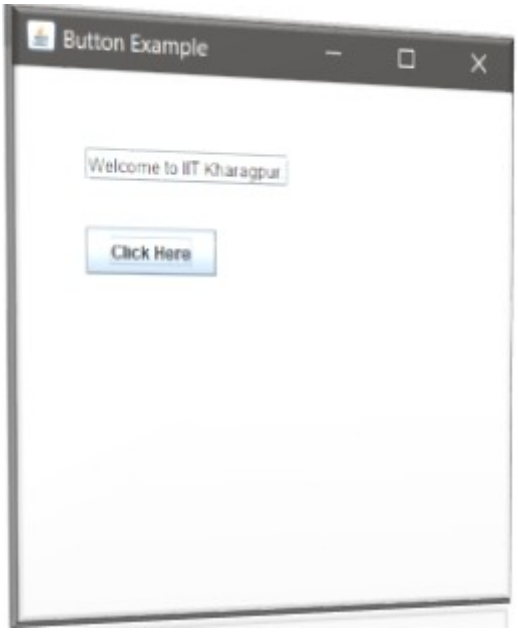


Illustration 10.6

```
// Button with Action Listner
import java.awt.event.*;‘‘
import javax.swing.*;’’‘‘
```

```
public class ButtonExample {‘‘
public static void main(String[] args) {‘‘
‘‘‘‘JFrame f=new JFrame("Button Example");’’
‘‘‘‘final JTextField tf=new JTextField();’’
‘‘‘‘tf.setBounds(50,50, 150,20);’’
‘‘‘‘JButton b=new JButton("Click Here");’’
‘‘‘‘b.setBounds(50,100,95,30);’’
‘‘‘‘b.addActionListener(new ActionListener(){‘‘
‘‘‘‘public void actionPerformed(ActionEvent e){‘‘
‘‘‘‘‘‘‘‘‘‘‘‘tf.setText("Welcome to Javatpoint.");’’
‘‘‘‘‘‘‘‘}’’
‘‘‘‘});’’
‘‘‘‘f.add(b);f.add(tf);’’
‘‘‘‘f.setSize(400,400);’’
‘‘‘‘f.setLayout(null);’’
‘‘‘‘f.setVisible(true);’’
}’’
}’’
```

Java JButton : Displaying image on the button



Illustration 10.7

```
// Button with image
```

```
import javax.swing.*;‘‘‘‘‘‘
public class ButtonExample{‘‘‘‘
ButtonExample(){‘‘‘‘
          JFrame f=new JFrame("Button Example");‘‘‘‘‘‘‘‘‘‘‘‘
          JButton b=new JButton(new ImageIcon("D:\\icon.png"));‘‘‘‘
          b.setBounds(100,100,100, 40);‘‘‘‘
          f.add(b);‘‘‘‘
          f.setSize(300,400);‘‘‘‘
          f.setLayout(null);‘‘‘‘
          f.setVisible(true);‘‘‘‘
          f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘‘‘
‘‘‘‘}‘‘‘‘‘‘‘
          public static void main(String[] args) {‘‘‘‘
                    new ButtonExample();‘‘‘‘
          }‘‘‘‘
}
```

Java Swing JLabel:

The object of **JLabel class** is a component for placing text in a container. It is used to display a single line of read only text. The text can be changed by an application but a user cannot edit it directly. It inherits **JComponent class.**



Below is the declaration for javax.swing.JLabel class. **public�class�JLabel�extends�JComponent�implements�SwingConstants,�Accessible�**

Illustration 10.8

```
import javax.swing.*;
class LabelExample‘‘
{‘‘
public static void main(String args[]){‘‘
```

```
‘‘‘‘JFrame f= new JFrame("Label Example");’‘
‘‘‘‘JLabel l1,l2;’‘
‘‘‘‘l1=new JLabel("First Label.");’‘
‘‘‘‘l1.setBounds(50,50, 100,30);’‘
‘‘‘‘l2=new JLabel("Second Label.");’‘
‘‘‘‘l2.setBounds(50,100, 100,30);’‘
‘‘‘‘f.add(l1); f.add(l2);’‘
‘‘‘‘f.setSize(300,300);’‘
‘‘‘‘f.setLayout(null);’‘
‘‘‘‘f.setVisible(true);’‘
‘‘‘‘}’‘
}’‘
```

Java JLabel : An example with ActionListener

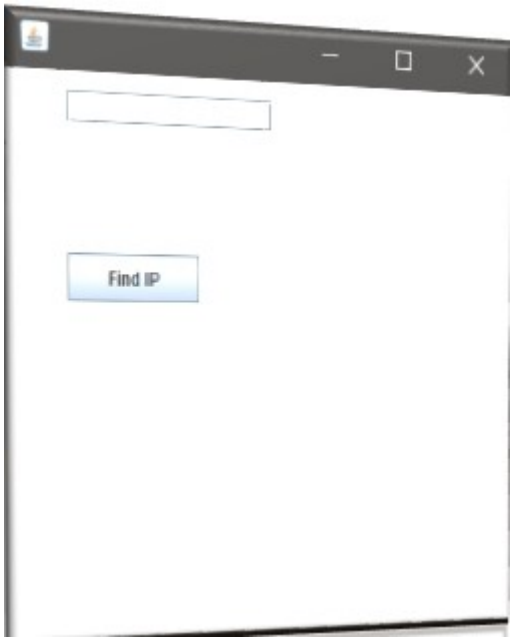


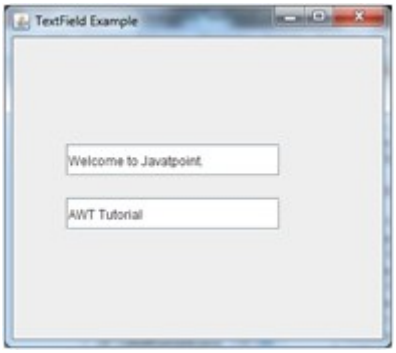
Illustration 10.9

```
import javax.swing.*;’‘
import java.awt.*;’‘
import java.awt.event.*;
‘‘
public class LabelExample extends Frame implements ActionListener{‘‘
```

```
‘‘‘‘JTextField tf; JLabel l; JButton b;’‘
‘‘‘‘LabelExample(){‘‘
‘‘‘‘‘‘‘‘tf=new JTextField();’‘
‘‘‘‘‘‘‘‘tf.setBounds(50,50, 150,20);’‘
‘‘‘‘‘‘‘‘l=new JLabel();’‘
‘‘‘‘‘‘‘‘l.setBounds(50,100, 250,20);’‘‘‘‘‘
‘‘‘‘‘‘‘‘b=new JButton("Find IP");’‘
‘‘‘‘‘‘‘‘b.setBounds(50,150,95,30);’‘
‘‘‘‘‘‘‘‘b.addActionListener(this);’‘‘‘
‘‘‘‘‘‘‘‘add(b);add(tf);add(l);’‘‘‘
‘‘‘‘‘‘‘‘setSize(400,400);’‘
‘‘‘‘‘‘‘‘setLayout(null);’‘
‘‘‘‘‘‘‘‘setVisible(true);’‘
‘‘‘‘}’‘
```

Java Swing JTextField:

The object of a **JTextField class** is a text component that allows the editing of a single line text. It inherits **JTextComponent class.**



Below is the declaration for javax.swing.JTextField class. **public class JTextField extends JTextComponent implements SwingConstants**

Class JTextField : Constructors

Constructor	Description
`JTextField()`	Creates a new TextF
`JTextField(String text)`	Creates a new TextField initia specified text.
`JTextField(String text, int columns)`	Creates a new TextField initia specified text and colu
`JTextField(int columns)`	Creates a new empty TextFi specified number of co

Class JTextField : Methods

Methods	Desc
`void addActionListener(ActionListener l)`	It is used to add the specif action events from this tex
`Action getAction()`	It returns the currently set source, or null if no Actio
`void setFont(Font f)`	It is used to set the current
`void removeActionListener(ActionListener l)`	It is used to remove the sp that it no longer receives a textfield.

```
Illustration 10.10
/*
```

```
Simple calculator for Addition and Subtraction of numbers in TextField
*/

import javax.swing.*;‘‘
import java.awt.event.*;‘‘
public class TextFieldExample implements ActionListener{‘‘
‘‘‘‘JTextField tf1,tf2,tf3;‘‘
‘‘‘‘JButton b1,b2;‘‘
‘‘‘‘TextFieldExample(){‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame();‘‘
‘‘‘‘‘‘‘‘tf1=new JTextField();‘‘
‘‘‘‘‘‘‘‘tf1.setBounds(50,50,150,20);‘‘
‘‘‘‘‘‘‘‘tf2=new JTextField();‘‘
‘‘‘‘‘‘‘‘tf2.setBounds(50,100,150,20);‘‘
‘‘‘‘‘‘‘‘tf3=new JTextField();‘‘
‘‘‘‘‘‘‘‘tf3.setBounds(50,150,150,20);‘‘
‘‘‘‘‘‘‘‘tf3.setEditable(false);‘‘
‘‘‘‘‘‘‘‘b1=new JButton("+");‘‘
‘‘‘‘‘‘‘‘b1.setBounds(50,200,50,50);‘‘
‘‘‘‘‘‘‘‘b2=new JButton("-");‘‘
‘‘‘‘‘‘‘‘b2.setBounds(120,200,50,50);‘‘
‘‘‘‘‘‘‘‘b1.addActionListener(this);‘‘
‘‘‘‘‘‘‘‘b2.addActionListener(this);‘‘
‘‘‘‘‘‘‘‘f.add(tf1);f.add(tf2);f.add(tf3);f.add(b1);f.add(b2);
‘‘‘‘‘‘‘‘ f.setSize(300,300);‘‘
‘‘‘‘‘‘‘‘ f.setLayout(null);‘‘
‘‘‘‘‘‘‘‘ f.setVisible(true);‘‘
‘‘‘‘}‘‘‘‘‘‘
‘‘‘‘public void actionPerformed(ActionEvent e) {‘‘
‘‘‘‘‘‘‘‘String s1=tf1.getText();‘‘
‘‘‘‘‘‘‘‘String s2=tf2.getText();‘‘
‘‘‘‘‘‘‘‘int a=Integer.parseInt(s1);‘‘
‘‘‘‘‘‘‘‘int b=Integer.parseInt(s2);‘‘
‘‘‘‘‘‘‘‘int c=0;‘‘
‘‘‘‘‘‘‘‘if(e.getSource()==b1){‘‘
‘‘‘‘‘‘‘‘‘‘‘‘c=a+b;‘‘
‘‘‘‘‘‘‘‘}else if(e.getSource()==b2){‘‘
‘‘‘‘‘‘‘‘‘‘‘‘c=a-b;‘‘
‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘String result=String.valueOf(c);‘‘
‘‘‘‘‘‘‘‘tf3.setText(result);‘‘
```

```
‘‘‘‘}’’
          public static void main(String[] args) {‘‘
                    new TextFieldExample();’‘
          }
}
```

Java Swing JTextArea:

The object of a **JTextArea class** is a multi line region that displays text. It allows the editing of multiple line text. It inherits **JTextComponent class.**



Class JTextArea : constructors:

Constructor	Description
JTextArea()	Creates a text area that displays no text i
JTextArea(String s)	Creates a text area that displays specifie initially.
JTextArea(int row, int column)	Creates a text area with the specified nu rows and columns that displays no text i
JTextArea(String s, int row, int column)	Creates a text area with the specified nu rows and columns that displays specifie

Class JTextArea : methods:

Methods	Description
`void setRows(int rows)`	It is used to set specified number of r
`void setColumns(int cols)`	It is used to set specified number of c
`void setFont(Font f)`	It is used to set the specified font.
`void insert(String s, int position)`	It is used to insert the specified text o position.
`void append(String s)`	It is used to append the given text to t document.

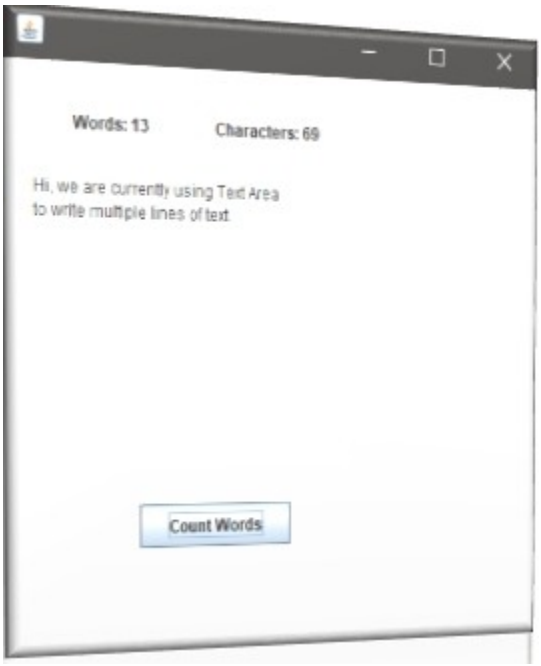
Illustration 10.11

```
/*
Simple program to find number of words and characters in a TextArea
*/


import javax.swing.*;‘‘
import java.awt.event.*;‘‘
public class TextAreaExample implements ActionListener{‘‘
JLabel l1,l2;‘‘
JTextArea area;‘‘
JButton b;‘‘
TextAreaExample() {‘‘
‘‘‘‘JFrame f= new JFrame();‘‘
‘‘‘‘l1=new JLabel();‘‘
‘‘‘‘l1.setBounds(50,25,100,30);‘‘
‘‘‘‘l2=new JLabel();‘‘
‘‘‘‘l2.setBounds(160,25,100,30);‘‘
‘‘‘‘area=new JTextArea();‘‘
‘‘‘‘area.setBounds(20,75,250,200);‘‘
‘‘‘‘b=new JButton("Count Words");‘‘
‘‘‘‘b.setBounds(100,300,120,30);‘‘
‘‘‘‘b.addActionListener(this);‘‘
```

```
‘‘‘‘f.add(l1);f.add(l2);f.add(area);f.add(b);
‘‘‘‘f.setSize(450,450);’‘
‘‘‘‘f.setLayout(null);’‘
‘‘‘‘f.setVisible(true);’‘
}’‘
public void actionPerformed(ActionEvent e){‘‘
‘‘‘‘String text=area.getText();’‘
‘‘‘‘String words[]=text.split("\\s");’‘
‘‘‘‘l1.setText("Words: "+words.length);’‘
‘‘‘‘l2.setText("Characters: "+text.length());’‘
}’‘
public static void main(String[] args) {‘‘
‘‘‘‘new TextAreaExample();’‘
}’‘
}’‘
```

Java JTextArea : An example with ActionListener:



Java JPasswordField: The object of a **JPasswordField class** is a text component specialized for password entry. It allows the editing of a single line of text. It inherits **JTextField class.**



The declaration for javax.swing.JPasswordField class is **public**&#65533;**class**&#65533;**JPasswordField**&#65533;**extends**&#65533;**JTextField.**
Class JPasswordField : Constructors:

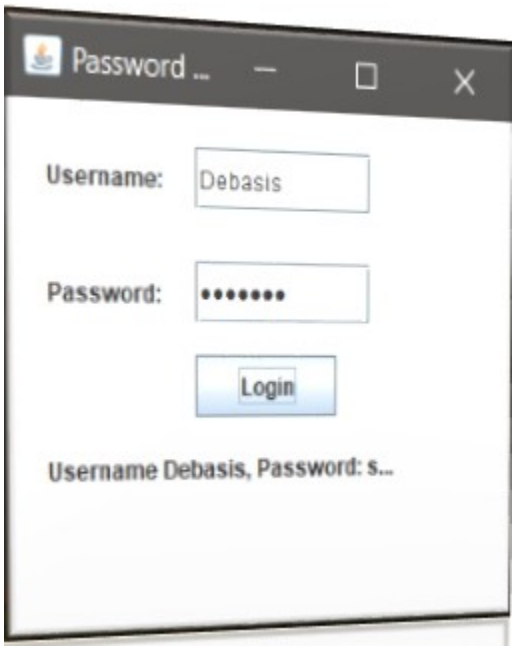
Constructor	Descripti
JPasswordField()	Constructs a new JPassword document, null starting text strin
JPasswordField(int columns)	Constructs a new empty JPas specified number o
JPasswordField(String text)	Constructs a new JPasswordFi specified te
JPasswordField(String text, int columns)	Construct a new JPasswordFie specified text and

Illustration 10.12

```
/*
Simple program to make a login GUI using TextField, PasswordField and Login Button
*/

import javax.swing.*;‘‘‘‘
import java.awt.event.*;‘‘
public class PasswordFieldExample {‘‘
‘‘public static void main(String[] args) {‘‘‘‘
‘‘‘‘JFrame f=new JFrame("Password Field Example");‘‘‘‘
‘‘‘‘final JLabel label = new JLabel();‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘label.setBounds(20,150, 200,50);‘‘
‘‘‘‘final JPasswordField value = new JPasswordField();‘‘
‘‘‘‘value.setBounds(100,75,100,30);‘‘
‘‘‘‘JLabel l1=new JLabel("Username:");‘‘‘‘
‘‘‘‘l1.setBounds(20,20, 80,30);‘‘‘‘
‘‘‘‘JLabel l2=new JLabel("Password:");‘‘‘‘
‘‘‘‘l2.setBounds(20,75, 80,30);‘‘‘‘
‘‘‘‘JButton b = new JButton("Login");‘‘
‘‘‘‘b.setBounds(100,120, 80,30);‘‘‘‘
‘‘‘‘final JTextField text = new JTextField();‘‘
‘‘‘‘text.setBounds(100,20, 100,30);‘‘‘‘
‘‘‘‘f.add(value);
‘‘‘‘f.add(l1);
‘‘‘‘f.add(label);
```

```
‘‘‘‘f.add(l2);
‘‘‘‘f.add(b);’‘‘‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘f.add(text);
‘‘ }’‘
‘‘‘‘f.setSize(300,300);’‘‘‘
‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘f.setVisible(true);’‘‘‘
‘‘‘‘b.addActionListener(new ActionListener() {‘‘
‘‘‘‘‘‘ public void actionPerformed(ActionEvent e) {‘‘‘‘‘‘
‘‘‘‘‘‘‘‘String data = "Username " + text.getText();’‘
‘‘‘‘‘‘‘‘data += ", Password: "+ new String(value.getPassword());’‘
‘‘‘‘‘‘‘‘label.setText(data);’‘‘‘‘‘‘‘‘‘
‘‘‘‘‘‘ }’‘
‘‘‘‘});’‘
‘‘ }’‘
}’‘
```



Java Swing JCheckBox: The JCheckBox class is used to create a checkbox. It is used to turn an option on (true) or off (false). Clicking on a JCheckBox changes its state from "on" to "off" or from "off" to "on ".It inherits JToggleButton class.



Below is the declaration for javax.swing.JCheckBox class. **public class JCheckBox extends JToggleButton implements Accessible**

Class JCheckBox : Constructors:

Constructor	*Description*
`JCheckBox()`	Creates an initially unselected check box button text, no icon.
`JChechBox(String s)`	Creates an initially unselected check box with te
`JCheckBox(String text, boolean selected)`	Creates a check box with text and specifies whet is initially selected.
`JCheckBox(Action a)`	Creates a check box where properties are taken f Action supplied.

Class JCheckBox : Methods:

Methods	*Descrip*
`AccessibleContext getAccessibleContext()`	It is used to get the Accessib with this JCheckBox.
`protected String paramString()`	It returns a string representa

Illustration 10.13

```
// A program to show use of CheckBox


import javax.swing.*;‘‘
public class CheckBoxExample’‘
{‘‘
‘‘‘‘ CheckBoxExample(){‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame("CheckBox Example");’‘
‘‘‘‘‘‘‘‘JCheckBox checkBox1 = new JCheckBox("C++");’‘
‘‘‘‘‘‘‘‘checkBox1.setBounds(100,100, 50,50);’‘
```

```
‘‘‘‘‘‘‘‘JCheckBox checkBox2 = new JCheckBox("Java", true);’’‘
‘‘‘‘‘‘‘‘checkBox2.setBounds(100,150, 50,50);’’‘
‘‘‘‘‘‘‘‘f.add(checkBox1);’’‘
‘‘‘‘‘‘‘‘f.add(checkBox2);’’‘
‘‘‘‘‘‘‘‘f.setSize(400,400);’’‘
‘‘‘‘‘‘‘‘f.setLayout(null);’’‘
‘‘‘‘‘‘‘‘f.setVisible(true);’’‘
‘‘‘‘ }’’‘
‘‘‘‘public static void main(String args[]){‘‘
‘‘‘‘‘‘new CheckBoxExample();’’‘
‘‘‘‘}
}’’‘
```



Java Swing JRadioButton: The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. It is widely used in exam systems or quiz. It should be added in ButtonGroup to select one radio button only.



Below is the declaration for javax.swing.JRadioButton class. **public**␣**class**␣**JRadioButton**␣**extends**␣**JToggleButton**␣**implements**␣**Accessible**␣␣

Class JRadioButton : Constructors:

Constructor	Description
JRadioButton()	Creates an unselected radio butt text.
JRadioButton(String s)	Creates an unselected radio butt specified text.
JRadioButton(String s, boolean selected)	Creates a radio button with the s text and selected status.

Class JRadioButton : Methods:

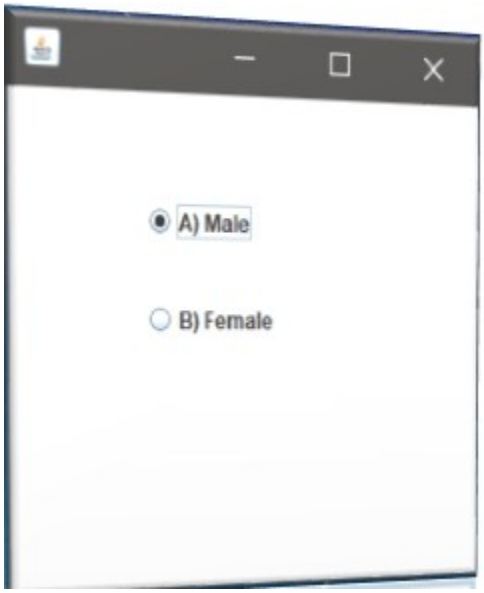
Methods	Description
void setText(String s)	It is used to set specified text on button.
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener(ActionListener a)	It is used to add the action listener to this object

Illustration 10.14

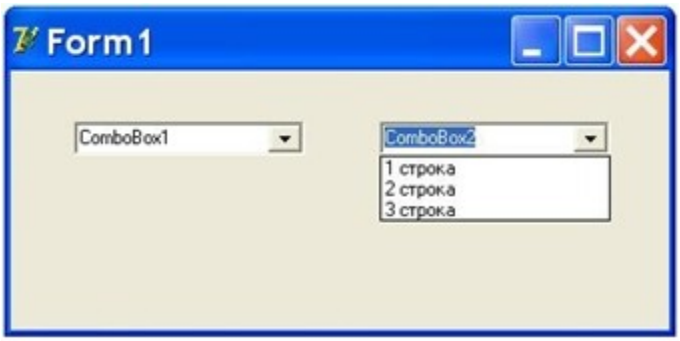
```
// A program to show use of RadioButton


import javax.swing.*;‘‘‘‘
public class RadioButtonExample {‘‘‘‘
JFrame f;‘‘‘‘
RadioButtonExample(){‘‘‘‘
f=new JFrame();‘‘‘‘
JRadioButton r1=new JRadioButton("A) Male");‘‘‘‘
```

```
JRadioButton r2=new JRadioButton("B) Female");‘‘‘‘
r1.setBounds(75,50,100,30);‘‘‘‘
r2.setBounds(75,100,100,30);‘‘‘‘
ButtonGroup bg=new ButtonGroup();‘‘‘‘
bg.add(r1);bg.add(r2);‘‘‘‘
f.add(r1);f.add(r2);‘‘‘‘‘‘
f.setSize(300,300);‘‘‘‘
f.setLayout(null);‘‘‘‘
f.setVisible(true);‘‘‘‘
}‘‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘new RadioButtonExample();‘‘‘‘
}}
```



Java Swing JComboBox: The object of Choice class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu. It inherits JComponent class.



Below is the declaration for javax.swing.JComboBox class. **public class JComboBox extends JComponent implements ItemSelectable, ListDataListener, ActionListener, Accessible**

Class JComboBox : Constructors:

Constructor	*Description*
`JComboBox()`	Creates a JComboBox with a default data
`JComboBox(Object[] items)`	Creates a JComboBox that contains the e in the specified array.
`JComboBox(Vector<?> items)`	Creates a JComboBox that contains the e in the specified Vector.

Class JComboBox : Methods:

Methods	*Descriptio*
`void addItem(Object anObject)`	It is used to add an item to the item list.
`void removeItem(Object anObject)`	It is used to delete an item to the item lis
`void removeAllItems()`	It is used to remove all the items from th
`void setEditable(boolean b)`	It is used to determine whether the JCon
`void addActionListener(ActionListener a)`	It is used to add the ActionListener.
`void addItemListener(ItemListener i)`	It is used to add the ItemListener.

Illustration 10.15

```
// A program to show use of ComboBox


import javax.swing.*;‘‘‘‘
public class ComboBoxExample {‘‘‘‘
JFrame f;‘‘‘‘
ComboBoxExample(){‘‘‘‘
‘‘‘‘f=new JFrame("ComboBox Example");‘‘‘‘
```

```
‘‘‘‘String country[]={"India","Aus","U.S.A","England","Newzealand"};’‘‘‘‘‘‘‘
‘‘‘‘JComboBox cb=new JComboBox(country);’‘‘‘
‘‘‘‘cb.setBounds(50, 50,90,20);’‘‘‘
‘‘‘‘f.add(cb);’‘‘‘‘‘‘‘
‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘f.setSize(400,500);’‘‘‘
‘‘‘‘f.setVisible(true);’‘‘‘‘‘‘‘
}’‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘new ComboBoxExample();’‘‘‘‘‘‘‘
}’‘‘‘
}
```



Java Swing JTable: The JTable class is used to display data in tabular form. It is composed of rows and columns



Class JTable : Constructors:

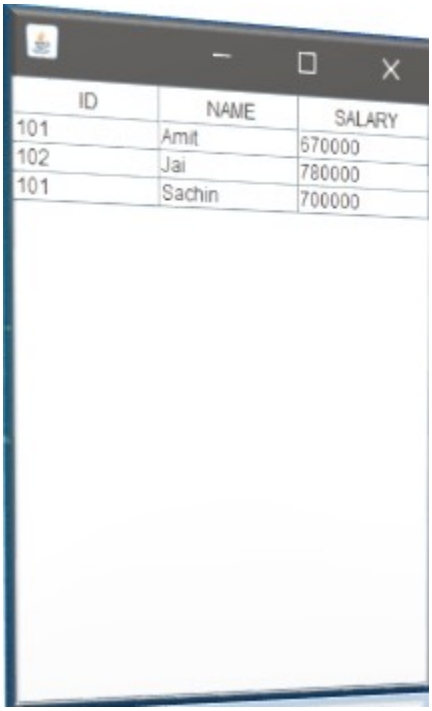
Constructor	*Descriptio*
`JTable()`	Creates a table with empty ce
`JTable(Object[][] rows, Object[] columns)`	Creates a table with the specif

Illustration 10.16

```
// A program to create a table and show some data


import javax.swing.*;‘‘‘‘
public class TableExample {‘‘‘‘
‘‘‘‘JFrame f;‘‘‘‘
‘‘‘‘TableExample(){‘‘‘‘
‘‘‘‘f=new JFrame();‘‘‘‘
‘‘‘‘String data[][]={ {"101","Amit","670000"},‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘{"102","Jai","780000"},‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘{"101","Sachin","700000"}};‘‘‘‘
‘‘‘‘String column[]={"ID","NAME","SALARY"};‘‘‘‘‘‘‘
‘‘‘‘JTable jt=new JTable(data,column);‘‘‘‘
‘‘‘‘jt.setBounds(30,40,200,300);‘‘‘‘‘‘‘‘‘
‘‘‘‘JScrollPane sp=new JScrollPane(jt);‘‘‘‘
‘‘‘‘f.add(sp);‘‘‘‘‘‘‘‘‘
‘‘‘‘f.setSize(300,400);‘‘‘‘
‘‘‘‘f.setVisible(true);‘‘‘‘
}‘‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘new TableExample();‘‘‘‘
}‘‘‘‘
```

```
}’‘
```



Java Swing JList: The object of JList class represents a list of text items. The list of text items can be set up so that the user can choose either one item or multiple items. It inherits JComponent class.



Below is the declaration for javax.swing.JList class. **public****class****JList****extends****JComponent****implements****Scrollable,Accessible**

Class JList : Constructors:

Constructor	Description
`JList()`	Creates a JList with an empty, read-only,
`JList(ary[] listData)`	Creates a JList that displays the elements specified array.
`JList(ListModel<ary> dataModel)`	Creates a JList that displays elements fro specified, non-null, model.

Class JList : Methods:

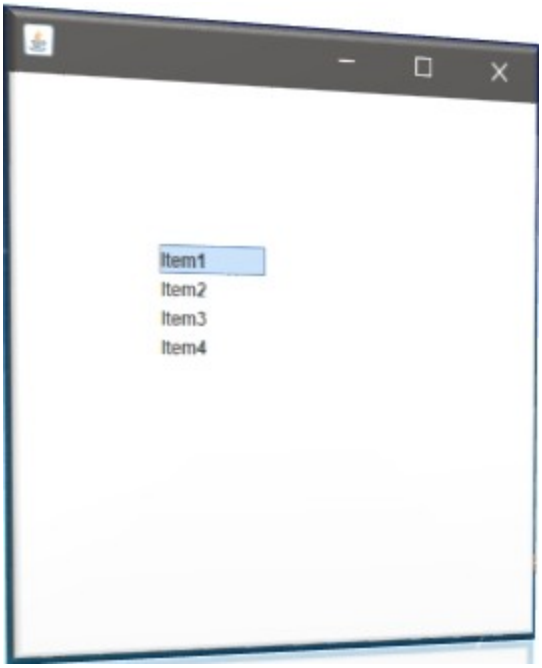
Methods	D
`Void addListSelectionListener(ListSelectionListener listener)`	It is used to ad be notified eac selection occu
`int getSelectedIndex()`	It is used to re selected cell in
`ListModel getModel()`	It is used to re holds a list of JList compon
`void setListData(Object[] listData)`	It is used to cr ListModel fro

Illustration 10.17

```
// A program to create a list and list some data


import javax.swing.*;‘‘
public class ListExample’‘
{‘‘
‘‘‘‘ ListExample(){‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame();’‘
‘‘‘‘‘‘‘‘DefaultListModel l1 = new DefaultListModel<>();’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item1");’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item2");’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item3");’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item4");’‘
‘‘‘‘‘‘‘‘‘‘JList list = new JList<>(l1);’‘
```

```
‘‘‘‘‘‘‘‘‘list.setBounds(100,100, 75,75);’‘
‘‘‘‘‘‘‘‘‘f.add(list);’‘
‘‘‘‘‘‘‘‘‘f.setSize(400,400);’‘
‘‘‘‘‘‘‘‘‘f.setLayout(null);’‘
‘‘‘‘‘‘‘‘‘f.setVisible(true);’‘
‘‘‘‘ }’‘
public static void main(String args[])’‘
‘‘‘‘{‘‘
‘‘ new ListExample();’‘
‘‘‘‘}}’‘
```



Java Swing JOptionPane: The JOptionPane class is used to provide standard dialog boxes such as message dialog box, confirm dialog box and input dialog box. These dialog boxes are used to display information or get input from the user. The JOptionPane class inherits JComponent class.



Below is the declaration for javax.swing.JOptainPane class. **public�class JOptionPane�extends�JComponent�implements�Accessible���**

Class JOptionPane : Constructors:

Constructor	Description
JOptionPane()	It is used to create a JOptionPane message.
JOptionPane(Object message)	It is used to create an instance of display a message.
JOptionPane(Object message, int messageType)	It is used to create an instance of display a message with specified and default options.

Class JOptionPane : Methods:

Methods	Descripti
JDialog createDialog(String title)	It is used to create and return a new parentle title.
static void showMessageDialog(Component parentComponent, Object message)	It is used to create an information-message
static void showMessageDialog(Component parentComponent, Object message, String title, int messageType)	It is used to create a message dialog with giv
static int showConfirmDialog(Component parentComponent, Object message)	It is used to create a dialog with the options title, Select an Option.
static String showInputDialog(Component parentComponent, Object message)	It is used to show a question-message dialog parented to parentComponent.
void setInputValue(Object newValue)	It is used to set the input value that was sele

Illustration 10.18

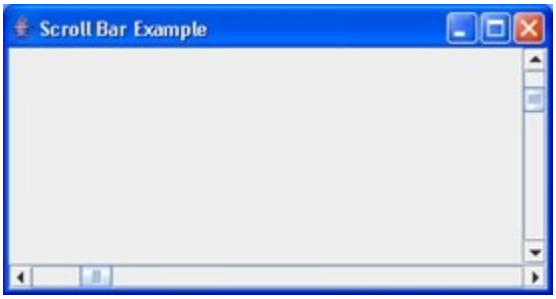
```
// A program to show some info to using JOptionPane


import javax.swing.*;‘‘
public class OptionPaneExample {‘‘
JFrame f;‘‘
OptionPaneExample(){‘‘
‘‘‘‘f=new JFrame();‘‘
‘‘‘‘JOptionPane.showMessageDialog(f,"Hello, Welcome to IIT Kharagpur.");‘‘
}‘‘
public static void main(String[] args) {‘‘
‘‘‘‘new OptionPaneExample();‘‘
```

```
}’‘
}’‘
```



Java Swing JScrollBar: The object of JScrollBar class is used to add horizontal and vertical scrollbar. It is an implementation of a scrollbar. It inherits JComponent class.



Below is the declaration for javax.swing.JScrollBar class. **public class JScrollBar extends JComponent implements Adjustable, Accessible**

Class JScrollBar: Constructors:

Constructor	Description
JScrollBar()	Creates a vertical scrollbar with the initial
JScrollBar(int orientation)	Creates a scrollbar with the specified orie and the initial values.
JScrollBar(int orientation, int value, int extent, int min, int max)	Creates a scrollbar with the specified orie value, extent, minimum, and maximum.

Illustration 10.19

```
import javax.swing.*;’‘
class ScrollBarExample’‘
{‘‘
ScrollBarExample(){‘‘
‘‘‘‘JFrame f= new JFrame("Scrollbar Example");’‘
```

```
 JScrollBar s=new JScrollBar();’‘
s.setBounds(100,100, 50,100);’‘
f.add(s);’‘
f.setSize(400,400);’‘
f.setLayout(null);’‘
f.setVisible(true);’‘
}’‘
public static void main(String args[])’‘
{‘‘
new ScrollBarExample();’‘
}}’‘
```

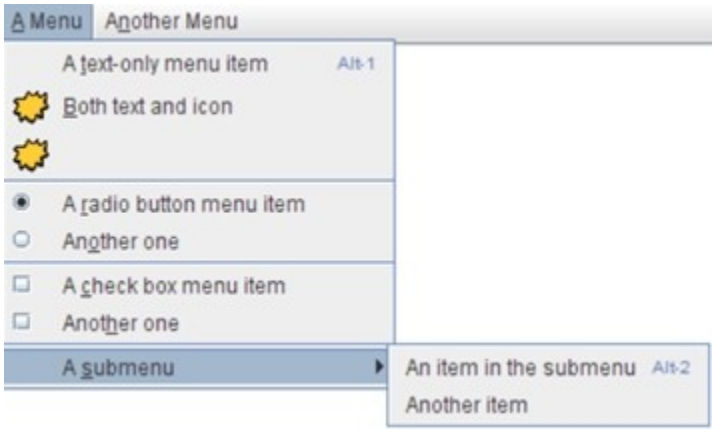


Java Swing JMenuBar: The JMenuBar class is used to display menu bar on the window or frame. It may have several menus.



Below is the declaration for javax.swing.JMenuBar class. **public class JMenuBar extends JComponent implements MenuElement, Accessible**

Java Swing JMenu: The object of JMenu class is a pull down menu component which is displayed from the menu bar. It inherits the JMenuItem class.



Below is the declaration for javax.swing.JMenuItem
class **JMenu**&#65533;**extends**&#65533;**JMenuItem**&#65533;**implements**&#65533;**MenuElement,**&#65533;**Accessible**

Class JMenuItem: The object of JMenuItem class adds a simple labeled menu item. The items used in a menu must belong to the JMenuItem or any of its subclass.

Below is the declaration for javax.swing.JMenuItem
class. **public**&#65533;**class**&#65533;**JMenuItem**&#65533;**extends**&#65533;**AbstractButton**&#65533;**implements**&#65533;**Accessible,**&#65533;**MenuElement**&#65533;&#65533;

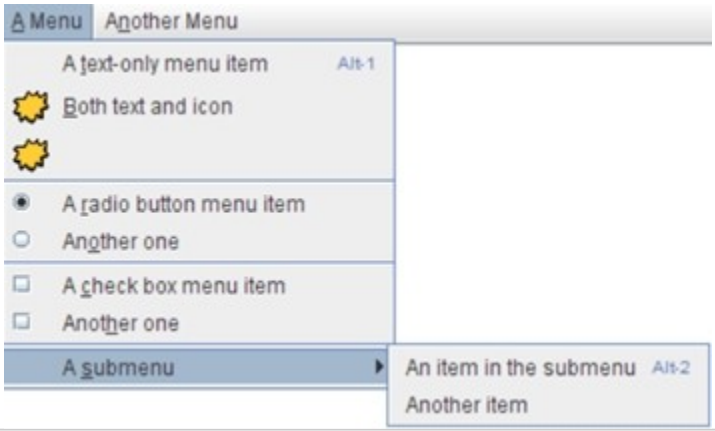
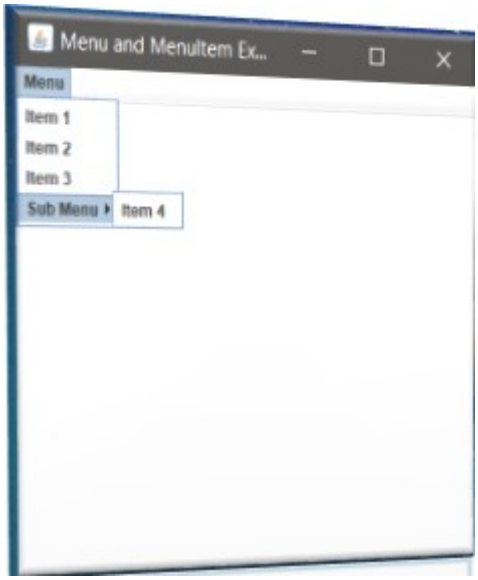


Illustration 10.20

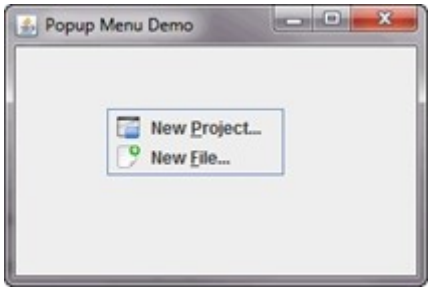
```
// A program to create a menu bar and show menu items


import javax.swing.*;‘‘
class MenuExample{‘‘
‘‘‘‘JMenu menu, submenu;’’
‘‘‘‘JMenuItem i1, i2, i3, i4, i5;’’
‘‘‘‘MenuExample(){‘‘
‘‘‘‘‘‘ JFrame f= new JFrame("Menu and MenuItem Example");’’
‘‘‘‘‘‘ JMenuBar mb=new JMenuBar();’’
‘‘‘‘‘‘ menu=new JMenu("Menu");’’
‘‘‘‘‘‘ submenu=new JMenu("Sub Menu");’’
‘‘‘‘‘‘ i1=new JMenuItem("Item 1");i2=new JMenuItem("Item 2");’’
‘‘‘‘‘‘ i3=new JMenuItem("Item 3");i4=new JMenuItem("Item 4");’’
‘‘‘‘‘‘ menu.add(i1); menu.add(i2); menu.add(i3);’’
```

```
‘‘‘‘‘‘ submenu.add(i4);
‘‘‘‘‘‘ menu.add(submenu);’‘‘‘‘‘‘‘ mb.add(menu);’‘
‘‘‘‘‘‘ f.setJMenuBar(mb);’‘
‘‘‘‘‘‘ f.setSize(400,400);’‘‘‘‘‘‘‘f.setLayout(null);’‘
‘‘‘‘‘‘ f.setVisible(true);’‘
          }
          public static void main(String args[]){‘‘
                    new MenuExample();’‘
          }
}
```



Java Swing JPopupMenu: JPopupMenu can be dynamically popped up at specific position within a component. It inherits the JComponent class.



Below is the declaration for javax.swing.JPopupMenu class. **public‘class‘JPopupMenu‘extends‘JComponent‘implements‘Accessible,‘MenuElement‘‘**

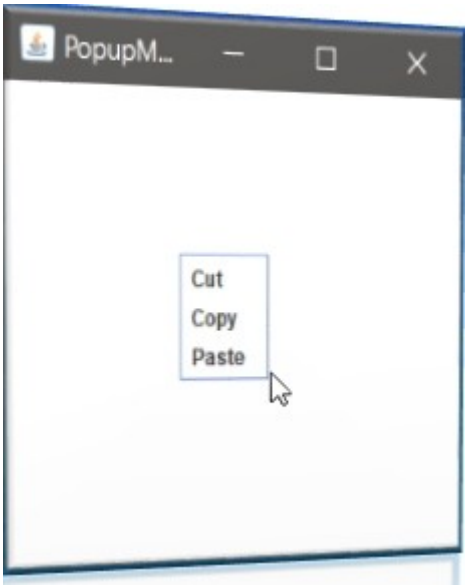
Class JPopupMenu : Constructors:

Constructor	Description
JPopupMenu()	Constructs a JPopupMenu without a "invoker".
JPopupMenu(String label)	Constructs a JPopupMenu with the specified title.

Illustration 10.21

```
// A program to show some menu using PopupMenu


import javax.swing.*;‘‘
import java.awt.event.*;‘‘
class PopupMenuExample {‘‘
‘‘‘‘ PopupMenuExample(){‘‘
‘‘‘‘‘‘‘‘ final JFrame f= new JFrame("PopupMenu Example");‘‘
‘‘‘‘‘‘‘‘ final JPopupMenu popupmenu = new JPopupMenu("Edit");‘‘
‘‘‘‘‘‘‘‘ JMenuItem cut = new JMenuItem("Cut");‘‘
‘‘‘‘‘‘‘‘ JMenuItem copy = new JMenuItem("Copy");‘‘
‘‘‘‘‘‘‘‘ JMenuItem paste = new JMenuItem("Paste");‘‘
‘‘‘‘‘‘‘‘ popupmenu.add(cut); popupmenu.add(copy); popupmenu.add(paste);‘‘‘‘‘‘‘‘
‘‘‘‘‘‘‘‘ f.addMouseListener(new MouseAdapter() {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘public void mouseClicked(MouseEvent e) {‘‘‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘popupmenu.show(f , e.getX(), e.getY());‘‘
‘‘‘‘‘‘‘‘‘‘‘‘} });‘‘
‘‘‘‘‘‘‘‘ f.add(popupmenu);‘‘
‘‘‘‘‘‘‘‘ f.setSize(300,300);‘‘
‘‘‘‘‘‘‘‘ f.setLayout(null);‘‘
‘‘‘‘‘‘‘‘ f.setVisible(true);‘‘
‘‘‘‘ }‘‘
public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘new PopupMenuExample();‘‘
}}‘‘
```



Java Swing JCheckBoxMenuItem: JCheckBoxMenuItem class represents checkbox which can be included on a menu. A JCheckBoxMenuItem can have text or a graphic icon or both, associated with it. MenuItem can be selected or deselected. MenuItems can be configured and controlled by actions.

Modifier and Type	*Class*	*Description*
protected class	JCheckBoxMenuItem.AccessibleJCheckBoxMenuItem	This class implements acce support for the JcheckBoxM class.

Class JCheckBoxMenuItem: Constructors:

Constructor	Description
`JCheckBoxMenuItem()`	It creates an initially unselected check box me icon.
`JCheckBoxMenuItem(Action a)`	It creates a menu item whose properties are ta supplied.
`JCheckBoxMenuItem(Icon icon)`	It creates an initially unselected check box me
`JCheckBoxMenuItem(String text)`	It creates an initially unselected check box me
`JCheckBoxMenuItem(String text, boolean b)`	It creates a check box menu item with the spe
`JCheckBoxMenuItem(String text, Icon icon)`	It creates an initially unselected check box me text and icon.
`JCheckBoxMenuItem(String text, Icon icon, boolean b)`	It creates a check box menu item with the spe state.

Class JCheckBoxMenuItem : Methods:

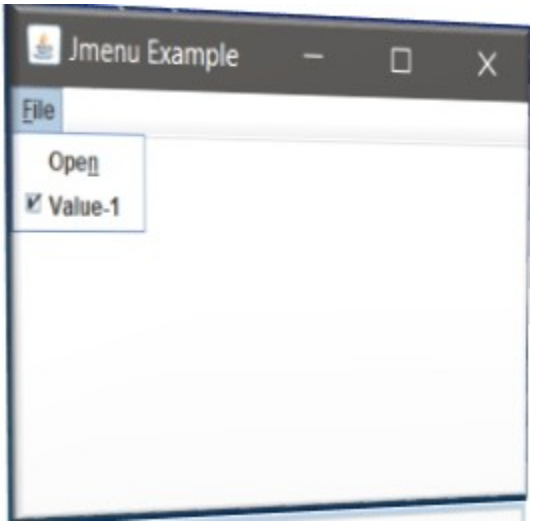
Modifier	Method	Description
AccessibleContext	`getAccessibleContext()`	It gets the AccessibleContext associated with this J
Object[]	`getSelectedObjects()`	It returns an array (length 1) containing the check b the check box is not selected.
boolean	`getState()`	It returns the selected-state of the item.
String	`getUIClassID()`	It returns the name of the L&F class that renders thi
protected String	`paramString()`	It returns a string representation of this JCheckBoxI
void	`setState(boolean b)`	It sets the selected-state of the item.

Illustration 10.22

```
// A program to show check box menu item in menu


import java.awt.event.ActionEvent;’‘
import java.awt.event.ActionListener;’‘
import java.awt.event.KeyEvent;’‘
import javax.swing.AbstractButton;’‘
import javax.swing.Icon;’‘
```

```
import javax.swing.JCheckBoxMenuItem;‘‘
import javax.swing.JFrame;‘‘
import javax.swing.JMenu;‘‘
import javax.swing.JMenuBar;‘‘
import javax.swing.JMenuItem;‘‘
public class JavaCheckBoxMenuItemExample {‘‘
‘‘‘‘public static void main(final String args[]) {‘‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("Jmenu Example");‘‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘‘‘‘‘JMenuBar menuBar = new JMenuBar();
‘‘‘‘‘‘‘‘JMenu fileMenu = new JMenu("File");‘‘
‘‘‘‘‘‘‘‘fileMenu.setMnemonic(KeyEvent.VK_F);‘‘
‘‘‘‘‘‘‘‘menuBar.add(fileMenu);‘‘
‘‘‘‘‘‘‘‘JMenuItem menuItem1 = new JMenuItem("Open", KeyEvent.VK_N);‘‘
‘‘‘‘‘‘‘‘fileMenu.add(menuItem1);‘‘
‘‘‘‘‘‘‘‘JCheckBoxMenuItem caseMenuItem = new JCheckBoxMenuItem("Option_1");‘‘
‘‘‘‘‘‘‘‘caseMenuItem.setMnemonic(KeyEvent.VK_C);
‘‘‘‘‘‘‘‘fileMenu.add(caseMenuItem);‘‘
‘‘‘‘‘‘‘‘ActionListener aListener = new ActionListener() {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘public void actionPerformed(ActionEvent event) {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘AbstractButton aButton = (AbstractButton) event.getSource();‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘boolean selected = aButton.getModel().isSelected();‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘String newLabel;‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘Icon newIcon;‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘if (selected) {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘newLabel = "Value-1";‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘} else {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘newLabel = "Value-2";‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘aButton.setText(newLabel);‘‘
‘‘‘‘‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘};
‘‘‘‘‘‘‘‘caseMenuItem.addActionListener(aListener);‘‘
‘‘‘‘‘‘‘‘frame.setJMenuBar(menuBar);‘‘
‘‘‘‘‘‘‘‘frame.setSize(350, 250);‘‘
‘‘‘‘‘‘‘‘frame.setVisible(true);‘‘
‘‘‘‘}‘‘
}‘‘
```



Java Swing JSeparator: The object of JSeparator class is used to provide a general purpose component for implementing divider lines. It is used to draw a line to separate widgets in a Layout. It inherits JComponent class.

Below is the declaration for javax.swing.JSeparator
class **public class JSeparator extends JComponent implements SwingConstants, Accessible**

Class JSeparator : Constructors:

Constructor	Description
JSeparator()	Creates a new horizontal separator.
JSeparator(int orientation)	Creates a new separator with the spe horizontal or vertical orientation.

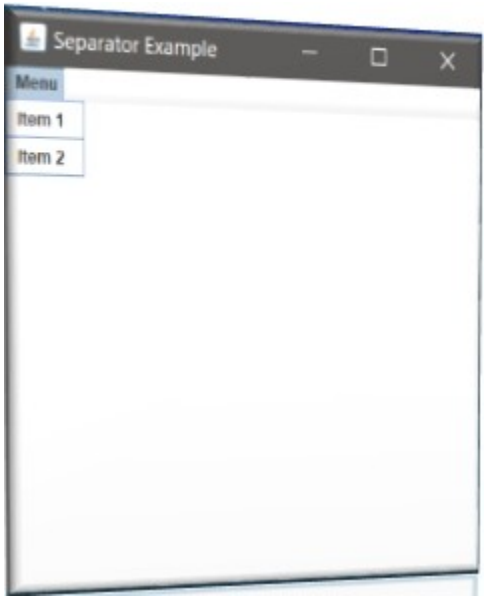
Class JSeparator : Methods:

Methods	Description
void setOrientation(int orientation)	It is used to set the orientation of the separator.
int getOrientation()	It is used to return the orientation of the separato

Illustration 10.23

```
import javax.swing.*;''''
class SeparatorExample{''''
''JMenu menu, submenu;''''
```

```
‘‘JMenuItem i1, i2, i3, i4, i5;’‘‘‘
‘‘SeparatorExample()’‘{‘‘‘‘
‘‘‘‘‘‘‘‘JFrame f = new JFrame("Separator Example");’‘‘‘
‘‘‘‘‘‘‘‘JMenuBar mb = new JMenuBar();’‘‘‘
‘‘‘‘‘‘‘‘menu = new JMenu("Menu");’‘‘‘
‘‘‘‘‘‘‘‘i1 = new JMenuItem("Item 1");’‘‘‘
‘‘‘‘‘‘‘‘i2 = new JMenuItem("Item 2");’‘‘‘‘‘
‘‘‘‘‘‘‘‘menu.add(i1);’‘
‘‘‘‘‘‘‘‘menu.addSeparator();’‘
‘‘‘‘‘‘‘‘menu.add(i2);’‘
‘‘‘‘‘‘‘‘mb.add(menu);’‘‘‘
‘‘‘‘‘‘‘‘f.setJMenuBar(mb);’‘‘‘
‘‘‘‘‘‘‘‘f.setSize(400,400);’‘‘‘
‘‘‘‘‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘‘‘‘‘f.setVisible(true);’‘‘‘
‘‘ }’‘‘‘
‘‘‘‘public static void main(String args[]){‘‘‘‘
‘‘‘‘‘‘‘‘new SeparatorExample();’‘‘‘
}}
```



Java Swing JProgressBar: The JProgressBar class is used to display the progress of the task. It inherits JComponent class.



Below is the declaration for javax.swing.JProgressBar class. **public class JProgressBar extends JComponent implements SwingConstants, Accessible**

Class JProgressBar : Constructors:

Constructor	Description
`JProgressBar()`	It is used to create a horizontal progress b
`JProgressBar(int min, int max)`	It is used to create a horizontal progress b minimum and maximum value.
`JProgressBar(int orient)`	It is used to create a progress bar with the can be either Vertical or Horizontal by usi SwingConstants.VERTICAL and SwingConstants.HORIZONTAL constants
`JProgressBar(int orient, int min, int max)`	It is used to create a progress bar with the minimum and maximum value.

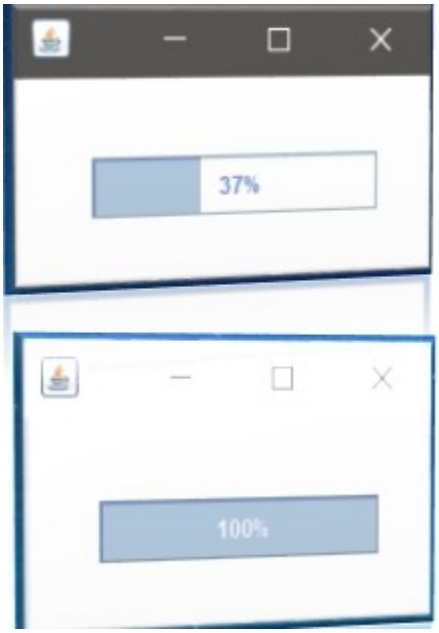
Class JProgressBar : Methods:

Method	Description
`void setStringPainted(boolean b)`	It is used to determine whether strin displayed.
`void setString(String s)`	It is used to set value to the progres
`void setOrientation(int orientation)`	It is used to set the orientation, it ma horizontal by using SwingConstants SwingConstants.HORIZONTAL co
`void setValue(int value)`	It is used to set the current value on

Illustration 10.24

```
// A program to show progress bar
```

```
import javax.swing.*;‘‘‘‘
public class ProgressBarExample extends JFrame{
‘‘JProgressBar jb; int i=0,num=0;‘‘‘‘
‘‘ProgressBarExample(){‘‘‘‘
‘‘‘‘jb=new JProgressBar(0,2000);‘‘‘‘
‘‘‘‘jb.setBounds(40,40,160,30);‘‘‘‘‘‘‘
‘‘‘‘jb.setValue(0);‘‘‘‘
‘‘‘‘jb.setStringPainted(true);‘‘‘‘
‘‘‘‘add(jb);‘‘‘‘
‘‘‘‘setSize(250,150);‘‘ setLayout(null);‘‘‘‘
‘‘}‘‘‘‘
‘‘public void iterate(){‘‘‘‘
‘‘‘‘while(i<=2000){‘‘‘‘
‘‘‘‘‘‘jb.setValue(i);‘‘‘‘
‘‘‘‘‘‘i=i+20;‘‘‘‘
‘‘‘‘‘‘try{Thread.sleep(150);}catch(Exception e){}‘‘‘‘}‘‘‘‘
‘‘}‘‘‘‘
‘‘public static void main(String[] args) {‘‘‘‘
‘‘‘‘‘‘ProgressBarExample m=new ProgressBarExample();‘‘‘‘
‘‘‘‘‘‘m.setVisible(true);‘‘‘‘
‘‘‘‘‘‘m.iterate();‘‘‘‘
‘‘}‘‘‘‘
}‘‘
```



Java Swing JTree: The JTree class is used to display the tree structured data or hierarchical data. JTree is a complex component. It has a 'root node' at the top most which is a parent for all nodes in the tree. It inherits JComponent class.



Below is the declaration for javax.swing.JTree class. **public class JTree extends JComponent implements Scrollable, Accessible**

Class JTree : Constructors:

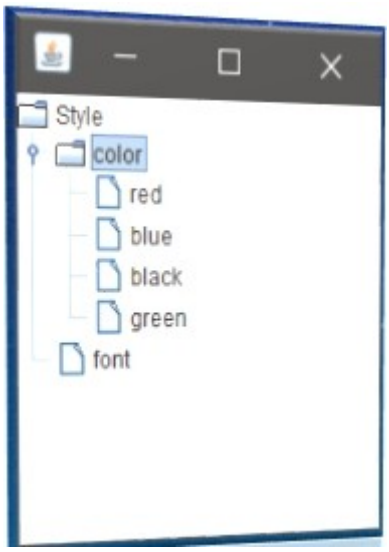
Constructor	Description
JTree()	Creates a JTree with a sample model.
JTree(Object[] value)	Creates a JTree with every element of the specified array as the child of a new root n
JTree(TreeNode root)	Creates a JTree with the specified TreeNod root, which displays the root node.

Illustration 10.25

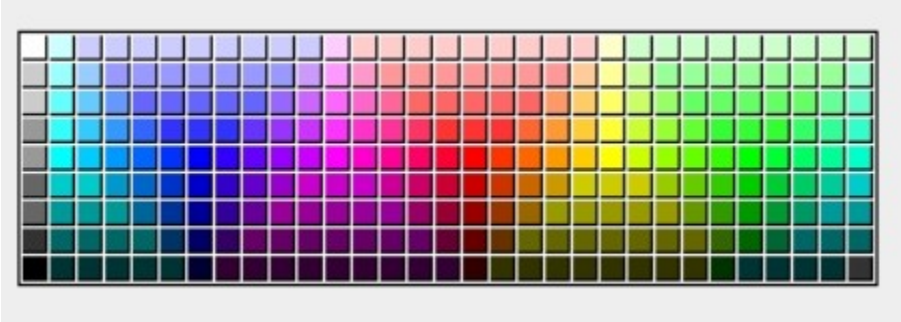
```
// A program to show Tree structured data arrangement


import javax.swing.*;‘‘
import javax.swing.tree.DefaultMutableTreeNode;’‘
public class TreeExample {‘‘
JFrame f;’‘
TreeExample(){‘‘
‘‘‘‘f=new JFrame();’‘
‘‘‘‘DefaultMutableTreeNode style=new DefaultMutableTreeNode("Style");’‘
‘‘‘‘DefaultMutableTreeNode color=new DefaultMutableTreeNode("color");’‘
‘‘‘‘DefaultMutableTreeNode font=new DefaultMutableTreeNode("font");’‘
‘‘‘‘style.add(color);’‘‘‘ style.add(font);’‘
‘‘‘‘DefaultMutableTreeNode red=new DefaultMutableTreeNode("red");’‘
‘‘‘‘DefaultMutableTreeNode blue=new DefaultMutableTreeNode("blue");’‘
‘‘‘‘DefaultMutableTreeNode black=new DefaultMutableTreeNode("black");’‘
‘‘‘‘DefaultMutableTreeNode green=new DefaultMutableTreeNode("green");’‘
‘‘‘‘color.add(red); color.add(blue); color.add(black); color.add(green);’‘‘‘‘‘
```

```
‘‘‘‘JTree jt=new JTree(style);’‘‘‘ f.add(jt);’‘
‘‘‘‘f.setSize(200,200);’‘
‘‘‘‘f.setVisible(true);’‘
}’‘
public static void main(String[] args) {‘‘
‘‘‘‘new TreeExample();’‘
}}’‘
```



Java Swing JColorChooser: The JColorChooser class is used to create a color chooser dialog box so that user can select any color. It inherits JComponent class.



Below is the declaration for javax.swing.JColorChooser class. **public**`&#65533;`**class JColorChooser**`&#65533;`**extends**`&#65533;`**JToggleButton**`&#65533;`**implements**`&#65533;`**Accessible**`&#65533;&#65533;&#65533;`

Class JColorChooser: Constructors:

Constructor	Description
JColorChooser()	It is used to create a color chooser panel with white color initially.
JColorChooser(color initialcolor)	It is used to create a color chooser panel with the specified color initially.

Class JColorChooser : Methods:

Method	
void addChooserPanel(AbstractColorChooserPanel panel)	It is use panel t
static Color showDialog(Component c, String title, Color initialColor)	It is use dialog

Illustration 10.26

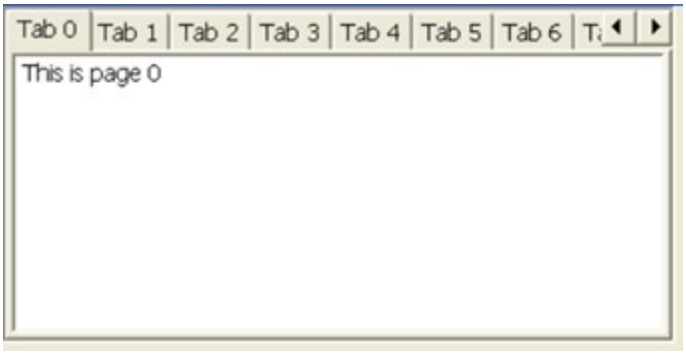
```
// A program to show choose colour using JColorChooser


import java.awt.event.*;‘‘‘‘
import java.awt.*;‘‘‘‘
import javax.swing.*;‘‘‘‘
public class ColorChooserExample extends JFrame implements ActionListener {‘‘‘‘
JButton b;Container c;‘‘‘‘
ColorChooserExample(){‘‘‘‘
‘‘‘‘c=getContentPane();‘‘‘‘
‘‘‘‘c.setLayout(new FlowLayout());‘‘‘‘‘‘‘‘
‘‘‘‘b=new JButton("color");‘‘‘‘
‘‘‘‘b.addActionListener(this);‘‘‘‘‘‘‘‘
‘‘‘‘c.add(b);‘‘ }‘‘‘‘
public void actionPerformed(ActionEvent e) {‘‘‘‘
Color initialcolor=Color.RED;‘‘‘‘
Color color=JColorChooser.showDialog(this,"Select a color",initialcolor);‘‘‘‘
c.setBackground(color);‘‘‘‘
}‘‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘ColorChooserExample ch=new ColorChooserExample();‘‘‘‘
‘‘‘‘ch.setSize(400,400);‘‘‘‘
‘‘‘‘ch.setVisible(true);‘‘‘‘
```

```
‘‘‘‘ch.setDefaultCloseOperation(EXIT_ON_CLOSE);’‘‘‘
}’‘‘‘
}
```



Java Swing JTabbedPane: The JTabbedPane class is used to switch between a group of components by clicking on a tab with a given title or icon. It inherits JComponent class.



Below is the declaration for javax.swing.JTabbedPane class. **public�class�JTabbedPane�extends�JComponent�implements�Serializable,�Accessible,�SwingConstants.��**

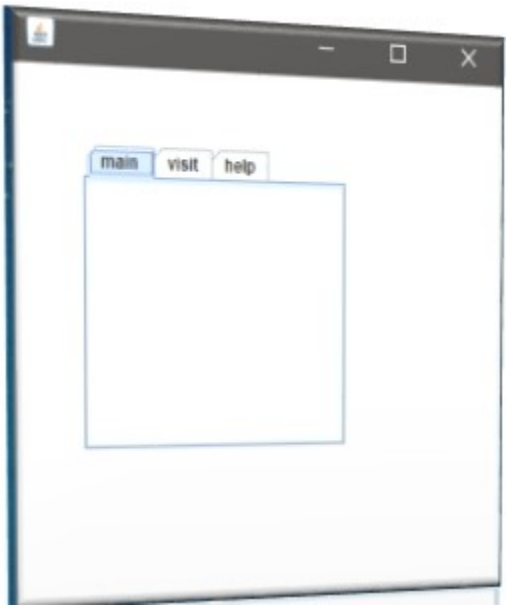
Class JTabbedPane : Constructors:

Constructor	Description
JTabbedPane()	Creates an empty TabbedPane with a defa placement of JTabbedPane.Top.
JTabbedPane(int tabPlacement)	Creates an empty TabbedPane with a spec tab placement.
JTabbedPane(int tabPlacement, int tabLayoutPolicy)	Creates an empty TabbedPane with a spec tab placement and tab layout policy.

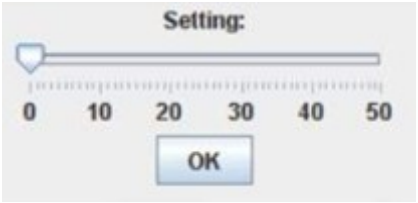
Illustration 10.27

```
// A program to show multiple tabs using JTabbedPane


import javax.swing.*;‘‘
public class TabbedPaneExample {‘‘
JFrame f;‘‘
TabbedPaneExample(){‘‘
‘‘‘‘f=new JFrame();‘‘
‘‘‘‘JTextArea ta=new JTextArea(200,200);‘‘
‘‘‘‘JPanel p1=new JPanel();‘‘
‘‘‘‘p1.add(ta);‘‘
‘‘‘‘JPanel p2=new JPanel();‘‘
‘‘‘‘JPanel p3=new JPanel();‘‘
‘‘‘‘JTabbedPane tp=new JTabbedPane();‘‘
‘‘‘‘tp.setBounds(50,50,200,200);‘‘
‘‘‘‘tp.add("main",p1);‘‘
‘‘‘‘tp.add("visit",p2);‘‘
‘‘‘‘tp.add("help",p3);‘‘‘‘
‘‘‘‘f.add(tp);‘‘
‘‘‘‘f.setSize(400,400);‘‘
‘‘‘‘f.setLayout(null);‘‘
‘‘‘‘f.setVisible(true); }‘‘
public static void main(String[] args) {‘‘
‘‘‘‘new TabbedPaneExample();‘‘
}}‘‘
```



Java Swing JSlider: The Java JSlider class is used to create the slider. By using JSlider, a user can select a value from a specific range.



Class JSlider : Constructors:

Constructor	*Description*
`JSlider()`	creates a slider with the initial value of 50 and range of
`JSlider(int orientation)`	creates a slider with the specified orientation set by eith JSlider.VERTICAL with the range 0 to 100 and initial
`JSlider(int min, int max)`	creates a horizontal slider using the given min and max
`JSlider(int min, int max, int value)`	creates a horizontal slider using the given min, max and
`JSlider(int orientation, int min, int max, int value)`	creates a slider using the given orientation, min, max a

Class JSlider : Methods:

Method	Description
public void setMinorTickSpacing(int n)	is used to set the minor tick spacing to the
public void setMajorTickSpacing(int n)	is used to set the major tick spacing to the
public void setPaintTicks(boolean b)	is used to determine whether tick marks a
public void setPaintLabels(boolean b)	is used to determine whether labels are pa
public void setPaintTracks(boolean b)	is used to determine whether track is paint

Illustration 10.28

```
// A program to implement slider input using JSlider


import javax.swing.*;‘‘
public class SliderExample1 extends JFrame{‘‘
public SliderExample1() {‘‘
          JSlider slider = new JSlider(JSlider.HORIZONTAL, 0, 50, 25);‘‘
          JPanel panel=new JPanel();‘‘
          panel.add(slider);‘‘
          add(panel);‘‘
          }‘‘
public static void main(String s[]) {‘‘
          SliderExample1 frame=new SliderExample1();‘‘
          frame.pack();‘‘
          frame.setVisible(true);‘‘
}‘‘
}‘‘
```



Java Swing JSpinner: The object of JSpinner class is a single line input field that allows the user to select a number or an object value from an ordered sequence.



Below is the declaration for javax.swing.JSpinner class. **public class JSpinner extends JComponent implements Accessible.**

Class JSpinner : Constructors:

Constructor	Description
`JSpinner()`	It is used to construct a spinner with an I SpinnerNumberModel with initial value minimum or maximum limits.
`JSpinner(SpinnerModel model)`	It is used to construct a spinner for a give

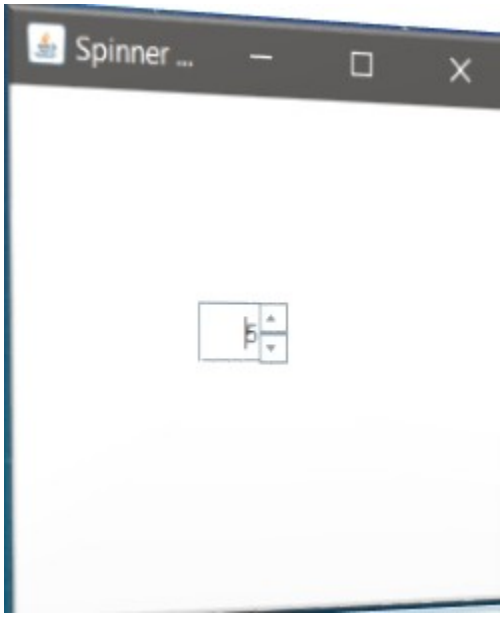
Class JSpinner : Methods:

Method	Desc
`void addChangeListener(ChangeListener listener)`	It is used to add a liste notified each time a cl occurs.
`Object getValue()`	It is used to return the model.

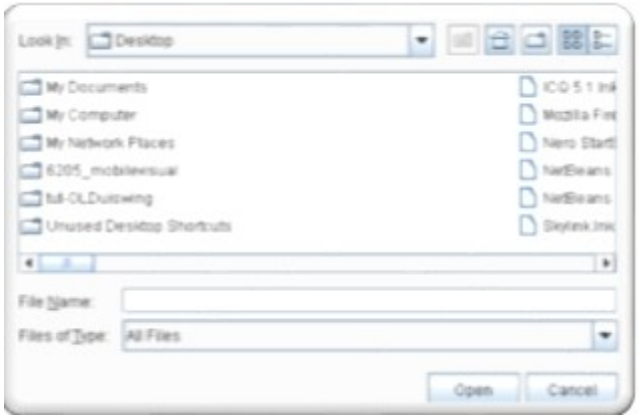
Illustration 10.29

```
import javax.swing.*;‘‘‘‘
‘‘public class SpinnerExample {‘‘
‘‘‘‘public static void main(String[] args) {‘‘‘‘
‘‘‘‘JFrame f=new JFrame("Spinner Example");‘‘‘‘
‘‘‘‘SpinnerModel value =‘‘
‘‘‘‘‘‘‘‘‘‘‘‘ new SpinnerNumberModel(5, //initial value’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘0, //minimum value’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘10, //maximum value’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘1); //step’‘
‘‘‘‘JSpinner spinner = new JSpinner(value);’‘
‘‘‘‘‘‘‘‘‘‘‘‘spinner.setBounds(100,100,50,30);’‘‘‘
```

```
‘‘‘‘‘‘‘‘‘‘‘‘f.add(spinner);’’’’
‘‘‘‘‘‘‘‘‘‘‘‘f.setSize(300,300);’’’’
‘‘‘‘‘‘‘‘‘‘‘‘f.setLayout(null);’’’’
‘‘‘‘‘‘‘‘‘‘‘‘f.setVisible(true);’’’’
‘‘}’’
}
```



Java Swing JFileChooser: The object of JFileChooser class represents a dialog window from which the user can select file. It inherits JComponent class.



Below is the declaration for javax.swing.JFileChooser class. **public class JFileChooser extends JComponent implements Accessible.**

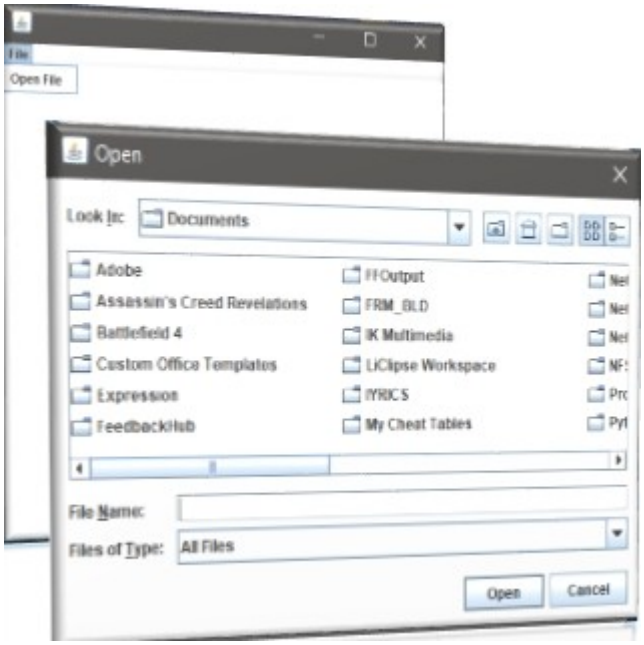
Class JFileChooser : Constructors:

Constructor	Description
JFileChooser()	Constructs a JFileChooser pointing to the user's directory.
JFileChooser(File currentDirectory)	Constructs a JFileChooser using the given File a
JFileChooser(String currentDirectoryPath)	Constructs a JFileChooser using the given path.

Illustration 10.30

```
import javax.swing.*;‘‘‘‘
import java.awt.event.*;‘‘‘‘
import java.io.*;‘‘‘‘
public class FileChooserExample extends JFrame implements ActionListener{‘‘‘‘
          JMenuBar mb;‘‘‘‘
          JMenu file;‘‘‘‘
          JMenuItem open;‘‘‘‘
          JTextArea ta;‘‘‘‘
          FileChooserExample(){‘‘‘‘
          open=new JMenuItem("Open File");‘‘‘‘
          open.addActionListener(this);‘‘‘‘‘‘‘‘‘‘‘‘
          file=new JMenu("File");‘‘‘‘
          file.add(open);‘‘‘‘‘‘‘‘‘‘‘‘
          mb=new JMenuBar();‘‘‘‘
          mb.setBounds(0,0,800,20);‘‘‘‘
          mb.add(file);‘‘‘‘‘‘‘‘‘‘‘‘‘‘
          ta=new JTextArea(800,800);‘‘‘‘
          ta.setBounds(0,20,800,800);‘‘‘‘‘‘‘‘‘‘‘‘‘‘
          add(mb);‘‘‘‘
          add(ta);}‘‘‘‘
          public void actionPerformed(ActionEvent e) {‘‘‘‘
          if(e.getSource()==open){
           JFileChooser fc=new JFileChooser();‘‘‘‘
                    int i=fc.showOpenDialog(this);‘‘‘‘
                    if(i==JFileChooser.APPROVE_OPTION){‘‘‘‘
                              File f=fc.getSelectedFile();‘‘‘‘
                              String filepath=f.getPath();‘‘‘‘
                              try{‘‘
                              BufferedReader br=new BufferedReader(new FileReader(filepath));‘‘‘‘
```

```
                                String s1="",s2="";’’’’’’’’’’’’’’’’’’’’’’’’
                                while((s1=br.readLine())!=null){’’’’
                                s2+=s1+"\n";’’’’
                                }’’’’
                                ta.setText(s2);’’’’
                                br.close();’’’’
                                }catch (Exception ex) {ex.printStackTrace();’’}’’’’’’’’’’’’’’’’
’’’’}}}’’’’’’’
public static void main(String[] args) {’’’’
’’’’FileChooserExample om=new FileChooserExample();’’’’
’’’’’’’’’’’’ om.setSize(500,500);’’’’
’’’’’’’’’’’’ om.setLayout(null);’’’’
’’’’’’’’’’’’ om.setVisible(true);’’’’
’’’’’’’’’’’’ om.setDefaultCloseOperation(EXIT_ON_CLOSE);}’’’’
}
’’’’
```



Java Swing JToggelButton: JToggleButton is used to create toggle button, it is two-states button to switch on or off.

Modifier and Type	Class	
protected class	JToggleButton.AccessibleJToggleButton	This class support for
static class	JToggleButton.ToggleButtonModel	The Toggle

Class JToggleButton : Constructors:

Constructor	Descri
JToggleButton()	It creates an initially unselected toggle butt
JToggleButton(Action a)	It creates a toggle button where properties a
JToggleButton(Icon icon)	It creates an initially unselected toggle butt
JToggleButton(Icon icon, boolean selected)	It creates a toggle button with the specified
JToggleButton(String text)	It creates an unselected toggle button with t
JToggleButton(String text, boolean selected)	It creates a toggle button with the specified
JToggleButton(String text, Icon icon)	It creates a toggle button that has the specif unselected.
JToggleButton(String text, Icon icon, boolean selected)	It creates a toggle button with the specified

Class JToggleButton : Methods:

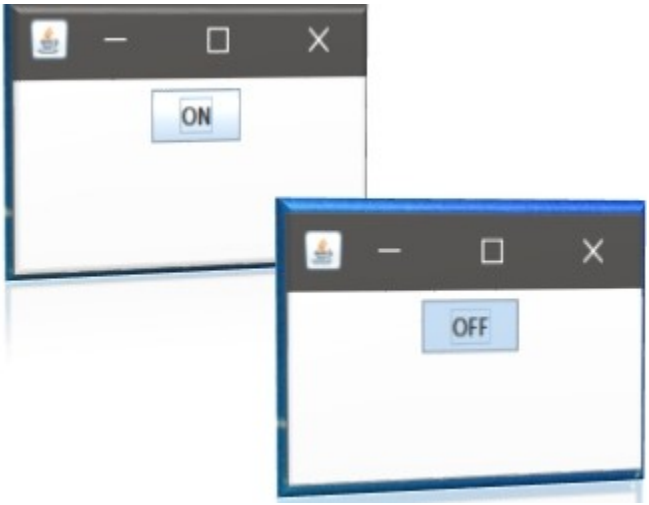
Modifier and Type	Method	Descriptio
AccessibleContext	`getAccessibleContext()`	It gets the AccessibleContext associated wi
String	`getUIClassID()`	It returns a string that specifies the name of this component.
protected String	`paramString()`	It returns a string representation of this JTo
void	`updateUI()`	It resets the UI property to a value from the

Illustration 10.31

```
// A program to add a Toggle button


import java.awt.FlowLayout;‘‘
import java.awt.event.ItemEvent;‘‘
import java.awt.event.ItemListener;‘‘
import javax.swing.JFrame;‘‘
import javax.swing.JToggleButton;
public class ToggleButtonExample extends JFrame implements ItemListener {‘‘
‘‘‘‘public static void main(String[] args) {‘‘
‘‘‘‘‘‘‘‘new ToggleButtonExample();‘‘
‘‘‘‘}‘‘
‘‘‘‘private JToggleButton button;‘‘
ToggleButtonExample() {‘‘
‘‘‘‘‘‘‘‘setTitle("JToggleButton with ItemListener Example");‘‘
‘‘‘‘‘‘‘‘setLayout(new FlowLayout());‘‘
‘‘‘‘‘‘‘‘setJToggleButton();‘‘
‘‘‘‘‘‘‘‘setAction();‘‘
‘‘‘‘‘‘‘‘setSize(200, 200);‘‘
‘‘‘‘‘‘‘‘setVisible(true);‘‘
‘‘‘‘‘‘‘‘setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘}
‘‘‘‘private void setJToggleButton() {‘‘
‘‘‘‘‘‘‘‘button = new JToggleButton("ON");‘‘
‘‘‘‘‘‘‘‘add(button);‘‘
‘‘‘‘}‘‘
‘‘‘‘private void setAction() {‘‘
```

```
‘‘‘‘‘‘‘‘button.addItemListener(this);’’
‘‘‘‘}’’
‘‘‘‘public void itemStateChanged(ItemEvent eve) {‘‘
‘‘‘‘‘‘‘‘if (button.isSelected())’’
‘‘‘‘‘‘‘‘‘‘‘‘button.setText("OFF");’’
‘‘‘‘‘‘‘‘else’’
‘‘‘‘‘‘‘‘‘‘‘‘button.setText("ON");’’
‘‘‘‘}’’
}’‘
```



Java Swing JToolBar: JToolBar container allows us to group other components, usually buttons with icons in a row or column. JToolBar provides a component which is useful for displaying commonly used actions or controls.

Modifier and Type	*Class*	*D*
protected class	JToolBar.AccessibleJToolBar	This class impl support for the
static class	JToolBar.Separator	A toolbar-spec

Class JToolBar : Constructors:

Constructor	Descriptio
`JToolBar()`	It creates a new tool bar; orientatio HORIZONTAL.
`JToolBar(int orientation)`	It creates a new tool bar with the s
`JToolBar(String name)`	It creates a new tool bar with the s
`JToolBar(String name, int orientation)`	It creates a new tool bar with a spe orientation.

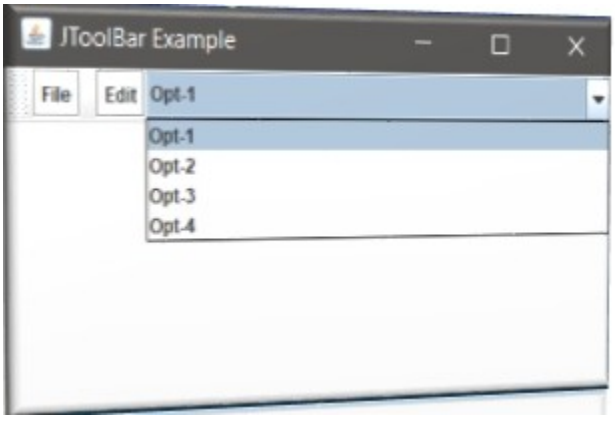
Class JToolBar : Methods:

Modifier and Type	Method	De
JButton	`add(Action a)`	It adds a new JButton which
protected void	`addImpl(Component comp, Object constraints, int index)`	If a JButton is being added,
void	`addSeparator()`	It appends a separator of de
protected PropertyChangeListener	`createActionChangeListener(JButton b)`	It returns a properly configu updates the control as chang default property change liste
protected JButton	`createActionComponent(Action a)`	Factory method which creat the JToolBar.
ToolBarUI	`getUI()`	It returns the tool bar's curre
void	`setUI(ToolBarUI ui)`	It sets the L&F object that r
void	`setOrientation(int o)`	It sets the orientation of the

Illustration 10.32

```
import java.awt.BorderLayout;’‘
import java.awt.Container;’‘
import javax.swing.JButton;’‘
import javax.swing.JComboBox;’‘
import javax.swing.JFrame;’‘
import javax.swing.JScrollPane;’‘
import javax.swing.JTextArea;’‘
import javax.swing.JToolBar;’‘
```

```
‘‘
public class ToolBarExample {‘‘
‘‘‘‘public static void main(final String args[]) {‘‘
‘‘‘‘‘‘‘‘JFrame myframe = new JFrame("JToolBar Example");’‘
‘‘‘‘‘‘‘‘myframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);’‘
‘‘‘‘‘‘‘‘JToolBar toolbar = new JToolBar();’‘
‘‘‘‘‘‘‘‘toolbar.setRollover(true);’‘
‘‘‘‘‘‘‘‘JButton button = new JButton("File");’‘
‘‘‘‘‘‘‘‘toolbar.add(button);’‘
‘‘‘‘‘‘‘‘toolbar.addSeparator();’‘
‘‘‘‘‘‘‘‘toolbar.add(new JButton("Edit"));’‘
‘‘‘‘‘‘‘‘toolbar.add(new JComboBox(new String[] { "Opt-1", "Opt-2", "Opt-3", "Opt-4" }));’‘
‘‘‘‘‘‘‘‘Container contentPane = myframe.getContentPane();’‘
‘‘‘‘‘‘‘‘contentPane.add(toolbar, BorderLayout.NORTH);’‘
‘‘‘‘‘‘‘‘JTextArea textArea = new JTextArea();’‘
‘‘‘‘‘‘‘‘JScrollPane mypane = new JScrollPane(textArea);’‘
‘‘‘‘‘‘‘‘contentPane.add(mypane, BorderLayout.EAST);’‘
‘‘‘‘‘‘‘‘myframe.setSize(450, 250);’‘
‘‘‘‘‘‘‘‘myframe.setVisible(true);’‘
‘‘‘‘}’‘
}’‘
```



Java Swing JViewport: The JViewport class is used to implement scrolling. JViewport is designed to support both logical scrolling and pixel-based scrolling. The viewport's child, called the view, is scrolled by calling the JViewport.setViewPosition() method.

Modifier and Type	Class	Description
protected class	JViewport.AccessibleJViewport	This class implements access support for the Jviewport cla
protected class	JViewport.ViewListener	A listener for the view.

Class JViewport : Fields:

Modifier and Type	Field	Descrip
static int	BACKINGSTORE_SCROLL_MODE	It draws viewport contents into an offscre
protected Image	backingStoreImage	The view image used for a backing store.
static int	BLIT_SCROLL_MODE	It uses graphics.copyArea to implement sc
protected boolean	isViewSizeSet	True when the viewport dimensions have b
protected Point	lastPaintPosition	The last viewPosition that we've painted, s backing store image is valid.
protected boolean	scrollUnderway	The scrollUnderway flag is used for comp
static int	SIMPLE_SCROLL_MODE	This mode uses the very simple method of scrollpane each time it is scrolled.

Class JViewport : Constructors:

Constructor	Description
`JViewport()`	Creates a JViewport.

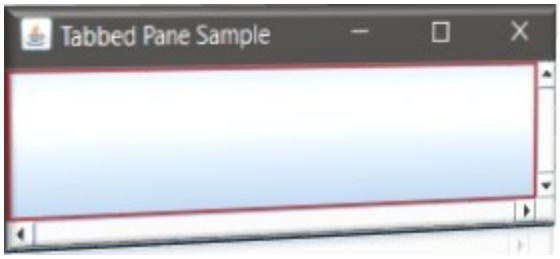
Class JViewport :Methods:

Modifier and Type	Method	Description
void	addChangeListener(ChangeListener l)	It adds a ChangeListener to the list that is notified position, or the viewport's extent size has changed
protected LayoutManager	createLayoutManager()	Subclassers can override this to install a different l constructor.
protected Jviewport.ViewListener	createViewListener()	It creates a listener for the view.
int	getScrollMode()	It returns the current scrolling mode.
Component	getView()	It returns the JViewport's one child or null.
Point	getViewPosition()	It returns the view coordinates that appear in the u viewport, or 0,0 if there's no view.
Dimension	getViewSize()	If the view's size hasn't been explicitly set, return t return the view's current size.
void	setExtentSize(Dimension newExtent)	It sets the size of the visible part of the view using
void	setScrollMode(int mode)	It used to control the method of scrolling the viewp
void	setViewSize(Dimension newSize)	It sets the size of the view.

Illustration 10.33

```
import java.awt.BorderLayout;’‘
import java.awt.Color;’‘
import java.awt.Dimension;’‘
import javax.swing.JButton;’‘
import javax.swing.JFrame;’‘
import javax.swing.JLabel;’‘
import javax.swing.JScrollPane;’‘
import javax.swing.border.LineBorder;’‘
public class ViewPortClass2 {’‘
‘‘‘‘public static void main(String[] args) {’‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("Tabbed Pane Sample");’‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);’‘
‘‘
‘‘‘‘‘‘‘‘JLabel label = new JLabel("Label");’‘
‘‘‘‘‘‘‘‘label.setPreferredSize(new Dimension(1000, 1000));’‘
‘‘‘‘‘‘‘‘JScrollPane jScrollPane = new JScrollPane(label);’‘
‘‘
‘‘‘‘‘‘‘‘JButton jButton1 = new JButton();’‘
‘‘‘‘‘‘‘‘jScrollPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);’‘
‘‘‘‘‘‘‘‘jScrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);’‘
```

```
‘‘‘‘‘‘‘‘jScrollPane.setViewportBorder(new LineBorder(Color.RED));’‘
‘‘‘‘‘‘‘‘jScrollPane.getViewport().add(jButton1, null);’‘
‘‘
‘‘‘‘‘‘‘‘frame.add(jScrollPane, BorderLayout.CENTER);’‘
‘‘‘‘‘‘‘‘frame.setSize(400, 150);’‘
‘‘‘‘‘‘‘‘frame.setVisible(true);’‘
‘‘‘‘}’‘
}
```



Java Swing JPanel: The JPanel is a simplest container class. It provides space in which an application can attach any other component. It inherits the JComponents class. It doesn't have title bar.



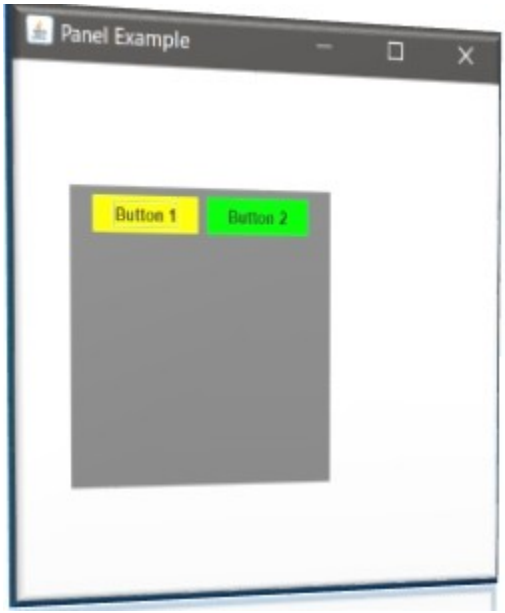
Below is the declaration for javax.swing.JPanel class. **public class JPanel extends JComponent implements Accessible**

Class JPanel : Constructors:

Constructor	Description
JPanel()	It is used to create a new JPanel with a d buffer and a flow layout.
JPanel(boolean isDoubleBuffered)	It is used to create a new JPanel with Flo and the specified buffering strategy.
JPanel(LayoutManager layout)	It is used to create a new JPanel with the layout manager.

Illustration 10.34

```
import java.awt.*;‘‘
import javax.swing.*;‘‘
public class PanelExample {‘‘
‘‘‘‘ PanelExample()‘‘{‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame("Panel Example");‘‘‘‘
‘‘‘‘‘‘‘‘JPanel panel=new JPanel();‘‘
‘‘‘‘‘‘‘‘panel.setBounds(40,80,200,200);‘‘‘‘
‘‘‘‘‘‘‘‘panel.setBackground(Color.gray);‘‘
‘‘‘‘‘‘‘‘JButton b1=new JButton("Button 1");‘‘‘‘
‘‘‘‘‘‘‘‘b1.setBounds(50,100,80,30);‘‘‘‘
‘‘‘‘‘‘‘‘b1.setBackground(Color.yellow);‘‘
‘‘‘‘‘‘‘‘JButton b2=new JButton("Button 2");‘‘
‘‘‘‘‘‘‘‘b2.setBounds(100,100,80,30);‘‘‘‘
‘‘‘‘‘‘‘‘b2.setBackground(Color.green);‘‘
‘‘‘‘‘‘‘‘panel.add(b1); panel.add(b2);‘‘
‘‘‘‘‘‘‘‘f.add(panel);‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setSize(400,400);‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setLayout(null);‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setVisible(true);‘‘‘‘
‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘new PanelExample();‘‘
‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘}‘‘
```



Java Swing JFrame: The javax.swing.JFrame class is a type of container which inherits the java.awt.Frame class. JFrame works like the main window where components like labels, buttons, textfields are added to create a GUI. Unlike Frame, JFrame has the option to hide or close the window with the help of setDefaultCloseOperation(int) method.

Modifier and Type	*Class*	*Desc*
protected class	JFrame.AccessibleJFrame	This class implements for the JFrame class.

Class JFrame : Fields:��

Modifier and Type	*Field*	*Description*
protected AccessibleContext	accessibleContext	The accessible context property.
static int	EXIT_ON_CLOSE	The exit application default window close operation.
protected JRootPane	rootPane	The JRootPane instance that manages the contentPane and optional menuBar for this frame, as well as the glassPane.
protected boolean	rootPaneCheckingEnabled	If true then calls to add and setLayout will be forwarded to the contentPane.

Class JFrame : Constructors:

Constructor	Description
`JFrame()`	It constructs a new frame that is initiall
`JFrame(GraphicsConfiguration gc)`	It creates a Frame in the specified GraphicsConfiguration of a screen dev title.
`JFrame(String title)`	It creates a new, initially invisible Fra specified title.
`JFrame(String title, GraphicsConfiguration gc)`	It creates a JFrame with the specified t specified GraphicsConfiguration of a s

Class JFrame : Methods:

Modifier and Type	Method	Descr
protected void	`addImpl(Component comp, Object constraints, int index)`	Adds the specified child Component.
protected JRootPane	`createRootPane()`	Called by the constructor methods to cr
protected void	`frameInit()`	Called by the constructors to init the JFr
void	`setContentPane(Containe contentPane)`	It sets the contentPane property
static void	`setDefaultLookAndFeelDecorated(boolean defaultLookAndFeelDecorated)`	Provides a hint as to whether or not new Window decorations (such as borders, v provided by the current look and feel.
void	`setIconImage(Image image)`	It sets the image to be displayed as the ic
void	`setJMenuBar(JMenuBar menubar)`	It sets the menubar for this frame.
void	`setLayeredPane(JLayeredPane layeredPane)`	It sets the layeredPane property.
JRootPane	`getRootPane()`	It returns the rootPane object for this fra
TransferHandler	`getTransferHandler()`	It gets the transferHandler property.

Illustration 10.35

```
import java.awt.FlowLayout;’‘
import javax.swing.JButton;’‘
import javax.swing.JFrame;’‘
```

```
import javax.swing.JLabel;‘‘
import javax.swing.Jpanel;‘‘
public class JFrameExample {‘‘
‘‘‘‘public static void main(String s[]) {‘‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("JFrame Example");‘‘
‘‘‘‘‘‘‘‘JPanel panel = new JPanel();‘‘
‘‘‘‘‘‘‘‘panel.setLayout(new FlowLayout());‘‘
‘‘‘‘‘‘‘‘JLabel label = new JLabel("JFrame By Example");‘‘
‘‘‘‘‘‘‘‘JButton button = new JButton();‘‘
‘‘‘‘‘‘‘‘button.setText("Button");‘‘
‘‘‘‘‘‘‘‘panel.add(label);‘‘
‘‘‘‘‘‘‘‘panel.add(button);‘‘
‘‘‘‘‘‘‘‘frame.add(panel);‘‘
‘‘‘‘‘‘‘‘frame.setSize(200, 300);‘‘
‘‘‘‘‘‘‘‘frame.setLocationRelativeTo(null);‘‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘‘‘‘‘frame.setVisible(true);‘‘
‘‘‘‘}‘‘
}‘‘
```



Java Swing Calculator:

```
Methods used :
```

```
add(Component c) : adds component to container.

addActionListenerListener(ActionListener d) : add actionListener for specified component

setBackground(Color c) : sets the background color of the specified container

setSize(int a, int b) : sets the size of container to specified dimensions.

setText(String s) : sets the text of the label to s.

getText() : returns the text of the label
```



Illustration 10.36

```
// Java program to create a simple calculator
// with basic +, -, /, * using java swing elements

import java.awt.event.*;
import javax.swing.*;
import java.awt.*;
class Calculator extends JFrame implements ActionListener {
          // create a frame
          static JFrame f;
          // create a textfield
          static JTextField l;
          // store oprerator and operands
          String s0, s1, s2;
          // default constrcutor
          Calculator()
```

```
        {
                s0 = s1 = s2 = "";
        }
        // main function
        public static void main(String args[])
        {
                // create a frame
                f = new JFrame("Swing Calculator");
                try {
                        // set look and feel
                        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
                }
                catch (Exception e) {
                        System.err.println(e.getMessage());
                }
                // create a object of class
                Calculator c = new Calculator();
                // create a textfield
                l = new JTextField(16);
                // set the textfield to non editable
                l.setEditable(false);
                // create number buttons and some operators
                JButton b0, b1, b2, b3, b4, b5, b6, b7, b8, b9, ba, bs, bd, bm, be, beq, beq1;
                // create number buttons
                b0 = new JButton("0");
                b1 = new JButton("1");
                b2 = new JButton("2");
                b3 = new JButton("3");
                b4 = new JButton("4");
                b5 = new JButton("5");
                b6 = new JButton("6");
                b7 = new JButton("7");
                b8 = new JButton("8");
                b9 = new JButton("9");
                // equals button
                beq1 = new JButton("=");
                // create operator buttons
                ba = new JButton("+");
                bs = new JButton("-");
                bd = new JButton("/");
                bm = new JButton("*");
```

```
                    beq = new JButton("C");
                    // create . button
                    be = new JButton(".");
                    // create a panel
                    JPanel p = new JPanel();
                    // add action listeners
                    bm.addActionListener(c);
                    bd.addActionListener(c);
                    bs.addActionListener(c);
                    ba.addActionListener(c);
                    b9.addActionListener(c);
                    b8.addActionListener(c);
                    b7.addActionListener(c);
                    b6.addActionListener(c);
                    b5.addActionListener(c);
                    b4.addActionListener(c);
                    b3.addActionListener(c);
                    b2.addActionListener(c);
                    b1.addActionListener(c);
                    b0.addActionListener(c);
                    be.addActionListener(c);
                    beq.addActionListener(c);
                    beq1.addActionListener(c);
                    // add elements to panel
                    p.add(l);
                    p.add(ba);
                    p.add(b1);
                    p.add(b2);
                    p.add(b3);
                    p.add(bs);
                    p.add(b4);
                    p.add(b5);
                    p.add(b6);
                    p.add(bm);
                    p.add(b7);
                    p.add(b8);
                    p.add(b9);
                    p.add(bd);
                    p.add(be);
                    p.add(b0);
                    p.add(beq);
```

```
                    p.add(beq1);
                    // set Background of panel
                    p.setBackground(Color.blue);
                    // add panel to frame
                    f.add(p);
                    f.setSize(200, 220);
                    f.show();
          }
          public void actionPerformed(ActionEvent e)
          {
                    String s = e.getActionCommand();
                    // if the value is a number
                    if ((s.charAt(0) >= '0' && s.charAt(0) <= '9') || s.charAt(0) == '.') {
                              // if operand is present then add to second no
                              if (!s1.equals(""))
                                        s2 = s2 + s;
                              else
                                        s0 = s0 + s;
                              // set the value of text
                              l.setText(s0 + s1 + s2);
                    }
                    else if (s.charAt(0) == 'C') {
                              // clear the one letter
                              s0 = s1 = s2 = "";
                              // set the value of text
                              l.setText(s0 + s1 + s2);
                    }
                    else if (s.charAt(0) == '=') {
                              double te;
                              // store the value in 1st
                              if (s1.equals("+"))
                                        te = (Double.parseDouble(s0) + Double.parseDouble(s2));
                              else if (s1.equals("-"))
                                        te = (Double.parseDouble(s0) - Double.parseDouble(s2));
                              else if (s1.equals("/"))
                                        te = (Double.parseDouble(s0) / Double.parseDouble(s2));
                                        else
                                                  te = (Double.parseDouble(s0) *
Double.parseDouble(s2));
                                        // convert it to string
                                        s0 = Double.toString(te);
```

```
                                        // place the operator
                                        s1 = s;
                                        // make the operand blank
                                        s2 = "";
                                }
                                // set the value of text
                                l.setText(s0 + s1 + s2);
‘‘‘‘‘‘‘‘}
‘‘‘‘}
}
```



Practice Questions

Practice 10.1

```
          //Simple Application

          import javax.swing.*;
          import java.awt.*;
          import java.awt.event.*;

          public class SimpleApplication implements ActionListener {
                    private static String labelPrefix = "Number of button clicks: ";
                    private int numClicks = 0;
                    private final JLabel label = new JLabel(labelPrefix + "0’‘‘‘");
```

```
            public Component createComponents() {
                        JButton button = new JButton("I'm a Swing button!");
                        button.addActionListener(this);
‘‘
                        /* An easy way to put space between a top-level container
                         * and its contents is to put the contents in a JPanel
                         * that has an "empty" border. */
                        JPanel pane = new JPanel(new GridLayout(0, 1));
                        pane.add(button);
                        pane.add(label);
                        pane.setBorder(BorderFactory.createEmptyBorder(30,30,10,30));’‘‘‘
                        //top,left,bottom,right
                        return pane;
            }

            public void actionPerformed(ActionEvent e) {
                        numClicks++;
                        label.setText(labelPrefix + numClicks);
            }

            private static void initLookAndFeel() {
‘‘ // METAL
                         String lookAndFeel = "javax.swing.plaf.metal.MetalLookAndFeel";
‘‘ // MOTIF
                        // String lookAndFeel = "com.sun.java.swing.plaf.motif.MotifLookAndFeel";
‘‘ // WINDOWS
                         // String lookAndFeel = "com.sun.java.swing.plaf.windows.WindowsLookAndFeel";

                        try {
                                                UIManager.setLookAndFeel(lookAndFeel);
                        }
                        catch (UnsupportedLookAndFeelException e){
                        }
                        catch(ClassNotFoundException e){
                        }
                        catch (Exception e){
                        }
            }
```

```
                private static void createAndShowGUI() {

                        //Set the look and feel.
                        initLookAndFeel();

                        //Make sure we have nice window decorations.
                        JFrame.setDefaultLookAndFeelDecorated(true);

                        //Create and set up the window.
                        JFrame frame = new JFrame("SimpleApplication");
                        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

                        // Create a new instance of the application class
                        SimpleApplication app = new SimpleApplication();
                        Component contents = app.createComponents();
                        frame.getContentPane().add(contents, BorderLayout.CENTER);

                        //Display the window.
                        frame.pack();
                        frame.setVisible(true);
                }

                public static void main(String[] args) {

                        //Schedule a job for the event-dispatching thread:
                        //creating and showing this application's GUI.

                        javax.swing.SwingUtilities.invokeLater(new Runnable() {
                                public void run() {
                                        createAndShowGUI();
                                }
                        });
                }
        }
```

Practice 9.2

```
        //Dialog example
```

```
import javax.swing.JOptionPane;

public class JOptionPaneExample {
          public static void main(String[] args) {
          String ans;

‘‘ //Input Dialog
‘‘ ans = JOptionPane.showInputDialog(null, "Speed in miles per hour?");
‘‘ double mph = Double.parseDouble(ans);
‘‘ double kph = 1.621 * mph;

‘‘ // Message Dialog
          JOptionPane.showMessageDialog(null, "KPH = " + kph);

‘‘ // Warning message
‘‘ JOptionPane.showMessageDialog(null,"Please pay attention", "A warning message",
          JOptionPane.WARNING_MESSAGE);

‘‘ // Question message
          JOptionPane.showMessageDialog(null,"Are you sleeping", "An question message",
           JOptionPane.QUESTION_MESSAGE );

          // Error message dialog
          JOptionPane.showMessageDialog(null, "Error!", "An error message",’‘
          ‘‘JOptionPane.ERROR_MESSAGE);

          // Input dialog with choice
          ‘‘ Object[] possibleValues = { "Enjoying", "Boring", "Sleep" };
          ‘‘ Object selectedValue = JOptionPane.showInputDialog(
                                    ‘‘null,
                                    "Choose one",
                                    "Input",
                                    JOptionPane.INFORMATION_MESSAGE,
                                    null,
                                    possibleValues,
                                    possibleValues[0]);

 JOptionPane.showMessageDialog(null, "Hey, you are now feeling = " + selectedValue);
```

```
          // Option Dialog
                     Object[] options = {"Yes", "No", "Don't know"};
                     int n = JOptionPane.showOptionDialog(
                                                  null,
                                        "Feeling hungry?",
                                        "A Silly Question",
                                        JOptionPane.YES_NO_CANCEL_OPTION,
                                        JOptionPane.QUESTION_MESSAGE,
                                        null,
                                        options,
                                        options[2]);

                                        ‘‘
                                        System.exit(0);
                     }
          }
```

Practice 9.3

```
          //example of Box layout

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.*;

          public class Password
          {
                     public static void main(String[] args)
                     {‘‘
                               JFrame frame = new JFrame("Password test");

                               frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
                               frame.getContentPane().add(new PasswordFrame(frame));
                               frame.setSize(new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT));
                               frame.setLocation(250,200);
                               frame.setResizable(false);
                               frame.show();
                     }

                     public static final int DEFAULT_WIDTH = 225;
                     public static final int DEFAULT_HEIGHT = 125;’‘
```

```
	}

	/**
		A frame that uses box layouts to organize various components.
	*/
	class PasswordFrame extends JPanel
	{
		public PasswordFrame(JFrame frame)
		{

			passFrame = frame;
			// construct the left vertical box
			JLabel label1 = new JLabel("Name");
			label1.setForeground(Color.yellow);

			JLabel label2 = new JLabel("Password");
			label2.setForeground(Color.yellow);
			label2.setDisplayedMnemonic('s');
			label2.setDisplayedMnemonicIndex(3);

			Box hbox1 = Box.createVerticalBox();
			hbox1.add(label1);
			// separate labels with a 10-pixel strut
			hbox1.add(Box.createVerticalStrut(10));
			hbox1.add(label2);

			// construct the right vertical box
			JTextField textField1 = new JTextField(10);
			textField1.setMaximumSize(textField1.getPreferredSize());

			// declare password field
			JPasswordField passwordField = new JPasswordField(10);
			passwordField.setMaximumSize(passwordField.getPreferredSize());
			passwordField.setEchoChar('@');
				// passwordField.setActionCommand(OK);
				// passwordField.addActionListener(this);
			label2.setLabelFor(passwordField);


//xxxxx.setComponentOrientation( ComponentOrientation.RIGHT_TO_LEFT);
```

```
Box hbox2 = Box.createVerticalBox();
hbox2.add(textField1);
// separate textfields with a 10-pixel strut
hbox2.add(Box.createVerticalStrut(10));
hbox2.add(passwordField);

// construct the top horizontal box
// by joining the left and roght horizontal boxes
Box hbox2a = Box.createHorizontalBox();
hbox2a.add(Box.createHorizontalStrut(10));
hbox2a.add(hbox1);
hbox2a.add(Box.createHorizontalStrut(10));
hbox2a.add(hbox2);
hbox2a.add(Box.createHorizontalStrut(10));

// construct the bottom horizontal box
JButton button1 = new JButton("‘‘ Ok’‘ ");
JButton button2 = new JButton("Cancel");

button1.setBackground(Color.gray);
button1.setForeground(Color.yellow);
button1.setBorder(BorderFactory.createRaisedBevelBorder());
button1.setActionCommand(OK);
button1.addActionListener(new buttonListener());

button2.setBackground(Color.gray);
button2.setForeground(Color.yellow);
button2.setBorder(BorderFactory.createRaisedBevelBorder());
button2.setActionCommand(CANCEL);
button2.addActionListener(new buttonListener());

Box hbox3 = Box.createHorizontalBox();
hbox3.add(button1);
          // use "glue" to push the two buttons apart
          // hbox3.add(Box.createGlue());
hbox3.add(Box.createHorizontalStrut(30));
hbox3.add(button2);

// add the two horizontal boxes inside a vertical box
```

```
                    Box vbox = Box.createVerticalBox();
                    vbox.add(hbox2a);
                    vbox.add(Box.createVerticalStrut(20));
                    vbox.add(hbox3);

                              // Container contentPane = getContentPane();
                              // contentPane.setBackground(Color.blue);
                              // contentPane.add(vbox, BorderLayout.CENTER);
                              //
                     add(vbox, BorderLayout.CENTER);
                     setBackground(Color.blue);


                    }

          class buttonListener implements ActionListener
          {
                    public void actionPerformed(ActionEvent e)
                    {
                    '' String buttonPressed = e.getActionCommand();

                              if (buttonPressed.equals(OK))
                              {
Container contentPane = messageDialog.getContentPane();
contentPane.add(new JLabel(" Success !!'' Wish you all the best"));
                              messageDialog.setSize(new Dimension(250,100));
                              messageDialog.setLocation(250+200, 200+100);
          contentPane.setBackground(new Color(252, 215, 180));
                              messageDialog.show();
                              }
                              else
                              {

                              }
                    }
          };

          JFrame passFrame;
          JDialog messageDialog = new JDialog(passFrame, "Login Message");

          private static String OK = "ok";
```

```
                    private static String CANCEL = "cancel";
          }
```

Practice 9.4

```
          //Example of Button Demo

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.AbstractButton;
          import javax.swing.JButton;
          import javax.swing.JPanel;
          import javax.swing.JFrame;
          import javax.swing.ImageIcon;

          public class ButtonDemo extends JPanel implements ActionListener {
                    protected JButton b1, b2, b3;

                    public ButtonDemo() {
                              ImageIcon leftButtonIcon = new ImageIcon("images/rightIcon.gif");
                              ImageIcon middleButtonIcon = new ImageIcon("images/midIcon.gif");
                              ImageIcon rightButtonIcon = new ImageIcon("images/leftIcon.gif");

                              b1 = new JButton("Disable middle button", leftButtonIcon);
                              b1.setVerticalTextPosition(AbstractButton.CENTER);
                              b1.setHorizontalTextPosition(AbstractButton.LEFT);
                              b1.setMnemonic(KeyEvent.VK_D);
                              b1.setActionCommand("disable");

                              b2 = new JButton("Middle button", middleButtonIcon);
                              b2.setVerticalTextPosition(AbstractButton.BOTTOM);
                              b2.setHorizontalTextPosition(AbstractButton.CENTER);
                              b2.setMnemonic(KeyEvent.VK_M);

                              b3 = new JButton("Enable middle button", rightButtonIcon);
                              //Use the default text position of CENTER, RIGHT.
                              b3.setMnemonic(KeyEvent.VK_E);
                              b3.setActionCommand("enable");
                              b3.setEnabled(false);
```

```
                    //Listen for actions on buttons 1 and 3.
                    b1.addActionListener(this);
                    b3.addActionListener(this);

                    b1.setToolTipText("Click this button to disable the middle button.");
                    b2.setToolTipText("This middle button does nothing when you click it.");
                    b3.setToolTipText("Click this button to enable the middle button.");

                    //Add Components to this container, using the default FlowLayout.
                    add(b1);
                    add(b2);
                    add(b3);
          }

          public void actionPerformed(ActionEvent e) {
                    if (e.getActionCommand().equals("disable")) {
                              b2.setEnabled(false);
                              b1.setEnabled(false);
                              b3.setEnabled(true);
                    } else {
                              b2.setEnabled(true);
                              b1.setEnabled(true);
                              b3.setEnabled(false);
                    }
          }

          public static void main(String[] args) {
                    JFrame frame = new JFrame("ButtonDemo");

                    frame.addWindowListener(new WindowAdapter() {
                              public void windowClosing(WindowEvent e) {
                                        System.exit(0);
                              }
                    });

                    frame.getContentPane().add(new ButtonDemo(), BorderLayout.CENTER);
                    frame.pack();
                    frame.setVisible(true);
          }
}
```

Practice 9.5

```
          //Flow layout Demo
          import java.awt.Container;
          import java.awt.FlowLayout;
          import javax.swing.JButton;
          import javax.swing.JFrame;

          import java.awt.Dimension;
          import java.awt.ComponentOrientation;

          public class FlowLayoutDemo {
                    public static boolean RIGHT_TO_LEFT = true;

                    public static void addComponents(Container contentPane) {
                              if (RIGHT_TO_LEFT) {
                                        contentPane.setComponentOrientation(
                                                  ComponentOrientation.RIGHT_TO_LEFT);
                              }
                              contentPane.setLayout(new FlowLayout());

                              contentPane.add(new JButton("Button 1"));
                              contentPane.add(new JButton("Button 2"));
                              contentPane.add(new JButton("Button 3"));
                              contentPane.add(new JButton("Long-Named Button 4"));
                              contentPane.add(new JButton("5"));
                    }

                    /**
                     * Create the GUI and show it.’‘For thread safety,
                     * this method should be invoked from the
                     * event-dispatching thread.
                     */
                    private static void createAndShowGUI() {
                              //Make sure we have nice window decorations.
                              JFrame.setDefaultLookAndFeelDecorated(true);

                              //Create and set up the window.
                              //JFrames decorated by the Java look and feel
                              //can't get smaller than their minimum size.
                              //We specify a skinnier minimum size than the
```

```
                        //content pane will cause the frame to request,
                        //so that you can see what happens when you
                        //drag the window so that it's narrower than a
                        //single row.
                        JFrame frame = new JFrame("FlowLayoutDemo") {
                                public Dimension getMinimumSize() {
                                        Dimension prefSize = getPreferredSize();
                                        return new Dimension(100, prefSize.height);
                                }
                        };
                        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

                        //Set up the content pane.
                        addComponents(frame.getContentPane());
                        //Display the window.
                        frame.pack();
                        frame.setVisible(true);
                }

                public static void main(String[] args) {
                        //Schedule a job for the event-dispatching thread:
                        //creating and showing this application's GUI.
                        javax.swing.SwingUtilities.invokeLater(new Runnable() {
                                public void run() {
                                        createAndShowGUI();
                                }
                        });
                }
        }
```

Practice 9.6

```
        //Example with font
        import java.awt.*;
        import java.awt.event.*;
        import javax.swing.*;
        import javax.swing.event.*;

        public class FontDialog
        {
```

```
‘‘ public static void main(String[] args)
‘‘ {‘‘
          ‘‘FontDialogFrame frame = new FontDialogFrame();
          ‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
          ‘‘frame.show();
‘‘ }
}

/**
‘‘ A frame that uses a grid bag layout to arrange font
‘‘ selection components.
*/
class FontDialogFrame extends JFrame
{‘‘
‘‘ public FontDialogFrame()
‘‘ {‘‘
          ‘‘setTitle("FontDialog");
          ‘‘setSize(WIDTH, HEIGHT);

          ‘‘Container contentPane = getContentPane();
          ‘‘GridBagLayout layout = new GridBagLayout();
          ‘‘contentPane.setLayout(layout);

          ‘‘ActionListener listener = new FontAction();

          ‘‘// construct components
          ‘‘
          ‘‘JLabel faceLabel = new JLabel("Face: ");

          ‘‘face = new JComboBox(new String[]
                    {‘‘
                              "Serif", "SansSerif", "Monospaced",
                              "Dialog", "DialogInput"
                    });
          ‘‘
          ‘‘face.addActionListener(listener);

          ‘‘JLabel sizeLabel = new JLabel("Size: ");

          ‘‘size = new JComboBox(new String[]
                    {
```

```
                        "8", "10", "12", "15", "18", "24", "36", "48"
                });

        ‘‘size.addActionListener(listener);

        ‘‘bold = new JCheckBox("Bold");
        ‘‘bold.addActionListener(listener);

        ‘‘italic = new JCheckBox("Italic");
        ‘‘italic.addActionListener(listener);

        ‘‘sample = new JTextArea();
        ‘‘sample.setText(
                "The quick brown fox jumps over the lazy dog");
        ‘‘sample.setEditable(false);
        ‘‘sample.setLineWrap(true);
        ‘‘sample.setBorder(BorderFactory.createEtchedBorder());

        ‘‘// add components to grid

        ‘‘GridBagConstraints constraints = new GridBagConstraints();

        ‘‘constraints.fill = GridBagConstraints.NONE;
        ‘‘constraints.anchor = GridBagConstraints.EAST;
        ‘‘constraints.weightx = 0;
        ‘‘constraints.weighty = 0;

        ‘‘add(faceLabel, constraints, 0, 0, 1, 1);
        ‘‘add(sizeLabel, constraints, 0, 1, 1, 1);

        ‘‘constraints.fill = GridBagConstraints.HORIZONTAL;
        ‘‘constraints.weightx = 100;
        ‘‘
        ‘‘add(face, constraints, 1, 0, 1, 1);
        ‘‘add(size, constraints, 1, 1, 1, 1);

        ‘‘constraints.weighty = 100;
        ‘‘constraints.fill = GridBagConstraints.NONE;
        ‘‘constraints.anchor = GridBagConstraints.CENTER;

        ‘‘add(bold, constraints, 0, 2, 2, 1);
```

```
                ‘‘add(italic, constraints, 0, 3, 2, 1);

                ‘‘constraints.fill = GridBagConstraints.BOTH;
                ‘‘add(sample, constraints, 2, 0, 1, 4);
        ‘‘ }
        ‘‘
        ‘‘ /**
                ‘‘A convenience method to add a component to given grid bag
                ‘‘layout locations.
                ‘‘@param c the component to add
                ‘‘@param constraints the grid bag constraints to use
                ‘‘@param x the x grid position
                ‘‘@param y the y grid position
                ‘‘@param w the grid width
                ‘‘@param h the grid height
        ‘‘ */
        ‘‘ public void add(Component c, GridBagConstraints constraints,int x, int y, int w, int h)
        ‘‘ {‘‘
                ‘‘constraints.gridx = x;
                ‘‘constraints.gridy = y;
                ‘‘constraints.gridwidth = w;
                ‘‘constraints.gridheight = h;
                ‘‘getContentPane().add(c, constraints);
        ‘‘ }

        ‘‘ public static final int WIDTH = 300;
        ‘‘ public static final int HEIGHT = 200;’‘

        ‘‘ private JComboBox face;
        ‘‘ private JComboBox size;
        ‘‘ private JCheckBox bold;
        ‘‘ private JCheckBox italic;
        ‘‘ private JTextArea sample;

        ‘‘ /**
                ‘‘An action listener that changes the font of the
                ‘‘sample text.
        ‘‘ */
        ‘‘ private class FontAction implements ActionListener
        ‘‘ {
                ‘‘public void actionPerformed(ActionEvent event)
```

```
                    ‘‘{‘‘
                              String fontFace = (String)face.getSelectedItem();
                              int fontStyle = (bold.isSelected() ? Font.BOLD : 0)
                                        + (italic.isSelected() ? Font.ITALIC : 0);
                              int fontSize = Integer.parseInt(
                                        (String)size.getSelectedItem());
                              Font font = new Font(fontFace, fontStyle, fontSize);
                              sample.setFont(font);
                              sample.repaint();
                    ‘‘}
          ‘‘ }
          }
```

Practice 9.7

```
          //Voting example

          import javax.swing.*;
          import java.awt.*;
          import java.awt.event.*;

          public class VoteDialog extends JPanel {
                    JLabel label;
                    JFrame frame;
                    String simpleDialogDesc = "The candidates";

                    public VoteDialog(JFrame frame) {
                              super(new BorderLayout());

                              this.frame = frame;
                              JLabel title;

                              //Create the components.
                              JPanel choicePanel = createSimpleDialogBox();

                              System.out.println("passed createSimpleDialogBox");

                              title = new JLabel("Click the \"Vote\" button"
                                                                  ‘‘ + " once you have selected a
candidate.",
```

```
                                                        ‘‘ JLabel.CENTER);

            label = new JLabel("Vote now!", JLabel.CENTER);
            label.setBorder(BorderFactory.createEmptyBorder(10,10,10,10));
            choicePanel.setBorder(BorderFactory.createEmptyBorder(20,20,5,20));

            //Lay out the main panel.
            add(title, BorderLayout.NORTH);’‘
            add(label, BorderLayout.SOUTH);’‘‘‘‘‘‘‘
            add(choicePanel, BorderLayout.CENTER);
    }

    void setLabel(String newText) {
            label.setText(newText);
    }

    private JPanel createSimpleDialogBox() {
            final int numButtons = 4;
            JRadioButton[] radioButtons = new JRadioButton[numButtons];

            final ButtonGroup group = new ButtonGroup();

            JButton voteButton = null;

            final String defaultMessageCommand = "default";
            final String yesNoCommand = "yesno";
            final String yeahNahCommand = "yeahnah";
            final String yncCommand = "ync";

            radioButtons[0] = new JRadioButton(
            ‘‘"Candidate 1: Sparky the Dog");
            radioButtons[0].setActionCommand(defaultMessageCommand);

            radioButtons[1] = new JRadioButton(
            ‘‘ "Candidate 2: Shady Sadie");
            radioButtons[1].setActionCommand(yesNoCommand);

            radioButtons[2] = new JRadioButton(
                    "Candidate 3: R.I.P. McDaniels");
            radioButtons[2].setActionCommand(yeahNahCommand);
```

```
                                radioButtons[3] = new JRadioButton(
                                          "Candidate 4: Duke the Java™ Platform Mascot");
                                radioButtons[3].setActionCommand(yncCommand);



                                for (int i = 0; i < numButtons; i++) {
                                          group.add(radioButtons[i]);
                                }

                                //Select the first button by default.
                                radioButtons[0].setSelected(true);

                                voteButton = new JButton("Vote");


                                voteButton.addActionListener(new ActionListener() {
                                          public void actionPerformed(ActionEvent e) {
                                                    String command =
group.getSelection().getActionCommand();


                                                    //ok dialog
                                                    if (command == defaultMessageCommand) {
                                                              JOptionPane.showMessageDialog(frame,
                                                              ‘‘ "This candidate is a dog. Invalid
vote.");


                                                    //yes/no dialog
                                                    } else if (command == yesNoCommand) {
                                                              int n =
JOptionPane.showConfirmDialog(frame,
                                                    ‘‘ ?This candidate is a convicted felon. \nDo you
still want
                                                              to vote for her?",
                                                              ‘‘ "A Follow-up Question",
                                                              ‘‘ JOptionPane.YES_NO_OPTION);
                                                              if (n == JOptionPane.YES_OPTION) {
                                                                        setLabel("OK. Keep an eye on your
wallet.");
                                                              } else if (n == JOptionPane.NO_OPTION) {
                                                                        setLabel("Whew! Good choice.");
                                                              } else {
                                                                        setLabel("It is your civic duty
to cast your vote.");
```

```
                                                  }

                                        //yes/no (with customized wording)
                                        } else if (command == yeahNahCommand) {
                                                  Object[] options = {"Yes, please", "No,
thanks"};
                                                  int n = JOptionPane.showOptionDialog(frame,
                                        '' "This candidate is deceased. \nDo you still want to
vote for him?",
                                                  '' "A Follow-up Question",
                                                  '' JOptionPane.YES_NO_OPTION,
                                                  '' JOptionPane.QUESTION_MESSAGE,
                                                  '' null,
                                                  '' options,
                                                  '' options[0]);
                                                  if (n == JOptionPane.YES_OPTION) {
                                                            setLabel("I hope you don't expect
much from your candidate.");
                                                  } else if (n == JOptionPane.NO_OPTION) {
                                                            setLabel("Whew! Good choice.");
                                                  } else {
                                                            setLabel("It is your civic duty
to cast your vote.");
                                                  }

                                        //yes/no/cancel (with customized wording)
                                        } else if (command == yncCommand) {
                                                  Object[] options = {"Yes!",
                                                                                                                                                  "N
o, I'll pass",
                                                                                                                                                  "W
ell, if I must"};
                                                  int n = JOptionPane.showOptionDialog(frame,
                                                                                                                                                  "D
uke is a cartoon mascot. \nDo you'‘"
                                                                                                                                                  +
"still want to cast your vote?",
                                                                                                                                                  "A
Follow-up Question",
            JOptionPane.YES_NO_CANCEL_OPTION,
            JOptionPane.QUESTION_MESSAGE,
            null,
```

```
          options,

          options[2]);
                                                    if (n == JOptionPane.YES_OPTION) {
                                                              setLabel("Excellent choice.");
                                                    } else if (n == JOptionPane.NO_OPTION) {
                                                              setLabel("Whatever you say. It's
your vote.");
                                                    } else if (n == JOptionPane.CANCEL_OPTION)
{
                                                              setLabel("Well, I'm certainly not
going to make you vote.");
                                                    } else {
                                                              setLabel("It is your civic duty
to cast your vote.");
                                                    }
                                          }
                                          return;
                                }
                        });
                        System.out.println("calling createPane");
                        return createPane(simpleDialogDesc + ":",
                                                    ‘‘radioButtons,
                                                    ‘‘voteButton);’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
‘
          }

          private JPanel createPane(String description,
                                                                        ‘‘JRadioButton[]
radioButtons,
                                                                        ‘‘JButton showButton) {
                        int numChoices = radioButtons.length;
                        JPanel box = new JPanel();
                        JLabel label = new JLabel(description);

                        box.setLayout(new BoxLayout(box, BoxLayout.PAGE_AXIS));
                        box.add(label);

                        for (int i = 0; i < numChoices; i++) {
                                  box.add(radioButtons[i]);
                        }
```

```
                JPanel pane = new JPanel(new BorderLayout());
                pane.add(box, BorderLayout.NORTH);
                pane.add(showButton, BorderLayout.SOUTH);
                System.out.println("returning pane");
                return pane;
        }

        /**
         * Create the GUI and show it.’‘For thread safety,
         * this method should be invoked from the
         * event-dispatching thread.
         */
        private static void createAndShowGUI() {
                //Make sure we have nice window decorations.
                JFrame.setDefaultLookAndFeelDecorated(true);
                JDialog.setDefaultLookAndFeelDecorated(true);
        ‘‘
                //Create and set up the window.
                JFrame frame = new JFrame("VoteDialog");
                frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

                //Set up the content pane.
                Container contentPane = frame.getContentPane();
                contentPane.setLayout(new GridLayout(1,1));
                contentPane.add(new VoteDialog(frame));

                //Display the window.
                frame.pack();
                frame.setVisible(true);
        }

        public static void main(String[] args) {
                //Schedule a job for the event-dispatching thread:
                //creating and showing this application's GUI.
                javax.swing.SwingUtilities.invokeLater(new Runnable() {
                        public void run() {
                                createAndShowGUI();
                        }
                });
        }
}
```

Practice 9.8

```
          //combo box with different window

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.*;

          public class CardWindow extends JFrame
                                                                        implements ItemListener {
                    boolean inAnApplet = true;

                    JPanel cards;
                    final static String BUTTONPANEL = "JPanel with JButtons";
                    final static String TEXTPANEL = "JPanel with JTextField";

                    public CardWindow() {
                              Container contentPane = getContentPane();

                     //Put the JComboBox in a JPanel to get a nicer look.
                              String comboBoxItems[] = { BUTTONPANEL, TEXTPANEL };
                              JPanel cbp = new JPanel();
                              JComboBox c = new JComboBox(comboBoxItems);
                              c.setEditable(false);
                              c.addItemListener(this);
                              cbp.add(c);

                              //Use the default layout manager, BorderLayout
                              contentPane.add(cbp, BorderLayout.NORTH);

                              cards = new JPanel();
                              cards.setLayout(new CardLayout());
          ‘‘
                              JPanel p1 = new JPanel();
                              p1.add(new JButton("Button 1"));
                              p1.add(new JButton("Button 2"));
                              p1.add(new JButton("Button 3"));

                              JPanel p2 = new JPanel();
```

```
                    p2.add(new JTextField("TextField", 20));

                    cards.add(p1, BUTTONPANEL);
                    cards.add(p2, TEXTPANEL);
                    contentPane.add(cards, BorderLayout.CENTER);

                    addWindowListener(new WindowAdapter() {
                              public void windowClosing(WindowEvent e) {
                                        if (inAnApplet) {
                                                  dispose();
                                        } else {
                                                  System.exit(0);
                                        }
                              }
                    });
          }

          public void itemStateChanged(ItemEvent evt) {
                    CardLayout cl = (CardLayout)(cards.getLayout());
                    cl.show(cards, (String)evt.getItem());
          }

          public static void main(String args[]) {
                    CardWindow window = new CardWindow();
                    window.inAnApplet = false;

                    window.setTitle("CardLayout");
                    window.pack();
                    window.setVisible(true);
          }
}
```

Practice 9.9

```
//tab window example

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
public class TabWindow extends JFrame {
        boolean inAnApplet = true;

        final static String BUTTONPANEL = "JPanel with JButtons";
        final static String TEXTPANEL = "JPanel with JTextField";

        public TabWindow() {
                Container contentPane = getContentPane();

                JTabbedPane tabbedPane = new JTabbedPane();

                JPanel p1 = new JPanel() {
                        //Force the window to be 400+ pixels wide.
                        public Dimension getPreferredSize() {
                                Dimension size = super.getPreferredSize();
                                size.width = 400;
                                return size;
                        }
                };
                p1.add(new JButton("Button 1"));
                p1.add(new JButton("Button 2"));
                p1.add(new JButton("Button 3"));
                tabbedPane.addTab(BUTTONPANEL, p1);

                JPanel p2 = new JPanel();
                p2.add(new JTextField("TextField", 20));
                tabbedPane.addTab(TEXTPANEL, p2);

                contentPane.add(tabbedPane, BorderLayout.CENTER);

                addWindowListener(new WindowAdapter() {
                        public void windowClosing(WindowEvent e) {
                                if (inAnApplet) {
                                        dispose();
                                } else {
                                        System.exit(0);
                                }
                        }
                });
        }
```

```
                public static void main(String args[]) {
                        TabWindow window = new TabWindow();
                        window.inAnApplet = false;

                        window.setTitle("TabWindow");
                        window.pack();
                        window.setVisible(true);
                }
        }
```

Practice 9.10

```
        //Create an applet for digital clock

        DigitalClockApplet.java

        import java.awt.Container;
        import java.awt.Dimension;
        import java.util.Date;
        import javax.swing.BorderFactory;
        import javax.swing.JApplet;
        import javax.swing.JCheckBox;
        import javax.swing.JLabel;
        import javax.swing.JPanel;
        import javax.swing.SwingConstants;

        public class DigitalClockApplet extends JApplet {
                ClockPanel panel;

                public void init() {
                        Container contentPane=getContentPane();
                        panel = new ClockPanel(650,150);
                        panel.resize(panel.getSize());
                        contentPane.resize(panel.getSize());
                        this.resize(panel.getSize());
                        contentPane.add(panel);
                        TimeThread tt = new TimeThread(panel);
                        tt.start();
                }
        }
```

```
class ClockPanel extends JPanel
{
           public JCheckBox jDate;
           public JCheckBox jDay;
           public JLabel jLabel1;
           public JCheckBox jSec;
           public boolean RUNCLOCK;
           ClockPanel(int wid, int hgt) {
                      jLabel1 = new JLabel();
                      jDay = new JCheckBox();
                      jDate = new JCheckBox();
                      jSec = new JCheckBox();
                      this.setLayout(null);
                      this.setSize(new Dimension(wid, hgt));
                      this.setPreferredSize(new Dimension(wid, hgt));
                      this.setBounds(0, 0, wid, hgt);
                      jLabel1.setFont(new java.awt.Font("Courier 10 Pitch", 1, 24)); // NOI18N
                      jLabel1.setForeground(new java.awt.Color(71, 113, 189));
                      jLabel1.setHorizontalAlignment(SwingConstants.CENTER);
                      jLabel1.setText("jLabel1");
                      jLabel1.setBorder(BorderFactory.createEtchedBorder());
                      this.add(jLabel1);
                      jLabel1.setBounds(30, 30, 500, 50);

                      jDay.setText("Show Day");
                      this.add(jDay);
                      jDay.setBounds(30, 100, 150, 22);

                      jDate.setText("Show Date");
                      this.add(jDate);
                      jDate.setBounds(190, 100, 180, 22);

                      jSec.setText("Show Seconds");
                      this.add(jSec);
                      jSec.setBounds(380, 100, 160, 22);

                      this.jDate.setSelected(true);
                      this.jDay.setSelected(true);
                      this.jSec.setSelected(true);
```

```
		}
	}


	class TimeThread extends Thread implements Runnable {
		ClockPanel cp = null;
		private int day;
		private String dayn;
		private String dat;
		private int sec;
		private int min;
		private int hours;
		public TimeThread (ClockPanel cfArg) {
			cp = cfArg;
			cp.RUNCLOCK = true;
		}

		public void run() {
			String dt = "";
			while (cp.RUNCLOCK) {
				Date date = new Date();
				min = date.getMinutes();
				hours = date.getHours();
				if (cp.jDay.isSelected()) day = date.getDay();
				else day=-1;
				//GET DAY NAME
				dayn="";
				if (day!=-1) {
					switch (day) {
						case 0: dayn = "Sunday"; break;
						case 1: dayn = "Monday"; break;
						case 2: dayn = "Tuesday"; break;
						case 3: dayn = "Wednesday"; break;
						case 4: dayn = "Thursay"; break;
						case 5: dayn = "Friday"; break;
						case 6: dayn = "Saturday"; break;
						default: dayn = "OOPS!"; break;
					}
				}
```

```
                                        //IS DATE REQD
                                        if (cp.jDate.isSelected()) dat = date.getDate() + "-" +
date.getMonth() + "-" + date.getYear();
                                        else dat="";

                                        dt = dayn + " " + dat + "‘‘" + hours + ":" + min;
                                        if (cp.jSec.isSelected()) {
                                                  sec = date.getSeconds();
                                                  dt += ":" + sec + "";
                                        }
                                        else sec=-1;
                                        //dt = date.toString();
                                        cp.jLabel1.setText(dt);
                                        try {
                                                  Thread.sleep(10);
                                        } catch (InterruptedException ex) {
                                                  System.out.println("Exception: " + ex.toString());
                                        }
                              }
                    }
          }

          DigitalClockApplet.htm

          < HTML>
          < HEAD>
          < / HEAD>
          < BODY BGCOLOR="000000">
          < CENTER>
          < applet code="DigitalClockApplet.class" WIDTH=660 HEIGHT=150>
          < / applet>
          < / CENTER>
          < / BODY>
          < / HTML>
```

Practice 9.11

```
          // Create an Application for Digital Clock

          import java.awt.Container;
          import java.util.Date;
```

```
          import javax.swing.BorderFactory;
          import javax.swing.JCheckBox;
          import javax.swing.JFrame;
          import javax.swing.JLabel;
          import javax.swing.JPanel;
          import javax.swing.SwingConstants;
          import javax.swing.WindowConstants;

          public class DigitalClock extends JFrame {
                    ClockPanel panel;
                    public DigitalClock() {
                              this.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
                              Container contentPane=this.getContentPane();
                              panel = new ClockPanel();
                              contentPane.add(panel);
                              java.awt.Dimension screenSize =‘‘‘‘
                              java.awt.Toolkit.getDefaultToolkit().getScreenSize();
                              this.setBounds((screenSize.width-573)/2, (screenSize.height-172)/2, 573,
172);
                    }

                    public static void main(String args[]) {
                               DigitalClock dc =‘‘new DigitalClock();
                               dc.setVisible(true);
                               TimeThread tt = new TimeThread(dc.panel);
                               tt.start();
                    }
          }
```

Practice 9.12

```
          //Create an applet for calculator

          CalcApplet.java

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.*;

          public class CalcApplet extends JApplet
          {
                    public void init()
```

```
                {
                Container’‘contentPane=getContentPane();
                CalculatorPanel panel=new CalculatorPanel();
                contentPane.add(panel);
                }
        }

        class CalculatorPanel extends JPanel
        {
                public CalculatorPanel()
                {
                setLayout(new BorderLayout());

                result = 0;
                lastCommand = "=";
                start = true;

                display = new JTextField("0.0");
                          display.setEditable(false);
                add(display,BorderLayout.NORTH);

                ActionListener insert = new InsertAction();
                ActionListener command = new CommandAction();

                panel = new JPanel();
                panel.setLayout(new GridLayout(8,4));

                addButton("sin",command);
                addButton("cos",command);
                addButton("tan",command);
                addButton("A/C",command);
                addButton("asin",command);
                addButton("acos",command);
                addButton("atan",command);
                addButton("n!",command);
                addButton("log",command);
                addButton("ln",command);
                addButton("Pi",insert);
                addButton("e",insert);
                addButton("x^(1/y)",command);
                addButton("x^y",command);
```

```
                addButton("Sqrt",command);
                addButton("/",command);
                addButton("9",insert);
                addButton("8",insert);
                addButton("7",insert);
                addButton("x",command);
                addButton("6",insert);
                addButton("5",insert);
                addButton("4",insert);
                addButton("+",command);
                addButton("3",insert);
                addButton("2",insert);
                addButton("1",insert);
                addButton("_",command);
                addButton("0",insert);
                addButton("+/-",command);
                addButton("=",command);
                addButton(".",insert);

                add(panel,BorderLayout.CENTER);
                }

                private void addButton(String label,ActionListener listener)
                {
                JButton button = new JButton(label);
                button.addActionListener(listener);
                panel.add(button);
                }


        private class InsertAction implements ActionListener
        {
                public void actionPerformed(ActionEvent event)
                {
                String input = event.getActionCommand();
                        if(start)
                        {
                        display.setText("");
                        start = false;
                        }
                ‘‘if(input.equals("e"))
```

```
                    {
                    ‘‘if(display.getText().equals("-"))
                    ‘‘ display.setText("-"+Math.E);
                              else
                              display.setText(""+Math.E);
                    ‘‘start=true;
                    }
          ‘‘else if(input.equals("Pi"))
                    {
                              if(display.getText().equals("-"))
                              ‘‘display.setText("-"+Math.PI);
                              else
                              ‘‘display.setText(""+Math.PI);
                    start=true;
                    }
          else
                    display.setText(display.getText()+input);
          }
}

private class CommandAction implements ActionListener
{
          public void actionPerformed(ActionEvent event)
          {
          String command = event.getActionCommand();
          String co = command;
          if(co.equals("A/C"))
                    {
                    result = 0;
                    display.setText(""+result);
                    lastCommand = "=";
                    start=true;
                    }
          else if((co.equals("+/-"))||(co.equals("Sin"))
                              ‘‘||(co.equals("Cos"))||(co.equals("Tan"))||(co.equals("Sqrt"))
                              ‘‘||(co.equals("n!"))||(co.equals("ln"))||(co.equals("aSin"))
                              ||(co.equals("aCos"))||(co.equals("aTan"))||(co.equals("log")))
                    ‘‘ {
                              lastCommand=command;
                              ‘‘calculate(Double.parseDouble(display.getText()));
                              ‘‘lastCommand="=";
```

```
                                        ‘‘ start=true;
                                        }
                                        ‘‘else if(start&&(lastCommand.equals("="))&&(result==0)
                                                                                ‘‘&&(display.getText().
equals("0.0")))
                                        ‘‘{
                                        if(command.equals("_"))
                                         {
                                        display.setText("-");
                                        start = false;
                                         }
                                        else
                                        ‘‘lastCommand = command;
                                         }
                    else
                                        {
                              ‘‘ calculate(Double.parseDouble(display.getText()));
                                         lastCommand=command;
                                         start= true;
                              }
                    }
          }

          public void calculate(double x)
          {
                    if(lastCommand.equals("+")) result += x;
                    else if(lastCommand.equals("_")) result -= x;
                    else if(lastCommand.equals("x")) result *= x;
                    else if(lastCommand.equals("/")) result /= x;
                    else if(lastCommand.equals("+/-")) result = -x;
                    else if(lastCommand.equals("Sin")) result = Math.sin(x*Math.PI/180.0);
                    else if(lastCommand.equals("Cos")) result = Math.cos(x*Math.PI/180.0);
                    else if(lastCommand.equals("Tan")) result = Math.tan(x*Math.PI/180.0);
                    else if(lastCommand.equals("=")) result = x;
                    else if(lastCommand.equals("x^y")) result = Math.pow(result,x);
                    else if(lastCommand.equals("Sqrt")) result = Math.sqrt(x);
                    else if(lastCommand.equals("n!")) result = fact((int)x);
                    else if(lastCommand.equals("ln")) result = Math.log(x);
                    else if(lastCommand.equals("aSin")) result = Math.asin(x)*180/Math.PI;
                    else if(lastCommand.equals("aCos")) result = Math.acos(x)*180/Math.PI;
                    else if(lastCommand.equals("aTan")) result = Math.atan(x)*180/Math.PI;
```

```
                    else if(lastCommand.equals("log")) result = Math.log(x)/Math.log(10.0);
                    else if(lastCommand.equals("x^(1/y)")) result = Math.pow(result,1.0/x);

                    display.setText(""+result);
          }

           private int fact(int n)
                    {
                              if((n==0)||(n==1)) return 1;
                              else return n*fact(n-1);
                    }

          private JTextField display;
          private JPanel panel;
          private double result;
          private String lastCommand;
          private boolean start;
          }

          CalcApplet.htm
          < HTML>
          < HEAD>
          < / HEAD>
          < BODY BGCOLOR="000000">
          < CENTER>
          < applet code="CalcApplet.class"WIDTH=400 HEIGHT=400>
          < / applet>
          < / CENTER>
          < / BODY>
          < / HTML>
```

Practice 9.13

```
          //Create an application for calculator

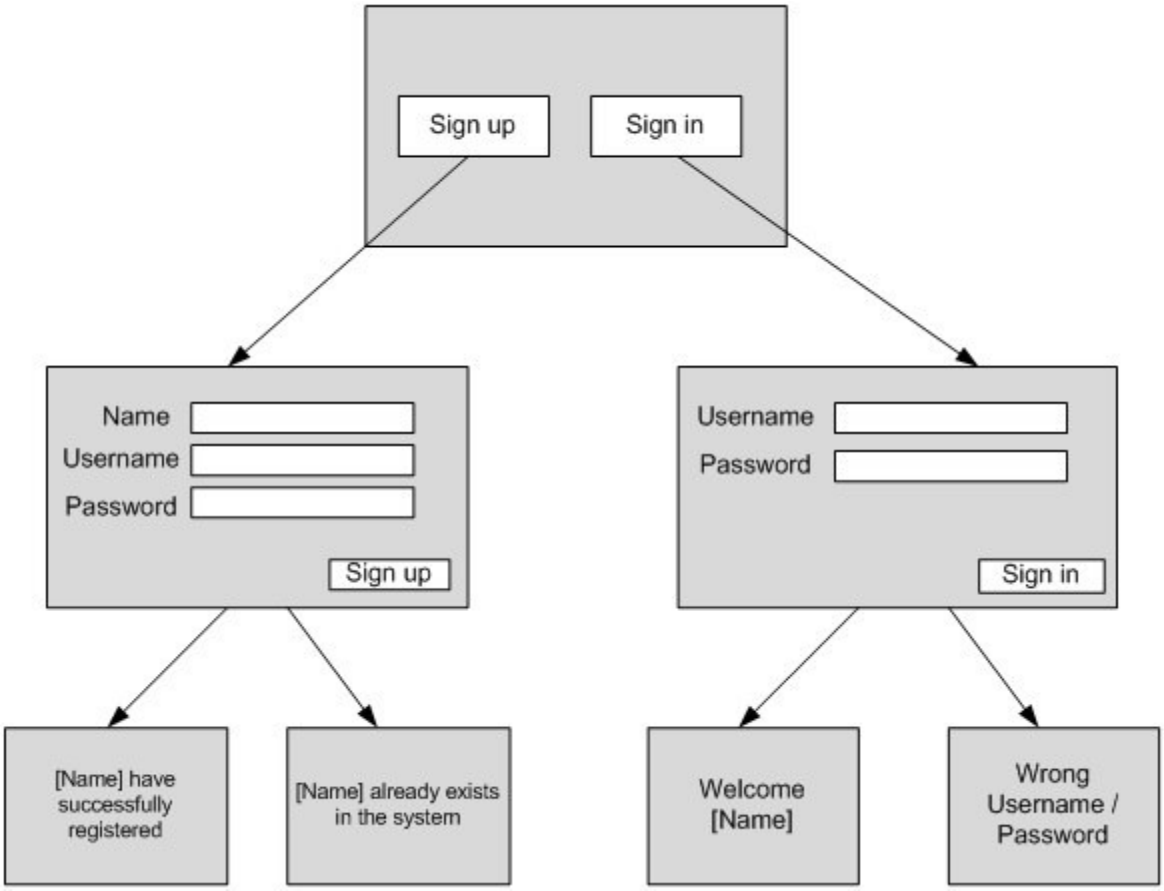
          import java.awt.BorderLayout;
          import java.awt.Container;
          import java.awt.GridLayout;
          import java.awt.event.ActionEvent;
          import java.awt.event.ActionListener;
          import javax.swing.JButton;
```

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class CalcApplication {
          public static void main(String args[]) {
                    JFrame’‘mainFrame = new JFrame();
                    mainFrame.setSize(400, 300);
                    Container’‘contentPane=mainFrame.getContentPane();
          CalculatorPanel panel=new CalculatorPanel();
          contentPane.add(panel);
                    mainFrame.setVisible(true);
          }
}
```

Assignment

Develop a swing application for login validation of a system as shown in the figure. The application shows two options: one ﷺign up﷽ and another ﷺign in﷽. When ﷺign up﷽ is selected, a new screen will appear for asking name, username, password. If the entered username and password do not exist in the database, it will show a successful message ﷺ[Name] has successfully registered﷽ and store corresponding username, password in the database, otherwise the message ﷺ[Name] already exists in the system﷽ will be displayed. On the other hand, when ﷺign In﷽ option is selected, the user will be prompted to enter username, password as shown in the figure. Eventually, the application checks whether username and password both match with some entry in the database. If such entry found in the database, then a welcome message ﷺelcome [Name]﷽ will appear, otherwise an unsuccessful login message ﷺrong username/password﷽ will be displayed. For message display, the swing dialogs can be used. You should have some login information in your database so that all four situations can be validated.



Q&A

Q: What is difference between invokeAndWait and invokeLater in Java?

A: Swing is not thread-safe and at same time you can not perform time consuming task in EDT thread. InvokeAndWait and InvokeLater method allows to enqueue task for EDT thread to perform, InvokeAndWait is a blocking method in Java and does it synchronously and invokeLater does it asynchronously. Since GUI can only be updated in Event dispatcher thread, if you need to show progress of any task, you can use these two methods.

Q: What is EDT thread in Swing?

A: EDT stands for Event dispatcher thread. EDT is one of the most important thing to learn about Swing, Since Swing is single-threaded all GUI drawing and event handling is done in EDT thread and that's why its recommended not to perform any time consuming task e.g. connecting to database or opening network connection in Event Dispatcher thread or EDT thread. If you do that, it may lead to frozen or hung GUI. This question leads to several other questions in Java, e.g. If you can not open database connection in EDT thread than how will you open db connection with button click etc. well this can be done by spawning a new thread from button click and opening db connection there.

Q: How to change a button from enable to disable after click ?

A: When button is clicked an action event is fired which can be captured by implementing ActionListener interface and actionPerformed(ActionEvent ae) method. You can then call button.setEnable(false) to disable this button.

Q: Why Swing is called light weight ?

A: Most of Swing component are inherited form JComponent and doesn't required a native peer and that's why they are referred as light weight component. light weight component implement look and feel in Java rather than using look and feel of native component and that's why Swing's look and feel remains same across platform.

Q: Is Swing Thread safe in Java ?

A: No, Swing is not thread-safe in Java. Swing components are not thread-safe they can not be modified from multiple threads. All swing components are designed to be updated by using Event dispatcher thread or EDT thread

Q: Which method of Swing are thread-safe?

A: Only couple of methods like repaint() and revalidate() are thread-safe in Swing, i.e. they can be called from multiple threads without additional synchronization in Java.

Q: What is difference between Container and Component ?

A: Main difference between Container and Component is that former can hold other components e.g. JFrame which is used as container to hold other components e.g. JButton.

Q: What is the purpose of action interface in Swing?

A: Action is performed on a state to allow it to change. It defines the interface that it is implementing. The library that is used for the action interface is javax.swing.Action. This action interface extends the ActionListener interface class that is being provided from the AWT. Action interface allow the concrete classes to implement the actionPerformed() method and provide the action that is associated with it. The use of actionPerformed() method allow the implementation of the behavior that is desired in the programming and allow it to show the functionality of the action class. The action can be added to the container class that accepts the parameters on an event like JMenu, JPopupMenu, or JtoolBar. The container used in this automatically registers the action that is taken on the event and it acts as an ActionListener of the user interface.

Q: Explain Thread Rule in Swing?

A: Once a swing component has been realized, i.e. the component has been painted on screen or it is ready to painted, all the code that might affect or depend on the state of that component should be executed in the event dispatching thread. This is called thread rule in swing..

Q: What is the use of double buffering in Swing?

A: Double-buffering is the process of drawing graphics into an off-screen image buffer and then copying the contents of the buffer to the screen all at once. For complex graphics, using double-buffering can reduce flickering.

Q: What is the purpose of Serialization in Swing?

A: Serialization is used when an object extends the Jcomponent class. The object's data in this case points out to the stream that is written or serialized and gives an output stream. This output stream can be sent through then network and it can be

saved in the file if required. The serialized file can be de-serialized in the memory. This will allow the operation to continue to perform the original state functions that were getting used. The serialization used in object allows easy retrieval of the object data and easy transmission of state data from one place to another. This can also be placed in custom made storage files that provide the transfer of the components to be fast and it uses the concept of virtual machine that transfers from one machine to another machine. It uses the remote method invocation to allow the use of distributed computing that provide with some exceptions to the use of transient keyword. The components can be serialized using the classes that can pass the reference to an object using the writeObject() method of the ObjectOutputStream object this way it calls the serialization function recursively.

Q: How do you classify Swing Components?

A: Swing components are classified under the following headings:

```
1.Top level containers – The containers at the top of any swing component hierarchy are:

Applet

Dialog

Frame

2.General purpose containers – The intermediate containers are

Panel

Scroll pane

Split pane

Tabbed pane

Tool bar

3.Special purpose containers – These are intermediate containers which play specific roles in the
user interface:

Internal frame

Layered pane

Root pane

4.Basic controls : The atomic components that exist primarily to get input from the user are

Buttons

Combo box

List

Menu

Slider

Spinner

TextField

5.Uneditable information displays: The atomic components which give information to the user are

Label

Progress bar

Tool tip

6.Interactive displays of highly formatted information – The atomic components that display
formatted information that can be modified by the user are
```

```
Color chooser
File chooser
Table
Text
Tree
```

Q: What is the main difference between Dialog and Frame?

A: In AWT, the top level window, which is itself not contained in another window, is called a FRAME.
On the other hand, a window that appears over existing window (mostly Frame) and forces the user to respond is called a DIALOG.

```
Frame f = new Frame(); // creates an object of type Frame.
```

A FRAME can exists on its own , where as a DIALOG cannot exist on its own. In other words, FRAME is a primary window, where as the DIALOG box is secondary window. The dialog boxes are of two types:

□'‘modal This dialog won't let the user interact with the remaining containers/windows until he deals with it.

□'‘modeless - The user can interact with other windows as well as the dialog box, e.g. the toolbox.
There is no default constructor for a DIALOG. The following statement constructs a dialog box:

```
public Dialog( Frame parent, String title, boolean modal) Here:

parent  The container to which the dialog belongs.

title  Title of the dialog

modal  True for modal type, false for modeless type.
```

Chapter 10

Java Swing

by Debasis Samanta

CONTENTS

- Introduction
- AWT versus Swing
- Swing in depth
- Java Swing examples
- Practice Questions
- Assignment
- Q&A

Introduction

Java Swing is a part of Java Foundation Classes (JFC) that is used to create window-based applications. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in Java. Java Swing provides better lightweight components than AWT. The javax.swing package provides classes for Java Swing components such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

AWT versus Swing

Java AWT	Java Swing
AWT components are platform-dependent.	Java swing components are platform-independent.
AWT components are heavyweight.	Swing components are lightweight.
AWT doesn't support pluggable look and feel.	Swing supports pluggable look and feel.
AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane, etc.
AWT doesn't follows MVC(Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC.

Figure 10.1 :AWT versus Swing

NoteThe Java Foundation Classes (JFC) are a set of GUI components which simplify the development of desktop applications.

Swing in depth

Hierarchy of Java Swing classes:

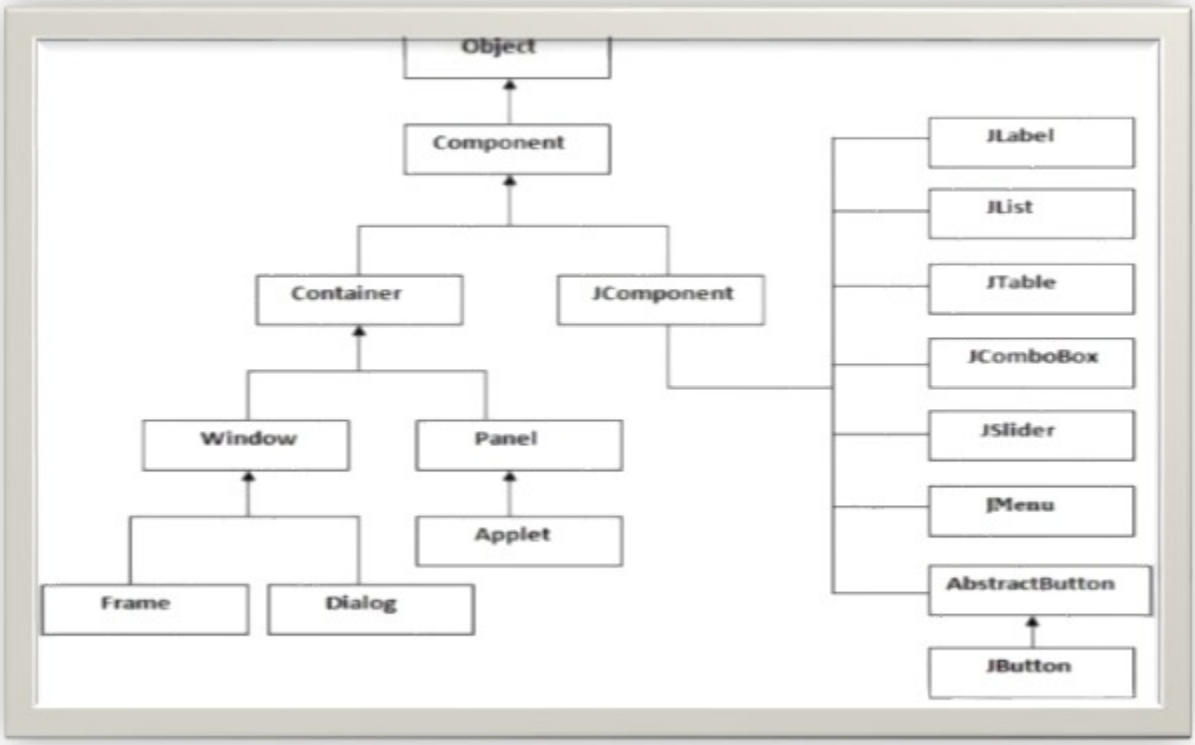


Figure 10.2 :Hierarchy of Java Swing classes

Commonly used methods in Component class:

Method	*Description*
`public void add(Component c)`	Add a component on another compon
`public void setSize(int width,int height)`	Sets size of the component.
`public void setLayout(LayoutManager m)`	Sets the layout manager for the comp
`public void setVisible(boolean b)`	Sets the visibility of the component.

Figure 10.3 :Commonly used methods in Component class:

Java Swing examples

There are two ways to create a frame:

- ''By creating the object of Frame class (Association)
- ''By extending Frame class (Inheritance)We can write the code of Swing inside the main(), constructor or any other method.

JFrame object :

Let's see a simple swing example where we are creating one button and adding it on the **JFrame** object inside the **main()** method.



Figure 10.4 :JFrame object

Illustration 10.1

```
// A program to add a label and button in a frame


import java.awt.FlowLayout;‘‘
import javax.swing.JButton;‘‘
import javax.swing.JFrame;‘‘
import javax.swing.JLabel;‘‘
import javax.swing.Jpanel;‘‘
public class JFrameExample {‘‘
‘‘‘‘public static void main(String s[]) {‘‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("JFrame Example");‘‘
‘‘‘‘‘‘‘‘JPanel panel = new JPanel();‘‘
‘‘‘‘‘‘‘‘panel.setLayout(new FlowLayout());‘‘
‘‘‘‘‘‘‘‘JLabel label = new JLabel("JFrame By Example");‘‘
‘‘‘‘‘‘‘‘JButton button = new JButton();‘‘
‘‘‘‘‘‘‘‘button.setText("Button");‘‘
‘‘‘‘‘‘‘‘panel.add(label);‘‘
‘‘‘‘‘‘‘‘panel.add(button);‘‘
‘‘‘‘‘‘‘‘frame.add(panel);‘‘
‘‘‘‘‘‘‘‘frame.setSize(200, 300);‘‘
‘‘‘‘‘‘‘‘frame.setLocationRelativeTo(null);‘‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘‘‘‘‘frame.setVisible(true);‘‘
```

```
‘‘‘‘}’‘
}’‘
```

Illustration 10.2

```
// A program to add panel to GUI

import java.awt.*;’‘
import javax.swing.*;’‘
public class PanelExample {‘‘
‘‘‘‘ PanelExample()’‘{‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame("Panel Example");’‘‘‘
‘‘‘‘‘‘‘‘JPanel panel=new JPanel();’‘
‘‘‘‘‘‘‘‘panel.setBounds(40,80,200,200);’‘‘‘
‘‘‘‘‘‘‘‘panel.setBackground(Color.gray);’‘
‘‘‘‘‘‘‘‘JButton b1=new JButton("Button 1");’‘‘‘
‘‘‘‘‘‘‘‘b1.setBounds(50,100,80,30);’‘‘‘
‘‘‘‘‘‘‘‘b1.setBackground(Color.yellow);’‘
‘‘‘‘‘‘‘‘JButton b2=new JButton("Button 2");’‘
‘‘‘‘‘‘‘‘b2.setBounds(100,100,80,30);’‘‘‘
‘‘‘‘‘‘‘‘b2.setBackground(Color.green);’‘
‘‘‘‘‘‘‘‘panel.add(b1);
                    panel.add(b2);’‘
‘‘‘‘‘‘‘‘f.add(panel);’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setSize(400,400);’‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setVisible(true);’‘‘‘
‘‘‘‘‘‘‘‘}’‘
‘‘‘‘‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘new PanelExample();’‘
‘‘‘‘‘‘‘‘}’‘
‘‘‘‘}’‘
```

We can also write all the codes of creating JFrame, JButton and method call inside the Java constructor.

Illustration 10.3

```
‘‘
//Create a Swing button
```

```
import javax.swing.*;‘‘
public class FirstSwingExample {‘‘
          public static void main(String[] args) {‘‘
                    //creating instance of JFrame
                    JFrame f=new JFrame();
‘‘‘‘‘‘‘‘‘          //creating instance of JButton’‘
                    JButton b=new JButton("click");
                    //x axis, y axis, width, height
                    b.setBounds(130,100,100, 40);
                    f.add(b);//adding button in JFrame
                    f.setSize(400,500);//400 width and 500 height’‘
                    f.setLayout(null);//using no layout managers’‘
                    f.setVisible(true);//making the frame visible’‘
          }’‘
}
```

Illustration 10.4

```
// Creating JFrame, JButton and method call inside the java constructor.

import javax.swing.*;‘‘
public class Simple {‘‘
          JFrame f;’‘
          Simple(){‘‘
          f=new JFrame();
          JButton b=new JButton("click");
          b.setBounds(130,100,100, 40);’‘
          f.add(b);
          f.setSize(400,500);
          f.setLayout(null);
          f.setVisible(true);
}
          public static void main(String[] args) {‘‘
                    new Simple();’‘
          }’‘
}
```

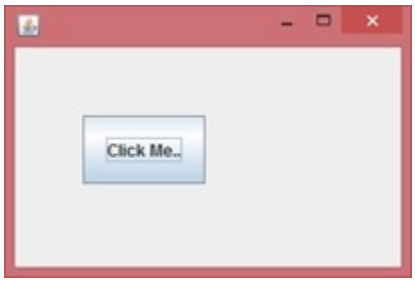
Illustration 10.5

```
// inherit the JFrame class, so there is no need to create the instance of JFrame class explicitly.


import javax.swing.*;
//inheriting JFrame’‘
public class Simple2 extends JFrame{
          JFrame f;’‘
          Simple2(){‘‘
                    JButton b=new JButton("click");
                    b.setBounds(130,100,100, 40);’‘
                    add(b);
                    setSize(400,500);’‘
          ‘‘‘‘‘‘ setLayout(null);’‘
                    setVisible(true);’‘
          }’‘
          public static void main(String[] args) {‘‘
                    new Simple2();’‘
          }
}
```

Java Swing JButton

The **JButton class** is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed. It inherits AbstractButton class. Below is the declaration for **javax.swing.JButton class.**
public class JButton extends AbstractButton implements Accessible



Class JButton : Constructors:

Constructor	Description
JButton()	It creates a button with no text and i
JButton(String s)	It creates a button with the specified
JButton(Icon i)	It creates a button with the specified

Class JButton : Methods:

Methods	Descrip
`void setText(String s)`	It is used to set specified text on but
`String getText()`	It is used to return the text of the bu
`void setEnabled(boolean b)`	It is used to enable or disable the bu
`void setIcon(Icon b)`	It is used to set the specified Icon or
`Icon getIcon()`	It is used to get the Icon of the butto
`void setMnemonic(int a)`	It is used to set the mnemonic on the
`void addActionListener(ActionListener a)`	It is used to add the action listener t

Creating a JButton : An example

Let's see a simple swing example where we are creating one button and adding it on the JFrame object inside the main() method. also illustarte Java JButton : An example with ActionListener

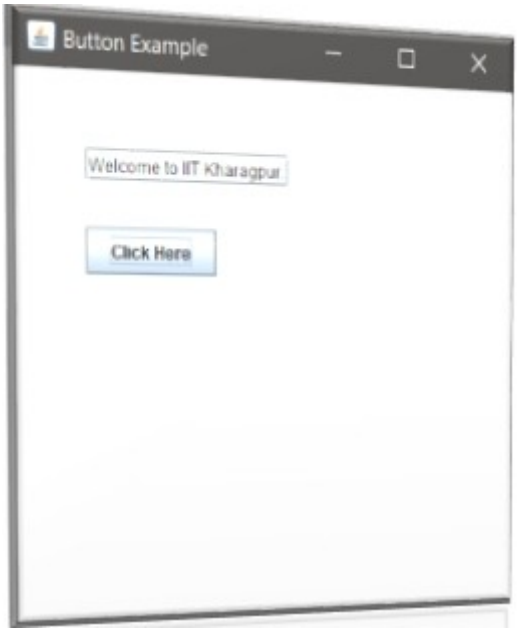


Illustration 10.6

```
// Button with Action Listner
import java.awt.event.*;‘‘
import javax.swing.*;’’‘‘
```

```
public class ButtonExample {‘‘
public static void main(String[] args) {‘‘
‘‘‘‘JFrame f=new JFrame("Button Example");’’
‘‘‘‘final JTextField tf=new JTextField();’’
‘‘‘‘tf.setBounds(50,50, 150,20);’’
‘‘‘‘JButton b=new JButton("Click Here");’’
‘‘‘‘b.setBounds(50,100,95,30);’’
‘‘‘‘b.addActionListener(new ActionListener(){‘‘
‘‘‘‘public void actionPerformed(ActionEvent e){‘‘
‘‘‘‘‘‘‘‘‘‘‘‘tf.setText("Welcome to Javatpoint.");’’
‘‘‘‘‘‘‘‘}’’
‘‘‘‘});’’
‘‘‘‘f.add(b);f.add(tf);’’
‘‘‘‘f.setSize(400,400);’’
‘‘‘‘f.setLayout(null);’’
‘‘‘‘f.setVisible(true);’’
}’’
}’’
```

Java JButton : Displaying image on the button



Illustration 10.7

```
// Button with image
```

```
import javax.swing.*;‘‘‘‘‘‘
public class ButtonExample{‘‘‘‘
ButtonExample(){‘‘‘‘
          JFrame f=new JFrame("Button Example");‘‘‘‘‘‘‘‘‘‘‘‘
          JButton b=new JButton(new ImageIcon("D:\\icon.png"));‘‘‘‘
          b.setBounds(100,100,100, 40);‘‘‘‘
          f.add(b);‘‘‘‘
          f.setSize(300,400);‘‘‘‘
          f.setLayout(null);‘‘‘‘
          f.setVisible(true);‘‘‘‘
          f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘‘‘
‘‘‘‘}‘‘‘‘‘‘‘
          public static void main(String[] args) {‘‘‘‘
                    new ButtonExample();‘‘‘‘
          }‘‘‘‘
}
```

Java Swing JLabel:

The object of **JLabel class** is a component for placing text in a container. It is used to display a single line of read only text. The text can be changed by an application but a user cannot edit it directly. It inherits **JComponent class.**



Below is the declaration for javax.swing.JLabel class. **public�class�JLabel�extends�JComponent�implements�SwingConstants,�Accessible�**

Illustration 10.8

```
import javax.swing.*;
class LabelExample‘‘
{‘‘
public static void main(String args[]){‘‘
```

```
‘‘‘‘JFrame f= new JFrame("Label Example");’‘
‘‘‘‘JLabel l1,l2;’‘
‘‘‘‘l1=new JLabel("First Label.");’‘
‘‘‘‘l1.setBounds(50,50, 100,30);’‘
‘‘‘‘l2=new JLabel("Second Label.");’‘
‘‘‘‘l2.setBounds(50,100, 100,30);’‘
‘‘‘‘f.add(l1); f.add(l2);’‘
‘‘‘‘f.setSize(300,300);’‘
‘‘‘‘f.setLayout(null);’‘
‘‘‘‘f.setVisible(true);’‘
‘‘‘‘}’‘
}’‘
```

Java JLabel : An example with ActionListener

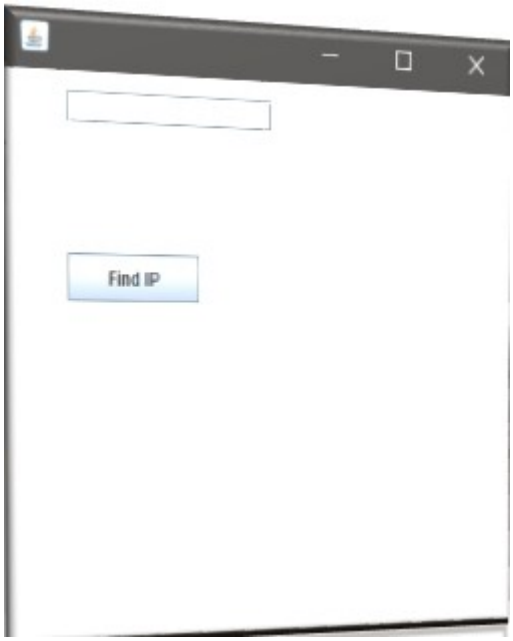


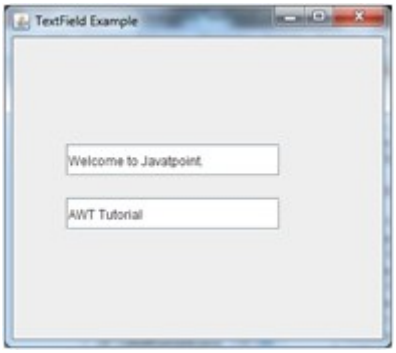
Illustration 10.9

```
import javax.swing.*;’‘
import java.awt.*;’‘
import java.awt.event.*;
‘‘
public class LabelExample extends Frame implements ActionListener{‘‘
```

```
‘‘‘‘JTextField tf; JLabel l; JButton b;’‘
‘‘‘‘LabelExample(){‘‘
‘‘‘‘‘‘‘‘tf=new JTextField();’‘
‘‘‘‘‘‘‘‘tf.setBounds(50,50, 150,20);’‘
‘‘‘‘‘‘‘‘l=new JLabel();’‘
‘‘‘‘‘‘‘‘l.setBounds(50,100, 250,20);’‘‘‘‘‘
‘‘‘‘‘‘‘‘b=new JButton("Find IP");’‘
‘‘‘‘‘‘‘‘b.setBounds(50,150,95,30);’‘
‘‘‘‘‘‘‘‘b.addActionListener(this);’‘‘‘
‘‘‘‘‘‘‘‘add(b);add(tf);add(l);’‘‘‘
‘‘‘‘‘‘‘‘setSize(400,400);’‘
‘‘‘‘‘‘‘‘setLayout(null);’‘
‘‘‘‘‘‘‘‘setVisible(true);’‘
‘‘‘‘}’‘
```

Java Swing JTextField:

The object of a **JTextField class** is a text component that allows the editing of a single line text. It inherits **JTextComponent class.**



Below is the declaration for javax.swing.JTextField class. **public class JTextField extends JTextComponent implements SwingConstants**

Class JTextField : Constructors

Constructor	Description
`JTextField()`	Creates a new TextF
`JTextField(String text)`	Creates a new TextField initia specified text.
`JTextField(String text, int columns)`	Creates a new TextField initia specified text and colu
`JTextField(int columns)`	Creates a new empty TextFi specified number of co

Class JTextField : Methods

Methods	Desc
`void addActionListener(ActionListener l)`	It is used to add the specif action events from this tex
`Action getAction()`	It returns the currently set source, or null if no Actio
`void setFont(Font f)`	It is used to set the current
`void removeActionListener(ActionListener l)`	It is used to remove the sp that it no longer receives a textfield.

```
Illustration 10.10
/*
```

```
Simple calculator for Addition and Subtraction of numbers in TextField
*/

import javax.swing.*;‘‘
import java.awt.event.*;‘‘
public class TextFieldExample implements ActionListener{‘‘
‘‘‘‘JTextField tf1,tf2,tf3;‘‘
‘‘‘‘JButton b1,b2;‘‘
‘‘‘‘TextFieldExample(){‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame();‘‘
‘‘‘‘‘‘‘‘tf1=new JTextField();‘‘
‘‘‘‘‘‘‘‘tf1.setBounds(50,50,150,20);‘‘
‘‘‘‘‘‘‘‘tf2=new JTextField();‘‘
‘‘‘‘‘‘‘‘tf2.setBounds(50,100,150,20);‘‘
‘‘‘‘‘‘‘‘tf3=new JTextField();‘‘
‘‘‘‘‘‘‘‘tf3.setBounds(50,150,150,20);‘‘
‘‘‘‘‘‘‘‘tf3.setEditable(false);‘‘
‘‘‘‘‘‘‘‘b1=new JButton("+");‘‘
‘‘‘‘‘‘‘‘b1.setBounds(50,200,50,50);‘‘
‘‘‘‘‘‘‘‘b2=new JButton("-");‘‘
‘‘‘‘‘‘‘‘b2.setBounds(120,200,50,50);‘‘
‘‘‘‘‘‘‘‘b1.addActionListener(this);‘‘
‘‘‘‘‘‘‘‘b2.addActionListener(this);‘‘
‘‘‘‘‘‘‘‘f.add(tf1);f.add(tf2);f.add(tf3);f.add(b1);f.add(b2);
‘‘‘‘‘‘‘‘ f.setSize(300,300);‘‘
‘‘‘‘‘‘‘‘ f.setLayout(null);‘‘
‘‘‘‘‘‘‘‘ f.setVisible(true);‘‘
‘‘‘‘}‘‘‘‘‘‘
‘‘‘‘public void actionPerformed(ActionEvent e) {‘‘
‘‘‘‘‘‘‘‘String s1=tf1.getText();‘‘
‘‘‘‘‘‘‘‘String s2=tf2.getText();‘‘
‘‘‘‘‘‘‘‘int a=Integer.parseInt(s1);‘‘
‘‘‘‘‘‘‘‘int b=Integer.parseInt(s2);‘‘
‘‘‘‘‘‘‘‘int c=0;‘‘
‘‘‘‘‘‘‘‘if(e.getSource()==b1){‘‘
‘‘‘‘‘‘‘‘‘‘‘‘c=a+b;‘‘
‘‘‘‘‘‘‘‘}else if(e.getSource()==b2){‘‘
‘‘‘‘‘‘‘‘‘‘‘‘c=a-b;‘‘
‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘String result=String.valueOf(c);‘‘
‘‘‘‘‘‘‘‘tf3.setText(result);‘‘
```

```
‘‘‘‘}’’
          public static void main(String[] args) {‘‘
                    new TextFieldExample();’‘
          }
}
```

Java Swing JTextArea:

The object of a **JTextArea class** is a multi line region that displays text. It allows the editing of multiple line text. It inherits **JTextComponent class.**



Class JTextArea : constructors:

Constructor	Description
JTextArea()	Creates a text area that displays no text i
JTextArea(String s)	Creates a text area that displays specifie initially.
JTextArea(int row, int column)	Creates a text area with the specified nu rows and columns that displays no text i
JTextArea(String s, int row, int column)	Creates a text area with the specified nu rows and columns that displays specifie

Class JTextArea : methods:

Methods	Description
`void setRows(int rows)`	It is used to set specified number of r
`void setColumns(int cols)`	It is used to set specified number of c
`void setFont(Font f)`	It is used to set the specified font.
`void insert(String s, int position)`	It is used to insert the specified text o position.
`void append(String s)`	It is used to append the given text to t document.

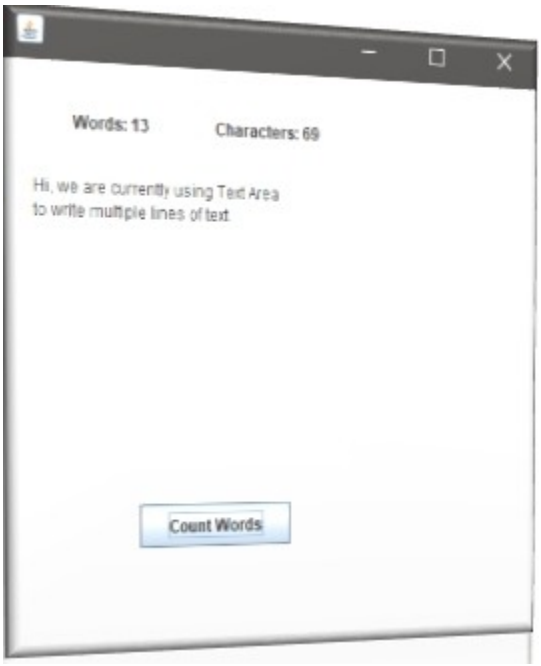
Illustration 10.11

```
/*
Simple program to find number of words and characters in a TextArea
*/


import javax.swing.*;‘‘
import java.awt.event.*;‘‘
public class TextAreaExample implements ActionListener{‘‘
JLabel l1,l2;‘‘
JTextArea area;‘‘
JButton b;‘‘
TextAreaExample() {‘‘
‘‘‘‘JFrame f= new JFrame();‘‘
‘‘‘‘l1=new JLabel();‘‘
‘‘‘‘l1.setBounds(50,25,100,30);‘‘
‘‘‘‘l2=new JLabel();‘‘
‘‘‘‘l2.setBounds(160,25,100,30);‘‘
‘‘‘‘area=new JTextArea();‘‘
‘‘‘‘area.setBounds(20,75,250,200);‘‘
‘‘‘‘b=new JButton("Count Words");‘‘
‘‘‘‘b.setBounds(100,300,120,30);‘‘
‘‘‘‘b.addActionListener(this);‘‘
```

```
‘‘‘‘f.add(l1);f.add(l2);f.add(area);f.add(b);
‘‘‘‘f.setSize(450,450);’‘
‘‘‘‘f.setLayout(null);’‘
‘‘‘‘f.setVisible(true);’‘
}’‘
public void actionPerformed(ActionEvent e){‘‘
‘‘‘‘String text=area.getText();’‘
‘‘‘‘String words[]=text.split("\\s");’‘
‘‘‘‘l1.setText("Words: "+words.length);’‘
‘‘‘‘l2.setText("Characters: "+text.length());’‘
}’‘
public static void main(String[] args) {‘‘
‘‘‘‘new TextAreaExample();’‘
}’‘
}’‘
```

Java JTextArea : An example with ActionListener:



Java JPasswordField: The object of a **JPasswordField class** is a text component specialized for password entry. It allows the editing of a single line of text. It inherits **JTextField class.**



The declaration for javax.swing.JPasswordField class is **public**&#65533;**class**&#65533;**JPasswordField**&#65533;**extends**&#65533;**JTextField.**
Class JPasswordField : Constructors:

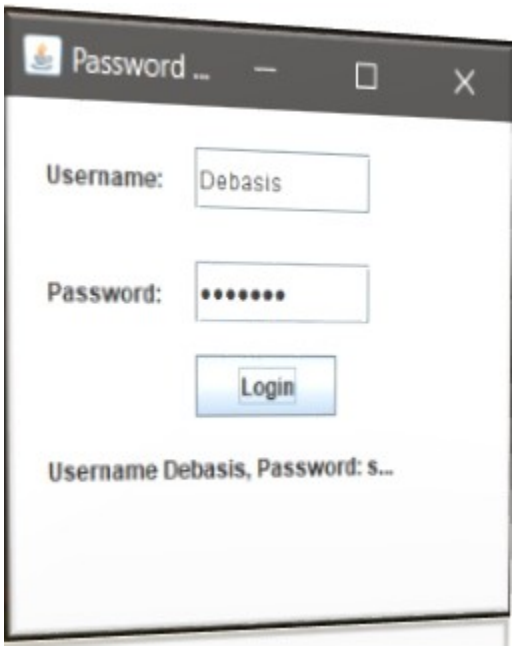
Constructor	Descripti
JPasswordField()	Constructs a new JPassword document, null starting text strin
JPasswordField(int columns)	Constructs a new empty JPas specified number o
JPasswordField(String text)	Constructs a new JPasswordFi specified te
JPasswordField(String text, int columns)	Construct a new JPasswordFie specified text and

Illustration 10.12

```
/*
Simple program to make a login GUI using TextField, PasswordField and Login Button
*/

import javax.swing.*;‘‘‘‘
import java.awt.event.*;‘‘
public class PasswordFieldExample {‘‘
‘‘public static void main(String[] args) {‘‘‘‘
‘‘‘‘JFrame f=new JFrame("Password Field Example");‘‘‘‘
‘‘‘‘final JLabel label = new JLabel();‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘label.setBounds(20,150, 200,50);‘‘
‘‘‘‘final JPasswordField value = new JPasswordField();‘‘
‘‘‘‘value.setBounds(100,75,100,30);‘‘
‘‘‘‘JLabel l1=new JLabel("Username:");‘‘‘‘
‘‘‘‘l1.setBounds(20,20, 80,30);‘‘‘‘
‘‘‘‘JLabel l2=new JLabel("Password:");‘‘‘‘
‘‘‘‘l2.setBounds(20,75, 80,30);‘‘‘‘
‘‘‘‘JButton b = new JButton("Login");‘‘
‘‘‘‘b.setBounds(100,120, 80,30);‘‘‘‘
‘‘‘‘final JTextField text = new JTextField();‘‘
‘‘‘‘text.setBounds(100,20, 100,30);‘‘‘‘
‘‘‘‘f.add(value);
‘‘‘‘f.add(l1);
‘‘‘‘f.add(label);
```

```
‘‘‘‘f.add(l2);
‘‘‘‘f.add(b);’‘‘‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘f.add(text);
‘‘ }’‘
‘‘‘‘f.setSize(300,300);’‘‘‘
‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘f.setVisible(true);’‘‘‘
‘‘‘‘b.addActionListener(new ActionListener() {‘‘
‘‘‘‘‘‘ public void actionPerformed(ActionEvent e) {‘‘‘‘‘‘
‘‘‘‘‘‘‘‘String data = "Username " + text.getText();’‘
‘‘‘‘‘‘‘‘data += ", Password: "+ new String(value.getPassword());’‘
‘‘‘‘‘‘‘‘label.setText(data);’‘‘‘‘‘‘‘‘‘
‘‘‘‘‘‘ }’‘
‘‘‘‘});’‘
‘‘ }’‘
}’‘
```



Java Swing JCheckBox: The JCheckBox class is used to create a checkbox. It is used to turn an option on (true) or off (false). Clicking on a JCheckBox changes its state from "on" to "off" or from "off" to "on ".It inherits JToggleButton class.



Below is the declaration for javax.swing.JCheckBox class. **public class JCheckBox extends JToggleButton implements Accessible**

Class JCheckBox : Constructors:

Constructor	*Description*
`JCheckBox()`	Creates an initially unselected check box button text, no icon.
`JChechBox(String s)`	Creates an initially unselected check box with te
`JCheckBox(String text, boolean selected)`	Creates a check box with text and specifies whet is initially selected.
`JCheckBox(Action a)`	Creates a check box where properties are taken f Action supplied.

Class JCheckBox : Methods:

Methods	*Descrip*
`AccessibleContext getAccessibleContext()`	It is used to get the Accessib with this JCheckBox.
`protected String paramString()`	It returns a string representa

Illustration 10.13

```
// A program to show use of CheckBox


import javax.swing.*;‘‘
public class CheckBoxExample’‘
{‘‘
‘‘‘‘ CheckBoxExample(){‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame("CheckBox Example");’‘
‘‘‘‘‘‘‘‘JCheckBox checkBox1 = new JCheckBox("C++");’‘
‘‘‘‘‘‘‘‘checkBox1.setBounds(100,100, 50,50);’‘
```

```
‘‘‘‘‘‘‘‘JCheckBox checkBox2 = new JCheckBox("Java", true);’’‘
‘‘‘‘‘‘‘‘checkBox2.setBounds(100,150, 50,50);’’‘
‘‘‘‘‘‘‘‘f.add(checkBox1);’’‘
‘‘‘‘‘‘‘‘f.add(checkBox2);’’‘
‘‘‘‘‘‘‘‘f.setSize(400,400);’’‘
‘‘‘‘‘‘‘‘f.setLayout(null);’’‘
‘‘‘‘‘‘‘‘f.setVisible(true);’’‘
‘‘‘‘ }’’‘
‘‘‘‘public static void main(String args[]){‘‘
‘‘‘‘‘‘new CheckBoxExample();’’‘
‘‘‘‘}
}’’‘
```



Java Swing JRadioButton: The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. It is widely used in exam systems or quiz. It should be added in ButtonGroup to select one radio button only.



Below is the declaration for javax.swing.JRadioButton class. **public**␣**class**␣**JRadioButton**␣**extends**␣**JToggleButton**␣**implements**␣**Accessible**␣␣

Class JRadioButton : Constructors:

Constructor	Description
JRadioButton()	Creates an unselected radio butt text.
JRadioButton(String s)	Creates an unselected radio butt specified text.
JRadioButton(String s, boolean selected)	Creates a radio button with the s text and selected status.

Class JRadioButton : Methods:

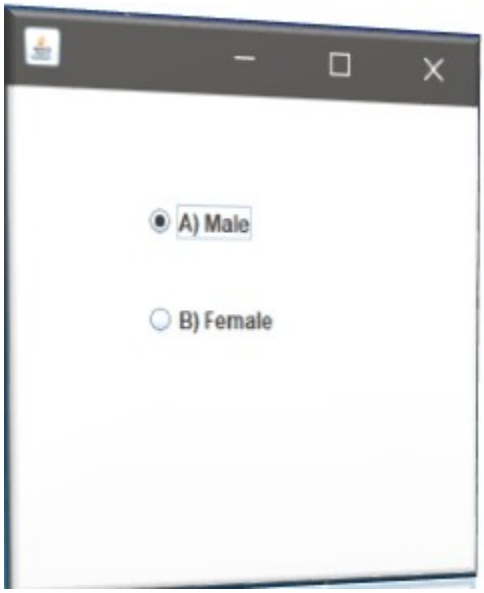
Methods	Description
void setText(String s)	It is used to set specified text on button.
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener(ActionListener a)	It is used to add the action listener to this object

Illustration 10.14

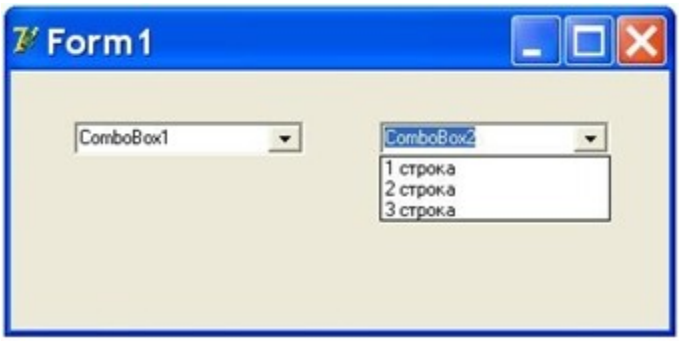
```
// A program to show use of RadioButton


import javax.swing.*;‘‘‘‘
public class RadioButtonExample {‘‘‘‘
JFrame f;‘‘‘‘
RadioButtonExample(){‘‘‘‘
f=new JFrame();‘‘‘‘
JRadioButton r1=new JRadioButton("A) Male");‘‘‘‘
```

```
JRadioButton r2=new JRadioButton("B) Female");‘‘‘‘
r1.setBounds(75,50,100,30);‘‘‘‘
r2.setBounds(75,100,100,30);‘‘‘‘
ButtonGroup bg=new ButtonGroup();‘‘‘‘
bg.add(r1);bg.add(r2);‘‘‘‘
f.add(r1);f.add(r2);‘‘‘‘‘‘
f.setSize(300,300);‘‘‘‘
f.setLayout(null);‘‘‘‘
f.setVisible(true);‘‘‘‘
}‘‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘new RadioButtonExample();‘‘‘‘
}}
```



Java Swing JComboBox: The object of Choice class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu. It inherits JComponent class.



Below is the declaration for javax.swing.JComboBox class. **public class JComboBox extends JComponent implements ItemSelectable, ListDataListener, ActionListener, Accessible**

Class JComboBox : Constructors:

Constructor	*Description*
`JComboBox()`	Creates a JComboBox with a default data
`JComboBox(Object[] items)`	Creates a JComboBox that contains the e in the specified array.
`JComboBox(Vector<?> items)`	Creates a JComboBox that contains the e in the specified Vector.

Class JComboBox : Methods:

Methods	*Descriptio*
`void addItem(Object anObject)`	It is used to add an item to the item list.
`void removeItem(Object anObject)`	It is used to delete an item to the item lis
`void removeAllItems()`	It is used to remove all the items from th
`void setEditable(boolean b)`	It is used to determine whether the JCon
`void addActionListener(ActionListener a)`	It is used to add the ActionListener.
`void addItemListener(ItemListener i)`	It is used to add the ItemListener.

Illustration 10.15

```
// A program to show use of ComboBox


import javax.swing.*;‘‘‘‘
public class ComboBoxExample {‘‘‘‘
JFrame f;‘‘‘‘
ComboBoxExample(){‘‘‘‘
‘‘‘‘f=new JFrame("ComboBox Example");‘‘‘‘
```

```
‘‘‘‘String country[]={"India","Aus","U.S.A","England","Newzealand"};’‘‘‘‘‘‘‘
‘‘‘‘JComboBox cb=new JComboBox(country);’‘‘‘
‘‘‘‘cb.setBounds(50, 50,90,20);’‘‘‘
‘‘‘‘f.add(cb);’‘‘‘‘‘‘‘
‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘f.setSize(400,500);’‘‘‘
‘‘‘‘f.setVisible(true);’‘‘‘‘‘‘‘
}’‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘new ComboBoxExample();’‘‘‘‘‘‘‘
}’‘‘‘
}
```



Java Swing JTable: The JTable class is used to display data in tabular form. It is composed of rows and columns



Class JTable : Constructors:

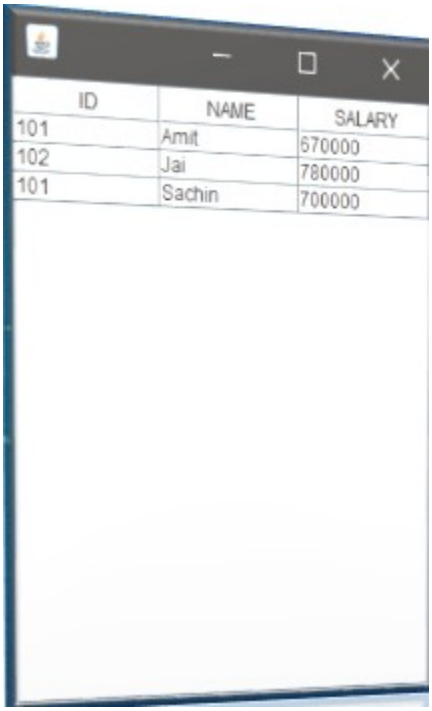
Constructor	*Descriptio*
`JTable()`	Creates a table with empty ce
`JTable(Object[][] rows, Object[] columns)`	Creates a table with the specif

Illustration 10.16

```
// A program to create a table and show some data


import javax.swing.*;‘‘‘‘
public class TableExample {‘‘‘‘
‘‘‘‘JFrame f;‘‘‘‘
‘‘‘‘TableExample(){‘‘‘‘
‘‘‘‘f=new JFrame();‘‘‘‘
‘‘‘‘String data[][]={ {"101","Amit","670000"},‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘{"102","Jai","780000"},‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘{"101","Sachin","700000"}};‘‘‘‘
‘‘‘‘String column[]={"ID","NAME","SALARY"};‘‘‘‘‘‘‘
‘‘‘‘JTable jt=new JTable(data,column);‘‘‘‘
‘‘‘‘jt.setBounds(30,40,200,300);‘‘‘‘‘‘‘‘‘
‘‘‘‘JScrollPane sp=new JScrollPane(jt);‘‘‘‘
‘‘‘‘f.add(sp);‘‘‘‘‘‘‘‘‘
‘‘‘‘f.setSize(300,400);‘‘‘‘
‘‘‘‘f.setVisible(true);‘‘‘‘
}‘‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘new TableExample();‘‘‘‘
}‘‘‘‘
```

```
}’‘
```



Java Swing JList: The object of JList class represents a list of text items. The list of text items can be set up so that the user can choose either one item or multiple items. It inherits JComponent class.



Below is the declaration for javax.swing.JList class. **public****class****JList****extends****JComponent****implements****Scrollable,Accessible**

Class JList : Constructors:

Constructor	Description
`JList()`	Creates a JList with an empty, read-only,
`JList(ary[] listData)`	Creates a JList that displays the elements specified array.
`JList(ListModel<ary> dataModel)`	Creates a JList that displays elements fro specified, non-null, model.

Class JList : Methods:

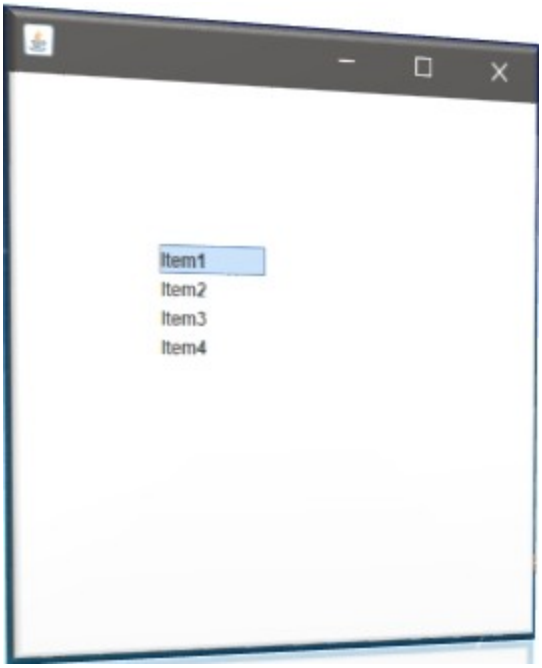
Methods	D
`Void addListSelectionListener(ListSelectionListener listener)`	It is used to ad be notified eac selection occu
`int getSelectedIndex()`	It is used to re selected cell in
`ListModel getModel()`	It is used to re holds a list of JList compon
`void setListData(Object[] listData)`	It is used to cr ListModel fro

Illustration 10.17

```
// A program to create a list and list some data


import javax.swing.*;‘‘
public class ListExample’‘
{‘‘
‘‘‘‘ ListExample(){‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame();’‘
‘‘‘‘‘‘‘‘DefaultListModel l1 = new DefaultListModel<>();’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item1");’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item2");’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item3");’‘
‘‘‘‘‘‘‘‘‘‘l1.addElement("Item4");’‘
‘‘‘‘‘‘‘‘‘‘JList list = new JList<>(l1);’‘
```

```
‘‘‘‘‘‘‘‘‘list.setBounds(100,100, 75,75);’‘
‘‘‘‘‘‘‘‘‘f.add(list);’‘
‘‘‘‘‘‘‘‘‘f.setSize(400,400);’‘
‘‘‘‘‘‘‘‘‘f.setLayout(null);’‘
‘‘‘‘‘‘‘‘‘f.setVisible(true);’‘
‘‘‘‘ }’‘
public static void main(String args[])’‘
‘‘‘‘{‘‘
‘‘ new ListExample();’‘
‘‘‘‘}}’‘
```



Java Swing JOptionPane: The JOptionPane class is used to provide standard dialog boxes such as message dialog box, confirm dialog box and input dialog box. These dialog boxes are used to display information or get input from the user. The JOptionPane class inherits JComponent class.



Below is the declaration for javax.swing.JOptainPane class. **public�class JOptionPane�extends�JComponent�implements�Accessible���**

Class JOptionPane : Constructors:

Constructor	Description
JOptionPane()	It is used to create a JOptionPane message.
JOptionPane(Object message)	It is used to create an instance of display a message.
JOptionPane(Object message, int messageType)	It is used to create an instance of display a message with specified and default options.

Class JOptionPane : Methods:

Methods	Descripti
JDialog createDialog(String title)	It is used to create and return a new parentle title.
static void showMessageDialog(Component parentComponent, Object message)	It is used to create an information-message
static void showMessageDialog(Component parentComponent, Object message, String title, int messageType)	It is used to create a message dialog with giv
static int showConfirmDialog(Component parentComponent, Object message)	It is used to create a dialog with the options title, Select an Option.
static String showInputDialog(Component parentComponent, Object message)	It is used to show a question-message dialog parented to parentComponent.
void setInputValue(Object newValue)	It is used to set the input value that was sele

Illustration 10.18

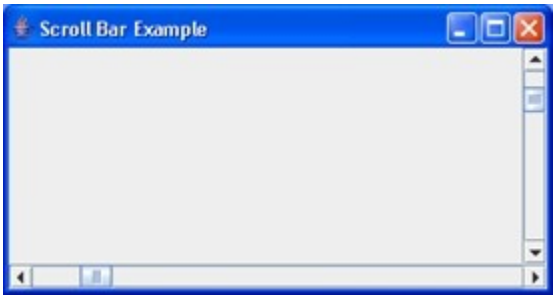
```
// A program to show some info to using JOptionPane


import javax.swing.*;‘‘
public class OptionPaneExample {‘‘
JFrame f;‘‘
OptionPaneExample(){‘‘
‘‘‘‘f=new JFrame();‘‘
‘‘‘‘JOptionPane.showMessageDialog(f,"Hello, Welcome to IIT Kharagpur.");‘‘
}‘‘
public static void main(String[] args) {‘‘
‘‘‘‘new OptionPaneExample();‘‘
```

```
}’‘
}’‘
```



Java Swing JScrollBar: The object of JScrollBar class is used to add horizontal and vertical scrollbar. It is an implementation of a scrollbar. It inherits JComponent class.



Below is the declaration for javax.swing.JScrollBar class. **public class JScrollBar extends JComponent implements Adjustable, Accessible**

Class JScrollBar: Constructors:

Constructor	Description
JScrollBar()	Creates a vertical scrollbar with the initial
JScrollBar(int orientation)	Creates a scrollbar with the specified orie and the initial values.
JScrollBar(int orientation, int value, int extent, int min, int max)	Creates a scrollbar with the specified orie value, extent, minimum, and maximum.

Illustration 10.19

```
import javax.swing.*;’‘
class ScrollBarExample’‘
{‘‘
ScrollBarExample(){‘‘
‘‘‘‘JFrame f= new JFrame("Scrollbar Example");’‘
```

```
 JScrollBar s=new JScrollBar();’‘
s.setBounds(100,100, 50,100);’‘
f.add(s);’‘
f.setSize(400,400);’‘
f.setLayout(null);’‘
f.setVisible(true);’‘
}’‘
public static void main(String args[])’‘
{‘‘
new ScrollBarExample();’‘
}}’‘
```

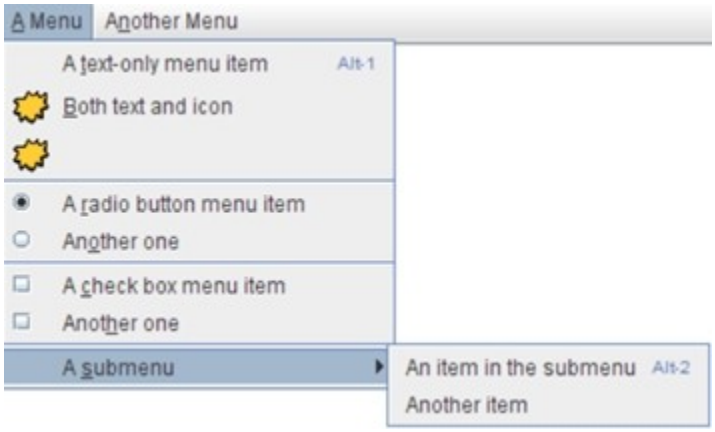


Java Swing JMenuBar: The JMenuBar class is used to display menu bar on the window or frame. It may have several menus.



Below is the declaration for javax.swing.JMenuBar class. **public class JMenuBar extends JComponent implements MenuElement, Accessible**

Java Swing JMenu: The object of JMenu class is a pull down menu component which is displayed from the menu bar. It inherits the JMenuItem class.



Below is the declaration for javax.swing.JMenuItem
class **JMenu**&#65533;**extends**&#65533;**JMenuItem**&#65533;**implements**&#65533;**MenuElement,**&#65533;**Accessible**

Class JMenuItem: The object of JMenuItem class adds a simple labeled menu item. The items used in a menu must belong to the JMenuItem or any of its subclass.

Below is the declaration for javax.swing.JMenuItem
class. **public**&#65533;**class**&#65533;**JMenuItem**&#65533;**extends**&#65533;**AbstractButton**&#65533;**implements**&#65533;**Accessible,**&#65533;**MenuElement**&#65533;&#65533;

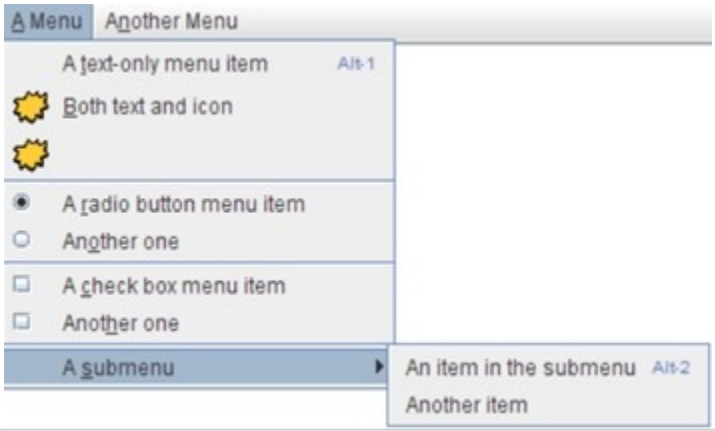
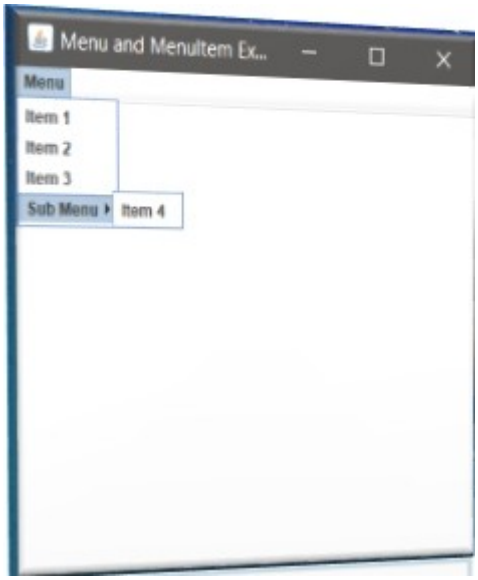


Illustration 10.20

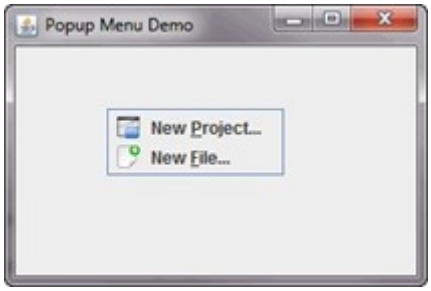
```
// A program to create a menu bar and show menu items


import javax.swing.*;‘‘
class MenuExample{‘‘
‘‘‘‘JMenu menu, submenu;’’
‘‘‘‘JMenuItem i1, i2, i3, i4, i5;’’
‘‘‘‘MenuExample(){‘‘
‘‘‘‘‘‘ JFrame f= new JFrame("Menu and MenuItem Example");’’
‘‘‘‘‘‘ JMenuBar mb=new JMenuBar();’’
‘‘‘‘‘‘ menu=new JMenu("Menu");’’
‘‘‘‘‘‘ submenu=new JMenu("Sub Menu");’’
‘‘‘‘‘‘ i1=new JMenuItem("Item 1");i2=new JMenuItem("Item 2");’’
‘‘‘‘‘‘ i3=new JMenuItem("Item 3");i4=new JMenuItem("Item 4");’’
‘‘‘‘‘‘ menu.add(i1); menu.add(i2); menu.add(i3);’’
```

```
‘‘‘‘‘‘ submenu.add(i4);
‘‘‘‘‘‘ menu.add(submenu);’‘‘‘‘‘‘‘ mb.add(menu);’‘
‘‘‘‘‘‘ f.setJMenuBar(mb);’‘
‘‘‘‘‘‘ f.setSize(400,400);’‘‘‘‘‘‘‘f.setLayout(null);’‘
‘‘‘‘‘‘ f.setVisible(true);’‘
          }
          public static void main(String args[]){‘‘
                    new MenuExample();’‘
          }
}
```



Java Swing JPopupMenu: JPopupMenu can be dynamically popped up at specific position within a component. It inherits the JComponent class.



Below is the declaration for javax.swing.JPopupMenu class. **public‘class‘JPopupMenu‘extends‘JComponent‘implements‘Accessible,‘MenuElement‘‘**

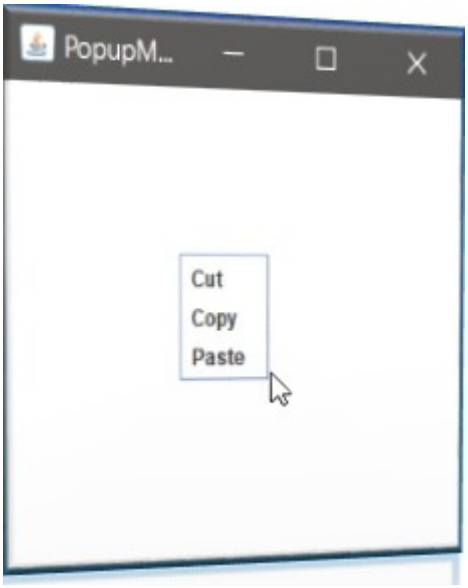
Class JPopupMenu : Constructors:

Constructor	Description
JPopupMenu()	Constructs a JPopupMenu without a "invoker".
JPopupMenu(String label)	Constructs a JPopupMenu with the specified title.

Illustration 10.21

```
// A program to show some menu using PopupMenu


import javax.swing.*;‘‘
import java.awt.event.*;‘‘
class PopupMenuExample {‘‘
‘‘‘‘ PopupMenuExample(){‘‘
‘‘‘‘‘‘‘‘ final JFrame f= new JFrame("PopupMenu Example");‘‘
‘‘‘‘‘‘‘‘ final JPopupMenu popupmenu = new JPopupMenu("Edit");‘‘
‘‘‘‘‘‘‘‘ JMenuItem cut = new JMenuItem("Cut");‘‘
‘‘‘‘‘‘‘‘ JMenuItem copy = new JMenuItem("Copy");‘‘
‘‘‘‘‘‘‘‘ JMenuItem paste = new JMenuItem("Paste");‘‘
‘‘‘‘‘‘‘‘ popupmenu.add(cut); popupmenu.add(copy); popupmenu.add(paste);‘‘‘‘‘‘‘‘
‘‘‘‘‘‘‘‘ f.addMouseListener(new MouseAdapter() {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘public void mouseClicked(MouseEvent e) {‘‘‘‘‘‘‘‘‘‘‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘popupmenu.show(f , e.getX(), e.getY());‘‘
‘‘‘‘‘‘‘‘‘‘‘‘} });‘‘
‘‘‘‘‘‘‘‘ f.add(popupmenu);‘‘
‘‘‘‘‘‘‘‘ f.setSize(300,300);‘‘
‘‘‘‘‘‘‘‘ f.setLayout(null);‘‘
‘‘‘‘‘‘‘‘ f.setVisible(true);‘‘
‘‘‘‘ }‘‘
public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘new PopupMenuExample();‘‘
}}‘‘
```



Java Swing JCheckBoxMenuItem: JCheckBoxMenuItem class represents checkbox which can be included on a menu. A JCheckBoxMenuItem can have text or a graphic icon or both, associated with it. MenuItem can be selected or deselected. MenuItems can be configured and controlled by actions.

Modifier and Type	*Class*	*Description*
protected class	JCheckBoxMenuItem.AccessibleJCheckBoxMenuItem	This class implements acce support for the JcheckBoxM class.

Class JCheckBoxMenuItem: Constructors:

Constructor	Description
`JCheckBoxMenuItem()`	It creates an initially unselected check box me icon.
`JCheckBoxMenuItem(Action a)`	It creates a menu item whose properties are ta supplied.
`JCheckBoxMenuItem(Icon icon)`	It creates an initially unselected check box me
`JCheckBoxMenuItem(String text)`	It creates an initially unselected check box me
`JCheckBoxMenuItem(String text, boolean b)`	It creates a check box menu item with the spe
`JCheckBoxMenuItem(String text, Icon icon)`	It creates an initially unselected check box me text and icon.
`JCheckBoxMenuItem(String text, Icon icon, boolean b)`	It creates a check box menu item with the spe state.

Class JCheckBoxMenuItem : Methods:

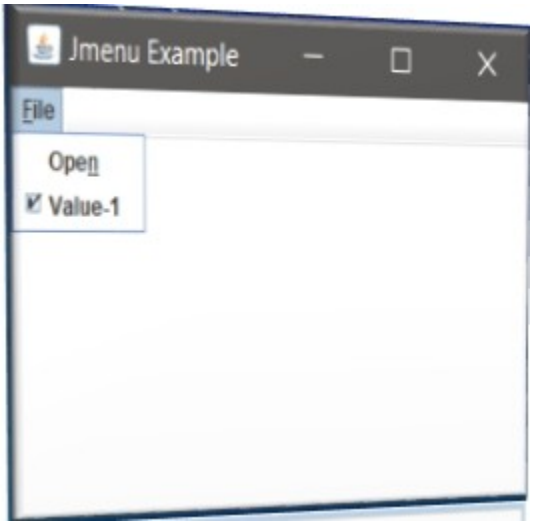
Modifier	Method	Description
AccessibleContext	`getAccessibleContext()`	It gets the AccessibleContext associated with this J
Object[]	`getSelectedObjects()`	It returns an array (length 1) containing the check b the check box is not selected.
boolean	`getState()`	It returns the selected-state of the item.
String	`getUIClassID()`	It returns the name of the L&F class that renders thi
protected String	`paramString()`	It returns a string representation of this JCheckBoxI
void	`setState(boolean b)`	It sets the selected-state of the item.

Illustration 10.22

```
// A program to show check box menu item in menu


import java.awt.event.ActionEvent;’‘
import java.awt.event.ActionListener;’‘
import java.awt.event.KeyEvent;’‘
import javax.swing.AbstractButton;’‘
import javax.swing.Icon;’‘
```

```
import javax.swing.JCheckBoxMenuItem;‘‘
import javax.swing.JFrame;‘‘
import javax.swing.JMenu;‘‘
import javax.swing.JMenuBar;‘‘
import javax.swing.JMenuItem;‘‘
public class JavaCheckBoxMenuItemExample {‘‘
‘‘‘‘public static void main(final String args[]) {‘‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("Jmenu Example");‘‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘‘‘‘‘JMenuBar menuBar = new JMenuBar();
‘‘‘‘‘‘‘‘JMenu fileMenu = new JMenu("File");‘‘
‘‘‘‘‘‘‘‘fileMenu.setMnemonic(KeyEvent.VK_F);‘‘
‘‘‘‘‘‘‘‘menuBar.add(fileMenu);‘‘
‘‘‘‘‘‘‘‘JMenuItem menuItem1 = new JMenuItem("Open", KeyEvent.VK_N);‘‘
‘‘‘‘‘‘‘‘fileMenu.add(menuItem1);‘‘
‘‘‘‘‘‘‘‘JCheckBoxMenuItem caseMenuItem = new JCheckBoxMenuItem("Option_1");‘‘
‘‘‘‘‘‘‘‘caseMenuItem.setMnemonic(KeyEvent.VK_C);
‘‘‘‘‘‘‘‘fileMenu.add(caseMenuItem);‘‘
‘‘‘‘‘‘‘‘ActionListener aListener = new ActionListener() {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘public void actionPerformed(ActionEvent event) {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘AbstractButton aButton = (AbstractButton) event.getSource();‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘boolean selected = aButton.getModel().isSelected();‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘String newLabel;‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘Icon newIcon;‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘if (selected) {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘newLabel = "Value-1";‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘} else {‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘newLabel = "Value-2";‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘aButton.setText(newLabel);‘‘
‘‘‘‘‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘};
‘‘‘‘‘‘‘‘caseMenuItem.addActionListener(aListener);‘‘
‘‘‘‘‘‘‘‘frame.setJMenuBar(menuBar);‘‘
‘‘‘‘‘‘‘‘frame.setSize(350, 250);‘‘
‘‘‘‘‘‘‘‘frame.setVisible(true);‘‘
‘‘‘‘}‘‘
}‘‘
```



Java Swing JSeparator: The object of JSeparator class is used to provide a general purpose component for implementing divider lines. It is used to draw a line to separate widgets in a Layout. It inherits JComponent class.

Below is the declaration for javax.swing.JSeparator
class **public class JSeparator extends JComponent implements SwingConstants, Accessible**

Class JSeparator : Constructors:

Constructor	Description
JSeparator()	Creates a new horizontal separator.
JSeparator(int orientation)	Creates a new separator with the spe horizontal or vertical orientation.

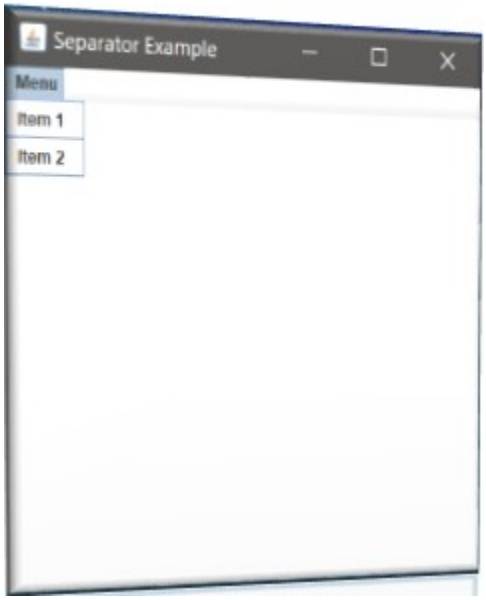
Class JSeparator : Methods:

Methods	Description
void setOrientation(int orientation)	It is used to set the orientation of the separator.
int getOrientation()	It is used to return the orientation of the separato

Illustration 10.23

```
import javax.swing.*;''''
class SeparatorExample{''''
''JMenu menu, submenu;''''
```

```
‘‘JMenuItem i1, i2, i3, i4, i5;’‘‘‘
‘‘SeparatorExample()’‘{‘‘‘‘
‘‘‘‘‘‘‘‘JFrame f = new JFrame("Separator Example");’‘‘‘
‘‘‘‘‘‘‘‘JMenuBar mb = new JMenuBar();’‘‘‘
‘‘‘‘‘‘‘‘menu = new JMenu("Menu");’‘‘‘
‘‘‘‘‘‘‘‘i1 = new JMenuItem("Item 1");’‘‘‘
‘‘‘‘‘‘‘‘i2 = new JMenuItem("Item 2");’‘‘‘‘‘
‘‘‘‘‘‘‘‘menu.add(i1);’‘
‘‘‘‘‘‘‘‘menu.addSeparator();’‘
‘‘‘‘‘‘‘‘menu.add(i2);’‘
‘‘‘‘‘‘‘‘mb.add(menu);’‘‘‘
‘‘‘‘‘‘‘‘f.setJMenuBar(mb);’‘‘‘
‘‘‘‘‘‘‘‘f.setSize(400,400);’‘‘‘
‘‘‘‘‘‘‘‘f.setLayout(null);’‘‘‘
‘‘‘‘‘‘‘‘f.setVisible(true);’‘‘‘
‘‘ }’‘‘‘
‘‘‘‘public static void main(String args[]){‘‘‘‘
‘‘‘‘‘‘‘‘new SeparatorExample();’‘‘‘
}}
```



Java Swing JProgressBar: The JProgressBar class is used to display the progress of the task. It inherits JComponent class.



Below is the declaration for javax.swing.JProgressBar class. **public class JProgressBar extends JComponent implements SwingConstants, Accessible**

Class JProgressBar : Constructors:

Constructor	Description
`JProgressBar()`	It is used to create a horizontal progress b
`JProgressBar(int min, int max)`	It is used to create a horizontal progress b minimum and maximum value.
`JProgressBar(int orient)`	It is used to create a progress bar with the can be either Vertical or Horizontal by usi SwingConstants.VERTICAL and SwingConstants.HORIZONTAL constants
`JProgressBar(int orient, int min, int max)`	It is used to create a progress bar with the minimum and maximum value.

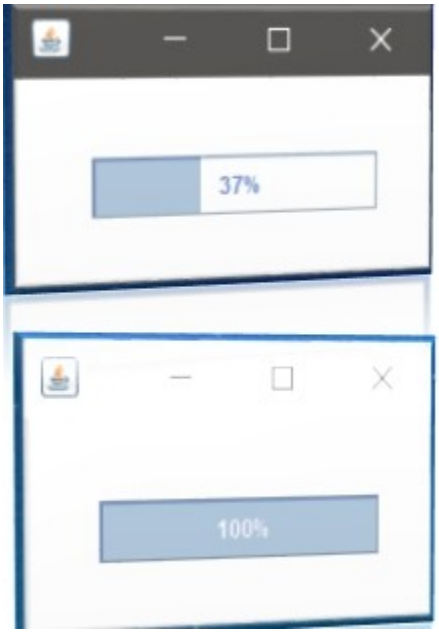
Class JProgressBar : Methods:

Method	Description
`void setStringPainted(boolean b)`	It is used to determine whether strin displayed.
`void setString(String s)`	It is used to set value to the progres
`void setOrientation(int orientation)`	It is used to set the orientation, it ma horizontal by using SwingConstants SwingConstants.HORIZONTAL co
`void setValue(int value)`	It is used to set the current value on

Illustration 10.24

```
// A program to show progress bar
```

```
import javax.swing.*;‘‘‘‘
public class ProgressBarExample extends JFrame{
‘‘JProgressBar jb; int i=0,num=0;‘‘‘‘
‘‘ProgressBarExample(){‘‘‘‘
‘‘‘‘jb=new JProgressBar(0,2000);‘‘‘‘
‘‘‘‘jb.setBounds(40,40,160,30);‘‘‘‘‘‘‘
‘‘‘‘jb.setValue(0);‘‘‘‘
‘‘‘‘jb.setStringPainted(true);‘‘‘‘
‘‘‘‘add(jb);‘‘‘‘
‘‘‘‘setSize(250,150);‘‘ setLayout(null);‘‘‘‘
‘‘}‘‘‘‘
‘‘public void iterate(){‘‘‘‘
‘‘‘‘while(i<=2000){‘‘‘‘
‘‘‘‘‘‘jb.setValue(i);‘‘‘‘
‘‘‘‘‘‘i=i+20;‘‘‘‘
‘‘‘‘‘‘try{Thread.sleep(150);}catch(Exception e){}‘‘‘‘}‘‘‘‘
‘‘}‘‘‘‘
‘‘public static void main(String[] args) {‘‘‘‘
‘‘‘‘‘‘ProgressBarExample m=new ProgressBarExample();‘‘‘‘
‘‘‘‘‘‘m.setVisible(true);‘‘‘‘
‘‘‘‘‘‘m.iterate();‘‘‘‘
‘‘}‘‘‘‘
}‘‘
```



Java Swing JTree: The JTree class is used to display the tree structured data or hierarchical data. JTree is a complex component. It has a 'root node' at the top most which is a parent for all nodes in the tree. It inherits JComponent class.



Below is the declaration for javax.swing.JTree class. **public class JTree extends JComponent implements Scrollable, Accessible**

Class JTree : Constructors:

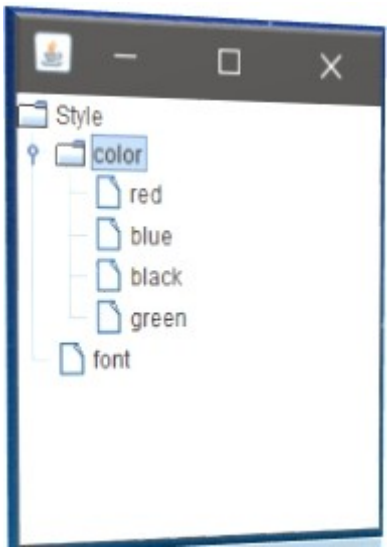
Constructor	Description
JTree()	Creates a JTree with a sample model.
JTree(Object[] value)	Creates a JTree with every element of the specified array as the child of a new root n
JTree(TreeNode root)	Creates a JTree with the specified TreeNod root, which displays the root node.

Illustration 10.25

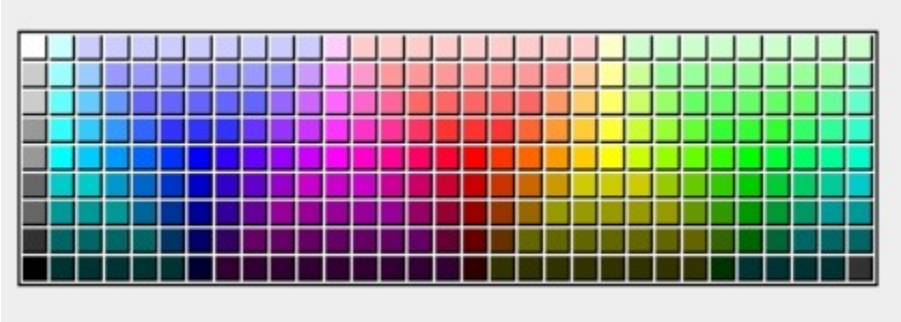
```
// A program to show Tree structured data arrangement


import javax.swing.*;‘‘
import javax.swing.tree.DefaultMutableTreeNode;’‘
public class TreeExample {‘‘
JFrame f;’‘
TreeExample(){‘‘
‘‘‘‘f=new JFrame();’‘
‘‘‘‘DefaultMutableTreeNode style=new DefaultMutableTreeNode("Style");’‘
‘‘‘‘DefaultMutableTreeNode color=new DefaultMutableTreeNode("color");’‘
‘‘‘‘DefaultMutableTreeNode font=new DefaultMutableTreeNode("font");’‘
‘‘‘‘style.add(color);’‘‘‘ style.add(font);’‘
‘‘‘‘DefaultMutableTreeNode red=new DefaultMutableTreeNode("red");’‘
‘‘‘‘DefaultMutableTreeNode blue=new DefaultMutableTreeNode("blue");’‘
‘‘‘‘DefaultMutableTreeNode black=new DefaultMutableTreeNode("black");’‘
‘‘‘‘DefaultMutableTreeNode green=new DefaultMutableTreeNode("green");’‘
‘‘‘‘color.add(red); color.add(blue); color.add(black); color.add(green);’‘‘‘‘‘
```

```
‘‘‘‘JTree jt=new JTree(style);’‘‘‘ f.add(jt);’‘
‘‘‘‘f.setSize(200,200);’‘
‘‘‘‘f.setVisible(true);’‘
}’‘
public static void main(String[] args) {‘‘
‘‘‘‘new TreeExample();’‘
}}’‘
```



Java Swing JColorChooser: The JColorChooser class is used to create a color chooser dialog box so that user can select any color. It inherits JComponent class.



Below is the declaration for javax.swing.JColorChooser class. **public**`&#65533;`**class JColorChooser**`&#65533;`**extends**`&#65533;`**JToggleButton**`&#65533;`**implements**`&#65533;`**Accessible**`&#65533;&#65533;&#65533;`

Class JColorChooser: Constructors:

Constructor	Description
JColorChooser()	It is used to create a color chooser panel with white color initially.
JColorChooser(color initialcolor)	It is used to create a color chooser panel with the specified color initially.

Class JColorChooser : Methods:

Method	
void addChooserPanel(AbstractColorChooserPanel panel)	It is use panel t
static Color showDialog(Component c, String title, Color initialColor)	It is use dialog

Illustration 10.26

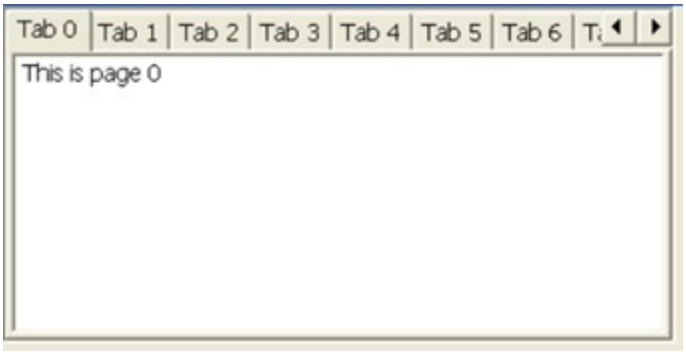
```
// A program to show choose colour using JColorChooser


import java.awt.event.*;‘‘‘‘
import java.awt.*;‘‘‘‘
import javax.swing.*;‘‘‘‘
public class ColorChooserExample extends JFrame implements ActionListener {‘‘‘‘
JButton b;Container c;‘‘‘‘
ColorChooserExample(){‘‘‘‘
‘‘‘‘c=getContentPane();‘‘‘‘
‘‘‘‘c.setLayout(new FlowLayout());‘‘‘‘‘‘‘‘
‘‘‘‘b=new JButton("color");‘‘‘‘
‘‘‘‘b.addActionListener(this);‘‘‘‘‘‘‘‘
‘‘‘‘c.add(b);‘‘ }‘‘‘‘
public void actionPerformed(ActionEvent e) {‘‘‘‘
Color initialcolor=Color.RED;‘‘‘‘
Color color=JColorChooser.showDialog(this,"Select a color",initialcolor);‘‘‘‘
c.setBackground(color);‘‘‘‘
}‘‘‘‘
public static void main(String[] args) {‘‘‘‘
‘‘‘‘ColorChooserExample ch=new ColorChooserExample();‘‘‘‘
‘‘‘‘ch.setSize(400,400);‘‘‘‘
‘‘‘‘ch.setVisible(true);‘‘‘‘
```

```
‘‘‘‘ch.setDefaultCloseOperation(EXIT_ON_CLOSE);’‘‘‘
}’‘‘‘
}
```



Java Swing JTabbedPane: The JTabbedPane class is used to switch between a group of components by clicking on a tab with a given title or icon. It inherits JComponent class.



Below is the declaration for javax.swing.JTabbedPane class. **public�class�JTabbedPane�extends�JComponent�implements�Serializable,�Accessible,�SwingConstants.��**

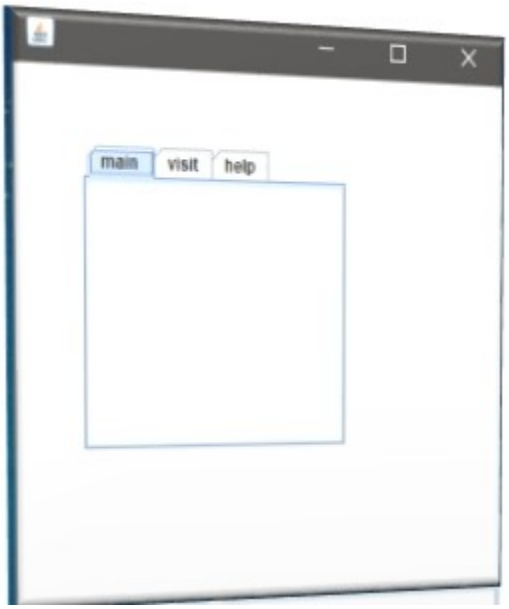
Class JTabbedPane : Constructors:

Constructor	Description
JTabbedPane()	Creates an empty TabbedPane with a defa placement of JTabbedPane.Top.
JTabbedPane(int tabPlacement)	Creates an empty TabbedPane with a spec tab placement.
JTabbedPane(int tabPlacement, int tabLayoutPolicy)	Creates an empty TabbedPane with a spec tab placement and tab layout policy.

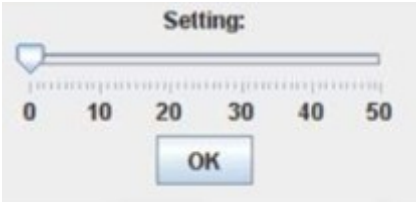
Illustration 10.27

```
// A program to show multiple tabs using JTabbedPane


import javax.swing.*;‘‘
public class TabbedPaneExample {‘‘
JFrame f;‘‘
TabbedPaneExample(){‘‘
‘‘‘‘f=new JFrame();‘‘
‘‘‘‘JTextArea ta=new JTextArea(200,200);‘‘
‘‘‘‘JPanel p1=new JPanel();‘‘
‘‘‘‘p1.add(ta);‘‘
‘‘‘‘JPanel p2=new JPanel();‘‘
‘‘‘‘JPanel p3=new JPanel();‘‘
‘‘‘‘JTabbedPane tp=new JTabbedPane();‘‘
‘‘‘‘tp.setBounds(50,50,200,200);‘‘
‘‘‘‘tp.add("main",p1);‘‘
‘‘‘‘tp.add("visit",p2);‘‘
‘‘‘‘tp.add("help",p3);‘‘‘‘
‘‘‘‘f.add(tp);‘‘
‘‘‘‘f.setSize(400,400);‘‘
‘‘‘‘f.setLayout(null);‘‘
‘‘‘‘f.setVisible(true); }‘‘
public static void main(String[] args) {‘‘
‘‘‘‘new TabbedPaneExample();‘‘
}}‘‘
```



Java Swing JSlider: The Java JSlider class is used to create the slider. By using JSlider, a user can select a value from a specific range.



Class JSlider : Constructors:

Constructor	*Description*
`JSlider()`	creates a slider with the initial value of 50 and range of
`JSlider(int orientation)`	creates a slider with the specified orientation set by eith JSlider.VERTICAL with the range 0 to 100 and initial
`JSlider(int min, int max)`	creates a horizontal slider using the given min and max
`JSlider(int min, int max, int value)`	creates a horizontal slider using the given min, max and
`JSlider(int orientation, int min, int max, int value)`	creates a slider using the given orientation, min, max a

Class JSlider : Methods:

Method	Description
public void setMinorTickSpacing(int n)	is used to set the minor tick spacing to the
public void setMajorTickSpacing(int n)	is used to set the major tick spacing to the
public void setPaintTicks(boolean b)	is used to determine whether tick marks a
public void setPaintLabels(boolean b)	is used to determine whether labels are pa
public void setPaintTracks(boolean b)	is used to determine whether track is paint

Illustration 10.28

```
// A program to implement slider input using JSlider


import javax.swing.*;‘‘
public class SliderExample1 extends JFrame{‘‘
public SliderExample1() {‘‘
          JSlider slider = new JSlider(JSlider.HORIZONTAL, 0, 50, 25);‘‘
          JPanel panel=new JPanel();‘‘
          panel.add(slider);‘‘
          add(panel);‘‘
          }‘‘
public static void main(String s[]) {‘‘
          SliderExample1 frame=new SliderExample1();‘‘
          frame.pack();‘‘
          frame.setVisible(true);‘‘
}‘‘
}‘‘
```



Java Swing JSpinner: The object of JSpinner class is a single line input field that allows the user to select a number or an object value from an ordered sequence.



Below is the declaration for javax.swing.JSpinner class. **public class JSpinner extends JComponent implements Accessible.**

Class JSpinner : Constructors:

Constructor	Description
`JSpinner()`	It is used to construct a spinner with an I SpinnerNumberModel with initial value minimum or maximum limits.
`JSpinner(SpinnerModel model)`	It is used to construct a spinner for a give

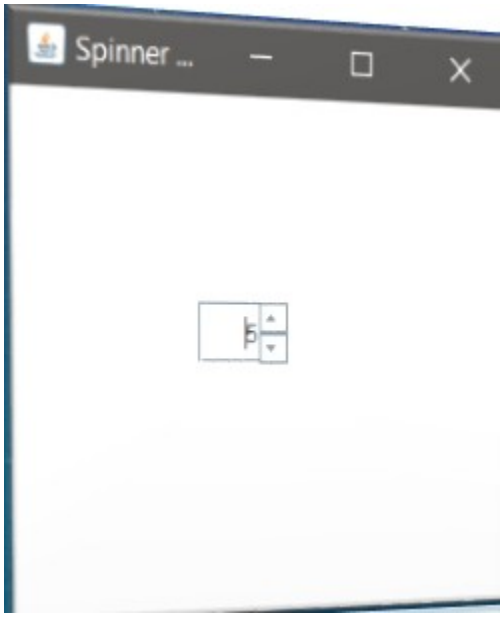
Class JSpinner : Methods:

Method	Desc
`void addChangeListener(ChangeListener listener)`	It is used to add a liste notified each time a cl occurs.
`Object getValue()`	It is used to return the model.

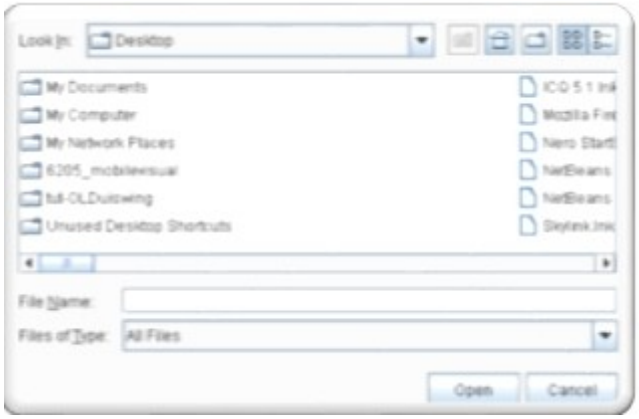
Illustration 10.29

```
import javax.swing.*;‘‘‘‘
‘‘public class SpinnerExample {‘‘
‘‘‘‘public static void main(String[] args) {‘‘‘‘
‘‘‘‘JFrame f=new JFrame("Spinner Example");‘‘‘‘
‘‘‘‘SpinnerModel value =‘‘
‘‘‘‘‘‘‘‘‘‘‘‘ new SpinnerNumberModel(5, //initial value’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘0, //minimum value’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘10, //maximum value’‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘1); //step’‘
‘‘‘‘JSpinner spinner = new JSpinner(value);’‘
‘‘‘‘‘‘‘‘‘‘‘‘spinner.setBounds(100,100,50,30);’‘‘‘
```

```
‘‘‘‘‘‘‘‘‘‘‘‘f.add(spinner);’’’’
‘‘‘‘‘‘‘‘‘‘‘‘f.setSize(300,300);’’’’
‘‘‘‘‘‘‘‘‘‘‘‘f.setLayout(null);’’’’
‘‘‘‘‘‘‘‘‘‘‘‘f.setVisible(true);’’’’
‘‘}’’
}
```



Java Swing JFileChooser: The object of JFileChooser class represents a dialog window from which the user can select file. It inherits JComponent class.



Below is the declaration for javax.swing.JFileChooser class. **public class JFileChooser extends JComponent implements Accessible.**

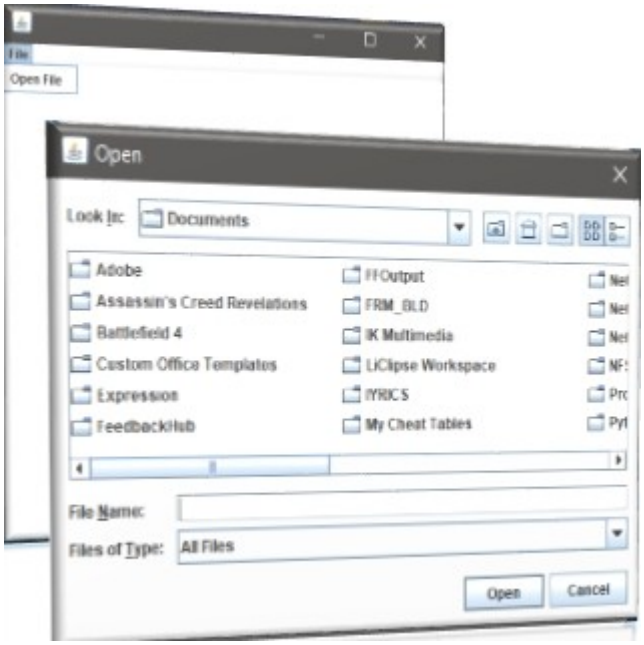
Class JFileChooser : Constructors:

Constructor	Description
JFileChooser()	Constructs a JFileChooser pointing to the user's directory.
JFileChooser(File currentDirectory)	Constructs a JFileChooser using the given File a
JFileChooser(String currentDirectoryPath)	Constructs a JFileChooser using the given path.

Illustration 10.30

```
import javax.swing.*;‘‘‘‘
import java.awt.event.*;‘‘‘‘
import java.io.*;‘‘‘‘
public class FileChooserExample extends JFrame implements ActionListener{‘‘‘‘
          JMenuBar mb;‘‘‘‘
          JMenu file;‘‘‘‘
          JMenuItem open;‘‘‘‘
          JTextArea ta;‘‘‘‘
          FileChooserExample(){‘‘‘‘
          open=new JMenuItem("Open File");‘‘‘‘
          open.addActionListener(this);‘‘‘‘‘‘‘‘‘‘‘‘
          file=new JMenu("File");‘‘‘‘
          file.add(open);‘‘‘‘‘‘‘‘‘‘‘‘
          mb=new JMenuBar();‘‘‘‘
          mb.setBounds(0,0,800,20);‘‘‘‘
          mb.add(file);‘‘‘‘‘‘‘‘‘‘‘‘‘‘
          ta=new JTextArea(800,800);‘‘‘‘
          ta.setBounds(0,20,800,800);‘‘‘‘‘‘‘‘‘‘‘‘‘‘
          add(mb);‘‘‘‘
          add(ta);}‘‘‘‘
          public void actionPerformed(ActionEvent e) {‘‘‘‘
          if(e.getSource()==open){
           JFileChooser fc=new JFileChooser();‘‘‘‘
                    int i=fc.showOpenDialog(this);‘‘‘‘
                    if(i==JFileChooser.APPROVE_OPTION){‘‘‘‘
                              File f=fc.getSelectedFile();‘‘‘‘
                              String filepath=f.getPath();‘‘‘‘
                              try{‘‘
                              BufferedReader br=new BufferedReader(new FileReader(filepath));‘‘‘‘
```

```
                                String s1="",s2="";’’’’’’’’’’’’’’’’’’’’’’’’
                                while((s1=br.readLine())!=null){’’’’
                                s2+=s1+"\n";’’’’
                                }’’’’
                                ta.setText(s2);’’’’
                                br.close();’’’’
                                }catch (Exception ex) {ex.printStackTrace();’’}’’’’’’’’’’’’’’’’
’’’’}}}’’’’’’’
public static void main(String[] args) {’’’’
’’’’FileChooserExample om=new FileChooserExample();’’’’
’’’’’’’’’’’’ om.setSize(500,500);’’’’
’’’’’’’’’’’’ om.setLayout(null);’’’’
’’’’’’’’’’’’ om.setVisible(true);’’’’
’’’’’’’’’’’’ om.setDefaultCloseOperation(EXIT_ON_CLOSE);}’’’’
}
’’’’
```



Java Swing JToggelButton: JToggleButton is used to create toggle button, it is two-states button to switch on or off.

Modifier and Type	Class	
protected class	JToggleButton.AccessibleJToggleButton	This class support for
static class	JToggleButton.ToggleButtonModel	The Toggle

Class JToggleButton : Constructors:

Constructor	Descri
JToggleButton()	It creates an initially unselected toggle butt
JToggleButton(Action a)	It creates a toggle button where properties a
JToggleButton(Icon icon)	It creates an initially unselected toggle butt
JToggleButton(Icon icon, boolean selected)	It creates a toggle button with the specified
JToggleButton(String text)	It creates an unselected toggle button with t
JToggleButton(String text, boolean selected)	It creates a toggle button with the specified
JToggleButton(String text, Icon icon)	It creates a toggle button that has the specif unselected.
JToggleButton(String text, Icon icon, boolean selected)	It creates a toggle button with the specified

Class JToggleButton : Methods:

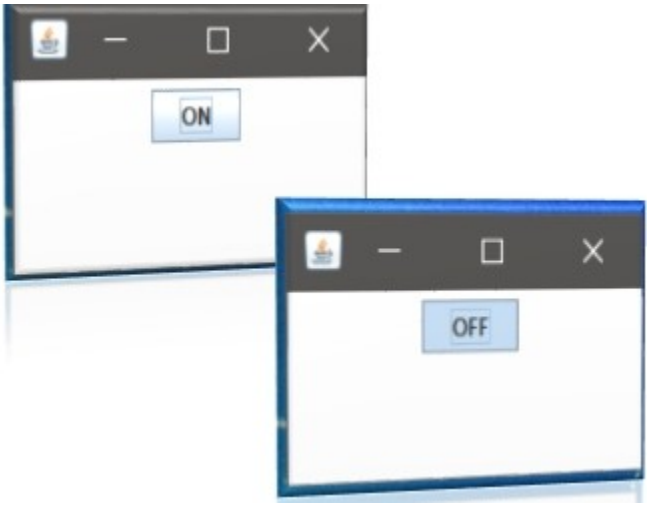
Modifier and Type	Method	Descriptio
AccessibleContext	`getAccessibleContext()`	It gets the AccessibleContext associated wi
String	`getUIClassID()`	It returns a string that specifies the name of this component.
protected String	`paramString()`	It returns a string representation of this JTo
void	`updateUI()`	It resets the UI property to a value from the

Illustration 10.31

```
// A program to add a Toggle button


import java.awt.FlowLayout;‘‘
import java.awt.event.ItemEvent;‘‘
import java.awt.event.ItemListener;‘‘
import javax.swing.JFrame;‘‘
import javax.swing.JToggleButton;
public class ToggleButtonExample extends JFrame implements ItemListener {‘‘
‘‘‘‘public static void main(String[] args) {‘‘
‘‘‘‘‘‘‘‘new ToggleButtonExample();‘‘
‘‘‘‘}‘‘
‘‘‘‘private JToggleButton button;‘‘
ToggleButtonExample() {‘‘
‘‘‘‘‘‘‘‘setTitle("JToggleButton with ItemListener Example");‘‘
‘‘‘‘‘‘‘‘setLayout(new FlowLayout());‘‘
‘‘‘‘‘‘‘‘setJToggleButton();‘‘
‘‘‘‘‘‘‘‘setAction();‘‘
‘‘‘‘‘‘‘‘setSize(200, 200);‘‘
‘‘‘‘‘‘‘‘setVisible(true);‘‘
‘‘‘‘‘‘‘‘setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘}
‘‘‘‘private void setJToggleButton() {‘‘
‘‘‘‘‘‘‘‘button = new JToggleButton("ON");‘‘
‘‘‘‘‘‘‘‘add(button);‘‘
‘‘‘‘}‘‘
‘‘‘‘private void setAction() {‘‘
```

```
‘‘‘‘‘‘‘‘button.addItemListener(this);’’
‘‘‘‘}’’
‘‘‘‘public void itemStateChanged(ItemEvent eve) {‘‘
‘‘‘‘‘‘‘‘if (button.isSelected())’’
‘‘‘‘‘‘‘‘‘‘‘‘button.setText("OFF");’’
‘‘‘‘‘‘‘‘else’’
‘‘‘‘‘‘‘‘‘‘‘‘button.setText("ON");’’
‘‘‘‘}’’
}’‘
```



Java Swing JToolBar: JToolBar container allows us to group other components, usually buttons with icons in a row or column. JToolBar provides a component which is useful for displaying commonly used actions or controls.

Modifier and Type	*Class*	*D*
protected class	JToolBar.AccessibleJToolBar	This class impl support for the
static class	JToolBar.Separator	A toolbar-spec

Class JToolBar : Constructors:

Constructor	Descriptio
`JToolBar()`	It creates a new tool bar; orientatio HORIZONTAL.
`JToolBar(int orientation)`	It creates a new tool bar with the s
`JToolBar(String name)`	It creates a new tool bar with the s
`JToolBar(String name, int orientation)`	It creates a new tool bar with a spe orientation.

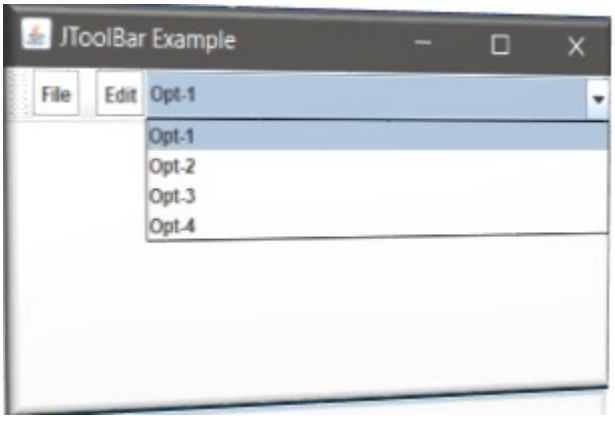
Class JToolBar : Methods:

Modifier and Type	Method	De
JButton	`add(Action a)`	It adds a new JButton which
protected void	`addImpl(Component comp, Object constraints, int index)`	If a JButton is being added,
void	`addSeparator()`	It appends a separator of de
protected PropertyChangeListener	`createActionChangeListener(JButton b)`	It returns a properly configu updates the control as chang default property change liste
protected JButton	`createActionComponent(Action a)`	Factory method which creat the JToolBar.
ToolBarUI	`getUI()`	It returns the tool bar's curre
void	`setUI(ToolBarUI ui)`	It sets the L&F object that r
void	`setOrientation(int o)`	It sets the orientation of the

Illustration 10.32

```
import java.awt.BorderLayout;’‘
import java.awt.Container;’‘
import javax.swing.JButton;’‘
import javax.swing.JComboBox;’‘
import javax.swing.JFrame;’‘
import javax.swing.JScrollPane;’‘
import javax.swing.JTextArea;’‘
import javax.swing.JToolBar;’‘
```

```
‘‘
public class ToolBarExample {‘‘
‘‘‘‘public static void main(final String args[]) {‘‘
‘‘‘‘‘‘‘‘JFrame myframe = new JFrame("JToolBar Example");’‘
‘‘‘‘‘‘‘‘myframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);’‘
‘‘‘‘‘‘‘‘JToolBar toolbar = new JToolBar();’‘
‘‘‘‘‘‘‘‘toolbar.setRollover(true);’‘
‘‘‘‘‘‘‘‘JButton button = new JButton("File");’‘
‘‘‘‘‘‘‘‘toolbar.add(button);’‘
‘‘‘‘‘‘‘‘toolbar.addSeparator();’‘
‘‘‘‘‘‘‘‘toolbar.add(new JButton("Edit"));’‘
‘‘‘‘‘‘‘‘toolbar.add(new JComboBox(new String[] { "Opt-1", "Opt-2", "Opt-3", "Opt-4" }));’‘
‘‘‘‘‘‘‘‘Container contentPane = myframe.getContentPane();’‘
‘‘‘‘‘‘‘‘contentPane.add(toolbar, BorderLayout.NORTH);’‘
‘‘‘‘‘‘‘‘JTextArea textArea = new JTextArea();’‘
‘‘‘‘‘‘‘‘JScrollPane mypane = new JScrollPane(textArea);’‘
‘‘‘‘‘‘‘‘contentPane.add(mypane, BorderLayout.EAST);’‘
‘‘‘‘‘‘‘‘myframe.setSize(450, 250);’‘
‘‘‘‘‘‘‘‘myframe.setVisible(true);’‘
‘‘‘‘}’‘
}’‘
```



Java Swing JViewport: The JViewport class is used to implement scrolling. JViewport is designed to support both logical scrolling and pixel-based scrolling. The viewport's child, called the view, is scrolled by calling the JViewport.setViewPosition() method.

Modifier and Type	Class	Description
protected class	JViewport.AccessibleJViewport	This class implements access support for the Jviewport cla
protected class	JViewport.ViewListener	A listener for the view.

Class JViewport : Fields:

Modifier and Type	Field	Descrip
static int	BACKINGSTORE_SCROLL_MODE	It draws viewport contents into an offscre
protected Image	backingStoreImage	The view image used for a backing store.
static int	BLIT_SCROLL_MODE	It uses graphics.copyArea to implement sc
protected boolean	isViewSizeSet	True when the viewport dimensions have b
protected Point	lastPaintPosition	The last viewPosition that we've painted, s backing store image is valid.
protected boolean	scrollUnderway	The scrollUnderway flag is used for comp
static int	SIMPLE_SCROLL_MODE	This mode uses the very simple method of scrollpane each time it is scrolled.

Class JViewport : Constructors:

Constructor	Description
`JViewport()`	Creates a JViewport.

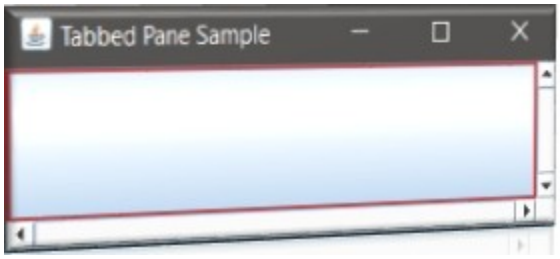
Class JViewport :Methods:

Modifier and Type	Method	Description
void	addChangeListener(ChangeListener l)	It adds a ChangeListener to the list that is notified position, or the viewport's extent size has changed
protected LayoutManager	createLayoutManager()	Subclassers can override this to install a different l constructor.
protected Jviewport.ViewListener	createViewListener()	It creates a listener for the view.
int	getScrollMode()	It returns the current scrolling mode.
Component	getView()	It returns the JViewport's one child or null.
Point	getViewPosition()	It returns the view coordinates that appear in the u viewport, or 0,0 if there's no view.
Dimension	getViewSize()	If the view's size hasn't been explicitly set, return t return the view's current size.
void	setExtentSize(Dimension newExtent)	It sets the size of the visible part of the view using
void	setScrollMode(int mode)	It used to control the method of scrolling the viewp
void	setViewSize(Dimension newSize)	It sets the size of the view.

Illustration 10.33

```
import java.awt.BorderLayout;’‘
import java.awt.Color;’‘
import java.awt.Dimension;’‘
import javax.swing.JButton;’‘
import javax.swing.JFrame;’‘
import javax.swing.JLabel;’‘
import javax.swing.JScrollPane;’‘
import javax.swing.border.LineBorder;’‘
public class ViewPortClass2 {’‘
‘‘‘‘public static void main(String[] args) {’‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("Tabbed Pane Sample");’‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);’‘
‘‘
‘‘‘‘‘‘‘‘JLabel label = new JLabel("Label");’‘
‘‘‘‘‘‘‘‘label.setPreferredSize(new Dimension(1000, 1000));’‘
‘‘‘‘‘‘‘‘JScrollPane jScrollPane = new JScrollPane(label);’‘
‘‘
‘‘‘‘‘‘‘‘JButton jButton1 = new JButton();’‘
‘‘‘‘‘‘‘‘jScrollPane.setHorizontalScrollBarPolicy(JScrollPane.HORIZONTAL_SCROLLBAR_ALWAYS);’‘
‘‘‘‘‘‘‘‘jScrollPane.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);’‘
```

```
‘‘‘‘‘‘‘‘jScrollPane.setViewportBorder(new LineBorder(Color.RED));’‘
‘‘‘‘‘‘‘‘jScrollPane.getViewport().add(jButton1, null);’‘
‘‘
‘‘‘‘‘‘‘‘frame.add(jScrollPane, BorderLayout.CENTER);’‘
‘‘‘‘‘‘‘‘frame.setSize(400, 150);’‘
‘‘‘‘‘‘‘‘frame.setVisible(true);’‘
‘‘‘‘}’‘
}
```



Java Swing JPanel: The JPanel is a simplest container class. It provides space in which an application can attach any other component. It inherits the JComponents class. It doesn't have title bar.



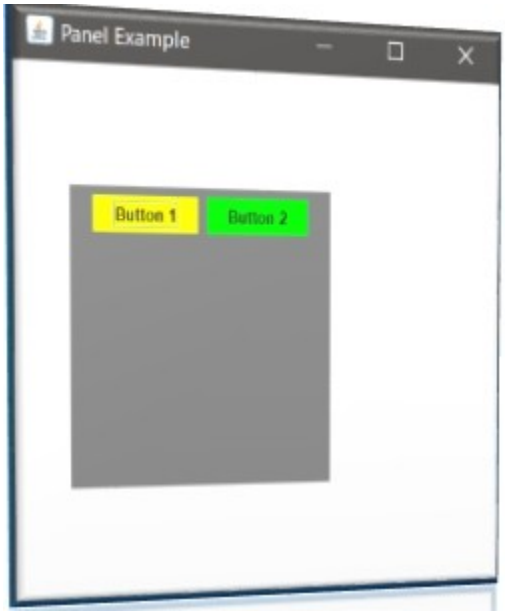
Below is the declaration for javax.swing.JPanel class. **public class JPanel extends JComponent implements Accessible**

Class JPanel : Constructors:

Constructor	Description
JPanel()	It is used to create a new JPanel with a d buffer and a flow layout.
JPanel(boolean isDoubleBuffered)	It is used to create a new JPanel with Flo and the specified buffering strategy.
JPanel(LayoutManager layout)	It is used to create a new JPanel with the layout manager.

Illustration 10.34

```
import java.awt.*;‘‘
import javax.swing.*;‘‘
public class PanelExample {‘‘
‘‘‘‘ PanelExample()‘‘{‘‘
‘‘‘‘‘‘‘‘JFrame f= new JFrame("Panel Example");‘‘‘‘
‘‘‘‘‘‘‘‘JPanel panel=new JPanel();‘‘
‘‘‘‘‘‘‘‘panel.setBounds(40,80,200,200);‘‘‘‘
‘‘‘‘‘‘‘‘panel.setBackground(Color.gray);‘‘
‘‘‘‘‘‘‘‘JButton b1=new JButton("Button 1");‘‘‘‘
‘‘‘‘‘‘‘‘b1.setBounds(50,100,80,30);‘‘‘‘
‘‘‘‘‘‘‘‘b1.setBackground(Color.yellow);‘‘
‘‘‘‘‘‘‘‘JButton b2=new JButton("Button 2");‘‘
‘‘‘‘‘‘‘‘b2.setBounds(100,100,80,30);‘‘‘‘
‘‘‘‘‘‘‘‘b2.setBackground(Color.green);‘‘
‘‘‘‘‘‘‘‘panel.add(b1); panel.add(b2);‘‘
‘‘‘‘‘‘‘‘f.add(panel);‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setSize(400,400);‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setLayout(null);‘‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘f.setVisible(true);‘‘‘‘
‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘‘‘‘‘public static void main(String args[]) {‘‘
‘‘‘‘‘‘‘‘new PanelExample();‘‘
‘‘‘‘‘‘‘‘}‘‘
‘‘‘‘}‘‘
```



Java Swing JFrame: The javax.swing.JFrame class is a type of container which inherits the java.awt.Frame class. JFrame works like the main window where components like labels, buttons, textfields are added to create a GUI. Unlike Frame, JFrame has the option to hide or close the window with the help of setDefaultCloseOperation(int) method.

Modifier and Type	*Class*	*Desc*
protected class	JFrame.AccessibleJFrame	This class implements for the JFrame class.

Class JFrame : Fields:��

Modifier and Type	*Field*	*Description*
protected AccessibleContext	accessibleContext	The accessible context property.
static int	EXIT_ON_CLOSE	The exit application default window close operation.
protected JRootPane	rootPane	The JRootPane instance that manages the contentPane and optional menuBar for this frame, as well as the glassPane.
protected boolean	rootPaneCheckingEnabled	If true then calls to add and setLayout will be forwarded to the contentPane.

Class JFrame : Constructors:

Constructor	Description
`JFrame()`	It constructs a new frame that is initiall
`JFrame(GraphicsConfiguration gc)`	It creates a Frame in the specified GraphicsConfiguration of a screen dev title.
`JFrame(String title)`	It creates a new, initially invisible Fra specified title.
`JFrame(String title, GraphicsConfiguration gc)`	It creates a JFrame with the specified t specified GraphicsConfiguration of a s

Class JFrame : Methods:

Modifier and Type	Method	Descr
protected void	`addImpl(Component comp, Object constraints, int index)`	Adds the specified child Component.
protected JRootPane	`createRootPane()`	Called by the constructor methods to cr
protected void	`frameInit()`	Called by the constructors to init the JFr
void	`setContentPane(Containe contentPane)`	It sets the contentPane property
static void	`setDefaultLookAndFeelDecorated(boolean defaultLookAndFeelDecorated)`	Provides a hint as to whether or not new Window decorations (such as borders, v provided by the current look and feel.
void	`setIconImage(Image image)`	It sets the image to be displayed as the ic
void	`setJMenuBar(JMenuBar menubar)`	It sets the menubar for this frame.
void	`setLayeredPane(JLayeredPane layeredPane)`	It sets the layeredPane property.
JRootPane	`getRootPane()`	It returns the rootPane object for this fra
TransferHandler	`getTransferHandler()`	It gets the transferHandler property.

Illustration 10.35

```
import java.awt.FlowLayout;’‘
import javax.swing.JButton;’‘
import javax.swing.JFrame;’‘
```

```
import javax.swing.JLabel;‘‘
import javax.swing.Jpanel;‘‘
public class JFrameExample {‘‘
‘‘‘‘public static void main(String s[]) {‘‘
‘‘‘‘‘‘‘‘JFrame frame = new JFrame("JFrame Example");‘‘
‘‘‘‘‘‘‘‘JPanel panel = new JPanel();‘‘
‘‘‘‘‘‘‘‘panel.setLayout(new FlowLayout());‘‘
‘‘‘‘‘‘‘‘JLabel label = new JLabel("JFrame By Example");‘‘
‘‘‘‘‘‘‘‘JButton button = new JButton();‘‘
‘‘‘‘‘‘‘‘button.setText("Button");‘‘
‘‘‘‘‘‘‘‘panel.add(label);‘‘
‘‘‘‘‘‘‘‘panel.add(button);‘‘
‘‘‘‘‘‘‘‘frame.add(panel);‘‘
‘‘‘‘‘‘‘‘frame.setSize(200, 300);‘‘
‘‘‘‘‘‘‘‘frame.setLocationRelativeTo(null);‘‘
‘‘‘‘‘‘‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);‘‘
‘‘‘‘‘‘‘‘frame.setVisible(true);‘‘
‘‘‘‘}‘‘
}‘‘
```



Java Swing Calculator:

```
Methods used :
```

```
add(Component c) : adds component to container.

addActionListenerListener(ActionListener d) : add actionListener for specified component

setBackground(Color c) : sets the background color of the specified container

setSize(int a, int b) : sets the size of container to specified dimensions.

setText(String s) : sets the text of the label to s.

getText() : returns the text of the label
```



Illustration 10.36

```
// Java program to create a simple calculator
// with basic +, -, /, * using java swing elements

import java.awt.event.*;
import javax.swing.*;
import java.awt.*;
class Calculator extends JFrame implements ActionListener {
          // create a frame
          static JFrame f;
          // create a textfield
          static JTextField l;
          // store oprerator and operands
          String s0, s1, s2;
          // default constrcutor
          Calculator()
```

```
        {
                s0 = s1 = s2 = "";
        }
        // main function
        public static void main(String args[])
        {
                // create a frame
                f = new JFrame("Swing Calculator");
                try {
                        // set look and feel
                        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
                }
                catch (Exception e) {
                        System.err.println(e.getMessage());
                }
                // create a object of class
                Calculator c = new Calculator();
                // create a textfield
                l = new JTextField(16);
                // set the textfield to non editable
                l.setEditable(false);
                // create number buttons and some operators
                JButton b0, b1, b2, b3, b4, b5, b6, b7, b8, b9, ba, bs, bd, bm, be, beq, beq1;
                // create number buttons
                b0 = new JButton("0");
                b1 = new JButton("1");
                b2 = new JButton("2");
                b3 = new JButton("3");
                b4 = new JButton("4");
                b5 = new JButton("5");
                b6 = new JButton("6");
                b7 = new JButton("7");
                b8 = new JButton("8");
                b9 = new JButton("9");
                // equals button
                beq1 = new JButton("=");
                // create operator buttons
                ba = new JButton("+");
                bs = new JButton("-");
                bd = new JButton("/");
                bm = new JButton("*");
```

```
                    beq = new JButton("C");
                    // create . button
                    be = new JButton(".");
                    // create a panel
                    JPanel p = new JPanel();
                    // add action listeners
                    bm.addActionListener(c);
                    bd.addActionListener(c);
                    bs.addActionListener(c);
                    ba.addActionListener(c);
                    b9.addActionListener(c);
                    b8.addActionListener(c);
                    b7.addActionListener(c);
                    b6.addActionListener(c);
                    b5.addActionListener(c);
                    b4.addActionListener(c);
                    b3.addActionListener(c);
                    b2.addActionListener(c);
                    b1.addActionListener(c);
                    b0.addActionListener(c);
                    be.addActionListener(c);
                    beq.addActionListener(c);
                    beq1.addActionListener(c);
                    // add elements to panel
                    p.add(l);
                    p.add(ba);
                    p.add(b1);
                    p.add(b2);
                    p.add(b3);
                    p.add(bs);
                    p.add(b4);
                    p.add(b5);
                    p.add(b6);
                    p.add(bm);
                    p.add(b7);
                    p.add(b8);
                    p.add(b9);
                    p.add(bd);
                    p.add(be);
                    p.add(b0);
                    p.add(beq);
```

```
                    p.add(beq1);
                    // set Background of panel
                    p.setBackground(Color.blue);
                    // add panel to frame
                    f.add(p);
                    f.setSize(200, 220);
                    f.show();
          }
          public void actionPerformed(ActionEvent e)
          {
                    String s = e.getActionCommand();
                    // if the value is a number
                    if ((s.charAt(0) >= '0' && s.charAt(0) <= '9') || s.charAt(0) == '.') {
                              // if operand is present then add to second no
                              if (!s1.equals(""))
                                        s2 = s2 + s;
                              else
                                        s0 = s0 + s;
                              // set the value of text
                              l.setText(s0 + s1 + s2);
                    }
                    else if (s.charAt(0) == 'C') {
                              // clear the one letter
                              s0 = s1 = s2 = "";
                              // set the value of text
                              l.setText(s0 + s1 + s2);
                    }
                    else if (s.charAt(0) == '=') {
                              double te;
                              // store the value in 1st
                              if (s1.equals("+"))
                                        te = (Double.parseDouble(s0) + Double.parseDouble(s2));
                              else if (s1.equals("-"))
                                        te = (Double.parseDouble(s0) - Double.parseDouble(s2));
                              else if (s1.equals("/"))
                                        te = (Double.parseDouble(s0) / Double.parseDouble(s2));
                                        else
                                                  te = (Double.parseDouble(s0) *
Double.parseDouble(s2));
                                        // convert it to string
                                        s0 = Double.toString(te);
```

```
                                        // place the operator
                                        s1 = s;
                                        // make the operand blank
                                        s2 = "";
                                }
                                // set the value of text
                                l.setText(s0 + s1 + s2);
‘‘‘‘‘‘‘‘}
‘‘‘‘}
}
```



Practice Questions

Practice 10.1

```
          //Simple Application

          import javax.swing.*;
          import java.awt.*;
          import java.awt.event.*;

          public class SimpleApplication implements ActionListener {
                    private static String labelPrefix = "Number of button clicks: ";
                    private int numClicks = 0;
                    private final JLabel label = new JLabel(labelPrefix + "0’‘‘‘");
```

```
            public Component createComponents() {
                        JButton button = new JButton("I'm a Swing button!");
                        button.addActionListener(this);
‘‘
                        /* An easy way to put space between a top-level container
                         * and its contents is to put the contents in a JPanel
                         * that has an "empty" border. */
                        JPanel pane = new JPanel(new GridLayout(0, 1));
                        pane.add(button);
                        pane.add(label);
                        pane.setBorder(BorderFactory.createEmptyBorder(30,30,10,30));’‘‘‘
                        //top,left,bottom,right
                        return pane;
            }

            public void actionPerformed(ActionEvent e) {
                        numClicks++;
                        label.setText(labelPrefix + numClicks);
            }

            private static void initLookAndFeel() {
‘‘ // METAL
                         String lookAndFeel = "javax.swing.plaf.metal.MetalLookAndFeel";
‘‘ // MOTIF
                        // String lookAndFeel = "com.sun.java.swing.plaf.motif.MotifLookAndFeel";
‘‘ // WINDOWS
                         // String lookAndFeel = "com.sun.java.swing.plaf.windows.WindowsLookAndFeel";

                        try {
                                                UIManager.setLookAndFeel(lookAndFeel);
                        }
                        catch (UnsupportedLookAndFeelException e){
                        }
                        catch(ClassNotFoundException e){
                        }
                        catch (Exception e){
                        }
            }
```

```
                private static void createAndShowGUI() {

                        //Set the look and feel.
                        initLookAndFeel();

                        //Make sure we have nice window decorations.
                        JFrame.setDefaultLookAndFeelDecorated(true);

                        //Create and set up the window.
                        JFrame frame = new JFrame("SimpleApplication");
                        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

                        // Create a new instance of the application class
                        SimpleApplication app = new SimpleApplication();
                        Component contents = app.createComponents();
                        frame.getContentPane().add(contents, BorderLayout.CENTER);

                        //Display the window.
                        frame.pack();
                        frame.setVisible(true);
                }

                public static void main(String[] args) {

                        //Schedule a job for the event-dispatching thread:
                        //creating and showing this application's GUI.

                        javax.swing.SwingUtilities.invokeLater(new Runnable() {
                                public void run() {
                                        createAndShowGUI();
                                }
                        });
                }
        }
```

Practice 9.2

```
        //Dialog example
```

```
import javax.swing.JOptionPane;

public class JOptionPaneExample {
          public static void main(String[] args) {
          String ans;

‘‘ //Input Dialog
‘‘ ans = JOptionPane.showInputDialog(null, "Speed in miles per hour?");
‘‘ double mph = Double.parseDouble(ans);
‘‘ double kph = 1.621 * mph;

‘‘ // Message Dialog
          JOptionPane.showMessageDialog(null, "KPH = " + kph);

‘‘ // Warning message
‘‘ JOptionPane.showMessageDialog(null,"Please pay attention", "A warning message",
          JOptionPane.WARNING_MESSAGE);

‘‘ // Question message
          JOptionPane.showMessageDialog(null,"Are you sleeping", "An question message",
           JOptionPane.QUESTION_MESSAGE );

          // Error message dialog
          JOptionPane.showMessageDialog(null, "Error!", "An error message",’‘
          ‘‘JOptionPane.ERROR_MESSAGE);

          // Input dialog with choice
          ‘‘ Object[] possibleValues = { "Enjoying", "Boring", "Sleep" };
          ‘‘ Object selectedValue = JOptionPane.showInputDialog(
                                    ‘‘null,
                                    "Choose one",
                                    "Input",
                                    JOptionPane.INFORMATION_MESSAGE,
                                    null,
                                    possibleValues,
                                    possibleValues[0]);

 JOptionPane.showMessageDialog(null, "Hey, you are now feeling = " + selectedValue);
```

```
          // Option Dialog
                     Object[] options = {"Yes", "No", "Don't know"};
                     int n = JOptionPane.showOptionDialog(
                                                  null,
                                        "Feeling hungry?",
                                        "A Silly Question",
                                        JOptionPane.YES_NO_CANCEL_OPTION,
                                        JOptionPane.QUESTION_MESSAGE,
                                        null,
                                        options,
                                        options[2]);

                                        ‘‘
                                        System.exit(0);
                     }
          }
```

Practice 9.3

```
          //example of Box layout

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.*;

          public class Password
          {
                     public static void main(String[] args)
                     {‘‘
                               JFrame frame = new JFrame("Password test");

                               frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
                               frame.getContentPane().add(new PasswordFrame(frame));
                               frame.setSize(new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT));
                               frame.setLocation(250,200);
                               frame.setResizable(false);
                               frame.show();
                     }

                     public static final int DEFAULT_WIDTH = 225;
                     public static final int DEFAULT_HEIGHT = 125;’‘
```

```
	}

	/**
		A frame that uses box layouts to organize various components.
	*/
	class PasswordFrame extends JPanel
	{
		public PasswordFrame(JFrame frame)
		{

			passFrame = frame;
			// construct the left vertical box
			JLabel label1 = new JLabel("Name");
			label1.setForeground(Color.yellow);

			JLabel label2 = new JLabel("Password");
			label2.setForeground(Color.yellow);
			label2.setDisplayedMnemonic('s');
			label2.setDisplayedMnemonicIndex(3);

			Box hbox1 = Box.createVerticalBox();
			hbox1.add(label1);
			// separate labels with a 10-pixel strut
			hbox1.add(Box.createVerticalStrut(10));
			hbox1.add(label2);

			// construct the right vertical box
			JTextField textField1 = new JTextField(10);
			textField1.setMaximumSize(textField1.getPreferredSize());

			// declare password field
			JPasswordField passwordField = new JPasswordField(10);
			passwordField.setMaximumSize(passwordField.getPreferredSize());
			passwordField.setEchoChar('@');
				// passwordField.setActionCommand(OK);
				// passwordField.addActionListener(this);
			label2.setLabelFor(passwordField);


//xxxxx.setComponentOrientation( ComponentOrientation.RIGHT_TO_LEFT);
```

```
Box hbox2 = Box.createVerticalBox();
hbox2.add(textField1);
// separate textfields with a 10-pixel strut
hbox2.add(Box.createVerticalStrut(10));
hbox2.add(passwordField);

// construct the top horizontal box
// by joining the left and roght horizontal boxes
Box hbox2a = Box.createHorizontalBox();
hbox2a.add(Box.createHorizontalStrut(10));
hbox2a.add(hbox1);
hbox2a.add(Box.createHorizontalStrut(10));
hbox2a.add(hbox2);
hbox2a.add(Box.createHorizontalStrut(10));

// construct the bottom horizontal box
JButton button1 = new JButton("‘‘ Ok’‘ ");
JButton button2 = new JButton("Cancel");

button1.setBackground(Color.gray);
button1.setForeground(Color.yellow);
button1.setBorder(BorderFactory.createRaisedBevelBorder());
button1.setActionCommand(OK);
button1.addActionListener(new buttonListener());

button2.setBackground(Color.gray);
button2.setForeground(Color.yellow);
button2.setBorder(BorderFactory.createRaisedBevelBorder());
button2.setActionCommand(CANCEL);
button2.addActionListener(new buttonListener());

Box hbox3 = Box.createHorizontalBox();
hbox3.add(button1);
          // use "glue" to push the two buttons apart
          // hbox3.add(Box.createGlue());
hbox3.add(Box.createHorizontalStrut(30));
hbox3.add(button2);

// add the two horizontal boxes inside a vertical box
```

```
                    Box vbox = Box.createVerticalBox();
                    vbox.add(hbox2a);
                    vbox.add(Box.createVerticalStrut(20));
                    vbox.add(hbox3);

                              // Container contentPane = getContentPane();
                              // contentPane.setBackground(Color.blue);
                              // contentPane.add(vbox, BorderLayout.CENTER);
                              //
                     add(vbox, BorderLayout.CENTER);
                     setBackground(Color.blue);


                    }

          class buttonListener implements ActionListener
          {
                    public void actionPerformed(ActionEvent e)
                    {
                    '' String buttonPressed = e.getActionCommand();

                              if (buttonPressed.equals(OK))
                              {
Container contentPane = messageDialog.getContentPane();
contentPane.add(new JLabel(" Success !!'' Wish you all the best"));
                              messageDialog.setSize(new Dimension(250,100));
                              messageDialog.setLocation(250+200, 200+100);
          contentPane.setBackground(new Color(252, 215, 180));
                              messageDialog.show();
                              }
                              else
                              {

                              }
                    }
          };

          JFrame passFrame;
          JDialog messageDialog = new JDialog(passFrame, "Login Message");

          private static String OK = "ok";
```

```
                    private static String CANCEL = "cancel";
          }
```

Practice 9.4

```
          //Example of Button Demo

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.AbstractButton;
          import javax.swing.JButton;
          import javax.swing.JPanel;
          import javax.swing.JFrame;
          import javax.swing.ImageIcon;

          public class ButtonDemo extends JPanel implements ActionListener {
                    protected JButton b1, b2, b3;

                    public ButtonDemo() {
                              ImageIcon leftButtonIcon = new ImageIcon("images/rightIcon.gif");
                              ImageIcon middleButtonIcon = new ImageIcon("images/midIcon.gif");
                              ImageIcon rightButtonIcon = new ImageIcon("images/leftIcon.gif");

                              b1 = new JButton("Disable middle button", leftButtonIcon);
                              b1.setVerticalTextPosition(AbstractButton.CENTER);
                              b1.setHorizontalTextPosition(AbstractButton.LEFT);
                              b1.setMnemonic(KeyEvent.VK_D);
                              b1.setActionCommand("disable");

                              b2 = new JButton("Middle button", middleButtonIcon);
                              b2.setVerticalTextPosition(AbstractButton.BOTTOM);
                              b2.setHorizontalTextPosition(AbstractButton.CENTER);
                              b2.setMnemonic(KeyEvent.VK_M);

                              b3 = new JButton("Enable middle button", rightButtonIcon);
                              //Use the default text position of CENTER, RIGHT.
                              b3.setMnemonic(KeyEvent.VK_E);
                              b3.setActionCommand("enable");
                              b3.setEnabled(false);
```

```
                    //Listen for actions on buttons 1 and 3.
                    b1.addActionListener(this);
                    b3.addActionListener(this);

                    b1.setToolTipText("Click this button to disable the middle button.");
                    b2.setToolTipText("This middle button does nothing when you click it.");
                    b3.setToolTipText("Click this button to enable the middle button.");

                    //Add Components to this container, using the default FlowLayout.
                    add(b1);
                    add(b2);
                    add(b3);
          }

          public void actionPerformed(ActionEvent e) {
                    if (e.getActionCommand().equals("disable")) {
                              b2.setEnabled(false);
                              b1.setEnabled(false);
                              b3.setEnabled(true);
                    } else {
                              b2.setEnabled(true);
                              b1.setEnabled(true);
                              b3.setEnabled(false);
                    }
          }

          public static void main(String[] args) {
                    JFrame frame = new JFrame("ButtonDemo");

                    frame.addWindowListener(new WindowAdapter() {
                              public void windowClosing(WindowEvent e) {
                                        System.exit(0);
                              }
                    });

                    frame.getContentPane().add(new ButtonDemo(), BorderLayout.CENTER);
                    frame.pack();
                    frame.setVisible(true);
          }
}
```

Practice 9.5

```
          //Flow layout Demo
          import java.awt.Container;
          import java.awt.FlowLayout;
          import javax.swing.JButton;
          import javax.swing.JFrame;

          import java.awt.Dimension;
          import java.awt.ComponentOrientation;

          public class FlowLayoutDemo {
                    public static boolean RIGHT_TO_LEFT = true;

                    public static void addComponents(Container contentPane) {
                              if (RIGHT_TO_LEFT) {
                                        contentPane.setComponentOrientation(
                                                  ComponentOrientation.RIGHT_TO_LEFT);
                              }
                              contentPane.setLayout(new FlowLayout());

                              contentPane.add(new JButton("Button 1"));
                              contentPane.add(new JButton("Button 2"));
                              contentPane.add(new JButton("Button 3"));
                              contentPane.add(new JButton("Long-Named Button 4"));
                              contentPane.add(new JButton("5"));
                    }

                    /**
                     * Create the GUI and show it.’‘For thread safety,
                     * this method should be invoked from the
                     * event-dispatching thread.
                     */
                    private static void createAndShowGUI() {
                              //Make sure we have nice window decorations.
                              JFrame.setDefaultLookAndFeelDecorated(true);

                              //Create and set up the window.
                              //JFrames decorated by the Java look and feel
                              //can't get smaller than their minimum size.
                              //We specify a skinnier minimum size than the
```

```
                        //content pane will cause the frame to request,
                        //so that you can see what happens when you
                        //drag the window so that it's narrower than a
                        //single row.
                        JFrame frame = new JFrame("FlowLayoutDemo") {
                                public Dimension getMinimumSize() {
                                        Dimension prefSize = getPreferredSize();
                                        return new Dimension(100, prefSize.height);
                                }
                        };
                        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

                        //Set up the content pane.
                        addComponents(frame.getContentPane());
                        //Display the window.
                        frame.pack();
                        frame.setVisible(true);
                }

                public static void main(String[] args) {
                        //Schedule a job for the event-dispatching thread:
                        //creating and showing this application's GUI.
                        javax.swing.SwingUtilities.invokeLater(new Runnable() {
                                public void run() {
                                        createAndShowGUI();
                                }
                        });
                }
        }
```

Practice 9.6

```
        //Example with font
        import java.awt.*;
        import java.awt.event.*;
        import javax.swing.*;
        import javax.swing.event.*;

        public class FontDialog
        {
```

```
‘‘ public static void main(String[] args)
‘‘ {‘‘
          ‘‘FontDialogFrame frame = new FontDialogFrame();
          ‘‘frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
          ‘‘frame.show();
‘‘ }
}

/**
‘‘ A frame that uses a grid bag layout to arrange font
‘‘ selection components.
*/
class FontDialogFrame extends JFrame
{‘‘
‘‘ public FontDialogFrame()
‘‘ {‘‘
          ‘‘setTitle("FontDialog");
          ‘‘setSize(WIDTH, HEIGHT);

          ‘‘Container contentPane = getContentPane();
          ‘‘GridBagLayout layout = new GridBagLayout();
          ‘‘contentPane.setLayout(layout);

          ‘‘ActionListener listener = new FontAction();

          ‘‘// construct components
          ‘‘
          ‘‘JLabel faceLabel = new JLabel("Face: ");

          ‘‘face = new JComboBox(new String[]
                    {‘‘
                              "Serif", "SansSerif", "Monospaced",
                              "Dialog", "DialogInput"
                    });
          ‘‘
          ‘‘face.addActionListener(listener);

          ‘‘JLabel sizeLabel = new JLabel("Size: ");

          ‘‘size = new JComboBox(new String[]
                    {
```

```
                        "8", "10", "12", "15", "18", "24", "36", "48"
                });

        ‘‘size.addActionListener(listener);

        ‘‘bold = new JCheckBox("Bold");
        ‘‘bold.addActionListener(listener);

        ‘‘italic = new JCheckBox("Italic");
        ‘‘italic.addActionListener(listener);

        ‘‘sample = new JTextArea();
        ‘‘sample.setText(
                "The quick brown fox jumps over the lazy dog");
        ‘‘sample.setEditable(false);
        ‘‘sample.setLineWrap(true);
        ‘‘sample.setBorder(BorderFactory.createEtchedBorder());

        ‘‘// add components to grid

        ‘‘GridBagConstraints constraints = new GridBagConstraints();

        ‘‘constraints.fill = GridBagConstraints.NONE;
        ‘‘constraints.anchor = GridBagConstraints.EAST;
        ‘‘constraints.weightx = 0;
        ‘‘constraints.weighty = 0;

        ‘‘add(faceLabel, constraints, 0, 0, 1, 1);
        ‘‘add(sizeLabel, constraints, 0, 1, 1, 1);

        ‘‘constraints.fill = GridBagConstraints.HORIZONTAL;
        ‘‘constraints.weightx = 100;
        ‘‘
        ‘‘add(face, constraints, 1, 0, 1, 1);
        ‘‘add(size, constraints, 1, 1, 1, 1);

        ‘‘constraints.weighty = 100;
        ‘‘constraints.fill = GridBagConstraints.NONE;
        ‘‘constraints.anchor = GridBagConstraints.CENTER;

        ‘‘add(bold, constraints, 0, 2, 2, 1);
```

```
                ‘‘add(italic, constraints, 0, 3, 2, 1);

                ‘‘constraints.fill = GridBagConstraints.BOTH;
                ‘‘add(sample, constraints, 2, 0, 1, 4);
        ‘‘ }
        ‘‘
        ‘‘ /**
                ‘‘A convenience method to add a component to given grid bag
                ‘‘layout locations.
                ‘‘@param c the component to add
                ‘‘@param constraints the grid bag constraints to use
                ‘‘@param x the x grid position
                ‘‘@param y the y grid position
                ‘‘@param w the grid width
                ‘‘@param h the grid height
        ‘‘ */
        ‘‘ public void add(Component c, GridBagConstraints constraints,int x, int y, int w, int h)
        ‘‘ {‘‘
                ‘‘constraints.gridx = x;
                ‘‘constraints.gridy = y;
                ‘‘constraints.gridwidth = w;
                ‘‘constraints.gridheight = h;
                ‘‘getContentPane().add(c, constraints);
        ‘‘ }

        ‘‘ public static final int WIDTH = 300;
        ‘‘ public static final int HEIGHT = 200;’‘

        ‘‘ private JComboBox face;
        ‘‘ private JComboBox size;
        ‘‘ private JCheckBox bold;
        ‘‘ private JCheckBox italic;
        ‘‘ private JTextArea sample;

        ‘‘ /**
                ‘‘An action listener that changes the font of the
                ‘‘sample text.
        ‘‘ */
        ‘‘ private class FontAction implements ActionListener
        ‘‘ {
                ‘‘public void actionPerformed(ActionEvent event)
```

```
                    ‘‘{‘‘
                              String fontFace = (String)face.getSelectedItem();
                              int fontStyle = (bold.isSelected() ? Font.BOLD : 0)
                                        + (italic.isSelected() ? Font.ITALIC : 0);
                              int fontSize = Integer.parseInt(
                                        (String)size.getSelectedItem());
                              Font font = new Font(fontFace, fontStyle, fontSize);
                              sample.setFont(font);
                              sample.repaint();
                    ‘‘}
          ‘‘ }
          }
```

Practice 9.7

```
          //Voting example

          import javax.swing.*;
          import java.awt.*;
          import java.awt.event.*;

          public class VoteDialog extends JPanel {
                    JLabel label;
                    JFrame frame;
                    String simpleDialogDesc = "The candidates";

                    public VoteDialog(JFrame frame) {
                              super(new BorderLayout());

                              this.frame = frame;
                              JLabel title;

                              //Create the components.
                              JPanel choicePanel = createSimpleDialogBox();

                              System.out.println("passed createSimpleDialogBox");

                              title = new JLabel("Click the \"Vote\" button"
                                                                  ‘‘ + " once you have selected a
candidate.",
```

```
                                                        ‘‘ JLabel.CENTER);

            label = new JLabel("Vote now!", JLabel.CENTER);
            label.setBorder(BorderFactory.createEmptyBorder(10,10,10,10));
            choicePanel.setBorder(BorderFactory.createEmptyBorder(20,20,5,20));

            //Lay out the main panel.
            add(title, BorderLayout.NORTH);’‘
            add(label, BorderLayout.SOUTH);’‘‘‘‘‘‘‘
            add(choicePanel, BorderLayout.CENTER);
    }

    void setLabel(String newText) {
            label.setText(newText);
    }

    private JPanel createSimpleDialogBox() {
            final int numButtons = 4;
            JRadioButton[] radioButtons = new JRadioButton[numButtons];

            final ButtonGroup group = new ButtonGroup();

            JButton voteButton = null;

            final String defaultMessageCommand = "default";
            final String yesNoCommand = "yesno";
            final String yeahNahCommand = "yeahnah";
            final String yncCommand = "ync";

            radioButtons[0] = new JRadioButton(
            ‘‘"Candidate 1: Sparky the Dog");
            radioButtons[0].setActionCommand(defaultMessageCommand);

            radioButtons[1] = new JRadioButton(
            ‘‘ "Candidate 2: Shady Sadie");
            radioButtons[1].setActionCommand(yesNoCommand);

            radioButtons[2] = new JRadioButton(
                    "Candidate 3: R.I.P. McDaniels");
            radioButtons[2].setActionCommand(yeahNahCommand);
```

```
                                radioButtons[3] = new JRadioButton(
                                          "Candidate 4: Duke the Java™ Platform Mascot");
                                radioButtons[3].setActionCommand(yncCommand);



                                for (int i = 0; i < numButtons; i++) {
                                          group.add(radioButtons[i]);
                                }

                                //Select the first button by default.
                                radioButtons[0].setSelected(true);

                                voteButton = new JButton("Vote");


                                voteButton.addActionListener(new ActionListener() {
                                          public void actionPerformed(ActionEvent e) {
                                                    String command =
group.getSelection().getActionCommand();


                                                    //ok dialog
                                                    if (command == defaultMessageCommand) {
                                                              JOptionPane.showMessageDialog(frame,
                                                              ‘‘ "This candidate is a dog. Invalid
vote.");


                                                    //yes/no dialog
                                                    } else if (command == yesNoCommand) {
                                                              int n =
JOptionPane.showConfirmDialog(frame,
                                                    ‘‘ ?This candidate is a convicted felon. \nDo you
still want
                                                              to vote for her?",
                                                              ‘‘ "A Follow-up Question",
                                                              ‘‘ JOptionPane.YES_NO_OPTION);
                                                              if (n == JOptionPane.YES_OPTION) {
                                                                        setLabel("OK. Keep an eye on your
wallet.");
                                                              } else if (n == JOptionPane.NO_OPTION) {
                                                                        setLabel("Whew! Good choice.");
                                                              } else {
                                                                        setLabel("It is your civic duty
to cast your vote.");
```

```
                                                  }

                                        //yes/no (with customized wording)
                                        } else if (command == yeahNahCommand) {
                                                  Object[] options = {"Yes, please", "No,
thanks"};
                                                  int n = JOptionPane.showOptionDialog(frame,
                                        '' "This candidate is deceased. \nDo you still want to
vote for him?",
                                                  '' "A Follow-up Question",
                                                  '' JOptionPane.YES_NO_OPTION,
                                                  '' JOptionPane.QUESTION_MESSAGE,
                                                  '' null,
                                                  '' options,
                                                  '' options[0]);
                                                  if (n == JOptionPane.YES_OPTION) {
                                                            setLabel("I hope you don't expect
much from your candidate.");
                                                  } else if (n == JOptionPane.NO_OPTION) {
                                                            setLabel("Whew! Good choice.");
                                                  } else {
                                                            setLabel("It is your civic duty
to cast your vote.");
                                                  }

                                        //yes/no/cancel (with customized wording)
                                        } else if (command == yncCommand) {
                                                  Object[] options = {"Yes!",
                                                                                                                                                  "N
o, I'll pass",
                                                                                                                                                  "W
ell, if I must"};
                                                  int n = JOptionPane.showOptionDialog(frame,
                                                                                                                                                  "D
uke is a cartoon mascot. \nDo you'‘"
                                                                                                                                                  +
"still want to cast your vote?",
                                                                                                                                                  "A
Follow-up Question",
            JOptionPane.YES_NO_CANCEL_OPTION,
            JOptionPane.QUESTION_MESSAGE,
            null,
```

```
          options,

          options[2]);
                                                    if (n == JOptionPane.YES_OPTION) {
                                                              setLabel("Excellent choice.");
                                                    } else if (n == JOptionPane.NO_OPTION) {
                                                              setLabel("Whatever you say. It's
your vote.");
                                                    } else if (n == JOptionPane.CANCEL_OPTION)
{
                                                              setLabel("Well, I'm certainly not
going to make you vote.");
                                                    } else {
                                                              setLabel("It is your civic duty
to cast your vote.");
                                                    }
                                          }
                                          return;
                                }
                        });
                        System.out.println("calling createPane");
                        return createPane(simpleDialogDesc + ":",
                                                    ‘‘radioButtons,
                                                    ‘‘voteButton);’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
‘
          }

          private JPanel createPane(String description,
                                                                        ‘‘JRadioButton[]
radioButtons,
                                                                        ‘‘JButton showButton) {
                        int numChoices = radioButtons.length;
                        JPanel box = new JPanel();
                        JLabel label = new JLabel(description);

                        box.setLayout(new BoxLayout(box, BoxLayout.PAGE_AXIS));
                        box.add(label);

                        for (int i = 0; i < numChoices; i++) {
                                  box.add(radioButtons[i]);
                        }
```

```
                JPanel pane = new JPanel(new BorderLayout());
                pane.add(box, BorderLayout.NORTH);
                pane.add(showButton, BorderLayout.SOUTH);
                System.out.println("returning pane");
                return pane;
        }

        /**
         * Create the GUI and show it.’‘For thread safety,
         * this method should be invoked from the
         * event-dispatching thread.
         */
        private static void createAndShowGUI() {
                //Make sure we have nice window decorations.
                JFrame.setDefaultLookAndFeelDecorated(true);
                JDialog.setDefaultLookAndFeelDecorated(true);
        ‘‘
                //Create and set up the window.
                JFrame frame = new JFrame("VoteDialog");
                frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

                //Set up the content pane.
                Container contentPane = frame.getContentPane();
                contentPane.setLayout(new GridLayout(1,1));
                contentPane.add(new VoteDialog(frame));

                //Display the window.
                frame.pack();
                frame.setVisible(true);
        }

        public static void main(String[] args) {
                //Schedule a job for the event-dispatching thread:
                //creating and showing this application's GUI.
                javax.swing.SwingUtilities.invokeLater(new Runnable() {
                        public void run() {
                                createAndShowGUI();
                        }
                });
        }
}
```

Practice 9.8

```
          //combo box with different window

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.*;

          public class CardWindow extends JFrame
                                                                        implements ItemListener {
                    boolean inAnApplet = true;

                    JPanel cards;
                    final static String BUTTONPANEL = "JPanel with JButtons";
                    final static String TEXTPANEL = "JPanel with JTextField";

                    public CardWindow() {
                              Container contentPane = getContentPane();

                     //Put the JComboBox in a JPanel to get a nicer look.
                              String comboBoxItems[] = { BUTTONPANEL, TEXTPANEL };
                              JPanel cbp = new JPanel();
                              JComboBox c = new JComboBox(comboBoxItems);
                              c.setEditable(false);
                              c.addItemListener(this);
                              cbp.add(c);

                              //Use the default layout manager, BorderLayout
                              contentPane.add(cbp, BorderLayout.NORTH);

                              cards = new JPanel();
                              cards.setLayout(new CardLayout());
          ‘‘
                              JPanel p1 = new JPanel();
                              p1.add(new JButton("Button 1"));
                              p1.add(new JButton("Button 2"));
                              p1.add(new JButton("Button 3"));

                              JPanel p2 = new JPanel();
```

```
                    p2.add(new JTextField("TextField", 20));

                    cards.add(p1, BUTTONPANEL);
                    cards.add(p2, TEXTPANEL);
                    contentPane.add(cards, BorderLayout.CENTER);

                    addWindowListener(new WindowAdapter() {
                              public void windowClosing(WindowEvent e) {
                                        if (inAnApplet) {
                                                  dispose();
                                        } else {
                                                  System.exit(0);
                                        }
                              }
                    });
          }

          public void itemStateChanged(ItemEvent evt) {
                    CardLayout cl = (CardLayout)(cards.getLayout());
                    cl.show(cards, (String)evt.getItem());
          }

          public static void main(String args[]) {
                    CardWindow window = new CardWindow();
                    window.inAnApplet = false;

                    window.setTitle("CardLayout");
                    window.pack();
                    window.setVisible(true);
          }
}
```

Practice 9.9

```
//tab window example

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
public class TabWindow extends JFrame {
        boolean inAnApplet = true;

        final static String BUTTONPANEL = "JPanel with JButtons";
        final static String TEXTPANEL = "JPanel with JTextField";

        public TabWindow() {
                Container contentPane = getContentPane();

                JTabbedPane tabbedPane = new JTabbedPane();

                JPanel p1 = new JPanel() {
                        //Force the window to be 400+ pixels wide.
                        public Dimension getPreferredSize() {
                                Dimension size = super.getPreferredSize();
                                size.width = 400;
                                return size;
                        }
                };
                p1.add(new JButton("Button 1"));
                p1.add(new JButton("Button 2"));
                p1.add(new JButton("Button 3"));
                tabbedPane.addTab(BUTTONPANEL, p1);

                JPanel p2 = new JPanel();
                p2.add(new JTextField("TextField", 20));
                tabbedPane.addTab(TEXTPANEL, p2);

                contentPane.add(tabbedPane, BorderLayout.CENTER);

                addWindowListener(new WindowAdapter() {
                        public void windowClosing(WindowEvent e) {
                                if (inAnApplet) {
                                        dispose();
                                } else {
                                        System.exit(0);
                                }
                        }
                });
        }
```

```
                public static void main(String args[]) {
                        TabWindow window = new TabWindow();
                        window.inAnApplet = false;

                        window.setTitle("TabWindow");
                        window.pack();
                        window.setVisible(true);
                }
        }
```

Practice 9.10

```
        //Create an applet for digital clock

        DigitalClockApplet.java

        import java.awt.Container;
        import java.awt.Dimension;
        import java.util.Date;
        import javax.swing.BorderFactory;
        import javax.swing.JApplet;
        import javax.swing.JCheckBox;
        import javax.swing.JLabel;
        import javax.swing.JPanel;
        import javax.swing.SwingConstants;

        public class DigitalClockApplet extends JApplet {
                ClockPanel panel;

                public void init() {
                        Container contentPane=getContentPane();
                        panel = new ClockPanel(650,150);
                        panel.resize(panel.getSize());
                        contentPane.resize(panel.getSize());
                        this.resize(panel.getSize());
                        contentPane.add(panel);
                        TimeThread tt = new TimeThread(panel);
                        tt.start();
                }
        }
```

```
class ClockPanel extends JPanel
{
           public JCheckBox jDate;
           public JCheckBox jDay;
           public JLabel jLabel1;
           public JCheckBox jSec;
           public boolean RUNCLOCK;
           ClockPanel(int wid, int hgt) {
                      jLabel1 = new JLabel();
                      jDay = new JCheckBox();
                      jDate = new JCheckBox();
                      jSec = new JCheckBox();
                      this.setLayout(null);
                      this.setSize(new Dimension(wid, hgt));
                      this.setPreferredSize(new Dimension(wid, hgt));
                      this.setBounds(0, 0, wid, hgt);
                      jLabel1.setFont(new java.awt.Font("Courier 10 Pitch", 1, 24)); // NOI18N
                      jLabel1.setForeground(new java.awt.Color(71, 113, 189));
                      jLabel1.setHorizontalAlignment(SwingConstants.CENTER);
                      jLabel1.setText("jLabel1");
                      jLabel1.setBorder(BorderFactory.createEtchedBorder());
                      this.add(jLabel1);
                      jLabel1.setBounds(30, 30, 500, 50);

                      jDay.setText("Show Day");
                      this.add(jDay);
                      jDay.setBounds(30, 100, 150, 22);

                      jDate.setText("Show Date");
                      this.add(jDate);
                      jDate.setBounds(190, 100, 180, 22);

                      jSec.setText("Show Seconds");
                      this.add(jSec);
                      jSec.setBounds(380, 100, 160, 22);

                      this.jDate.setSelected(true);
                      this.jDay.setSelected(true);
                      this.jSec.setSelected(true);
```

```
		}
	}


	class TimeThread extends Thread implements Runnable {
		ClockPanel cp = null;
		private int day;
		private String dayn;
		private String dat;
		private int sec;
		private int min;
		private int hours;
		public TimeThread (ClockPanel cfArg) {
			cp = cfArg;
			cp.RUNCLOCK = true;
		}

		public void run() {
			String dt = "";
			while (cp.RUNCLOCK) {
				Date date = new Date();
				min = date.getMinutes();
				hours = date.getHours();
				if (cp.jDay.isSelected()) day = date.getDay();
				else day=-1;
				//GET DAY NAME
				dayn="";
				if (day!=-1) {
					switch (day) {
						case 0: dayn = "Sunday"; break;
						case 1: dayn = "Monday"; break;
						case 2: dayn = "Tuesday"; break;
						case 3: dayn = "Wednesday"; break;
						case 4: dayn = "Thursay"; break;
						case 5: dayn = "Friday"; break;
						case 6: dayn = "Saturday"; break;
						default: dayn = "OOPS!"; break;
					}
				}
```

```
                                        //IS DATE REQD
                                        if (cp.jDate.isSelected()) dat = date.getDate() + "-" +
date.getMonth() + "-" + date.getYear();
                                        else dat="";

                                        dt = dayn + " " + dat + "‘‘" + hours + ":" + min;
                                        if (cp.jSec.isSelected()) {
                                                  sec = date.getSeconds();
                                                  dt += ":" + sec + "";
                                        }
                                        else sec=-1;
                                        //dt = date.toString();
                                        cp.jLabel1.setText(dt);
                                        try {
                                                  Thread.sleep(10);
                                        } catch (InterruptedException ex) {
                                                  System.out.println("Exception: " + ex.toString());
                                        }
                              }
                    }
          }

          DigitalClockApplet.htm

          < HTML>
          < HEAD>
          < / HEAD>
          < BODY BGCOLOR="000000">
          < CENTER>
          < applet code="DigitalClockApplet.class" WIDTH=660 HEIGHT=150>
          < / applet>
          < / CENTER>
          < / BODY>
          < / HTML>
```

Practice 9.11

```
          // Create an Application for Digital Clock

          import java.awt.Container;
          import java.util.Date;
```

```
          import javax.swing.BorderFactory;
          import javax.swing.JCheckBox;
          import javax.swing.JFrame;
          import javax.swing.JLabel;
          import javax.swing.JPanel;
          import javax.swing.SwingConstants;
          import javax.swing.WindowConstants;

          public class DigitalClock extends JFrame {
                    ClockPanel panel;
                    public DigitalClock() {
                              this.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
                              Container contentPane=this.getContentPane();
                              panel = new ClockPanel();
                              contentPane.add(panel);
                              java.awt.Dimension screenSize =‘‘‘‘
                              java.awt.Toolkit.getDefaultToolkit().getScreenSize();
                              this.setBounds((screenSize.width-573)/2, (screenSize.height-172)/2, 573,
172);
                    }

                    public static void main(String args[]) {
                               DigitalClock dc =‘‘new DigitalClock();
                               dc.setVisible(true);
                               TimeThread tt = new TimeThread(dc.panel);
                               tt.start();
                    }
          }
```

Practice 9.12

```
          //Create an applet for calculator

          CalcApplet.java

          import java.awt.*;
          import java.awt.event.*;
          import javax.swing.*;

          public class CalcApplet extends JApplet
          {
                    public void init()
```

```
                {
                Container’‘contentPane=getContentPane();
                CalculatorPanel panel=new CalculatorPanel();
                contentPane.add(panel);
                }
        }

        class CalculatorPanel extends JPanel
        {
                public CalculatorPanel()
                {
                setLayout(new BorderLayout());

                result = 0;
                lastCommand = "=";
                start = true;

                display = new JTextField("0.0");
                          display.setEditable(false);
                add(display,BorderLayout.NORTH);

                ActionListener insert = new InsertAction();
                ActionListener command = new CommandAction();

                panel = new JPanel();
                panel.setLayout(new GridLayout(8,4));

                addButton("sin",command);
                addButton("cos",command);
                addButton("tan",command);
                addButton("A/C",command);
                addButton("asin",command);
                addButton("acos",command);
                addButton("atan",command);
                addButton("n!",command);
                addButton("log",command);
                addButton("ln",command);
                addButton("Pi",insert);
                addButton("e",insert);
                addButton("x^(1/y)",command);
                addButton("x^y",command);
```

```
                addButton("Sqrt",command);
                addButton("/",command);
                addButton("9",insert);
                addButton("8",insert);
                addButton("7",insert);
                addButton("x",command);
                addButton("6",insert);
                addButton("5",insert);
                addButton("4",insert);
                addButton("+",command);
                addButton("3",insert);
                addButton("2",insert);
                addButton("1",insert);
                addButton("_",command);
                addButton("0",insert);
                addButton("+/-",command);
                addButton("=",command);
                addButton(".",insert);

                add(panel,BorderLayout.CENTER);
                }

                private void addButton(String label,ActionListener listener)
                {
                JButton button = new JButton(label);
                button.addActionListener(listener);
                panel.add(button);
                }


        private class InsertAction implements ActionListener
        {
                public void actionPerformed(ActionEvent event)
                {
                String input = event.getActionCommand();
                        if(start)
                        {
                        display.setText("");
                        start = false;
                        }
                ‘‘if(input.equals("e"))
```

```
                    {
                    ‘‘if(display.getText().equals("-"))
                    ‘‘ display.setText("-"+Math.E);
                              else
                              display.setText(""+Math.E);
                    ‘‘start=true;
                    }
          ‘‘else if(input.equals("Pi"))
                    {
                              if(display.getText().equals("-"))
                              ‘‘display.setText("-"+Math.PI);
                              else
                              ‘‘display.setText(""+Math.PI);
                    start=true;
                    }
          else
                    display.setText(display.getText()+input);
          }
}

private class CommandAction implements ActionListener
{
          public void actionPerformed(ActionEvent event)
          {
          String command = event.getActionCommand();
          String co = command;
          if(co.equals("A/C"))
                    {
                    result = 0;
                    display.setText(""+result);
                    lastCommand = "=";
                    start=true;
                    }
          else if((co.equals("+/-"))||(co.equals("Sin"))
                              ‘‘||(co.equals("Cos"))||(co.equals("Tan"))||(co.equals("Sqrt"))
                              ‘‘||(co.equals("n!"))||(co.equals("ln"))||(co.equals("aSin"))
                              ||(co.equals("aCos"))||(co.equals("aTan"))||(co.equals("log")))
                    ‘‘ {
                              lastCommand=command;
                              ‘‘calculate(Double.parseDouble(display.getText()));
                              ‘‘lastCommand="=";
```

```
                                        ‘‘ start=true;
                                        }
                                        ‘‘else if(start&&(lastCommand.equals("="))&&(result==0)
                                                                                ‘‘&&(display.getText().
equals("0.0")))
                                        ‘‘{
                                        if(command.equals("_"))
                                         {
                                        display.setText("-");
                                        start = false;
                                         }
                                        else
                                        ‘‘lastCommand = command;
                                         }
                    else
                                        {
                              ‘‘ calculate(Double.parseDouble(display.getText()));
                                         lastCommand=command;
                                         start= true;
                              }
                    }
          }

          public void calculate(double x)
          {
                    if(lastCommand.equals("+")) result += x;
                    else if(lastCommand.equals("_")) result -= x;
                    else if(lastCommand.equals("x")) result *= x;
                    else if(lastCommand.equals("/")) result /= x;
                    else if(lastCommand.equals("+/-")) result = -x;
                    else if(lastCommand.equals("Sin")) result = Math.sin(x*Math.PI/180.0);
                    else if(lastCommand.equals("Cos")) result = Math.cos(x*Math.PI/180.0);
                    else if(lastCommand.equals("Tan")) result = Math.tan(x*Math.PI/180.0);
                    else if(lastCommand.equals("=")) result = x;
                    else if(lastCommand.equals("x^y")) result = Math.pow(result,x);
                    else if(lastCommand.equals("Sqrt")) result = Math.sqrt(x);
                    else if(lastCommand.equals("n!")) result = fact((int)x);
                    else if(lastCommand.equals("ln")) result = Math.log(x);
                    else if(lastCommand.equals("aSin")) result = Math.asin(x)*180/Math.PI;
                    else if(lastCommand.equals("aCos")) result = Math.acos(x)*180/Math.PI;
                    else if(lastCommand.equals("aTan")) result = Math.atan(x)*180/Math.PI;
```

```
                    else if(lastCommand.equals("log")) result = Math.log(x)/Math.log(10.0);
                    else if(lastCommand.equals("x^(1/y)")) result = Math.pow(result,1.0/x);

                    display.setText(""+result);
          }

           private int fact(int n)
                    {
                              if((n==0)||(n==1)) return 1;
                              else return n*fact(n-1);
                    }

          private JTextField display;
          private JPanel panel;
          private double result;
          private String lastCommand;
          private boolean start;
          }

          CalcApplet.htm
          < HTML>
          < HEAD>
          < / HEAD>
          < BODY BGCOLOR="000000">
          < CENTER>
          < applet code="CalcApplet.class"WIDTH=400 HEIGHT=400>
          < / applet>
          < / CENTER>
          < / BODY>
          < / HTML>
```

Practice 9.13

```
          //Create an application for calculator

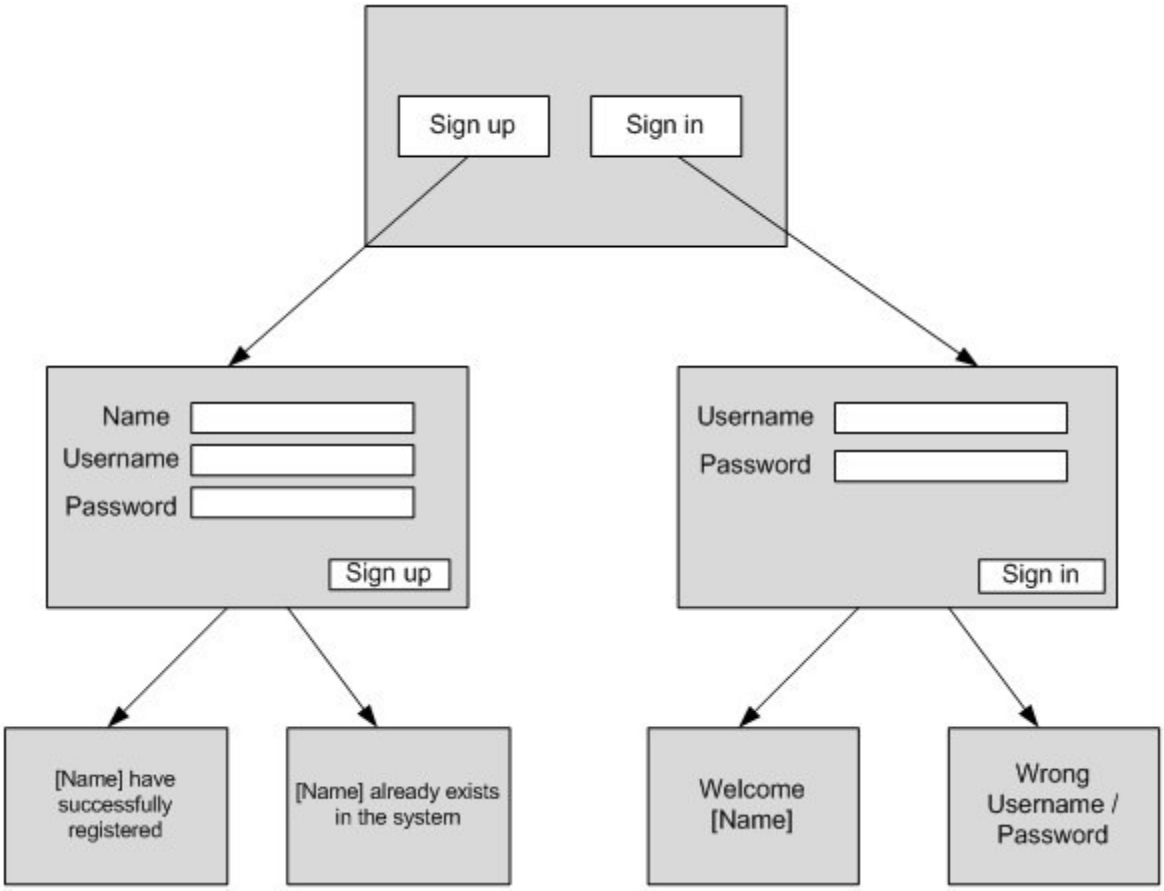
          import java.awt.BorderLayout;
          import java.awt.Container;
          import java.awt.GridLayout;
          import java.awt.event.ActionEvent;
          import java.awt.event.ActionListener;
          import javax.swing.JButton;
```

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class CalcApplication {
          public static void main(String args[]) {
                    JFrame’‘mainFrame = new JFrame();
                    mainFrame.setSize(400, 300);
                    Container’‘contentPane=mainFrame.getContentPane();
          CalculatorPanel panel=new CalculatorPanel();
          contentPane.add(panel);
                    mainFrame.setVisible(true);
          }
}
```

Assignment

Develop a swing application for login validation of a system as shown in the figure. The application shows two options: one ﷺign up﷽ and another ﷺign in﷽. When ﷺign up﷽ is selected, a new screen will appear for asking name, username, password. If the entered username and password do not exist in the database, it will show a successful message ﷺ[Name] has successfully registered﷽ and store corresponding username, password in the database, otherwise the message ﷺ[Name] already exists in the system﷽ will be displayed. On the other hand, when ﷺign In﷽ option is selected, the user will be prompted to enter username, password as shown in the figure. Eventually, the application checks whether username and password both match with some entry in the database. If such entry found in the database, then a welcome message ﷺelcome [Name]﷽ will appear, otherwise an unsuccessful login message ﷺrong username/password﷽ will be displayed. For message display, the swing dialogs can be used. You should have some login information in your database so that all four situations can be validated.



Q&A

Q: What is difference between invokeAndWait and invokeLater in Java?

A: Swing is not thread-safe and at same time you can not perform time consuming task in EDT thread. InvokeAndWait and InvokeLater method allows to enqueue task for EDT thread to perform, InvokeAndWait is a blocking method in Java and does it synchronously and invokeLater does it asynchronously. Since GUI can only be updated in Event dispatcher thread, if you need to show progress of any task, you can use these two methods.

Q: What is EDT thread in Swing?

A: EDT stands for Event dispatcher thread. EDT is one of the most important thing to learn about Swing, Since Swing is single-threaded all GUI drawing and event handling is done in EDT thread and that's why its recommended not to perform any time consuming task e.g. connecting to database or opening network connection in Event Dispatcher thread or EDT thread. If you do that, it may lead to frozen or hung GUI. This question leads to several other questions in Java, e.g. If you can not open database connection in EDT thread than how will you open db connection with button click etc. well this can be done by spawning a new thread from button click and opening db connection there.

Q: How to change a button from enable to disable after click ?

A: When button is clicked an action event is fired which can be captured by implementing ActionListener interface and actionPerformed(ActionEvent ae) method. You can then call button.setEnable(false) to disable this button.

Q: Why Swing is called light weight ?

A: Most of Swing component are inherited form JComponent and doesn't required a native peer and that's why they are referred as light weight component. light weight component implement look and feel in Java rather than using look and feel of native component and that's why Swing's look and feel remains same across platform.

Q: Is Swing Thread safe in Java ?

A: No, Swing is not thread-safe in Java. Swing components are not thread-safe they can not be modified from multiple threads. All swing components are designed to be updated by using Event dispatcher thread or EDT thread

Q: Which method of Swing are thread-safe?

A: Only couple of methods like repaint() and revalidate() are thread-safe in Swing, i.e. they can be called from multiple threads without additional synchronization in Java.

Q: What is difference between Container and Component ?

A: Main difference between Container and Component is that former can hold other components e.g. JFrame which is used as container to hold other components e.g. JButton.

Q: What is the purpose of action interface in Swing?

A: Action is performed on a state to allow it to change. It defines the interface that it is implementing. The library that is used for the action interface is javax.swing.Action. This action interface extends the ActionListener interface class that is being provided from the AWT. Action interface allow the concrete classes to implement the actionPerformed() method and provide the action that is associated with it. The use of actionPerformed() method allow the implementation of the behavior that is desired in the programming and allow it to show the functionality of the action class. The action can be added to the container class that accepts the parameters on an event like JMenu, JPopupMenu, or JtoolBar. The container used in this automatically registers the action that is taken on the event and it acts as an ActionListener of the user interface.

Q: Explain Thread Rule in Swing?

A: Once a swing component has been realized, i.e. the component has been painted on screen or it is ready to painted, all the code that might affect or depend on the state of that component should be executed in the event dispatching thread. This is called thread rule in swing..

Q: What is the use of double buffering in Swing?

A: Double-buffering is the process of drawing graphics into an off-screen image buffer and then copying the contents of the buffer to the screen all at once. For complex graphics, using double-buffering can reduce flickering.

Q: What is the purpose of Serialization in Swing?

A: Serialization is used when an object extends the Jcomponent class. The object's data in this case points out to the stream that is written or serialized and gives an output stream. This output stream can be sent through then network and it can be

saved in the file if required. The serialized file can be de-serialized in the memory. This will allow the operation to continue to perform the original state functions that were getting used. The serialization used in object allows easy retrieval of the object data and easy transmission of state data from one place to another. This can also be placed in custom made storage files that provide the transfer of the components to be fast and it uses the concept of virtual machine that transfers from one machine to another machine. It uses the remote method invocation to allow the use of distributed computing that provide with some exceptions to the use of transient keyword. The components can be serialized using the classes that can pass the reference to an object using the writeObject() method of the ObjectOutputStream object this way it calls the serialization function recursively.

Q: How do you classify Swing Components?

A: Swing components are classified under the following headings:

```
1.Top level containers – The containers at the top of any swing component hierarchy are:

Applet

Dialog

Frame

2.General purpose containers – The intermediate containers are

Panel

Scroll pane

Split pane

Tabbed pane

Tool bar

3.Special purpose containers – These are intermediate containers which play specific roles in the
user interface:

Internal frame

Layered pane

Root pane

4.Basic controls : The atomic components that exist primarily to get input from the user are

Buttons

Combo box

List

Menu

Slider

Spinner

TextField

5.Uneditable information displays: The atomic components which give information to the user are

Label

Progress bar

Tool tip

6.Interactive displays of highly formatted information – The atomic components that display
formatted information that can be modified by the user are
```

```
Color chooser
File chooser
Table
Text
Tree
```

Q: What is the main difference between Dialog and Frame?

A: In AWT, the top level window, which is itself not contained in another window, is called a FRAME.
On the other hand, a window that appears over existing window (mostly Frame) and forces the user to respond is called a DIALOG.

```
Frame f = new Frame(); // creates an object of type Frame.
```

A FRAME can exists on its own , where as a DIALOG cannot exist on its own. In other words, FRAME is a primary window, where as the DIALOG box is secondary window. The dialog boxes are of two types:

□'‘modal This dialog won't let the user interact with the remaining containers/windows until he deals with it.

□'‘modeless - The user can interact with other windows as well as the dialog box, e.g. the toolbox.
There is no default constructor for a DIALOG. The following statement constructs a dialog box:

```
public Dialog( Frame parent, String title, boolean modal) Here:

parent  The container to which the dialog belongs.

title  Title of the dialog

modal  True for modal type, false for modeless type.
```

Chapter 11

JDBC

by Debasis Samanta

CONTENTS

- Introduction
- MySQL Basics and setup installation
- JDBC: Java Database Connectivity
- JDBC: Characteristics
- Relational database
- Types of JDBC Drivers
- JDBC Architectures
- Inside JDBC
- Connecting to Database
- Examples

Introduction

MySQL Basics and installation

ꕥ ’‘SQL stands for Structured Query Language. It is a kind of language(mostly like English). <li< style="box-sizing: border-box;">SQL is undoubtedly the most popular and widely-used open source Database Management System (DBMS) language.</li<>
ꕥ ’‘It is simple to set up and use.
ꕥ ’‘SQL is mainly used to create a table, enter data into table, update records in a table and retrieving data from a table.
NOTE: SQL is not a programming language like C, C++, Java, etc. SQL is not case sensitive.



MySQL:

```
MySQL is one of the most popular Database Management System (DBMS).
MySQL is a database system used on the web.
MySQL is a database system that runs on a server.
MySQL is ideal for both small and large applications.
MySQL is very fast, reliable, and easy to use.
MySQL uses standard SQL.
MySQL works on a number of platforms.
MySQL is free to download and use.
```

```
MySQL is developed, distributed, and supported by Oracle Corporation.

MySQL is named after co-founder Monty Widenius's daughter: My
```

MySQL Setup:

```
1. Download: dev.mysql.com/downloads’‘‘‘ -->‘‘‘‘‘‘MySQL Community Server’‘ -->‘‘‘‘‘‘Archive Version (64/36 bit as per your OS)


2. Extract the file in C:// drive.


3. Rename the directory to 'mysql'


4. Create the data folder in D:// drive. [D://data/] (Storing data in a different partition ensures data Backup)


5. Create my.ini with the following details:
                    ---------------------------------------------
                                        [mysqld]

                                        # installation directory
                                        basedir="c:\mysql\"

                                        # data directory
                                        datadir="d:\data\"
                    ---------------------------------------------



6. Start command prompt and change directory to C://mysql/bin/


7. Type the command: mysqld --console --initialize (Remember the password generated)
                                ----------------------------
                                ‘‘TEMP PASS:
                                ----------------------------



8. Open another command prompt and change directory to C://mysql/bin/


9. Type the command: mysql -u root -p


10. Enter the password from step 7.


11. Type the command: ALTER USER 'root'@'localhost' IDENTIFIED BY 'root'; (Our new password for root user will be 'root')
```

```
12. Type the command: EXIT (This will exit the console)

13. Now start a new MySql console as in Step 9 with the password set in Step 11.

That's It!
```

MySQL Statements:
CREATE:

```
To begin with, the table creation command requires the following details-
Name of the table
Name of the fields
Definitions for each field
Syntax
                    CREATE TABLE table_name (
                              column_name1 column_type1,
                              column_name2 column_type2...);
```

DESC:
In practice, you use the DESC statement which is a shorthand of the DESCRIBE statement. These statements are used to view the structure, datatype, keys and constraints used in a table.

```
Syntax
‘‘ DESC table_name;
```

INSERT
You can insert data into the existing MySQL table by using the mysql> prompt or by using any script like PHP or any programming language using proper drivers using the INSERT statement.

```
Syntax
          INSERT INTO table_name VALUES(Value1,Value2,..., Value n);
```

UPDATE
UPDATE statement is used to modify previously inserted data in a table.

```
Syntax
          UPDATE table_name SET
                    ColName1=Value1,
                    ColName2=Value2,
                    ,...,
                    WHERE ColName1=Value1;
```

SELECT

SELECT statement is used to retrieve data from a table.

```
Syntax
          SELECT < colname 1>,< colname 2>,...,< colname p>
          FROM table_name
          WHERE < Condition>;
```

DELETE
DELETE FROM statement is used to delete a record from any MySQL table.

```
Syntax
                    DELETE FROM table_name
                    WHERE < Condition>;
```

DROP
DROP TABLE statement is used to drop an existing MySQL table, but you need to be very careful while deleting any existing table because the data lost will not be recovered after deleting a table.

```
Syntax
                    DROP TABLE table_name;
```

```
Illustration 11.1
//Performing simple SQL operations in MySQL console (CREATE,INSERT,UPDATE,DROP etc)

create DATABASE test;
use test;

create table JavaCourse(
          Roll Integer primary key,
          Name Varchar(30),
          Marks Integer not null,
          Grade Varchar(2));
‘‘‘‘
desc JavaCourse;

insert into JavaCourse values (01,'Debasish', 75, 'A');
insert into JavaCourse values(02,'Nilanjan', 85, 'EX');
insert into JavaCourse values(03,'Tauheed', 65, 'B');
insert into JavaCourse values(04,'Priyabrata', 78, 'A');

update JavaCourse set Name='Debasis' where Name='Debasish';’‘
```

```
update JavaCourse set Marks=85, Grade='Ex' where Name='Debasis';

select * from JavaCourse;

select name, roll from JavaCourse;

select * from JavaCourse where marks>80;

select * from JavaCourse where name like ‘%jee%‘;

select * from JavaCourse order by marks;

delete from JavaCourse where marks < 66;

delete from JavaCourse;

drop table JavaCourse;
```

JDBC: Java Database Connectivity

```
JDBC is a standard Java API for handling database related activities.

In Java, there is a package java.sql having number of classes for database related programming.

It includes java.sql.DriverManager class and two interfaces java.sql.Driver and java.sql.Connection.
```

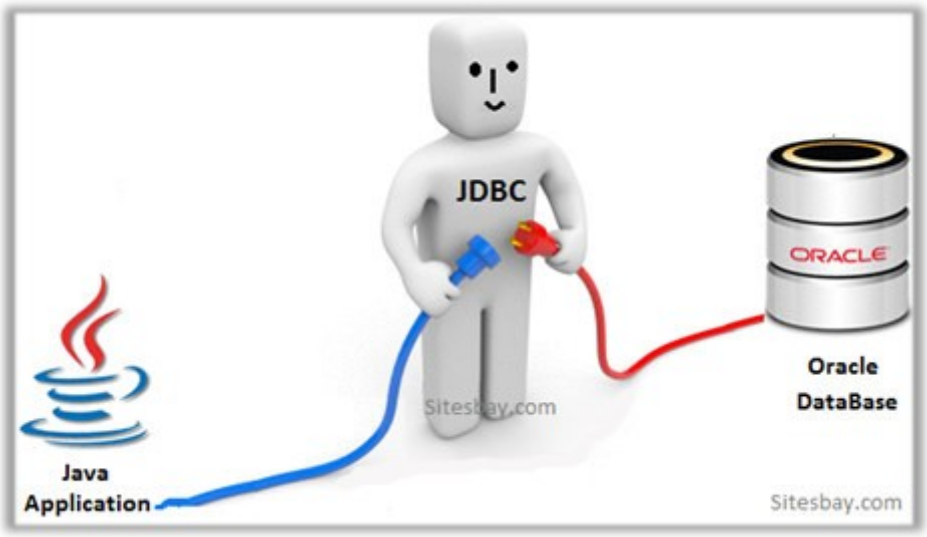


Illustration 11.2

// Downloading and extracting the latest version of JDBC driver

Download link:

https://dev.mysql.com/downloads/connector/j/

Steps:

1. Extract the ZIP file.
2. Find the'JDBC MySQL Connector.jar' file and rename to 'mysql' for convinience.
3. Place the mysql.jar file along with any program using JDBC and use the following command during execution:

```
java -cp mysql.jar;.
```

Note: Compilation doesnot require the Classpath of JDBC driver.

JDBC: Characteristics:

JDBC is a SQL (Structured Query Language)-level API, which is very popular for RDBMS (Relational Database Management System).

It is compatible with the most of the popular database management systems, namely OracleDB, MySQL, Sybase, Microsoft SQL, etc.

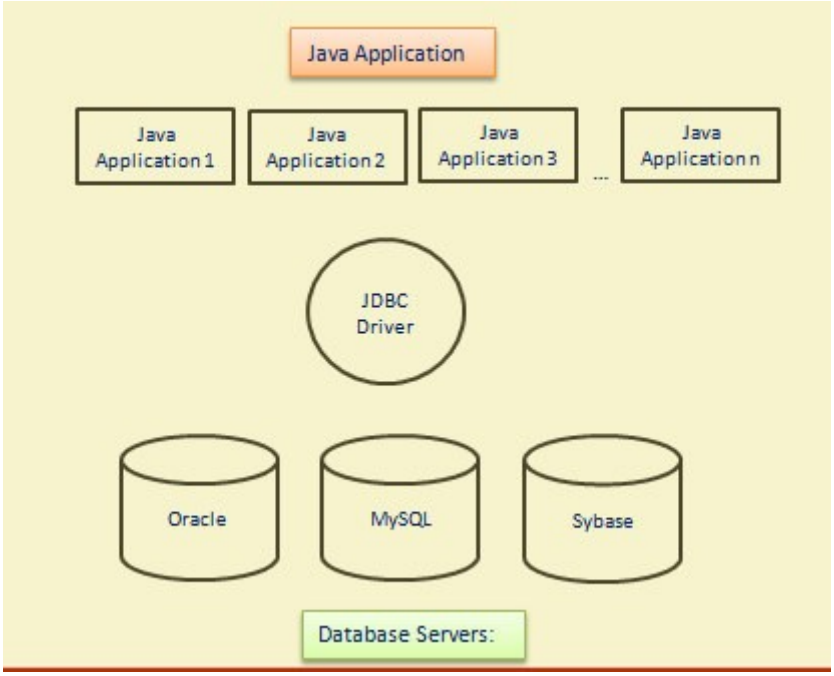
It is simple and easy to implement.

In this course, we shall refer to JDBC to connect to MySQL database server.

The approach is same to any other database.

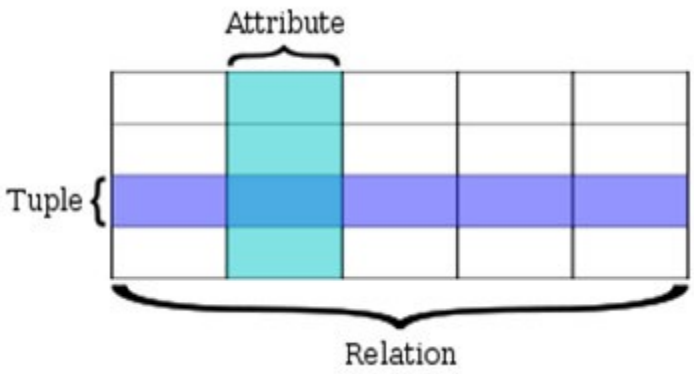
Note:

The JDBC driver for different database is different. But, as an end-user, we don't have to bother about their implementation.



Relational database:

- ''A relational database is a database that allows for queries which typically use Structured Query Language (SQL) to store and retrieve data.
- ''A relational database stores information by means of tables. A table is referred to as a relation in the sense that it is a collection of objects of the same type (rows).Examples: MS SQL Server, IBM DB2, Oracle, MySQL , etc.



Why JDBC?

```
Write once, run anywhere
‘‘ Multiple client and server platforms.


Object-relational mapping
‘‘ Databases optimized for searching/indexing.
‘‘ Objects optimized for engineering/flexibility.


Network independence
‘‘ Works across Internet Protocol.


Database independence
‘‘ Java can access any database vendor.
```

JDBC: Structure:

```
JDBC is a SQL-level API. It means that the JDBC allows to construct SQL statements and embed them inside
Java API calls.

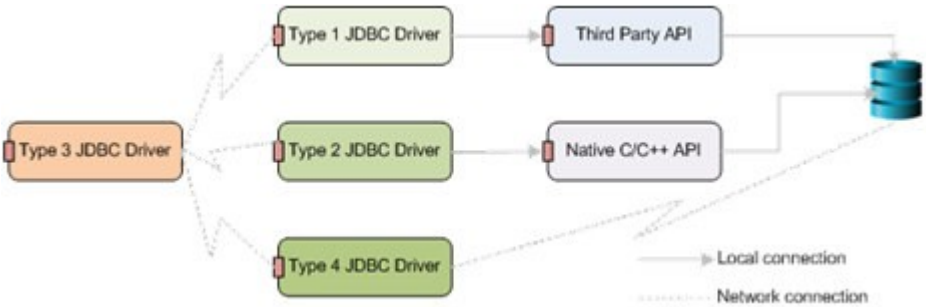


The JDBC API is an implementation to interact a particular database engine. This implementation is called
JDBC Driver.
```

Types of JDBC Drivers:

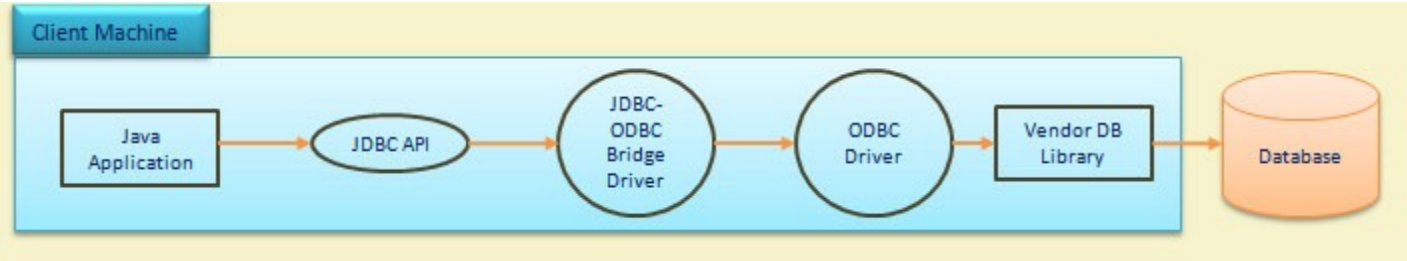
JDBC Driver is a software component that enables Java application to interact with the database.

```
There are 4 types of JDBC drivers:
Type 1 : JDBC-ODBC bridge driver
Type 2 : Native-API driver (partially Java driver)
Type 3 : Network Protocol driver (fully Java driver)
Type 4 : Thin driver (fully Java driver)
```



JDBC: Type 1 JDBC-ODBC

Type - I : JDBC-ODBC bridge driver
The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.



Note: Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

```
Advantages:

’‘Easy to use.

’‘Can be easily connected to any database.



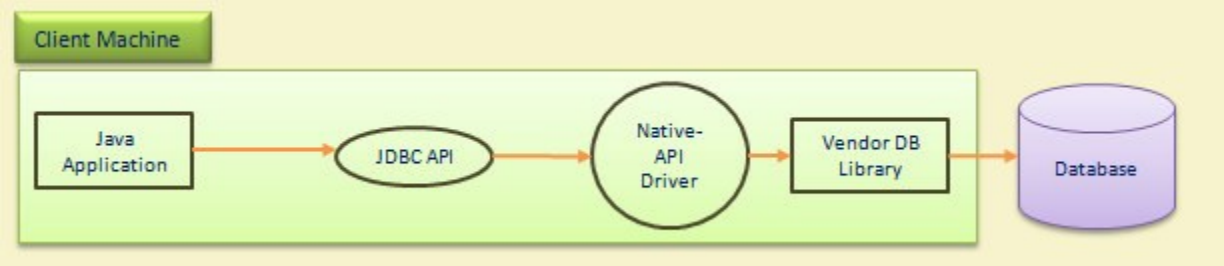
Disadvantages:

’‘Performance degraded because JDBC method call is converted into the ODBC function calls.

’‘The ODBC driver needs to be installed on the client machine.
```

JDBC: Type 2 - Native-API

Type - II : Native-API driver (partially Java driver)
The native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.




```
Advantages:

’‘Performance upgraded than JDBC-ODBC bridge driver.
```

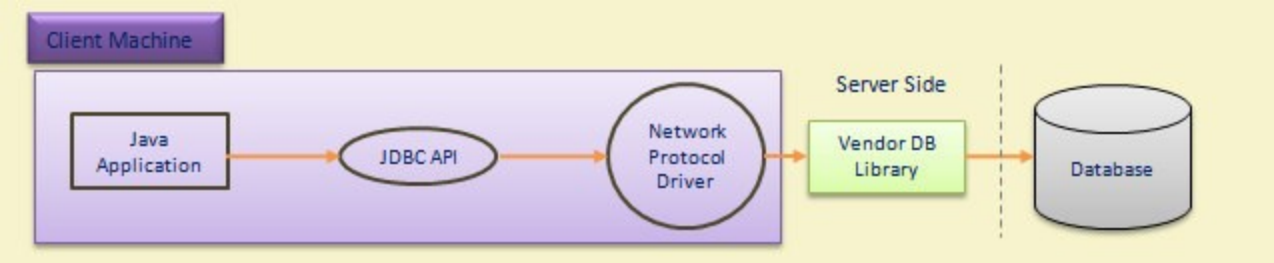
Disadvantages:

’‘The native driver needs to be installed on the each client machine.

’‘The vendor client library needs to be installed on client machine.

JDBC: Type 3 - Network Protocol

Type - III : Network Protocol driver (fully Java driver)
The network protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.



Advantages:

’‘No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantages:

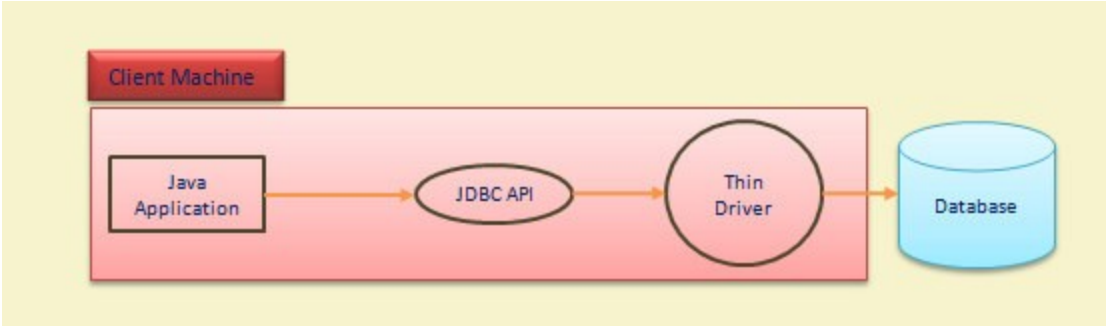
’‘Network support is required on client machine.

’‘Requires database-specific coding to be done in the middle tier.

’‘Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

JDBC: Type 4 - Thin Driver

Type -IV : Thin driver (fully Java driver)
The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.



Advantages:

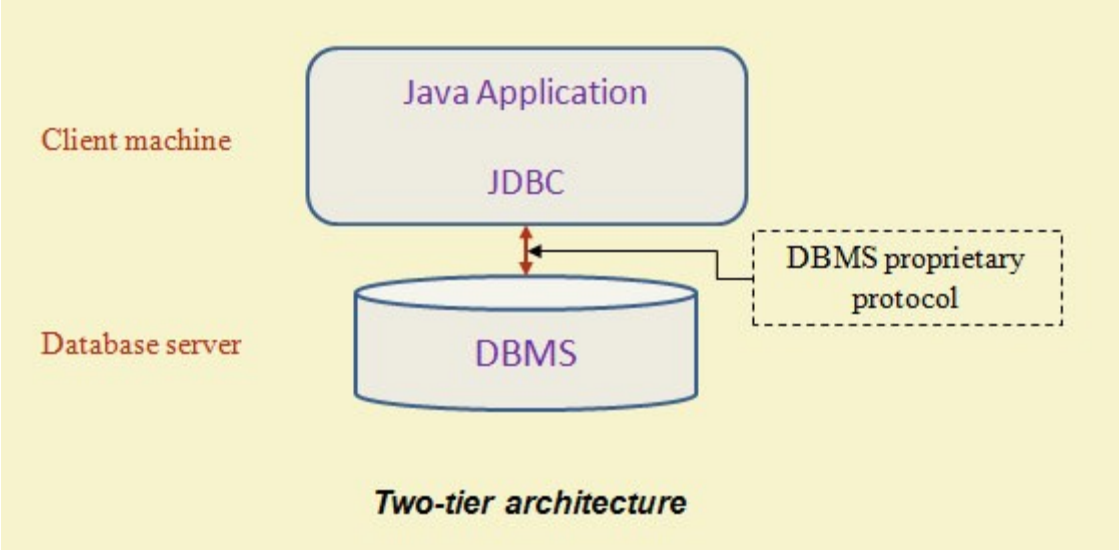
’‘Better performance than all other drivers.

’‘No software is required at client side or server side.

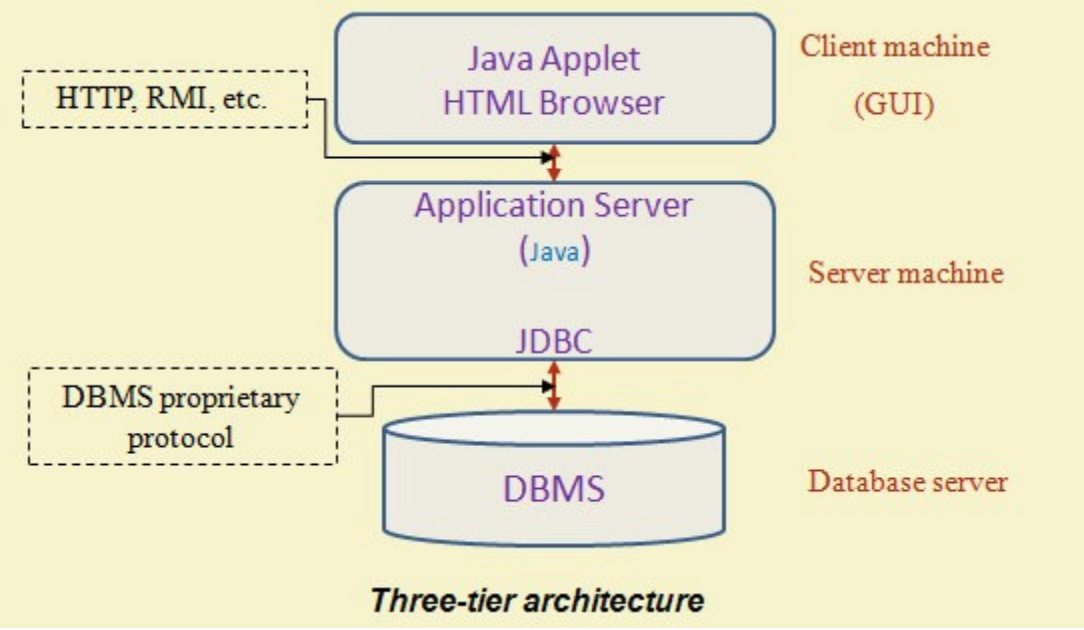
```
Disadvantages:
’‘Drivers depend on the database.
```

JDBC Architectures:

JDBC Architecture : Two-tier



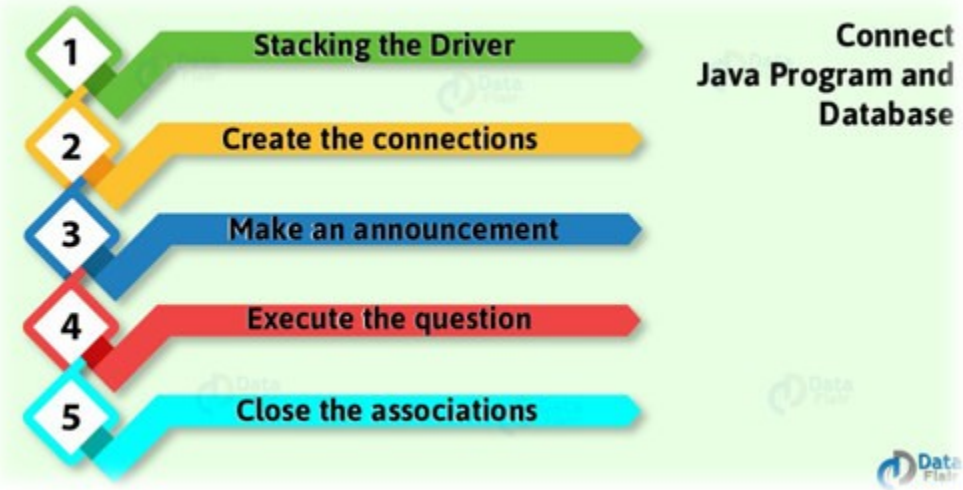
JDBC Architecture : Three-tier



Inside JDBC:

JDBC steps

```
Load a JDBC driver.
Create connections.
Connect to the data source.
Execute SQL statement(s).
Map the results to data structures.
```



JDBC driver

```
Acts as the gateway to a database.


Not actually a Windows "driver", it is just a .jar file.


For MySQL JDBC driver:
‘‘‘‘‘‘‘‘‘‘‘‘‘‘ mysql-connector-java-8.0.12-bin.jar (current)
```

JDBC driver installation

```
Download the driver, extract .jar file and add its path into your $CLASSPATH


Linux: use command:
‘‘‘‘ export CLASSPATH=$CLASSPATH::.


Windows: Add the path of .jar file to system variable CLASSPATH
```

Connecting to Database:

Common JDBC components

Class/Interface	Description
DriverManager	This **class** manages a list of database drivers. Matches connection requests from the Java app proper database driver using communication sub protocol. The first driver that recognizes a cer under JDBC will be used to establish a database Connection.
Driver	This **interface** handles the communications with the database server. You will interact dire objects very rarely. In stead, you use DriverManager objects, which manages objects of this type the details associated with working with Driver objects.
Connection	This **interface** with all methods for contacting a database. The connection object represents context, i.e., all communication with database is through connection object only.
Statement	You use objects created from this **interface** to submit the SQL statements to the databas interfaces accept parameters in addition to executing stored procedures.
ResultSet	This **class** retrieves data from a database after you execute an SQL query using Statement obj iterator to allow you to move through its data.
SQLException	This **class** handles any errors that occur in a database application.

Loading JDBC driver

```
For MySql:
‘‘‘‘‘‘    Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();

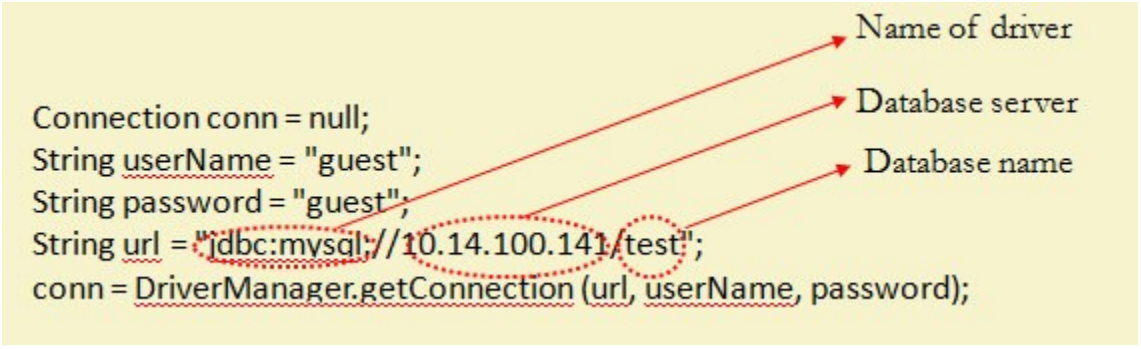
For Oracle:’‘
‘‘‘‘‘‘    Class.forName("oracle.jdbc.driver.OracleDriver").newInstance();

For MS SQL Server :’‘
‘‘‘‘‘‘ Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver").newInstance();
```

Establishing connection

```
Create a Connection object

Use the DriverManager to grab a connection with the getConnection() method
```



Three types of statements

Types	Description
Statement	For executing a simple SQL statement
PreparedStatement	For executing a precompiled SQL statement
CallableStatement	For executing a database stored procedure

Executing SQL statements
Create a statement object from the connection

```
Statement stmt = null;
stmt = conn.createStatement();

stmt.execute("show tables");
stmt.execute("insert into JavaCourse values (01,'Debasis', 85, 'Ex')");
```

Useful methods in Statement class

Methods	Description
executeQuery()	Executes SQL query and returns the data in a table (ResultSet) object. This method is SQL command that expects a return data from a database.
executeUpdate()	Used to execute INSERT, UPDATE, DELETE, CREATE TABLE, DROP TABLE, ALTER Returns the number of rows that are affected in the database
execute()	Generic method for executing simple statements, stored procedures, prepared statements. used when the statement is either related to query or update. This method returns true(*if q a row*) otherwise returns false.
getMaxRows()	Determines the number of rows a ResultSet can contain.

PrepareStatement : An example

```
PreparedStatement pstmt = null;

String QryString = "INSERT INTO JavaCourse (Roll,Name,Marks,Grade) VALUE
pstmt = conn.prepareStatement(QryString);

pstmt.setInt(1, 12);
pstmt.setString(2, "ABC");
pstmt.setInt(3, 64);
pstmt.setString(4, "Ex");
pstmt.executeUpdate();
```

ResultSet

- ‘A ResultSet provides access to a table of data generated by executing a Statement.
- ‘Only one ResultSet per Statement can be open at once.
- ‘The table rows are retrieved in sequence.
- ‘A ResultSet maintains a cursor pointing to its current row of data.
- ‘The next() method moves the cursor to the next row.
- ‘you can't rewind.

Useful ResultSet methods

Methods	Description
`boolean next()`	– attempts to move to the next row in ResultSet – the first call to next() positions cursor at the first r – returns false if there are no more rows
`Type getType(int columnIndex)`	– returns the given field as the given type – fields indexed starting at 1 (not 0)
`Type getType(String columnName)`	– same, but uses name of field – less efficient
`void close()`	– disposes of the ResultSet – allows you to re-use the Statement that created it
`int findColumn(String columnName)`	– looks up column index given column name

Matching with Java and SQL data types

SQL type	Java class	ResultSet method
BIT	Boolean	getBoolean()
CHAR	String	getString()
VARCHAR	String	getString()
DOUBLE	Double	getDouble()
FLOAT	Double	getDouble()
INTEGER	Integer	getInt()
REAL	Double	getFloat()
DATE	java.sql.Date	getDate()
TIME	java.sql.Time	getTime()
TIMESTAMP	java.sql.TimeStamp	getTimestamp()

Map the results to data structures

```
ResultSet rs = null;
String NameString, RollString, MarksString, GradeString;

stmt.execute("SELECT * FROM JavaCourse");
rs = stmt.getResultSet();

while (rs.next()) {
    NameString = rs.getString("Name");
    RollString = rs.getString("Roll");
    MarksString = rs.getString("Marks");
    GradeString = rs.getString("Grade");
}
```

JDBC exceptions

SQLException is an Exception class which provides information on database access errors.

```
try{
  ….

}

catch (SQLException ex){
                // handle any errors
                System.out.println("SQLException: " + ex.getMessage());
                System.out.println("SQLState: " + ex.getSQLState());
                System.out.println("VendorError: " + ex.getErrorCode());
}
```

Examples:

Illustration 11.3

```
// Create JDBC connection to MySQL Database server


import java.sql.*;
public class Connect {
          public static void main (String[] args) {
                    Connection conn = null;
                    try{
                              String userName = "root";
                              String password = "root";
                              String url = "jdbc:mysql://localhost:3306/test";
                              //Class.forName ("com.mysql.jdbc.Driver").newInstance (); // This is
depricated
                              Class.forName ("com.mysql.cj.jdbc.Driver").newInstance ();// This is the
newest driver
                              conn = DriverManager.getConnection (url, userName, password);
                              System.out.println ("Database connection established");
                    }
                    catch (Exception e) {
                              System.err.println ("Cannot connect to database server:"+e);
                    } finally {
                              if (conn != null) {
                                        try {
                                                  conn.close ();
                                                  System.out.println ("Database connection
terminated:"+e);
                                        } catch (Exception e) { /* ignore close errors */ }
                              }’‘‘‘
```

```
                    }
          }
}
```

Illustration 11.4

```
// Create table in MySQL database using JDBC


import java.sql.*;
import java.sql.ResultSet;
public class CreateTable{
          public static void main (String[] args)'' {
                    Connection conn = null;
                    Statement stmt = null;
                    ResultSet rs = null;
                    String TableName;
                    try''{
                              String userName = "root";
                              String password = "root";
                              String url = "jdbc:mysql://localhost:3306/test";
                              Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                              conn = DriverManager.getConnection (url, userName, password);
                              stmt = conn.createStatement();
                              //Creating table
                              stmt.execute("create table JavaCourse(Roll Integer primary key, Name
Varchar(30), Marks Integer not null, Grade Varchar(2))");''
                    } catch (SQLException ex){
                              System.out.println("SQLException: " + ex.getMessage());
                              System.out.println("SQLState: " + ex.getSQLState());
                              System.out.println("VendorError: " + ex.getErrorCode());
                    }
                    catch (Exception e){
                              System.err.println ("Cannot connect to database server");
                    }
                    finally {
                              if (rs != null) {
                                        try {
                                                  rs.close();
                                        } catch (SQLException sqlEx) {}
                                        rs = null;
```

```
                                }
                                if (stmt != null) {
                                        try {
                                                stmt.close();
                                        } catch (SQLException sqlEx) {}
                                        stmt = null;
                                }
                                if (conn != null) {
                                        try {
                                                conn.close ();
                                        } catch (Exception e) { /* Ignore code for closing errors */ }
                                }       ‘‘
                        }
                }
}
```

Illustration 11.6

```
// Insert data in MySQL database using JDBC

import java.sql.*;
import java.sql.ResultSet;
public class InsertRecord{
        public static void main (String[] args){
                Connection conn = null;
                Statement stmt = null;
                ResultSet rs = null;
                String NameString, RollString, MarksString, GradeString;

                try{
                        String userName = "root";
                        String password = "root";
                        String url = "jdbc:mysql://localhost:3306/test";
                        Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                        conn = DriverManager.getConnection (url, userName,
password);’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                        stmt = conn.createStatement();’‘
                        stmt.execute("insert into JavaCourse values (01,'Debasis', 75, 'A')");
                        stmt.execute("insert into JavaCourse values(02,'Nilanjan', 85, 'EX')");
                        stmt.execute("insert into JavaCourse values(03,'Tauheed', 65, 'B')");
```

```
                    stmt.execute("insert into JavaCourse values(04,'Priyabrata', 78, 'A')");
                    }
                catch (SQLException ex){
                    // handle any errors
                    System.out.println("SQLException: " + ex.getMessage());
                    System.out.println("SQLState: " + ex.getSQLState());
                    System.out.println("VendorError: " + ex.getErrorCode());
                }
                catch (Exception e){
                    System.err.println ("Cannot connect to database server");
                }
                finally {
                    if (rs != null) {
                        try {
                            rs.close();
                        } catch (SQLException sqlEx) { } // ignore
                        rs = null;
                    }
                    if (stmt != null) {
                        try {
                            stmt.close();
                        } catch (SQLException sqlEx) { } // ignore
                        stmt = null;
                    }
                    if (conn != null) {
                        try {
                            conn.close ();// System.out.println ("Database
connection terminated");
                        } catch (Exception e) { /* ignore close errors */ }
                    }
                }
        }
}
```

Illustration 11.7

```
// Update data in MySQL database using JDBC

import java.sql.*;
import java.sql.ResultSet;
```

```
public class UpdateRecord{
          public static void main (String[] args){
                    Connection conn = null;
                    Statement stmt = null;
                    ResultSet rs = null;
                    String NameString, RollString, MarksString, GradeString;
                    try{
                              String userName = "root";
                              String password = "root";
                              String url = "jdbc:mysql://localhost:3306/test";
                              Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                              conn = DriverManager.getConnection (url, userName, password);
                              stmt = conn.createStatement();
                              stmt.execute("update JavaCourse set Name='Debasis' where Name='Debasish'");
                              //stmt.execute("update JavaCourse set Marks=85, Grade='Ex' where
Name='Debasis'");
                    }
                    catch (SQLException ex){
                              // handle any errors
                              System.out.println("SQLException: " + ex.getMessage());
                              System.out.println("SQLState: " + ex.getSQLState());
                              System.out.println("VendorError: " + ex.getErrorCode());
                    }
                    catch (Exception e){
                              System.err.println ("Cannot connect to database server");
                    }
                    finally {
                              if (rs != null) {
                              try {
                                        rs.close();
                                        } catch (SQLException sqlEx) { } // ignore
                                        rs = null;
                              }
                              if (stmt != null) {
                                        try {
                                                  stmt.close();
                                        } catch (SQLException sqlEx) { } // ignore
                                        stmt = null;
                              }
                              if (conn != null) {
                                        try {
```

```
                                                    conn.close ();
                                               // System.out.println ("Database connection terminated");
                                               } catch (Exception e) { /* ignore close errors */ }
                                    }
                         }
          }
}
```

Illustration 11.8

```
// Delete table data in MySQL database using JDBC

import java.sql.*;
import java.sql.ResultSet;
public class DeleteRecord{
          public static void main (String[] args){
                    Connection conn = null;
                    Statement stmt = null;
                    ResultSet rs = null;
                    String NameString, RollString, MarksString, GradeString;
                    try{
                              String userName = "root";
                              String password = "root";
                              String url = "jdbc:mysql://localhost:3306/test";
                              Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                              conn = DriverManager.getConnection (url, userName, password);
                              stmt = conn.createStatement();’‘
                              stmt.execute("delete from JavaCourse where marks <66");
                              //stmt.execute("delete from JavaCourse");
                    }
                    catch (SQLException ex){
                              // handle any errors
                              System.out.println("SQLException: " + ex.getMessage());
                              System.out.println("SQLState: " + ex.getSQLState());
                              System.out.println("VendorError: " + ex.getErrorCode());
                    }
                    catch (Exception e){
                              System.err.println ("Cannot connect to database server:"+e);
                    }
                    finally {
```

```
                              if (rs != null) {
                                        try {
                                                  rs.close();
                                        } catch (SQLException sqlEx) { } // ignore
                                        rs = null;
                              }
                              if (stmt != null) {
                                        try {
                                                  stmt.close();
                                        } catch (SQLException sqlEx) { } // ignore
                                        stmt = null;
                              }
                              if (conn != null) {
                                        try {
                                                  conn.close ();
                                        // System.out.println ("Database connection terminated");
                                        }
                                        catch (Exception e) { /* ignore close errors */ }
                              }
                    }
          }
}
```

Illustration 11.9

```
// Drop a table in MySQL database using JDBC

import java.sql.*;
import java.sql.ResultSet;
public class DropTable{
          public static void main (String[] args){
                    Connection conn = null;
                    Statement stmt = null;
                    ResultSet rs = null;
                    String NameString, RollString, MarksString, GradeString;
                    try{
                              String userName = "root";
                              String password = "root";
                              String url = "jdbc:mysql://localhost:3306/test";
                              Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
```

```
                    conn = DriverManager.getConnection (url, userName, password);
                    stmt = conn.createStatement();’‘
                    stmt.execute("drop table JavaCourse");
          }
          catch (SQLException ex){
                    // handle any errors
                    System.out.println("SQLException: " + ex.getMessage());
                    System.out.println("SQLState: " + ex.getSQLState());
                    System.out.println("VendorError: " + ex.getErrorCode());
          }
          catch (Exception e){
                    System.err.println ("Cannot connect to database server:"+e);
          }
          finally {
                    if (rs != null) {
                              try {
                                        rs.close();
                              } catch (SQLException sqlEx) { } // ignore
                              rs = null;
                    }
                    if (stmt != null) {
                              try {
                                        stmt.close();
                              } catch (SQLException sqlEx) { } // ignore
                              stmt = null;
                    }
                    if (conn != null) {
                              try {
                                        conn.close ();
                              // System.out.println ("Database connection terminated");
                              }
                              catch (Exception e) { /* ignore close errors */ }
                    }
          }
     }
}
```

Illustration 11.10

```
/*
```

```
A program to select all data present in a table and calculate average marks
*/


import java.sql.*;
import java.sql.ResultSet;
public class SelectRecord’‘ {
          public static void main (String[] args){
                    Connection conn = null;
                    Statement stmt = null;
                    ResultSet rs = null;
                    int TotalMarks=0, Num_Student=0;
                    float Avg_Marks;
                    String NameString, RollString, MarksString, GradeString;
                    try{
                              String userName = "root";
                              String password = "root";
                              String url = "jdbc:mysql://localhost:3306/test";
                              Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                              conn = DriverManager.getConnection (url, userName, password);
                              stmt = conn.createStatement();’‘
                              stmt.execute("SELECT * FROM JavaCourse");
                              rs = stmt.getResultSet();
                              System.out.println("\n\n -------’‘Results’‘---------\n");
                              while (rs.next()) {
                              NameString = rs.getString("Name");
                              RollString = rs.getString("Roll");
                              MarksString = rs.getString("Marks");
                              GradeString = rs.getString("Grade");
                              TotalMarks = TotalMarks + Integer.parseInt(MarksString);
                              System.out.println("Name: = "+NameString+"\t\t"+"Roll: =
"+RollString+"\t\t"+"Marks: = "+MarksString+"\t\t"+"Grade: = "+GradeString+"\n");
                              } //end while
                              rs.last();
                              Num_Student = rs.getRow();
                              Avg_Marks = TotalMarks / Num_Student;
                              System.out.println("\n\n ------- AVERAGE Marks = "+Avg_Marks+"--------");
                    }
                    catch (SQLException ex){
                              // handle any errors
                              System.out.println("SQLException: " + ex.getMessage());
                              System.out.println("SQLState: " + ex.getSQLState());
```

```
                    System.out.println("VendorError: " + ex.getErrorCode());
                }
                catch (ArithmeticException e) {
                    System.out.println("Division by zero.");
                }
                catch (Exception e) {
                    System.err.println ("Cannot connect to database server");
                }
                finally {
                    if (rs != null) {
                        try {
                            rs.close();
                        } catch (SQLException sqlEx) { } // ignore
                        rs = null;
                    }
                    if (conn != null) {
                        try {
                            conn.close ();
                        // System.out.println ("Database connection terminated");
                        }
                        catch (Exception e) { /* ignore close errors */ }
                    }
                }
            }
}
```

Illustration 11.11

```
/*
Create a table in MySQL DB using JDBC
        - First show all the existing tables
        - Insert the new table
        - Show all the current tables
*/
import java.sql.*;
import java.sql.ResultSet;
public class CreateTable{
‘‘ public static void main (String[] args)’‘ {
‘‘‘‘Connection conn = null;
        Statement stmt = null;
```

```
          ResultSet rs = null;
‘‘‘‘‘‘‘‘‘ String TableName;
‘‘‘‘‘‘‘‘‘ try’‘{
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘String userName = "root";
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘String password = "root";
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘String url = "jdbc:mysql://localhost:3306/test";
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘conn = DriverManager.getConnection (url, userName, password);
                                        stmt = conn.createStatement();
                                        stmt.execute("show tables");
                                ‘‘‘‘rs = stmt.getResultSet();
                                ‘‘‘‘System.out.println("Result before creating the table");
                                ‘‘‘‘while (rs.next()) {
                                        ‘‘ TableName = rs.getString("Tables_in_test");’‘// To create a
table having name JavaCourse
                                        ‘‘ System.out.println("Table Name: = "+TableName+"\n");
                                        }
                                        stmt.execute("create table JavaCourse(Roll Integer primary key,
Name Varchar(30), Marks Integer not null, Grade Varchar(2))");
                                        stmt.execute("show tables");
                                        rs = stmt.getResultSet();
                                        System.out.println("Result after creating the table\n");
                        ‘‘‘‘       while (rs.next()) {
                        ‘‘‘‘                ‘‘ TableName = rs.getString("Tables_in_test");
                        ‘‘‘‘                ‘‘ System.out.println("Table Name: = "+TableName+"\n");
                        ‘‘‘‘       }’‘
                        ‘‘‘‘} catch (SQLException ex){
                        ‘‘‘‘                ‘‘ System.out.println("SQLException: " + ex.getMessage());
                        ‘‘‘‘                ‘‘ System.out.println("SQLState: " + ex.getSQLState());
                        ‘‘‘‘                ‘‘ System.out.println("VendorError: " + ex.getErrorCode());
                        ‘‘‘‘}
                        ‘‘‘‘catch (Exception e){
                        ‘‘‘‘                ‘‘ System.err.println ("Cannot connect to database server");
                        ‘‘‘‘}
‘‘‘‘‘‘‘‘‘‘‘‘finally {
                                        ‘‘ if (rs != null) {
                                                try {
                                                        rs.close();
                                                } catch (SQLException sqlEx) {}
                                                        rs = null;
                                        ‘‘ }
```

```
                                        ‘‘if (stmt != null) {
                                                  ‘‘try {
                                                            ‘‘stmt.close();
                                                  ‘‘} catch (SQLException sqlEx) {}
                                                            ‘‘stmt = null;
                                        ‘‘ }
                                        ‘‘if (conn != null) {
                                                  try {
                                                            ‘‘conn.close ();
                                                            } catch (Exception e) { /* Ignore code for
closing errors */ }
                                        }                                   ‘‘
                    }
‘‘ }
}
```

Illustration 11.12

```
/*
Insert data in MySQL DB using JDBC
          - First show all the existing data
          - Insert the new data
          - Show all the current data
*/
import java.sql.*;
import java.sql.ResultSet;
public class InsertRecord{
          public static void main (String[] args){
                    Connection conn = null;
                    Statement stmt = null;
                    ResultSet rs = null;
                    String NameString, RollString, MarksString, GradeString;

                    try{
                              String userName = "root";
                              String password = "root";
                              String url = "jdbc:mysql://localhost:3306/test";
                              Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                              conn = DriverManager.getConnection (url, userName,
password);’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                              stmt = conn.createStatement();’‘
```

```
                                stmt.execute("SELECT * FROM JavaCourse");
                                rs = stmt.getResultSet();
                                System.out.println("\n\n ------- Result Before Insert---------\n");
                                while (rs.next()) {
                                NameString = rs.getString("Name");
                                RollString = rs.getString("Roll");
                                MarksString = rs.getString("Marks");
                                GradeString = rs.getString("Grade");
                                System.out.println("Name: = "+NameString+"\t\t"+"Roll: =
"+RollString+"\t\t"+"Marks: = "+MarksString+"\t\t"+"Grade: = "+GradeString+"\n");
                                } //end while

                                stmt.execute("insert into JavaCourse values (01,'Debasish', 75, 'A')");
                                stmt.execute("insert into JavaCourse values(02,'Nilanjan', 85, 'EX')");
                                stmt.execute("insert into JavaCourse values(03,'Tauheed', 65, 'B')");
                                stmt.execute("insert into JavaCourse values(04,'Priyabrata', 78, 'A')");

                                stmt.execute("SELECT * FROM JavaCourse");
                                rs = stmt.getResultSet();
                                System.out.println("\n\n ------- Result After Insert---------\n");
                                while (rs.next()) {
                                NameString = rs.getString("Name");
                                RollString = rs.getString("Roll");
                                MarksString = rs.getString("Marks");
                                GradeString = rs.getString("Grade");
                                System.out.println("Name: = "+NameString+"\t\t"+"Roll:
="+RollString+"\t\t"+"Marks: = "+MarksString+"\t\t"+"Grade: = "+GradeString+"\n");
                                } //end while
                        }
                        catch (SQLException ex){
                                // handle any errors
                                System.out.println("SQLException: " + ex.getMessage());
                                System.out.println("SQLState: " + ex.getSQLState());
                                System.out.println("VendorError: " + ex.getErrorCode());
                        }
                        catch (Exception e){
                                System.err.println ("Cannot connect to database server");
                        }
                        finally {
                                if (rs != null) {
```

```
                                        try {
                                                rs.close();
                                        } catch (SQLException sqlEx) { } // ignore
                                        rs = null;
                                }
                                if (stmt != null) {
                                        try {
                                                stmt.close();
                                        } catch (SQLException sqlEx) { } // ignore
                                        stmt = null;
                                }
                                if (conn != null) {
                                        try {
                                                conn.close ();// System.out.println ("Database
connection terminated");
                                        } catch (Exception e) { /* ignore close errors */ }
                                }
                        }
                }
}
```

Illustration 11.13

```
/*
Update data in MySQL DB using JDBC
        - First show all the existing data
        - Update the new data
        - Show all the current data
*/
import java.sql.*;
import java.sql.ResultSet;
public class UpdateRecord{
        public static void main (String[] args){
                Connection conn = null;
                Statement stmt = null;
                ResultSet rs = null;
                String NameString, RollString, MarksString, GradeString;
                try{
                        String userName = "root";
                        String password = "root";
```

```
                                String url = "jdbc:mysql://localhost:3306/test";
                                Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                                conn = DriverManager.getConnection (url, userName, password);
                                stmt = conn.createStatement();’‘
                                stmt.execute("SELECT * FROM JavaCourse");
                                rs = stmt.getResultSet();
                                System.out.println("\n\n ------- Result Before Update ---------\n");
                                while (rs.next()) {
                                          NameString = rs.getString("Name");
                                          RollString = rs.getString("Roll");
                                          MarksString = rs.getString("Marks");
                                          GradeString = rs.getString("Grade");
                                          System.out.println("Name: = "+NameString+"\t\t"+"Roll: =
"+RollString+"\t\t"+"Marks: = "+MarksString+"\t\t"+"Grade: = "+GradeString+"\n");
                                } //end while
                                // Update a row
                                stmt.execute("update JavaCourse set Marks=85, Grade='Ex' where
Name='Debasish Kundu'");
                                stmt.execute("SELECT * FROM JavaCourse");
                                rs = stmt.getResultSet();
                                System.out.println("\n\n ------- Result After Update ---------\n");
                                while (rs.next()) {
                                          NameString = rs.getString("Name");
                                          RollString = rs.getString("Roll");
                                          MarksString = rs.getString("Marks");
                                          GradeString = rs.getString("Grade");
                                          System.out.println("Name: = "+NameString+"\t\t"+"Roll: =
"+RollString+"\t\t"+"Marks: = "+MarksString+"\t\t"+"Grade: = "+GradeString+"\n");
                                } //end while
                      }
                      catch (SQLException ex){
                                // handle any errors
                                System.out.println("SQLException: " + ex.getMessage());
                                System.out.println("SQLState: " + ex.getSQLState());
                                System.out.println("VendorError: " + ex.getErrorCode());
                      }
                      catch (Exception e){
                                System.err.println ("Cannot connect to database server");
                      }
                      finally {
                                if (rs != null) {
                                          try {
```

```
                                        rs.close();
                                } catch (SQLException sqlEx) { } // ignore
                                rs = null;
                        }
                        if (stmt != null) {
                                try {
                                        stmt.close();
                                } catch (SQLException sqlEx) { } // ignore
                                stmt = null;
                        }
                        if (conn != null) {
                                try {
                                        conn.close ();
                                // System.out.println ("Database connection terminated");
                                }’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘catch (Exception e) { /* ignore close errors */
}
                        }
                }
        }
}
```

Illustration 11.14

```
/*
Delete table data in MySQL DB using JDBC
        - First show all the existing data
        - Delete the data
        - Show all the currently present data
*/
import java.sql.*;
import java.sql.ResultSet;
public class DeleteRecord{
        public static void main (String[] args){
                Connection conn = null;
                Statement stmt = null;
                ResultSet rs = null;
                String NameString, RollString, MarksString, GradeString;
                try{
                        String userName = "root";
                        String password = "root";
```

```
                                String url = "jdbc:mysql://localhost:3306/test";
                                Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
                                conn = DriverManager.getConnection (url, userName, password);
                                stmt = conn.createStatement();’‘
                                stmt.execute("SELECT * FROM JavaCourse");
                                rs = stmt.getResultSet();
                                System.out.println("\n\n ------- Result Before Update ---------\n");
                                while (rs.next()) {
                                          NameString = rs.getString("Name");
                                          RollString = rs.getString("Roll");
                                          MarksString = rs.getString("Marks");
                                          GradeString = rs.getString("Grade");
                                          System.out.println("Name: = "+NameString+"\t\t"+"Roll: =
"+RollString+"\t\t"+"Marks: = "+MarksString+"\t\t"+"Grade: = "+GradeString+"\n");
                                } //end while



                                // Delete data with marks less than value 66
                                stmt.execute("delete from JavaCourse where marks <66");



                                stmt.execute("SELECT * FROM JavaCourse");
                                rs = stmt.getResultSet();
                                System.out.println("\n\n ------- Result After Deletion ---------\n");
                                while (rs.next()) {
                                          NameString = rs.getString("Name");
                                          RollString = rs.getString("Roll");
                                          MarksString = rs.getString("Marks");
                                          GradeString = rs.getString("Grade");
                                          System.out.println("Name: = "+NameString+"\t\t"+"Roll: =
"+RollString+"\t\t"+"Marks: = "+MarksString+"\t\t"+"Grade: = "+GradeString+"\n");
                                } //end while
                     }
                     catch (SQLException ex){
                                // handle any errors
                                System.out.println("SQLException: " + ex.getMessage());
                                System.out.println("SQLState: " + ex.getSQLState());
                                System.out.println("VendorError: " + ex.getErrorCode());
                     }
                     catch (Exception e){
```

```
                    System.err.println ("Cannot connect to database server");
                }
                finally {
                    if (rs != null) {
                        try {
                            rs.close();
                        } catch (SQLException sqlEx) { } // ignore
                        rs = null;
                    }
                    if (stmt != null) {
                        try {
                            stmt.close();
                        } catch (SQLException sqlEx) { } // ignore
                        stmt = null;
                    }
                    if (conn != null) {
                        try {
                            conn.close ();
                            // System.out.println ("Database connection terminated");
                            }‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘catch (Exception e) { /* ignore close errors */
}
                    }
                }
            }
}
```

Chapter 12

The Applet Package

by Debasis Samanta

CONTENTS

- Introduction
- Class Applet
- Interface AppletContext
- Interface AppletStub
- Interface AudioClip

Introduction

The java.applet Package contains classes and interfaces for creating applets. This package also provides several interfaces that connect an applet to its document and to resource for playing audio. This is in fact a fundamental package for the creation of applets in Java. The various components in this package is listed below :

```
          Classes
                    Class Applet’‘


          Interfaces
                    Interface AppletContext’‘
                    Interface AppletStub’‘
                    Interface AudioClip’‘
```

Class Applet

An applet is a small program that is not intended to be run on its own, but to be embedded inside another application. The Applet class must be the superclass of any applet that is to be embedded in a Web page or viewed by the Java Applet Viewer. The Applet class provides a standard interface between applets and their environment. The structure of this class is stated as below :

```
‘‘‘‘ public’‘class’‘java.applet.Applet
‘‘‘‘                 extends’‘java.awt.Panel’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public Applet();
                    ‘‘ //     The default constructor for an applet.


‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public void destroy();
‘‘‘‘      public AppletContext getAppletContext();
‘‘‘‘      public String getAppletInfo();
‘‘‘‘      public AudioClip getAudioClip(URL’‘url);
‘‘‘‘      public AudioClip getAudioClip(URL’‘url, String’‘name);
‘‘‘‘      public URL getCodeBase();
‘‘‘‘      public URL getDocumentBase();
‘‘‘‘      public Image getImage(URL’‘url);
‘‘‘‘      public Image getImage(URL’‘url, String’‘name);
‘‘‘‘      public String getParameter(String’‘name);
‘‘‘‘      public String[][] getParameterInfo();
‘‘‘‘      public void init();
‘‘‘‘      public boolean isActive();
‘‘‘‘      public void play(URL’‘url);
```

```
‘‘‘‘      public void play(URL’‘url, String’‘name);
‘‘‘‘      public void resize(Dimension’‘d);
‘‘‘‘      public void resize(int’‘width, int’‘height);
‘‘‘‘      public final void setStub(AppletStub’‘stub);
‘‘‘‘      public void showStatus(String’‘msg);
‘‘‘‘      public void start();
‘‘‘‘      public void stop();
          }

The methods in this class are described in Table 1.1.
```

Table 1.1

Method	Description
destroy()	This method is called by the browser or applet viewer to inform thisapplet that it is being reclaimed and that it should destroy any resources that it has allocated. The `stop` method will always be called before `destroy`. A subclass of `Applet` should override this method if it has any operation that it wants to perform before it is destroyed. For example, an applet with threads would use the `init` method to create the threads and the `destroy` method to kill them.
getAppletContext()	Determines this applet's context, which allows the applet to query and affect the environment in which it runs. This environment of an applet represents the document that contains the applet.
getAppletInfo()	Returns information about this applet. An applet should override this method to return a String containing information about the author, version, and copyright of the applet.
getAudioClip(URL url)	Returns the `AudioClip` object specified by the URL argument. This method always returns immediately, whether or not the audio clip exists. When this applet attempts to play the audio clip, the data will be loaded.
getAudioClip(URL url, String name)	Returns the `AudioClip` object specified by the URL and name arguments.

Table 1.1 (Contd.)

getCodeBase()	**Returns :** The URL *of this applet.*
getDocumentBase()	**Returns :** The URL *of the document that contains this applet.*
getImage(URL url)	Returns an `Image` object that can then be painted on the screen. The url that is passed as an argument must specify an absolute URL.162162
getImage(URL url, String name)	Returns an Image object that can then be painted on the screen.

	The url argument must specify an absolute URL. The name argument is a specifier that is relative to the url argument. This method always returns immediately, whether or not the image exists. When this applet attempts to draw the image on the screen, the data will be loaded. The graphics primitives that draw the image will incrementally paint on the screen.
getParameter(String name)	Returns the value of the named parameter in the HTML tag. For example, if this applet is specified as : <applet code="Clock" width=50 height=50> <param name= Color value ="blue"> </applet> A call to `getParameter("Color")` returns the value "blue". The name argument is case insensitive.
getParameterInfo()	Returns information about the parameters than are understood by this applet. An applet should override this method to return an array of `Strings` describing these parameters.
init()	This method is called by the browser or applet viewer to inform this applet that it has been loaded into the system. It is always called before the first time that the `start` method is called. A subclass of `Applet` should override this method if it has initialization to perform. For example, an applet with threads would use the `init` method to create the threads and the `destroy` method to kill them.
isActive()	Determines if this applet is active. An applet is marked active just before its `start` method is called. It becomes inactive immediately after its `stop` method is called. **Returns :** true if the applet is active; false otherwise.
play(URL url)	Plays the audio clip at the specified absolute URL. Nothing happens if the audio clip cannot be found.

Table 1.1 (Contd.)

play(URL url, String name)	Plays the audio clip given the URL and a specifier that's relative to it. Nothing happens if the audio clip cannot be found.
resize(Dimension d)	Requests that this applet be resized with the specified dimension.
resize(int width, int height)	Requests that this applet be resized with the specified parameters.
setStub(AppletStub stub)	Sets this applet's stub. This is done by automatically by the system.
showStatus(String msg)	Requests that the argument string be displayed in the "status window." Many browsers and applet viewers provide such a "status window" where the application can inform users of its current state.
start()	This method is called by the browser or applet viewer to inform this applet that it should start its execution. It is called after the `init` method and each time the applet is revisited in a Web page. A subclass of Applet should override this method if it has any operation that it wants to perform each time the Web page

	containing it is visited. For example, an applet with animation might want to use the start method to resume animation, and the stop method to suspend the animation.
stop()	This method is called by the browser or applet viewer to inform this applet that it should stop its execution. It is called when the Web page that contains this applet has been replaced by another page and also just before the applet is to be destroyed. A subclass of Applet should override this method if it has any operation that it wants to perform each time the Web page containing it is no longer visible. For example, an applet with animation might want to use the start method to resume animation, and the stop method to suspend the animation.

Interface AppletContext

This interface corresponds to an applet's environment: the document containing the applet and the other applets in the same document. The methods in this interface can be used by an applet to obtain information about its environment.

```
‘‘ public’‘interface’‘java.applet.AppletContext
          {
‘‘‘‘‘‘‘‘  //        Methods
‘‘‘‘       public abstract Applet getApplet(String’‘name);
‘‘‘‘       public abstract Enumeration getApplets();
‘‘‘‘       ublic abstract AudioClip getAudioClip(URL’‘url);
‘‘‘‘       public abstract Image getImage(URL’‘url);
‘‘‘‘       public abstract void showDocument(URL’‘url);
‘‘‘‘       public abstract void showDocument(URL’‘url, String’‘target);
‘‘‘‘
          public abstract void showStatus(String’‘status);
          }
‘‘‘‘Methods are stated in the Table 1.2.
```

Table 1.2

Method	Description
getApplet(String name)	Finds and returns the applet in the document represented by this applet context with the given name. The name can be set in the HTML tag by setting the name attribute.
getApplets()	Finds all the applets in the document represented by this applet context. **Returns :** an enumeration of all applets in the document represented by this applet context.
getAudioClip(URL url)	Creates an audio clip. **Returns :** The audio clip at the specified URL.
getImage(URL url)	Returns an `Image` object that can then be painted on the screen. The url argument that is passed as an argument

	must specify an absolute URL. This method always returns immediately, whether or not the image exists. When the applet attempts to draw the image on the screen, the data will be loaded. The graphics primitives that draw the image will incrementally paint on the screen.
showDocument(URL url)	Replaces the Web page currently being viewed with the given URL. This method may be ignored by applet contexts that are not browsers.
showDocument(URL url, String target)	Requests that the browser or applet viewer show the Web page indicated by the url argument. The target argument indicates where to display the frame. The target argument is interpreted as follows : Argument’“““““Function self:’“““Show the current frame parent:’“““Show the parent frame top-most:’“““Show the top most frame An applet viewer or browser is free to ignore.

Interface AppletStub

When an applet is first created, an applet stub is attached to it using the applet's setStub method. This stub serves as the interface between the applet and the browser environment or applet viewer environment in which the applet is running.

```
‘‘‘‘public’‘interface’‘java.applet.AppletStub
          ‘‘{
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract void appletResize(int’‘width, int’‘height);
‘‘‘‘      public abstract AppletContext getAppletContext();
‘‘‘‘      public abstract URL getCodeBase();
‘‘‘‘      public abstract URL getDocumentBase();
‘‘‘‘      public abstract String getParameter(String’‘name);
‘‘‘‘      public abstract boolean isActive();
          ‘‘}


‘‘‘‘Methods are described in Table 1.3
```

Table 1.3

Method	Description
appletResize(int width, int height)	Called when the applet wants to be resized.
getAppletContext()	**Returnsc :** the applet's context.
getCodeBase()	**Returns :** The URL of the applet.

getDocumentBase()	**Returns :** the URL of the document containing the applet.
getParameter(String name)	Returns the value of the named parameter in the HTML tag. 165
isActive()	Determines if the applet is active. An applet is active just before its start method is called. It becomes inactive immediately after its stop method is called. **Returns :** true if the applet is active; false otherwise.

Interface AudioClip

The AudioClip interface is a simple abstraction for playing a sound clip. Multiple AudioClip items can be playing at the same time, and the resulting sound is mixed together to produce a composite.

```
          public‘‘interface’‘java.applet.AudioClip
          {
‘‘‘‘‘‘‘‘// Methods
‘‘‘‘      public abstract void loop();
‘‘‘‘      public abstract void play();
‘‘‘‘      public abstract void stop();
          }

          Methods are described in Table 1.4.
```

Table 1.4

Method	Description
loop()	Starts playing this audio clip in a loop.
play()	Starts playing this audio clip. Each time this method is called, the clip is restarted from the beginning.
stop()	Stops playing this audio clip.

Chapter 13

The Java Language Package

by Debasis Samanta

CONTENTS

- Introduction
- Class Boolean
- Class Character
- Class Class
- Class ClassLoader
- Class Compiler
- Class Double
- Class Float
- Class Integer
- Class Long
- Class Math
- Class Number
- Class Object
- Class Process
- Class Runtime
- Class SecurityManager
- Class String
- Class StringBuffer
- Class System
- Class Thread
- Class ThreadGroup
- Class Throwable
- Interface Cloneable
- Interface Runnable
- Class ArithmeticException
- Class ArrayIndexOutOfBoundsException
- Class ArrayStoreException
- Class ClassCastException
- Class ClassNotFoundException
- Class CloneNotSupportedException
- Class Exception
- Class IllegalAccessException
- Class IllegalArgumentException
- Class IllegalMonitorStateException
- Class IllegalThreadStateException
- Class IndexOutOfBoundsException
- Class InstantiationException
- Class InterruptedException
- Class NegativeArraySizeException
- Class NoSuchMethodException
- Class NullPointerException
- Class NumberFormatException
- Class RuntimeException
- Class SecurityException
- Class StringIndexOutOfBoundsException
- Class AbstractMethodError
- Class ClassCircularityError
- Class ClassFormatError
- Class Error
- Class IllegalAccessError
- Class IncompatibleClassChangeError
- Class InstantiationError
- Class InternalError
- Class LinkageError
- Class NoClassDefFoundError
- Class NoSuchFieldError
- Class NoSuchMethodError
- Class OutOfMemoryError

- Class StackOverflowError
- Class ThreadDeath
- Class UnknownError
- Class UnsatisfiedLinkError
- Class VerifyError
- Class VirtualMachineError

Introduction

The Java Language Package java.lang provides 21 classes, 2 interfaces, 21 classes for exceptions, and 19 classes for errors; it forms the core of the Java language and the JVM. This is the only package that is automatically imported into every Java program. Following are the list of classes, interfaces, classes of exceptions, and classes of errors :

```
Classes’‘‘‘
‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
Class Boolean                    ‘‘‘‘
Class Character
Class Class
Class ClassLoader
Class Compiler
Class Double
Class Float
Class Integer
Class Long
Class Math
Class Number
Class Object
Class Process
Class Runtime
Class SecurityManager
Class String
Class StringBuffer
Class System
Class Thread
Class ThreadGroup
Class Throwable

Exceptions


Class ArithmeticException
Class ArrayIndexOutOfBoundsException
Class ArrayStoreException
Class ClassCastException
```

Class ClassNotFoundException

Class CloneNotSupportedException

Class Exception

Class IllegalAccessException

Class IllegalArgumentException

Class IllegalMonitorStateException

Class IllegalThreadStateException

Class IndexOutOfBoundsException

Class InstantiationException

Class InterruptedException

Class NegativeArraySizeException

Class NoSuchMethodException

Class NullPointerException

Class NumberFormatException

Class RuntimeException

Class SecurityException

Class StringIndexOutOfBoundsException

Errors

Class AbstractMethodError

Class ClassCircularityError

Class ClassFormatError

Class Error

Class IllegalAccessError

Class IncompatibleClassChangeError

Class InstantiationError

Class InternalError

Class LinkageError

Class NoClassDefFoundError

Class NoSuchFieldError

Class NoSuchMethodError

Class OutOfMemoryError

Class StackOverflowError

Class ThreadDeath

Class UnknownError

Class UnsatisfiedLinkError

Class VerifyError

Class VirtualMachineError

Interfaces

```
Interface Cloneable

Interface Runnable
```

Class Boolean

This class provides us to define primitive type boolean for an object. An object of type Boolean contains a single field whose type is boolean. In addition, this class provides a number of methods for converting a boolean to a String and a String to a boolean, as well as other constants and methods useful when dealing with a boolean. The various component in this class is stated below :

```
          public‘‘final’‘class’‘java.lang.Boolean
‘‘‘‘                                    extends’‘java.lang.Object {
‘‘‘‘‘‘     //       Member elements
‘‘‘‘       public final static Boolean FALSE;
‘‘‘‘       public final static Boolean TRUE;


‘‘‘‘‘‘     //        Constructors
                     public Boolean(boolean’‘value);
                     //        Creates’‘a Boolean object initialized with the value argument.
                     public Boolean(String’‘s);
                     /*        Creates a Boolean object initialized with the value true if the string
argument is not null and is equal, ignoring case, to the string "true". Otherwise, creates a Boolean
object initialized with the value false. */
                     // Methods
                     public boolean booleanValue();
                     public boolean equals(Object’‘obj);
                     public static boolean’‘getBoolean(String’‘name);
                     public int hashCode();
                     public String toString();
                     public static Boolean valueOf(String’‘s);
‘‘‘‘}
The performance of various methods in the class Boolean are listed in Table 2.1 :
```

Table 2.1

Methods	Description
booleanValue()	**Returns :** The primitive boolean value of this object.
equals(Object obj)	The result is true if and only if the argument is not null and is a Boolean object that contains the same boolean value as this object.**Returns** : True if the

	objects are the same; false otherwise.
getBoolean(String name)	The result is true if and only if the system property named by the argument exists and is equal, ignoring case to the string ⍰true⍰.
hashCode()	**Returns :** a hash code value for this object.
toString()	If this object contains the value true, a string equal to "true" is returned. Otherwise, a string equal to "false" is returned.
valueOf(String s)	A new Boolean object is constructed. This Boolean contains the value true if the string argument is not null and is equal, ignoring case, to the string "true".**Returns :** the Boolean value represented by the string.

Class Character

This class provides us a value of the primitive type char in an object. An object of type Character contains a single field whose type is char. In addition, this class provides a number of methods for determining the type of a character, and converting characters from uppercase to lowercase and vice versa.

Many of the methods of class Character are defined in terms of a "Unicode attribute table" that specifies a name for every defined Unicode code point. The table also includes other attributes, such as a decimal value, an uppercase equivalent, a lowercase equivalent, and/or a titlecase equivalent. The Unicode attribute table is available on the World Wide Web as the file :

```
ftp ://unicode.org/pub/MappingTables/UnicodeData1.1.5.txt
```

The various component in this class is stated as below :

```
          public‘‘final’‘class’‘java.lang.Character extends’‘java.lang.Object’‘‘‘  ‘‘‘‘{

                                //                  Member elements

                                public final static int MAX_RADIX;

                                /*        The constant value of this field is the largest value ( say, 36)
permitted for the radix argument in radix-conversion methods such as the digit method, the forDigit
method, and the toString method of class Integer.

                                */

                                public final static char MAX_VALUE;

                                //        The constant value of this field is the largest value (e.g.
'\uffff' of type char.


                                public final static int MIN_RADIX;

                                /*        The constant value of this field is the smallest value ( say, 2)
permitted for the radix argument in radix-conversion methods such as the digit method, the forDigit
method, and the toString method of class Integer.

                                */

                                public final static char MIN_VALUE;

                                //        The constant value of this field is the smallest value (e.g.
'\u0000' of type char.)
```

```
                                        // Constructor
                                public Character(char’‘value);
                                //        Creates a Character object and initializes it so that it
represents the value argument.


                                //                  Methods
                                public char charValue();
                                public static int digit(char’‘ch, int’‘radix);
                                public boolean equals(Object’‘obj);
                                public static char forDigit(int’‘digit, int’‘radix);
                                public int hashCode();
                                public static boolean isDefined(char’‘ch);
                                public static boolean isDigit(char’‘ch);
                                public static boolean isJavaLetter(char’‘ch);
                                public static boolean isJavaLetterOrDigit(char’‘ch);
                                public static boolean isLetter(char’‘ch);
                                public static boolean isLetterOrDigit(char’‘ch);
                                public static boolean isLowerCase(char’‘ch);
                                public static boolean isSpace(char’‘ch);
                                public static boolean isTitleCase(char’‘ch);
                                public static boolean isUpperCase(char’‘ch);
                                public static char toLowerCase(char’‘ch);
                                public String toString();
                                public static char toTitleCase(char’‘ch);
                                public static char toUpperCase(char’‘ch);
          ‘‘‘‘}
          The function of various methods are listed in Table 2.2.
```

Table 2.2

Method	Description
charValue()	Returns :the char value represented by this object.
digit(char ch, int radix)	Returns the numeric value of the character ch in the specified radix. If the radix is not in the range MIN_RADIX <= RADIX <= MAX_RADIX or if the ch is not a valid digit in the specified radix, -1 is returned. A character is a valid digit if either of the following is true : The method `isDigit` is true, and the Unicode decimal digit value of the character (or its single-character decomposition) is less than the specified radix. The character is one of the uppercase Latin letters 'A' through 'Z' and its code is less than

	radix +�A�-10. The character is one of the lowercase Latin letters 'a' through 'z' and its code is less that radix +�A�-10. In this case, ch - �A� + 10 is returned.
equals(Object obj)	The result is true if and only if the argument is not null and is a `Character` object that represents the same char value as this object. **Returns :** true if the objects are the same; false otherwise.
forDigit(int digit, int radix)	Determines the character representation for a specific digit in the specified radix. If the value of radix is not a valid radix, or the value of digit is not a valid digit in the specified radix, the null character ('\u0000') is returned. The radix argument is valid if it is greater than or equal to MIN_RADIX and less than or equal to MAX_RADIX. The digit atgument is valid if 0 <= digit <= radix. If the digit is less than 10, then �0� + digit is returned. Otherwise, the value �a� + digit - 10 is returned. .
hashCode()	**Returns :** a hash code value for this object.
isDefined(char ch)	Determines if a character has a defined meaning in Unicode. A character is defined if at least one of the following is true : It has an entry in the Unicode attribute table. Its value is in the range �\u3040� <= ch <= �\u9FA5� Its value is in the range �\uF900� <= ch <= �\uFA2� **Returns :** true if the character has a defined meaning in Unicode; false otherwise.
isDigit(char ch)	**Returns :** true if the character has a defined meaning in Unicode; false otherwise.
isJavaLetter(char ch)	Determines whether the specified character is a "Java" letter, that is, the character is permissible as the first character in an identifier in the Java language. A character is considered to be a Java letter if and only if it is a letter, the ASCII dollar sign character '$', or the underscore character '_'. **Returns :** true if the character is a Java letter; false otherwise.
isJavaLetterOrDigit(char ch)	Determines whether the specified character is a "Java" letter or digit, that is, the character is permissible as a non-initial character in an identifier in the Java language. A character is considered to be a Java letter or digit if and only if it is a letter, a digit, the ASCII dollar sign character '$', or the underscore character '_'. **Returns :**true if the character is a Java letter or digit; false otherwise.
isLetter(char ch)	Determines whether the specified character is a letter. **Returns :** true if the character is a letter; false otherwise.
isLetterOrDigit(char ch)	**Returns :** true, if the character is a letter or digit; false

	otherwise.
isLowerCase(char ch)	Determines whether the specified character is a lowercase character. A character is lowercase if it is not in the range '\u2000' through '\u2FFF.,**Returns :** true if the character is lowercase; false otherwise.
isSpace(char ch)	Determines if the character `ch` is wihite space. This method returns true for the following five characters only： `\t`\u0009 HORIZONTAL TABULATION `\n` \u000A NEW LINE `\f` \u000C FORM FEED `\r` \u000D CARRIAGE RETURN `` \u0020 SPACE
isTitleCase(char ch)	Determines if the character is a titlecase character.There are four Unicode characters whose printed representations look like pairs of Latin letters. For example, there is an uppercase letter that looks like "LJ" and the corresponding lowercase letter looks like "lj". A third form, which looks like "Lj" that is the appropriate form to use when rendering a word in lowercase with initial capitals, as for a book title..
isUpperCase(char ch)	Determines whether the specified character is an uppercase character. A character is uppercase if it is not in the range '\u2000' through '\u2FFF'. **Returns :** true, if the character is uppercase; false otherwise.
toLowerCase(char ch)	The given character is mapped to its lowercase equivalent; if the character has no lowercase equivalent, the character itself is returned. A character has a lowercase equivalent if and only if a lowercase mapping is specified for the character in the Unicode attribute table. Note that some Unicode characters in the range '\u2000' through '\u2FFF' have lowercase mappings; this method does map such characters to their lowercase equivalents even though the method `isUpperCase` does not return true for such characters.
toString()	Converts this `Character` object to a string. The result is a string whose length is 1 and whose sole component is the primitive `char` value represented by this object..
toTitleCase(char ch)	Converts the character argument to titlecase. A character has a titlecase equivalent if and only if a titlecase mapping is specified for the character in the Unicode attribute table. Note that some Unicode characters in the range '\u2000' through '\u2FFF' have titlecase mappings; this method does map such characters to their titlecase equivalents even though the method `isTitleCase` does not return

	true for such characters.
toUpperCase(char ch)	Converts the character argument to uppercase. A character has an uppercase equivalent if and only if a titlecase mapping is specified for the character in the Unicode attribute table. **Returns :** The uppercase equivalent of the character, if any; otherwise the character itself.

Class Class

Instances of the class Class represent classes and interfaces in running a Java application. There is no member elements and public constructor for the class Class. Class objects are constructed automatically by the JVM as classes are loaded and or by calls to the defineClass method in the class ClassLoader. The structure of this class is given below :

```
          public‘‘final’‘class’‘java.lang.Class
‘‘‘‘                                    extends’‘java.lang.Object’‘
          ‘‘{
‘‘‘‘‘‘‘‘             // Methods
‘‘‘‘      public static Class forName(String’‘className);
‘‘‘‘      public ClassLoader getClassLoader();
‘‘‘‘      public Class[] getInterfaces();
‘‘‘‘      public String getName();
‘‘‘‘      public Class getSuperclass();
‘‘‘‘      public boolean isInterface();
‘‘‘‘      public Object newInstance();
‘‘‘‘      public String toString();
          ‘‘}
The operation performed by the various methods in it are tabulated in Table 2.3.
          ‘‘
```

Table 2.3

Method	Description
forName(String className)	Returns the Class object associated with the class with the given string name. For example, the following code fragment returns the runtime Class descriptor for the class named `java.lang.Thread` : Class t = Class.forName("java.lang.Thread") **Throws :** `ClassNotFoundException`, if the class could not be found.
getClassLoader()	Determines the class loader. **Returns :** The class loader that created the class or

	interface represented by this object, or null if the class was not created by a class loader.
getInterfaces()	Determines the interfaces implemented by the class or interface represented by this object. If this object represents a class, the return value is an array containing objects representing all interfaces implemented by the class. The order of the interface objects in the array corresponds to the order of the interface names in the implements clause of the declaration of the class represented by this object. If this object represents an interface, the array contains objects representing all interfaces extended by the interface. The order of the interface objects in the array corresponds to the order of the interface names in the extends clause of the declaration of the interface represented by this object. If the class or interface implements no interfaces, the method returns an array of length 0. **Returns :**an array of interfaces implemented by this class.
getName()	**Returns** : The fully qualified name of the class or interface represented by this object.
getSuperclass()	If this object represents any class other than the class `Object`, then the object that represents the superclass of that class is returned. If this object is the one that represents the class `Object` or this object represents an interface, null is returned.
isInterface()	**Returns :** true if this object represents an interface; false otherwise.
newInstance()	Creates a new instance of a class. **Returns** : a newly allocated instance of the class represented by this object. This is done exactly as if by a new expression with an empty argument list. **Throws :** `InstantiationException`, If an application tries to instantiate an abstract class or an interface, or if the instantiation fails for some other reason. It also throws `IllegalAccessException`, If the class or initializer is not accessible.
toString()	Converts the object to a string. The string representation is the string "class" or "interface" followed by a space and then the fully qualified name of the class.

Class ClassLoader

The class ClassLoader is an abstract class. Applications implement subclasses of ClassLoader in order to extend the manner in which the JVM dynamically loads classes. Normally, the JVM loads classes from the local file system in a platform- dependent manner. For example, on UNIX systems, the Virtual Machine loads classes from the directory defined by the CLASSPATH environment variable. However, some classes may not originate from a file; they may originate from other sources, such as the network, or they could be constructed by an application. The method defineClass convets an array of bytes into an instance of class Class. Instances of this newly defined class can be created using the newInstance method in class Class.

The methods and constructors of objects created by a ClassLoader may reference other classes. To determine the class(es) referred to, the Java Virtual Machine calls the loadClass method of the class loader that oroginally created

the class. If the JVM only needs to determine if the class exist and if it does exist to know its superclass, the resolve flag is set to false. However, if an instance of the class is being craeted or any of its method is being called, the class must also be resolved. In this case, the resolve flag is set to true, and the resolveClass method should called. The class body is described as below :

```
          public‘‘abstract class’‘java.lang.ClassLoader
‘‘‘‘                                                extends’‘java.lang.Object
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      protected ClassLoader();
                    /*        Creates a new class loader and initializes it. If there is a security
manager, its checkCreateClassLoader method is called. It may result in a security exception, if the
current thread does not have permission to create a new class loader.
                    */


‘‘‘‘‘‘    //        Methods
‘‘‘‘      protected final Class defineClass(byte’‘data[], int’‘offset, int’‘length);
‘‘‘‘      protected final Class findSystemClass(String’‘name);
‘‘‘‘      protected abstract Class loadClass(String’‘name, boolean’‘resolve);
‘‘‘‘      protected final void resolveClass(Class’‘c);
          }



          Operation performed by the various methods in this class is listed in Table 2.4.
```

Table 2.4

Method	Description
defineClass(byte data[], int offset, int length)	Converts an array of bytes into an instance of class `Class`. **Returns :** The class object which was created from the data. **Throws :** `ClassFormatError`, if the data does not contain a valid `Class`.
findSystemClass(String name)	Finds the system class with the specified name, loading it in if necessary. A syster class is a class loaded from the local file system in a platform- dependent way. It has no class loader.
loadClass(String name, boolean resolve)	Requests the class loader to load a class with the specified name. The `loadClass` method is called by the Java Virtual Machine when a class loade by a class loader first references another class. Every subclass of class `ClassLoader` must define this method. If the resolve flag is true, the metho should call the `resolveClass` method on the resulting class object. Class loader should use a hash table or other cache to avoid defining classes with the same name multiple times. **Returns :** the resulting `Class`, or null if it was not found. **Throws :** `ClassNotFoundException`, if the `ClassLoader` cannot find a definition for the class.
resolveClass(Class c)	Resolves the class so that an instance of the class can be created, or so that one its methods can be called. This method should be called by `loadClass` if the

	resolve flag is true.

Class Compiler

The class Compiler is provided in support of Java-to-native-code compilers and related services. When the JVM first starts, it determines if a system property java.compiler exists. If so, it is assumed to be the name of a library (whose exact location and type is platform-dependent); the loadLibrary method in class System is called to load that library. If this loading suceeds, the function named java.lang.Compiler.start() in that library is called. In case, if there is no compiler available, these methods do nothing.

```
          The body of this class is here under.
          public‘‘final’‘class’‘java.lang.Compiler
‘‘‘‘                                                extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public static Object command(Object’‘any);
‘‘        public static boolean compileClass(Class’‘clazz);
‘‘‘‘      public static boolean compileClasses(String’‘string);
‘‘‘‘      public static void disable();
‘‘‘‘      public static void enable();
          }
          Methode are described in Table 2.5.
```

Table 2.5

Method	Description
command(Object any)	The Compiler should examine the argument type and its fields and perform some documented operation. No specific operations are required. **Returns :** a compiler-specific value, or null if no compiler is available.
compileClass(Class clazz)	The indicated class is compiled. **Returns :** true if the compilation succeeded; false if the compilation failed or no compiler is available.
compileClasses(String string)	Indicates that the application would like the third-party software to compile all classes whose name matches the indicated string. **Returns :** true if the compilation succeeded; false if the compilation failed or no compiler is available.
disable()	Cause the Compiler to cease operation.
enable()	Cause the Compiler to resume operation.

Class Double

This class wraps a value of the primitive type double in an object. An object of type Double contains a single field whose type is double. In addition, this class provides a number of methods for converting a double to a String and a String to a double, as well as other constants and methods useful when dealing with a double. The various components in this class is listed as below :

```
                    public‘‘final‘‘class‘‘java.lang.Double
                                                                    extends‘‘java.lang.Number
                                {
                                          //        Member elements
                                public final static double MAX_VALUE;
                                //        MAX_VALUE = 1.79769313486231570e+308d, The largest positive
value of type double.
                                public final static double MIN_VALUE;
                                //        MIN_VALUE = 4.94065645841246544e-324, The smallest positive
finite value of type double.
                                public final static double NaN;
                                //        NaN = 0.0 / 0.0 , a Not-a-Number value of type double.
                                public final static double NEGATIVE_INFINITY;
                                //        NaN = 0.0 / 0.0 , a‘‘Not-a-Number value of type double.
                                public final static double POSITIVE_INFINITY;
                                //        POSITIVE_INFINITY = 1.0 / 0.0 , the‘‘positive infinity of type
double.
                                          //        Constructors
                                public Double(double‘‘value);
                                //        Constructs a newly allocated Double object that represents the
primitive double argument.
                                public Double(String‘‘s);
                                /*        Constructs a newly allocated Double object that represents the
floating- point value of type double represented by the string. The string is converted to a double value
as if by the valueOf method. This method throws NumberFormatException,‘‘if the string does not contain a
parsable number.’‘
                                */        ‘‘
                                //        Methods
                                public static long doubleToLongBits(double‘‘value);
                                public double doubleValue();
                                public boolean equals(Object‘‘obj);
                                public float floatValue();
                                public int hashCode();
                                public int intValue();
                                public boolean isInfinite();
                                public static boolean isInfinite(double‘‘v);
                                public boolean isNaN();
                                public static boolean isNaN(double‘‘v);
                                public static double longBitsToDouble(long‘‘bits);
```

```
                    public long longValue();
                    public String toString();
                    public static String toString(double'‘d);
                    public static Double valueOf(String'‘s);
               ‘‘‘‘}
```

The performance of various methods in this class are stated in Table 2.6

Table 2.6

Method	Description
doubleToLongBits(double value)	The result is a representation of the floating-point argument according to the IEEE 754 floating-point "double format" bit layout. Bit 63 represents the sign of the floating-point number. Bits 62-52 represent the exponent. Bits 51-0 represent the significand (sometimes called the mantissa) of the floating-point number. If the argument is positive infinity, the result is 0x7ff0000000000000L. If the argument is negative infinity, the result is 0xfff0000000000000L. If the argument is NaN, the result is 0x7ff8000000000000L.
doubleValue()	The double value represented by this object.
equals(Object obj)	The result is true if and only if the argument is not null and is a `Double` object that represents a double that has the identical bit pattern to the bit pattern of the double represented by this object.
floatValue()	the double value represented by this object is converted to type `float` and the result of the conversion is returned.
hashCode()	a hash code value for this object.
intValue()	the double value represented by this object is converted to type 191`int` and the result of the conversion is returned.
isInfinite()	True if the value represented by this object is positive infinity or negative infinity; false otherwise.
isInfinite(double v)	true if the value of the argument is positive infinity or negative infinity; false otherwise.
isNaN()	true if the value represented by this object is NaN; false otherwise.
isNaN(double v)	true if the value of the argument is NaN; false otherwise.
longBitsToDouble(long bits)	The argument is considered to be a representation of a floating-point value according to the IEEE 754 floating-

	point "double precision" bit layout. That floating-point value is returned as the result. If the argument is 0x7f800000 00000000L, the result is positive infinity. If the argument is 0xff80000000000000L, the result is negative infinity. If the argument is any value in the range 0x7ff0000000000001L through 0x7fffffff ffffffffL or in the range 0xfff0000 000000001L through 0xffffffffffffffffL, the result is NaN. All IEEE 754 NaN values are, in effect, lumped together by the Java language into a single value.
longValue()	the double value represented by this object is converted to type long and the result of the conversion is returned.
toString()	The primitive double value represented by this object is converted to a string exactly as if by the method toString of one argumen.
toString(double d)	Creates a string representation of the double argument.
valueOf(String s)	Parses a string into a `Double`. A newly constructed Double initialized to the value represented by the string argument. **Throws :** `NumberFormatException`, if the string does not contain a parsable number.

Class Float

This class wraps a value of type float in an object. An object of type Float contains a single field whose type is float. In addition, this class provides a number of methods for converting a float to a String and a String to a float, as well as other constants and methods useful when dealing with a float. The structure of this class is as stated below :

```
          public‘‘final’‘class’‘java.lang.Float
‘‘‘‘                                                extends’‘java.lang.Number’‘
          {
‘‘‘‘‘‘‘‘ //        Member elements
                    public final static float MAX_VALUE;
                              //        MAX_VALUE’‘= 3.40282346638528860e+38, is the largest positive
finite value of type          float.
                    public final static float MIN_VALUE;
                              //        MIN_VALUE’‘ = 1.40129846432481707e-45, is the smallest positive
value of type float.
                    public final static float NaN;
                              //        NaN = 0.0f/0.0f , is the Not-a-Number value of type float.
                    public final static float NEGATIVE_INFINITY;
                              //        NEGATIVE_INFINITY’‘= -1.0f/0.0f , is the negative infinity of
type float.
                    public final static float POSITIVE_INFINITY;
                              //        POSITIVE_INFINITY = 1.0f/0.0f’‘, is the positive infinity of
type float.
```

```
                              //        Constructors
                    public Float(double’‘value);
                              //        Constructs a newly allocated Float object that represents the
argument converted to type float.
                    public Float(float’‘value);
                              //        Constructs a newly allocated Float object that represents the
primitive float argument.
                    public Float(String’‘s);
                              /*        Constructs a newly allocated Float object that represents the
floating-point value of type float represented by the string. The string is converted to a float value as
if by the valueOf method. Throws NumberFormatException, if the string does not contain a parsable
number.’‘
                              */
                              //        Methods
                    public double doubleValue();
                    public boolean equals(Object’‘obj);
                    public static int floatToIntBits(float’‘value);
                    public float floatValue();
                    public int hashCode();
                    public static float intBitsToFloat(int’‘bits);
                    public int intValue();
                    public boolean isInfinite();
                    public static boolean isInfinite(float’‘v);
                    public boolean isNaN();
                    public static boolean isNaN(float’‘v);
                    public long longValue();
                    public String toString();
                    public static String toString(float’‘f);
                    public static Float valueOf(String’‘s);
          }
Operations performed by the various methods in trhis class are listed in Table 2.7.
```

Table 2.7

Method	Description
doubleValue()	The float value represented by this object is converted to type double and the result of the conversion is returned.
equals(Object obj)	**Returns:** true if the objects are the same; false otherwise. Note that in most cases, for two instances of class Float, f1 and f2, the value of f1.equals(f2) is true if and only if f1.floatValue() == f2.floatValue() also has the value true. However, there are two exceptions:

	If f1 and f2 both represent Float.NaN,then the equals method returns true, even though Float.NaN==Float.NaN has the value false. If f1 represents +0.0f while f2 represents -0.0f, or vice versa, the equal test has the value false, even though 0.0f==-0.0f has the value true.
floatToIntBits(float value)	**Returns:** the bits that represent the floating point number. The result is a representation of the floating-point argument according to the IEEE 754 floating-point "single precision" bit layout. Bit 31 represents the sign of the floating-point number. Bits 30-23 represent the exponent. Bits 22-0 represent the significand (sometimes called the mantissa) of the floating-point number. If the argument is positive infinity, the result is 0x7f800000. If the argument is negative infinity, the result is 0xff800000. If the argument is `NaN`, the result is 0x7fc00000.
floatValue()	**Returns:** The float value represented by this object is returned.
hashCode()	**Returns**: a hash code value for this object.
intBitsToFloat(int bits)	**Returns:** the single format floating-point value with the same bit pattern. The argument is considered to be a representation of a floating-point value according to the IEEE 754 floating-point "single precision" bit layout. That floating-point value is returned as the result. If the argument is 0x7f800000, the result is positive infinity. If the argument is 0xff800000, the result is negative infinity. If the argument is any value in the range 0x7f800001 through 0x7f8fffff or in the range 0xff800001 through 0xff8fffff, the result is NaN. All IEEE 754 NaN values are, in effect, lumped together by the Java language into a single value194.
intValue()	**Returns:** The float value represented by this object is converted to type int and the result of the conversion is returned.
isInfinite()	**Returns:** true if the value value represented by this object is positive infinity or negative infinity; false otherwise.
isInfinite(float v)	**Returns:** true if the argument is positive infinity or negative infinity; false otherwise.
isNaN()	**Returns:** true if the value value represented by this object is NaN; false otherwise.
isNaN(float v)	**Returns:** true if the argument is NaN; false otherwise.
longValue()	**Returns:** The float value represented by this object is converted to type long and the result of the conversion is returned.

toString()	The primitive float value represented by this object is converted to a String exactly as if by the method toString of one argument **Returns:** a String representation of this object.
toString(float f)	Creates a string representation of the float argument. The values NaN, NEGATIVE_INFINITY, POSITIVE_INFINITY, -0.0, and +0.0 are represented by the strings "NaN", "-Infinity", "Infinity", "-0.0" and "0.0", respectively. **Returns:** a string representation of the argument.
valueOf(String s)	Parses a string into a `Float`. **Returns:** a newly constructed Float initialized to the value represented by the String argument. **Throws** : `NumberFormatException`, If the string does not contain a parsable number.

Class Integer

This class wraps a value of the primitive type int in an object. An object of type Integer contains a single field whose type is int. In addition, this class provides a number of methods for converting an int to a String and a String to an int, as well as other constants and methods useful when dealing with an int. The body of this class is as stated as below :

```
          public‘‘final’‘class’‘java.lang.Integer
‘‘‘‘                                                  extends’‘java.lang.Number’‘
          {
                    //        Member elements
          public final static int MAX_VALUE;
          //        MAX_VALUE = 2147483647, the largest value of type int.
          public final static int MIN_VALUE;
          //        MIN_VALUE = -2147483648, the smallest value of type int.


          //        Constructors
          public Integer(int’‘value);
          //        Constructs a newly allocated Integer object that represents the primitive int
argument.
          public Integer(String’‘s);
          /*        Constructs a newly allocated Integer object that represents the value represented by
the string The string is converted to an int value as if by the valueOf method.
                    Throws : NumberFormatException, if the String does not contain a parsable integer.
          */




          //        Methods
          public double doubleValue();
          public boolean equals(Object’‘obj);
          public float floatValue();
```

```
public static Integer getInteger(String’‘nm);
public static Integer’‘getInteger(String’‘nm, intval);
public static Integer getInteger(String’‘nm, Integer’‘val);
public int hashCode();
public int intValue();
public long longValue();
public static int parseInt(String’‘s);
public static int parseInt(String’‘s, int’‘radix);
public static String toBinaryString(int’‘i);
public static String toHexString(int’‘i);
public static String toOctalString(int’‘i);
public String toString();
public static String toString(int’‘i);
public static String’‘toString(int’‘i, int’‘radix);
public static Integer valueOf(String’‘s);
public static Integer valueOf(String’‘s, int’‘radix);
}

Methods are stated in Table 2.8.
```

Table 2.8

Method	Description
doubleValue()	**Returns:** The int value represented by this object is converted to type double and the result of the conversion is returned.
equals(Object obj)	The result is true if and only if the argument is not null and is an `Integer` object that contains the same int value as this object.
floatValue()	**Returns:** the int value represented by this object is converted to type float and the result of the conversion is returned.
getInteger(String nm)	Determines the integer value of the system property with the specified name. The first argument is treated as the name of a system property to be obtained as if by the method `System.getProperty`.The string value of this property is then interpreted as an integer value and an `Integer` object representing this value is returned.
getInteger(String nm, int val)	Determines the integer value of the system property with the specified name. The first argument is treated as the name of a system property to be obtained as if by the method `System.getProperty`. The string value of this property is then interpreted as an integer value and

	an `Integer` object representing this value is returned.
getInteger(String nm, Integer val)	The first argument is treated as the name of a system property to be obtained as if by the method `System.getProperty`. The string value of this property is then interpreted as an integer value and an `Integer` object representing this value is returned. If the property value begins with "0x" or "#", not followed by a minus sign, the rest of it is parsed as a hexadecimal integer exactly as for the method Integer.valueOf with radix 16. If the property value begins with "0" then it is parsed as an octal integer exactly as for the method Integer.valueOf with radix 8. Otherwise the property value is parsed as a decimal integer exactly as for the method Integer.valueOf with radix 10. The second argument is the default value. If there is no property of the specified name, or if the property does not have the correct numeric format, then the second argument is returned.
hashCode()	**Returns:** a hash code value for this object.
intValue()	**Returns:** The int value represented by this object is returned.
longValue()	**Returns:** The int value represented by this object is converted to type long and the result of the conversion is returned.
parseInt(String s)	Parses the string argument as a signed decimal integer. The characters in the string must all be decimal digits, except that the first character may be an ASCII minus sign '-' to indicate a negative value. **Returns:** the integer represented by the argument in decimal. **Throws** :`NumberFormatException`, if the string does not contain a parsable integer.
parseInt(String s, int radix)	Parses the string argument as a signed integer in the radix specified by the second argument. The characters in the string must all be digits of the specified radix (as determined by whether `Character.digit` returns a nonnegative value), except that the first character may be an ASCII minus sign **-** to indicate a negative value. The resulting integer value is returned.198 **Returns:** the integer represented by the string argument in the specified radix. **Throws** : `NumberFormatException`, if the string does not contain a parsable integer.
toBinaryString(int i)	Creates a string representation of the integer argument as an unsigned integer in base 2. The unsigned integer value is the argument plus 2^5 if the argument is negative; otherwise it is equal to the argument. This value is converted to a string of ASCII digits in binary (base 2) with no extra leading 0's. **Returns:** the string representation of the unsigned integer value represented by the argument in binary (base 2).

toHexString(int i)	Creates a string representation of the integer argument as an unsigned integer in base 16. The unsigned integer value is the argument plus 2^5 if the argument is negative; otherwise it is equal to the argument. This value is converted to a string of ASCII digits in hexadecimal (base 16) with no extra leading 0's. **Returns:** the string representation of the unsigned integer value represented by the argument in hexadecimal (base 16).
toOctalString(int i)	Creates a string representation of the integer argument as an unsigned integer in base 8. The unsigned integer value is the argument plus 2^5 if the argument is negative; otherwise it is equal to the argument. This value is converted to a string of ASCII digits in octal (base 8) with no extra leading 0's. **Returns:** the string representation of the unsigned integer value represented by the argument in octal (base 8).
toString()	**Returns:** a string representation of the value of this object in base 10.
toString(int i)	**Returns:** a string representation of the argument in base 10.
toString(int i, int radix)	Creates a string representation of the first argument in the radix specified by the second argument. If the radix is smaller than Character.MIN_RADIX or larger than Character. MAX_RADIX, then the radix 10 is used instead. If the first argument is negative, the first element of the result is the ASCII minus characer '-' . If the first argument is not negative, no sign character appears in the result. The following ASCII characters are used as digits: 0123456789abcdefghijklmnopqrstuvwxyz **Returns:** a string representation of the argument in the specified radix.
valueOf(String s)	**Returns:** a newly constructed Integer initialized to the value. represented by the string argument. **Throws** : `NumberFormatException`, if the string does not contain a parsable integer.
valueOf(String s, int radix)	**Returns:** a newly constructed Integer initialized to the value represented by the string argument in the specified radix . 198198**Throws :** `NumberFormatException`, If the String does not contain a parsable integer.

Class Long

This class wraps a value of the primitive type long in an object. An object of type Long contains a single field whose type is long. In addition, this class provides a number of methods for converting a long to a String and a String to a long, as well as other constants and methods useful when dealing with a long.

```
public‘‘final’‘class’‘java.lang.Long
```

```
                                                        extends‘‘java.lang.Number’‘
          {
                              //        Member elements
                    public final static long MAX_VALUE;
                    //        MAX_VALUE = 9223372036854775807L , the largest value of type long.
                    public final static long MIN_VALUE;
                    //        MIN_VALUE = -9223372036854775808L , the smallest value of type long.


                    //        Constructors
                    public Long(long’‘value);
                    //        Constructs a newly allocated Long object that represents the primitive long
argument.
                    public Long(String’‘s);
                    /*        Constructs a newly allocated Long object that represents the value
represented by the string. The string is converted to an long value as if by the valueOf method.
                              Throws : NumberFormatException, if the String does not contain a parsable
long integer.
                    */
                    //        Methods
                    public double doubleValue();
                    public boolean equals(Object’‘obj);
                    public float floatValue();
                    public static Long getLong(String’‘nm);
                    public static Long getLong(String’‘nm, long’‘val);
                    public static Long getLong(String’‘nm, Long’‘val);
                    public int hashCode();
                    public int intValue();
                    public long longValue();
                    public static long parseLong(String’‘s);
                    public static long parseLong(String’‘s, int’‘radix);
                    public static String toBinaryString(long’‘i);
                    public static String toHexString(long’‘i);
                    public static String toOctalString(long’‘i);
                    public String toString();
                    public static String toString(long’‘i);
                    public static String toString(long’‘i, int’‘radix);
                    public static Long valueOf(String’‘s);
                    public static Long valueOf(String’‘s, int’‘radix);
          }


          Methods are described in Table 2.9.
```

Table 2.9

Method	Description
doubleValue()	**Returns:**The long value represented by this object is converted to type double and the result of the conversion is returned.
equals(Object obj)	The result is true if and only if the argument is not null and is a `Long` object that contains the same long value as this object. **Returns :** true if the objects are the same; false otherwise.
floatValue()	**Returns:** The long value represented by this object is converted to type float and the result of the conversion is returned.
getLong(String nm)	Determines the long value of the system property with the specified name. The first argument is treated as the name of a system property to be obtained as if by the method `System.getProperty`. The string value of this property is then interpreted as a long value and a Long object representing this value is returned. If there is no property with the specified name, or if the property does not have the correct numeric format, then null is returned.
getLong(String nm, long val)	Determines the long value of the system property with the specified name. The first argument is treated as the name of a system property to be obtained as if by the method `System.getProperty`. The string value of this property is then interpreted as a long value and a `Long` object representing this value is returned.
getLong(String nm, Long val)	Determines the long value of the system property with the specified name. The first argument is treated as the name of a system property to be obtained as if by the method `System.getProperty`. The string value of this property is then interpreted as a long value and a Long object representing this value is returned. If the property value begins with "0x" or "#", not followed by a minus sign, the rest of it is parsed as a hexadecimal integer exactly as for the method Long.valueOf with radix 16. If the property value begins with "0" then it is parsed as an octal integer exactly as for the method Long.valueOf with radix 8. Otherwise the property value is parsed as a decimal integer exactly as for the method Long.valueOf with radix 10. The second argument is the default value. If there is no property of the specified name, or if the property does not have the correct numeric format, then the second argument is returned.

hashCode()	Returns: a hash code value for this object.
intValue()	Returns: the long value represented by this object is converted to type int and the result of the conversion is returned.
longValue()	**Returns:** the long value represented by this object is returned.
parseLong(String s)	Parses the string argument as a signed decimal long. The characters in the string must all be decimal digits, except that the first character may be an ASCII minus sign '-' to indicate a negative value. **Returns :** the long represented by the argument in decimal. T**hrows** : `NumberFormatException`, if the string does not contain a parsable long.
parseLong(String s, int radix)	Parses the string argument as a signed long in the radix specified by the second argument. The characters in the string must all be digits of the specified radix (as determined by whether `Character.digit` returns a nonnegative value), except that the first character may be an ASCII minus sign **-** to indicate a negative value. The resulting long value is returned. **Returns:** the long represented by the string argument in the specified radix. **Throws** : `NumberFormatException`, if the string does not contain a parsable integer
toBinaryString(long i)	Creates a string representation of the long argument as an unsigned integer in base 2. The unsigned long value is the argument plus 2^8 if the argument is negative; otherwise it is equal to the argument. This value is converted to a string of ASCII digits in binary (base 2) with no extra leading 0's. **Returns:** the string representation of the unsigned long value represented by the argument in binary (base 2).
toHexString(long i)	Creates a string representation of the long argument as an unsigned integer in base 16. The unsigned long value is the argument plus2^{16} if the argument is negative; otherwise it is equal to the argument. This value is converted to a string of ASCII digits in hexadecimal (base 16) with no extra leading 0's. **Returns:** the string representation of the unsigned long value represented by the argument in hexadecimal (base 16).
toOctalString(long i)	Creates a string representation of the long argument as an unsigned integer in base 8. The unsigned long value is the argument plus 2^8 if the argument is negative; otherwise it is equal to the argument. This value is converted to a string of ASCII digits in octal (base 8) with no extra leading 0's. **Returns:** the string representation of the unsigned long value represented by the argument in octal (base 8).
toString()	**Returns:** a string representation of this object in base 10.
toString(long i)	**Returns:** a string representation of the argument in base 10.

***t*oString**(long i, int radix)	Creates a string representation of the first argument in the radix specified by the second argument. If the radix is smaller than Character.MIN_RADIX or larger than Character.MAX_RADIX, then the radix 10 is used instead. If the first argument is negative, the first element of the result is the ASCII minus sign '-' . If the first argument is not negative, no sign character appears in the result. The following ASCII characters are used as digits. 0123456789abcdefghijklmnopqrstuvwxyz. **Returns:** a string representation of the argument in the specified radix.
valueOf(String s)	**Returns:** a newly constructed Long initialized to the value represented by the string argument. **Throws** : NumberFormatException, if the String does not contain a parsable
valueOf(String s, int radix)	**Returns:** a newly constructed Long initialized to the value represented by the string argument in the specified radix. **Throws** *:* NumberFormatException, If the String does not contain a parsable long.

Class Math

The class Math contains methods for performing basic numerical operations such as the elementary exponential, logarithm, square root, and trigononetric functions. The structure of this class is stated as below :

```
          public‘‘final’‘class’‘java.lang.Math
‘‘‘‘                                    extends’‘java.lang.Object
          {
‘‘‘‘‘‘‘‘ //        Member elements
‘‘‘‘      public final static double E;
                    /*        E = 2.7182818284590452354 , the double value that is closer than any other
to e, the base of the natural logarithms.
                    */
‘‘‘‘‘‘    public final static double PI;
                    /*        PI = 3.14159265358979323846 , the double value that is closer than any
other to p, the ratio of the circumference of a circle to its diameter.
                    */
‘‘‘‘‘‘    //        Methods
‘‘‘‘      public static double abs(double’‘a);
‘‘‘‘      public static float abs(float’‘a);
‘‘‘‘      public static int abs(int’‘a);
‘‘‘‘      public static long abs(long’‘a);
‘‘‘‘      public static double acos(double’‘a);
‘‘‘‘      public static double asin(double’‘a);
‘‘‘‘      public static double atan(double’‘a);
‘‘‘‘      public static double atan2(double’‘a, double’‘b);
‘‘        ‘‘‘‘public static double ceil(double’‘a);
```

```
‘‘        ‘‘‘‘public static double cos(double’‘a);
‘‘‘‘      public static double exp(double’‘a);
‘‘‘‘      public static double floor(double’‘a);
‘‘        ‘‘‘‘public static double IEEEremainder(double’‘f1, double’‘f2);
‘‘‘‘      public static double log(double’‘a);
‘‘‘‘      public static double max(double’‘a, double’‘b);
‘‘‘‘      public static float max(float’‘a, float’‘b);
‘‘‘‘      public static int max(int’‘a, int’‘b);
‘‘‘‘      public static long max(long’‘a, long’‘b);
‘‘        ‘‘‘‘public static double min(double’‘a, double’‘b);
‘‘‘‘      public static float min(float’‘a, float’‘b);
‘‘‘‘‘‘    public static int min(int’‘a, int’‘b);
‘‘‘‘      public static long min(long’‘a, long’‘b);
‘‘‘‘      public static double pow(double’‘a, double’‘b);
‘‘‘‘      public static double random();
‘‘        ‘‘‘‘public static double rint(double’‘a);
‘‘        ‘‘‘‘public static long round(double’‘a);
‘‘        ‘‘‘‘public static int round(float’‘a);
‘‘‘‘      public static double sin(double’‘a);
‘‘        ‘‘‘‘public static double sqrt(double’‘a);
‘‘‘‘      public static double tan(double’‘a);
          }
          Methods are described as in Table 2.10.
```

Table 2.10

Method	Description
abs(double a)	Calculates the absolute value of the argument. **Returns:** the absolute value of the argument.
abs(float a)	Calculates the absolute value of the argument. **Returns:** the absolute value of the argument.
abs(int a)	Calculates the absolute value of the argument. If the argument is not negative, the argument is returned. If the argument is negative, the negation of the argument is returned. Note that if the argument is equal to the value of `Integer.MIN_VALUE`, the most negative representable int value, the result is that same value, which is negative. **Returns:** the absolute value of the argument.
abs(long a)	Calculates the absolute value of the argument.If the argument is not negative, the argument is returned. If the argument is negative, the negation of the argument is returned. Note that if the argument is equal to the value

	of `Long.MIN_VALUE`, the most negative representable int value, the result is that same value, which is negative. **Returns:** the absolute value of the argument.
acos(double a)	**Returns:** the arc cosine of the argument.
asin(double a)	**Returns:** the arc sine of the argument.
atan(double a)	**Returns:** the arc tangent of the argument.
atan2(double a, double b)	**Returns:** the t component of the polar coordinate the (p, t) at corresponds to the cartesian coordinate . (a.b).
ceil(double a)	Returns the smallest (closest to negative infinity) double value that is not less than the argument and is equal to a mathematical integer.
cos(double a)	Returns: the cosine of the argument.
exp(double a)	**Returns:** the value e^a, where where e is the base of the natural logarithms.
floor(double a)	Returns the largest (closest to positive infinity) double value that is not greater than the argument and is equal to a mathematical integer.
IEEEremainder(double f1, double f2)	Computes the remainder operation on two arguments as prescribed by the IEEE 754 standard: the remainder value is mathematically equal to f1/f2 where f2 is the mathematical integer closest to the exact mathematical value of the quotient f1/f2 205205, and if two mathematical integers are equally close to the f1/f2 then f2 is the integer that is even. If the remainder is zero, its sign is the same as the sign of the first argument. **Returns:** the remainder when f1 is divided by f2.
log(double a)	**Returns:** the value log a, the natural logarithm of a.
max(double a, double b)	**Returns:** the larger of a and b.
max(float a, float b)	**Returns:** the larger of a and b
max(int a, int b)	**Returns:** the larger of a and b.
max(long a, long b)	**Returns:** the larger of a and b.
min(double a, double b)	**Returns:** the smaller of a and b
min(float a, float b)	**Returns:** the smaller of a and b
min(int a, int b)	**Returns:** the smaller of a and b.
min(long a, long b)	**Returns:** the smaller of a and b.

pow(double a, double b)	**Returns:** the value a^b.
random()	**Returns:** a pseudorandom double between 0.0 and 1.0.
rint(double a)	Calculates the closest integer to the argument. **Returns:** the closest double value to a that is equal to a mathematical integer. If two double values that are mathematical integers are equally close to the value of the argument, the result is the integer value that is even.
round(double a)	Calculates the closest long to the argument. If the argument is negative infinity or any value less than or equal to the value of `Long.MIN_VALUE`, the result is equal to the value of `Long.MIN_VALUE`. If the argument is positive infinity or any value greater than or equal to the value of Long.MAX_VALUE, the result is equal to the value of `Long.MAX_VALUE`.. **Returns:** the value of the argument rounded to the nearest long value.
round(float a)	Calculates the closest int to the argument. If the argument is negative infinity or any value less than or equal to the value of `Integer.MIN_VALUE`, the result is equal to the value of `Integer.MIN_VALUE`. If the argument is positive infinity or any value greater than or equal to the value of `Integer.MAX_VALUE`, the result is equal to the value of `Integer.MAX_VALUE`. **Returns:** the value of the argument rounded to the nearest int value.
sin(double a)	**Returns:** the sine of the argument
sqrt(double a)	**Returns:** the value of square root of `a`. If the argument is NaN or less than zero, the result is NaN.
tan(double a)	**Returns:** the tangent of the argument.

Class Number

The abstract class Number is the superclass of classes Float. Double, Integer, and Long. Subclasses of Number must provide methods to convert the represented numeric value to int, long, float, and double.

```
          public‘‘abstract class’‘java.lang.Number
‘‘‘‘                                              extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract double doubleValue();
‘‘‘‘      public abstract float floatValue();
‘‘‘‘      public abstract int intValue();
‘‘‘‘      public abstract long longValue();
          }
```

```
Methods are as stated in Table 2.11.
```

Table 2.11

Method	Description
doubleValue()	**Returns:** The numeric value represented by this object is returned after conversion to type double.
floatValue()	**Returns:** The numeric value represented by this object is returned after conversion to type float.
intValue()	**Returns:** The numeric value represented by this object is returned after conversion to type int.
longValue()	**Returns :** The numeric value represented by this object is returned after conversion to type long.

Class Object

Class Object is the root of the class hierarchy. Every class has Object as a superclass. parent. All objects, including arrays, implement the methods of this class.

```
public''class''java.lang.Object
{
                        //        Constructors
          public Object();
          //        Allocates a new instance of class Object.
          //        Methods
          protected Object clone();
          public boolean equals(Object''obj);
          protected void finalize();
          public final Class getClass();
          public int hashCode();
          public final void notify();
          public final void notifyAll();
          public String toString();
          public final void wait();
          public final void wait(long''timeout);
          public final void wait(long''timeout, int''nanos);
}
Methods are described in Table 2.12.
```

Table 2.12

Method	Description
clone()	The clone method of class `Object` creates a new object of the same class as this object. It then initializes each of the new object's fields by assigning it the same value as the corresponding field in this object. No constructor is called. The `clone` method of class `Object` will only clone an object whose class indicates that it is willing for its instances to be cloned. A class indicates that its instances can be cloned by declaring that it implements the `Cloneable` Interface. **Throws :** `OutOfMemoryError` , if there is not enough memory. Also throws `CloneNotSupportedException`, when the object's class does not support the Cloneable interface. Subclasses that override the clone method can also throw this exception to indicate that an instance cannot be cloned.
equals(Object obj)	Indicates whether some other object is "equal to" this one. The equals method implements an equivalence relation : It is reflexive: for any reference value x, x.equals(x) should return true. It is symmetric: for any reference values x and y, x.equals(y) should return true if and only if y.equals(x) returns true. It is transitive: for any reference values x, y, and z, if x.equals(y) returns true and y.equals(z) returns true then x.equals(z) should return true. It is consistent: for any reference values x and y, multiple invocations of x.equals(y) consistently return true or consistently return false. For any reference value x, x.equals(null) should return false. For any reference value x, x.equals(null) should return false. The equals method for class Object implements the most discriminating possible equivalence relation on objects; that is, for any reference values x and y, this method returns true if and only if x and y refer to the same object (x==y has the value true).
finalize()	The finalize method is called by the garbage collector on an object when garbage collection determines that there are no more references to the object. A subclass overrides the finalize method in order to dispose of system resources or to perform other cleanup. Any exception thrown by the finalize method causes the finalization of this object to be halted, but is otherwise ignored. The finalize method in `Object` does nothing
getClass()	Determines the run-time class of an object. **Returns:** the object of type `Class` that represents the run-time class of the object .
hashCode()	Calculates a hash code value for the object. This method is supported for the benefit of hash tables such as those provided by java.util.Hashtable.The general contract of `hashCode` is : Whenever invoked on the same object more than once during an execution of a Java application, the `hashCode` method must

	consistently return the same integer. This integer need not remain consistent from one execution of an application to another execution of the same application. 208 If two objects are equal according to the equals method, then calling the `hashCode` method on each of the two objects must prtoduce the same integer result.
notify()	Wakes up a single thread that is waiting on this object's monitor. A thread waits on an object's monitor by calling one of the wait methods.This method should only be called by a thread that is the owner of this object's monitor. A thread becomes the owner of the object's monitor in one of three ways: By executing a synchronized instance method of that object. By executing the body of a synchronized statement that synchronizes on the object. For objects of type Class, by executing a synchronized static method of that class. Only one thread at a time can only an object's monitor. **Throws :** IllegalMonitorStateException, If the current thread is not the owner of this object's monitor
notifyAll()	Wakes up all threads that are waiting on this object's monitor. A thread waits on an object's monitor by calling one of the wait methods.This method should only be called by a thread that is the owner of this object's monitor. **Throws** `IllegalMonitorStateException`, If the current thread is not the owner of this object's monitor.
toString()	Creates a string representation of the object. In general, the `toString` method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. The `toString` method for class Object returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object.
wait()	Waits to be notified by another thread of a change in this object. The current thread must own this object's monitor. The thread releases ownership of this monitor and waits until another thread notifies threads waiting on this object's monitor to wake up either through a call to the notify method or the `notufyAll` method. The thread then waits until it can re-obtain ownership of the monitor and resumes execution.This method should only be called by a thread that is the owner of this object's monitor. See the notify method for a description of the ways in which a thred can become the owner of a monitor. **Throws :** `IllegalMonitorStateException`, if the current thread is not the owner of the object's monitor. Also **Throws :** `InterruptedException`, if another

	thread has interrupted this thread.
wait(long timeout)	Waits to be notified by another thread of a change in this object. The current thread must own this object's monitor. The thread releases ownership of this monitor and waits until either of the following two conditions has occurred : Another thread notifies threads waiting on this object's monitor to wake up either through a call to the notify method or the notifyAll method. The timeout period, specified by the timeout argument is milliseconds, has elapsed. The thread then waits until it can re-obtain ownership of the monitor and resumes execution. This method should only be called by a thread that is the owner of this object's monitor.
wait(long timeout, int nanos)	This method is similar to the wait method of one argument, but it allows finer control over the amount of time to wait for a notif ication before giving up.

Class Process

The execution methods returns an instance of a subclass of Process that can be used to control the process and obtain information about it.

```
          public‘‘abstract class’‘java.lang.Process
                                                              extends’‘java.lang.Object’‘
          {
                    //        Constructors
‘‘‘‘      public Process();
                    //        The default constructor.

‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract void destroy();
‘‘‘‘      public abstract int exitValue();
‘‘‘‘      public abstract InputStream getErrorStream();
‘‘‘‘      public abstract InputStream getInputStream();
‘‘‘‘      public abstract OutputStream getOutputStream();
‘‘‘‘      public abstract int waitFor();
          }
          Methods are as shown in Table 2.13.
```

Table 2.13”

Method	Description

destroy()	Kills the subprocess.
exitValue()	**Returns:** the exit value of the subprocess. **Throws :** IllegalThreadStateException, if the subprocess has not yet terminated.
getErrorStream()	Gets the error stream of the subprocess. **Returns:** the input stream connected to the error stream of the subprocess. This stream is usually unbuffered.
getInputStream()	Gets the input stream of the subprocess. **Returns:** the input stream connected to the normal output of the subprocess. This stream is usually buffered.
getOutputStream()	Gets the output stream of the subprocess. **Returns:** the output stream connected to the normal input of the subprocess. This stream is usually buffered.
waitFor()	Waits for the subprocess to complete. This method returns immediately if the subprocess has already terminated. **Returns:** the exit value of the process. **Throws** : InterruptedException., if the waitFor was interrupted.

Class Runtime

Every Java application has a single instance of class Runtime which allows the application to interface with the environment in which the application is running. The current runtime can be obtained from the getRuntime method. An application cannot create its own instance of this class.

```
          public‘‘class’‘java.lang.Runtime
‘‘‘‘                                    extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public Process exec(String’‘command);
‘‘‘‘      public Process exec(String’‘command, String’‘envp[]);
‘‘‘‘      public Process exec(String’‘cmdarray[]);
‘‘‘‘      public Process exec(String’‘cmdarray[], String’‘envp[]);
‘‘‘‘      public void exit(int’‘status);
‘‘‘‘      public long freeMemory();
‘‘‘‘      public void gc();
‘‘‘‘      public InputStream getLocalizedInputStream(InputStream in);’‘
‘‘‘‘      public OutputStream getLocalizedOutputStream(OutputStream out);’‘
‘‘‘‘      public static Runtime getRuntime();
‘‘‘‘      public void load(String’‘filename);
‘‘‘‘      public void loadLibrary(String’‘libname);
‘‘‘‘      public void runFinalization();
```

```
‘‘‘‘      public long totalMemory();
‘‘‘‘      public void traceInstructions(boolean’‘on);
‘‘‘‘      public void traceMethodCalls(boolean’‘on);
          }
          Methods are described as in Table 2.14.
```

Table 2.14”

Method	Description
exec(String command)	Executes the string command in a separate process.The command argument is parsed into tokens and then executed as a command in a separate process. This method has exactly the same effect as exec(command, null). **Returns:** a `Process` object for managing the subprocess. **Throws :** `SecurityException`, if the current thread cannot create a subprocess.
exec(String command, String envp[])	Executes the string command in a separate process with the specified environment..This method breaks the command string into tokens and creates a new array cmdarray containing the tokens; it then performs the call exec(cmdarray, envp). **Returns:** a Process object for managing the subprocess. **Throws** : `SecurityException`, if the current thread cannot create a subprocess.
exec(String cmdarray[])	Executes the string command in a separate process with the specified environment.. The command specified by the tokens in cmdarray is executed as a command in a separate process. This has exactly the same effect as exec(cmdarray, null). **Returns:** a Process object for managing the subprocess. **Throws** : `SecurityException`, if the current thread cannot create a subprocess.
exec(String cmdarray[], String envp[])	Executes the string command in a separate process with the specified arguments and the specified environment. If there is a security manager, its checkExec method is called with the first component of the array cmdarray as its argument. This may result in asecurity exeception. Given an array of strings cmdarray, representing the tokens of a command line, and an array of strings envp, representing an "environment" that defines system properties, this method creates a new process in which to execute the specified command. **Returns:** a `Process` object for managing the subprocess. **Throws** : `SecurityException`, if the current thread cannot create a subprocess

exit(int status)	Terminates the currently running Java Virtual Machine. This method never returns normally. If there is a security manager, its `checkExit` methodis called with the status as its argument. This may rseult in a security exception. The argument serves as a status code; by convention, a nonzero status code indicates abnormal termination **Throws** : `SecurityException`, if the current thread cannot exit with the specified status.
freeMemory()	Determines the amount of free memory in the system. The value returned by this method is always less than the value returned by the total-Memory method. Calling the gc method may result in increasing the value returned by `freeMemory`. **Returns:** an approximation to the total amount of memory currently available for future allocated objects, measured in bytes, is returned.
gc()	Calling this method suggests that the Java Virtual Machine expend effort toward recycling unused objects in order to make the memory they currently occupy available for quick reuse. When control returns from the method call, the Java Virtual Machine has made its best effort to recycle all unused objects. The name gc stands for "garbage collector." The Java Virtual Machine performs this recycling process automatically as needed even if the gc method is not invoked explicitly.
getLocalizedInputStream (InputStream in)	Creates a localized version of an input stream. This method takes an InputStream and returns an `InputStream` equivalent to the argument in all respects except that it is localized : as characters in the local character set are read from the stream, they are automatically converted from the local charascter set to Unicode. If the argument is already a localized stream, it may be returned as the result. **Returns:** a localized input stream.
getLocalizedOutputStream (OutputStream out)	Creates a localized version of an output stream. This method takes an OutputStream and returns an `OutputStream` equivalent to the argument in all respects except that it is localized : as Unicode character set are written to the stream, they are automatically converted to the local character set. If the argument is already a localized stream, it may be returned as the result. **Returns:** a localized output stream.
getRuntime()	**Returns:** the Runtime object associated with the current Java application.
load(String filename)	Loads the given filename as a dynamic library. The filename argument must be a complete path name. If there is a security manager, its checkLink method is called with tyhe filename as its argument. This may result in a security exception.

	Throws : `UnsatisfiedLinkError`, if the file does not exist. **Throws** : `SecurityException`, if the current thread cannot load the specified dynamic library.
loadLibrary(String libname)	Loads the dynamic library with the specified library name. The mapping from a library name to a specific filename is done is a system-specific manner. First, if there is a security manager, its `checkLink` method is called with the filename as its argument. this may result in security exception. If this method is called more than once with the same library name, the second and subsequent calls are ignored. **Throws** : `UnsatisfiedLinkError`**,** if the library does not exist **Throws** : `SecurityException`, if the current thread cannot load the specified dynamic library.
runFinalization()	Calling this method suggests that the Java Virtual Machine expend effort toward running the finalize methods of objects that have been foun to be discarded but whose finalized methods have not yet been run. when control returns from the method call, the Java Virtual Machine has made a best effort to complete all outstanding finalizations. The Java Virtual Machine performs the finalization process automatically as needed if the runFinalization method is not invoked explicitly.
totalMemory()	Determines the total amount of memory in the Java Virtual Machine. **Returns:** the total amount of memory currently available for allocating objects, measured in bytes.
traceInstructions(boolean on)	If the boolean argument is true, this method asks the Java Virtual Machine to print out a detailed trace of each instruction in the Java Virtual Machine as it is executed. The virtual machine may ignore this request if it does not support this feature. The destination of the trace output is system-dependent. If the boolean argument is false, this method causes the Java Virtual Machine to stop performing a detailed instruction trace it is performing.
traceMethodCalls(boolean on)	If the boolean argument is true, this method asks the Java Virtual Machine to print out a detailed trace of each method in the Java Virtual Machine as it is called. The virtual machine may ignore this request if it does not support this feature. The destination of the trace output is system-dependent. If the boolean argument is false, this method causes the Java Virtual Machine to stop performing a detailed method trace it is performing.

Class SecurityManager

The security manager is an abstract class that allows applications to implement a security policy. It allows an application to determine, before performing a possibly unsafe or sensitive operation, what the operation is and whether the operation is being performed by a class created via a class loader rather than installed locally. Classes loaded via a class loader (especially if they have been downloaded over a network) may be less trustworthy than classes from files installed locally. The application has the option of allowing or disallowing the operation

The SecurityManager class contains a large number of methods whose names begin with the word check. These methods are called by various methods in the Java libraries before those methods perform certain potentially sensitive operations.

The security manager is thereby given an opportunity to prevent completion of the operation by throwing an exception. A security manager routine simply returns if the operation is permitted, but throws a SecurityException if the operation is not permitted. The only exception to this convention is checkTopLevelWindow, which returns a boolean value.

The current security manager is set by the setSecurityManager method in class System. The current security manager is obtained by the getSecurityManager method :

```
          public abstract’‘class’‘java.lang.SecurityManager
‘‘‘‘                                                extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Member elements
‘‘‘‘      protected boolean inCheck;
                    //        This field is true if there is a security check in progress; false
otherwise.


‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      protected SecurityManager();
                    /*        Constructs a new SecurityManager. An application is not allowed to create a
new security manager if there is already a current security manager.Throws :’‘SecurityException, if a
security manager already exists.’‘
                    */
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public void’‘checkAccept(String’‘host, int’‘port);
‘‘‘‘      public void checkAccess(Thread’‘g);
‘‘‘‘      public void checkAccess(ThreadGroup’‘g);
‘‘‘‘      public void checkConnect(String’‘host, int’‘port);
‘‘‘‘      public void checkConnect(String’‘host, int’‘port, Object’‘context);
‘‘‘‘      public void checkCreateClassLoader();
‘‘‘‘      public void checkDelete(String’‘file);
‘‘‘‘      public void checkExec(String’‘cmd);
‘‘‘‘      public void checkExit(int’‘status);
‘‘‘‘      public void checkLink(String’‘lib);
‘‘‘‘      public void checkListen(int’‘port);
‘‘‘‘      public void checkPackageAccess(String’‘pkg);
‘‘‘‘      public void checkPackageDefinition(String’‘pkg);
‘‘‘‘      public void checkPropertiesAccess();
```

```
‘‘‘‘      public void checkPropertyAccess(String’‘key);
‘‘‘‘      public void checkRead(FileDescriptor’‘fd);
‘‘‘‘      public void checkRead(String’‘file);
‘‘‘‘      public void checkRead(String’‘file, Object’‘context);
‘‘‘‘      public void checkSetFactory();
‘‘‘‘      public boolean checkTopLevelWindow(Object’‘window);
‘‘‘‘      public void checkWrite(FileDescriptor’‘fd);
‘‘‘‘      public void checkWrite(String’‘file);
‘‘‘‘      protected int classDepth(String’‘name);
‘‘‘‘      protected int classLoaderDepth();
‘‘‘‘      protected ClassLoader currentClassLoader();
‘‘‘‘      protected Class[] getClassContext();
‘‘‘‘      public boolean getInCheck();
‘‘‘‘      public Object getSecurityContext();
‘‘‘‘      protected boolean inClass(String’‘name);
‘‘‘‘      protected boolean inClassLoader();
          }
Operation performed by the various methods are described in Table 2.15.
```

Table 2.15

Method	Description
checkAccept(String host, int port)	This method should throw a SecurityException if the calling thread is not permitted to accepting a socket connection from the specified host and port number. This method is invoked for the current security manager by the accept method of class `ServerSocket`. The `checkAccept` method for class `SecurityManager` always throws a `SecurityException`.
checkAccess(Thread g)	This method should throw a `SecurityException` if the calling thread is not allowed to modify the thread argument. This method is invoked for the current security manager by the stop, suspend, resume, `setPriority`, `setName`, and `setDaemon` methods of class `Thread`. The checkAccess method for class `SecurityManager` always throws a `SecurityException`.
checkAccess(ThreadGroup g)	This method should throw a `SecurityException` if the calling thread is not allowed to modify the thread argument. This method is invoked for the current security manager by the `stop`, `suspend`, `resume`, `setPriority`, `setName`, and `setDaemon` methods of class `Thread`. The `checkAccess` method for class `SecurityManager` always throws

	a `SecurityException`.
checkConnect(String host, int port)	This method should throw a `SecurityException` if the calling thread is not allowed to open a socket connection to the specified host and port number. A port number of of -1 indicates that the calling method is attempting to determine the IP address of the specified host name. The checkConnect method for class SecurityManager always throws a SecurityException.
checkConnect(String host, int port, Object context)	This method should throw a `SecurityException` if the specified security context is not allowed to open a socket connection to the specified host and port number. A port number of of -1 indicates that the calling method is attempting to determine the IP address of the specified host name. The `checkConnect` method for class `SecurityManager` always throws a `SecurityException`.
checkCreateClassLoader()	This method should throw a SecurityException if the calling thread is not allowed to create a new class loader216.The `checkCreateClassLoader` method for class `SecurityManager` always throws a `SecurityException`.
checkDelete(String file)	This method should throw a `SecurityException` if the calling thrad is not allowed to delete the specified file.This method is invoked for the current security manager by the `delete` method of class `File`. The `checkDelete` method for class `SecurityManager` always throws a `Security Exception`.
checkExec(String cmd)	This method should throw a `SecurityException` if the calling thread is not allowed to create a subprocss. This method is invoked for the current security manager by the `exec` methods of class `Runtime`. The `checkExec` method for class `SecurityManager` always throws a `SecurityException`.
checkExit(int status)	This method should throw a `SecurityException` if the calling thread is not allowed to cause the Java Virtual Machine to halt with the specified access code. This method is invoked for the current security manager by the `exit` method of class `Runtime`. A status of 0 indicates success; other values indicate various errors. The `checkExit` method for class `SecurityManager` always throws a `SecurityException`.
checkLink(String lib)	This method should throw a `SecurityException` if the calling thread is not allowed to dynamic link the library code specified by the string argument file. The argument is either a simple library name or a complete file name. This method is invoked for the current security manager by

	methodd `load`, and `loadLibrary` of class `Runtime`. The `checkLink` method for class `SecurityManager` always throws a `SecurityException`.
checkListen(int port)	This method should throw a `SecurityException` if the calling thread is not allowed to wait for a connection request on the specified local port number. The `checkListen` method for class `SecurityManager` always throws a `SecurityException`.
checkPackageAccess(String pkg)	This method should throw a `SecurityException` if the calling thread is allowed to access the package specified by the argument.This method is used by the `loadClass` method of class loaders. The `checkPackageAccess` method for class `SecurityManager` always throws a `SecurityException`.
checkPackageDefinition(String pkg)	This method should throw a `SecurityException` if the calling thread is not allowed to define classes in the package specified by the argument. This method is used by the `loadClass` method of some class loaders. The `checkPackageDefinition` method for class `SecurityManager` always throws a SecurityException.
checkPropertiesAccess()	This method should throw a `SecurityException` if the calling thread is not allowed to access or modify the system properties. This method is used by the `getProperties` and `setProperties` methods of class `System`. The `checkPropertiesAccess` method for class `SecurityManager` always throws a `SecurityException`.
checkPropertyAccess(String key)	This method should throw a `SecurityException` if the calling thread is not allowed to access the system property with the specified key name. This method is used by the getProperty method of class `System`. The checkPropertiesAccess method for class `SecurityManager` always throws a `SecurityException`.
checkRead(FileDescriptor fd)	This method should throw a `SecurityException` if the calling thread is not allowed to read from the specified file descriptor. The `checkRead` method for class `SecurityManager` always throws a `SecurityException`.
checkRead(String file)	This method should throw a `SecurityException` if the calling thread is not allowed to read the file specified by the string argument. The `checkRead` method for class `SecurityManager` always throws a `SecurityException`.

checkRead(String file, Object context)	This method should throw a `SecurityException` if the specified security context is not allowed to read the file specified by the string argument. The context must be a security context returned by a previous call to getSecurityContext. The `checkRead` method for class `SecurityManager` always throws a `SecurityException`.
checkSetFactory()	This method should throw a `SecurityException` if the calling thread is not allowed to set the socket factor used by ServerSocket or Socket, or the stream handler factory used by URL. The `checkSetFactory` method for class SecurityManager always throws a `SecurityException`.
checkTopLevelWindow(Object window)	This method should return false if the calling thread is not trusted to bring up the top-level window indicated by the window argument. In this case, the caller can still decide to show the window, but the window should include some sort of visual warning. If the method returns true, then the window can be shown without any special restrictions. The `checkSetFactory` method for class `SecurityManager` always return false. **Returns :** true if the caller is trusted to put up top-level windows; false otherwise.
checkWrite(FileDescriptor fd)	This method should throw a `SecurityException` if the calling thread is not allowed to write to the specified file descriptor218. The `checkWrite` method for class `SecurityManager` always throws a `SecurityException`. **Throws** : `SecurityException`, if the caller does not have permission to access the specified file descriptor.
checkWrite(String file)	This method should throw a SecurityException if the calling thread is not allowed to write to the file specified by the string argument. The `checkWrite` method for class SecurityManager always throws a SecurityException., **Throws** : `SecurityException`, if the caller does not have permission to access the specified file.
classDepth(String name)	Determines the stack depth of a given class. **Returns :** The depth on the stack frame of the first occurrence of a method from a class with the specified name; -1 if such a frame cannot be found.
classLoaderDepth()	Determines the stack stack of the most recently executing method from a class defined using a class loader. **Returns :** the depth on the stack frame of the most recent occurrence of a method from a class defined using a class loader; returns -1 if there is no occurrence of a method from a class defined using a class loader
currentClassLoader()	Determines the most recent class loader executing on the stack. **Returns :** The class loader of the most recent occurrence on the stack of a method from a class defined using a

	class loader; returns null if there is no occurrence on the stack of a method from a class defined using a class loader.
getClassContext()	Calculates the current execution stack, which is returned as an array of classes. The length of the array is the number of methods on the execution stack. The element at index 0 is the class of the currently executing method. The element at index 1 is the class of that method's caller. And so forth. **Returns** : the execution stack.
getInCheck()	**Returns :** the value of the inCheck field. This field should contain true if a security check is in progress; false otherwise.
getSecurityContext()	Creates an object when encapsulates the current execution environment. The result of this method is used by the three-argument `checkConnect` method and by the two-argument `checkRead` method. These methods are needed because a trusted method may be called upon to read a file or open a socket on behalf of another method. The trusted method needs to determine if the other (possibly untrusted) method would be allowed to perform the operation.on its own.. **Returns :** an implementation-dependent object which encapsulates sufficient information about the current execution environment to perform some security checks later.
inClass(String name)	**Returns :** true if a method from a class with the specified name is on the execution stack; false otherwise.
inClassLoader()	**Returns :** true if a method from a class defined using a class loader is on the execution stack.

Class String

The String class represents character strings. All string literals in Java programs, such as "abc" are implemented as instances of this class. Strings are constant, their values cannot be changed after they are created. String buffers support mutable strings.The class String includes methods for examining individual characters of the sequence, for comparing strings, for searching strings, for extracting substrings, and for creating a copy of a string with all characters translated to uppercase or to lowercase. Following is the structure of this class :

```
          public‘‘final’‘class’‘java.lang.String
‘‘‘‘                                              extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘//          Constructors
                    public String();
                    //        Allocates a new String containing no characters.
                    public String(byte’‘ascii[], int’‘hibyte);
                    /*        Allocates a new String containg characters constructed from an array of 8-
bit integer values. Each character c in the resultoing string is constructed from the corresponding
component b in the byte array such that :
```

```
                                                  c == (char)(((hibyte & 0xff) << 8 | (b & 0xff))

                              Parameters :

                                        ascii - the bytes to be converted to characters’‘

                                        hibyte - the top 8 bits of each 16-bit Unicode character

                    */

                    public String(byte’‘ascii[], int’‘hibyte, int’‘offset, int’‘count);

                    /*        Allocates a new String constructed from a subarray of an array of 8-bit
integer values. The offset argument is the index of the first byte of the subarray, and the count argument
specifies the length of the subarray. Each byte in the subarray is converted to a char as specified in the
method
above.’‘‘‘
```

```
                    public int indexOf(String‘‘str);
                    public int indexOf(String‘‘str, int‘‘fromIndex);
                    public String intern();
                    public int lastIndexOf(int‘‘ch);
                    public int lastIndexOf(int‘‘ch, int‘‘fromIndex);
                    public int lastIndexOf(String‘‘str);
                    public int lastIndexOf(String‘‘str, int‘‘fromIndex);
                    public int length();
                    public boolean regionMatches(boolean‘‘ignoreCase, int‘‘toffset, String other,
int‘‘ooffset, int‘‘len);
                    public boolean regionMatches(int‘‘toffset, String‘‘other, int‘‘ooffset,int‘‘len);
                    public String replace(char‘‘oldChar,char‘‘newChar);
                    public boolean startsWith(String‘‘prefix);
                    public boolean startsWith(String‘‘prefix, int toffset);
                    public String substring(int‘‘beginIndex);
                    public String substring(int‘‘beginIndex, int endIndex);
                    public char[] toCharArray();
                    public String toLowerCase();
                    public String toString();
                    public String toUpperCase();
                    public String trim();
                    public static String valueOf(boolean‘‘b);
                    public static String valueOf(char‘‘c);
                    public static String valueOf(char‘‘data[]);
                    public static String valueOf(char‘‘data[], int‘‘offset, int‘‘count);
                    public static String valueOf(double‘‘d);
                    public static String valueOf(float‘‘f);
                    public static String valueOf(int‘‘i);
                    public static String valueOf(long‘‘l);
                    public static String valueOf(Object‘‘obj);
          }
          Methods are stated in Table 2.16.
```

Table 2.16

Method	Description
charAt(int index)	**Returns :** The character at the specified index of this string. The first character is at index 0. **Throws :** `StringIndexOutOfBoundsException`, if the index is out of range.

compareTo(String anotherString)	Compares two strings lexicographically. **Returns :** The value 0 if the argument string is equal to this string; a value less than 0 if this string is lexicographically less than the string argument; and a value greater than 0 if this string is lexicographically greater than the string argument.
concat(String str)	Concatenates the string argument to the end of this string. If the length of the argument string is zero, then this object is returned. **Returns :** A string that represents the concatenation of this object's characters followed by the string argument's characters.
copyValueOf(char data[])	**Returns:** a String that contains the characters of the character array.
copyValueOf(char data[], int offset, int count)	**Returns :** a String that contains the characters of the specified subarray of the character array.
endsWith(String suffix)	**Returns :** true, if the character sequence represented by the argument is a suffix of the character sequence represented by this object; false otherwise.
equals(Object anObject)	The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as this object. **Returns :** true, if the String's are equal; false otherwise.
equalsIgnoreCase(String anotherString)	The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as this object, where case is ignored. Two characters are considered the same, ignoring case, if at least one of the following is true : The two characters are the same (as compared by the == operator). Applying the method Character.toUppercase to each character produces the same result. Applying the method Character.toLowercase to each character produces the same result. Two sequence of characters are the same, ignoring case, if the sequences have the same length and corresponding characters are the same, ignoring case. **Returns :** true, if the String's are equal, ignoring case; false otherwise.
getBytes(int srcBegin, int srcEnd, byte dst[], int dstBegin)	Copies characters from this string into the destination byte array. Each byte receives the 8 low-order bits of the corresponding character. The first character to be copied is at index srcBegin; the last character to be copied is at index srcEnd-1. The total number of characters to be copied is srcEnd-srcBegin. The characters, converted to bytes, are copied into the subarray of dst starting at index dstBegin and ending at index: `dstbegin+(srcEnd-srcBegin)-1`

getChars(int srcBegin, int srcEnd, char dst[], int dstBegin)	Copies characters from this string into the destination character array. The first character to be copied is at index srcBegin; the last character to be copied is at index srcEnd-1 (thus the total number of characters to be copied is srcEnd-srcBegin). The characters are copied into the subarray of dst starting at index dstBegin and ending at index : 223 `dstbegin+(srcEnd-srcBegin)-1`
hashCode()	**Returns :** a hash code value for this object.
indexOf(int ch)	**Returns :** the index of the first occurrence of the character in the character sequence represented by this object, or -1 if the character does not occur.
indexOf(int ch, int fromIndex)	**Returns :** the index of the first occurrence of the character in the character sequence represented by this object that is greater than or equal to fromIndex, or -1 if the character does not occur.
indexOf(String str)	**Returns :** if the string argument occurs as a substring within this object, then the index of the first character of the first such substring is returned; if it does not occur as a substring, -1 is returned.
indexOf(String str, int fromIndex)	**Returns :** if the string argument occurs as a substring within this object at a starting index no smaller than fromIndex then the index of the first character of the first such substring is returned. If it does not occur as a substring starting at fromIndex or beyond, -1 is returned.
intern()	Creates a canonical representation for the string object. If s and t are strings such that s.equals(t), it is guaranteed that s.intern() == t.intern(). **Returns :** a string that has the same contents as this string, but is guaranteed to be from a pool of unique strings.
lastIndexOf(int ch)	**Returns :** the index of the last occurrence of the character in the character sequence represented by this object, or -1 if the character does not occur.
lastIndexOf(int ch, int fromIndex)	**Returns :** the index of the last occurrence of the character in the character sequence represented by this object that is less than or equal to from-Index, or -1 if the character does not occur before that point.
lastIndexOf(String str)	**Returns :** if the string argument occurs as a substring within this object, then the index of the first character of the last such substring is returned. If it does not occur as a substring, -1 is returned.
lastIndexOf(String str, int fromIndex)	**Returns :** if the string argument occurs as a substring within this object at a starting index no greater than fromIndex then the

	index of the first character of the lastsuch substring is returned. If it does not occur as a substring starting at fromIndex or earlier, -1 is returned.
length()	**Returns :** the length of the sequence of characters represented by this object is returned.
regionMatches(boolean ignoreCase,int toffset, String other, int ooffset, int len)	Determines if two string regions are equal. If `toffset` or `ooffset` is negative, or if toffset+length is greater than the length of this string, or if ooffset+length is greater than the length of the string argument, then this method returns false.
regionMatches(int toffset, String other,int ooffset, int len)	Determines if two string regions are equal. If `toffset` or `ooffset` is negative, or if toffset+length is greater than the length of this string, or if ooffset+length is greater than the length of the string argument, then this method returns false224.
replace(char oldChar, char newChar)	Returns a new string resulting from replacing all occurrences of `oldChar` in this nstring with `newChar`. If the character `oldChar` does not occur in the character sequence represented by this object, then this string is returned.
startsWith(String prefix)	**Returns :** true if the character sequence represented by the argument is a prefix of the character sequence represented by this string; false otherwise.
startsWith(String prefix, int toffset)	**Returns :** true if the character sequence represented by the argument is a prefix of the substring of this object starting at index toffset; false otherwise.
substring(int beginIndex)	Creates a new string that is a substring of this string. The substring begins at the specified index and extends to the end of this string. **Returns :** The specified substring. **Throws :** `StringIndexOutOfBoundsException`, if the `beginIndex` is out of range.
substring(int beginIndex, int endIndex)	Creates a new string that is a substring of this string. The substring begins at the specified `beginIndex` and extends to the character at index `endIndex` - 1. **Throws :** `StringIndexOutOfBoundsException`, If the `beginIndex` or the `endIndex` is out of range.
toCharArray()	**Returns :** a newly allocated character array whose length is the length of this string and whose contents is initialized to contain the character sequence represented by this string.
toLowerCase()	Converts a string to lowercase. If no character in this string has a different lowercase version, then this string is returned. Otherwise, a new string is allocated, whose length is identical to this string, and such that each character which has a difference lowercase version is mapped to this lower case equivalent.

	Returns : the string, converted to lowercase.
toString()	This object (which is already a string!) is itself returned.
toUpperCase()	Converts a string to uppercase. If no character in this string has a different uppercase version, then this string is returned. Otherwise, a new string is allocated, whose length is identical to this string, and such that each character which has a difference uppercase version is mapped to this uppercase equivalent. **Returns** : the string, converted to uppercase.
trim()	Removes white space from both ends of a string. All characters that have codes less than or equal to '\u0020' (the space character) are considered to be white space. **Returns :** this string, with whitespace removed from the front and end.
valueOf(boolean b)	Creates the string representation of the boolean argument. **Returns :** if the argument is true, a string equal to "true" is returned; otherwise, a string equal to "false" is returned.
valueOf(char c)	Creates the string representation of the char argument. **Returns :** a newly allocated string of length one containing as its single characer the argument c.
valueOf(char data[])	Creates the string representation of the char array argument. **Returns :** a newly allocated string representing the same sequence of characters contained in the character array argument.
valueOf(char data[], int offset, int count)	Creates the string representation of a specific subarray of the char array argument. The offset argument is the index of the first character of the subarray. The count argument specifies the length of the subarray. **Returns :** a newly allocated string representing the sequence of characters contained in the subarray of the character array argument.
valueOf(double d)	Creates the string representation of the double argument. The representation is exactly the one returned by the Double-.toString method of one argument. **Returns :** a newly allocated string containing a string representation of the double argument.
valueOf(float f)	Creates the string representation of the float argument. The representation is exactly the one returned by the Float.toString method of one argument. **Returns :** a newly allocated string containing a string representation of the float argument.
valueOf(int i)	Creates the string representation of the int argument. The representation is exactly the one returned by the `Integer.toString` method of one argument. **Returns:** a newly allocated string containing a string representation of the int argument.

valueOf(long l)	Creates the string representation of a the long argument. The representation is exactly the one returned by the `Long.toString` method of one argument. **Returns :** a newly allocated string containing a string representation of the long argument.
valueOf(Object obj)	Creates the string representation of the `Object` argument. **Returns :** if the argument is null, then a string equal to "null"; otherwise the value of obj.toString() is returned.

Class StringBuffer

A string buffer implements a mutable sequence of characters. String buffers are safe for use by multiple threads. The methods are synchronized where necessary so that all the operations on any particular instance behave as if they occur in some serial order. String buffers are used by the compiler to implement the binary string concatentation operator +.

For example the code x = "a" + 4 + "c" is compiled to the equivalent of :

x=newStringBuffer().append("a").append(4).append("c").toString()

The principal operations on a StringBuffer are the append and insert methods, which are overloaded so as to accept data of any type. Each effectively converts a given datum to a string and then appends or inserts the characters of that string to the string buffer. The append method always adds these characters at the end of the buffer; the insert method adds the characters at a specified point. For example, if z refers to a string buffer object whose current contents are "start" then the method call z.append("le") would cause the string buffer to contain "startle" while z.insert(4, "le") would alter the string buffer to contain "starlet".

Every string buffer has a capacity. As long as the length of the character sequence contained in the string buffer does not exceed the capacity, it is not necessary to allocate a new internal buffer array. If the internal buffer overflows, it is automatically made larger.

```
          The structure of this class is shown as below :

                    public'‘class'‘java.lang.StringBuffer

                                                              extends'‘java.lang.Object'‘

                    {

                                        // Constructors

                              public StringBuffer();

                              //        Constructs a string buffer with no characters in it, and an
initial capacity of'‘16 characters.

                              public StringBuffer(int'‘length);

                              /*        Constructs a string buffer with no characters in it, and an
initial capacity specified by the length argument.

                                        Throws :'‘NegativeArraySizeException, if the length argument is
less than zero.

                              */

                              public StringBuffer(String'‘str);

                              /*        Constructs a string buffer so that it represents the same
sequence of characters as the string argument. The initial capacity of the string buffer is 16 plus the
length of the string argument.

                              */

                                        //        Methods
```

```
                                public StringBuffer append(boolean‘‘b);
                                public StringBuffer append(char‘‘c);
                                public StringBuffer append(char‘‘str[]);
                                public StringBuffer‘‘append(char‘‘str[], int‘‘offset, int‘‘len);
                                public StringBuffer append(double‘‘d);
                                public StringBuffer append(float‘‘f);
                                public StringBuffer append(int‘‘i);
                                public StringBuffer append(long‘‘l);
                                public StringBuffer append(Object‘‘obj);
                                public StringBuffer append(String‘‘str);
                                public int capacity();
                                public char charAt(int‘‘index);
                                public void ensureCapacity(int‘‘minimumCapacity);
                                public void getChars(int‘‘srcBegin, int‘‘srcEnd,    char‘‘dst[], int
dstBegin);
                                public StringBuffer insert(int‘‘offset, boolean‘‘b);
                                public StringBuffer insert(int‘‘offset, char‘‘c);
                                public StringBuffer insert(int‘‘offset, char‘‘str[]);
                                public StringBuffer insert(int‘‘offset, double‘‘d);
                                public StringBuffer insert(int‘‘offset, float‘‘f);
                                public StringBuffer insert(int‘‘offset, int‘‘i);
                                public StringBuffer insert(int‘‘offset, long‘‘l);
                                public StringBuffer insert(int‘‘offset, Object‘‘obj);
                                public StringBuffer insert(int‘‘offset, String‘‘str);
                                public int length();
                                public StringBuffer length();
                                public void setCharAt(int‘‘index, char‘‘ch);
                                public void setLength(int‘‘newLength);
                                public String toString();
                    }
```

Methods are described in Table 2.17.

Table 2.17

Method	Description
append(boolean b)	Appends the string representation of the boolean argument to the string buffer. The argument is converted to a string as if by the method String.valueOf, and the characters of that string are then appended to this string buffer. **Returns :** this string buffer.

append(char c)	Appends the string representation of the char argument to this string buffer. Argument is appended to the contents of this string buffer. The length of this string buffer increases by one. **Returns:** this string buffer
append(char str[])	Appends the string representation of the char array argument to this string buffer. The characters of the array argument are appended, in order, to the contents of this string buffer. The length of this string buffer increases by the length of the argument. **Returns:** this string buffer.
append(char str[], int offset, int len)	Appends the string representation of a subarray of the char array argument to this string buffer. Characters of the character array str, starting at index offset, are appended, in order, to the contents of this string buffer. The length of this string buffer increases by the value of len. **Returns :** this string buffer.
append(double d)	Appends the string representation of the double argument to this string buffer. The argument is converted to a string as if by the method String.valueOf , and the characters of that string are then appended to this string buffer.228 **Returns:** this string buffer.
append(float f)	Appends the string representation of the float argument to this string buffer. The argument is converted to a string as if by the method `String.valueOf`, and the characters of that string are then appended to the string buffer. **Returns :**this string buffer.
append(int i)	Appends the string representation of the int argument to this string buffer. The argument is converted to a string as if by the method `String.valueOf` , and the characters of that string are then appended to the string buffer. **Returns :**this string buffer.
append(long l)	Appends the string representation of the long argument to this string buffer. The argument is converted to a string as if by the method `String.valueOf` , and the characters of that string are then appended to the string buffer. **Returns :**this string buffer.
append(Object obj)	Appends the string representation of the Object argument to this string buffer. The argument is converted to a string as if by the method `String.valueOf` , and the characters of that string are then appended to the string buffer. **Returns :** this string buffer.
append(String str)	Appends the string to this string buffer. The characters of the String argument are appended, in order, to the contents of this string buffer, increasing the length of this string buffer by the length of the argument.
capacity()	**Returns :** the current capacity of this string buffer.

charAt(int index)	Determines the character at a specific index in this string buffer. The first character of a string buffer is at index 0, the next at index 1, and so on, for array indexing. The index argument must be greater than or equal to 0, and less than the length of this string buffer. **Returns:** the character at the specified index of this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the index is invalid.
ensureCapacity(int minimumCapacity)	If the current capacity of this string buffer is less than the argument, then a new internal buffer is allocated with greater capacity. The new capacity is the larger of: the `minimumCapacity` argument twice the old capacity, plus 2 If the `minimumCapacity` argument is nonpositive, this method takes no action and simply returns.
getChars(int srcBegin, int srcEnd char dst[], int dstBegin)	Characters are copied from this string buffer into the destination character array dst. The first character to be copied is at index `srcBegin`; the last character to be copied is at index `srcEnd-1`. The total number of characters to be copied is `srcEnd-srcBegin`. The characters are copied into the subarray of dst starting at index `dstBegin` and ending at index : `dstBegin+(srcEnd-srcBegin)-1`. **Throws:**`StringIndexOutOfBoundsException`, if there is an invalid index into the buffer.
insert(int offset, boolean b)	Inserts the string representation of the boolean argument into this string buffer. The second argument is converted to a string as if by the method String.valueOf , and the characters of that string are then appended to the string buffer at the indicated offset.The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the offset in invalid.
insert(int offset, char c)	Inserts the string representation of the char argument into this string buffer. The second argument is inserted into the contents of this string buffer at the position indicated by offset. The length of this string buffer increases by one. The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the offset is invalid.
insert(int offset, char str[])	Inserts the string representation of the char array argument into this string buffer. The characters of the array argument are inserted into the contents of this string buffer at the position indicated by offset. The length of this string buffer increases by the length of the argument.

	Throws:`StringIndexOutOfBoundsException`, if the offset is invalid.
insert(int offset, double d)	Inserts the string representation of the double argument into this string buffer. The second argument is converted to a string as if by the method String.valueOf , and the characters of that string are then inserted into this string buffer at the indicated offset. The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffer. **Returns:** this string buffer. **Throws :** StringIndexOutOfBoundsException, if the offset in invalid.
insert(int offset, float f)	Inserts the string representation of the float argument into this string buffer. The second argument is converted to a string as if by the method String.valueOf , and the characters of that string are then inserted into this string buffer at the indicated offset. The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffer. **Returns: :**this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the offset is invalid.
insert(int offset, int i)	Inserts the string representation of the second int argument into this string buffer. The second argument is converted to a string as if by the method String.valueOf , and the characters of that string are then indserted into this string buffer at the indicated offset. The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffer.230 **Returns :** this string buffer. **Throws**:`StringIndexOutOfBoundsException`, if the offset in invalid.
insert(int offset, long l)	Inserts the string representation of the long argument into this string buffer. The second argument is converted to a string as if by the method String.valueOf , and the characters of that string are then inserted into this string buffer at the indicated offset. The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffrer. **Returns :** this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the offset is invalid
insert(int offset, Object obj)	Inserts the string representation of the Object argument into this string buffer. The second argument is converted to a string as if by the method String.valueOf , and the characters of that string are then inserted into this string buffer at the indicated offset. The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffer. **Returns :** this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the offset is invalid.

insert(int offset, String str)	Inserts the string into this string buffer. The characters of the String argument are inserted, in order, into this string buffer at the indicated `offset`. The length of the of this string buffer is increased by the length of the argument. The offset argument must be greater than or equal to 0, and less than or equal to the length of this string buffer. **Returns:** this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the `offset` is out of range.
length()	**Returns :** the number of characters in this string buffer.
reverses()	The character sequence contained in this string buffer is replaced by the reverse of the sequence. **Returns :** this string buffer.
setCharAt(int index, char ch)	The character at the specified index of this string buffer is set to `ch`. The `offset` argument must be greater than or equal to 0, and less than the length of this string buffer. **Throws:**`StringIndexOutOfBoundsException`, if the index is invalid.
setLength(int newLength)	If the `newLength` argument is less than the current length of the string buffer, the string buffer is turncated to contain exactly the number of characters given by the `newLength` argument. If the `newLength` argument is greater than or equal to the current length, sufficient null characters ('\u0000') are appended to the string buffer so that length becomes the `newLength` argument. The `newLength` argument must be greater than or equal to zero. **Throws:**`StringIndexOutOfBoundsException`, if the newLength argument is invalid.
toString()	A new `String` object is allocated and initialized to contain the character sequence currently represented by this string buffer. This String is then returned. Subsequent changes to the string buffer do not affect the contents of the String.

Class System

The System class contains a number of useful member elements, and methods. It cannot be instantiated. Among the facilities provided by the System class are standard input, standard output, and error output streams, access to externally defined "properties", a means of loading files and libraries, and a utility method for quickly copying a portion of an array.

```
          public‘‘final’‘class’‘java.lang.System
                                                              extends’‘java.lang.Object’‘
          {
                                  //        Member elements
                     public static PrintStream err;
                     /*        The "standard" output stream. This stream is already open and ready to
accept output data. Typically this stream corresponds to display output or another output destination
specified by the host environment or user. By convention, this output stream is used to display error
```

```
messages or other information that should come to the immediate attention of a user even if the principal
output stream, the value of the variable out, has been redirected to a file or other destination that is
typically not continuously monitored.

                    */

                    public static InputStream in;

                    /*        The "standard" input stream. This stream is already open and ready to
supply input data. Typically this stream corresponds to keyboard input or another input source specified
by the host environment or user.

                    */

                    public static PrintStream out;

                    /*        The "standard" output stream. This stream is already open and ready to
accept output data. Typically this stream corresponds to display output or another output destination
specified by the host environment or user. For simple standalone Java applications, a typical way to write
a line of output data is System.out.println(data); See the println methods in class PrintStream.

                    */

                              //        Methods

                    public static void arraycopy(Object'‘src, int'‘src_position,Object'‘dst,
int'‘dst_position, int'‘length);

                    public static long currentTimeMillis();

                    public static void exit(int'‘status);

                    public static void gc();

                    public static Properties getProperties();

                    public static String getProperty(String'‘key);

                    public static String getProperty(String'‘key, String'‘def);

                    public static SecurityManager getSecurityManager();

                    public static void load(String'‘filename);

                    public static void loadLibrary(String'‘libname);

                    public static void runFinalization();

                    public static void setProperties(Properties'‘props);

                    public static void setSecurityManager(SecurityManager'‘s);

          }

          Methods are described as in Table 2.18.
```

Table 2.18

Method	Description
arraycopy(Object src, int srcOffset, Object dst, int dstOffset, int length)	A subsequence of array components are copied from the source array referenced by `src` to the destination array referenced by `dst`. The number of components copied is equal to the `length` argument. The components at positions `srcOffset` through `srcOffset+length-1` in the source array are copied into positions `dstOffset` through `dstOffset+length-1`, respectively, of the destination array. If the `src` and `dst` arguments refer to the same array

	object, then the copying is performed as if the components at positions `srcOffset` through `srcOffset+length-1` were first copied to a temporary array with length components and then the contents of the temporary array were copied into positions `dstOffset` through `dstOffset+length-1` of the argument array. If any of the following is true, an `ArrayStoreException` is thrown and the destination is not modified: □''The src argument refers to an object that is not an array. □''The dst argument refers to an object that is not an array. □''The src argument and dst argument refer to arrays whose component types are different primitive types. □''The src argument refers to an array with a primitive component type and the dst argument refers to an array with a reference component type. □''The src argument refers to an array with a reference component type and the dst argument refers to an array with a primitive component type.The srcOffset argument is negative. □''The dstOffset argument is negative. □''The length argument is negative. □''srcOffset+length is greater than src.length, the length of the source array. □''dstOffset+length is greater than dst.length, the length of the destination array. **Throws :** `ArrayIndexOutOfBoundsException`, if copy would cause access of data outside array bounds; `ArrayStoreException`, if an element in the src array could could not be stored into the dest array due to a type mismatch
currentTimeMillis()	Gets the current time. **Returns :** the difference, measured in milliseconds, between the current time and midnight, January 1, 1970 UTC.
exit(int status)	Terminates the currently running Java Virtual Machine. The argument serves as a status code; by convention, a nonzero status code indicates abnormal termination. This method calls the `exit` method in class `Runtime`. This method never returns normally. **Throws** : SecurityException, if the current thread cannot exit with the specified status.
gc()	This method call the `gc` method in class `Runtime`.Calling this method suggests that the Java Virtual Machine expend effort toward recycling unused objects in order to make the memory they currently occupy available for quick reuse. When

	control returns from the method call, the Java Virtual Machine has made a best effort to reclaim space from all unused objects.

getProperties()	Determines the current system properties. If there is a security manager, its `checkPropertiesAccess method` is called with no arguments. This may result in a security exception. The current set of system properties is returned as a `Properties` object. If there is no current set of system properties, a set of system properties is first created and initialized. This set of system properties always includes values for the following keys:
	`Key Description of associated value`
	`java.version Java version number`
	`java.vendor Java-vendor-specific string`
	`java.vendor.url Java vendor URL`
	`java.home Java installation directory`
	`java.class.version Java class format version number`
	`java.class.path Java classpath`
	`os.name Operating system name`
	`os.arch Operating system architecture`
	`os.version Operating system version`
	`file.separator File separator ("/" on Unix)`
	`path.separator Path separator (":" on Unix)`
	`line.separator Line separator ("\n" on Unix)`
	`user.name User account name`
	`user.home User home directory`
	`user.dir User's current working directory`
	Throws : SecurityException, if the current thread cannot access the system properties.

getProperty(String key)	Gets the system property indicated by the specified key. First, if there is a security manager, its checkPropertyAccess method is called with the key as its argument. This may result in a system exception. If there is no current set of system properties, a set of system properties is first created and initialized in the same manner as for the 234`getProperties` method. **Returns :** the string value of the system property, or null if there is no property with that key. **Throws** : `SecurityException`, if the current thread cannot access the system properties or the specified property.
getProperty(String key, String def)	Gets the system property indicated by the specified key. First, if there is a security manager, its checkPropertyAccess method is called with the key as its argument. If there is no current set of system properties, a set of system properties is first created and initialized in the same manner as for the getProperties method.

	Returns : the string value of the system property, or the default value if there is no property with that key. **Throws** : SecurityException, if the current thread cannot access the system properties or the specified property.
getSecurityManager()	**Returns :** if a security manager has already been established for the current application, then that security manager is returned; Otherwise, null is returned.
load(String filename)	Loads the given filename as a dynamic library. The filename argument must be a complete path name. This method calls the `load` method in class `Runtime`. **Throws** : `UnsatisfiedLinkError`, if the file does not exist; **Throws :** `SecurityException`, if the current thread cannot load the specified dynamic library.
loadLibrary(String libname)	Loads the system library specified by the libname argument. The manner in which a library name is mapped to the actual system library is system dependent. **Throws** : UnsatisfiedLinkError, if the library does not exist; **Throws** : SecurityException, if the current thread cannot load the specified dynamic library.
runFinalization()	This method calls the `runFinalization` method in class `Runtime`. Calling this method suggests that the Java Virtual Machine expend effort toward running the finalize methods of objects that have been found to be discarded but whose finalize methods have not yet been run. When control returns from the method call, the Java Virtual Machine has made a best effort to complete all outstanding finalizations.
setProperties(Properties props)	Sets the system properties to the `Properties` argument. First, if there is a security manager, its `checkPropertiesAccess` method is called with no arguments. This may result in a security exception. The argument becomes the current set of system properties for use by the `getProperty` method. If the argument is null, then the current set of system properties is forgotten. **Throws** : `SecurityException`, if the current thread cannot set the system properties.
setSecurityManager(SecurityManager s)	If a security manager has already been established for the currently running Java application, a `SecurityException` is thrown. Otherwise, the argument is established as the current security manager. If the argument is null and no security manager has been established, then no action is taken

	and the method simply returns. **Throws** : SecurityException, if the security manager has already been set.

Class Thread

Thread is an important class that allows to run multiple program at a time. A thread is corresponds to a control of a program. Every thread has a priority. Threads with higher priority are executed in preference to threads with lower priority. Each thread may or may not also be marked as a d&#65533;mon. When code running in some thread creates a new Thread object, the new thread has its priority initially set equal to the priority of the creating thread, and is a d&#65533;mon thread if and only if the creating thread is a d&#65533;mon. When a Java Virtual Machine starts up, there is usually a single non-d&#65533;mon thread (which typically calls the method named main of some designated class). The Java Virtual Machine continues to execute threads until either of the following occurs:

The exit method of class Runtime has been called and the security manager has permitted the exit operation to take place.

All threads that are not d&#65533;mon threads have died, either by returning from the call to the run method or by performing the stop method. Let us see the various components that are available in this class.

```
          public‘‘class’‘java.lang.Thread

                                                extends’‘java.lang.Object’‘

                    implements java.lang.Runnable’‘

                     {

                                        //        Member elements

                                public final static int MAX_PRIORITY;

                                //        MAX_PRIORITY = 10; The maximum priority that a thread can have.

                                public final static int MIN_PRIORITY;

                                //        MIN_PRIORITY = 1 ;’‘The minimum priority that a thread can have.

                                public final static int NORM_PRIORITY;

                                //        NORM_PRIORITY = 5 ; The default priority that is assigned to a
thread.


                                        //        Constructors

                                public Thread();

                                /*        Allocates a new Thread object. This constructor has the same
effect as Thread(null, null, gname) where gname is a newly generated name. Automatically generated names
are of the form "Thread-"+n where n is an integer.

                                */

                                public Thread(Runnable’‘target);

                                /*        Allocates a new Thread object. This constructor has the same
effect as Thread(null, target, gname), where gname is a newly generated name. Automatically generated
names are of the form "Thread-"+n where n is an integer. Here, target’‘is the object whose run method is
called.

                                */

                                public Thread(Runnable’‘target, String’‘name);

                                //        Allocates a new Thread object. This constructor has the same
effect as Thread(null, target, name).

                                public Thread(String’‘name);
```

```
                                //        Allocates a new Thread object. This constructor has the same
effect as Thread(null, null, name).

                                public Thread(ThreadGroup’‘group, Runnable’‘target);

                                /*        Allocates a new Thread object. This constructor has the same
effect as Thread(group, target, gname), where gname is a newly generated name. Automatically generated
names are of the form "Thread-"+n where n is an integer.

                                          Throws : SecurityException’‘if the current thread cannot create
a thread in the specified thread group.

                                */

                                public Thread(ThreadGroup’‘group, Runnable’‘target, String’‘name);

                                /*        Allocates a new Thread object so that it has target as its run
object, has the specified name as its name, and belongs to the thread group referred to by group.

                                          If group is not null, the checkAccess method of that thread
group is called with no arguments; this may result in throwing a SecurityException; if group is null, the
new process belongs to the same group as the thread this is created the new thread.

                                          If the target argument is not null, the run method of the target
is called when this thread is started. If the target argument is null, this thread's run method is called
when this thread is started.

                                          The priority of the newly created thread is set equal to the
priority of the thread creating it, that is, the currently running thread. The method setPriority may be
used to change the priority to a new value.

                                          The newly created thread is a initially marked as being a d□mon
thread if and only if the thread creating it is currently marked as a d□mon thread. The method setDaemon
may be used to change whether or not a thread is a daemon.

                                          Throws :’‘SecurityException,’‘if the current thread cannot
create a thread in the specified thread group.

                                */

                                public Thread(ThreadGroup’‘group, String’‘name);

                                /*        Allocates a new Thread object. This constructor has the same
effect as Thread(group, null, name).

                                          Throws :’‘SecurityException, if the current thread cannot create
a thread in the specified thread group.

                                */



                                //        Methods


                                public static int activeCount();

                                public void checkAccess();

                                public int countStackFrames();

                                public static Thread currentThread();

                                public void destroy();

                                public static void dumpStack();

                                public static int enumerate(Thread’‘tarray[]);

                                public final String getName();

                                public final int getPriority();

                                public final ThreadGroup getThreadGroup();

                                public void interrupt();
```

```
                        public static boolean interrupted();
                        public final boolean isAlive();
                        public final boolean isDaemon();
                        public boolean isInterrupted();
                        public final void join();
                        public final void'‘join(long'‘millis);
                        public final void join(long'‘millis, int'‘nanos)
                        public final void resume();
                        public void run();
                        public final void setDaemon(boolean'‘on);
                        public final void setName(String'‘name);
                        public final void setPriority(int'‘newPriority);
                        public static void sleep(long'‘millis);
                        public static void sleep(long'‘millis, int'‘nanos)
                        public void start();
                        public final void stop();
                        public final void'‘stop(Throwable'‘obj);
                        public final void suspend();
                        public String toString();
                        public static void yield();
}

Methods are described as in Table 2.19.
```

Table 2.19

Method	Description
activeCount()	**Returns :** the current number of threads in this thread's thread group.
checkAccess()	Determines if the currently running thread has permission to modify this thread. If there is a security manager, its checkAccess method is called with this method as its argument. This may result in throwing a security exception. 238 **Throws :** SecurityException, if the current thread is not allowed to access the thread.
countStackFrames()	Counts the number of stack frames in this thread. The thread must be suspended. **Returns :** the number of stack frames in this thread. **Throws :** IllegalThreadStateException, if this thread is not suspended.
currentThread()	Find the currently executing thread.

	Returns : the currently executing thread.
destroy()	Destroys this thread, without any cleanup. Any monitors it has locked remain locked.
dumpStack()	Prints a stack trace of the current thread. Used only for debugging.
enumerate(Thread tarray[])	Copies into the array argument every thread in this thread's thread group. This method simple calls the enumerate method of this thread's thread group withy the array argument. **Returns :** the number of threads put into the array.
getName()	**Returns :** this thread's name.
getPriority()	**Returns :** this thread's current priority.
getThreadGroup()	**Returns :** this thread's thread group.
interrupt()	Interrupts this thread.
interrupted()	**Return** : true if the current thread has been interrupted; false otherwise.
isAlive()	Determines if this thread is alive A thread is alive if it has been started and has not yet died. **Returns :** true if this thread is alive; false otherwise.
isDaemon()	**Returns :** true if this thread is a d mon thread; false otherwise.
isInterrupted()	**Returns :**true if this thread has been interrupted; false otherwise.
join()	Waits for this thread to die. **Throw :** `InterruptedException`, another thread has interrupted the current thread.
join(long millis)	Waits at most millis milliseconds for for this thread to die. A timeout of 0 means to wait forever. **Throws** : `InterruptedException`, another thread has interrupted the current thread.
join(long millis, int nanos)	Waits at most millis milliseconds plus nanos nanoseconds for this thread to die. **Throws :** `InterruptedException`, another thread has interrupted the current thread.
resume()	Resumes a suspended thread. First, the checkAccess method of this thread is called with no arguments. this may result in throwing a security exception (in the current thread). If the thread is alive but suspended. it is resumed and is permitted to make progress in its execution. **Throws** : `SecurityException`, if the current thread cannot modify this thread.

run()	If this thread was constructed using a separate Runnable run object , then that Runnable object's `run` method is called. Otherwise, this method does nothing and rteturns. Subclasses of Thread should override this method.
setDaemon(boolean on)	Marks this thread as either a d&#65533;mon thread or a user thread. The Java Virtual Machine exits when the only threads running are all d&#65533;mon threads. This method must be called before the thread is started.239239 **Throws** : `IllegalThreadStateException`, if this thread is active.
setName(String name)	Changes the name of this thread to be equal to the argument name. First the checkAccess method of this thread is called with no arguments. This may result in throwing a security exception. **Throws** : `SecurityException`, if the current thread cannot modify this thread.
setPriority(int newPriority)	Changes the priority of this thread. First the checkAccess method of this thread is called with no arguments. This may result in throwing a security exception. Otherwise, the priority of this thread is set to the smaller of the specified newPriority and the maximum permitted priority of the thread&#65533;s thread group. **Throws** : `IllegalArgumentException`, if the priority is not in the range MIN_PRIORITY to MAX_PRIORITY. Also can **Throws** : `SecurityException`, if the current thread cannot modify this thread.
sleep(long millis)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds. The thread does not lose ownership of any monitors. **Throws** : `InterruptedException`, another thread has interrupted this thread.
sleep(long millis, int nanos)	Causes the currently executing thread to sleep (cease execution) for the specified number of milliseconds plus the specified number of nanoseconds. The thread does not lose ownership of any monitors. **Throws** : `InterruptedException`, another thread has interrupted this thread.
start()	Causes this thread to begin execution; the Java Virtual Machine calls the run method of this thread. The result is that two threads are running concurrently: the current thread (which returns from the call to the start method) and the other thread (which executes its run method). **Throws** `IllegalThreadStateException` , if the thread was already started.
stop()	Forces this thread to stop executing. First, the `checkAccess` method of this thread is called with no arguments. This may result in throwing a security exception (in the current thread). The thread represented

	by the this thread is forced to stop whatever it is doing abnormally and to throw a newly created `ThreadDeath` object as an exception. It is permitted to stop a thread that has not yet been started. If the thread is eventually started, it immediately terminates. An application should not normally try to catch `ThreadDeath` unless it must do some extraordinary cleanup operation (note that the throwing of ThreadDeath will cause finally clauses of try statements to be executed before the thread officially dies). If a catch clause does catch a ThreadDeath object, it is important to rethrow the object so that the thread actually dies. The top-level error handler that reacts to otherwise uncaught exceptions does not print out a message or otherwise notify the application if the current uncaught exception is an instance of `ThreadDepth`. **Throws** : `SecurityException`, if the current thread cannot modify this thread.
suspend()	Suspends this thread. First, the checkAccess method of this thread is called with no arguments. This may result in throwing a security exception (in the current thread). If the thread is alive, it is suspended and makes no further progress unless and until it is resumed. **Throws** : `SecurityException` , if the current thread cannot modify this thread.
toString()	**Returns :** a string representation of this thread.
yield()	Causes the currently executing thread object to temporarily pause and allow other threads to execute.

Class ThreadGroup

A thread group represents a set of threads. In addition, a thread group can also include other thread groups. The thread groups form a tree in which every thread group except the initial thread group has a parent. A thread is allowed to access information about its own thread group, but not to access information about its thread group's parent thread group.

```
          public‘‘class’‘java.lang.ThreadGroup
                                                              extends’‘java.lang.Object’‘
          {
                                //        Constructors
                     public ThreadGroup(String’‘name);
                     //        Constructs a new thread group, whose parent is the thread group of the
currently running thread.
                     public ThreadGroup(ThreadGroup’‘parent, String’‘name);’‘
                     /*        Creates a new thread group whose parent is the specified thread group.
Throws :’‘NullPointerException,’‘if the thread group argument is null. and also SecurityException, if the
current thread cannot create a thread in the specified thread group.
                     */
```

```
                    //        Methods
                public int activeCount();
                public int activeGroupCount();
                public final void checkAccess();
                public final void destroy();
                public int enumerate(Thread’‘list[]);
                public int enumerate(Thread’‘list[], boolean’‘recurse);
                public int enumerate(ThreadGroup’‘list[]);
                public int enumerate(ThreadGroup’‘list[], boolean’‘recurse);’‘
                public final int getMaxPriority();
                public final String getName();
                public final ThreadGroup getParent();
                public final boolean isDaemon();
                public void list();
                public final boolean parentOf(ThreadGroup’‘g);
                public final void resume();
                public final void setDaemon(boolean’‘daemon);
                public final void’‘setMaxPriority(int’‘pri);
                public final void stop();
                public final void suspend();
                public String toString();
                public void uncaughtException(Thread’‘t, Throwable’‘e);’‘
        }

        Methods are stated in Table 2.20.
```

Table 2.20

Method	Description
activeCount()	**Returns :** the number of active threads in this thread group and in any other thread group that has this thread group as an ancestor.
activeGroupCount()	**Returns :** the number of active thread groups with this thread group as an ancestor.
checkAccess()	Determines if the currently running thread has permission to modify this thread group. If there is a security manager, its checkAccess method is called withn this thread group as its argument. This may result in throwing a security exception. **Throws :** SecurityException, if the current thread is not allowed to access this thread group.

destroy()	Destroys this thread group and all of its sub-groups. This thread group must be empty, indicating that all threads that had been in this thread group have since stopped. **Throws** : IllegalThreadStateException, if the thread group is not empty or if the thread group has already destroyed. Also **Throws** : SecurityException, if the current thread cannot modify this thread group.
enumerate(Thread list[])	Copies into the specified array every active thread in this thread group and its sub-groups. An application should use the activeCount method to get an estimate of how big the array should be. If the array is too short to hold all the threads, the extra threads are silently ignored.243 **Returns :** the number of threads put into the array.
enumerate(Thread list[], boolean recurse)	Copies into the specified array every active thread in this thread group. If the recurse flag is true references to every active thread in this thread's sub- groups are also included. If the array is too short to hold all the threads, the extra threads are silently ignored. An application should use the activeCount method to get an estimate of how big the array should be. **Returns :** the number of threads placed into the array.
enumerate(ThreadGroup list[])	Copies into the specified array references to every active sub-group in this thread group. An application should use the activeGroupCount method to get an estimate of how big the array should be. If the array is too short to hold all the threads, the extra threads are silently ignored. **Returns :** the number of thread groups put into the array.
enumerate(ThreadGroup list[], boolean recurse)	Copies into the specified array references to every active sub-group in this thread group. If the recurse flag is true references to every active, all sub- thread groups of the sub-thread groups and so forth are also included. An application should use the activeGroupCount method to get an estimate of how big the array should be. **Returns :** the number of thread groups put into the array.
getMaxPriority()	**Returns :**the maximum priority that a thread in this thread group can have.
getName()	**Returns :** the name of this thread group.
getParent()	**Returns :** the parent of this thread group. The top-level thread group is the only thread group whose parent is null.
isDaemon()	Determines if this thread group is a d&#65533;mon thread group. A d&#65533;mon thread group is automatically destroyed when its last thread is stopped or its last thread group is destroyed. **Returns:** true if this thread group is a d&#65533;mon thread group; false otherwise.
list()	Prints information about this thread group to the standard output. This method is useful only for debugging.
parentOf(ThreadGroup g)	Determines if this thread group is either the thread group argument or one of its ancestor thread

	groups. **Returns :** true if this thread group is the thread group argument or one of its ancestor thread groups; false otherwise.
resume()	Resumes all processes in this thread group. First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exception. This method then calls the resume method on all the threads in this thread group and in all of its sub-thread groups. **Throws** : SecurityException, if the current thread is not allowed to access this thread group or any of the threads in the thread group.
setDaemon(boolean daemon)	Sets whether this thread group is a d&#65533;mon thread group. First, the checkAccess method of this thread group is called with no arguments; this may result in a security exceptiion. Marks whether this thread is a d&#65533;mon thread group. A d&#65533;mon thread group is automatically destroyed when its last thread is stopped or its last thread group is destroyed. **Throws** : SecurityException, if the current thread cannot modify this thread.
setMaxPriority(int pri)	Sets the maximum priority of the group. First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exceptiion. Threads in the thread group that already have a higher priority are not affected. **Throws** : SecurityException, if the current thread cannot modify this thread group.
stop()	Stops all processes in this thread group. First, the checkAccess method of this thread group is called with no arguments; this may result in a security exceptiion. This method then calls the stop method on all the threads in this thread group and in all of its sub-thread groups. **Throws** : SecurityException, if the current thread is not allowed to access this thread group or any of the threads in the thread group.
suspend()	Suspends all processes in this thread group. First, the `checkAccess` method of this thread group is called with no arguments; this may result in a security exception. This method then calls the suspend method on all the threads in this thread group and in all of its sub-thread groups. **Throws** : SecurityException, if the current thread is not allowed to access this thread group or any of the threads in the thread group.
toString()	**Returns :** a string representation of this thread group.
uncaughtException(Thread t, Throwable e)	The Java Virtual Machine calls this method when a thread in this thread group stops because of an uncaught exception. The `uncaughtException` method of `ThreadGroup` does the following :

	□''If this thread group has a parent thread group, the `uncaughtException` method of that parent is called with the same two arguments. □''Otherwise, this method determines if the `Throwable` argument is an instance of `ThreadDeath`. if so, nothing dspecial is done. Otherwise, the Throwable's `printStackTrack` method is called to print a stack back trace to the standard error stream. Applications can override this method in subclasses of `ThreadGroup` to provide alternative handling of uncaught exceptions.

Class Throwable

The Throwable class is the superclass of all errors and exceptions in the Java language. Only objects that are instances of this class (or of one of its subclasses) are thrown by the Java Virtual Machine or can be thrown by the Java throw statement. Similarly, only this class or one of its subclasses can be the argument type in a catch clause. A Throwable contains a snapshot of the execution stack of its thread at the time it was created. It can also contain a message string that gives more information about the error.

```
          public''class''java.lang.Throwable
                                                extends''java.lang.Object''
          {
                                //        Constructors
                     public Throwable();
                     //        Constructs a new Throwable with no detail message. The stack trace is
automatically filled in.
                     public Throwable(String''message);
                     //        Constructs a new Throwable with the specified detail message. The stack
trace is automatically filled in.
                                //        Methods
                     public Throwable fillInStackTrace();
                     public String getMessage();
                     public void printStackTrace();
                     public void printStackTrace(PrintStream''s);
                     public String toString();
          }
          Methods are described in Table 2.21.
```

Table 2.21

Method	Description
fillInStackTrace()	Fills in the execution stack trace. This method is useful when an application is re-throwing an error or exception. For example:

	try {
	a = b / c;
	} catch(ArithmeticThrowable e) {
	a = Number.MAX_VALUE;
	throw e.fillInStackTrace();
	}

getMessage()	**Returns :** the detail message of this Throwable, or null if this Throwable does not have a detailed message.
printStackTrace()	Prints this Throwable and its backtrace to the standard error output stream.
printStackTrace(PrintStream s)	Prints this Throwable and its backtrace to the specified print stream.
toString()	**Returns :** a string representation of this Throwable.

Interface Cloneable

A class implements the Cloneable interface to indicate to the clone method in class Object that it is legal for that method to make a field-for-field copy of instances of that class. Attempts to clone instances that do not implement the Cloneable interface result in the exception CloneNotSupportedException being thrown.

public interface java.lang.Cloneable { }

Interface Runnable

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. The class must define a method of no arguments called run.

```
          public‘‘interface’‘java.lang.Runnable
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract void run();
                    /*        When an object implementing interface Runnable is used to create a thread,
starting the thread causes the object's run method to be called in that separately executing thread.
                    */
          }
```

Class ArithmeticException

Thrown when an exceptional arithmetic condition has occurred. For example, an integer "divide by zero" throws an instance of this class.

```
          public‘‘class’‘java.lang.ArithmeticException
                                                extends’‘java.lang.RuntimeException’‘
          {
                              //        Constructors
                    public ArithmeticException();
                    //        Constructs an ArithmeticException with no detail message.
```

```
                    public ArithmeticException(String‘‘s);
                    //        Constructs an ArithmeticException with the specified detail message.
          }
```

Class ArrayIndexOutOfBoundsException

Thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.

```
          public‘‘class‘‘java.lang.ArrayIndexOutOfBoundsException
                                                extends‘‘java.lang.IndexOutOfBoundsException‘‘
          {
                              //        Constructors
                    public ArrayIndexOutOfBoundsException();
                    //        Constructs an ArrayIndexOutOfBoundsException with no detail message.
                    public ArrayIndexOutOfBoundsException(int‘‘index);
          ‘‘ /* Constructs a new ArrayIndexOutOfBoundsException class with an argument indicating the
illegal index.*/
                    public ArrayIndexOutOfBoundsException(String‘‘s);
                    /*        Constructs a ArrayIndexOutOfBoundsException with the specified detail
          message.*/
          }
```

Class ArrayStoreException

Thrown to indicate that an attempt has been made to store the wrong type of object into an array of objects.

```
          public‘‘class‘‘java.lang.ArrayStoreException
                                                extends‘‘java.lang.RuntimeException‘‘
          {
                              //        Constructors
                    public ArrayStoreException();
                    //        Constructs a ArrayStoreException with no detail message.


                    public ArrayStoreException(String‘‘s);
                    //        Constructs a ArrayStoreException with the specified detail message.
          }
```

Class ClassCastException

Thrown to indicate that the code has attempted to cast an object to a subclass of which it is not an instance.

```
          public‘‘class‘‘java.lang.ClassCastException
```

```
‘‘‘‘                                    extends’‘java.lang.RuntimeException’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public ClassCastException();
                    //        Constructs a ClassCastException with no detail message.
          public ClassCastException(String’‘s);
                    //        Constructs a ClassCastException with the specified detail message.
          }
```

Class ClassNotFoundException

Thrown when an application tries to load in a class through its string name using any one of the following, but no definition for the class with the specifed name could be found.

- ’‘the forName method in class Class
- ’‘the findSystemClass method in class ClassLoader
- ’‘the loadClass method in class ClassLoader

```
          public’‘class’‘java.lang.ClassNotFoundException
                                                        extends’‘java.lang.Exception’‘
                    {
                                        //        Constructors
                    public ClassNotFoundException();
                              //        Constructs a ClassNotFoundException with no detail message.
                    public ClassNotFoundException(String’‘s);
                              //        Constructs a ClassNotFoundException with the specified detail
message.            }
```

Class CloneNotSupportedException

Thrown to indicate that the clone method in class Object has been called to clone an object, but that the object's class does not implement the clonable interface. Applications that override the clone method can also throw this exception to indicate that an object could not or should not be cloned.

```
          public’‘class’‘java.lang.CloneNotSupportedException
                                                        extends’‘java.lang.Exception’‘
                    {
                                        //        Constructors
                    public CloneNotSupportedException();
                              //        Constructs a CloneNotSupportedException with no detail message.


                    public CloneNotSupportedException(String’‘s);
                              //        Constructs a CloneNotSupportedException with the specified
detail message.
```

```
}
```

Class Exception

The class Exception and its subclasses are a form of Throwable that indicates conditions that a reasonable application might want to catch.

```
public‘‘class’‘java.lang.Exception
                                        extends’‘java.lang.Throwable’‘
{
                        //      Constructors
public Exception();
                //      Constructs an Exception with no specified detail message.
public Exception(String’‘s);
                //      Constructs a Exception with the specified detail message.
}
```

Class IllegalAccessException

Thrown when an application tries to load in a class through its string name using any one of the following, but the currently executing method does not have access to the definition of the specified class, because the class is not public and in another package :

- ’‘the forName method
- ’‘the findSystemClass method in class ClassLoader
- ’‘the loadClass method in class ClassLoader An instance of this class can also be thrown when an application tries to create an instance of a class using the newInstance method in class Class, but the current method does not have access to the appropriate zero-argument constructor.

```
public‘‘class’‘java.lang.IllegalAccessException
                                        extends’‘java.lang.Exception’‘
{
                //      Constructors
public IllegalAccessException();
//      Constructs a IllegalAccessException without’‘any’‘message.

public IllegalAccessException(String’‘s);
//      Constructs a IllegalAccessException with a detail message.
}
```

Class IllegalArgumentException

Thrown to indicate that a method has been passed an illegal or inappropriate argument.

```
public‘‘class’‘java.lang.IllegalArgumentException
```

```
                                                    extends‘‘java.lang.RuntimeException’‘
                    {
                                        //        Constructors
                    public IllegalArgumentException();
                                //        Constructs an IllegalArgumentException with no detail message.


                    public IllegalArgumentException(String‘‘s);
                                //        Constructs an IllegalArgumentException with the specified detail
message.


                    }
```

Class IllegalMonitorStateException

Thrown to indicate that a thread has attempted to wait on an object's monitor or to notify other threads waiting on an object's monitor, without owning the specified monitor.

```
          public‘‘class’‘java.lang.IllegalMonitorStateException
                                                extends‘‘java.lang.RuntimeException’‘
          {
                                //        Constructors
                    public IllegalMonitorStateException();
                    //        Constructs an IllegalMonitorStateException with no detail message.
                    public IllegalMonitorStateException(String‘‘s);
                    /*        Constructs an IllegalMonitorStateException with the specified detail
message.
                    */
          }
```

Class IllegalThreadStateException

Thrown to indicate that a thread is not in an appropriate state for the requested operation.

```
          public‘‘class’‘java.lang.IllegalThreadStateException
‘‘‘‘                                    extends‘‘java.lang.IllegalArgumentException’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public IllegalThreadStateException();
                    //        Constructs an IllegalThreadStateException with no detail message.


‘‘‘‘      public IllegalThreadStateException(String‘‘s);
                    //        Constructs an IllegalThreadStateException with the specified detail
message.
```

```
        }
```

Class IndexOutOfBoundsException

Instances of this class are thrown indicate that an index of some sort (such as to an array, to a string, or to a vector) is out of range. Applications can subclass this class to indicate similar exceptions.

```
        public‘‘class’‘java.lang.IndexOutOfBoundsException
‘‘‘‘                                            extends’‘java.lang.RuntimeException’‘
        {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘    public IndexOutOfBoundsException();
                //        Constructs an IndexOutOfBoundsException with no detail message.

‘‘‘‘    public IndexOutOfBoundsException(String’‘s);
                //        Constructs an IndexOutOfBoundsException with the specified detail message.
        }
```

Class InstantiationException

Thrown when an application tries to create an instance of a class using the newInstance method in class Class, but the the specified class object cannot be instantiated because it is an interface or is an abstract class.

```
        public‘‘class’‘java.lang.InstantiationException
                                            extends’‘java.lang.Exception’‘
        {
                        //        Constructors
                public InstantiationException();
                //        Constructs an InstantiationException with no detail message.
                public InstantiationException(String’‘s);
                //        Constructs an InstantiationException with the specified detail message.
        }
```

Class InterruptedException

Thrown when a thread is waiting, sleeping. or otherwise paused for a long time and another thread interrupts it using the the interrupt method in class Thread.

```
        public‘‘class’‘java.lang.InterruptedException
```

```
                                                extends’‘java.lang.Exception’‘
        {
                                //        Constructors
                        public InterruptedException();
                        //        Constructs an InterruptedException with no detail message.
                        public InterruptedException(String’‘s);
                        //        Constructs an InterruptedException with the specified detail message.
        }
```

Class NegativeArraySizeException

Thrown if an application tries to create an array with negative size.

```
        public’‘class’‘java.lang.NegativeArraySizeException
‘‘‘‘                                    extends’‘java.lang.RuntimeException’‘
        {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘    public NegativeArraySizeException();
                        //        Constructs a NegativeArraySizeException with no detail message.


‘‘‘‘    public NegativeArraySizeException(String’‘s);
                        //        Constructs a NegativeArraySizeException with the specified detail message.
        }
```

Class NoSuchMethodException

Thrown hen there is no succh method exist.

```
        public’‘class’‘java.lang.NoSuchMethodException
                                                extends’‘java.lang.Exception’‘
        {
                                //        Constructors
                        public NoSuchMethodException();
                        //        Constructs a NoSuchMethodException without a detail message.


                        public NoSuchMethodException(String’‘s);
                        //        Constructs a NoSuchMethodException with a detail message.
        }
```

Class NullPointerException

Thrown when an application attempts to use null in a case where an object is required. These include:

- ’‘Call the instance method of a null object.
- ’‘Accessing or modifying the field of the null object.
- ’‘Taking the length of null as if it were an array.
- ’‘Accessing or modifying the slots of null as if it were an array.
- ’‘Throwing null as if it were a Throwable value. Applications should throw instances of this class to indicate other illegal uses of the null object.

```
public’‘class’‘java.lang.NullPointerException
                                        extends’‘java.lang.RuntimeExceptio’‘
{
                    //      Constructors
          public NullPointerException();
          //      Constructs a NullPointerException with no detail message.
          public NullPointerException(String’‘s);
          //      Constructs a NullPointerException with the specified detail message.
}
```

Class NumberFormatException

Thrown to indicate that the application has attempted to convert a string to one of the numeric types, but that the string does not have the appropriate format.

```
public’‘class’‘java.lang.NumberFormatException
‘‘‘‘                                    extends’‘java.lang.IllegalArgumentException’‘
{
‘‘‘‘‘‘‘‘ //      Constructors
          public NumberFormatException();
          //      Constructs a NumberFormatException with no detail message.

          public NumberFormatException(String’‘s);
          //      Constructs a NumberFormatException with the specified detail message.
}
```

Class RuntimeException

RuntimeException is the superclass of those exceptions which can be thrown during the normal operation of the Java Virtual Machine. A method is not required to declare in its throws clause any subclasses of RuntimeException that might be thrown during the execution of the method but not caught.

```
```

```
public''class''java.lang.RuntimeException
                                        extends''java.lang.Exception''
{
                    //      Constructors
          public RuntimeException();
          //      Constructs a RuntimeException with no detail message.
          public RuntimeException(String''s);
          //      Constructs a RuntimeException with the specified detail message.
}
```

Class SecurityException

Thrown by the security manager to indicate a security violation.

```
public''class''java.lang.SecurityException
                                        extends''java.lang.RuntimeException''
{
          //      Constructors
          public SecurityException();
          //      Constructs a SecurityException with no detail message.
          public SecurityException(String''s);
          //      Constructs a SecurityException with the specified detail message.
}
```

Class StringIndexOutOfBoundsException

Thrown by the charAt method in class String and by other String methods to indicate that an index is either negative or greater than or equal to the size of the string.

```
public''class''java.lang.StringIndexOutOfBoundsException
                                extends''java.lang.IndexOutOfBoundsException''
{
                    //      Constructors
          public StringIndexOutOfBoundsException();
          //      Constructs a StringIndexOutOfBoundsException with no detail message.
          public StringIndexOutOfBoundsException(int''index);
          /*      Constructs a new StringIndexOutOfBoundsException class with an argument
indicating''the illegal index.
          */
          public StringIndexOutOfBoundsException(String''s)
          /*      Constructs a StringIndexOutOfBoundsException with the specified detail
     message.
          */
```

```
        }
```

Class AbstractMethodError

Thrown when an application tries to call an abstract method. Normally, this error is caught by the compiler; this error can only occur at runtime if the definition of some class has incompatibably changed since the currently executing method was last compiled.

```
        public‘‘class’‘java.lang.AbstractMethodError
‘‘‘‘                                    extends’‘java.lang.IncompatibleClassChangeError
        {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘    public AbstractMethodError();
                  //        Constructs an AbstractMethodError with no detail message.


‘‘‘‘    public AbstractMethodError(String’‘s);
                  //        Constructs an AbstractMethodError with the specified detail message.
        }
```

Class ClassCircularityError

```
‘‘‘‘public’‘class’‘java.lang.ClassCircularityError
‘‘‘‘                                    extends’‘java.lang.LinkageError
        {
‘‘‘‘‘‘  //        Constructors
‘‘‘‘    public ClassCircularityError();
                  //        Constructs a ClassCircularityError with no detail message.


‘‘‘‘    public ClassCircularityError(String’‘s);
                  //        Constructs a ClassCircularityError with the specified detail message.
        }
```

Class ClassFormatError

Thrown when the Java Virtual Machine attempts to read a class file and determines that the file is malformed or otherwise cannot be interpreted as a class file.

```
        public’‘class’‘java.lang.ClassFormatError
                                                extends’‘java.lang.LinkageError’‘
        {
                          //        Constructors
                  public ClassFormatError();
```

```
                    //        Constructs a ClassFormatError with no detail message.


          ‘‘ public ClassFormatError(String’‘s);
                    //        Constructs a ClassFormatError with the specified detail message.
          }
```

Class Error

Thrown due to some abnormal conditions.

```
          public’‘class’‘java.lang.Error
‘‘‘‘                                    extends’‘java.lang.Throwable’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
                    public Error();
                    //        Constructs an Error with no specified detail message.


                    public Error(String’‘s);
                    //        Constructs an Error with the specified detail message.
          }
```

Class IllegalAccessError

Thrown if an application attempts to access or modify a field, or to call a method that it does not have access to. Normally, this error is caught by the compiler; this error can only occur at runtime if the definition of a class has incompatibably changed.

```
          public’‘class’‘java.lang.IllegalAccessError
                                                  extends’‘java.lang.IncompatibleClassChangeError’‘
          {
                              //        Constructors
          ‘‘ public IllegalAccessError();
                    //        Constructs an IllegalAccessError with no detail message.


          ‘‘ public IllegalAccessError(String’‘s);
                    //        Constructs an IllegalAccessError with the specified detail message.
          }
```

Class IncompatibleClassChangeError

An instance of a subclass of IncompatibleClassChangeError is thrown to indicate that an incompatible class change has occurred to some class definition. The definition of some class, upon which the currently executing method depends, has since changed.

```
          public‘‘class’‘java.lang.IncompatibleClassChangeError
‘‘‘‘                                    extends’‘java.lang.LinkageError’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public IncompatibleClassChangeError();
                    //        Constructs an IncompatibleClassChangeError with no detail message.



‘‘‘‘      public IncompatibleClassChangeError(String’‘s);
                    //        Constructs an IncompatibleClassChangeError with the specified detail
message.
          }
```

Class InstantiationError

Thrown when an application tries to use the Java new construct to instantiate an abstract class or an interface. Normally, this error is caught by the compiler; this error can only occur at runtime if the definition of a class has incompatibably changed.

```
          public‘‘class’‘java.lang.InstantiationError
‘‘‘‘                                    extends’‘java.lang.IncompatibleClassChangeError’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public InstantiationError();
                    //        Constructs an InstantiationError with no detail message.


‘‘‘‘      public InstantiationError(String’‘s);
                    //        Constructs an InstantiationError with the specified detail message.
          }
```

Class InternalError

Thrown to indicate some unexpected internal error has occurred in the Java Virtual Machine.

```
          public‘‘class’‘java.lang.InternalError
‘‘‘‘                                    extends’‘java.lang.VirtualMachineError’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public InternalError();
                    //        Constructs an InternalError with no detail message.


‘‘‘‘      public InternalError(String’‘s);
```

```
                    //        Constructs an InternalError with the specified detail message.
          }
```

Class LinkageError

LinkageError indicate that a class has some dependency on another class.

```
          public‘‘class’‘java.lang.LinkageError
‘‘‘‘                                    extends’‘java.lang.Error’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public LinkageError();
                    //        Constructs a LinkageError with no detail message.


‘‘‘‘      public LinkageError(String’‘s);
                    //        Constructs a LinkageError with the specified detail message.
          }
```

Class NoClassDefFoundError

Thrown if the Java Virtual Machine or a classloader tries to load in the definition of a class (as part of a normal method call or as part of creating a new instance using the new expression) and no definition of the class could be found.

```
          public‘‘class’‘java.lang.NoClassDefFoundError
‘‘‘‘                                    extends’‘java.lang.LinkageError’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public NoClassDefFoundError();
                    //        Constructs a NoClassDefFoundError with no detail message.


‘‘‘‘      public NoClassDefFoundError(String’‘s);
                    //        Constructs a NoClassDefFoundError with the specified detail message.
          }
```

Class NoSuchFieldError

Thrown if an application tries to access or modify a specified field of an object, and that object no longer has that field. Normally, this error is caught by the compiler; this error can only occur at runtime if the definition of a class has incompatibably changed.

```
          public‘‘class’‘java.lang.NoSuchFieldError
‘‘‘‘                                    extends’‘java.lang.IncompatibleClassChangeError’‘
```

```
          {
‘‘‘‘‘‘‘‘  //        Constructors
                    public NoSuchFieldError();
                    //        Constructs a NoSuchFieldException with no detail message.


                    public NoSuchFieldError(String’‘s);
                    //        Constructs a NoSuchFieldException with the specified detail message.
          }
```

Class NoSuchMethodError

Thrown if an application tries to call a specified method of a class (either static or instance),and that class no longer has a definition of that method.

```
          public’‘class’‘java.lang.NoSuchMethodError
‘‘‘‘                                    extends’‘java.lang.IncompatibleClassChangeError’‘
          {
‘‘‘‘‘‘‘‘  //        Constructors
‘‘‘‘      public NoSuchMethodError();
                    //        Constructs a NoSuchMethodException with no detail message.


          ‘‘ public NoSuchMethodError(String’‘s);
                    //        Constructs a NoSuchMethodException with the specified detail message.
          }
```

Class OutOfMemoryError

Thrown when the Java Virtual Machine is out of cannot allocate an object because it is out of memory, and no more memory could be made available by the garbage collector.

```
‘‘‘‘public’‘class’‘java.lang.OutOfMemoryError
‘‘‘‘                                    extends’‘java.lang.VirtualMachineError’‘
          {
‘‘‘‘‘‘‘‘  //        Constructors
          public OutOfMemoryError();
                    //        Constructs an OutOfMemoryError with no detail message.


          public OutOfMemoryError(String’‘s);
                    //        Constructs an OutOfMemoryError with the specified detail message.
          }
```

Class StackOverflowError

Thrown when a stack overflow occurs because an application recurses too deeply.

```
          public‘‘class’‘java.lang.StackOverflowError
‘‘‘‘                                    extends’‘java.lang.VirtualMachineError’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public StackOverflowError();
                    //        Constructs a StackOverflowError with no detail message.


‘‘‘‘      public StackOverflowError(String’‘s);
                    //        Constructs a StackOverflowError with the specified detail message.
          }
```

Class ThreadDeath

An instance of ThreadDeath is thrown in the victim thread when the stop method with zero arguments in class Thread is called.

```
          public‘‘class’‘java.lang.ThreadDeath
‘‘‘‘                                    extends’‘java.lang.Error’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
          ‘‘ public ThreadDeath();
                    //        Constructs a new thread death object.
          }
```

Class UnknownError

Thrown when an unknown but serious exception has occurred in the Java Virtual Machine.

```
          public‘‘class’‘java.lang.UnknownError
‘‘‘‘                                    extends’‘java.lang.VirtualMachineError’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
                    public UnknownError();
                              //        Constructs an UnknownError with no detail message.


                    public UnknownError(String’‘s);
                              //        Constructs an UnknownError with the specified detail message.
          }
```

Class UnsatisfiedLinkError

Thrown if the The Java Virtual Machine cannot find an appropriate native-language definition of a method declared native.

```
public‘‘class’‘java.lang.UnsatisfiedLinkError
                                        extends’‘java.lang.LinkageError’‘
{
                    //      Constructors
public UnsatisfiedLinkError();
          //      Constructs an UnsatisfiedLinkError with no detail message.


public UnsatisfiedLinkError(String’‘s);
          //      Constructs an UnsatisfiedLinkError with the specified detail message.
}
```

Class VerifyError

Thrown when the "verifier" detects that a class file, though well-formed, contains some sort of internal inconsistency or security problem.

```
          public‘‘class’‘java.lang.VerifyError
          ‘‘‘‘                          extends’‘java.lang.LinkageError’‘
          {
‘‘‘‘‘‘‘‘ //      Constructors
                    public VerifyError();
                              //      Constructs an VerifyError with no detail message.


                    public VerifyError(String’‘s);
                              //      Constructs an VerifyError with the specified detail message.
          }
```

Class VirtualMachineError

Thrown to indicate that the Java Virtual Machine is broken or has run out of resources necessary for it to continue operating.

```
          public‘‘class’‘java.lang.VirtualMachineError
‘‘‘‘                          extends’‘java.lang.Error’‘
          {
‘‘‘‘‘‘‘‘ //      Constructors
‘‘‘‘      public VirtualMachineError();
                    //      Constructs a VirtualMachineError with no detail message.
```

```
‘‘‘‘      public VirtualMachineError(String’‘s);
                    //        Constructs a VirtualMachineError with the specified detail message.
          }
```

Chapter’‘14

The Java Utility Package

by Debasis Samanta

CONTENTS

- Introduction
- Class BitSet
- Class Date
- Class Dictionary
- Class Hashtable
- Class Observable
- Class Properties
- Class Random
- Class Stack
- Class StringTokenizer
- Class Vector
- Interface Enumeration
- Interface Observer
- The awt Event Handler
- Class EmptyStackException
- Class NoSuchElementException

Introduction

The Package java.util contains miscellaneous utility classes, and related interfaces, and exceptions. It includes generic data structures like Dictionary, Hashtable, Stack, Vector, string manipulation, random number generation, system properties, notification and enumeration of data structures, and calender and date utilities.

The java.util package also contains the Observer interface and Observable class, which allow objects to notify one another when they change.

Following is the list of classes, interfaces, and exceptions which belong to this package.

```
          Classes
```

```
                    Class BitSet’‘
                    Class Date’‘
                    Class Dictionary’‘
                    Class Hashtable’‘
                    Class Observable’‘
                    Class Properties’‘
                    Class Random’‘
                    Class Stack’‘
                    Class StringTokenizer’‘
                    Class Vector’‘
          Interfaces
                    Interface Enumeration’‘
                    Interface Observer’‘
          Exceptions
                    Class EmptyStackException’‘
                    Class NoSuchElementException’‘
```

Class BitSet

This class implements a vector of bits that grows as needed. Each component of the bit set has a boolean value. The bits of a BitSet are inded by nonnegative integers. Individual bits can be examined, set, or cleared. By default, all bits in the set initially have the value false. Every bit set has a current size, which is the number of bits currently in the bit set. Following is the whole class BitSet :

```
                    public’‘final’‘class’‘java.util.BitSet extends’‘java.lang.Object’‘
                    implements java.lang.Cloneable’‘
                    {
                                        //        Constructors
                              public BitSet();
                              //        Creates a new bit set. All bits are initially false.
                              public BitSet(int’‘nbits);
                              //        Creates a bit set whose initial size is nbits, the specified
number of bits. All bits are initially false.


                               // Methods
                              public void and(BitSet’‘set);
                              public void clear(int’‘bit);
                              public Object clone();
                              public boolean equals(Object’‘obj);
                              public boolean get(int’‘bit);
                              public int hashCode();
                              public void or(BitSet’‘set);
                              public void set(int’‘bit);
```

```
                    public int size();
                    public String toString();
                    public void xor(BitSet’‘set);
                }

    All the mellthods are described in Table 3.1.
```

Table 3.1

Method	Description
and(BitSet set)	Performs a logical ***and*** of this target bit set with the argument bit set. This bit set is modified so that each bit in it has the value true if and only if it both initially had the value true and the corresponding bit in the bit set argument also had the value true.
clear(int bit)	Sets the bit specified by the index to false.
clone()	The clone of the bit set is another bit set that has exactly the same bits set to true as this bit set and the same current size.
equals(Object obj)	The result is true if and only if the argument is not null and is a Bitset object that has exactly the same set of bits set. The current sizes of the two bit sets are not compared, false otherwise.
get(int bit)	Returns the value of the bit with the specified index.
hashCode()	Returns a hash code value for this bit set.

Table 3.1(contd...)"

or(BitSet set)	Performs a logical ***or*** of this bit set with the bit set argument. This bit set is modified so that a bit in it has the value true if and only if it either already had the value true or the corresponding bit in the bit set argument has the value true.44
set(int bit)	Sets the bit specified by the index to true.
size()	Returns the number of bits currently in this bit set.
toString()	Returns a string representation of this bit set.
xor(BitSet set)	Performs a logical x***or*** of this bit set with the bit set argument. This bit set is modified so that a bit in it has the value true if and only if one of the following statements holds: The bit initially has the value true, and the corresponding bit in the argument has the value false. The bit initially has the value false, and the corresponding bit in the argument has the value true.

Class Date

While the Date class is intended to reflect UTC (Coordinated Universal Time), it may not do so exactly, depending on the host environment of the Java Virtual Machine. Nearly all modern operating systems assume that 1 day = 24 x 60

x 60 = 86400 seconds in all cases. In UTC, however, about once every year or two there is an extra second, called a "leap second." The leap second is always added as the last second of the day, and always on December 31 or June 30. For example, the last minute of the year 1995 was 61 seconds long, thanks to an added leap second. Most computer clocks are not accurate enough to be able to reflect the leap-second distinction.

The class Date provides an abstraction of dates and times. Dates may be constructed from a year, month, date (day of month), hour, minute, and second. Those six components, as well as the day of the week, may be extracted from a date. Dates may also be compared and converted to a readable string form. A date is represented to a precision of one millisecond. As examples, To print today's date:

```
System.out.println("today = " + new Date());
```

To find out the day of the week for some particular date, for example, January 05, 1964 :

```
new Date(64, 0, 15).getDay();
```

Some computer standards are defined in terms of GMT (Greenwich Mean Time), which is equivalent to UT (Universal Time). GMT is the "civil" name for the standard; UT is the "scientific" name for the same standard. The distinction between UTC and UT is that UTC is based on an atomic clock and UT is based on astronomical observations, which for all practical purposes is an invisibly fine hair to split. Because the earth's rotation is not uniform-it slows down and speeds up in complicated ways-UT does not always flow uniformly. Leap seconds are introduced as needed into UTC so as to keep UTC within 0.9 seconds of UT1, which is a version of UT with certain corrections applied. There are other time and date systems as well; for example, the time scale used by GPS (the satellite-based Global Positioning System) is synchronized to UTC but is not adjusted for leap seconds.
In all methods of class Date that accept or return year, month, date, hours, minutes, and seconds values, the following representations are used :

- ’‘A year y represented by the integer y - 1900.
- ’‘A month is represented by an integer form 0 to 11; 0 is January, 1 is February, and so on; thus 11 is December.
- ’‘A date (day of month) is represented by an integer from 1 to 31 in the usual manner.
- ’‘An hour is represented by an integer from 0 to 23. Thus the hour from midnight to 1 AM is hour 0, and the hour from noon to 1 PM is hour 12.
- ’‘A minute is represented by an integer from 0 to 59 in the usual manner.
- ’‘A second is represented by an integer from 0 to 60; the value 60 occurs only for leap seconds and even then only in Java implementations that actually track leap seconds correctly.

In all cases, arguments given to methods for these purposes need not fall within the indicated ranges; for example, a date may be specified as January 32 and is interpreted as meaning February 1.

```
          Let us see the class stucture for the above mentioned facts.


          public’‘class’‘java.util.Date extends’‘java.lang.Object

          {

                                //        Constructors

                                public Date();

                                /*        Allocates a Date object and initializes it so that it represents
the time at which it was allocated measured to the nearest millisecond.

                                */

                                public Date(int’‘year, int’‘month, int’‘date);

                                /*        Allocates a Date object and initializes it so that it represents
midnight, local time, at the beginning of the day specified by the year, month, and date arguments.

                                */

                                public Date(int’‘year, int’‘month, int’‘date,int’‘hrs, int’‘min);
```

```
                                /*        Allocates a Date object and initializes it so that it represents
the specified hour and minute, local time, of the date specified by the year, month, and date arguments.

                                */

                                public Date(int’‘year, int’‘month, int’‘date,int’‘hrs, int’‘min, int’‘sec);

                                /*        Allocates a Date object and initializes it so that it represents
the specified hour, minute, and second, local time of the date specified by the year, month, and date
arguments.

                                */

                                public Date(long’‘date);

                                /*        Allocates a Date object and initializes it to represent the
specified number of milliseconds since January 1, 1970, 00:00:00GMT.

                                */

                                public Date(String’‘s);

                                /*        Allocates a Date object and initializes it that that it
represents the date and time indicated by the string s, which is interpreted as if by the parse method.

                                */

                                          // Methods

                                public boolean after(Date’‘when);

                                public boolean before(Date’‘when);

                                public boolean equals(Object’‘obj);

                                public int getDate();

                                public int getDay();

                                public int getHours();

                                public int getMinutes();

                                public int getMonth();

                                public int getSeconds();

                                public long getTime();

                                public int getTimezoneOffset();

                                public int getYear();

                                public int hashCode();

                                public static long parse(String’‘s);

                                public void setDate(int’‘date);

                                public void setHours(int’‘hours);

                                public void setMinutes(int’‘minutes);

                                public void setMonth(int’‘month);

                                public void setSeconds(int’‘seconds);

                                public void setTime(long’‘time);

                                public void setYear(int’‘year);

                                public String toGMTString();

                                public String toLocaleString();

                                public String toString();

                                public static long UTC(int’‘year, int’‘month, int’‘date,int’‘hrs, int’‘min,
int’‘sec);
```

```
                }


        The methods are listed in Table 3.2.
```

Table 3.2

Method	Description
after(Date when)	**Returns:** true if this date is after the argument date; false otherwise.
before(Date when)	**Returns:** true if this date is before the argument date; false otherwise.
equals(Object obj)	The result is true if and only if the argument is not null and is a Date object that represents the same point in time, to the millisecond, as this object .
getDate()	**Returns:** the day of the month represented by this date. The value returned is between 1 and 31.
getDay()	**Returns:** the day of the week represented by this date. The value returned is between 0 and 6, where 0 represents Sunday.
getHours()	**Returns:** the hour represented by this date. The value returned is between 0 and 23, where 0 represents midnight.
getMinutes()	**Returns:** the number of minutes past the hour represented by this date. The value returned is between 0 and 59.
getMonth()	**Returns:** the month represented by this date. The value returned is between 0 and 11, with the value 0 representing January.
getSeconds()	**Returns:** the number of seconds past the minute represented by this date. The value returned is between 0 and 60. The value 60 can only occur on those Java Virtual Machines that take leap seconds into account.
getTime()	**Returns:** the number of milliseconds since January 1, 1970, 00:00:00 GMT represented by this date.
getTimezoneOffset()	Determines the local time zone offset. The time zone noffset is the number of minutes that must be added to Greenwich Mean Time to give the local time zone.
getYear()	**Returns:** the year represented by this date, minus 1900.
hashCode()	**Returns:** a hash code value for this object.

Table 3.2 (Contd..)

parse(String s)	Given a string representing a time, parse it and return the time value.

	This method recognizes most standard syntaxes. It accepts many syntaxes; in particular, it recognizes the IETF standard date syntax: "Sat, 12 Aug 1995 13:30:00 GMT". It also understands the continental US time zone abbreviations, but for general use, a timezone offset should be used: "Sat, 12 Aug 1995 13:30:00 GMT+0430" (4 hours, 30 minutes west of the Greenwich meridian). If no time zone is specified, the local time zone is assumed. GMT and UTC are considered equivalent.777
setDate(int date)	Sets the day of the month of this date to the specified value.
setHours(int hours)	Sets the hour of this date to the specified value.
setMinutes(int minutes)	Sets the minutes of this date to the specified value.
setMonth(int month)	Sets the month of this date to the specified value.
setSeconds(int seconds)	Sets the seconds of this date to the specified value..
setTime(long time)	Sets this date to represent the specified number of milliseconds since January 1, 1970 00:00:00 GMT.
setYear(int year)	Sets the year of this date to be the specified value plus 1900.
toGMTString()	Creates a string representation of this date. The result is of the form : "12 Aug 1995 02:30:00 GMT"; the day of the month is always one or two digits. The other fields have exactly the width shown. The timezone is always given as "GMT".
toLocaleString()	Creates a string representation of this date is an implementation-dependent form.
toString()	Creates a canonical string representation of the date. The result is of the form : "Mon Jul 12 02:30:00 PDT 1998".
UTC(int year, int month, int date, int hrs, int min, int sec)	Determines the date and time based on the arguments. The arguments are interpreted in UTC, not in the local time zone.

Class Dictionary

The Dictionary class is the abstract parent of any class, such as Hashtable which maps keys to values. Any non-null object can be used as a key and as a value.

```
public‘‘abstract’‘class’‘java.util.Dictionary extends’‘java.lang.Object’‘
{
                        //        Constructors
            public Dictionary();
            //        The default constructor.


            //        Methods
            public abstract Enumeration elements();
```

```
                public abstract Object get(Object’‘key);
                public abstract boolean isEmpty();
                public abstract Enumeration keys();
                public abstract Object put(Object’‘key, Object’‘value);
                public abstract Object remove(Object’‘key);
                public abstract int size();
        }

        Methods are described in Table 3.3.
```

Table 3.3

Method	Description
elements()	**Returns:** An enumeration *of the values in the dictionary.*
get(Object key)	**Returns:** the value to which the key is mapped in this dictionary; null if the key is not mapped to any value is thise dictionary.
isEmpty()	**Returns:** true if this dictionary maps no keys to values; false otherwise.
keys()	**Returns:** An enumeration *of the keys in this dictionary.*
put(Object key, Object value)	**Returns:** the previous value to which the key was mapped in the dictionary, or null if the key did not have a previous mapping.
remove(Object key)	Removes the key (and its corresponding value) from this dictionary. This method does nothing if the key is not in this dictionary.
size()	**Returns:** the number of keys in this dictionary.

Class Hashtable

This class implements a hash table, which maps keys to values. Any non-null object can be used as a key or as a value.To successfully store and retrieve objects from a hash table, the objects used as keys must implement the hashCode method and the equals method.

An instance of Hashtable has two parameters that affect its efficiency: its capacity and its load factor. The load factor should be between 0.0 and 1.0. When the number of entries in the hash table exceeds the product of the load factor and the current capacity, the capacity is increased by calling the rehash method. Larger load factors use memory more efficiently, at the expense of larger expected time per lookup.

If many entries are to be made into a Hashtable, creating it with a sufficiently large capacity may allow the entries to be inserted more efficiently than letting it perform automatic rehashing as needed to grow the table.

As an example, following is a code to create a hashtable of numbers. It uses the names of the numbers as keys :

```
Hashtable’‘numbers’‘=‘‘new’‘Hashtable();
numbers.put("one",’‘new’‘Integer(1));
numbers.put("two",’‘new’‘Integer(2));
numbers.put("three",’‘new’‘Integer(3));
```

To retrieve a number use the following code:

```
Integer’‘n’‘=‘‘(Integer)numbers.get("two");
if’‘(n’‘!=‘‘null)’‘{
        System.out.println("two’‘=‘‘"‘‘+’‘n);
}
```

Now, let us see the class structure.

```
public’‘class’‘java.util.Hashtable      extends’‘java.util.Dictionary’‘
implements java.lang.Cloneable’‘
{
        //      Constructors
    public Hashtable();
    //      Constructs a new empty hashtable.
    public Hashtable(int’‘initialCapacity);
    /*      Constructs a new, empty hash table with the specified initial capacity.
            Throws : IllegalArgumentException, if the initial capacity is less than or equal to zero.
    */
    public Hashtable(int’‘initialCapacity, float’‘loadFactor);
    /*      Constructs a new, empty hashtable with the specified initial capacity and the specified load factor.
            Throws : IllegalArgumentException, if the initial capacity is less than or equal to zero, or if the load factor is less than or equal to zero.
    */
        //      Methods
    public void clear();
    public Object clone();
    public boolean contains(Object’‘value);
    public boolean containsKey(Object’‘key);
    public Enumeration elements();
    public Object get(Object’‘key);
```

```
                    public boolean isEmpty();
                    public Enumeration keys();
                    public Object put(Object’‘key, Object’‘value);
                    protected void rehash();
                    public Object remove(Object’‘key);
                    public int size();
                    public String toString();
                }
```

Methods are stated in Table 3.4.

Table 3.4

.

Method	Description
clear()	Clears this hash table so that it contains no keys.
clone()	Creates a shallow copy of this hash table. The keys and values themselves are not cloned.
contains(Object value)	**Returns :** true if some key maps to the value argument in this hash table; false otherwise. **Throws** : `NullPointerException`, If the value is null.
containsKey(Object key)	**Returns:** true if the specified object is a key in this hash table; false otherwise.
elements()	**Returns:** an enumeration.
get(Object key)	**Returns:** the value to which the key is mapped in this hash table; null if the key is not mapped to any value is this hash table.
isEmpty()	**Returns :** true if this hash table maps no keys to values; false otherwise.
keys()	**Returns:** an enumeration *of the keys in this hash table.*
put(Object key, Object value)	**Returns :** the previous value of of the specified key in this hash table, or null if it did not have one. **Throws** : `NullPointerException`, If the value is null.
rehash()	Rehashes the contents of the hash table into a hash table with a larger capacity. This method is called automatically when the number of keys in the hash table exceeds this hash table's capacity and load factor.
remove(Object key)	Removes the key (and its corresponding value) from this hash table. This method does nothing if the key is not in the hash table.

size()	**Returns:** the number of keys in this hash table.
toString()	**Returns:** a string representation of this hash table.

Class Observable

This class represents an observable object, or "data" in the model-view paradigm. It can be subclassed to represent an object that the application wants to have observed.

An observable object can have one or more observers After an observerable instance changes, an application calling the Observerable's notifyObservers method causes all of its observers to be notified of the change by a call to their update method.

```
          The class structure is as shown below :


          public‘‘class’‘java.util.Observable
‘‘‘‘                                              extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public Observable();
                    //        The default constructor.


‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public void addObserver(Observer’‘o);
‘‘‘‘      protected void clearChanged();
‘‘‘‘      public int countObservers();
‘‘‘‘      public void deleteObserver(Observer’‘o);
‘‘‘‘      public void deleteObservers();
‘‘‘‘      public boolean hasChanged();
‘‘‘‘      public void notifyObservers();
‘‘‘‘      public void notifyObservers(Object’‘arg);
‘‘‘‘      protected void setChanged();
          }


          Methods are described in Table 3.5.
```

Table 3.5

Method	Description
addObserver(Observer o)	Adds an observer to the set of observers for this object.
clearChanged()	Indicates that this object has no longer changed, or that it has already notified all of its observers of its most recent change.

	This method is call automatically by the `notifyObservers` methods .
countObservers()	**Returns:** the number of observers of this object.
deleteObserver(Observer o)	Deletes an observer from the set of observers of this object.
deleteObservers()	Clears the observer list so that this object no longer has any observers.
hasChanged()	Determines if this object has changed. **Returns:** true if the `setChanged` method has been called more recently than the `clearChanged` method on this object; false otherwise.
notifyObservers()	If this object has changed, as indicated by the `hasChanged` method, then notify all of its observers and then call the `clearChanged` method to indicate that this object has no longer changed. Each observer has its update method called with two arguments: the observable object and null argument
notifyObservers(Object arg)	If this object has changed, as indicated by the `hasChanged` method. 12 then notify all of its observers and then call the `clearChanged` method to indicate that this object has no longer changed. Each observer has its update method called with two arguments: the observable object and the arg argument
setChanged()	Indicates that this object has changed.

Class Properties

The Properties class represents a persistent set of properties. The Properties can be saved to a stream or loaded from a stream. Each key and and its corresponding value in the property list is a string.

A property list can contain another property list as its "defaults"; this second property list is searched if the property key is not found in the original property list.

```
          public‘‘class’‘java.util.Properties
                                                              extends’‘java.util.Hashtable’‘
          {
                                //        Member elements
                     protected Properties defaults;
                     //        A property list which contains contains default values for any keys not
found in this property list.


                                //        Constructors
                     public Properties();
                     //        Creates an empty property list with no default values.
                     public Properties(Properties’‘defaults);
                     //        Creates an empty property list with the specified defaults.
```

```
                    //        Methods
            public String getProperty(String‘‘key);
            public String getProperty(String‘‘key, String‘‘defaultValue);’‘
            public void list(PrintStream’‘out);
            public void load(InputStream’‘in);
            public Enumeration propertyNames();
            public void save(OutputStream’‘out, String’‘header);
      }
      Methods are listed in Table 3.6.
```

Table 3.6

Method	Description
getProperty(String key)	Searches for the property with the specified key in this property list. **Returns:** the value in this property list with the specified key value. If the key is not found in this property list, the default property list and its defaults, recursively, are then checked. The method returns null if the property is not found.
getProperty(String key, String defaultValue)	Searches for the property with the specified key in this property list. **Returns:** the value in this property list with the specified key value. If the key is not found in this property list, the default property list, and its defaults, recursively, are then checked. The method returns the default value argument if the property is not found.
list(PrintStream out)	Prints this property list out to the specified output stream. This method is useful for debugging.
load(InputStream in)	Reads a property list from an input stream. **Throws :** IOException, if an error occurred when reading from the input stream.13
propertyNames()	**Returns:** An enumeration *of all the keys in this property list, including the keys in the default property list.*
save(OutputStream out, String header)	Stores this property list to the specified output stream. The string header is printed as a comment at the beginning of the stream.

Class Random

An instance of this class is used to generate a stream of pseudo-random numbers. The class uses a 48-bit seed, which is modified using a linear congruential formula.

If two instances of Random are created with the same seed, and the same sequence of method calls is made for each, they will generate and return identical sequences of numbers.

Many applications will find the random method in class Math simpler to use.

```
          public’‘class’‘java.util.Random extends’‘java.lang.Object’‘
          {
                                    //        Constructors
                    public Random();
                    //        Creates a new random number generator. Its seed is initialized to a value
based on the current time.
                    public Random(long’‘seed);
                    //        Creates a new random number generator using a single long seed.


                    //        Methods
                    public double nextDouble();
                    public float nextFloat();
                    public double nextGaussian();
                    public int nextInt();
                    public long nextLong();
                    public void setSeed(long’‘seed);
          }


          Table 3.7 is to list the methods in’‘this class.
```

Table 3.7

Method	Description
nextDouble()	**Returns:** the next pseudorandom, uniformally distributed double value between 0.0 and 1.0 from this random number generator's sequence.
nextFloat()	**Returns:** the next pseudorandom, uniformally distributed float value between 0.0 and 1.0 from this random number generator's sequence.
nextGaussian()	**Returns:** the next pseudorandom, Gaussian ("normally") distributed double value with mean 0.0 and standard deviation 1.0 from this random number generator's sequence.
nextInt()	**Returns:** the next pseudorandom, uniformally distributed int value from this random number generator's sequence.
nextLong()	**Returns:** the nextpseudorandom, uniformally distributed long value from this random number generator's sequence.
setSeed(long seed)	Sets the seed of this random number generator using a single long seed.

Class Stack

The Stack class represents a last-in-first-out (LIFO) stack of objects.

```
          public‘‘class’‘java.util.Stack extends’‘java.util.Vector’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public Stack();
                    //        Creates a new stack with no elements.

‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public boolean empty();
‘‘‘‘      public Object peek();
‘‘‘‘      public Object pop();
‘‘‘‘      public Object push(Object’‘item);
‘‘‘‘      public int search(Object’‘o);
          }


          Methods are shown in Table 3.8.
```

Table 3.8

Method	Description
empty()	**Returns:** true if this stack is empty; false otherwise.
peek()	Looks at the object at the top of this stack without removing it from the stack. **Throws :** `EmptyStackException`, if this stack is empty.
pop()	Removes the object at the top of this stack and returns that object as the value of this function. **Throws :** `EmptyStackException`, if this stack is empty.
push(Object item)	Pushes an `item` onto the top of this stack.
search(Object o)	Determines if an object is on this stack. **Returns:** The distance from the top of the stack that at which the object is located at; the return value -1 indicates that the object is not on the stack.

Class StringTokenizer

The string tokenizer class allows an application to break a string into tokens. The tokenization method is much simpler than the one used by the StreamTokenizer class. The StringTokenizer methods do not distinguish among identifiers, numbers, and quoted strings, not does it recognize and skip comments.

The set of delimiters (the characters that separate tokens) may be specified either at creation time or on a per-token basis.

An instance of StringTokenizer behaves in one of two ways, depending on whether it was created with the return Tokens flag having the value true or false:

If the flag is false, delimiter characters merely serve to separate tokens. A token is a maximal sequence of consecutive characters that are delimiters.

If the flag is true, delimiters characters are considered to be tokens. A token is either one delimiter character, or a maximal sequence of consecutive characters that are not delimiters.

```
          The following is one example of the use of the tokenizer. The code :


          StringTokenizer’‘st’‘=‘‘new’‘StringTokenizer("I love Java !");
                              while’‘(st.hasMoreTokens())’‘{
‘‘‘‘‘‘                                        println(st.nextToken());
                              }
          Prints the following output’‘:
                              I
                              love
                              java
                              !


          The class is furnished below :


          public’‘class’‘java.util.StringTokenizer extends’‘java.lang.Object’‘
          implements java.util.Enumeration’‘
          {
                              //        Constructors
                    public StringTokenizer(String’‘str);
                    /*        Constructs a string tokenizer for the specified string. The tokenizer uses
the default delimiter set, which is " \t\n\r", the space character, the tab character, the newline
character, and the carriage return character.
                    */
                    public StringTokenizer(String’‘str, String’‘delim);
                    /*        Constructs a string tokenizer for the specified string. The characters in
the delim argument are the delimiters for separating tokens.
                    */
                    public StringTokenizer(String’‘str, String’‘delim,’‘boolean’‘returnTokens);
                    /*        Constructs a string tokenizer for the specified string. The characters in
the delim argument are the delimiters for separating tokens. If the returnTokens flag is true, then the
delimiter characters are also returned as tokens. Each delimiter is returned as a string of length one. If
the flag is false, the delimiter characters are skipped and only serve as separators between tokens.
                    */


                              //        Methods
                    public int countTokens();
                    public boolean hasMoreElements();
                    public boolean hasMoreTokens();
```

```
                public Object nextElement();
                public String nextToken();
                public String nextToken(String’‘delim);
        }

        Methods are described in Table3.9.
```

Table 3.9

Method	Description
countTokens()	Calculates the number of times that this tokenizer's `nextToken` method can be called before it generates an exception. **Returns:** the number of tokens remaining in the string using thie current delimiter set.
hasMoreElements()	This method returns the same value as the following `has MoreTokens` method. It exists so that this class can implement the enumeration interface. **Returns:** true if there are more tokens; false otherwise.
hasMoreTokens()	**Returns:** true if there are more tokens available from this tokenizer's string; false otherwise.
nextElement()	This method returns the same value as the `nextToken` method, except that its declared return value is `Object` rather than `String`. **Throws :** `NoSuchElementException`, if there are no more tokens in this tokenizer's string.
nextToken()	**Returns:** the next token from this string tokenizer. **Throws :** NoSuchElementException, if there are no more tokens in this tokenizer's string.
nextToken(String delim)	Gets the next token in this stringt tokenizer's string. The new delimiter set remains the default after this call. **Throws :** `NoSuchElementException`, if there are no more tokens in this string.

Class Vector

The Vector class implements a growable array of objects. Like an array, it contains components that can be accessed using an integer index. However, the size of a Vector can grow or shrink as needed to accommodate adding and removing items after the Vector has been created.

Each vector tries to optimize storage management by maintaining a capacity and a capacityIncrement. The capacity is always at least as large as the vector size; it is usually larger because as components are added to the vector, the vector's storage increases in chunks the size of capacityIncrement. An application can increase the capacity of a vector before inserting a large number of components; this reduces the amount of incremental reallocation.

```
          The class Vector is stated below :


          public‘‘class’‘java.util.Vector extends’‘java.lang.Object’‘

          implements java.lang.Cloneable’‘

          {

‘‘‘‘‘‘‘  //        Member elements

‘‘‘‘      protected int capacityIncrement;

                    /*        The amount by which the capacity of the vector is automatically incremented
when its size becomes greater than its capacity. If the capacity is 0, the capacity of the vector is
doubled each time it needs to grow.

                    */

‘‘‘‘      protected int elementCount;

                    //        The number of valid components in the vector.

‘‘‘‘      protected Object elementData[];

                    /*        The array buffer into which the components of the vector are stored. The
length of this array buffer is the (current) capacity of the vector.

                    */

‘‘‘‘‘‘‘  //        Constructors

‘‘‘‘      public Vector();

                    //        Constructs an empty vector.

                    public Vector(int’‘initialCapacity);

                    //        Constructs an empty vector. Its initial capacity is the specified argument
initialCapacity.

‘‘‘‘      public Vector(int’‘initialCapacity, int’‘capacityIncrement);

                    //        Constructs an empty vector with the specified capacity and the specified
capacity increment.


‘‘‘‘‘‘‘  //        Methods

‘‘‘‘      public final void addElement(Object’‘obj);

‘‘‘‘      public final int capacity();

‘‘‘‘      public Object clone();

‘‘‘‘      public final boolean contains(Object’‘elem);

‘‘‘‘      public final void copyInto(Object’‘anArray[]);

‘‘‘‘      public final Object elementAt(int’‘index);

‘‘‘‘      public final Enumeration elements();

‘‘‘‘      public final void ensureCapacity(int’‘minCapacity);

‘‘‘‘      public final Object firstElement();

‘‘‘‘      public final int indexOf(Object’‘elem);

‘‘‘‘      public final int indexOf(Object’‘elem, int’‘index);

‘‘‘‘      public final void insertElementAt(Object’‘obj, int’‘index);

‘‘‘‘      public final boolean isEmpty();

‘‘‘‘      public final Object lastElement();

‘‘‘‘      public final int lastIndexOf(Object’‘elem);
```

```
‘‘‘‘      public final int lastIndexOf(Object’‘elem, int’‘index);
‘‘‘‘      public final void removeAllElements();
‘‘‘‘      public final boolean removeElement(Object’‘obj);
‘‘‘‘      public final void removeElementAt(int’‘index);
‘‘‘‘      public final void setElementAt(Object’‘obj, int’‘index);
‘‘‘‘      public final void setSize(int’‘newSize);
‘‘‘‘      public final int size();
‘‘‘‘      public final String toString();
‘‘‘‘      public final void trimToSize();
          }


          Methods are decribed in Table 3.10.
```

Table 3.10

Method	Description
addElement(Object obj)	Adds the specified component to the end of this vector, increasing its size by one. The capacity of this vector is increased if its size becomes greater than its capacity.
capacity()	**Returns :** the current capacity of this vector.
clone()	**Returns :** a clone of this vector.
contains(Object elem)	**Returns :** true if the specified object is an component in this vector; false otherwise.
copyInto(Object anArray[])	Copies the components of this vector into the specified array. The array must be big enough to hold all the objects in this vector.
elementAt(int index)	**Returns :** the component at the specified index. **Throws :** `ArrayIndexOutOfBoundsException`, if an invalid index was given.
elements()	**Returns :** an enumeration *of the components of this vector.*
ensureCapacity(int minCapacity)	Increases the capacity of this vector, if necessary, to ensure that it can hold at least the number of components specified by the minimum capacity argument.
firstElement()	**Returns :** the first component of this vector. **Throws :** `NoSuchElementException,` if this vector has no components.
indexOf(Object elem)	**Returns :** the index of the first occurrence of the argument in this vector; returns -1 if the object is not found.

indexOf(Object elem, int index)	**Returns :** the index of the first occurrence of the object argument in thisn vector at position index or later in the vector; returns -1 if the object is not found.

Table 3.10 (Contd..)

insertElementAt(Object obj, int index)	Inserts the specified object as an component in this vector at the specified index. Each component in this vector with an index greater or equal to the specified index is shifted upward to have an index one greater than the value it had previously. The index must be a value greater than or equal to 0 and less than or equal to the current size of the vector. **Throws :** ArrayIndexOutOfBoundsException, If the index was invalid.
isEmpty()	**Returns :** true if this vector has no components; false otherwise.
lastElement()	**Returns :** the last component of the vector. **Throws :** `NoSuchElementException`, If this vector is empty.
lastIndexOf(Object elem)	**Returns :** the index of the last occurrence of the argument in this vector; returns -1 if the object is not found.
lastIndexOf(Object elem, int index)	Searches backwards for the specified object, starting from the specified index and returns an index to it. **Returns :** the index of the last occurrence of the object argument in this vector at position less than index in the vector; returns -1 if the object is not found.
removeAllElements()	Removes all components from this vector and sets its size to zero.
removeElement(Object obj)	Removes the first occurrence of the argument from this vector. If the object is found in this vector, each component in the vector with an index greater or equal to the object's index is shifted downward to have an index one smaller than the value it had previously. **Returns:** true if the argument was a component of this vector; false otherwise.
removeElementAt(int index)	Deletes the component at the specified index. Each component in this vector with an index greater or equal to the specified index is shifted downward to have an index one smaller than the value it had previously. **Throws :** `ArrayIndexOutOfBoundsException`, if the index was invalid.
setElementAt(Object obj, int index)	Sets the component at the specified index of this vector to be the specified object. The previous component at that position is discarded. **Throws :** `ArrayIndexOutOfBoundsException`, if the index was invalid.
setSize(int newSize)	Sets the size of this vector. If the new size is greater than the current size, new null items are added to the end of the vector. If

	the new size is less than the current size, all components at index `newSize` and greater are discarded.
size()	**Returns :** the number of components in this vector.
toString()	Creates a string representation of this vector. **Returns :** a string representation of this vector.
trimToSize()	Trims the capacity of this vector to be the vector's current size . An application can use this operation to minimize the storage of a vector.

Interface Enumeration

An object that implements the Enumeration interface generates a series of elements, one at a time. Successive calls to the nextElement method return successive elements of the series. For example, to print all elements of a Vector v , the code :

```
          for’‘(Enumeration’‘e’‘=‘‘v.elements();’‘e.hasMoreElements();)’‘
          {‘‘‘‘‘‘‘‘                    System.out.println(e.nextElement());
          }’‘


          Methods are provided to enumerate through the elements of a vector, the keys of a hash table ,
and the values in a hash table. Enumerations are also used to specify the input streams to a
SequenceInputStream.


          public’‘interface’‘java.util.Enumeration
          {
‘‘‘‘‘‘‘‘ // Methods
‘‘‘‘      public abstract boolean hasMoreElements();
                    //        Returns : true if this enumeration contains more elements; false otherwise.
‘‘‘‘      public abstract Object nextElement();
                    /*        Returns : the next element of this enumeration.’‘Throws :
NoSuchElementException, if the element not found.
                    */
          }
```

Interface Observer

A class can implement the Observer interface when it wants to be informed if observable objects changes.

```
          public’‘interface’‘java.util.Observer
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract void update(Observable’‘o, Object’‘arg);
                    /*        This method is called whenever the observed object is changed. An
application calls an observable object's notifyObservers method’‘to have all the object's observers
notified of the change.
                    */
          }
```

Class EmptyStackException

Thrown by methods in the Stack class to indicate that the stack is empty.

```
          public‘‘class’‘java.util.EmptyStackException extends’‘java.lang.RuntimeException{
‘‘‘‘‘‘‘‘  //        Constructors
‘‘‘‘      public EmptyStackException();
                    //        Constructs a new EmptyStackException with no detail message.
          }
```

Class NoSuchElementException

Thrown by the nextElement method of an Enumeration to indicate that there are no more elements in the enumeration.

```
          public‘‘class’‘java.util.NoSuchElementException extends’‘java.lang.RuntimeException’‘
          {
‘‘‘‘‘‘‘‘  //        Constructors
‘‘‘‘public NoSuchElementException();
                    //        Constructs a NoSuchElementException with no detail message.
          public NoSuchElementException(String’‘s);
                    //        Constructs a NoSuchElementException with the specified detail message.
          }
```

Chapter 15

The Abstract Window Toolkit (AWT) Package

by Debasis Samnata

CONTENTS

- Introduction
- Class BorderLayout
- Class Button
- Class Canvas
- Class CardLayout
- Class Checkbox
- Class CheckboxGroup

- Class CheckboxMenuItem
- Class Choice
- Class Color
- Class Component
- Class Container
- Class Dialog
- Class Dimension
- Class Event
- Class FileDialog
- Class FlowLayout
- Class Font
- Class FontMetrics
- Class Frame
- Class Graphics
- Class GridBagConstraints
- Class GridBagLayout
- Class GridLayout
- Class Image
- Class Insets
- Class Label
- Class List
- Class MediaTracker
- Class Menu
- Class MenuBar
- Class MenuComponent
- Class MenuItem
- Class Panel
- Class Point
- Class Polygon
- Class Rectangle
- Class Scrollbar
- Class TextArea
- Class TextComponent
- Class TextField
- Class Toolkit
- Class Window
- Interface LayoutManager
- Interface MenuContainer
- Class AWTException
- Class AWTError

Introduction

The java.awt Package provides an integrated set of classes and interfaces that manages standard graphical user interface (GUI). It contains GUI components such as buttons, lists, menus, and text areas. In addition to these , it also includes containers such as windows, menu bars etc. and higher level components such as diolog boxes, check boxes, scroll bars etc. The AWT contains two more packages : java.awt.image and java.awt.peer. These two packages will be presented in Chapter 21 and Chapter 22 respectively. The various constituents in the java.awt package is listed below :

```
    ````Classes
```
```

```
Class BorderLayout                          Class MenuBar
Class Button                                        Class MenuComponent
Class Canvas                                        Class MenuItem
Class CardLayout                            Class Panel
Class CheckBox                                      Class Point
Class CheckBoxGroup                         Class Polygon
Class CheckBoxMenuItem                      ‘‘‘‘‘‘‘‘Class Rectangle
Class Choice                                        Class ScrollBar
Class Color                                 ‘‘‘‘‘‘‘‘Class TextArea
Class Component                                     Class TextComponent
Class Container                                     Class TextField
Class Dialog                                        Class Toolkit
Class Dimension                                     Class Window
Class Event
Class FileDialog                            Interfaces
Class FlowLayout                            Interface LayoutManager
Class Font                          ‘‘‘‘‘‘‘‘Interface MenuContainer
Class FontMatrices
Class Frame                                 ‘‘‘‘‘‘‘‘Errors
Class Graphics                                      Class AWTException
Class GridBagConstraints’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘Class AWTError
Class GridBagLayout
Class GridLayout
Class Image
Class Insets
Class Label
Class List
Class MediaTracker
Class Menu
```

Class BorderLayout

A BorderLayout lays out a container to contain number of GUI element like buttons, labels, Grids texts etc. Following is the varopus methods in this class :

```
public’‘class’‘java.awt.BorderLayout extends’‘java.lang.Object’‘
          implements java.awt.LayoutManager’‘
          {
                    //        Constructors
                    public BorderLayout();
```

```
                                //        Default constructor, constructs a new border layout.
                                public BorderLayout(int'‘hgap, int'‘vgap);


                                /*        Creates a new border layout with the specified horizontal and
vertical gaps. The horizontal and vertical gaps specify the space between the components.
                                */


                                //        Methods
                                public void'‘addLayoutComponent(String'‘name, Component'‘comp);
                                public void layoutContainer(Container'‘target);
                                public Dimension minimumLayoutSize(Container'‘target);
                                public Dimension preferredLayoutSize(Container'‘target);
                                public void removeLayoutComponent(Component'‘comp);
                                public String toString();
                        }
```

Methods in this class are described in Table 4.1.

Table 4.1

Method	Description
addLayoutComponent(String'‘‘‘‘‘‘‘ name,Component'‘comp)	Adds the specified component to this border layout using the indicated tag.Most applications do not call this method directly. This method is called when a component is added to a container using the two-argument add method.
layoutContainer(Container'‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘target)'‘‘‘	Lays out the container argument using this border layout. This method reshapes the components in the specified container in order to satisfy the constraints of this BorderLayout object. Most applications do not call this method directly. This method is called when a container calls its layout method.
minimumLayoutSize(Container'‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ target)	Determines the minimum size of the target container using this layout manager.This method is called when a container calls its minimumSize method. Most applications do not call this method directly. **Returns :** the minimum dimensions

	needed to lay out the subcomponents of the specified container.
preferredLayoutSize(Container’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ target)	Determines the preferred size of the target container using this layout manager.’‘This’‘method is called when a container calls its preferredSize method. Most applications do not call this method directly. **Returns:** the preferred dimensions to lay out the subcomponents of the specified container.
removeLayoutComponent’‘‘‘‘‘‘‘‘‘‘‘‘‘‘ (Component’‘comp)	Removes the specified component from this border layout. This method is called when a container calls its remove or removeAll methods. Most applications do not call this method directly.
toString()	**Returns :** a string representation of this border layout.

Class Button

This class creates a labeled button.

```
public’‘class’‘java.awt.Button extends’‘java.awt.Component’‘
          {
//        Constructors
                    public Button();
                    //        Creates a button with no label.
                    public Button(String’‘label);
                    //        Creates a button with the indicated label.

                    //        Methods
                    public void addNotify();
                    public String getLabel();
                    protected String paramString();
                    public void setLabel(String’‘label);
          }
```

Table 4.2

Method	Description

addNotify()	This method calls the createButton method of the button's toolkit in order to create a ButtonPeer for this button. This peer allows the application to change the look of a button without changing its functionality. Most applications do not call this method directly.
getLabel()	**Returns:** the label of this button, or null if this button has no label.
paramString()	Returns the parameter string representing the state of this button. This string is useful for debugging. **Returns:** the parameter string of this button.
setLabel(String‘‘label)	Changes this button's label to be the String argument.

Class Canvas

A Canvas component represents a blank rectangular area of the screen onto which the application can draw or from which the application can trap input events from the user. An application must subclass the Canvas class in order to get useful functionality such as creating a custom component. The paint method must be overridden in order to perform custom graphics on the canvas. The AWT sends the canvas all mouse, keyboard, and focus events that occur over it. The gotFocus, lostFocus, keyDown, keyUp, mouseEnter, mouseExit, mouseMove, mouseDrag, mouseDown, and mouseUp methods may be overridden in order to catch user events.

```
public’‘class’‘java.awt.Canvas extends’‘java.awt.Component’‘
          {
                    //        Constructors
                    public Canvas();



                    //        Methods
                    public void addNotify();
                    public void paint(Graphics’‘g);
          }
```

Table 4.3

Method	Description
addNotify()	This method calls the `createCanvas` method of the canvas's toolkit in order to create a CanvasPeer for this canvas. This peer allows the application to change the look of a canvas without changing its functionality.’‘Most applications do not call this method directly.
paint(Graphics’‘g)	This method is called to repaint this canvas. Most applications should override this method in order to perform some useful operation.’‘The `paint` method provided by Canvas redraws this canvas's rectangle in the background color.

Class CardLayout

A layout manager for a container that contains several "cards". Only one card is visible at a time, allowing the application to flip through the cards. This class provides us to manage such cards.

```
public‘‘class’‘java.awt.CardLayout extends’‘java.lang.Object’‘
          implements java.awt.LayoutManager’‘
          {
                    //        Constructors
                    public CardLayout();
                    //        Default constructor, creates a new card layout.
                    public CardLayout(int’‘hgap, int’‘vgap);
                    /*        Creates a new card layout with the specified horizontal and vertical gaps.
The horizontal gaps are placed and the left and right edge. The vertical gaps are placed at the top and
bottom edge.
                    */
                    //        Methods
                    public void addLayoutComponent(String’‘name, Component’‘comp);’‘
                    public void first(Container’‘target);
                    public void last(Container’‘target);
                    public void layoutContainer(Container’‘target);
                    public Dimension minimumLayoutSize(Container’‘target);
                    public void next(Container’‘target);
                    public Dimension preferredLayoutSize(Container’‘target);
                    public void previous(Container’‘target);
                    public void removeLayoutComponent(Component’‘comp);
                    public void show(Container’‘target, String’‘name);
                    public String toString();
          }
```

Table 4.4

Method	Description
addLayoutComponent(String’‘name,’‘‘‘‘‘‘‘‘‘‘‘‘ Component’‘comp)	Adds the specified component to the card layout using the indicated name tag. The show method can be used to display the component with the specified tag.
first(Container’‘target)	Flips to the first card of the container.
last(Container’‘target)	Flips to the last card of the container.
layoutContainer(Container’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ target)	Lays out the container argument using this card layout.‘‘Each component in the target container is reshaped to be the

	size of the container minus space for surrounding insets, horizontal gaps, and vertical gaps.‘‘Most applications do not call this method directly. This method is called when a container calls its layout method.
minimumLayoutSize’“““““““ (Container’‘target)	Determines the minimum size of the container argument using this card layout.‘‘The minimum width of a card layout is the largest minimum width of the cards in the container, plus twice the horizontal gap, plus the left and right insets. The minimum height of a card layout is the largest minimum height of the cards in the container, plus twice the vertical gap, plus the top and bottom insets.‘‘Most applications do not call this method directly. This method is called when a container calls its layout method. **Returns :** the minimum dimensions needed to lay out the subcomponents of the specified container.
next(Container’‘target)	Flips to the next card of the specified container. If the currently visible card is the last one, this method flips to the first card in the layout.
preferredLayoutSize’“““““““ (Container’‘target)	Determines the preferred size of the container argument using this card layout.‘‘The preferred width of a card layout is the largest preferred width of the cards in the container, plus twice the horizontal gap, plus the left and right insets. The preferred height of a card layout is the largest preferred height of the cards in the container, plus twice the vertical gap, plus the top and bottom insets.‘‘ Most applications do not call this method directly. This method is called when a container calls its `preferredSize` method. **Returns:** the preferred dimensions to lay out the

	subcomponents of the specified container.
previous(Container‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘target)	Flips to the previous card of the specified container. If the currently visible card is the first one, this method flips to the last card in the layout.
removeLayoutComponent‘‘‘‘‘‘‘‘(Component‘‘comp)	Removes the specified component from the layout. This method is called when a container calls its remove or removeAll methods. Most applications do not call this method directly.
show(Container‘‘target,‘‘‘‘‘‘‘‘String‘‘name)	Flips to the component that was added to this layout with the specified name tag. If no such component exists, then nothing happens.
toString()	**Returns:** a String representation of this card layout.

Class CheckBox

A check box is a graphical component that has an "on" (true) and "off" (false) state. Clicking on the check box changes its state from "on" to "off" or from "off" to "on". When the check box is clicked, AWT sends an action event to the check box. This event's target is the check box, and its object is a Boolean giving the new state of the check box. An application should override the action method of the check box or of one of its containing windows in order to cause some action to occur. Optionally, several checkboxes can be grouped together into a CheckboxGroup. At most one button in a group can be in the "on" state at any given time. Pushing a check box to turn it "on" forces any other check box in the group that is "on" to become "off".

```
public‘‘class‘‘java.awt.Checkbox extends‘‘java.awt.Component‘‘
          {
                    //        Constructors
                    public Checkbox();
                    //        Creates a check box with no label. The check box is set to "off" and is not
part of any check box group.
                    public Checkbox(String‘‘label);
                    //        Creates a check box with the specified label. The check box is set to "off"
and is not part of any check box group.
                    public Checkbox(String‘‘label, CheckboxGroup‘‘group, boolean‘‘state);
                    //        Creates a check box with the specified label, in the specified check box
group, and set to the specified state.


                    //        Methods
                    public void addNotify();
                    public CheckboxGroup getCheckboxGroup();
```

```
                    public String getLabel();
                    public boolean getState();
                    protected String paramString();
                    public void setCheckboxGroup(CheckboxGroup’‘g);
                    public void setLabel(String’‘label);
                    public void setState(boolean’‘state);
          }
```

Table 4.5

Method	Description
addNotify()	This method calls the `createCheckbox` method of the check box's toolkit in order to create a CheckboxPeer for this button. This peer allows the application to change the look of a check box without changing its functionality. Most applications do not call this method directly
getCheckboxGroup()	Determines this check box's group. **Returns:’‘**thischeck box's group, or null if it is not part of a check box group.
getLabel()	**Returns:** the label of this check box, or null if this check box has no label.
getState()	Determines if this check box is "on" or "off.". **Returns:** the state of this check box. The value true indicates "on", false indicates "off".
paramString()	Returns the parameter string representing the state of this check box. This string is useful for debugging. **Returns:** the parameter string of this check box.
setCheckboxGroup’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘ (CheckboxGroup’‘g)	Sets the group of this check box to be the specified check box group. If this check box is already in a different check box group, it is first taken out of that group.
setLabel(String’‘label)	Sets this check box's label to be the string argument.
setState(boolean’‘state)	Sets this check box to the specifed boolean state: true indicates "on"; false indicates "off".

Class CheckboxGroup

This class is used to group together a set of Checkbox buttons. Exactly one check box button in a CheckboxGroup can be in the "on" state at any given time. Pushing any button turns it "on" and forces any other button that is "on" to become "off". When a check box in a check box group is clicked, AWT sends an action event to that check box. This event's target is the check box, and its object is a value equal to Boolean.TRUE . No action event is sent to the check

box (if any) that is turned off. An application should override the action method of the check box or of one of its parent containers in order to cause some action to occur.

```
public‘‘class’‘java.awt.CheckboxGroup extends’‘java.lang.Object’‘
          {
                    //        Constructors
                    public CheckboxGroup();
                    //        Default constructor,’‘creates a new CheckboxGroup.

                    //        Methods
                    public Checkbox getCurrent();
                    public void setCurrent(Checkbox’‘box);
                    public String toString();
          }
```

Table 4.6

Method	Description
getCurrent()	**Returns:** the check box in this check box group that is currently "on", or null if all are "off".
setCurrent(Checkbox’‘box)	If the indicated check box argument belongs to this check box group, this method sets that check box to be "on", and all other checkboxes in this group to be "off".’‘If the check box argument is null or belongs to a different check box group, then this method does nothing.
toString()	**Returns:** a string representation of this check box group.

Class CheckboxMenuItem

This class represents a check box that can be included in a menu. Clicking on the check box in the menu changes its state from "on" to "off" or from "off" to "on". When a check box menu item is clicked, the AWT sends an action event to the check box menu item's containing frame. The event's target is the check box menu item, and its object is the string label of the check box.

```
public‘‘class’‘java.awt.CheckboxMenuItem extends’‘java.awt.MenuItem
          {
                    //        Constructors
                    public CheckboxMenuItem(String’‘label);
                    //        Creates a check box with the specified label. The check box is initially
set to "off".
                    //        Methods
                    public void addNotify();
                    public boolean getState();
                    public String paramString();
                    public void setState(boolean t);
```

```
        }
```

Table 4.7

Method	Description
addNotify()	This method calls the createCheckboxMenuItem method of the check box's toolkit in order to create a CheckboxMenuItemPeer for this button. This peer allows the application to change the look of a check box menu item without changing its functionality. Most applications do not call this method directly.
getState()	Determines whether this check box menu item is "on" or "off.". **Returns:** the state of this check box menu item: true indicates "on", false indicates "off".
paramString()	Returns the parameter string representing the state of this check box menu item. This string is useful for debugging. **Returns:** the parameter string of this check box menu item.
setState(boolean'‘t)	Sets this check box menu item to the specifed boolean state: true indicates "on"; false indicates "off".

Class Choice

The Choice class presents a pop-up menu of choices. The current choice is displayed as the title of the menu. After any choice is made, AWT sends an action event to the choice menu. The event's target is the choice menu, and its object is the string label of the currently selected item. An application should override the action method of the choice menu or of one of its parent containers in order to cause some action to occur.

```
public'‘class'‘java.awt.Choice extends'‘java.awt.Component'‘
          {
                    //        Constructors
                    public Choice();
                    /*        Creates a new choice menu. The menu initially has no items in it. By
default, the first item added to the choice menu becomes the selected item, until a different selection is
made by the user or by calling one of the select methods.
                    */
                    //        Methods
                    public void addItem(String'‘item);
                    public void addNotify();
                    public int countItems();
                    public String getItem(int'‘index);
                    public int getSelectedIndex();
                    public String getSelectedItem();
                    protected String paramString();
                    public void select(int'‘pos);
```

```
                    public void select(String’‘str);
          }
```

Table 4.8

Method	Description
addItem(String’‘item)	Adds an item to this choice menu. **Throws :** `NullPointerException`, if the item's value is equal to null
addNotify()	This method calls the createChoice method of this object's toolkit in order to create a ChoicePeer for this button. This peer allows the application to change the look of a choice mneu without changing its functionality. Most applications do not call this method directly.
countItems()	**Returns:** the number of menu items in this choice menu.
getItem(int’‘index)	Returns: the string at the specified index in this choice menu.
getSelectedIndex()	**Returns:** the index of the currently selected item in this choice menu.
getSelectedItem()	**Returns:** a string representation of the currently selected item in this choice menu.
paramString()	Returns the parameter string representing the state of this choice menu. This string is useful for debugging. **Returns:** the parameter string of this choice menu.
select(int’‘pos)	Sets the selected item in this choice menu to be the item at the specified position. **Throws :** `IllegalArgumentException`,’‘if the choice item position is invalid.
select(String’‘str)	Sets the selected item in this choice menu to be the choice whose name is equal to the specified string. If more than one choice is equal to the specified string, the one with the smallest index whose name matches is selected.

Class Color

This class encapsulate colors using the RGB format. In this format, the red, blue, and green components of a color a each represented by an integer in the range 0-255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component.

```
public’‘final’‘class’‘java.awt.Color extends’‘java.lang.Object
{
                    //        Member elements
                    public final static Color black;
```

```
                    public final static Color blue;
                    public final static Color cyan;
                    public final static Color darkGray;
                    public final static Color gray;
                    public final static Color green;
                    public final static Color lightGray;
                    public final static Color magenta;
                    public final static Color orange;
                    public final static Color pink;
                    public final static Color red;
                    public final static Color white;
                    public final static Color yellow;


                    //        Constructors
                    public Color(float‘‘r, float‘‘g, float‘‘b);
                    /*        Creates a color with the specified red, green, and blue values, where each
of the values is in the range 0.0-1.0. The value 0.0 indicates no contribution from the primary color
component. The value 1.0 indicates the maximum intensity of the primary color component. The actual color
used in rendering depends on finding the best match given the color space available for a given output
device.
                    */
                    public Color(int‘‘rgb);
                    /*        Creates a color with the the specified RGB value, where the red component
is in bits 16-23 of the argument, the green component is in bits 8-15, of the argument, and the blue
component is in bits 0-7. The value 0 indicates no contribution from the primary color component.
                    */
                    public Color(int‘‘r, int‘‘g, int‘‘b);
                    /*        Creates a color with the specified red, green, and blue components. The
three arguments must each be in the range 0-255.    ‘‘
                    */
                    //        Methods
                    public Color brighter();
                    public Color darker();
                    public boolean equals(Object‘‘obj);
                    public int getBlue();
                    public static Color getColor(String‘‘nm);
                    public static Color getColor(String‘‘nm, Color‘‘v);
                    public static Color getColor(String‘‘nm, int‘‘v);
                    public int getGreen();
                    public static Color getHSBColor(float‘‘h, float‘‘s, float‘‘b);
                    public int getRed();
                    public int getRGB();
                    public int hashCode();
```

```
                public static int HSBtoRGB(float‘‘hue, float‘‘saturation, float brightness);


                public static float[] RGBtoHSB(int‘‘r, int‘‘g, int‘‘b, float‘‘hsbvals[]);

                ‘‘

                public String toString();
        }
```

Table 4.9

Method	Description
brighter()	**Returns:** a brighter version of this color.
darker()	**Returns:** a darker version of this color.
equals(Object‘‘obj)	The result is true if and only if the argument is not null and is a Color object that has the same red, green, and blue value as this object. **Returns:** true if the objects are the same; false otherwise.
getBlue()	Returns the blue component of this color. The result is in the range 0 to 255.
getColor(String‘‘nm)	Finds a color in the system properties.The first argument is treated as the name of a system property to be obtained as if by the method System.getProperty . The string value of this property is then interpreted as an integer value (see Integer.getInteger for information on how the string value is interpreted as an integer). This integer is then converted to a color by using the Color constructor that takes one integer argument If the specified property is not found, or could not be parsed as an integer, then null is returned.
getColor(String‘‘nm,Color‘‘v)	Finds a color in the system properties.‘‘ The first argument is treated as the name of a system property to be obtained as if by the method `System.getProperty.The string value of this property is then interpreted as an integer value. This integer is then converted to a color by using the color constructor that takes one integer argument.If the specified property is not found, or could not be parsed as an integer, then the color specified by the second argument is returned instead.`
getColor(String‘‘nm,‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘int‘‘v)	Finds a color in the system properties. The first argument is treated as the name of a system property to be obtained.‘‘If the specified property is not found, or could not be parsed as an integer, then the integer value `v` is used instead.‘‘ This integer is then converted to a color by using the Color constructor that takes one integer argument.‘‘
getGreen()	Returns the green component of this color. The result is in the

	range 0 to 255.
getHSBColor(float‘‘h, float‘‘s, float‘‘b)	Determines the hue, saturation, and brightness of a color. Each of the three components should each be a floating point number in the range
getRed()	Returns the red component of this color. The result is in the range 0 to 255.
getRGB()	

Chapter 16

The AWT Image Package

by Debasis Samanta

CONTENETS

- Introduction
- Class ColorModel
- Class CropImageFilter
- Class DirectColorModel
- Class FilteredImageSource
- Class ImageFilter
- Class IndexColorModel
- Class MemoryImageSource
- Class PixelGrabber
- Class RGBImageFilter
- Interface ImageConsumer
- Interface ImageObserver
- Interface ImageProducer

Introduction

The java.awt.image Package contains classes and interfaces for performing sophisticated image processing. The classes in this package provides for managaing image data, color models, cropping, color filtering, setting pixel values, and grabbing snapshots. The various component in this package is listed below :

```
        Classes
                  ‘‘Class ColorModel
                  ‘‘Class CropImageFilter
                  ‘‘Class DirectColorModel
```

```
                    ‘‘Class FilteredImageSource
                    ‘‘Class ImageFilter
                    ‘‘Class IndexColorModel
                    ‘‘Class MemoryImageSource
                    ‘‘Class PixelGrabber
                    ‘‘Class RGBImageFilter

          Interfaces
                    ‘‘Interface ImageConsumer
                    ‘‘Interface ImageObserver
                    ‘‘Interface ImageProducer
```

Class ColorModel

This abstract class is the superclass for all classes that encapsulate methods for translating from pixel values to their alpha (transparency), red, green, and blue components. The java.awt.image classes IndexColorModel and DirectColorModel are subclasses of this class.

```
          public’‘abstract’‘class’‘java.awt.image.ColorModel extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Member elements
‘‘‘‘      protected int pixel_bits;
                    //        The number of bits per pixel.

‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public ColorModel(int’‘bits);
                    //        Constructs a ColorModel which describes a pixel with the specified number
of bits.

‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract int getAlpha(int’‘pixel);
‘‘‘‘      public abstract int getBlue(int’‘pixel);
‘‘‘‘      public abstract int getGreen(int’‘pixel);
‘‘‘‘      public int getPixelSize();
‘‘‘‘      public abstract int getRed(int’‘pixel);
‘‘‘‘      public int getRGB(int’‘pixel);
‘‘‘‘      public static ColorModel getRGBdefault();
          }
```

Table 5.1

Methods	Description

getAlpha(int pixel)	Determines the alpha transparency of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates that the pixel is completely transparent. The value 255 indicates that the pixel is opaque.
getBlue(int pixel)	Determines the blue component of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component **Returns:** the blue color component represented by the pixel value.
getGreen(int pixel)	Determines the green component of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component**.** **Returns:** The green color component ranging from 0 to 255.
getPixelSize()	**Returns:** the number of bits per pixel in this color model.
getRed(int pixel)	Determines the red component of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component. **Returns:** the red color component ranging from 0 to 255.

Table 5.1 (contd.)

getRGB(int pixel)	Calculates a single integer representing the alpha, red, green, and blue components of a pixel in this color model. The components are each scaled to be a value between 0 and 255 . The integer returned is the number such that bits 24-31 are the alpha value, 16-23 are the red value, bits 8-15 are the green value, and bits 0-7 are the blue value.105 **Returns:** an integer representing this color in RGB format.
getRGBdefault()	Returns the default Abstract Window Toolkit color model. The Abstract Window Toolkit represents each pixel as a 32-bit integer. Bits 24-31 are the alpha transparency, bits 16-23 are the red value, bits 8- 15 are the green value, and bits 0-7 are the blue value. This method returns a `ColorModel` object which describes that pixel format and can be used to extract alpha, red, green, and blue values from such color values.

Class CropImageFilter

The cropped image filter is an image filter for cropping images. This class extends the basic ImageFilter class to extract a given rectangular region of an existing image and provides a source for new image containing only the extracted region.

This class is meant to be used in conjunction with a FilteredImageSource to produce cropped versions of existing images.

```
          public‘‘class’‘java.awt.image.CropImageFilter extends’‘java.awt.image.ImageFilter

          {

‘‘‘‘‘‘‘‘  //        Constructors

‘‘‘‘      public CropImageFilter(int’‘x, int’‘y, int’‘w, int h);

                    /*        Constructs a cropped image filter that extracts the absolute rectangular
region of pixels from its source Image as specified by the x, y, w, and h parameters.

                    */

‘‘‘‘‘‘‘‘  //        Methods

‘‘‘‘      public void setDimensions(int’‘w, int’‘h);

‘‘‘‘      public void setPixels(int’‘x, int’‘y, int’‘w, int’‘h, ColorModel’‘model,byte’‘pixels[],
int’‘off, int’‘scansize);

‘‘‘‘      ‘‘        public void setProperties(Hashtable’‘props);

          }

                    Methods are stated in Table 5.2.
```

Table 5.2

Methods	Description
setDimensions(int w, int h)	The image producer calls the `setDimensions` of the image consumer to tell it the width and height of the image. The setDimensions method of `CroppedImageFilter` ignores its arguments. It calls the `setDimensions` method of its image consumer with the width and height arguments of the constructor106106.
setPixels(....)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. The `setPixels` method of `CroppedImageFilter` determines if the specified rectangle intersects its cropping region. If so, it calls the `setPixels` method after modifying the x, y, w, h, and offset arguments to reflect only the intersecting region.
setProperties(Hashtable props)	The image producer calls the `setProperties` method of the image consumer to let it know of additional properties of the image. The `setProperties` method of `CroppedImageFilter` adds the property "croprect" with the value = new `Rectangle`(x, y, width, height) to the table of properties, and then calls the `setProperties` method of its image consumer with the modified properties table.

Class DirectColorModel

The DirectColorModel is a color model which specifies a translation from pixel values to alpha, red, green, and blue components using the actual bits of the pixel value.

Many of the methods in this class are final : the underlying native graphics code makes assumptions about the layout and operation of this class and those assumptions are reflected in the implementations of the methods here that are

marked final. Applications can subclass this class for other reaons, but they cannot override or modify the behavior of the final methods.

```
          public''class''java.awt.image.DirectColorModel extends''java.awt.image.ColorModel''
          {
                                //        Constructors
                     public DirectColorModel(int''bits, int''rmask, int gmask, int''bmask);
                     /*        Constructs a direct color model in which each of the given masks specify
which bits in the pixels contain the red, green, and blue components.
                                Pixels described by this color model all have alpha components of 255,
indicating that they are fully opaque.
                     */
                     public DirectColorModel(int''bits, int''rmask, int gmask, int''bmask,int''amask);
                     /*        Constructs a direct color model in which each of the given masks specify
which bits in the pixels contain the alpha, red, green, and blue components.                */


                                //        Methods
                     public final int getAlpha(int''pixel);
                     public final int getAlphaMask();
                     public final int getBlue(int''pixel);
                     public final int getBlueMask();
                     public final int getGreen(int''pixel);
                     public final int getGreenMask();
                     public final int getRed(int''pixel);
                     public final int getRedMask();
                     public final int getRGB(int''pixel);
          }


          Methods are stated in Table 5.3.
```

Table 5.3

Methods	Description
getAlpha(int pixel)	Determines the alpha transparency of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates that the pixel is completely transparent. The value 255 indicates that the pixel is opaque. **Returns:** the alpha transparency represented by the pixel value.
getAlphaMask()	**Returns:** a mask indicating which bits in a pixel contain the alpha transparency component in this color model.
getBlue(int pixel)	Determines the blue component of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component.

	Returns: the blue color component represented by the pixel value.
getBlueMask()	**Returns:** a mask indicating which bits in a pixel contain the blue color component in this color model.
getGreen(int pixel)	Determines the green component of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component. **Returns:** the blue color component represented by the pixel value.
getGreenMask()	**Returns:** a mask indicating which bits in a pixel contain the green color component in this color model.
getRed(int pixel)	Determines the red component of a pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component. **Returns:** the red color component represented by the pixel value.
getRedMask()	**Returns:** a mask indicating which bits in a pixel contain the red color component in this color model.
getRGB(int pixel)	Calculates a single integer representing the alpha, red, green, and blue components of the pixel in this color model. The components are each scaled to be a value between 0 and 255. The integer returned is the number such that bits 24-31 are the alpha value, 16-23 are the red value, bits 8-15 are the green value, and bits 0-7 are the blue value. This format is the pixel format of the default RGB colormodel. **Returns:** an integer representing this color in RGB format.

Class FilteredImageSource

A FilteredImagedSource is an implementation of the image producer interface which takes an existing image and an image filter and uses them to produce a new filtered version of the original image.

```
          public‘‘class’‘java.awt.image.FilteredImageSource extends’‘java.lang.Object’‘
‘‘‘‘implements java.awt.image.ImageProducer’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public FilteredImageSource(ImageProducer’‘orig, ImageFilter’‘imgf);
                    //        Constructs a new image producer from an existing image producer and an an
image filter.

‘‘‘‘‘‘‘‘ //        Methods
```

```
‘‘‘‘      public void addConsumer(ImageConsumer’‘ic);
‘‘‘‘      public boolean isConsumer(ImageConsumer’‘ic);
‘‘‘‘      public void removeConsumer(ImageConsumer’‘ic);
‘‘‘‘      public void requestTopDownLeftRightResend(ImageConsumer’‘ic);
‘‘‘‘      public void startProduction(ImageConsumer’‘ic);
          }

          Methods are stated in Table 5.4
```

Table 5.4

Methods	Description
addConsumer(ImageConsumer ic)	Registers the image consumer argument as wanting th image produced by this image producer. The `addConsumer` method of `FilteredImageSource` calls its image filter's `getFilterInstance` method to creates a new filter instance for the image consumer argument. The resulting filter instance is then passed as an argument to the image producer's `addConsumer` method.
isConsumer(ImageConsumer ic)	**Returns:** true if the specified image consumer argument *is currently registered with this image producer as one of its consumers; false otherwise.*
removeConsumer(ImageConsumer ic)	Removes the specified image consumer object from the list of consumers registered to receive the image data from this image producer. The `removeConsumer` method of `FilteredImageSource` calls its image producer's `removeConsumer` method with the image filter than was created when this image consumer was first registered.
requestTopDownLeftRightResend*(...)*	An image consumer sends this message to request that the image producer attempt to resend the image data one more time in top-down, left-to-right order. 109This method of `FilteredImageSource` calls its image producer's `requestTopDownLeftRightResent` method with the image filter than was created when this image consumer was first registered.
startProduction(ImageConsumer ic)	Registers the image consumer argument as wanting the image produced by this image producer. In addition, this method forces the image producer to start an immediate reconstruction of the image data. The `startProduction` method of `FilteredImageSource` calls its image filter's `getFilterInstance` method to creates a new filter instance for the image consumer argument. The resulting filter instance is then passed as an argument to the image producer's `startProduction` method.

Class ImageFilter

This class is the superclass of all classes that are meant to filter the data delivered from an ImageProducer to an ImageConsumer. This class and its subclasses are meant to be used in conjunction with a FilteredImageSource object to produce filtered versions of existing images.

The image filter implemented by this class is the "null filter" which has no effect on the data passing through. Filters should subclass this class and override the methods in order to modify the data as necessary.

```
          public’‘class’‘java.awt.image.ImageFilter extends’‘java.lang.Object’‘

          implements java.awt.image.ImageConsumer’‘

                                          java.lang.Cloneable’‘

          {

                              //        Member elements

                    protected ImageConsumer consumer;

                    /*        The image consumer’‘ of the particular image data stream for which this
image filter is filtering data. The field is not initialized by the constructor, but by the call to the
getFilterInstance method. when the FilteredImageSource’‘ is creating a unique instance of this object for
a particular image data stream.

                    */

                              //        Constructors

                    public ImageFilter();

                    //        The default constructor for this method.


                              //        Methods

                    public Object clone();

                    public ImageFilter getFilterInstance(ImageConsumer’‘ic)

                    public void imageComplete(int’‘status);

                    public void resendTopDownLeftRight(ImageProducer’‘ip);

                    public void setColorModel(ColorModel’‘model);

                    public void setDimensions(int’‘width, int’‘height);

                    public void setHints(int’‘hints);

                    public void setPixels(int’‘x, int’‘y, int’‘w, int’‘h,
ColorModel’‘model,byte’‘pixels[], int’‘off, int’‘scansize);

                    public void setPixels(int’‘x, int’‘y, int’‘w, int’‘h,
ColorModel’‘model,int’‘pixels[], int’‘off, int’‘scansize);

                    public void setProperties(Hashtable’‘props);

          }


          Methods are described in Table 5.5.
```

Table 5.5

Methods	Description
clone()	**Returns:** a clone of this object. .
getFilterInstance(ImageConsumer ic)	Creates an image filter which filters the image for the specified image consumer. The `getFilterInstance` method of `ImageFilter` clones the object. and sets its

	consumer field to the image consumer argument. Subclasses can override this method, if they require more setup than this.
imageComplete(int status)	The image producer calls the `imageComplete` method when it has completed an image or it has encountered an error in producing or loading the image. This method of `ImageFilter` calls the `imageComplete` method of the image consumer with the identical argument.
resendTopDownLeftRight (ImageProducer ip)	An image consumer calls `the resendTopDownLeftRight` of its producer to request that the pixel data be resent in that order. An image filter can respond to this request in one of three ways: The filter can forward this request to the image producer using itself as the requesting image consumer. This is the default behavior provided by the `resendTopDownLeftRight` method of ImageFilter. The filter can resend the pixels in the right order on its own (presumably because the generated pixels have been saved in some sort of buffer). The filter can ignore this request.
1. **setColorModel**(ColorModel model)	The image producer calls the `setColorModel` method to specify the color model for the majority of the subsequent setPixels method calls.. The `setColorModel` method of `ImageFilter` calls the `setColorModel` method of the image consumer with the identical argument.
setDimensions(int width, int height)	The image producer calls the `setDimensions` of the image consumer to tell it the width and height of the image.The`setDimensions` method of `ImageFilter` calls the `setDimensions` method of its image consumer with the identical arguments.
setHints(int hints)	The image producer calls the `setHints` method of the image consumer to indicate the order in which the bits are to be delivered. The `setHints` method of `ImageFilter` calls the setHints method of its image consumer with the identical argument.
setPixels(..byte pixels[]..)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. The `setPixels` method of `ImageFilter` calls the `setPixels` method of its image consumer with the identical arguments.
setPixels(..int pixels[],.)	The image producer calls the setPixels method of the image consumer one or more times to deliver the pixels of the image. The `setPixels` method of ImageFilter calls the setPixels method of its image consumer with the identical arguments.

setProperties(Hashtable props)	The `setProperties` method of `ImageFilter` calls the `setProperties` method of its image consumer with the properties argument after modifying it slightly: If the hashtable already has no key "filters", a string representation of the current filter is added to the hashtable under the key "filter". If such a key is already in the hashtable, a string representation of the current filter is appended to the property (which must be a String) already stored under that key, and stores this value back in the hashtable.

Class IndexColorModel

The index color model class specifies a color model in a which a pixel value is converted into alpha, red, green, and blue component by using the pixel value as an index into a color map. Each entry in the color map gives the alpha, red, green, and blue component for the corresponding pixel. An optional transparent pixel can be specified. This pixel is completely transparent, independent of the alpha value recorded for that pixel value. The maximum size of the color map is 256 entries. This color model is similar to an X11 PseudoColor visual..

Many of the methods in this class are final: the underlying native graphics code makes assumptions about the layout and operation of this class and those assumptions are reflected in the implementations of the methods here that are marked final. Applications can subclass this class for other reaons, but they cannot override or modify the behavior of the final methods.

```
          public‘‘class’‘java.awt.image.IndexColorModel extends’‘java.awt.image.ColorModel’‘

          {

‘‘‘‘‘‘‘‘ //        Constructors

‘‘‘‘      public IndexColorModel(int’‘bits, int’‘size, byte’‘r[], byte’‘g[],byte’‘b[]);

                    /*        Constructs an IndexColorModel from the given arrays of red, green, and blue
components.

                              Pixels described by this color model all have alpha components of 255
(fully opaque).

                              Each of the three arrays must have at least size elements, and it must be
the case that size < 2bits . These first size elements of the arrays are the red, green, and blue values
for pixels in the range 0 <= i < size . Pixels in the range size <= i <= 2bits’‘have red, green, and blue
values of’‘0.

                    */

‘‘        public IndexColorModel(int’‘bits, int’‘size, byte’‘r[], byte’‘g[],byte’‘b[], byte’‘a[]);

                    //        Constructs an IndexColorModel from the given arrays of red, green, blue and
alpha components.

‘‘        public IndexColorModel(int’‘bits, int’‘size, byte’‘r[], byte’‘g[],byte’‘b[], int’‘trans);

                    /*        Constructs an IndexColorModel from the given arrays of red, green, and blue
components. Pixels described by this color model all have alpha components of 255 (fully opaque), except
for the transparent pixel.

                    */

‘‘        public IndexColorModel(int’‘bits, int’‘size, byte’‘cmap[],int’‘start,boolean’‘hasalpha);

                    /*        Constructs an index color model from a single array of packed red, green,
blue and optional alpha components. If the hasalpha argument is false, then the cmap array must have
length of at least start + 3 x size. The red, green, and blue components for a pixel i in the range 0
<=‘‘i < size are in the three elements of the cmap array starting at cmap[start + 3i] Its alpha component
is 255 (fully opaque). If the hasalpha argument is true, then the cmap array must have length of at least
```

```
start + 4 x size. The red, green, blue, and alpha components for a pixel i in the range'' 0 <=''i < size
are in the four elements of the cmap array starting at cmap[start + 4i]. Pixels in the range''size <=i <
2bits have red, green, and blue values of 0, Their alpha component is 0 if hasalpha is true, and 255
otherwise.

                    */

''        public IndexColorModel(int''bits, int''size, byte''cmap[], int''start,boolean''hasalpha,
int''trans);

                    /*        Constructs an index color model from a single array of packed red, green,
blue, and optional alpha values.

                              The color model is constructed the same way as described in''. In addition,
the specified transparent index represents a pixel which is considered entirely transparent regardless of
any alpha value specified for it. The array must have enough values in it to fill all of the needed
component arrays of the specified size.

                    */

''''''''  //        Methods

                    public final int getAlpha(int''pixel);

                    public final void getAlphas(byte''a[]);

                    public final int getBlue(int''pixel);

                    public final void getBlues(byte''b[]);

                    public final int getGreen(int''pixel);

                    public final void getGreens(byte''g[]);

                    public final int getMapSize();

                    public final int getRed(int''pixel);

                    public final void getReds(byte''r[]);

                    public final int getRGB(int''pixel);

                    public final int getTransparentPixel();

          }
```

Methods are described in Table 5.6.

Table 5.6.

Methods	Description
getAlpha(int pixel)	Determines the alpha transparency value of the pixel in this color model. The value ranges from 0 to 255. The value 0 indicates that the pixel is completely transparent. The value 255 indicates that the pixel is opaque.
getAlphas(byte a[])	Copies the array of alpha transparency values from this color model into the given array. Only the initial entries of the array as specified by the `getMapSize` method are written.
getBlue(int pixel)	Determines the blue component of the pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component**.** **Returns:** the blue color component represented by the pixel value.

getBlues(byte b[])	Copies the array of blue color components from this color model into the given array. Only the initial entries of the array as specified by the getMapSize method are written.
getGreen(int pixel)	Determines the green component of the pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component. **Returns :** the blue color component represented by the pixel value. .
getGreens(byte g[])	Copies the array of green color components from this color model into the given array. Only the initial entries of the array as specified by the getMapSize method are written.
getMapSize()	**Returns:** the value of the size argument when creating this index color model.
getRed(int pixel)	Determines the red component of the pixel in this color model. The value ranges from 0 to 255. The value 0 indicates no contribution from this primary color. The value 255 indicates the maximum intensity of this color component. **Returns:** the red color component represented by the pixel value.
getReds(byte r[])	Copies the array of red color components from this color model into the given array. Only the initial entries of the array as specified by the getMapSize method are written.
getRGB(int pixel)	Calculates a single integer representing the alpha, red, green, and blue components of the pixel in this color model. The components are each scaled to be a value between 0 and 255 . The integer returned is the number such that bits 24-31 are the alpha value, 16-23 are the red value, bits 8-15 are the green value, and bits 0-7 are the blue value.114 This format is the format used by the default RGB color model. **Returns:** an integer representing this color in RGB format.
getTransparentPixel()	**Returns:** the index of the transparent pixel in this color model, or -1 if there is no transparent pixel.
getRGB(int pixel)	Calculates a single integer representing the alpha, red, green, and blue components of the pixel in this color model. The components are each scaled to be a value between 0 and 255 . The integer returned is the number such that bits 24-31 are the alpha value, 16-23 are the red value, bits 8-15 are the green value, and bits 0-7 are the blue value. This format is the format used by the default RGB color model. **Returns:** an integer representing this color in RGB format.
getTransparentPixel()	**Returns:** the index of the transparent pixel in this color model,

	or -1 if there is no transparent pixel.

Class MemoryImageSource

A memory image source is an implementation of the image producer interface. It uses an array to produce pixel values for the image. An application can use the method createImage in class Component to create an Image from a MemoryImageSource.

```
          public‘‘class’‘java.awt.image.MemoryImageSource extends’‘java.lang.Object’‘

          implements java.awt.image.ImageProducer’‘

          {

                              //        Constructors

                    public MemoryImageSource(int’‘w, int’‘h, ColorModel cm,
byte’‘pix[],int’‘off,’‘int’‘scan);

                    /*        Constructs an image producer object which uses an array of bytes to produce
data for the image object. The pixel at coordinate’‘(i, j)’‘is stored in the pixel array at index (j x
scan +i + offset).

                    */

                    public MemoryImageSource(int’‘w, int’‘h, ColorModel cm,
byte’‘pix[],int’‘off,’‘int’‘scan,’‘Hashtable’‘props);

                    /*        Constructs an image producer’‘ object which uses an array of bytes to
produce data for the image object. The pixel at coordinate (i, j) is stored in the pixel array at index (j
x scan +i + offset).In addition, the image has the properties indicated in the hash table argument.

                    */

                    public MemoryImageSource(int’‘w, int’‘h, ColorModel cm, int’‘pix[], /*   Constructs
an image producer’‘ object which uses an array of integers to produce data for the image object. The pixel
at coordinate’‘(i, j) is stored in the pixel array at index (j x scan +i + offset).

                    */

                    public MemoryImageSource(int’‘w, int’‘h, ColorModel cm,       ‘‘int’‘pix[],int’‘off,
int scan, Hashtable’‘props);

                    /*        Constructs an image producer object which uses an array of integers to
produce data for the image object. The pixel at coordinate (i, j)’‘ is stored in the pixel array at index
(j x scan +i + offset).In addition, the image has the properties indicated in the hash table argument.

                    */

                    public MemoryImageSource(int’‘w, int’‘h, int pix[], int off, int’‘scan);

                    /*        Constructs an image producer’‘object which uses an array of integers to
produce data for the image object. The pixel at coordinate’‘( i, j) is stored in the pixel array at index
(j x scan +i + offset).The resulting image uses the default RGB color model.

                    */

                    public MemoryImageSource(int’‘w, int’‘h, int pix[],int’‘off,int scan,
Hashtable’‘props);

                    /*        Constructs an image producer’‘object which uses an array of integers to
produce data for the image object. The pixel at coordinate (i, j)’‘is stored in the pixel array at index
(j x scan +i + offset). The resulting image uses the default RGB color model.In addition, the image has
the properties indicated in the hash table argument.

                    */

                    /*        Note : The parameters in the above constructors :

                    w- the width of the image’‘
```

```
                h - the height of the image’‘
                cm - the color model used the pixels’‘
                pix - the array of pixel values’‘
                off - the offset of the first pixel in the array
                scan - the number of pixels in the array per line
                props - a hash table of properties
        */
                //        Methods
                public void addConsumer(ImageConsumer’‘ic);
                public boolean isConsumer(ImageConsumer’‘ic);
                ublic void removeConsumer(ImageConsumer’‘ic);
                public void          requestTopDownLeftRightResend(ImageConsumer’‘ic);
                public void ctstartProduion(ImageConsumer’‘ic);
        }
        The methods are stated in Table 5.7.
```

Table 5.7

Methods	Description
addConsumer(ImageConsumer ic)	Registers the image consumer argument as wanting the image produced by this image producer. The `addConsumer` method of `MemoryImageSourc`e, since it already has the image data, immediately delivers the data to the image consumer.
isConsumer(ImageConsumer ic)	Determines if the image consumer argument is registered with this image producer as a consumer. Because the memory image source delivers data immediately to its image consumer, the memory image source can have at most one consumer at a time. **Returns:** true if the specified image consumer argument is currently registered with this image producer as one of its consumers; false otherwise.
removeConsumer(ImageConsumer ic)	Removes the specified image consumer object from the list of consumers registered to receive the image data.
requestTopDownLeftRightResend(ImageConsumer ic)	An image consumer sends this message to request that the image producer attempt to resend the image data one more time in top-down, left-to-right order. The `requestTopDownLeftRightResend` method of memoryImageSource ignores this request, since it always sends its data in that order.
startProduction(ImageConsumer ic)	Registers the image consumer argument as wanting the image produced by this image producer. In addition, this method forces the image producer to start an immediate reconstruction of the image

	data. The `addConsumer` method of `MemoryImageSource`, since it already has the image data, immediately delivers the data to the image consumer.

Class PixelGrabber

The pixel grabber implements an image consumer which can be attached to an image or image producer object to retrieve a subset of the pixels in that image. Pixels are stored in the array in the default RGB color model. Most applications need to call only the grabPixel methods and the status method of this class. The remaining methods are part of the ImageConsumer interface and allow the pixel grabber to receive the image from the image producer.

```
          public‘‘class’‘java.awt.image.PixelGrabber extends’‘java.lang.Object’‘

          implements java.awt.image.ImageConsumer’‘

          {

                              //        Constructors

                    public PixelGrabber(Image’‘img, int’‘x, int’‘y, int’‘w, int’‘h, int’‘pix[],int’‘off,
int’‘scansize);

                    /*        Creates a new pixel grabber object to grab the rectangular section of
pixels from the specified image into the specified array. The pixels are stored in the array in the
default RGB color model. The pixel data for the coordinate (i, j), where (i, j) is inside the indicted
rectangle, is stored in the array at index

                              (j - scan) + (i-x) + offset of the pixel array. The x and y coordinates
indicate the upper left corner of the rectangle of pixels to retrieve from the image, relative to the
default (unscaled) size of the image.

                    */



                    public PixelGrabber(ImageProducer’‘ip, int’‘x, int’‘y, int’‘w, int’‘h, int’‘pix[],
int’‘off, int’‘scansize);

                    /*        Creates a new pixel grabber object to grab the rectangular section of
pixels from the specified image producer into the specified array. The pixels are stored in the array in
the default RGB color model. The pixel data for the coordinate (i,j), where’‘(i, j) is inside the indicted
rectangle, is stored in the array at the index’‘

                              (j - scan) + (i-x) + offset of the pixel array. The x and y coordinates
indicate the upper left corner of the rectangle of pixels to retrieve from the image, relative to the
default (unscaled) size of the image.

                    /*        Note : The various parameters used in these constructors are :

                                        img - the image from which to retrieve pixels’‘

                              x - the x coordinate of the upper left corner’‘

                              y - the y coordinate of the upper left corner’‘

                              w - the width of the rectangle to retrieve’‘

                              h - the height of the rectangle to retrieve’‘

                              pix - the array of integers into which to place the RGB pixels retrieved
from the image’‘

                              scansize - the distance from the start of one row of pixels to the start
of’‘the next row in the array.

                    */

                              //        Methods
```

```
                    public boolean grabPixels();
                    public boolean grabPixels(long'‘ms);
                    public void imageComplete(int'‘status);
                    public void setColorModel(ColorModel'‘model);
                    public void setDimensions(int'‘width, int'‘height)
                    public void setHints(int'‘hints);
                    public void setPixels(int'‘srcX, int'‘srcY, int'‘srcW, int'‘srcH,
                    ColorModel'‘model, byte'‘pixels[], int'‘srcOff, int'‘srcScan);
                    public void setPixels(int'‘srcX, int'‘srcY, int'‘srcW,
int'‘srcH,ColorModel'‘model,'‘int'‘pixels[], int'‘srcOff,'‘int'‘srcScan);
                    public void setProperties(Hashtable'‘props);
                    public int status();
          }


          Methods are described in Table 5.8.
```

Table 5.8'‘

Methods	Description
grabPixels()	Requests the image or image producer to start delivering pixels to this image consumer. It waits for all of the pixels in the rectangle of interest to be delivered. **Returns:** true if the pixels were successfully grabbed; false on abort or error. **Throws** : `InterruptedException`, If another thread has interrupted this thread.
grabPixels(long ms)	Requests the image or image producer to start delivering pixels to this image consumer. It waits for all of the pixels in the rectangle of interest to be delivered, or until the specified timeout has elapsed. **Returns:** true if the pixels were successfully grabbed; false on abort, error or timeout.**Throws :** InterruptedException , If another thread has interrupted this thread.
imageComplete(int status)	The image producer calls the imageComplete method when it has completed an image or it has errored in producing or loading the image. The `imageComplete` method of `PixelGrabber` notifies all processes waiting for the pixels to wake up. It uses the value of the status flag to determine whether the image was successfully retrieved or not.
setColorModel(ColorModel model)	The image producer calls the `setColorMode`l method to specify the color model for the majority of the subsequent `setPixel`s method calls. The `setColorMode`l method of `PixelGrabber` ignores this call.

setDimensions(int width, int height)	The image producer calls the setHints method of the image consumer to indicate the order in which the bits will be delivered.
setHints(int hints)	The image producer calls the `setHints` method of the image consumer to indicate the order in which the bits will be delivered. The `setHints` method of `PixelGrabber` ignores the hints.
setPixels(.,byte pixels[],.)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. The `setPixels` method of `PixelGrabber` places the bits, if appropriate, into the array of bits passed to it by a call to the `grabPixels` method.
setPixels(..int pixels[],..)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. The `setPixels` method of `PixelGrabber` places the bits, if appropriate, into the array of bits passed to it by a call to the `grabPixels` method.
setProperties(Hashtable props)	The image producer calls the `setProperties` method of the image consumer to let it know of additional properties of the image. The 119`setProperties` method of `PixelGrabber` ignores the hints.
status()	Returns the bitwise OR of the appropriate ImageObserver interface flag.

Class RGBImageFilter

This class provides an easy way to create an image filter which modifies the pixels of the original image by converting them one at a time in the default RGB color model. Objects of this class are meant to be used in conjunction with a filtered image source object to produce filtered versions of existing images.

This class is an abstract class. It provides the calls needed to channel all the pixel data through a single method which converts pixels, one at a time, into the default RGB color model, regardless of the color model being used by the image producer. The only method which needs to be defined to create a useable image filter is the filterRGB method.

```
          public‘‘abstract’‘class’‘java.awt.image.RGBImageFilter extends’‘java.awt.image.ImageFilter

          {

‘‘‘‘‘‘‘‘ //        Member elements

‘‘‘‘      protected boolean canFilterIndexColorModel;

                    /*        Setting this value to true indicates that the the value returned by the
filterRGB method’‘is independent of the x and y arguments, and depends only on the rgb argument.
Subclasses of’‘RGBImageFilter should set this field to true in their constructor if their filterRGB method
does not depend on the coordinate of the pixel being filtered. Filtering the colormap entries of an
indexed color map can be much faster than filtering every pixel. The default value is false.

                    */

‘‘‘‘      protected ColorModel newmodel;

                    //        This field is used to remember the newcm argument passed to the
substituteColorModel method.
```

```
‘‘‘‘      protected ColorModel origmodel;
                    //        This field is used to remember the oldcm argument passed to the
substituteColorModel method.


‘‘‘‘‘‘‘‘  //        Constructors
‘‘‘‘      public RGBImageFilter();
                    //        The default constructor.
‘‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public IndexColorModel filterIndexColorModel(IndexColorModel’‘icm);
‘‘‘‘      public abstract int filterRGB(int’‘x, int’‘y, int’‘rgb);
‘‘‘‘      public void filterRGBPixels(int’‘x, int’‘y, int’‘w, int’‘h, int’‘pixels[],int’‘off,
int’‘scansize);
                    public void setColorModel(ColorModel’‘model);


‘‘‘‘      public void setPixels(int’‘x, int’‘y, int’‘w, int’‘h, ColorModel’‘model,byte’‘pixels[],
;int’‘off, int’‘scansize);
‘‘‘‘      public void setPixels(int’‘x, int’‘y, int’‘w, int’‘h ColorModel’‘model,int’‘pixels[], int’‘off,
int’‘scansize);
‘‘‘‘      public void substituteColorModel(ColorModel’‘oldcm, ColorModel’‘newcm);
          }
          Methods in this class is described as in Table 5.9.
```

Table 5.9.

Methods	Description
filterIndexColorModel(IndexColorModel icm)	Filters an index color model object by running each entry in its color table through the `filterRGB` method . The call to `filterRGB` has the x and y arguments set to -1 as a flag to indicate that a color table entry is being filtered rather than an actual pixel value.
filterRGB(int x, int y,int rgb)	Specifies a method to convert a single input pixel, whose value is specified in the default RGB color model, to a new pixel value also in the default RGB color model. Subclasses of RGBImageFilter must provide a definition for this method. If the value of the field canFilterIndexColorModel is true, then the value returned by this method must not depend on the x and y coordinates. If the x and y arguments are both -1, this method is being called by the `filterIndexColorModel` method. **Returns** : the new value of the pixel, in the default RGB color model.
filterRGBPixels(...)	Filters a buffer of pixels in the default RGB color model by passing them one by one through the filterRGB method. The `setPixels` method of the filter's consumer is then called with the resulting buffer and the

	color model argument set to the default RGB color model. Only pixels that fall within the specified rectangle are modified. The value of the pixel at coordinate (i, j) stored in the pixel array at index (j x scan + I + offset).
setColorModel(ColorModel model)	The image producer calls the `setColorModel` method to specify the color model for the majority of the subsequent setPixels method calls. The setColorModel method of RGBImageFilter determines if the color model argument is an index color model and if the `canFilterIndexColorModel` field is true. If both conditions are true, the method creates a new color model by calling the filterIndexColorModel method on the model argument. The original color model and the newly created color model are then passed as arguments to the `substituteColorModel` method. In addition, the `setColorModel` method of the filter's consumer is called with the newly created color model. If either condition is false, the method calls the 121the `setColorModel` method of its consumer with the default RGB color map.
setPixels(..byte pixels[],..)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. The `setPixels` method of `RGBImageFilter` looks to see if the color model is the same one that has already been converted and remembered for substitution by a previous call to the `substituteColorModel` mrethod. If so, it calls the `setPixels` method of its consumer, changing the color model argument to be the alternative color model. Otherwise, the method converts the buffer of byte pixels to the default RGB color model and passes the converted buffer to the filterRGBPixels method to be converted one by one.
setPixels(..int pixels[], ..)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. The setPixels method of `RGBImageFilter` looks to see if the color model is the same one that has already been converted and remembered for substitution by a previous call to the substituteColorModel method. If so, it calls the `setPixels` method of the filter's consumer, changing the color model argument to be the alternative color model. Otherwise, the method converts the buffer of byte pixels to the default RGB color model and passes the converted buffer to the filterRGBPixels method to be converted one by one.
substituteColorModel(ColorModel oldcm, ColorModel newcm)	Registers two color model objects for substitution: If the oldcm is the color model during any subsequent call to either of the `setPixels` methods, the newcm argument is substituted and the pixels passed through unmodifed.

Interface ImageConsumer

The image consumer interface specifies the methods that all image consumers must implement. An image consumer is an object interested in data produced by the image producers. When a consumer is added to an image producer, the producer delivers all the data about the image using the method calls defined in this interface.

```
                    public‘‘interface’‘java.awt.image.ImageConsumer

                    {

                                        // status value for the imageComplete method

                    ‘‘ public final static int IMAGEABORTED;

                                /*      IMAGEABORTED = 4, Argument to the imageComplete
method’‘indicating that the image creation process was deliberately aborted.

                                */

                    ‘‘ public final static int IMAGEERROR;

                                /*      IMAGEERROR = 1, Argument to the imageComplete method’‘indicating
that an error was encountered while producing the image.

                                public final static int SINGLEFRAMEDONE;

                                /*      SINGLEFRAMEDONE = 2, Argument to the imageComplete method
indicating that one frame of the image is complete but there are more frames to be delivered.

                                */

                    ‘‘ public final static int STATICIMAGEDONE;

                                /*      STATICIMAGEDONE = 3,’‘Argument to the imageComplete
method’‘indicating that the image is complete and there are no more pixels or frames to be delivered.

                                */


                                ‘‘// hints used by the setHints method

                    ‘‘ public final static int COMPLETESCANLINES;

                                /*      COMPLETESCANLINES = 4,’‘Flag in the setHints method’‘indicating
that the pixels will be delivered in (multiples of) complete scanlines at a time.

                                */

                    ‘‘ public final static int RANDOMPIXELORDER;

                                /*      RANDOMPIXELORDER = 1,’‘Flag in the setHints method’‘indicating
that the pixels will be delivered in a random order.

                                */

                    ‘‘ public final static int SINGLEFRAME;

                                /*      SINGLEFRAME = 8, Flag in the setHints method’‘indicating that
the image contains a single static image. The pixels will be defined in calls to the setPixels methods;
the image producer then calls the imageComplete method’‘ with the STATICIMAGEDONE flag’‘after which no
more image data is delivered.

                                */


                    ‘‘ public final static int SINGLEPASS;

                                //      SINGLEPASS = 8, Flag in the setHints method’‘ indicating that
each pixel will be delivered only’‘once.

                    ‘‘ public final static int TOPDOWNLEFTRIGHT;

                                /*      TOPDOWNLEFTRIGHT = 2,Flag in the setHints method indicating that
the pixels will be delivered in a top-down, left-to-right order.
```

```
                    */



                                //        Methods
                ‘‘ public abstract void imageComplete(int’‘status);
                ‘‘ public abstract void setColorModel(ColorModel’‘model);
                ‘‘ public abstract void        setDimensions(int’‘width, int’‘height);
                ‘‘ public abstract void setHints(int’‘hintflags);
                ‘‘ public abstract void        setPixels(int’‘x, int’‘y, int’‘w, int’‘h,
                ‘‘ ColorModel’‘model, byte’‘pixels[], int’‘off, int’‘scansize);
                ‘‘ public abstract void        setPixels(int’‘x, int’‘y, int’‘w, int’‘h,
                ‘‘ ColorModel’‘model,’‘int’‘pixels[], int’‘off, int’‘scansize);
                ‘‘ public abstract void setProperties(Hashtable’‘props);
                }
                Methods are stated as in Table 5.10.
```

Table 5.10

Methods	Description
imageComplete(int status)	The image producer calls the imageComplete method when one of the following conditions has occurred : it has delivered all the pixels that the source image contains, a single frame of a multi-frame animation has been completed, an error in loading or producing the image has occurred. the image production was explicitly aborted by the application. The image consumer should remove itself from the list of consumers registered with the image producer unless it is interested in subsequent frames.
setColorModel(ColorModel model)	The image producer calls the `setColorModel` method to specify the color model for the majority of the subsequent setPixels method calls. Each set of pixels delivered using the setPixels method includes its own color model, so the image consumer should not assume that the model argument is the color model argument in every subsequent setPixels method call. A notable case where multiple `ColorModel` objects may be seen is a filtered image where for each set of pixels that it filters, the filter determines whether the pixels can be sent on untouched, using the original `ColorModel`, or should be modified (filtered) and passed on using a `ColorModel` more convenient for the filtering process.
setDimensions(int width, int height)	The image producer calls the `setDimensions` of the image consumer to indicate the width and height of the image.

setHints(int hintflags)	The image producer calls the setHints method of the image consumer to indicate the order in which the bits will be delivered. The image producer is allowed to deliver the pixels in any order, but the image consumer may be able to scale or convert the pixels more efficiently or with higher quality if it knows some information about how the pixels will be presented.The image producer should call the setHints method before any calls to the image consumer's setPixels method. The hintflags agument is a bit mask of hints about the manner in which the pixels are delivered.124
setPixels(..byte pixels[],..)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. Each call specifies the location and size of the rectangle of source pixels contained in the array of pixels.The specified color model object should be used to convert the pixels into their corresponding color and alpha components. The pixel at coordinate (i, j) is stored in the pixel array at index (j - y) x scan + (i-x) + offset. The pixels delivered using this method are all stored as bytes.
setPixels(...int pixels[],..)	The image producer calls the `setPixels` method of the image consumer one or more times to deliver the pixels of the image. Each call specifies the location and size of the rectangle of source pixels that are contained in the array of pixels. The specified color model object should be used to convert the pixels into their corresponding color and alpha components. The pixel at coordinate)(i, j) is stored in the pixel array at index (j - y) x scan + (i-x) + offset. The pixels delivered using this method are all stored as integers.
setProperties(Hashtable props)	The image producer calls the `setProperties` method of the image consumer to indicate additional properties of the image. All keys to the hash table are strings. The corresponding values depend on the string.

Interface ImageObserver

The image observer interface specifies the methods that all image observers must implement. An image observer is interested in receiving asynchronous notifications about the image as the image is being constructed.

```
‘‘ public’‘interface’‘java.awt.image.ImageObserver
          {
                    // flags for the infoflags argument to imageUpdate
                    public final static int ABORT;
                    /*        ABORT = 128 : This flag in the infoflags argument to imageUpdate’‘indicates
that the image was aborted before production was complete.
                    */
                    public final static int ALLBITS;
```

```
                    /*        ALLBITS = 32 : This flag in the infoflags argument to imageUpdate'‘
indicates that a static image is now complete and can be drawn in its final form. The x, y, width, and
height arguments to the imageUpdate method should be ignored when this flag is set in the status

                    */

                    public final static int ERROR;

                    /*        ERROR = 64 : This flag in the infoflags argument to imageUpdate'‘indicates
that an image which was being tracked asynchronously has encountered an error. No further information will
become available, and drawing the image will fail. Whenever this flag is set, the ABORT flag must also be
set.

                    */

                    public final static int FRAMEBITS;

                              /*        FRAMEBITS = 16 : This flag in the infoflags argument to
imageUpdate'‘indicates that another complete frame of a multi-frame image can now be drawn. The x, y,
width, and height arguments to the imageUpdate method should be ignored when this flag is set in the
status.

                              /*

                    public final static int HEIGHT;

                              /*        HEIGHT = 2 : This flag in the infoflags argument to
imageUpdate'‘ indicates that the height of the base image is now available and can be taken from the
height argument to the imageUpdate method.

                              */

                              public final static int PROPERTIES;

                              /*        PROPERTIES = 4 : This flag in the infoflags argument to
imageUpdate indicates that the properties of the image are now available.

                              */

                    public final static int SOMEBITS;

                              /*        SOMEBITS = 8 : This flag in the infoflags argument to
imageUpdate'‘ indicates that the pixels needed for drawing a scaled variation of the image are now
available.

                              */




                    public final static int WIDTH;

                              /*        WIDTH'‘= 2 : This flag in the infoflags argument to
imageUpdate'‘indicates that the width of the base image is now available and can be taken from the width
argument to the imageUpdate method.

                              */


                              //        Methods

                    public abstract boolean        imageUpdate(Image'‘img, int'‘infoflags, int'‘x,int'‘y,
int'‘width, int'‘height);

                    /*        This image observer method is called when previously requested information
about an image becomes available.

                              Asynchronous interfaces are method calls such as getWidth, getHeight, and
drawImage'‘ which take an image observer as an argument. These methods register the caller as being
interested either in information about the image or about an output version of the image. This method
should return true if further calls to imageUpdate are needed by this image observer; false if it needs no
more information.

                              The infoflags argument should be the OR of the following flags :
```

```
                              WIDTH’‘HEIGHT PROPERTIES’‘SOMEBITS’‘FRAMEBITS’‘ALLBITS’‘ERROR ABORT’‘

                              The interpretation of the x, y, width, and height arguments depends on the
infoflags’‘argument.
                     */
          }
```

Interface ImageProducer

The image producer interface specifies the methods that all image producers must implement. Every image contains an image producer which can reconstruct the image whenever it is needed by an image consumer.

```
          public’‘interface’‘java.awt.image.ImageProducer
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract void addConsumer(ImageConsumer’‘ic);
‘‘‘‘      public abstract boolean isConsumer(ImageConsumer’‘ic);
‘‘‘‘      public abstract void removeConsumer(ImageConsumer’‘ic);
‘‘‘‘      public abstract void requestTopDownLeftRightResend(ImageConsumer’‘ic);
‘‘‘‘      public abstract void startProduction(ImageConsumer’‘ic);
          }
          Methods are described in Table 5.11.
```

Table 5.11

Methods	Description
addConsumer(ImageConsumer ic)	Registers the image consumer argument as wanting information about this image. The image producer may, at its discretion, start delivering the image data immediately, or it may wait until the next image reconstruction is forced by a call to the `startProduction` method.
isConsumer(ImageConsumer ic)	**Returns :** true, if the specified image consumer argument is currently registered with this image producer as one of its consumers; false otherwise.
removeConsumer(ImageConsumer ic)	Removes the specified image consumer object from the list of consumers registered to receive the image data. It is not an error to remove a consumer that is not registered. The image producer should stop sending data to this consumer as soon as it is feasible.

requestTopDownLeftRightResend(ImageConsumer ic)	An image consumer inokes this method to request that the image producer attempt to resend the image data one more time in top-down, left-to-right order. If the data cannot be resent in that order, the image producer ignores this call. If the data can be resent in that order, the image producer should respond by executing the following minimum set of image consumer method calls127 : ic.setHints(TOPDOWNLEFTRIGHT | *otherhints*); ic.setPixels(...); // As many times as needed ic.imageComplete(); An image consumer might call this method so that it can use a higher quality conversion algorithm which depends on receiving the pixels in order.
startProduction (ImageConsumer ic)	Registers the image consumer argument as wanting information about this image. In addition, this method forces the image producer to start an immediate reconstruction of the image data. Then data will be delivered both to this image consumer and to any other image consumers which may have already been registered with the producer using the `addConsumer` method.

Chapter 17

The AWT Peer Package

by Debasis Samanta

CONTENTS

- Introduction
- Interface ButtonPeer
- Interface CanvasPeer
- Interface CheckboxMenuItemPeer
- Interface CheckboxPeer
- Interface ChoicePeer
- Interface ComponentPeer
- Interface ContainerPeer

- Interface DialogPeer
- Interface FileDialogPeer
- Interface FramePeer
- Interface LabelPeer
- Interface ListPeer
- Interface MenuBarPeer
- Interface MenuComponentPeer
- Interface MenuItemPeer
- Interface MenuPeer
- Interface PanelPeer
- Interface ScrollbarPeer
- Interface TextAreaPeer
- Interface TextComponentPeer
- Interface TextFieldPeer
- Interface WindowPeer

Introduction

The java.awt.peer Package includes interfaces used to connect AWT components to their platform specific implementations such as Motif widgets or Microsoft Windows controls. The interfaces in the java.awt.peer are need to be used only if one wants to create a syste specific implementation. This package contains the interfaces as listed below :

```
        Interfaces:


                ‘‘ Interface ButtonPeer
                ‘‘ Interface CanvasPeer
                ‘‘ Interface CheckboxMenuItemPeer
                ‘‘ Interface CheckboxPeer
                ‘‘ Interface ChoicePeer
                ‘‘ Interface ComponentPeer
                ‘‘ Interface ContainerPeer
                ‘‘ Interface DialogPeer
                ‘‘ Interface FileDialogPeer
                ‘‘ Interface FramePeer
                ‘‘ Interface LabelPeer
                ‘‘ Interface ListPeer
                ‘‘ Interface MenuBarPeer
                ‘‘ Interface MenuComponentPeer
                ‘‘ Interface MenuItemPeer
                ‘‘ Interface MenuPeer
                ‘‘ Interface PanelPeer
                ‘‘ Interface ScrollbarPeer
                ‘‘ Interface TextAreaPeer
```

```
                    ‘‘ Interface TextComponentPeer
                    ‘‘ Interface TextFieldPeer
                    ‘‘ Interface WindowPeer
```

Interface ButtonPeer

The button peer interface specifies the methods that all implementations of Abstract Window Toolkit buttons must define.

```
          public’‘interface’‘java.awt.peer.ButtonPeer extends java.awt.peer.ComponentPeer
‘‘
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘‘‘‘‘public abstract void setLabel(String’‘label);
                    //        Changes the button's label to be the String argument.


          }
```

Interface CanvasPeer

The canvas peer interface specifies the methods that all implementations of Abstract Window Toolkit canvases must define. In general, canvas implementations only need to implement methods required of all component peers.

```
          public’‘interface’‘java.awt.peer.CanvasPeer extends java.awt.peer.ComponentPeer’‘
          {‘‘‘‘
          }
```

Interface CheckboxMenuItemPeer

The check box menu peer interface specifies the methods that all implementations of Abstract Window Toolkit check box menus must define

```
          public’‘interface’‘java.awt.peer.CheckboxMenuItemPeer extends java.awt.peer.MenuItemPeer’‘
                    {
                              //        Methods
          ‘‘‘‘public abstract void setState(boolean’‘t);
                              //        Sets the check box to the specifed boolean state: true indicates
"on"; false indicates "off."


                    }
```

Interface CheckboxPeer

The check box peer interface specifies the methods that all implementations of Abstract Window Toolkit check boxes must define

```
          public‘‘interface‘‘java.awt.peer.CheckboxPeer extends java.awt.peer.ComponentPeer‘‘
                    {
                              //        Methods
                    public abstract void setCheckboxGroup(CheckboxGroup‘‘g);

                              //        Sets the group of the checkbox to be the specified CheckboxGroup

                    public abstract void setLabel(String‘‘label);

                              //        Changes the check box's label to be the string argument.
                    public abstract void setState(boolean‘‘state);

                              //        Sets the check box to the specifed boolean state: true indicates
"on"; false indicates "off"
                    }
```

Interface ChoicePeer

The choice menu peer interface specifies the methods that all implementations of Abstract Window Toolkit choice menus must define.

```
          public‘‘interface‘‘java.awt.peer.ChoicePeer extends java.awt.peer.ComponentPeer
                    {
                              //        Methods
                    public abstract void addItem(String‘‘item, int‘‘index);

                              //        Adds an item to the choice menu at the specified position.

                    public abstract void select(int‘‘index);

                    //        Sets the selected item to be the item at the specified position.
                    }
```

Interface ComponentPeer

The component peer interface specifies the methods that all implementations of Abstract Window Toolkit components must define.

```
          public‘‘interface‘‘java.awt.peer.ComponentPeer
           {
‘‘‘‘‘‘‘‘ //        Methods
                    public abstract int checkImage(Image‘‘img, int‘‘w, int‘‘h, ImageObserver‘‘o);
                    public abstract Image createImage(ImageProducer‘‘p);
```

```
            public abstract Image createImage(int'‘w , int'‘h );
            public abstract void disable();
            public abstract void dispose();
            public abstract void enable();
            public abstract ColorModel getColorModel();
            public abstract FontMetrics getFontMetrics(Font'‘font);
            public abstract Graphics getGraphics();
            public abstract Toolkit getToolkit();
            public abstract boolean handleEvent(Event'‘e);
            public abstract void hide();
            public abstract Dimension minimumSize();
            public abstract void nextFocus();
            public abstract void paint(Graphics'‘g);
            public abstract Dimension preferredSize();
            public abstract boolean'‘prepareImage(Image'‘img, int'‘w, int'‘h,ImageObserver'‘o);
            public abstract void print(Graphics'‘g);
            public abstract void repaint(long'‘tm, int'‘x, int'‘y,
int'‘width,int'‘height);
            public abstract void requestFocus();
            public abstract void reshape(int'‘x, int'‘y, int width, int'‘height);
            public abstract void setBackground(Color'‘c);
            public abstract void setFont(Font'‘f);
            public abstract void setForeground(Color'‘c);
            public abstract void show();
}


Methods are stated in Table 6.1.
```

Table 6.1

Method	Description
checkImage(.....)	Returns the status of the construction of a scaled screen representation of the specified image. This method does not cause the image to begin loading. An application must use the prepareImage method to force the loading of an image. **Returns:** the bitwise inclusive OR of the `ImageObserver` flags for the data that is currently available.
createImage(ImageProducer p)	Creates an image from the specified image producer.
createImage(int w , int h)	Creates an off-screen drawable image to be used for double buffering.
disable()	Makes the component insensitive to user input.
dispose()	Disposes of the component peer and any resources used by it.

enable()	Makes the component sensitive to user input. This is the default.
getColorModel()	Gets the component's color model which is an abstract class that encapsulates how to translate between pixel values of an image and its red, green, blue, and alpha components.
getFontMetrics(Font font)	**Returns :** the font metrics for this component; if the component is not currently on the screen, this method returns null.
getGraphics()	**Returns :** the graphics context of this component; this method returns null if the component is not currently on the screen.
getToolkit()	**Returns :** the toolkit that component peer is part of.

Table 6.1 (Contd..)

handleEvent(Event e)	This method is called when any event occurs inside the component. The method returns true to indicate that it has successfully handled the action; or false if the event that triggered the action should be passed up to the component's containing object.
hide()	Hides the component.
minimumSize()	**Returns :** the minimum size of this component.
nextFocus()	Moves the focus to the next component
paint(Graphics g)	Paints the component.The coordinator of the graphics context is the top-left corner of the components. The clipping region of the graphics context is the rectangle of the component.133
preferredSize()	**Returns :** the preferred size of this component.
prepareImage(....)	Prepares an image for rendering on this component at the specified width and height. The image data is downloaded asynchronously in another thread and an appropriately scaled screen representation of the image is generated.
print(Graphics g)	Prints this component.
repaint(....)	Repaints the specified rectangle of the component.
requestFocus()	Requests the input focus.
reshape(...)	Reshapes the component to the specified bounding rectangle.
setBackground(Color c)	Sets the background color for the component.
setFont(Font f)	Sets the font of the component.
setForeground(Color c)	Sets the foreground color for the component.
show()	Shows the component: if the component had been made invisible by a call to the hide method, make the component visible again.

Interface ContainerPeer

The container peer interface specifies the methods that all implementations of Abstract Window Toolkit containers must define.

```
          public‘‘interface’‘java.awt.peer.ContainerPeer extends java.awt.peer.ComponentPeer’‘
          ‘‘‘‘{
                                //        Methods
                    public abstract Insets insets();
                    /*        Determines the insets of the container, which indicate the size of the
border of the container. A Frame, for example, has a top inset that corresponds to the height of the
frame's title bar.
                    */
          ‘‘ }
```

Interface DialogPeer

The dialog window peer interface specifies the methods that all implementations of Abstract Window Toolkit dialog windows must define.

```
          public‘‘interface’‘java.awt.peer.DialogPeer extends java.awt.peer.WindowPeer’‘
                    {
                                          //        Methods
                              public abstract void setResizable(boolean’‘resizeable);
                              //        Sets the resizable flag.

                              public abstract void setTitle(String’‘title);
                              //        Sets the title of the dialog window.
                    }
```

Interface CanvasPeer

The dialog window peer interface specifies the methods that all implementations of Abstract Window Toolkit dialog windows must define.

```
          public‘‘interface’‘java.awt.peer.DialogPeer extends java.awt.peer.WindowPeer’‘
          ‘‘‘‘{
                                //        Methods
                    public abstract void setResizable(boolean’‘resizeable);
                    //        Sets the resizable flag.

                    public abstract void setTitle(String’‘title);
                    //        Sets the title of the dialog window.
          ‘‘ }
```

Interface FileDialogPeer

The file dialog window peer interface specifies the methods that all implementations of Abstract Window Toolkit file dialog windows must define

```
          public‘‘interface’‘java.awt.peer.FileDialogPeer extends java.awt.peer.DialogPeer’‘
          ‘‘‘‘{
‘‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract void setDirectory(String’‘dir);

                    //        Sets the directory of the file dialog window to be the specified directory.

‘‘‘‘      public abstract void setFile(String’‘file);

                    //        Sets the selected file for the file dialog window to be the specified file.

‘‘        ‘‘‘‘public abstract void setFilenameFilter(FilenameFilter’‘filter);

                    //        Sets the filename filter’‘for this file dialog window to the specified
filter.
          ‘‘ }
```

Interface FramePeer

The frame peer interface specifies the methods that all implementations of Abstract Window Toolkit frames must define.

```
          public‘‘interface’‘java.awt.peer.FramePeer extends java.awt.peer.WindowPeer’‘
          {
‘‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract void setCursor(int’‘cursorType);
‘‘‘‘      public abstract void setIconImage(Image’‘im);
‘‘‘‘      public abstract void setMenuBar(MenuBar’‘mb);
‘‘‘‘      public abstract void setResizable(boolean’‘resizeable);
‘‘‘‘      public abstract void setTitle(String’‘title);
          }



          Methods are stated in Table 6.2.
```

Table 6.2

Method	Description
setCursor(int cursorType)	Sets the cursor image to be one of the predefined cursors.

setIconImage(Image im)	Sets the image to display when this frame is iconized. Note that not all platforms support the concept of iconizing a window.
setMenuBar(MenuBar mb)	Sets the menubar of this frame to the specified menu bar.
setResizable(boolean resizeable)	Determines whether this frame should be resizable.
setTitle(String title)	Sets the title of this frame to the specified title.

Interface LabelPeer

The label peer interface specifies the methods that all implementations of Abstract Window Toolkit label must define

```
          public‘‘interface’‘java.awt.peer.LabelPeer          extends java.awt.peer.ComponentPeer’‘
          {
‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract void setAlignment(int’‘alignment);

                    //        Sets the alignment for this label to the specified alignment.

‘‘‘‘      public abstract void setText(String’‘label);

                    //        Sets the text for this label to the specified text.
          }
```

Interface ListPeer

The scrolling list peer interface specifies the methods that all implementations of Abstract Window Toolkit scrolling lists must define.

```
          public‘‘interface’‘java.awt.peer.ListPeer extends java.awt.peer.ComponentPeer’‘
          {
‘‘‘‘‘‘‘‘

                    //        Methods
‘‘‘‘      public abstract void addItem(String’‘item, int’‘index);
‘‘‘‘      public abstract void clear();
‘‘‘‘      public abstract void delItems(int’‘start, int’‘end);
‘‘‘‘      public abstract void deselect(int’‘index);
‘‘‘‘      public abstract int[] getSelectedIndexes();
‘‘‘‘      public abstract void makeVisible(int’‘index);
‘‘‘‘      public abstract Dimension minimumSize(int’‘v);
‘‘‘‘      public abstract Dimension preferredSize(int’‘v);
‘‘‘‘      public abstract void select(int’‘index);
‘‘‘‘      public abstract void setMultipleSelections(boolean’‘v);
```

```
        }

        Methods are stated in Table 6.3.
```

Table 6.3

Method	Description
addItem(String item, int index)	Adds the specified string to the scrolling list at the specified position. The index argument is 0-based. If the index is -1, or greater than or equal to the number of items already in the list, then the item is added at the end of the list.
clear()	Removes all items from the scrolling list.
delItems(int start, int end)	Deletes the items in the range from the scrolling list.
deselect(int index)	Deselects the item at the specified index.
getSelectedIndexes()	**Returns :** an array of the selected indexes on the scrolling list.
makeVisible(int index)	Forces the item at the specified index to be visible.
minimumSize(int v)	**Returns :** the minimum dimensions needed to display the specified number of rows in a scrolling list.
preferredSize(int v)	**Returns :** the preferred dimensions needed to display the specified number of rows in a scrolling list.
select(int index)	Selects the item at the specified index.
setMultipleSelections(boolean v)	Sets whether this scolling list allows multiple selections.

Interface MenuBarPeer

The menu bar peer interface specifies the methods that all implementations of Abstract Window Toolkit menu bars must define.

```
        public‘‘interface’‘java.awt.peer.MenuBarPeer extends java.awt.peer.MenuComponentPeer’‘
        {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘    public abstract void addHelpMenu(Menu’‘m);

                //        Sets the help menu on the menu bar to be the specified menu.

‘‘‘‘    public abstract void addMenu(Menu’‘m);

                //        Adds the specified menu to the menu bar

‘‘‘‘    public abstract void delMenu(int’‘index);
```

```
                    //        Removes the menu located at the specified index from the menu bar.


          }
```

Interface MenuComponentPeer

The menu component peer interface specifies the methods that all implementations of Abstract Window Toolkit menu components must define.

```
          public‘‘interface’‘java.awt.peer.MenuComponentPeer
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract void dispose();
                    //        Disposes of the menu component peer and any resources used by it.
          }
```

Interface MenuItemPeer

The MenuItemPeer interface specifies the methods that all implementations of Abstract Window Toolkit menu items must define.

```
          public‘‘interface’‘java.awt.peer.MenuItemPeer extends java.awt.peer.MenuComponentPeer’‘
          {
                    //        Methods
‘‘‘‘      public abstract void disable();


                    //        Disables this menu item. It can no longer be selected by the user..


‘‘        ‘‘‘‘public abstract void enable();


                    //        Enables this menu item. It can be selected by the user.


‘‘‘‘      public abstract void setLabel(String’‘label);


                    //        Changes the menu item's label to be the specified string.
          }
```

Interface MenuPeer

The menu peer interface specifies the methods that all implementations of Abstract Window Toolkit menus must define.

```
          public‘‘interface’‘java.awt.peer.MenuPeer extends java.awt.peer.MenuItemPeer’‘
                    {
                                        //        Methods
                    public abstract void addItem(MenuItem’‘item);
                              //        Adds the specified menu item to this menu.


                    public abstract void addSeparator();
                              //        Adds a separator line to this menu.


                    public abstract void delItem(int’‘index);
                              //        Deletes the item at the specified index from this menu.
          ‘‘‘‘}
```

Interface PanelPeer

The panel peer interface specifies the methods that all implementations of Abstract Window Toolkit panels must define.

```
          public‘‘interface’‘java.awt.peer.PanelPeer extends java.awt.peer.ContainerPeer’‘
                    {
                    }
```

Interface ScrollbarPeer

The scroll bar peer interface specifies the methods that all implementations of Abstract Window Toolkit scroll bars must define.

```
          public‘‘interface’‘java.awt.peer.ScrollbarPeer extends java.awt.peer.ComponentPeer’‘
          ‘‘‘‘{
‘‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract void setLineIncrement(int’‘l);
                    /*        Sets the line increment of the scroll bar. The line increment is the value
that is added to or subtracted from the value of the scrollbar when the user hits the line down or line up
gadget.
                    */
‘‘‘‘      public abstract void setPageIncrement(int’‘l);
                    /*        Sets the page increment of the scroll bar. The page increment is the value
that is added to or subtracted from the value of the scrollbar when the user hits the page down or page up
gadget.
                    */
‘‘‘‘      public abstract void setValue(int’‘value);
                    //        Sets the value of the scroll bar to the specified value.


‘‘‘‘      public abstract void setValues(int’‘value, int’‘visible int’‘min, int’‘max );
```

```
                    //        Sets several parameters of the scroll bar simultaneously.
‘‘‘‘‘‘‘}
```

Interface TextAreaPeer

The text area peer interface specifies the methods that all implementations of Abstract Window Toolkit text areas must define

```
          public’‘interface’‘java.awt.peer.TextAreaPeer extends java.awt.peer.TextComponentPeer’‘
          ‘‘‘‘{
                              //        Methods
                    public abstract void insertText(String’‘txt, int’‘pos);
                              //        Inserts the specified text at the specified position.


                    public abstract Dimension minimumSize(int’‘rows, int’‘cols);
                              //        Returns : the minimum dimensions needed for the text area.


                    public abstract Dimension preferredSize(int’‘rows, int’‘cols);
                              //        Returns : the preferred dimensions needed for the text area.


                    public abstract void replaceText(String’‘txt, int’‘start,      int’‘end);
                              /*        Replaces the text from the start (inclusive) index to the end
(exclusive) index with the new text specified.
                              */
          ‘‘‘‘}
```

Interface TextComponentPeer

The text component peer interface specifies the methods that all implementations of Abstract Window Toolkit text components must define

```
          public’‘interface’‘java.awt.peer.TextComponentPeer extends java.awt.peer.ComponentPeer’‘
          ‘‘‘‘{
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract int getSelectionEnd();
                    //        Returns : selected text's end position.


‘‘‘‘      public abstract int getSelectionStart();
                    //        Returns : selected text's start position.


‘‘‘‘      public abstract String getText();
```

```
                    //        Returns : the text of this text component.


‘‘‘‘      public abstract void select(int’‘selStart, int’‘selEnd);

                    //        Selects the text in the text component from the start (inclusive) index to
the end                       (exclusive) index.


                    public abstract void setEditable(boolean’‘editable)

                              /*        Sets the text component to be user editable if the boolean
argument is true. If the flag is false, sets the text component so that the user cannot change its
contents.

                              */

                    public abstract void setText(String’‘l);

                              /*        Sets the text of this text component to the specified text.

          ‘‘‘‘}
```

Interface TextFieldPeer

The text field peer interface specifies the methods that all implementations of Abstract Window Toolkit text fields must define.

```
          public’‘interface’‘java.awt.peer.TextFieldPeer extends java.awt.peer.TextComponentPeer’‘

          {

‘‘‘‘‘‘‘‘ //        Methods

                    public abstract Dimension minimumSize(int’‘cols);

                              /*        Returns : the minimum dimensions needed to display the text
field with the specified number of columns.

                              */

                    public abstract Dimension preferredSize(int’‘cols);

                              /*        Returns : the preferred dimensions needed to display the text
field with the specified number of columns.

                              */


                    public abstract void setEchoCharacter(char’‘c);

                              /*        Sets the echo character for this text field. Any character that
the user types in the text field is echoed in the text field as the echo character. An echo character is
useful for fields where the user input shouldn't be echoed to the screen such as in the case of a text
field for typing in a password.

                              */

          }
```

Interface WindowPeer

The window peer interface specifies the methods that all implementations of Abstract Window Toolkit windows must define.

```
          public‘‘interface’‘java.awt.peer.WindowPeer extends java.awt.peer.ContainerPeer’‘
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract void toBack();
                    //        Sends this window to the back.


‘‘‘‘      public abstract void toFront();
                    //        Brings this window to the front.
          }
```

Chapter 18

The Java I/O Package

by Debasis Samanta

CONTENTS

- Introduction
- Class BufferedInputStream
- Class BufferedOutputStream
- Class ByteArrayInputStream
- Class ByteArrayOutputStream
- Class DataInputStream
- Class DataOutputStream
- Class File
- Class FileDescriptor
- Class FileInputStream
- Class FileOutputStream
- Class FilterInputStream
- Class FilterOutputStream
- Class InputStream
- Class LineNumberInputStream
- Class OutputStream
- Class PipedInputStream
- Class PipedOutputStream
- Class PrintStream
- Class PushbackInputStream
- Class RandomAccessFile
- Class SequenceInputStream
- Class StreamTokenizer
- Class StringBufferInputStream
- Interface DataInput

- Interface DataOutput
- Interface FilenameFilter
- Class EOFException
- Class FileNotFoundException
- Class IOException
- Class InterruptedIOException
- Class UTFDataFormatException

Introduction

The package java.io provide 23 classes, 3 interfaces, and 5 error classes that provides to manage a set of input and output streams to read data from and write data to files, strings, and otherr sources. Java streams are byte oriented and the classes defined in the java.io package can utilized to implrement more sophisticated stream manipulation. Follwing is the various component in ths package.

```
‘‘ Classes
‘‘
          ‘‘Class BufferedInputStream
          ‘‘Class BufferedOutputStream
          ‘‘Class ByteArrayInputStream
          ‘‘Class ByteArrayOutputStream
          ‘‘Class DataInputStream
          ‘‘Class DataOutputStream
          ‘‘Class File
          ‘‘Class FileDescriptor
          ‘‘Class FileInputStream
          ‘‘Class FileOutputStream
          ‘‘Class FilterInputStream
          ‘‘Class FilterOutputStream
          ‘‘Class InputStream
          ‘‘Class LineNumberInputStream
          ‘‘Class OutputStream
          ‘‘Class PipedInputStream
          ‘‘Class PipedOutputStream
          ‘‘Class PrintStream
          ‘‘Class PushbackInputStream
          ‘‘Class RandomAccessFile
          ‘‘Class SequenceInputStream
          ‘‘Class StreamTokenizer
          ‘‘Class StringBufferInputStream
‘‘
‘‘Interfaces
‘‘
```

```
          ‘‘Interface DataInput’‘

          ‘‘Interface DataOutput

          ‘‘Interface FilenameFilter’‘

‘‘Exceptions

          ‘‘

          ‘‘Class EOFException

          ‘‘Class FileNotFoundException

          ‘‘Class IOException

          ‘‘Class InterruptedIOException

          ‘‘Class UTFDataFormatException
```

Class BufferedInputStream

The class implements a buffered input stream. By setting up a such an input stream, an application can read bytes from a stream without necessarily causing a call to the underlying system for each byte read. The data is read by blocks into a buffer; subsequent reads can access the data directly from the buffer. The structure of this class is shown as berlow :

```
public’‘class’‘java.io.BufferedInputStream extends’‘java.io.FilterInputStream

          {

                    //        Member elements

                    protected byte buf [];

                    //        An array of bytes, bufffer,’‘where data is stored.

                    protected int count ;

                    //        The index one greater than the index of the last valid byte in the buffer.

                    protected int marklimit ;

                    /*        The maximum read ahead allowed after a call to the mark method’‘ before
subsequent calls to the reset method’‘fail.

                    */

                    protected int markpos ;

                    /*        The value of the pos field’‘at the time the last mark method’‘ was called.
The value of this field is -1 if there is no current mark.

                    */

                    protected int pos ;

                    /*        The current position in the buffer. This is the index of the next character
to be read from the buf’‘ array.

                    */


                    // Constructors

                    public BufferedInputStream (InputStream’‘in);

                    /*        Creates a new buffered input stream to read data from the specified input
stream with a default 512-byte buffer size.
```

```
                    public BufferedInputStream (InputStream’‘in, int size);
                    /*        Creates a new buffered input stream to read data from the specified input
stream with the specified buffer size.
                    */                  ‘‘
                    //        Methods
                    public int available ()
                    public void mark (int’‘readlimit);
                    public boolean markSupported ();
                    public int read ();
                    public int read (byte’‘b[], int’‘off, int’‘len);
                    public void reset ();’‘
                    public long skip (long’‘n);
          }
          The methods are listed in Table 7.1.’‘
```

Table 7.1

Method	Description
available `()`	Determines the number of bytes that can be read from this input stream without blocking.
mark `(int readlimit)`	Marks the current position in this input stream. A subsequent call to the `reset` method repositions the stream at the last marked position so that subsequent reads re-read the same bytes. The `readlimit` arguments tells the input stream to allow that many bytes to be read before the mark position gets invalidated.
markSupported `()`	Determines if this input stream supports the mark and reset methods. The `markSupported` method of Buffered-Input-Stream returns true.
read `()`	Reads the next byte of data from this buffered input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned. This method blocks until either input data is available, the end of the stream is detected, or an exception is thrown.
read `(byte b[], int off, int len)`	Reads up to `len` bytes of data from this buffered input stream into an array of bytes. This method blocks until some input is available.
reset `()`	Repositions this stream to the position at the time the `mark` method was last called on this input stream.

skip `(long n)`	Skips over and discards n bytes of data from the input stream. The skip method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly zero. The actual number of bytes skipped is returned.

Class BufferedOutputStream

The class implements a buffered output stream. By setting up a such an output stream, an application can write bytes to the underlying output stream without necessarily causing a call to the underlying system for each byte written. The data is written into a buffer, and then written to the underlying stream if the buffer reaches its capacity, the buffer output stream is closed, or the buffer output stream is explicity flushed. The various components in this class is furnished as below :

```
public‘‘class‘‘java.io.BufferedOutputStream extends‘‘java.io.FilterOutputStream‘‘
          {
‘‘‘‘‘‘‘‘//          Member elements
‘‘‘‘      protected byte buf[];
                    //        The buffer, array of bytes,‘‘where data is stored.
‘‘‘‘      protected int count;
                    //        The number of valid bytes in the buffer.


‘‘‘‘‘‘‘‘//          Constructors
‘‘‘‘      public BufferedOutputStream(OutputStream‘‘out);
                    /*        Creates a new buffered output stream to write data to the specified
underlying output stream with a default 512-byte buffer size.
                    */
‘‘‘‘      public BufferedOutputStream(OutputStream‘‘out, int size);
                    /*        Creates a new buffered output stream to write data to the specified
underlying output stream with the specified buffer size.
                    */
‘‘‘‘‘‘‘‘//          Methods
‘‘‘‘      public void flush();
‘‘‘‘      public void write(byte‘‘b[], int‘‘off, int‘‘len);
‘‘‘‘      public void write(int‘‘b);
          }
          The operation performed by the methods in this class is stated in Table 7.2
```

Table 7.2

Method	Description
flush()	Flushes this buffered output stream. This forces any buffered output bytes to be written out to the underlying output stream. **Throws** : `IOException`, if an I/O error occurs.

write(byte b[], int off, int len);	Writes `len` bytes from the specified byte array starting at offset `off` to this buffered output stream. **Throws** : `IOException`, if an I/O error occurs.
write(int b);	Writes the specified byte to this buffered output stream. **Throws** : `IOException`, if an I/O error occurs.

Class ByteArrayInputStream

This class allows an application to create an input stream in which the bytes read are supplied by the contents of a byte array. Applications can also read bytes from a string by using a StringBufferInputStream.

```
public‘‘class’‘java.io.ByteArrayInputStream extends’‘java.io.InputStream’‘
          {
                    //        Member elements
                    protected byte buf[];
                    //        The buffer, byte array containing the data.
                    protected int count;
                              //        The index one greater than the last valid character in the input
stream buffer.
                    protected int pos;
                    //        The index of the next character to read from the input stream buffer.    ‘‘

                    //        Constructors
                    public ByteArrayInputStream(byte’‘buf[]);
                    /*        Creates a new byte array input stream which reads data from the specified
byte array. The byte array is not copied. */
                    public ByteArrayInputStream(byte’‘buf[], int offset, int’‘length);
                    /*        Creates a new byte array input stream which reads data from the specified
byte array. Up to length characters are to be read from the byte array, starting at the indicated offset.
                              */’‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘‘
                    //        Methods
                    public int available();
                    public int read();
                    public int read(byte’‘b[], int’‘off, int’‘len);
                    public void reset();
                    public long skip(long’‘n);
          }

          The operation performed by the various methods in ths class is stated in Table 7.3 :
```

Table 7.3’‘

Method	Description

available()	Determines the number of bytes that can be read from this input stream without blocking. The available method of `ByteArrayInputStream` returns the value of, which is the number of bytes remaining to be read from the input buffer.
read()	Reads the next byte of data from this input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned.
read(byte b[], int off, int len);	Reads up to len bytes of data into an array of bytes from this input stream. This read method cannot block.
reset()	Resets this input stream to begin reading from the same position at which it first started reading from the buffer.
skip(long n)	Skips n bytes of input from this input stream. Fewer bytes might be skipped if the end of the input stream is reached.

Class ByteArrayOutputStream

This class implements an output stream in which the data is written into a byte array. The buffer automatically grows as data is written to it. The structure of the class as shown below

```
          public‘‘class’‘java.io.ByteArrayOutputStream extends’‘java.io.OutputStream’‘
          {
                    //        Member elements
                    protected byte buf[];
                    //        The buffer where data is stored.
                    protected int count;
                    //        The number of valid bytes in the buffer.

                    // Constructors
                    public ByteArrayOutputStream();
                    //        Creates a new byte array output stream.
                    public ByteArrayOutputStream(int’‘size);
                    /*        Creates a new byte array output stream. The buffer capacity is initially 32
bytes, though its size increases if necessary.
                    */

                    //        Methods
                    public void reset();
                    public int size();
                    public byte[] toByteArray();
                    public String toString();
                    public String toString(int’‘hibyte);
```

```
                public void write(byte'‘b[], int'‘off, int'‘len);
                public void write(int'‘b);
                public void writeTo(OutputStream'‘out);
        }
        The operation of the various methods are stated in Table 7.4.
```

Table 7.4

Method	Description
reset()	Resets the `count` field of this byte array output stream to zero, so that all currently accumulated output in the output stream is disdcarded. The out stream can be used again, reusing the already allocated buffer space.
size()	Returns the value of the count field, which is the number of valid bytes in this output stream.
toByteArray()	Creates a newly allocated byte array whose size is the current size of this output stream and into which the valid contents of the buffer have been copied.
toString()	Creates a newly allocated string whose size is the current size of this output stream and into which the valid contents of the buffer have been copied. Each character c in the resulting string is constructed from the corresponding element b in the byte array such that c == (char)(b & 0xff)
toString(int hibyte)	Creates a newly allocated string whose size is the current size of the output stream and into which the valid contents of the buffer have been copied. Each character c in the resulting string is constructed from the corresponding element b in the byte array such that c == (char)(((hibyte & 0xff) << 8) | (b & 0xff))
write(byte b[], int off, int len);	Writes `len` bytes from the specified byte array starting at offset off to this byte array output stream.
write(int b);	Writes the specified byte to this byte array output stream.
writeTo(OutputStream out);	Writes the complete contents of this byte array output stream to the specified output stream argument, as if by calling the output stream's write method using out `write(buf, 0, count).`

Class DataInputStream

A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way. An application uses a data output stream to write data which can later be read by a data input stream. Data input streams and data output streams represent Unicode strings in a format that is a slight modification of UTF-8. The two differences between this format and the "standard" UTF-8 format are the following:

- ''The null byte'\u0000' is encoded in two-byte format rather than one-byte, so that the encoded strings never have embedded nulls.
- ''Only the one-byte, two-byte, and three-byte formats are used.

```
public‘‘class‘‘java.io.DataInputStream extends‘‘java.io.FilterInputStream‘‘
          implements java.io.DataInput‘‘
                    {
                    //        Constructors
                    public DataInputStream(InputStream‘‘in);
                    //        Creates a new data input stream to read data from the specified input
stream.
                    //        Methods
                    public final int read(byte‘‘b[]);
                    public final int read(byte‘‘b[], int‘‘off, int‘‘len);
                    public final boolean readBoolean();
                    public final byte readByte();
                    public final char readChar();
                    public final double readDouble();
                    public final float readFloat();
                    public final void readFully(byte‘‘b[]);
                    public final void readFully(byte‘‘b[], int‘‘off, int‘‘len);
                    public final int readInt();
                    public final String readLine();
                    public final long readLong();
                    public final short readShort();
                    public final int readUnsignedByte();
                    public final int readUnsignedShort();
                    public final String readUTF();
                    public final static String readUTF(DataInput‘‘in);
                    public final int skipBytes(int‘‘n);
          ‘‘ }
          Table 7.5 represents the operation performed by various methods in this class.
```

Table 7.5

Method	Description
read(byte b[])	Reads up to byte.length bytes of data from this data input stream into an array of bytes. This method blocks until some input is available. The read method of `DataInputStream` calls the the read method of its underlying input stream with three arguments b, 0, and b.length, and returns whatever value the method returns.
read(byte b[], int off, int len)	Reads up to len bytes of data from this data input stream into an array of bytes. This method blocks until some input is available. The read method of `DataInputStream` calls the read method of its underlying input stream with the same arguments and

	returns whatever value the method returns.
readBoolean()	Reads a boolean from this data input stream. This method reads a single byte from the underlying input stream. a value of 0 represents `false`. Any other value represents `true`. This method blocks until eother the byte is read, the end of the stream is deleted, or an exception is thrown.
readByte()	Reads a signed 8-bit value from this data input stream. This method reads a byte from the underlying input stream. If the byte read is b, where 0 <= b <= 255, then the result is (byte)(b). This method blocks until either the byte is read, the end of the stream is detected, or an exception is thrown.
readChar()	Reads a Unicode character from this data input stream. This method reads two bytes from the underlying input stream. If the bytes read, in order, are b1 and b2, where 0<= b21, b2 <= 255, then the result is equal (char)((b1 << 8) | b2) This method blocks until either the two bytes are read, the end of the stream is detected, or an exception is thrown.
readDouble()	Reads a double from this data input stream. This method reads a long value as if by the readLong method and then converts that long to a double using the longBitsToDouble method in class Double. This method blocks until either the eight bytes are read, the end of the stream is detected, or an exception is thrown.
readFloat()	Reads a float from this data input stream. This method reads an int value as if by the readInt method and then converts that into a float using the `inBitsToFloat` method in class `Float`. This method blocks until either the four bytes are read, the end of the stream is deleted, or an exception is thrown. **Throws:** `EOFException` If this input stream reaches the end before reading four bytes.
readFully(byte b[])	Reads full bytes from this data input stream into the byte array. This method reads repeatedly from the underlying stream until all the bytes are read. This method blocks until either all the bytes are read, the end of the stream is deleted, or an exception is thrown. **Throws :** `EOFException` If this input stream reaches the end before reading all bytes.
readFully(byte b[], int off, int len)	Reads exactly len bytes from this data input stream into the byte array. This method reads repeatedly from the underlying stream until all the bytes are read. This method blocks until either all the bytes are read, the end of the stream is deleted, or an exception is thrown. **Throws** EOFException If this input stream reaches the end before reading all bytes.
readInt()	Reads a signed 32-bit integer from this data input stream. This method reads four bytes from the underlying input stream. If the read bytes are in order, b1, b2, b3, and b4, where 0<= b1, b2, b3, b4 <= 255 then the result is equal to (b1 << 24) | (b2 << 16) + (b3 << 8) + b4

	This method blocks until either the four bytes are read, the end of the stream is detected, or an exception is thrown. **Throws :** `EOFException` If this input stream reaches the end before reading two bytes.
readLine()	Reads the next line of text from this data input stream. This method successively reads bytes from the underlying input stream until it reaches the end of a line of text.A line of text is terminated by a carriage return character ('\r'), a newline character ('\n'), a carriage return character immediately followed by a newline character, or the end of the input stream. The line-terminating character(s), if any, are included as part of the string returned. This method blocks until either a newline character is read, a carriage return and the byte following it are read (to see if it is a newline), the end of the stream is detected, or an exception is thrown.
readLong()	Reads a signed 64-bit integer from this data input stream. This method reads eight bytes from the underlying input stream. If the bytes read, in order are b1, b2, b3, b4, b5, b6, b7, and b8, where 0<= b1, b2, b3, b4, b5, b6, b7, b8 <= 255 then the result is equal to ((long)b1 << 56) +((long)b2 << 48)+ ((long)b3 << 40)+ ((long)b4 << 32)+ ((long)b5 << 24) +(b6 << 16)+(b7 << 8) + b8 This method blocks until either the eight bytes are read, the end of the stream is detected, or an exception is thrown.
readShort()	Reads a signed 16-bit number from this data input stream. The method reads two bytes from the underlying input stream. If the two bytes read, in order are b1, b2, where each of the twop values is between 0 and 255, inclusive, then the result is equal to : (short)((b1 << 8) | b2). This method blocks until either the two bytes are read, the end of the stream is detected, or an exception is thrown. **Throws:** `EOFException`, if this input stream reaches the end before reading two bytes.
readUnsignedByte()	Reads an unsigned 8-bit number from this data input stream. This method reads a byte from this data input stream's underlying input stream and returns that byte. This method blocks until either the byte is read, the end of the stream is detected, or an exception is thrown. **Throws:** `EOFException` if this input stream reaches the end before reading two bytes.
readUnsignedShort()	Reads an unsigned 16-bit number from this data input stream. This method reads two bytes from the underlying input stream. If the bytes read are in order, b1, b2, where, 0 <= b1,b2 <= 255, then the result is equal to *(b1 << 8) | b2*). This method blocks until either the two bytes are read, the end of the stream is detected, or an exception is thrown. **Throws:** `EOFException` if this input stream reaches the end before reading two bytes.
readUTF()	Reads in a string which has been encoded using a modified UTF-

	8 format from this data input stream. This method calls readUTF(this). See the following method for a more complete description of the format. This method blocks until either all the bytes are read, the end of the stream is detected, or an exception is thrown.
readUTF(DataInput in)	Reads in a string from the specified data input stream. The string has been encoded using a modified using a modified UTF-8 format. The first two bytes from are read as if by readUnsignedShort. This value gives the number of following bytes that are in the encoded string. (Note : not the length of the resulting string). The following bytes are then interpreted as vbyte encoding characters in the UTF-8 format, and are conveted into characters.This method blocks until all the bytes are read, the end of the stream is detected, or an exception is thrown.1111 **Throws:** `UTFDataFormatException` if the bytes do not represent a valid UTF-8 encoding of a Unicode string.
skipBytes(int n)	Skips exactly n bytes of input in the underlying input stream. this method blocks until either all the bytes are skipped, the end of the stream is detected, or an exception is thrown. **Throws:** `EOFException`, if this input stream reaches the end before skipping all the bytes.

Note : In addition to the respective exception, all the methods in this class throws an Ioexception if an I/O error occurs.

Class DataOutputStream

A data output stream lets an application write primitive Java data types to an output stream in a portable way. An application can then use a data input stream to read the data back in. This class contains various constituents as shown below :

```
public‘‘class’‘java.io.DataOutputStream   extends java.io.FilterOutputStream’‘
          ‘‘implements java.io.DataOutput’‘
          ‘‘ {
                    //        Member elements
                    protected int written;
                    //        The number of bytes written to the data output stream.
                    //        Constructors
                    public DataOutputStream(OutputStream’‘out);
                    //        Creates a new data output stream to write data to the specified underlying
output stream
                    // Methods
                    public void flush();
                    public final int size();
                    public void write(byte’‘b[], int’‘off, int’‘len)’‘‘‘
                    public void write(int’‘b)’‘‘‘
                    public final void writeBoolean(boolean’‘v)’‘‘‘
```

```
public final void writeByte(int‘‘v)’‘‘‘
public final void writeBytes(String‘‘s)’‘‘‘
public final void writeChar(int‘‘v’‘‘‘
public final void writeChars(String‘‘s)’‘‘‘
public final void writeDouble(double‘‘v)’‘‘‘
public final void writeFloat(float‘‘v)’‘‘‘
public final void writeInt(int‘‘v)’‘‘‘
public final void writeLong(long‘‘v)’‘‘‘
public final void writeShort(int‘‘v)’‘‘‘
public final void writeUTF(String‘‘str)
 }
The operations of the different methods are listed in Table 7.6.
```

Table 7.6

Method	Description
flush()	Flushes this data output stream. This forces any buffered output bytes to be written out to the stream. The `flush` method of `DataOuputStream` calls the flush method of its underlying output stream.
size()	Determines the number of bytes written to this data output stream.
write(byte b[], int off, int len)	Writes len bytes from the specified byte array starting at offset off to the underlying output stream.
write(int b)	Writes the specified byte to the underlying output stream.
writeBoolean(boolean v)	Writes the specified boolean value to the underlying output stream.
writeByte(int v)	Writes out a byte to the underlying output stream as a one byte value.
writeBytes(String s)	Writes out the string to the underlying output stream as a sequence of bytes. Each character in the string is written out. in sequence, by discarding its high eight bits.
writeChar(int v)	Writes a char to the underlying output stream as a two byte value, high byte first.
writeChars(String s)	Writes a string to the underlying output stream as a sequence of characters. Each character is written to the data outout stream as if by the `writeChar` method.
writeDouble(double v)	Converts the double argument to a long using the `doubleToLongBits` method in class `double`, and then writes the long value to the undserlying output stream as an eight byte quantity, high byte first.

writeFloat(float v)	Converts the float argument to an int using the `floatToIntBits` method in class float, and then writes the int value to the undserlying output stream as a four byte quantity, high byte first.
writeInt(int v)	Writes an int to the underlying output stream as four bytes, high byte first.
writeLong(long v)	Writes a long to the underlying output stream as eight bytes, high byte first.
writeShort(int v)	Writes a short to the underlying output stream as two bytes, high byte first.
writeUTF(String s)	Writes out a string to the underlying output stream using UTF-8 encoding in a machine independent manner.First, two bytes are written to the output stream as if by the `writeShort` method giving the number of bytes to follow. This value is the the number of bytes actually written out, not the length of the string. Following the length, each character of the string is output, in sequence, using the UTF-8 encoding for each character.13

Note : All the methods in this class returns IOException error, if an IO error occurs.

Class File

Instances of this class represent the name of a file or directory on the host file system. A file is specified by a path name, which can either be an absolute path name or a path name relative to the current working directory. The path name must follow the naming conventions of the host platform. The File class is intended to provide an abstraction that deals with most of the machine- dependent complexities of files and path names in a machine-independent fashion. The class as defined is stated below :

```
public‘‘class’‘java.io.File extends’‘java.lang.Object’‘
          {
                    //        Member elements
                    public final static String pathSeparator;
                    /*        The system-dependent path separator string. This field is initialized to
contain the value of the system property. */
                    public final static char pathSeparatorChar;
                    /*        The system-dependent path separator character. This field is initialized to
contain the first character of the value of the system property. This character is often used to separate
file names in a sequence of files given as a path list.
                    */
                    public final static String separator;
                    /*        The system-dependent path separator string. This field is initialized to
contain the value of the system property.’‘*/
                    public final static char separatorChar;
                    /*        The system-dependent path separator string. This field is initialized to
contain the first character of the value of the system property. This character separates the directory
and fike components in a file name.
                    */
```

```
                    //        Constructors
                    public File(File''dir, String''name);
                    /*        Creates a File instance that represents the file with the specified name in
the specified directory. If the directory argument is null. the resulting File instance represents a file
in
                    the (system-dependent) current directory whose path name is the name argument.
Otherwise, the the file instance represents a file whose path nmae is the path nmae of the directory,
followed by the separator charactor, followed by the name argument.''*/
                    public File(String''path);
                    /*        Creates a File instance that represents the file whose path name is the
given path argument.
                              Throws : NullPointerException, If the file path is equal to null.
                    */''
                    public File(String''path, String''name);
                    /*        Creates a File instance whose path name is the pathname of the specified
directory, followed by the separator character, followed by the name argument. */''''''''
                    //        Methods
                    public boolean canRead();
                    public boolean canWrite();
                    public boolean delete();
                    public boolean equals(Object''obj);
                    public boolean exists();
                    public String getAbsolutePath();
                    public String getName();
                    public String getParent();
                    public String getPath();
                    public int hashCode();
                    public boolean isAbsolute();
                    public boolean isDirectory();
                    public boolean isFile();
                    public long lastModified();
                    public long length();
                    public String[] list();
                    public String[] list(FilenameFilter''filter);
                    public boolean mkdir();
                    public boolean mkdirs();
                    public boolean renameTo(File''dest);
                    public String toString();
          }
```

The description of the methods is listed in Table 7.7.

Table 7.7

Method	Description
canRead()	Determines if the application can read from the specified file. **Returns:** true if the file specified by this object exists and the application can read the file; false otherwise **Throws :** `SecurityException` if a security manager exists, its `checkRead` method is called with the path name of this `File` to see if the application is allowed read access to the file.
canWrite()	Determines if the application can write to this file. **Returns :** true if the application is allowed to write to a file whose name is specified by this object; false otherwise. **Throws :** `SecurityException` if a security manager exists, its `checkWrite` method is called with the path name of this `File` to see if the application is allowed write access to the file.
delete( )	Deletes the file specified by this object. **Returns:** true if the file is successfully deleted; false otherwise. **Throws:** `SecurityException` if a security manager exists, its `checkDelete` method is called with with the path name of this File to see if the application is allowed to delete the file.
equals(Object obj)	The result is true if and only if the argument is not null and is a `File` object whose path name is equal to the path name of this object.
exists()	Determines if this File exists. **Returns:** true if the file specified by this object exists; false otherwise **Throws** `SecurityException` if a security manager exists, its `checkRead` method is called with the path name of this `File` to see if the application is allowed read access to the file.
getAbsolutePath()	If this object represents an absolute path name then return the path name. Otherwise, return a path name that is a concatenation of the current user directory, the separator character, and the path name of this file object. The system property `user.dir` contains the current user directory
getName()	Returns the name of the file represented by this object. The name is everything in the path name after the last occurrence of the separator character.
getParent()	Returns the parent directory of the file represented by this object. The parent directory is everything in the path name before the last occurrence of the separator character or null if the separator character does not appear in the path name.
getPath()	Returns the path name represented by this File object.
hashCode()	Returns a hash code value for this File object.

isAbsolute()	Determines if this File represents an absolute path name. The definition of an absolute path name is system -dependent. **Returns :** true if the path name indicated by the File object is an absolute path name; false otherwise.
isDirectory()	Determines if the file represented by this File object is a directory. **Returns :** true if this File exists and is a directory; false otherwise. **Throws** : `SecurityException` if a security manager exists, its `checkRead` method is called with the path name of this File to see if the application is allowed read access to the file.
isFile()	Determines if the file represented by this File object is a "normal" file i.e. it is not a directory.Any non-directory file created by a Java application is guaranteed to be a normal file. **Returns:** true if the file specified by this object exists and is a "normal" file; false otherwise. **Throws** `SecurityException` if a security manager exists, its `checkRead` method is called with the path name of this File to see if the application is allowed read access to the file.
lastModified()	Determines the time that the file represented by this File object was last modified. It should not be interpreted as an absolute time. It returns 0L if the specified file does not exist. **Throws :** SecurityException if a security manager exists, its `checkRead` method is called with the path name of this File to see if the application is allowed read access to the file.
length()	Determines the length of the file represented by this File object. It returns 0L if the specified file does not exist. **Throws:** `SecurityException` if a security manager exists, its `checkRead` method is called with the path name of this File to see if the application is allowed read access to the file
list()	Lists the files in the directory specified by this File.
list(FilenameFilter filter)	Lists the files in the directory specified by this File that satisfy the specified filter. **Returns:** an array of file names in the specified directory that satisfy the filter. This list does not include the current directory or the parent directory
mkdir()	Creates a directory whose path name is specified by this File object. It returns true if the directory could be created; false otherwise.
mkdirs()	Creates a directory whose path name is specified by this File object. In addition, create all parent directories as necessary. It returns true if the directory (or directories) could be created; false otherwise.

renameTo(File dest)	Renames the file specified by this File object to have the path name given by the File argument.
toString()	Returns a string representation of this object.

Class FileDescriptor

Instances of the file descriptor class serve as an opaque handle to the underlying machine-specific structure representing an open file or an open socket. Applications should not create their own file descriptors. This class is shown below :

```
public‘‘final’‘class’‘java.io.FileDescriptor extends’‘java.lang.Object
           {
          //        Fields
          public final static FileDescriptor err;
          //        A handle to the standard error stream.
          public final static FileDescriptor in;
          //        A handle to the standard input stream
          public final static FileDescriptor out;
          //        A handle to the standard output stream.
          //        Constructors
          public FileDescriptor(         );
          //        The default constructor.


          //        Methods
          public boolean valid(          );
          /*        This method returns true if the file descriptor object represents a valid, open file
or socket; false otherwise.*/
 }
```

Class FileInputStream

This class is to manage a file input stream as an input stream for reading data from a File or from a FileDescriptor.

```
public‘‘class’‘java.io.FileInputStream extends’‘java.io.InputStream’‘
          {
‘‘‘‘‘‘‘‘//          Constructors
‘‘‘‘      public FileInputStream(File’‘file);
                    /*        Creates an input file stream to read from the specified File object. Throws
: FileNotFoundException,if the file is not found.
                    */
```

```
‘‘‘‘      public FileInputStream(FileDescriptor’‘fdObj);

                    /*        Creates an input file stream to read from the specified file descriptor.

‘‘‘‘‘‘‘‘Throws : SecurityException’‘ if a security manager exists, its checkRead method is called with the
file descriptor to see if the application is allowed to read from the specified file descriptor. This may
result in a security exception.

                    */

‘‘‘‘      public FileInputStream(String’‘name);

                    /*        Throws : FileNotFoundException, if the file is not found
and’‘SecurityException,’‘if a security manager exists, its checkRead method’‘ is called with the name
argument t to see if the application is allowed read access to the file. This may result in a security
exception.

                    */

‘‘‘‘‘‘‘‘//          Methods

                    public int available();

‘‘‘‘      public void close();

‘‘‘‘      protected void finalize();

‘‘‘‘      public final FileDescriptor getFD();

‘‘‘‘      public int read();

‘‘‘‘      public int read(byte’‘b[]);

‘‘‘‘      public int read(byte’‘b[], int’‘off, int’‘len);

‘‘‘‘      public long skip(long’‘n);

          }


          Table 7.8 lists the operations performed by various methods in this class.
```

Table 7.8

Method	Description
available()	Returns the number of bytes that can be read from this file input stream without blocking.
close()	Closes this file input stream and releases any system resources associated with the stream.
finalize()	This finalize method ensures that the close method of this file input stream is called when there are no more references to it.
getFD()	Returns the file descriptor object.
read()	Reads a byte of data from this input stream. This method blocks if no input is yet available. It returns -1 if the end of the file is reached.
read(byte b[])	Reads up to b.length bytes of data from this input stream into an array of bytes. This method blocks until some input is available. It returns -1

	is there is no more data because the end of the file has been reached.
read(byte b[], int off, int len)	Reads up to len bytes of data from this input stream into an array of bytes. This method blocks until some input is available.It returns -1 if is there is no more data because the end of the file has been reached.
skip(long n)	Skips over and discards n bytes of data from the input stream. The skip method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly zero. The actual number of bytes skipped is returned.

Class FileOutputStream

A file output stream is an output stream for writing data to a File or to a FileDescriptor.

```
public‘‘class’‘java.io.FileOutputStream extends’‘java.io.OutputStream’‘
          {
                              //        Constructors
                    public FileOutputStream(File’‘file);
                    /*        Creates an file output stream to write to the specified File object. Throws
: IOException, If the file could not be opened for writing. It also throws’‘SecurityException,’‘if a
security manager exists, its checkWrite method is called with the path name’‘ of the File argument to see
if the application is allowed write access to the file. This may result in a security exception.
                    */
                    public FileOutputStream(FileDescriptor’‘fdObj);
                    /*        Creates an output file stream to write to the specified file descriptor.
                    Throws :’‘SecurityException, if a security manager exists, its checkWrite method is
called with the file descriptor to see if the application is allowed to write to the specified file
descriptor.
                    */
                    public FileOutputStream(String’‘name);
                    /*        Creates an output file stream to write to the file with the specified name.
                              Throws :’‘IOException, if the file could not be opened for writing. It also
throws SecurityException, if a security manager exists, its checkWrite method is called with the name
argument to see if the application is allowed write access to the file.
                    */
                              //        Methods
                    public void close();
                    protected void finalize();
                    public final FileDescriptor getFD();
                    public void write(byte’‘b[]);
                    public void write(byte’‘b[], int’‘off, int’‘len);
                    public void write(int’‘b);
          }
```

```
          The methods are being tabulated in Table 7.9.
```

Table 7.9

Method	Descriptor
close()	Closes this file output stream and releases any system resources associated with this stream.
finalize()	This finalize method ensures that the close method of this file output stream is called when there are no more references to this stream.
getFD()	Returns the file descriptor object *associated with this stream*
write(byte b[])	Writes b.length bytes from the specified byte array to this file output stream.
write(byte b[], int off, int len)	Writes `len` bytes from the specified byte array starting at offset off to this file output stream.
write(int b)	Writes the specified amount of byte to this file output stream.

Class FilterInputStream

This class is the superclass of all classes that filter input streams. These streams sit on top of an already existing input stream (the underlying input stream), but provide additional functionality. The class structure is shown as below :

```
‘‘‘‘public’‘class’‘java.io.FilterInputStream extends’‘java.io.InputStream’‘
          {
‘‘‘‘‘‘‘‘//          Member elements
‘‘‘‘      protected InputStream in;
                    //        This indicates the input stream under consideration.


‘‘‘‘‘‘‘‘//          Constructors
‘‘‘‘      protected FilterInputStream(InputStream’‘in);
                    //        Creates an input stream filter built on top of the specified input stream.


‘‘‘‘‘‘‘‘//          Methods
‘‘‘‘      public int available();
‘‘‘‘      public void close();
‘‘‘‘      public void mark(int’‘readlimit);
‘‘‘‘      public boolean markSupported();
‘‘‘‘      public int read();
‘‘‘‘      public int read(byte’‘b[]);
```

```
‘‘‘‘      public int read(byte’‘b[], int’‘off, int’‘len);
‘‘‘‘      public void reset();
‘‘‘‘      public long skip(long’‘n);
          }

          The methods are described in Table 7.10.
```

Table 7.10

Method	Description
available()	Determines the number of bytes that can be read from this input stream without blocking.
close()	Closes this input stream and releases any system resources associated with the stream. The `close` method of `FilterInputStream` calls the `close` method of its underlying input stream.
mark(int readlimit)	Marks the current position in this input stream. A subsequent call to the reset method22 repositions this stream at the last marked position so that subsequent reads re-read the same bytes. The readlimit arguments tells this input stream to allow that many bytes to be read before the mark position gets invalidated.
markSupported()	Determines if this input stream supports the mark and reset methods. The `markSupported` method of `FilterInputStream` calls the `markSupported` method of its underlying input stream and returns whatever value that method returns.
read()	Reads the next byte of data from this stream from this input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned. This method blocks until either input data is available, the end of the stream is detected, or an exception is thrown.
read(byte b[])	Reads up to byte`.length` bytes of data from this input stream into an array of bytes. This method blocks until some input is available.
read(byte b[], int off, int len)	Reads up to `len` bytes of data from this input stream into an array of bytes. This method blocks until some input is available. It returns -1 is there is no more data because the end of the stream has been reached.
reset()	Repositions this stream to the position at the time the mark method was last called on this input stream.
skip(long n)	Skips over and discards n bytes of data from the input stream. The skip method may, for a variety of reasons, end up skipping

	over some smaller number of bytes, possibly zero. The actual number of bytes skipped is returned.

Note : All these method throw IOException, if an I/O error occurs.

Class FilterOutputStream

This class is the superclass of all classes that filter output streams. These streams sit on top of an already existing output stream (the underlying output stream), but provide additional functionality.

The class FilterOutputStream itself simply overrides all methods of OutputStream with versions that pass all requests to the underlying output stream. Subclasses of FilterOutputStream may further override some of these methods as well as provide additional methods and fields. The class structure is as below :

```
public‘‘class‘‘java.io.FilterOutputStream extends‘‘java.io.OutputStream‘‘
          {


‘‘‘‘‘‘‘‘//          Member elements
‘‘‘‘      protected OutputStream out;
                    //                  The‘‘ output stream under consideration.
‘‘‘‘‘‘    //        Constructors
‘‘‘‘      public FilterOutputStream(OutputStream‘‘out);
                    //        Creates an output stream filter built on top of the specified underlying
output stream.


‘‘‘‘‘‘‘‘// Methods
‘‘‘‘      public void close();
‘‘‘‘      public void flush();
‘‘‘‘      public void write(byte‘‘b[]);
‘‘‘‘      public void write(byte‘‘b[], int‘‘off, int‘‘len);
‘‘‘‘      public void write(int‘‘b);
          }
          Table 7.11 describes the methods of‘‘this class :
```

Table 7.11

Method	Description
close()	Closes this output stream and releases any system resources associated with the stream. The `close` method of `FilterOutputStream` calls its flush method and then calls the close method of its underlying output stream.
flush()	Flushes this output stream and forces any buffered output bytes to be written out to the stream. The `flush` method

	of `FilterOutputStream` calls the flush method of it underlying output stream
write(byte b[])	The `write` method of `FilterOutputStream` calls its write method of three arguments with the arguments b, 0, and b.length.
write(byte b[], int off, int len)	Writes `len` bytes from the specified byte array starting at offset `off` to this output stream. Note that this method does not call the `write` method of its underlying input stream with the same arguments. Subclasses of `FilterOutputStream` should provide a more efficient implementation of this method.
write(int b)	Writes the specified byte to this output stream. The write method of `FilterOutputStream` calls the `write` method of its underlying output stream.

Note : All the metohods in this class Throws IOException , if an I/O error occurs.

Class InputStream

This class is an abstract class that is the superclass of all classes representing an input stream of bytes. Applications that need to define a subclass of InputStream must always provide a method that returns the next byte of input.

```
public‘‘abstract’‘class’‘java.io.InputStream extends’‘java.lang.Object’‘
          {
                    //        Constructors
                    public InputStream();
                    //        The default constructor. This constructor is only called by subclasses.

                    //        Methods
                    public int available();
                    public void close();
                    public void mark(int’‘readlimit);
                    public boolean markSupported();
                    public abstract int read();
                    public int read(byte’‘b[]);
                    public int read(byte’‘b[], int’‘off, int’‘len);
                    public void reset();
                    public long skip(long’‘n);
          }
          Table 7.12 lists the methods.
```

Table 7.12

Method	Description

available()	Determines the number of bytes that can be read from this input stream without blocking. The available method of `InputStream` returns 0. This method should be overridden by subclasses. **Returns:** the number of bytes that can be read from this input stream without blocking
close()	Closes this input stream and releases any system resources associated with the stream.
mark(int readlimit)	Marks the current position in this input stream. A subsequent call to the reset method repositions this stream at the last marked position so that subsequent reads re-read the same bytes.
markSupported()	Determines if this input stream supports the mark and reset methods. **Returns:** true if this true type supports the mark and reset method; false otherwise.
read()	Reads the next byte of data from this input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned. This method blocks until either input data is available, the end of the stream is detected, or an exception is thrown. A subclass must provide an implementation of this method. **Returns :** the next byte of data, or -1 if the end of the stream is reached
read(byte b[])	Reads up to b.length bytes of data from this input instream into an array of bytes. The read method of `InputStream` calls the the read method of three arguments with the arguments b, 0, and b.length. **Returns:** the total number of bytes read into the buffer, or -1 is there is no more data because the end of the stream has been reached.
read(byte b[], int off, int len)	Reads up to `len` bytes of data from this input stream into an array of bytes. This method blocks until some input is available. If the first argument is null, up to len bytes are read and discarded. This method of `InputStream` reads a single byte at a time using the read method of zero arguments to fill in the array. **Returns:** the total number of bytes read into the buffer, or -1 is there is no more data because the end of the stream has been reached.
reset()	Repositions this stream to the position at the time the mark method was last called on this input stream
skip(long n)	Skips over and discards n bytes of data from this input stream. The `skip` method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly zero. The actual number of bytes skipped is returned. This method of `InputStream` creates a byte array of length n and then reads into it until n bytes have been read or the end of the stream has been reached.

Class LineNumberInputStream

A line is a sequence of bytes ending with either a carriage return character ('\r'), a newline character ('\n'), or a carriage return character followed immediately by a line feed character. In all three cases the line terminating character(s) are returned as a single newline character.

This class is an input stream filter that provides the added functionality of keeping track of the current line number.The line number begins at zero, and is incremented by 1 when a read returns a newline character.

The class is shown as below :

```
The class is shown as below :
          public‘‘class’‘java.io.LineNumberInputStream extends’‘java.io.FilterInputStream’‘
          {
‘‘‘‘‘‘‘‘ // Constructors
‘‘‘‘      public LineNumberInputStream(InputStream’‘in);
                    /*        Constructs a new line number input stream that reads its input from the
specified input stream. */


‘‘‘‘‘‘‘‘ // Methods
‘‘‘‘      public int available();
‘‘‘‘      public int getLineNumber();
‘‘‘‘      public void mark(int’‘readlimit);
‘‘‘‘      public int read();
‘‘‘‘      public int read(byte’‘b[], int’‘off, int’‘len);
‘‘‘‘      public void reset();
‘‘‘‘      public void setLineNumber(int’‘lineNumber);
‘‘‘‘      public long skip(long’‘n);
          }
          The desription of these metods is given in Table 7.13.
```

Table 7.13

Methods	Description
available()	Determines the number of bytes that can be read from this input stream without blocking. Note that if the underlying input stream 26 is able to supply k input characters without blocking, the LineNumberInputStream can guarantee only to provide characters without blocking, because the k characters from the underlyhing input stream might consist of pairs of '\r' and '\n', which are converted to just '\n' characters.
getLineNumber()	Returns the current line number.

mark(int readlimit)	Marks the current position in this input stream. A subsequent call to the reset method repositions this stream at the last marked position so that subsequent reads re-read the same bytes. This method remembers the current line number in a private variable, and then calls the mark method of the underlying input stream.
read()	Reads the next byte of data from this input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned. This method blocks until either input data is available, the end of the stream is detected, or an exception is thrown. The read method of `LineNumberInputStream` calls the `read` method of the underlying input stream. It checks for carriage returns and newline characters in the input, and modifies the current line number as appropriate. A carriage return character or a carriage return followed by a newline character are both converted into a single newline character.
read(byte b[], int off, int len)	Reads up to len bytes of data from this input stream into an array of bytes. This method blocks until some input is available. **Returns:** the total number of bytes read into the buffer, or -1 is there is no more data because the end of this stream has been reached.
reset()	Repositions this stream to the position at the time the mark method was last called on this input stream.This method resets the line number to be the line number at the time the `mark` method was called, and then calls the `mark` method of the underlying input stream.
setLineNumber(int lineNumber)	Sets the line number to the specified argument.
skip(long n)	Skips over and discards n bytes of data from the input stream. The `skip` method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly zero. The actual number of bytes skipped is returned. This method creates a byte array of length n and then reads into it until n bytes have been read or the end of the stream has been reached. **Returns:** the actual number of bytes skipped.

Note : All the methods in this class Throws IOExvception, if an I/O error occurs.

Class OutputStream

This class is an abstract class that is the superclass of all classes representing an output stream of bytes. Applications that need to define a subclass of OutputStream must always provide at least a method that writes one byte of output. The class is given below :

```
public‘‘abstract’‘class’‘java.io.OutputStream extends’‘java.lang.Object’‘
          {
                                //        Constructors
                    public OutputStream();
                    //        The default constructor.
```

```
                              //        Methods
                    public void close();
                    public void flush();
                    public void write(byte‘‘b[]);
                    public void write(byte‘‘b[], int‘‘off, int‘‘len);
                    public abstract void write(int‘‘b);
          }
          The Table 7.14 is to list the function of these methods.
```

Table 7.14

Method	Description
close()	Closes this output stream and releases any system resources associated with this stream.
flush()	Flushes this output stream and forces any buffered output bytes to be written out.
write(byte b[])	Writes b.length bytes from the specified byte array to this output stream.
write(byte b[], int off, int len)	Writes len bytes from the specified byte array starting at offset off to this output stream.
write(int b)	Writes the specified byte to this output stream.

Class PipedInputStream

A piped input stream is the receiving end a communications pipe. Two threads can communicate by having one thread send data through a piped output stream and having the other thread read the data through a piped input stream.

```
public‘‘class‘‘java.io.PipedInputStream extends‘‘java.io.InputStream‘‘
‘‘‘‘{
‘‘‘‘‘‘‘‘// Constructors
          ‘‘public PipedInputStream();


          /* Creates a piped input stream that is not yet connected to a piped output stream. It must be
connected to a piped output stream, either by the receiver‘‘ or the sender, before being used.
          */
                    public PipedInputStream(PipedOutputStream‘‘src);
                              /*Creates a piped input stream connected to the specified piped output
stream.Throws IOException , If an I/O error occurs. */
                               // Methods
                    public void close();
```

```
                    public void connect(PipedOutputStream‘‘src);
                    public int read();
                    public int read(byte‘‘b[], int‘‘off, int‘‘len);
‘‘‘‘}
The functions description is given in Table 7.15
```

Table 7.15

Method	Description
close()	Closes this piped input stream and releases any system re associated with the stream and close in class `InputStr`
connect(PipedOutputStream src)	Connects this piped input stream to a sender.
read()	Reads the next byte of data from this piped input stream. value byte is returned as an int in the range 0 to 255. If n is available because this end of the stream has been reach value -1 is returned. This method blocks until either inpu available, the end of the stream is detected, or an excepti thrown. It overrides read in class `InputStream` and **throws** IOExeption , if the pip broken. **Returns:** the next byte of data, or -1 if the end of the stre reached.
read(byte b[], int off, int len)	Reads up to len bytes of data from this piped input strean an array of bytes. This method blocks until at least one b input is available. It overrides read in class `InputStre` **Returns :**the total number of bytes read into the buffer, o there is no more data because the end of the stream has b reached. and **throws** an IOException if an I/O error occu

Class PipedOutputStream

A piped output stream is the sending end a communications pipe. Two threads can communicate by having one thread send data through a piped output stream and having the other thread read the data through a piped input stream.

```
public‘‘class‘‘java.io.PipedOutputStream extends‘‘java.io.OutputStream‘‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public PipedOutputStream();
                    /*        Creates a piped output stream that is not yet connected to a piped input
stream. It must be connected to a piped input stream, either by the receiver‘‘ or the sender, before being
used.
                    */
‘‘‘‘      public PipedOutputStream(PipedInputStream‘‘snk);
```

```
                    //        Creates a piped output stream connected to the specified piped input stream
snk.
‘‘‘‘‘‘     //        Methods
‘‘‘‘       public void close();
‘‘‘‘       public void connect(PipedInputStream’‘snk);
‘‘‘‘       public void write(byte’‘b[], int’‘off, int’‘len);
‘‘‘‘       public void write(int’‘b);
           }
           The metods are described in Table 7.16.
```

Table 7.16

Method	Description
close()	Closes this piped output stream and releases any system resources associated with this stream.
connect(PipedInputStream snk)	Connects this piped output stream to a receivier.
write(byte b[], int off, int len)	Writes len bytes from the specified byte array starting at offset off to this piped output stream.
write(int b)	Writes the specified byte to the piped output stream.

Note : All the metods in this class Throws an IOException, if an I/O error occurs.

Class PrintStream

A print stream implements an output stream filter that provides convenient methods for printing types other than bytes and arrays of bytes. In addition, the print stream overrides many of the InputStream methods so as not to throw an IOException. Instead, an I/O exception causes an internal flag to be set, which the application can check by a call to the checkError method.Only the lower 8 bits of any 16-bit quantity are printed to the stream.

An application can specify at creation time whether a print stream should be flushed every time a newline character is written

```
Here are some examples of the use of a print stream:
                    System.out.println("Hello’‘world!");
                    System.out.print("x’‘=‘‘");
                    System.out.println(x);
                    System.out.println("y’‘=‘‘"‘‘+’‘y);
           Following is the various components in it.


           public’‘class’‘java.io.PrintStream extends’‘java.io.FilterOutputStream’‘
           {
```

```
                     //       Constructors
                    public PrintStream(OutputStream’‘out);
                    /*       Constructs a new print stream that writes its output to the specified
underlying output stream. */
                    public PrintStream(OutputStream’‘out, boolean’‘autoflush);
                    /*       Constructs a new print stream that writes its output to the specified
underlying output stream. In addition, if the autoflush flag is true, then the underlying output stream's
flush method is called any time a newline character is printed.
                    */
                              //        Methods
                    public boolean checkError();
                    public void close();
                    public void flush();
                    public void print(boolean’‘b);
                    public void print(char’‘c);
                    public void print(char’‘s[]);
                    public void print(double’‘d);
                    public void print(float’‘f);
                    public void print(int’‘i);
                    public void print(long’‘l);
                    public void print(Object’‘obj);
                    public void print(String’‘s);
                    public void println();
                    public void println(boolean’‘b);
                    public void println(char’‘c);
                    public void println(char’‘s[]);
                    public void println(double’‘d);
                    public void println(float’‘f);
                    public void println(int’‘i);
                    public void println(long’‘l);
                    public void println(Object’‘obj);
                    public void println(String’‘s);
                    public void write(byte’‘b[], int’‘off, int’‘len);
                    public void write(int’‘b);
}
          The methods are summarized in Table 7.17.
```

Table 7.17

Method	Description

checkError()	Flushes this print stream's underlying output stream, and returns a boolean indicating if there has been an error on the underlying output stream. Errors are cumulative; once the print stream has encounted an error, this method will continue to return true on all successive calls.
close()	Closes this print stream and releases any resources associated with the underlying output stream. This method calls the `close` method of its underlying output stream However, if that close method throws an `IOException`, this method catches that exception and indicates, instead, that the underlying stream has gotten an error.
flush()	Flushes this print stream. This forces any buffered output bytes to be written to the underlying stream.
print(boolean b)	Prints the string "true" to the underlying output stream if the value of the boolean argument is true; otherwise, prints the string "false" to the underlying output stream.
print(char c)	Prints the low eight bits of the character argument to this print stream's underlying output stream.
print(char s[])	Prints the low eight bits of each of the characters in the character array to this print stream's underlying output stream.
print(double d)	Prints the string representation of the double to this print stream's underlying output stream. The string representation is identical to the one returned by the toString method of class `Double` with the argument d.
print(float f)	Prints the string representation of the float to this print stream's underlying output stream. The string representation is identical to the one returned by the `toString` method of class `Float` with the argument f.
print(int i)	Prints the string representation of the int to this print stream's underlying output stream. The string representation is identical to the one returned by the `toString` method of class `Integer` with the argument i.
print(long l)	Prints the string representation of the long to this print stream's underlying output stream. The string representation is identical to the one returned by the `toString` method of class `Long` with the argument l.
print(Object obj)	Prints the string representation of the Object to this print stream's underlying output stream. The string representation is identical to the one returned by calling the Object argument's `toString` method.
print(String s)	If the string argument is null, the string "null" is printed to this print stream's underlying output stream. Otherwise, the low eight bits of each of the characters in the string is printed to the underlying output stream.
println()	Prints a newline character to this print stream's underlying output stream.

println(boolean b)	Prints the string "true" followed by a newline character to this print stream's underlying output stream if the value of the boolean argumentis true; otherwise, prints the string "false" followed by a newline character to the underlying output stream.
println(char c)	Prints the low eight bits of the character argument followed by a newline characer to this print stream's underlying output stream.
println(char s[])	Prints the low eight bits of each of the characters in the character array, followed by a newline character, to this print stream's underlying output stream.
println(double d)	Prints the string representation of the double followed by a newline to this print stream's underlying output stream. The string representation is identical to the one returned by the `toString` method of class `Double` with the argument d.
println(float f)	Prints the string representation of the float followed by a newline to this print stream's underlying output stream. The string representation is identical to the one returned by the `toString` method of class `Integer` with the argument f.
println(int i)	Prints the string representation of the int followed by a newline to this print stream's underlying output stream. The string representation is identical to the one returned by the `toString` method of class `Integer` with the argument i.
println(long l)	Prints the string representation of the long followed by a newline to this print stream's underlying output stream. The string representation is identical to the one returned by the `toString` method of class `Long` with the argument l.
println(Object obj)	Prints the string representation of the `Object` followed by a newline to this print stream's underlying output stream. The string representation is identical to the one returned by calling the Object argument's `toString` method.
println(String s)	If the string argument is null, the string "null" followed by a newline character is printed to this print stream's underlying output stream. Otherwise, the low eight bits of each of the characters in the string, followed by a newline character, is printed to the underlying output stream.
write(byte b[], int off, int len)	Writes `len` bytes from the specified byte array starting at offset off to this print stream's underlying output stream.
write(int b)	Writes the specified byte to this print stream. This method calls the write method of its underlying stream. In addition, if the character is a newline character and autoflush is turned on, then the print stream's flush method is called.

Class PushbackInputStream

This class is an input stream filter that provides a one-byte push back buffer. This feature allows an application to "unread" the last character that it read. The next time that a read is performed on the input stream filter, the "unread" character is re-read.

This functionality is useful in situations where it is useful for a fragment of code to read an indefinite number of data bytes that are delimited by particular byte values; after reading the terminating byte, the code fragment can "unread" it, so that the next read operation on the input stream will re-read the byte that was pushed back.

```
public‘‘class‘‘java.io.PushbackInputStream extends‘‘java.io.FilterInputStream‘‘
          {
‘‘‘‘‘‘‘‘ //        Member element
‘‘‘‘      protected int pushBack;
                    /*        A character that has been "unread" and that will be the next byte read. The
value -1 indicates no character in the buffer.
                    */
‘‘‘‘‘‘    //        Constructors
‘‘‘‘      public PushbackInputStream(InputStream‘‘in);
                    /*        Constructs a new pushback input stream that reads its input from the
specified input stream. */
                    //        Methods
‘‘‘‘      public int available();
‘‘‘‘      public boolean markSupported();
‘‘‘‘      public int read();
‘‘‘‘      public int read(byte b[], int off, int len);
‘‘‘‘      public void unread(int‘‘ch);
          }
          The metods are explained in Table 7.18.
```

Table 7.18

Methods	Description
available()	Determines the number of bytes that can be read from this input stream without blocking. The available method of `PushbackInputStream` calls the available method of its underlying input stream ; it returns that value if there is no character that has been pushed back, or that value plus one if there is a character that has been pushed back.
markSupported()	Determines if the input stream supports the `mark` and `reset` methods. **Return :** true if this stream type supports the `mark` and and `reset` methods; false otherwise.
read()	Reads the next byte of data from this input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned. This method blocks until either input data is

	available, the end of the stream is detected, or an exception is thrown.
read(byte b[], int off, int len)	Reads up to `len` bytes of data from this input stream into an array of bytes. This method blocks until at least one byte of input is available.
unread(int ch)	Pushes back a character so that it is read again by the next call to the read method on this input stream.

Class RandomAccessFile

This is the one important class known for I/O manipulation. Instances of this class support both reading and writing to a random access file. An application can modify the position in the file at which the next read or write occurs. The structure of this class is as shown below :

```
public‘‘class’‘java.io.RandomAccessFile extends’‘java.lang.Object’‘

          implements java.io.DataOutput’‘java.io.DataInput’‘

          {

                              //        Constructors

                    public RandomAccessFile(File’‘file, String’‘mode);

                    /*        Creates a random access file stream to read from, and optionally to write
to, the file specified by the File argument. The mode argument must either be equal to "r" or to "rw",
indicating either to open the file for input, or for both input and output, respectively.

                    */

                    public RandomAccessFile(String’‘name, String’‘mode);

                    /*        Creates an random access file stream to read from, and optionally to write
to, a file with the specified name. The mode argument must either be equal to "r" or "rw", indicating
either to open the file for input or for both input and output. */

                    /*        Constructors in this class may throw the exceptions as indicated below :

                              Throws : IOException, If an I/O error occurs.

                              Throws IllegalArgumentException, f the mode argument is not equal to "r" or
to "rw".’‘

                              Throws : SecurityException, If a security manager exists, its checkRead
method’‘ is called with the path name of the File argument to see if the application is allowed read
access to the file. If the mode argument is equal to "rw", its checkWrite method’‘ also is called with the
path name to see if the application is allowed write access to the file.

                    */

                    //        Methods

                    public void close();

                    public final FileDescriptor getFD();

                    public long getFilePointer();

                    public long length();

                    public int read();

                    public int read(byte’‘b[]);

                    public int read(byte’‘b[], int’‘off, int’‘len);

                    public final boolean readBoolean();

                    public final byte readByte();
```

```
                public final char readChar();
                public final double readDouble();
                public final float readFloat();
                public final void readFully(byte'‘b[]);
                public final void readFully(byte'‘b[], int'‘off, int'‘len);
                public final int readInt();
                public final String readLine();
                public final long readLong();
                public final short readShort();
                public final int readUnsignedByte();
                public final int readUnsignedShort();
                public final String readUTF();
                public void seek(long'‘pos);
                public int skipBytes(int'‘n);
                public void write(byte'‘b[]);
                public void write(byte'‘b[], int'‘off, int'‘len);
                public void write(int'‘i);
                public final void writeBoolean(boolean'‘v);
                public final void writeByte(int'‘v);
                public final void writeBytes(String'‘s);
                public final void writeChar(int'‘v);
                public final void writeChars(String'‘s);
                public final void writeDouble(double'‘v);
                public final void writeFloat(float'‘v);
                public final void writeInt(int'‘v);
                public final void writeLong(long'‘v);
                public final void writeShort(int'‘v);
                public final void writeUTF(String'‘str);
        }
        The function of various methods in this class is stated in Table 7.19.
```

Table 7.19

Method	Description
close()	Closes this random access file stream and releases any system resources associated with the stream.
getFD()	Returns the file descriptor object associated with this stream.
getFilePointer()	Returns the offset from the beginning of the file, in bytes, at which the next read or write occurs.
length()	Returns the length of this file.

read()	Reads a byte of data from this file. This method blocks if no input is yet available , or -1 if the end of the file is reached.
read(byte b[])	Reads up to b.length bytes of data from this file into an array of bytes. This method blocks until at least one byte of input is available. Returns -1 if there is no more data because the end of this file has been reached.
read(byte b[], int off, int len)	Reads up to len bytes of data from this file into an array of bytes. This method blocks until at least one byte of input is available. Returns -1 if there is no more data because the end of the file has been reached.
readBoolean()	Reads a boolean from this file. This method reads a single byte from the file. A value of 0 represents false. Any other value represents true. This method blocks until either the byte is read, the end of the stream is detected, or an exception is thrown.37
readByte()	Reads a signed 8-bit value from this file. This method reads a byte from the file. If the byte read is b, where 0<= b<=255, then the result is (byte)(b).
readChar()	Reads a Unicode character from this file. This method reads two bytes from the file. If the bytes read, in order, are b1 and b2, where 0<= b1, b2 <= 255, the result is equal to (char)((b1 << 8) | b2).
readDouble()	Reads a double from this file. This method reads a long value as if by the `readLong` method and then converts that long to a double using the `longBitsToDouble` method in class `Double`.
readFloat()	Reads a float from this file. This method reads an int value as if by the `readInt` method and then converts that int to a float using the `intBitsToFloat` method in class `Float`
readFully(byte b[])	Reads b.length bytes from this file into the byte array. This method reads repeatedly from the file until all the bytes are read.
readFully(byte b[], int off, int len)	Reads exactly `len` bytes from this file into the byte array. This method reads repeatedly from the file until all the bytes are read.
readInt()	Reads a signed 32-bit integer from this file. This method reads four bytes from the file. If the bytes read, in order, are b1, b2, b3, and b4 where 0 <= b1, b2, b3, b4 <= 255, then the result is equal to (b1 << 24) | (b2 << 16) + (b3 << 8) + b4.
readLine()	Reads the next line of text from this file. This method successively reads bytes from the file until it reaches the end of a line of text. A line of text is terminated by a carriage return character ('\r'), a newline character ('\n'), a carriage return character immediately followed by a newline character, or the end of the input stream. The line-terminating character(s), if any, are included as part of the string returned.
readLong()	Reads a signed 64-bit integer from this file. This method reads eight bytes from the file. If the bytes read, in order, are b1, b2, b3, b4, b5, b6, b7, and b8, where 0<= b1, b2, b3, b4, b5, b6, b7

	<= 0, then the result is equal to ((long)b1 << 56) + ((long)b2 << 48) + ((long)b3 << 40) + ((long)b4 << 32) + ((long)b5 << 24) + ((long)b6 << 16) + ((long)b7 << 8) + b8)
readShort()	Reads a signed 16-bit number from this file. The method reads two bytes from this file. If the two bytes read, in order, are b1 and b2, where each of the two values is between 0 and 255, inclusive, then the result is equal to (short)((b1 << 8) | b2)
readUnsignedByte()	Reads an unsigned 8-bit number from this filr. This method reads a byte from this file and returns that byte.
readUnsignedShort()	Reads an unsigned 16-bit number from this file. This method reads two bytes from the file. If the bytes read, in order, are b1 and b2, where 0 <= b1, b2 <= 255, then the result is equal to (b1 << 8) | b2)
readUTF()	Reads in a string from this file. The string has been encoded using a modified using a modified UTF-8 format.
seek(long pos)	Sets the offset from the beginning of this file at which the next read or write occurs.
skipBytes(int n)	Skips exactly n bytes of input.
write(byte b[])	Writes b.length bytes from the specified byte array starting at offset off to this file.
write(byte b[], int off, int len)	Writes `len` bytes from the specified byte array starting at offset off to this file.
write(int i)	Writes the specified byte to this file.
writeBoolean(boolean v)	Writes a boolean to the file as a one-byte value. The value true is written out as the value (byte)1; the value false is written out as the value (byte)0.
writeByte(int v)	Writes out a byte to the file as a one-byte value.
writeBytes(String s)	Writes out the string to the file as a sequence of bytes. Each character in the string is written out, in sequence, by discarding its high eight bits.
writeChar(int v)	Writes a char to the file as a two-byte value, high byte first.
writeChars(String s)	Writes a string to the file as a sequence of characters. Each character is written to the data output stream as if by the `writeChar` method.
writeDouble(double v)	Converts the double argument to a long using the `doubleToLongBits` method in class `Double`, and then writes that long value to the file as an eight-byte quantity, high-byte first.
writeFloat(float v)	Converts the float argument to an int using the `floatToIntBits` method in class Float, and then writes that int value to the file as a four- byte quantity, high-byte first.
writeInt(int v)	Writes an int to the file as four bytes, high-byte first.

writeLong(long v)	Writes a long to the file as eight bytes, high-byte first.
writeShort(int v)	Writes a short to the file as two bytes, high-byte first.
writeUTF(String str)	Writes out a string to the file using UTF-8 encoding in a machine-independent manner. First, two bytes are written to the file as if by the `writeShort` method giving the number of bytes to follow. This value is the number of bytes actually written out, not the length of the string. Following the length, each character of the string is output, in sequence, using the UTF-8 encoding for each character.

```
Note :    1) All the read methods block until either the byte is read, the end of the stream is detected, or an exception is thrown.''

          ''''2) All the read methods throws an EOFException, if an end of file encounters.

          ''''3) All the methods in this class throws IOException if I/O errors occur.
```

Class SequenceInputStream

The sequence input stream class allows an application to combine several input streams serially and make them appear as if they were a single input stream. Each input stream is read from, in turn, until it reaches the end of the stream. The sequence input stream class then closes that stream and automatically switches to the next input stream.

```
          public''class''java.io.SequenceInputStream extends''java.io.InputStream''
                    {


                              //        Constructors
                              public SequenceInputStream(Enumeration''e);
                              /*        Constructs a new sequence input stream initialized to the specified enumeration''of input streams. Each object in the enumeration must be an InputStream.
                              */
                              public SequenceInputStream(InputStream''s1, InputStream''s2);
                              /*        Constructs a new sequence input stream initialized to read first from the input stream s1, and then from the input stream s2.
                                        */
                                        //        Methods
                              public void close();
                              public int read();
                              public int read(byte''buf[], int''pos, int''len);
                    }
                    Methods are listed in Table 7.20.
```

Table 7.20

Method	Description
close()	Closes this input stream and releases any system resources associated with the stream.
read()	Reads the next byte of data from this input stream. The byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned. This method blocks until either input data is available, the end of the stream is detected, or an exception is thrown.
read(byte buf[], int pos, int len)	Reads up to len bytes of data from this input stream into an array of bytes. This method blocks until at least one byte of input is available. If the first argument is null, up to len bytes are read and discarded.

Class StreamTokenizer

The StreamTokenizer class takes an input stream and parses it into "tokens", allowing the tokens to be read one at a time. The parsing process is controlled by a table and a number of flags that can be set to various states. The stream tokenizer can recognize identifiers, numbers, quoted strings, and various comment styles.

Each byte read from the input stream is regarded as a character in the range '\u0000' through '\u00FF'. The character value is used to look up five possible attributes of the character: whitespace, alphabetic, numeric, tring quote, and comment character. Each character can have zero or more of these attributes. In addition an instance has four flags. These flags indicate:

- ’‘Whether line terminators are to be returned as tokens or treated as whitespace.
- ’‘Whether C-style comments are to be recognized and skipped.
- ’‘Whether C++-style comments are to be recognized and skipped.
- ’‘Whether the characters of identifiers are converted to lowercase.

A typical application first constructs an instance of this class, sets up the syntax tables, and then repeatedly loops calling the nextToken method in each iteration of the loop until it returns the value TT_EOF. Let us see the structure of this class.

```
public’‘class’‘java.io.StreamTokenizer extends’‘java.lang.Object’‘
          {
                    //        Member elements
                    public double nval;
                    /*        If the current token is a number, this field contains the value of that
number. The current token is a number when the value of the ttype field’‘is TT_NUMBER.
                    */
                    public String sval;
                              /*        If the current token is a word token, this field contains a
string giving the characters of the word token. When the current token is a quoted string token, this
field contains the body of the string. The current token is a word when when the value of the ttype
field’‘ is TT_WORD.
                              */
```

```
                    public int ttype;

                                /*        After a call to the nextToken method’‘, this field contains the
type of the token just read. For a single character token, its value is the single character, converted to
an integer. For a quoted string token its value is the quote character. Otherwise, its value is one of the
following:

                                          TT_WORD’‘ indicates that the token is a word.

                                          TT_NUMBER’‘indicates that the token is a number.

                                          TT_EOL’‘ indicates that the end of line has been read. The field
can only have                             this value if the eolIsSignficant method’‘ has been called with
the argument true.

                                          TT_EOF’‘indicates that the end of the input stream has been
reached.

                                */

                    //        Constructors

                    public StreamTokenizer(InputStream’‘i);

                    /*        Creates a stream tokenizer that parses the specified input stream. The
stream tokenizer is initialized to the following default state:

                               All byte values 'A' through 'Z', 'a' through 'z', and '\u00A0' through
'\u00FF' are considered to be alphabetic.

                               All byte values '\u0000' through '\u0020' are considered to be whitespace.

                               '/' is a comment character.

                              Single quote '\'' and double quote '"' are string quote characters.

                              Numbers are parsed .

                              End of lines are not treated as whitespace, not as separate tokens.

                              C-style and C++-style comments are not recognized.

                    */

                              //        Methods

                    public void commentChar(int’‘ch);

                    public void eolIsSignificant(boolean’‘flag);

                    public int lineno();

                    public void lowerCaseMode(boolean’‘fl);

                    public int nextToken();

                    public void ordinaryChar(int’‘ch);

                    public void ordinaryChars(int’‘low, int’‘hi);

                    public void parseNumbers();

                    public void pushBack();

                    public void quoteChar(int’‘ch);

                    public void resetSyntax();

                    public void slashStarComments(boolean’‘flag);

                    public String toString();

                    public void whitespaceChars(int’‘low, int’‘hi);

                    public void wordChars(int’‘low, int’‘hi);

}
```

All these methods are listed in Table 7.21.

Table 7.21

Method	Description
commentChar(int ch)	Specified that the character argument starts a single line comment. All characters from the comment character to the end of the line are ignored by this stream tokenizer.
eolIsSignificant(boolean flag)	If the flag argument is true, this tokenizer treats end of lines as tokens; the nextToken method returns TT_EOL and also sets the ttype field to this value when an end of line is read. If the flag is false, end of line characters are treated as whitespace and serve only to separate tokens.
lineno()	Returns the current line number of this stream tokenizer.
lowerCaseMode(boolean fl)	If the flag argument is true, then the value in the sval field is lowercased whenever a word token is returned (the ttype field has the value TT_WORD) by the nextToken method of this tokenizer. If the flag argument is false, then the sval field is not modified.
nextToken()	Parses the next token from the input stream of this tokenizer. The type of the next token is returned in the `ttype` field. Additional information about the token may be in the `nval` field or the `sval` field of this tokenizer.
ordinaryChar(int ch)	Specifies that the character argument is "ordinary" in this tokenizer. It removes any special significance the character has as a comment character, word component, string delimiter, whitespace, or number character. When such a character is encountered by the parser, the parser treates it as a single-character token and sets ttype field to the character value.
ordinaryChars(int low, int hi)	Specifies that all characters *c* in the range are "ordinary" in this tokenizer.
parseNumbers()	Specifies that numbers should be parsed by this tokenizer. The syntax table of this tokenizer is modified so that each of the twelve characters : 0 1 2 3 4 5 6 7 8 9 . - has the "numeric" attribute. When the parser encounters a word token that has the format of a double precision floating point number, it treates the token as a number rather than a word, by setting the the `ttype` field to the value TT_NUMBER and putting the numeric value of the token into the `nval` field.
pushBack()	Causes the next call to the `nextToken` method of this tokenizer to return the current value in the ttype field 4242 , and not to modify the value in the `nval` or `sval` field.
quoteChar(int ch)	Specifies that matching pairs of this character delimit string constants in this tokenizer.
resetSyntax()	Resets this tokenizer's syntax table so that all characters are

	"ordinary."
slashStarComments(boolean flag)	If the flag argument is true, this stream tokenizer recognizes C++ style comments. Any occurrence of two consecutive slash characters ('/') is treated as the beginning of a comment that extends to the end of the line. If the flag argument is false, then C++ style comments are not treated specially. If the flag argument is true, this stream tokenizer recognizes C style comments. All text between successive occurrences of /* and */ are discarded. If the flag argument is false, then C style comments are not treated specially.
toString()	Returns the string representation of the current stream token.
whitespaceChars(int low, int hi)	Specifies that all characters c *in the range are whitespace character. Whitepsace characters serve only to separate tokens in the input stream.*
wordChars(int low, int hi)	Specifies that all characters *c* in the range are word constituents. A word token consists of a word constitutent followed by zero or more word constituents or number constituents.

Class StringBufferInputStream

This class allows an application to create an input stream in which the bytes read are supplied by the contents of a string. Applications can also read bytes from a byte array by using a ByteArrayInputStream. Only the low eight bits of each character in the string are used by this class.

```
public‘‘class’‘java.io.StringBufferInputStream extends’‘java.io.InputStream’‘
          {
                    //        Member elements
                    protected String buffer;
                    //        The string from which bytes are read.
                    protected int count;
                    //        The number of valid characters in the input stream buffer.
                    protected int pos;
                    //        The index of the next character to read from the input stream buffer.
                              //        Constructors
                    public StringBufferInputStream(String’‘s);
                    //        Creates a string input stream to read data from the specified string.
                              //        Methods
                    public int available();
                    public int read();
                    public int read(byte’‘b[], int’‘off, int’‘len);
                    public void reset();
                    public long skip(long’‘n);
```

```
        }
        Methods are described in Table 7.22.
```

Table 7.22

Method	Description
available()	Determines the number of bytes that can be read from the input stream without blocking.
read()	Reads the next byte of data from this input stream. The value byte is returned as an int in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned.
read(byte b[], int off, int len)	Reads up to `len` bytes of data from this input stream into an array of bytes or returns -1 is there is no more data because the end of the stream has been reached..
reset()	Resets the input stream to begin reading from the first character of this input stream's underlying buffer.
skip(long n)	Skips n bytes of input from this input stream. Fewer bytes might be skipped if the end of the input stream is reached.

Interface DataInput

The data input interface is implemented by streams that can read primitive Java data types from a stream in a machine-independent manner. The constitutents in this interface is listed below :

```
public‘‘interface’‘java.io.DataInput
        {
‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract boolean readBoolean();
‘‘‘‘      public abstract byte readByte();
‘‘‘‘      public abstract char readChar();
‘‘‘‘      public abstract double readDouble();
‘‘‘‘      public abstract float readFloat();
‘‘‘‘      public abstract void readFully(byte’‘b[]);
‘‘‘‘      public abstract void readFully(byte’‘b[], int’‘off, int’‘len);
‘‘‘‘      public abstract int readInt();
‘‘‘‘      public abstract String readLine();
‘‘‘‘      public abstract long readLong();
‘‘‘‘      public abstract short readShort();
‘‘‘‘      public abstract int readUnsignedByte();
‘‘‘‘      public abstract int readUnsignedShort();
‘‘‘‘      public abstract String readUTF();
```

```
‘‘‘‘      public abstract int skipBytes(int’‘n);
          }
          The methods are described in Table 7.23.
```

Table 7.23

Method	Description
readBoolean()	Reads a boolean value from the input stream.
readByte()	Reads a signed eight-bit value from the input stream.
readChar()	Reads a Unicode char value from the input stream.
readDouble()	Reads a double value from the input stream.
readFloat()	Reads a float value from the input stream, high byte.
readFully(byte b[])	Reads b.length bytes into the byte array. This method blocks until all the bytes are read.
readFully(byte b[], int off, int len)	Reads b.len bytes into the byte array. This method blocks until all the bytes are read.
readInt()	Reads an int value from the input stream.
readLine()	Reads the next line of text from the input stream.
readLong()	Reads a long value from the input stream.
readShort()	Reads a 16-bit value from the input stream.
readUnsignedByte()	Reads an unsigned eight-bit value from the input stream.
readUnsignedShort()	Reads an unsigned 16-bit value from the input stream.
readUTF()	Reads in a string that has been encoded using a modified UTF-8 format.
skipBytes(int n)	Skips exactly n bytes of input.

Note : All these methods Throws EOFException, if this stream reaches the end before skipping all the bytes. Throws IOException, if an I/O error occurs.

Interface DataOutput

The data output interface is implemented by streams that can write primitive Java data types to an output stream in a machine-independent manner. The interface is as shown below :

```
public’‘interface’‘java.io.DataOutput
```

```
          {
‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract void write(byte’‘b[]);
‘‘‘‘      public abstract void write(byte’‘b[], int’‘off, int’‘len);
‘‘‘‘      public abstract void write(int’‘i);
‘‘‘‘      public abstract void writeBoolean(boolean’‘v);
‘‘‘‘      public abstract void writeByte(int’‘v);
‘‘‘‘      public abstract void writeBytes(String’‘s);
‘‘‘‘      public abstract void writeChar(int’‘v);
‘‘‘‘      public abstract void writeChars(String’‘s);
‘‘‘‘      public abstract void writeDouble(double’‘v);
‘‘‘‘      public abstract void writeFloat(float’‘v);
‘‘‘‘      public abstract void writeInt(int’‘v);
‘‘‘‘      public abstract void writeLong(long’‘v);
‘‘‘‘      public abstract void writeShort(int’‘v);
‘‘‘‘      public abstract void writeUTF(String’‘str);
          }
          The methods are described in Table 7.24.
```

Table 7.24

Method	Description
write(byte b[])	Writes b.length bytes from the specified byte array to this output stream.
write(byte b[], int off, int len)	Writes `len` bytes from the specified byte array starting at offset off to this output stream
write(int i)	Writes the specified byte to this data output stream.
writeBoolean(boolean v)	Writes a boolean value to this output stream.
writeByte(int v)	Writes an 8-bit value to this output stream.
writeBytes(String s)	Writes out the string to this output stream as a sequence of bytes.
writeChar(int v)	Writes a char value of v to this output stream.
writeChars(String s)	Writes a string to this output stream as a sequence of characters.
writeDouble(double v)	Writes a double value to this output stream.
writeFloat(float v)	Writes a float value to this output stream.
writeInt(int v)	Writes an int value to this output stream.

writeLong(long v)	Writes a long value to this output stream
writeShort(int v)	Writes a 16-bit value to this output stream
writeUTF(String str)	Writes out a Unicode string that by encoded it using modified UTF-8 format.

Note : All these methods Throws IOException, If an I/O error occurs.

Interface FilenameFilter

Instances of classes that implement this interface are used to filter filenames. These instances are used to filter directory listings in the list method of class File and by the Abstract Window Toolkit's file dialog component.

```
public‘‘interface’‘java.io.FilenameFilter

          {

‘‘‘‘‘‘‘‘  //        Methods

‘‘‘‘       public abstract boolean accept(File’‘dir, String’‘name);

                    /*        Determines whether a specified file should be included in a file list.
Returns: true if name should be included in file list; false otherwise.

                    */

          }
```

Class EOFException

Signals that an end-of-file has been reached unexpectedly during input. This exception is mainly used by data input streams, which generally expect a binary file in a specific format, and for which an end-of-stream is an unusual condition. Most other input streams return a special value on end of stream.

```
public‘‘class’‘java.io.EOFException extends’‘java.io.IOException’‘

          {

‘‘‘‘‘‘‘‘  //        Constructors

‘‘‘‘       public EOFException();

                    //        Constructs an EOFException with no detail message.

‘‘‘‘       public EOFException(String’‘s);

                    //        Constructs an EOFException with the specified detail message.

‘‘‘‘}
```

Class FileNotFoundException

Signals that a file could not be found.

```
public‘‘class’‘java.io.FileNotFoundException extends’‘java.io.IOException’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public FileNotFoundException();
                    //        Constructs a FileNotFoundException with no detail message.
‘‘‘‘      public FileNotFoundException(String’‘s);
                    //        Constructs a FileNotFoundException with the specified detail message.
          }
```

Class IOException

Signals that an I/O exception of some sort has occurred.

```
public’‘class’‘java.io.IOException extends’‘java.lang.Exception’‘
          {
                              //        Constructors
                    public IOException();
                    //        Constructs an IOException with no detail message.
                    public IOException(String’‘s);
                    //        Constructs an IOException with the specified detail message.
          }
```

Class InterruptedIOException

Signals that an I/O operation has been interrupted.

```
public’‘class’‘java.io.InterruptedIOException extends’‘java.io.IOException’‘
          {
‘‘‘‘‘‘‘‘ //        Member elements
‘‘‘‘      public int bytesTransferred;
                    /*        Reports how many bytes had been transferred as part of the IO operation
before it was interrupted. */

‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public InterruptedIOException();
                    //        Constructs an InterruptedIOException with no detail message.
‘‘‘‘      public InterruptedIOException(String’‘s);
                    //        Constructs an InterruptedIOException with the specified detail message.
          }
```

Class UTFDataFormatException

Signals that a malformed UTF-8 string has been read in a data input stream or by any class that implements the data input interface

```
public‘‘class’‘java.io.UTFDataFormatException extends’‘java.io.IOException
          {
‘‘‘‘‘‘‘‘ //        Constructors
                    public UTFDataFormatException();
                              //        Constructs an UTFDataFormatException with no detail message.
                    public UTFDataFormatException(String’‘s);
                              //        Constructs an UTFDataFormatException with the specified detail
message.
          }
```

Chapter 19

The Java Networking Package

by Debasis Samanta

CONTENTS

- Introduction
- Class ContentHandler
- Class DatagramPacket
- Class DatagramSocket
- Class InetAddress
- Class ServerSocket
- Class Socket
- Class SocketImpl
- Class URL
- Class URLConnection
- Class URLEncoder
- Class URLStreamHandler
- Throwing Exceptions
- Interface ContentHandlerFactory
- Interface SocketImplFactory
- Interface URLStreamHandlerFactory
- Class MalformedURLException
- Class ProtocolException
- Class SocketException

- Class UnknownHostException
- Class UnknownServiceException

Introduction

The java.net package contains classes and interfaces, and related exception that provides a URL and a URL connection, TCP, UDP, IP, and support socket connections. It also includes classes that represents an Internetaddress, and binary-to- text converter. Following is the diffetent constituent in this package :

```
          Classes
                    Class ContentHandler’‘
                    Class DatagramPacket’‘
                    Class DatagramSocket’‘
                    Class InetAddress’‘
                    Class ServerSocket’‘
                    Class Socket’‘
                    Class SocketImpl’‘
                    Class URL’‘
                    Class URLConnection’‘
                    Class URLEncoder’‘
                    Class URLStreamHandler’‘
          Interfaces
                    Interface ContentHandlerFactory’‘
                    Interface SocketImplFactory’‘
                    Interface URLStreamHandlerFactory’‘
          Exceptions
                    Class MalformedURLException’‘
                    Class ProtocolException’‘
                    Class SocketException’‘
                    Class UnknownHostException’‘
                    Class UnknownServiceException’‘
‘‘‘‘
```

Class ContentHandler

The abstract class ContentHandler is the superclass of all classes that read an Object from a URL Connection. An application does not generally call the getContentHandler method in this class directly. Instead, an application calls the getContent method in class URL or in URLConnection. The application's content handler factory (an instance of a class that implements the interface ContentHandlerFactory set up by a call to setContentHandler and is called with a String giving the MIME type of the object being received on the socket. The factory returns an instance of a subclass of ContentHandler, and its getContent method is called to create the object.

```
This class has the following structure :
```

```
          public‘‘abstract’‘class’‘java.net.ContentHandler extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public ContentHandler();
                    //        The default constructor for class ContentHandler.


‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract Object getContent(URLConnection’‘urlc);
                    /*        Given an URL connect stream positioned at the beginning of the
representation of an object, this method reads that stream and creates an object from it. Returns : The
object read by the ContentHandler.’‘Throws : IOException,’‘if an IO error occurs while reading the object.
                    */
          }
```

Class DatagramPacket

This class represents a datagram packet. Datagram packets are used to implement a connectionless packet delivery service. Each message is routed from one machine to another based solely on information contained within that packet. Multiple packets sent from a machine to another might be routed differently, and might arrive in any order.

```
The structure of this class is shown below :


          public‘‘final’‘class’‘java.net.DatagramPacket extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public DatagramPacket(byte’‘ibuf[], int’‘ilength);
                    /*        Constructs a DatagramPacket for receiving packets of length ilength. The
length argument must be less than or equal to ibuf.length.
                    */
‘‘‘‘      public DatagramPacket(byte’‘ibuf[], int’‘ilength,   InetAddress’‘iaddr,int iport);
                    /*        Constructs a DatagramPacket for sending packets of length ilength to the
specified port number on the specified host. The length argument must be less than or equal to
ibuf.length.
                    */
‘‘‘‘‘‘    //        Methods
‘‘‘‘      public InetAddress getAddress();
‘‘‘‘      public byte[] getData();
‘‘‘‘      public int getLength();
‘‘‘‘      public int getPort();
          }


          The operation performed by the various methods are listed in Table 8.1.
```

Table 8.1

Method	Description
getAddress()	**Returns :** the IP address *of the machine to which this datagram is being sent, or from which the datagram was received.*
getData()	**Returns :** the data received, or the data to be sent.
getLength()	**Returns :** the length of the data to be sent, or the length of the data received.
getPort()	**Returns :** the port number on the remote host to which this datagram is being sent, or from which the dagram was received

Class DatagramSocket

This class represents a socket for sending and receiving datagram packets. A datagram socket is the sending or receiving point for a connectionless packet delivery service. Each packet sent or received on a datagram socket is individually addressed and routed. Multiple packets sent from a machine to another may be routed differently, and may arrive in any order.

```
public‘‘class’‘java.net.DatagramSocket extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public DatagramSocket();
                    /*        Constructs a datagram socket and binds it to any available port on the
local host machine.
                              Throws : SocketException,’‘if the socket could not be opened, or the socket
could not bind the specified local port.
                    */
‘‘‘‘      public DatagramSocket(int’‘port);
                    /*        Constructs a datagram socket and binds it to the specified port on the
local host                                        machine.
                              Throws : SocketException,’‘if the socket could not be opened, or the socket
could not bind the specified local port.
                    */
          ‘‘‘‘      //        Methods
‘‘‘‘      public void close();
‘‘‘‘      protected void finalize();
‘‘‘‘      public int getLocalPort();
‘‘‘‘      public void receive(DatagramPacket’‘p);
‘‘‘‘      public void send(DatagramPacket’‘p);
          }
          Methods are described in Table 8.2.
```

Table 8.2

Method	Description
close()	Closes this datagram socket.

finalize()	Ensures that this socket is closed if there are no longer any references to this socket.
getLocalPort()	**Returns :** the port number on the local host to which this socket is bound.
receive(DatagramPacket p)	Receives a datagram packet from this socket. When this method returns, the DatagramPacket's buffer is filled with the data received. The datagram packet also contains the sender's IP address, and the port number on the sender's machine. This method blocks until a datagram is received. The length field of the datagram packet object contains the length of the received message. If the message is longer than the buffer length, the message is truncated. **Throws :** `IOException`, if an I/O error occurs.
send(DatagramPacket p)	Sends a datagram packet from this socket. The DatagramPacket includes information indicating the data to be sent, its length, the IP address of the remote host, and the port number on the remote host. **Throws :** `IOException`, if an I/O error occurs.

Class InetAddress

This class represents an Internet Protocol (IP) address. Applications should use the methods getLocalHost, getByName , or getAllByName to create a new InetAddress instance. The class structure is shown as below :

```
public‘‘final’‘class’‘java.net.InetAddress extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public boolean equals(Object’‘obj);
‘‘‘‘      public byte[] getAddress();
‘‘‘‘      public static InetAddress[] getAllByName(String’‘host);
‘‘‘‘      public static InetAddress getByName(String’‘host);
‘‘‘‘      public String getHostName();
‘‘‘‘      public static InetAddress getLocalHost();
‘‘‘‘      public int hashCode();
‘‘‘‘      public String toString();
          }


          Methods are described in Table 8.3.
```

Table 8.3

Method	Description
equals(Object obj)	The result is true if and only if the argument is not null and

	it represents the same IP address as this object.
getAddress()	Determines the raw IP address of this `InetAddress` object. The result is in network byte order : the highest order byte of the address is in `getAddress()`.
getAllByName(String host)	**Returns :** an array of all the IP addresses for a given host name. **Throws :** `UnknownHostException`, if no IP address for the host could be found.
getByName(String host)	Determines the IP address of a host, given the host's name. The host name can either be a machine name, such as "java.sun.com" or a string representing its IP address, such as "198.87.26.123".
getHostName()	**Throws :** `UnknownHostException`, if no IP address for the host could be found.193193
getLocalHost()	**Returns :** the IP address of the local host. **Throws :** `UnknownHostException` , if no IP address for the host could be found.
hashCode()	**Returns :** a hash code value for this IP address.
toString()	**Returns :** a string representation of this IP address.

Class ServerSocket

This class implements server sockets. A server socket waits for requests to come in over the network. It performs some operation based on that request, and then possibly returns a result to the requester.

The actual work of the server socket is performed by an instance of the SocketImpl class. An application can change the socket factory that creates the socket implementation to configure itself to create sockets appropriate to the local firewall

```
public‘‘final’‘class’‘java.net.ServerSocket extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public ServerSocket(int’‘port);
                    /*        Creates a server socket on a specified port. A port of 0 creates a socket
on any free port.
                              The maximum queue length for incoming connection indications (a request to
connect) is set to 50. If a connection indication arrives when the queue is full, the connection is
refused.
                              If the application has specified a server socket factory, that factory's
createSocketImpl method’‘is called to create the actual socket implementation. Otherwise a "plain"
socket’‘ is created.
                              Throws :’‘IOException, if an IO error occurs when opening the socket.
                    */
‘‘‘‘      public ServerSocket(int’‘port, int’‘count);
                    /*        Creates a server socket and binds it to the specified local port number. A
port number of 0 creates a socket on any free port.
```

```
                              The maximum queue length for incoming connection indications (a request to
connect) is set to the count parameter. If a connection indication arrives when the queue is full, the
connection is refused.

                              If the application has specified a server socket factory , that factory's
createSocketImpl                                  method’‘is called to create the actual socket
implementation. Otherwise a "plain" socket’‘                                is created.

                              Throws :’‘IOException, if an IO error occurs when opening the socket.

                    */

‘‘‘‘‘‘     //        Methods

‘‘‘‘       public Socket accept();

‘‘‘‘       public void close();

‘‘         ‘‘‘‘public InetAddress getInetAddress();

‘‘‘‘       public int getLocalPort();

‘‘‘‘       public static void setSocketFactory(SocketImplFactory’‘fac);

‘‘‘‘       public String toString();

           }
```

Methods are described in Table 8.4

Table 8.4

Method	Description
accept()	Listens for a connection to be made to this socket and accepts it. The method blocks until a connection is made. **Throws :** `IOException`, If an I/O error occurs when waiting for a connection.
close()	Closes this socket. **Throws :** `IOException`, If an I/O occurs error when closing the socket.
getInetAddress()	**Returns** : The address to which this socket is connected, or null if the socket is not yet connected.
getLocalPort()	**Returns :** the port number to which this socket is listening.
setSocketFactory(SocketImplFactory fac)	Sets the server socket implementation factory for the application. The factory can be specified only once. When an application creates a new server socket, the socket implementation factory's createSocketImpl method is called to create the actual socket implementation. **Throws :** SocketException, if the factory has already been defined, and **Throws :** IOException, If an I/O error occurs when setting the socket factory.
toString()	**Returns :** a string representation of this socket.

Class Socket

This class implements client sockets (also called just "sockets"). A socket is a end point for communication between two machines.

The actual work of the socket is performed by an instance of the SocketImpl class. An application, by changing the socket factory that creates the socket implementation, can configure itself to create sockets appropriate to the local firewall.

```
public‘‘final‘‘class‘‘java.net.Socket extends‘‘java.lang.Object‘‘

          {

‘‘‘‘‘‘‘  //        Constructors

‘‘‘‘      public Socket(InetAddress‘‘address, int‘‘port);

                    /*        Creates a stream socket and connects it to the specified port number at the
specified IP address. If the application has specified a socket factory, that factory's createSocketImpl
method‘‘is called to create the actual socket implementation. Otherwise a "plain" socket‘‘ is created.

                              Throws : IOException, if an I/O error occurs when creating the socket.

                    */

‘‘‘‘      public Socket(InetAddress‘‘address, int‘‘port, boolean‘‘stream);

                    /*        Creates a socket and connects it to the specified port number at the
specified IP address. If the stream argument is true, this creates a stream socket. If the stream argument
is false, it creates a datagram socket.

                              Throws : IOException,‘‘If an I/O error occurs when creating the socket.

                    */

‘‘        ‘‘‘‘public Socket(String‘‘host, int‘‘port);

                    /*        Creates a stream socket and connects it to the specified port number on the
named host.

                    Throws : IOException,‘‘If an I/O error occurs when creating the socket.

                    */‘‘

‘‘‘‘      public Socket(String‘‘host, int‘‘port, boolean stream);

                    /*        Creates a stream socket and connects it to the specified port number on the
named host.If the stream argument is      true, this creates a stream socket. If the stream argument is
false, it creates a datagram socket.

                              Throws : IOException,‘‘If an I/O error occurs when creating the socket.

                    */


‘‘‘‘‘‘‘  //        Methods

‘‘‘‘      public void close();

‘‘‘‘      public InetAddress getInetAddress();

‘‘‘‘      public InputStream getInputStream();

‘‘‘‘      public int getLocalPort();

‘‘‘‘      public OutputStream getOutputStream();

‘‘‘‘      public int getPort();

‘‘‘‘      public static void setSocketImplFactory(SocketImplFactory‘‘fac);

‘‘‘‘      public String toString();

          }

          Methods are described in Table 8.5
```

Table 8.5

Method	Description
close()	Closes this socket. **Throws :** IOException, if an I/O error occurs when closing this socket.
getInetAddress()	**Returns :** the remote IP address to which this socket is connected.
getInputStream()	**Returns :** an input stream for reading bytes from this socket. **Throws :** `IOException`, If an I/O error occurs when creating the input stream.
getLocalPort()	**Returns :** the local port number to which this socket is connected.
getOutputStream()	**Returns:** an output stream for writing bytes to this socket. **Throws :** `IOException`, If an I/O error occurs when creating the output stream.
getPort()	**Returns :** the remote port number to which this socket is connected.
setSocketImplFactory(SocketImplFactory fac)	Sets the client socket implementation factory for the application. The factory can be specified only once. When an application creates a new client socket, the socket implementation factory's createSocketImpl method is called to create the actual socket implementation. **Throws :** SocketException, if the factory is already defined and **Throws :** IOException, if an I/O error occurs when setting the socket factory.
toString()	**Returns :** a string representation of this socket.

Class SocketImpl

The abstract class SocketImpl is a common superclass of all classes that actually implement sockets. It is used to create both client and server sockets. A "plain" socket implements these methods exactly as described, without attempting to go through a firewall or proxy.

```
public‘‘abstract’‘class’‘java.net.SocketImpl extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Member elements
‘‘‘‘      protected InetAddress address;
                    //        The IP address of the remote end of this socket.
‘‘        ‘‘‘‘protected FileDescriptor fd;
                    //        The file descriptor object for this socket.
```

```
‘‘‘‘      protected int localport;
                    //        The local port number to which this socket is connected.
‘‘‘‘      protected int port;
                    //        The port number on the remote host to which this socket is connected.

‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public SocketImpl();
                    //        The default constructor for a socket implementation.

‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      protected abstract void accept(SocketImpl’‘s);
‘‘‘‘      protected abstract int available();
‘‘‘‘      protected abstract void bind(InetAddress’‘host, int’‘port);
‘‘‘‘      protected abstract void close();
‘‘‘‘      protected abstract void connect(InetAddress’‘address, int’‘port);
‘‘‘‘      protected abstract void connect(String’‘host, int’‘port);
‘‘‘‘      protected abstract void create(boolean’‘stream);
‘‘‘‘      protected FileDescriptor getFileDescriptor();
‘‘‘‘      protected InetAddress getInetAddress();
‘‘‘‘      protected abstract InputStream getInputStream();
‘‘‘‘      protected int getLocalPort();
‘‘‘‘      protected abstract OutputStream getOutputStream();
‘‘‘‘      protected int getPort();
‘‘‘‘      protected abstract void listen(int’‘count);
‘‘‘‘      public String toString();
          }

          Methods are described in Table 8.6.
```

Table 8.6

Method	Description
accept(SocketImpl s)	Accepts a connection. **Throws :** IOException, if an I/O error occurs when accepting the connection.
available()	**Returns:** the number of bytes that can be read from this socket without blocking. 197197 **Throws :** `IOException`, if an I/O error occurs when determining the number of bytes available.
bind(InetAddress host, int port)	Binds this socket to the specified port number on the specified host. **Throws :** `IOException` , if an I/O error occurs when binding this socket.
close()	Closes this socket.

	Throws : `IOException`, if an I/O error occurs when closing this socket
connect(InetAddress address, int port)	Connects this socket to the specified port number on the specified host. **Throws :** `IOException`, if an I/O error occurs when attempting a connection.
connect(String host, int port)	Connects this socket to the specified port on the named host. **Throws :** `IOException`, if an I/O error occurs when connecting to the remote host.
create(boolean stream)	Creates a socket. **Throws :** `IOException`, if an IO error occurs while creating the socket.
getFileDescriptor()	**Returns :** the value of this socket's fd field.
getInetAddress()	**Returns :** the value of this socket's address field.
getInputStream()	**Returns :** a stream for reading from this socket. **Throws :** `IOException`, f an I/O error occurs when creating the input stream.
getLocalPort()	**Returns :** the value of this socket's localport field.
getOutputStream()	**Returns :** an output stream for writing to this socket. **Throws :** `IOException`, If an I/O error occurs when creating the output stream
getPort()	**Returns :** the value of this socket's port field.
listen(int count)	Sets the maximum queue length for incoming requests to this socket to the count argument. If a connection request arrives when the queue is full, the connection is refused. **Throws :** IOException, if an I/O error occurs when creating the queue.
toString()	**Returns :** a string representation of this socket.

Class URL

Class URL represents a Uniform Resource Locator-a pointer to a "resource" on the World Wide Web. A resource can be something as simple as a file or a directory, or it can be a reference to a more complicated object, such as a query to a database or to a search engine.

The class structure is shown below :

```
public‘‘final’‘class’‘java.net.URL extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘  //        Constructors
‘‘‘‘       public URL(String’‘spec);
```

```
                    /*        Creates a URL object from the String representation. This constructor is
equivalent to a call to the two-argument constructor with a null first argument.

                              Throws :‘‘MalformedURLException,’‘ if the string specifies an unknown
protocol.

                    */

‘‘‘‘      public URL(String’‘protocol, String’‘host, int’‘port, String’‘file);

                    /*        Creates a URL object from the specified protocol, host, port number, and
file.

                              Throws :‘‘MalformedURLException,’‘ if the string specifies an unknown
protocol.

                    */

‘‘        public URL(String’‘protocol, String’‘host, String’‘file);

                    /*        Creates an absolute URL from the specified protocol name, host name, and
file name. The default port for the specified protocol is used.

                              Throws :‘‘MalformedURLException,’‘ if the string specifies an unknown
protocol.

                    */

          public URL(URL’‘context, String’‘spec);

                    /*        Creates a URL by parsing the String specification within a specified
context: If the context argument is not null and the spec argument is a partial URL specification, then
any of the strings missing components are inherited from the context argument.

                              Throws :‘‘MalformedURLException, if no protocol is specified, or an unknown
protocol is found.

                    */


‘‘‘‘‘‘‘‘ //        Methods

‘‘‘‘      public boolean equals(Object’‘obj);

‘‘‘‘      public final Object getContent();

‘‘‘‘      public String getFile();

‘‘‘‘      public String getHost();

‘‘‘‘      public int getPort();

‘‘‘‘      public String getProtocol();

‘‘‘‘      public String getRef();

‘‘‘‘      public int hashCode();

‘‘‘‘      public URLConnection openConnection();

‘‘‘‘      public final InputStream openStream();

‘‘‘‘      public boolean sameFile(URL’‘other);

‘‘‘‘      public static void setURLStreamHandlerFactory(URLStreamHandlerFactory fac);

‘‘‘‘      public String toExternalForm();

‘‘‘‘      public String toString();

          }
```

Methods are stated in Table 8.7.

Table 8.7

Method	Description
equals(Object obj)	The result is true if and only if the argument is not null and is a URL object that represents the same URL as this object. Two URL objects are equal if they have the same protocol, reference the same host, the same port number on the host, and the same information on the host.
getContent()	**Returns:** the contents of this URL. **Throws :** IOException, if an I/O exception occurs.
getFile()	**Returns :** the information field of this URL.
getHost()	**Returns :** the host name field of this URL.
getPort()	**Returns :** the port number of this URL.
getProtocol()	**Returns :** the name of the protocol of this URL.
getRef()	**Returns** : the anchor (also known as the "reference") of this URL.
hashCode()	**Returns :** a hash code for this URL.
openConnection()	Creates (if not already in existence) a URLConnection object that represents a connection to the remote object referred to by the URL. **Throws :** IOException, if an I/O exception occurs.
openStream()	Opens a connection to this URL and return a stream for reading from that connection. **Throws :** IOException, if an I/O exception occurs.
sameFile(URL other)	Returns true if the this URL and the other argument both refer to the same resource; the two URLs might not both contain the same anchor.
setURLStreamHandlerFactory (URLStreamHandlerFactory fac)	Sets an application's `URLStreamHandlerFactory`. This method can be called at most once by an application. **Throws :** `Error` , if the application has already set a factory.
toExternalForm()	Constructs a string representation of this URL. The string is created by calling the `toExternalForm` method of the stream protocol handler for this object.
toString()	**Returns :** a string representation of this object.

Class URLConnection

The abstract class URLConnection is the superclass of all classes that represent a communications link between the application and a URL. Instances of this class can be used both to read from and to write to the resource referenced by the URL. In general, creating a connection to a URL is a multi-step process.

First, the connection object is created by invoking openConnection on a URL, then setup parameters are manipulated, then the actual connection to the remote object is made, and finally the remote object is available. There are a set of properties that apply before the connection is made, and a set that apply after:

Most of these properties have a set and a get method. The first three in the "before" column also have corresponding "Default" properties that specify the default values used in subsequent requests where the property is not specified.

In the common case, all of these properties can be ignored: the pre-connection properties default to sensible values, and the post-connection properties are often uninteresting. For most clients of this interface, there are only two interesting methods: getInputStream and getObject, which are mirrored in the URL class by convenience methods.

```
This very useful class is having the following constituents :


          public’‘abstract’‘class’‘java.net.URLConnection extends’‘java.lang.Object’‘

          {

                              //        Member elements

                    protected boolean allowUserInteraction;

                    /*        This value of this field can be set by the setAllowUserInteraction method.
Its value can be accessed by the getAllowUserInteraction method . Its default value is the value of the
argument method to the last call to the setDefaultAllowUserInteraction method.

                              If true, this URL is being examined in a context which it makes sense to
allow user interactions such as popping up an authentication dialog. If false, then no user interaction is
allowed.

                    */

                    protected boolean connected;

                    /*        If false, this connection object has not created a communications link to
the specified URL. If true, the communications link has been established.

                    */

                    protected boolean doInput;

                    /*        This variable is set by the setDoInput method. Its value can be accessed by
the getDoInput method .

                              A URL connection can be used for input and/or output. Setting the doInput
flag to true indicates that the application intends to read data from the URL connection.

                              The default value of this field is true.

                    */

                    protected boolean doOutput;

                    /*        This variable is set by the setDoOutput method. Its value can be accessed
by the                                   getDoInput method.

                              A URL connection can be used for input and/or output. Setting the do-Output
flag to true indicates that the application intends to write data to the URL connection.

                              The default value of this field is false.

                    */

                    protected long ifModifiedSince;

                    /*        Some protocols support skipping the fetching of the object unless the
object has been modified more recently than a certain time.

                              A non-zero value is interpreted as the time corresponding to if-Modified-
Since seconds since January 1, 1970, GMT. The object is fetched only if it has been modified more recently
than that time.

                              This variable is set by the setIfModifiedSince method . Its value can be
accessed by the getIfModifiedSince method.
```

```
                              The default value of this field is 0, indicating that the fetching must
always occur.
                    */
                    protected URL url;
                    /*        The URL representing the remote object on the World Wide Web to which this
connection is opened.
                              The value of this field can be accessed by the getURL method.
                              The default value of this variable is the value of the URL argument in the
URLConnection constructor.
                    */
                    protected boolean useCaches;
                    /*        This field is set by the setUseCaches method. Its value can be accessed by
the getUseCaches method . Its default value is the value of the last argument to the method
setDefaultUseCaches.
                              If true, the protocol is allowed to use caching whenever it can. If false,
the protocol must always try to get a fresh copy of the object.
                    */
                    //        Constructors
                    protected URLConnection(URL'‘url);
                    /*        Constructs a URL connection to the specified URL. A connection to the
object referenced by the URL is not created.
                    */
                              //        Methods
                    public abstract void connect();
                    public boolean getAllowUserInteraction();
                    public Object getContent();
                    public String getContentEncoding();
                    public int getContentLength();
                    public String getContentType();
                    public long getDate();
                    public static boolean getDefaultAllowUserInteraction();
                    public static String getDefaultRequestProperty(String'‘key);
                    public boolean getDefaultUseCaches();
                    public boolean getDoInput();
                    public boolean getDoOutput();
                    public long getExpiration();
                    public String getHeaderField(int'‘n);
                    public String getHeaderField(String'‘name);
                    public long getHeaderFieldDate(String'‘name, long'‘Default);'‘
                    public int getHeaderFieldInt(String'‘name, int'‘Default);
                    public String getHeaderFieldKey(int'‘n);
                    public long getIfModifiedSince();
                    public InputStream getInputStream();
```

```
                public long getLastModified();
                public OutputStream getOutputStream();
                public String getRequestProperty(String’‘key);
                public URL getURL();
                public boolean getUseCaches();
                protected static String guessContentTypeFromName(String’‘fname);
                protected static String guessContentTypeFromStream(InputStream’‘is);
                public void’‘setAllowUserInteraction(boolean’‘allow);
                public static void setContentHandlerFactory(ContentHandlerFactory’‘fac);
                public static void setDefaultAllowUserInteraction(boolean’‘default);
                public static void setDefaultRequestProperty(String’‘key, String’‘value);
                public void setDefaultUseCaches(boolean’‘uCaches);
                public void setDoInput(boolean’‘doinput);
                public void setDoOutput(boolean’‘dooutput);
                public void setIfModifiedSince(long’‘ modify);
                public void setRequestProperty(String’‘key, String’‘value);’‘
                public void setUseCaches(boolean’‘uCaches);
                public String toString();
        }

        Methods are stated in Table 8.8.
```

Table 8.8

Method	Description
connect()	Opens a communications link to the resource referenced by this URL, if such a connection hasn't already been established. If the connect method is called when the connection has already been opened (indicated by the connected field having the value true), the call is ignored. **Throws :** `IOException`, if an IO error occurs while opening the connection.
getAllowUserInteraction()	**Returns :** the value of the `allowUserInteraction` *field for this object.*
getContent()	Retrieves the contents of this URL connection. **Throws :** `IOException`, if an IO error occurs while getting the content. **Throws :** `UnknownServiceException,` if the protocol does not support the content type.
getContentEncoding()	**Returns :** the content encoding of the

	resource that the URL references, or null if not known.
getContentLength()	**Returns :** the content length of the resource that this connection's URL referncees, or -1 if the content length is not known.
getContentType()	**Returns :** the content type of the resource that the URL references, or null if not known.
getDate()	**Returns :** the sending date of the resource that the URL refernces, or 0 if not known. The value returned is the number of seconds since January 1, 1970 GMT.
getDefaultAllowUserInteraction()	**Returns :** The default value of the `allowUserInteraction` *field.*
getDefaultRequestProperty(String key)	Gets the value of the default request property. Default request properties are set for every connection.
getDefaultUseCaches()	**Returns :** the default value of this URLConnection's `useCaches` *flag.*
getDoInput()	**Returns :** the value of this URLConnection's `doInput` *flag.*.
getDoOutput()	**Returns :** the value of this URLConnection's doOutput *flag.*
getExpiration()	**Returns :** the expiration date of the resource that this URL references, or 0 if not known. The value is number of seconds since January 1, 1970 GMT.
getHeaderField(int n)	Gets the value for the nth header field. It returns null if there are fewer than n fields.
getHeaderField(String name)	**Returns :** the value of the named header field, or null if not known.
getHeaderFieldDate(String name, long Default)	**Returns :** the value of the field, parsed as a date. The result is the number of seconds since January 1, 1970 GMT represented by the named field. The value of the Default argument is returned if the field is missing or malformed.
getHeaderFieldInt(String name, int Default)	**Returns :** the value of the named field, parsed as an integer. The Default value is

	returned if the field is missing or malformed.
getHeaderFieldKey(int n)	**Returns :** the key for the nth header field, or null if there are fewer than n fields.
getIfModifiedSince()	**Returns :** the value of this object's ifModifiedSince *flag*.
getInputStream()	**Returns :** an input stream that reads from this open connection. **Throws :** `IOException` (If an IO error occurs while creating the input stream), `UnknownServiceException` , if the protocol does not support input.
getLastModified()	**Returns :** the date the resource referenced by this URLConnection was last modified, or 0 if not known. The result is the number of seconds since January 1, 1970 GMT.203203
getOutputStream()	**Returns :** an output stream that writes to this connection. **Throws :** `IOException` (If an IO error occurs while creating the output stream), `UnknownServiceException` , if the protocol does not support output.
getRequestProperty(String key)	**Returns :** The value of the named general request property for this connection.
getURL()	**Returns :** the value of this URLConnection's URL *flag*.
getUseCaches()	**Returns :** the value of this URLConnection's useCaches *flag*.
guessContentTypeFromName (String fname)	**Returns :** a guess as to what the content type of the object is, based upon its name.
guessContentTypeFromStream (InputStream is)	Tries to determine the type of an input stream based on the characters at the beginning of the input stream.
setAllowUserInteraction(boolean allow)	Set the value of the `allowUserInteraction`.
setContentHandlerFactory(ContentHandlerFactory fac)	Sets the ContentHandlerFactory of an application. It can be called at most once by an application. **Throws :** Error, if the factory has already been defined.
setDefaultAllowUserInteraction(boolean default)	Sets the default value of the allow-User-Interaction flag for this URLConnection to

	the specified value.
setDefaultRequestProperty(String key, String value)	Sets the default value of a general request property. When a URL-Connection is created, it is initialized with these properties.
setDefaultUseCaches(boolean uCaches)	Sets the default value of the useCaches flag for this URLConnection to the specified value.
setDoInput(boolean doinput)	Sets the value of the doInput flag for this URLConnection to the specified value.
setDoOutput(boolean dooutput)	Sets the value of the doOutput flag for this URL-Connection to the specified value.
setIfModifiedSince(long modify)	Sets the value of the ifModifiedSince field of this URLConnection to the specified value.
setRequestProperty(String key, String value)	Sets the general request property.
setUseCaches(boolean uCaches)	Sets the value of the useCaches field of this URL-Connection to the specified value.
toString()	**Returns :** a string representation of this URLConnection.

Class URLEncoder

The class contains a utility method for converting a String into a MIME format called "x- www-form-urlencoded" format. To convert a String, each character is examined in turn :

- ’‘The ASCII characters 'a' through 'z', 'A' through 'Z', and '0' through '9' remain the same.
- ’‘The space character ' ' is converted into a plus sign '+'.
- ’‘All other characters are converted into the three-character string "%xy" where xy is the two-digit hexidecimal representation of the lower 16-bits of the character.

```
public’‘class’‘java.net.URLEncoder extends’‘java.lang.Object’‘
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public static String encode(String’‘s);
                    //        Translates a String into x-www-form-urlencoded format.
          }
```

Class URLStreamHandler

The abstract class URLStreamHandler is the common superclass for all stream protocol handlers. A stream protocol handler knows how to make a connection for a particular protocol type, such as http, ftp, or gopher.

```
public’‘abstract’‘class’‘java.net.URLStreamHandler extends’‘java.lang.Object
          {
```

```
                    //        Constructors
                public URLStreamHandler();
                //        The default constructor.
                //        Methods
                protected abstract URLConnection openConnection(URL'‘u);
                /*        Opens a connection to the object referenced by the URL argument.
                          Returns: a URLConnection object for the URL.
                          Throws : IOException, if an IO error occurs while opening the conection.
                */
                protected void parseURL(URL'‘u, String'‘spec, int'‘start, int'‘limit);
                /*        Parses the string representation of a URL into a URL object.
                          limit - the character position to stop parsing at. This is the end of the
string or the position of the "#" character, if present.
                */
                 protected void setURL(URL'‘u, String'‘protocol, String'‘host,
int'‘port,String'‘file, String'‘ref);
                //        Sets the fields of the URL argument to the indicated values.
                protected String toExternalForm(URL'‘u);
                //        Converts a URL of a specific protocol to a String.
          }
```

Interface ContentHandlerFactory

This interface defines a factory for content handlers. This interface is used by the URLStreamHandler class to create a ContentHandler for a MIME type. An implemention of this interface should map a MIME type into an instance of ContentHandler.

```
public'‘interface'‘java.net.ContentHandlerFactory
          {
‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract ContentHandler createContentHandler(String'‘mimetype);
                    //        Creates a new ContentHandler to read an object from a URLStreamHandler.
          }
```

Interface SocketImplFactory

This interface defines a factory for socket implementations. It is used by the classes Socket and ServerSocket to create actual socket implementions.

```
public'‘interface'‘java.net.SocketImplFactory
          {
‘‘‘‘‘‘‘  //        Methods
‘‘‘‘      public abstract SocketImpl createSocketImpl();
```

```
                    //        Returns : a new instance of SocketImpl.
          }
```

Interface URLStreamHandlerFactory

This interface defines a factory for URL stream protocol handlers. It is used by the URL class to create a URLStreamHandler for a specific protocol.

```
public‘‘interface’‘java.net.URLStreamHandlerFactory
          {
‘‘‘‘‘‘‘‘ //        Methods
‘‘‘‘      public abstract URLStreamHandler createURLStreamHandler(String’‘protocol);
                    //        Returns : a URLStreamHandler’‘for the specific protocol like "ftp", "http",
"nntp", etc.
          }
```

Class ProtocolException

Thrown to indicate than there is an error in the underlying protocol, such as a TCP error.

```
public‘‘class’‘java.net.ProtocolException extends’‘java.io.IOException’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
                    public ProtocolException();
                    //        Constructs a new ProtocolException with no detail message.
                    public ProtocolException(String’‘host);
                    //        Constructs a new ProtocolException with the specified detail message.
          }
```

Class SocketException

Thrown to indicate that some error occurred while attempting to use a socket.

```
public‘‘class’‘java.net.SocketException extends’‘java.io.IOException’‘
          {
‘‘‘‘‘‘‘‘ //         Constructors
‘‘‘‘      public SocketException();
                    //        Constructs a new SocketException with no detail message.
          ‘‘‘‘public SocketException(String’‘msg);
                    //        Constructs a new SocketException with the specified detail message.
          }
```

Class UnknownHostException

Thrown to indicate that the IP address of a host could not be determined.

```
public‘‘class’‘java.net.UnknownHostException extends’‘java.io.IOException’‘
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public UnknownHostException();
                    //        Constructs a new UnknownHostException with no detail message.
‘‘‘‘      public UnknownHostException(String’‘host);
                    //        Constructs a new UnknownHostException with the specified detail message.
          }
```

Class UnknownServiceException

Thrown to indicate that an unknown service exception has occurred. Either the MIME type returned by a URL connection does not make sense, or the application is attempting to write to a read-only URL connection.

```
public‘‘class’‘java.net.UnknownServiceException extends’‘java.io.IOException
          {
‘‘‘‘‘‘‘‘ //        Constructors
‘‘‘‘      public UnknownServiceException();
                    //        Constructs a new UnknownServiceException with no detail message.
‘‘‘‘      public UnknownServiceException(String’‘msg);
                    //        Constructs a new UnknownServiceException with the specified detail message.


          }
```